Writing Advanced Applications for the Java™ 2 Platform

by Calvin Austin and Monica Pawlan

copyright ® 1995-99 Sun Microsystems, Inc.

As used in this document, the terms “Java™ virtual machine” or “Java VM” mean a virtual machine for the Java platform.

Preface

As an experienced developer on the Java™ platform, you undoubtedly know how fast moving and comprehensive the platform is. Its many application programming interfaces (APIs) provide a wealth of functionality for all aspects of application and system-level programming. While there are many good books and online documents that detail all the parameters that an API has, finding a book that brings these APIs together and uses them to solve an advanced business problem has always been a challenge.

This book fills that void by presenting the design, development, test, deployment and debugging phases for an enterprise-worthy auction application. It is not purely a reference for the Java APIs, but a practical, hands-on guide to building successful projects with the Java platform. Like any good handbook on your car or house, it includes an entire section on what to do if things do not go so well. You will find sections that detail everything from what steps to take when troubleshooting bugs to tips on performance.

While the example application does not cover every possible programming scenario, it explores many common situations and leaves you with a solid base of knowledge so you can go on and use the Java platform to design, build, debug, and deploy your own solutions. Developing one application throughout the book is a tool to help you fast-track learning new features. For example, you gain a working knowledge of RMI in one section, and a following section on CORBA explains the similarities and differences between the two.

You can get a download of the example application source code and explore more information on any topic presented here by visiting the Java Developer Connection℠ (JDC) web site at `http://developer.java.sun.com`, or the main Java web site at `http://java.sun.com`.

The example for this book is an auction application because of the growing popularity of and interest in web-based electronic commerce. The example runs on a real application server using Enterprise JavaBeans™ technology, which is particularly well-suited to electronic commerce applications. Later chapters expand the core example by adding advanced functionality, improvements, and alternative solutions to do some of the things you get for free when you use the Enterprise JavaBeans platform. Additional topics important to applications development such as security, transaction management, and performance tuning are also presented.

This book is for developers with more than a beginning level of understanding of writing programs in the Java programming language. The example application is written with the

Java 2 platform APIs and explained in terms of functional hows and whys, so if you need help installing the Java platform, setting up your environment, or getting your first application to work, you should first read a more introductory book such as *Essentials of the Java Programming Langauge: A Hands-On Guide* or *The Java Tutorial* .

Acknowledgements

We would like to thank Tony Squier for writing the code for the Thread Pooling (page 342) section, and the web session code for the Servlets (page 145) section. Tony also helped on the initial design and content, and with Joe Sam Shirah, came up with the idea for a book like this.

Special thanks to Isaac Elias, Daniel Liu, Mark Horwath, Satya Dodda, and Mary Dageforde for their contributions to the advanced examples and all the Java Developer Connection members who sent in suggestions and corrections. Also, the Printing Graphics in Project Swing (page 259) and Writing a Security Manager (page 422) sections used code adapted from *The Java Tutorial* and *The Java Tutorial Continued* by Kathy Walrath, Mary Campione, and The Tutorial Team.

We would also like to thank the following for reviewing and checking the book for accuracy Bob Bell, Rama Roberts, Erik Larsen, <<add A-W reviewers>>.

Contents

Preface 3
Acknowledgements 4
1: Matching Project Requirements with Technology 1
Project Requirements 2
Interview User Base 2
Auction House Requirements 2
User Requirements 2
Model the Project 2
Activity Diagram 6
Choosing the Software 7
Duke's Auction Demonstration 7
Home Page 8
Registration Page 9
Items Closing Today 10
All Items 11
Search for Items 11
Sell Items 12
2: Auction House Application 13
A Multitiered Application with Enterprise Beans 14
Thin-Client Programs and Multitiered Architecture 14
Entity and Session Bean Differences 16
Auction House Workings 17
Developing and Running Applications 19
How Multitiered Applications Work 19
How Enterprise Beans are used in the Example 20
AuctionServlet 20
Entity Bean Classes 22
AuctionItem Entity Bean 22
Auction Items Table 23
Registration Entity Bean 24
Registration Table 24
Session Bean Classes 24
Bidder Session Bean 25
Seller Session Bean 25
Container Classes 25

Examining a Container-Managed Bean 26
Member Variables 26
Create Method 26
Entity Context Methods 27
Load Method 27
Store Method 27
Connection Pooling 27
Deployment Descriptor 28
XML Deployment Descriptor 29
Container-Managed finder Methods 29
Finder-Based Search 29
AuctionServlet.searchItems 31
BidderBean.getMatchingItemsList 32
AuctionItemHome.findAllMatchingItems 32
AuctionItemBean Deployment Descriptor 32
Code for this Chapter 33
AuctionItem 33
AuctionItemHome 34
AuctionItemBean 34
AuctionItemPK 37
Registration 37
RegistrationHome 37
RegistrationBean 38
RegistrationPK 39
Bidder 39
BidderHome 40
BidderBean 40
Seller 43
SellerHome 43
SellerBean 43
AuctionServlet 44
3: Data and Transaction Management 55
Bean-Managed Persistence & the JDBC Platform 55
Connect to Database 56
Create Method 56
Load Method 57
Refresh Method 57
Store Method 58
Find Method 59

Managing Transactions 59
Why Manage Transactions? 60
Session Synchronization 60
Container-Managed Example 61
Session Synchronization Code 61
Transaction Commit Mode 63
Bean-Managed finder Methods 68
AuctionServlet.searchItems 68
SearchBean 70
Database Connection 70
Get Matching Items List 70
Create Method 71
Code for this Chapter 72
RegistrationBean (SQL) 72
SellerBean (Session Synchronization) 75
searchItems Method 77
SearchBean 77
4: Distributed Computing 81
Lookup Services 82
Java Naming and Directory Interface (JNDI) 82
CORBA Naming Service 84
Interoperable Object References (IOR) 87
Remote Method Invocation (RMI) 88
RMI Over Internet Inter-ORB Protocol (IIOP) 89
Improving Lookup Performance 91
Remote Method Invocation 91
About RMI 92
RMI in the Auction Application 92
Establishing Remote Communications 97
RegistrationServer Class 98
Registration Interface 103
RegistrationHome Interface 104
ReturnResults Interface 104
SellerBean Class 105

Common Object Request Broker Architecture (CORBA) 107
IDL Mapping Scheme 108
Quick Reference 108
Other IDL keywords and types 112
CORBA in the Auction Application 115
Object Request Broker 117
Helper and Holder classes 120
Garbage Collection 121
CORBA Callbacks 121
Using the Any type 124
In Conclusion 128
JDBC Technology 129
JDBC Drivers 129
Database Connections 130
Statements 131
Caching Database results 133
Result Sets 133
Scrolling Result Sets 134
Controlling Transactions 140
Escaping Characters 142
Mapping Database Types 143
Mapping Date types 144
Servlets 145
HttpServlet 145
The init Method 146
The destroy Method 146
The service Method 147
HTTP Requests 148
Using Cookies in servlets 149
HTTP Error Codes 153
Reading GET and POST Values 153
Threading 158
HTTPS 159

Code for this Chapter 160
RegistrationServer (lookup) 161
SellerBean (lookup) 163
RegistrationServer (RMI) 165
Registration (RMI) 169
RegistrationHome (RMI) 169
RegistrationPK (RMI) 170
SellerBean (RMI) 170
ReturnResults (RMI) 172
AuctionServlet (RMI) 173
RegistrationServer (CORBA) 181
Registration.idl (CORBA) 187
SellerBean (CORBA) 187
Seller (CORBA) 191
AuctionServlet (CORBA) 191
Login Servlet 200
Session (Servlets) 204
SessionCache (Servlets) 205
5: JNI Technology. 207
JNI Example 207
About the Example 207
Generate the Header File 209
Method Signature 209
Implement the Native Method 210
Compile the Dynamic or Shared Object Library 211
Run the Example 211
Strings and Arrays 212
Passing Strings 212
Passing Arrays 213
Pinning Array 215
Object Arrays 216
MultiDimensional Arrays 217
Other Programming Issues 220
Language issues 220
Calling Methods 222
Accessing Fields 225
Threads and Synchronization 226
Memory Issues 227
Invocation 228
Attaching Threads 229
6: Project Swing:
Building a User Interface 233

Components and Data Models 234
Lightweight Components 234
Ordering Components 235
Specialized Event Handling 255
Project Swing Directions 256
Printing API 256
What is in the Package? 256
Printing an AWT Component 257
Printing a Project Swing Component 259
Printing Graphics in Project Swing 259
Print Dialog 261
Page Setup Dialog 262
Printing a Collection of Pages 262
Advanced Printing 263
Multiple Components Per Page 264
Components Larger Than One Page 265
Printing A JTable Component 268
Print a Sales Report 269
Code for this Chapter 270
AuctionClient 270
PrintButton (AWT) 280
PrintButton (Project Swing) 281
PrintPanel 282
ComponentPrinterFrame 283
ShapesPrint 284
Print2Button 287
Report 288
SalesReport 291
7: Debugging Applets, Applications, and Servlets. 297
In a Rush? 298
Collecting Evidence 298
Installation and Environment 298
Class Path 299
Class Loading 300
Including Debug Code 301
Turning Debug Information on at Runtime 301
Creating Debug and Production Releases at Runtime 302
Using Diagnostic Methods 302
Adding Debug Information 303

Running Tests and Analyzing 304
Getting Behind the Seat with jdb 304
Simple jdb Test Drive 305
Remote Debugging 310
Using Auto-Pilot 313
Creating a Session Log 314
Servlet Debugging 314
Running servletrunner in Debug Mode 315
Running Java WebServer in Debug Mode 316
Abstract Window Toolkit Debugging 318
Using AWTEventListener 319
Analyzing Stack Traces 320
Sending a signal to the Java VM 320
The Java VM Generates a Stack Trace 321
Core Files 321
Using Debugging Tools or API Calls 321
Which Release Generated The Stack Trace? 321
Which Platform Generated the Stack Trace? 322
Which Thread Package was Used? 322
What are the Thread States? 322
Examining Monitors 324
Putting the Steps Into Practice 327
Expert's Checklist 330
Stack Trace Examples 331
Stack Trace 1 331
Stack Trace 2 332
Version Issues 334
JDK 1.0.2 334
JDK 1.1 335
Java 2 Platform 335
Internet Explorer 337
8: Performance Techniques 339
Improving Applet Download Speed 339
Packaging Images into One Class 339
Using JAR Files 341
Thread Pooling 342
Connection Pooling 344
Wrapper Classes 345
Deadlocks and Hangs 346
Closing Connections 347
Example Application 347

Performance Features and Tools 348
Java Virtual Machine Features 348
Just-In-Time Compilers 352
Third-Party Tools 354
Performance Analysis 354
Profiling 355
Analyze a Program 355
Operating System Performance Tools 360
Caching Client/Server Applications 363
Caching One Object 363
Caching Many Objects 365
Code for this Chapter 369
MyApplet 369
XPM Definition 375
Pool 380
Worker 382
HttpServerWorker 383
HttpServer 384
HttpServerListener 386
HttpListener 386
HttpClient 387
JDCConnectionDriver 391
JDCConnectionPool 391
JDCConnection 393
RegistrationBean 395
9: Deploying the Auction Application 399
JAR File Format 399
Bundle and Deploy the HTML Files 400
Bundle and Deploy the Enterprise Beans 401
Bundle and Deploy the Applet Program 405
Deploy to Solaris Operating System 406
Get Downloads 406
Extract Downloaded Files 406
Install Java Plug-In 407
Install Java Plug-In Patches 407
Install Netscape Communicator 408
Check the Installation 408
Install the HTML Converter 408
Security Policy File 409
Run the Administration Applet 410

Deploy to Win32 Platform 411
Get Downloads 411
Install JRE with Java Plug-In 412
Install the HTML Converter 412
Security Policy File 412
Run the Administration Applet 413
How Does It Work? 415
10: Signed Applets & Security Managers 417
Signed Applets 417
Signed Applet Example 417
Intranet Developer 419
End User 421
Running an Application with a Policy File 422
Signed Applets in JDK 1.1 422
Writing a Security Manager 422
The FileIO Program 423
The PasswordSecurityManager Class 423
Reference Information 426
Code for this Chapter 426
SignedAppletDemo 426
PasswordSecurityManager 427
FileIO 428
A: Security and Permissions 431
Overview 432
Knowing Which Permissions 432
AllPermission 433
AWTPermission 433
FilePermission 434
NetPermission 435
PropertyPermission 436
ReflectPermission 437
RuntimePermission 437
SecurityPermission 440
SerializablePermission 443
SocketPermission 444
B: Classes, Methods, and Permissions 447
java.awt.Graphics2D 448
java.awt.Toolkit 449
java.awt.Window 449
java.beans.Beans 449
java.beans.Introspector 449
java.beans.PropertyEditorManager 450

java.io.File 450
java.io.FileInputStream 450
java.io.FileOutputStream 451
java.io.ObjectInputStream 451
java.io.ObjectOutputStream 451
java.io.RandomAccessFile 451
java.lang.Class 452
java.lang.ClassLoader 453
java.lang.Runtime 454
java.lang.SecurityManager 454
java.lang.System 454
java.lang.Thread 455
java.lang.ThreadGroup 456
java.lang.reflect.AccessibleObject 457
java.net.Authenticator 457
java.net.DatagramSocket 458
java.net.HttpURLConnection 459
java.net.InetAddress 459
java.net.MulticastSocket 459
java.net.ServerSocket 460
java.net.Socket 460
java.net.URL 460
java.net.URLConnection 461
java.net.URLClassLoader 461
java.rmi.activation.ActivationGroup 461
java.rmi.server.RMISocketFactory 461
java.security.Identity 461
java.security.IdentityScope 462
java.security.Permission 462
java.security.Policy 462
java.security.Provider 462
java.security.SecureClassLoader 463
java.security.Security 463
java.security.Signer 463
java.util.Locale 463
java.util.zip.ZipFile 464
C: Security Manager Methods 465
D: API Reference 469
ActionListener Interface 469
WindowListener Interface 469
Graphics Class 469

Graphics2D class 469
Book Class 470
PageFormat Class 470
Printable Interface 470
PrinterJob Class 470
Toolkit Class 471
ByteArrayOutputStream Class 471
DataOutputStream Class 471
Double Class 471
SecurityManager Class 471
System Class 472
Naming Class 472
RMISocketFactory Class 472
CallableStatement Interface 472
Connection Interface 472
DatabaseMetaData Interface 473
DriverManager Class 473
PreparedStatement Interface 473
ResultSet Interface 474
Statement Interface 474
ArrayList Class 474
Calendar Class 474
Date Class 475
Enumeration Interface 475
HashMap Class 475
Iterator Interface 475
LinkedList Class 476
List Class 476
EntityBean Interface 476
SessionBean Interface 476
UserTransaction Interface 477
RemoteObject Class 477
Cookie Class 477
HttpServlet Class 477
HttpServletRequest Interface 477
HttpServletResponse Interface 478
ServletConfig Interface 478
ServletRequest Interface 478
ServletResponse Interface 478
Box Class 478
DefaultCellEditor Class 479

JButton Class 479
JComponent Class 479
JFrame Class 479
JLabel Class 479
JScrollPane Class 479
JTable Class 480
JTree Class 480
JViewPort Class 480
ListSelectionModel Interface 480
SwingUtilities Class 481
DefaultTableCellRenderer Class 481
DefaultTableModel Class 481
TableColumn Class 481
TableColumnModel Interface 481
DefaultMutableTreeNode Class 482
TreePath Class 482
Any Class 482
IntHolder Class 482
ORB Class 482
NameComponent Class 483
NamingContext Interface 483
JNI C methods 483
Index 485

1: Matching Project Requirements with Technology

One challenge in writing a book on advanced application development for the Java™ platform is to find a project small enough to write about, while at the same time, complex enough to warrant advanced programming techniques. The project presented in this book is a web-based auction house. The application is initially written for the Enterprise JavaBeans™ platform. Later chapters expand the core example by adding advanced functionality, improvements, and alternative solutions to do some of the things you get for free when you use the Enterprise JavaBeans platform.

To keep the discussion simple, the example application has only a basic set of transactions for posting and bidding on auction items. However, the application scales to handle multiple users, provides a three-tiered transaction-based environment, controls security, and integrates legacy-based systems.

This chapter covers how to determine project requirements and model the important steps that should always come before coding begins.

Covered in this Chapter

- Project Requirements (page 2)
- Choosing the Software (page 7)

- Duke's Auction Demonstration (page 7)

Project Requirements

The first step in determining project requirements is to interview the user base to find out what they want in an online auction. This is an important step, and one that cannot be overrated because a solid base of user-oriented information helps you define your key application capabilities.

2: Auction House Application (page 13) walks through the application code, explains how the Enterprise JavaBeans platform works, and tells you how to run a live demonstration. If you have never seen or used an online auction, Duke's Auction Demonstration (page 7) shows mock-ups of the example auction application HTML pages.

Interview User Base

For the sake of discussion and to keep things simple, this section assumes interviews with the user base found the following auction house and user requirements:

Auction House Requirements

- Require buyer and seller information
- Bill sellers for posting item
- Record and report the day's transactions

User Requirements

- Bid on or sell an item
- Search or view items for sale
- Notify buyer and seller of sale

Model the Project

After analyzing the requirements, you can build a use case diagram to gain a better understanding of the elements needed in the application and how they interact. A use case diagram shows the relationships among actors and use cases within the system. A use case is a unique function in a system, and an actor is the person or software that performs the action or use case. For example, a buyer is the actor that performs the function (use case) of bidding on an auction item, and the seller is the actor that performs the use case of posting an item for auc-

tion. Not all actors are people, though. For example, the software is the actor that determines when an item has closed, finds the highest bidder, and notifies the buyer and seller of a sale.

The Unified Modeling Language (UML) (`http://www.rational.com/uml/resources/documentation/notation/notation52.jtmpl`) is the tool of choice for creating use case diagrams. The Use Case diagram in Figure 1 uses UML to describe the buyer and seller use cases for the online auction application. In UML, squares group systems, stick figures represent actors, ovals denote use cases, and lines show how actors use the system.

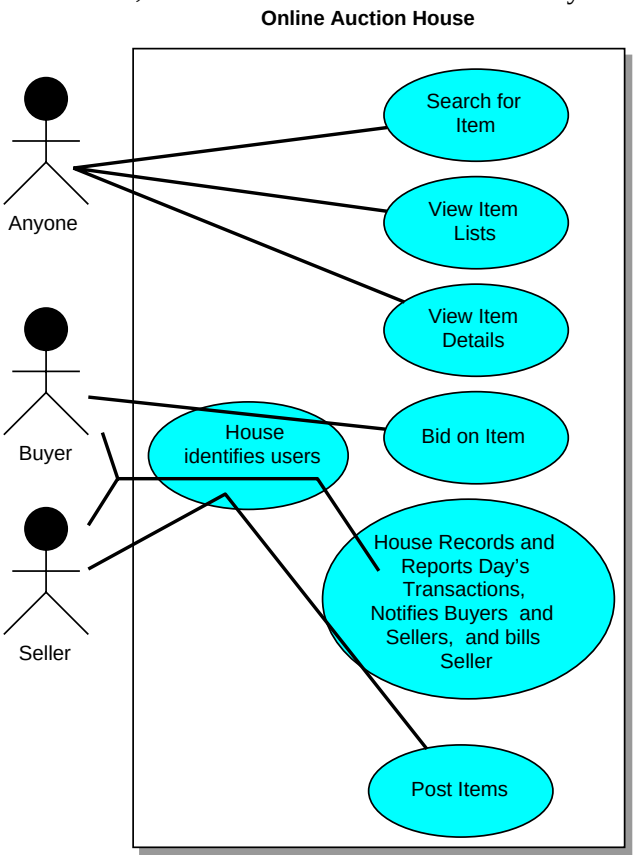


Figure 1 Use Case Diagram

The following descriptions further define the project. These descriptions are not part of UML, but are a helpful tool in project definition.

House Identifies Buyers and Sellers

An auction application is used by buyers and sellers. A buyer needs to know who the seller is to pay him or her, and the seller needs to know who the buyers are to answer product ques-

tions and to finalize the sale. So, to post or bid on an auction item, buyers and sellers are required to register. Registration needs to get the following information from buyers and sellers:

- User ID and password for buying and selling.
- Email address so highest bidder and seller can communicate when item closes.
- Credit card information so auction can charge sellers for listing their items.

Once registered, a user can post or bid on an item for sale.

House Determines Highest Bidder

Nightly, the auction application queries the database to record and report the day's transactions. The application finds items that have closed and determines the highest bidder.

House Notifies Buyers and Sellers

The auction application uses email to notify the highest bidder and seller of the sale, and debit the seller's account.

Anyone Searches for an Item

Sellers and buyers enter a search string to locate all auction items in the database.

Anyone Views Items for Sale

To popularize the auction and encourage new buyers and sellers, the application allows anyone to view auction items without requiring user ID and password identification. To keep things simple, the auction lets anyone view summarized lists of items in the following three ways:

- All items up for auction
- New items listed today
- Items due to close today

Anyone Views Item Details

The summarized lists of auction items should link each item to the following detailed information. Detail information on auction items is available to anyone without identification.

- Item Summary
- Auction Item number
- Current price
- Number of bids

- Date posted for auction
- Date item closes
- Seller ID
- Highest bid
- Item description

Seller Posts Items for Sale

To post an item for sale, a seller needs to identify himself or herself and describe the item for sale, as follows:

- User ID and password for seller identification
- Summary description of item
- Starting Price for bidding
- Detailed description of item
- Number of days item is available for bidding

Buyer Bids on Items

The detailed summary page for each item lets registered users identify themselves and bid on the item by providing the following information:

- User ID
- Password
- Bid amount

Activity Diagram

The activity diagram in Figure 2 outlines the flow of tasks within the auction house as a whole. The solid black circle on the left shows the beginning of activities, and the white circles with black dots in the center denote where activities end.

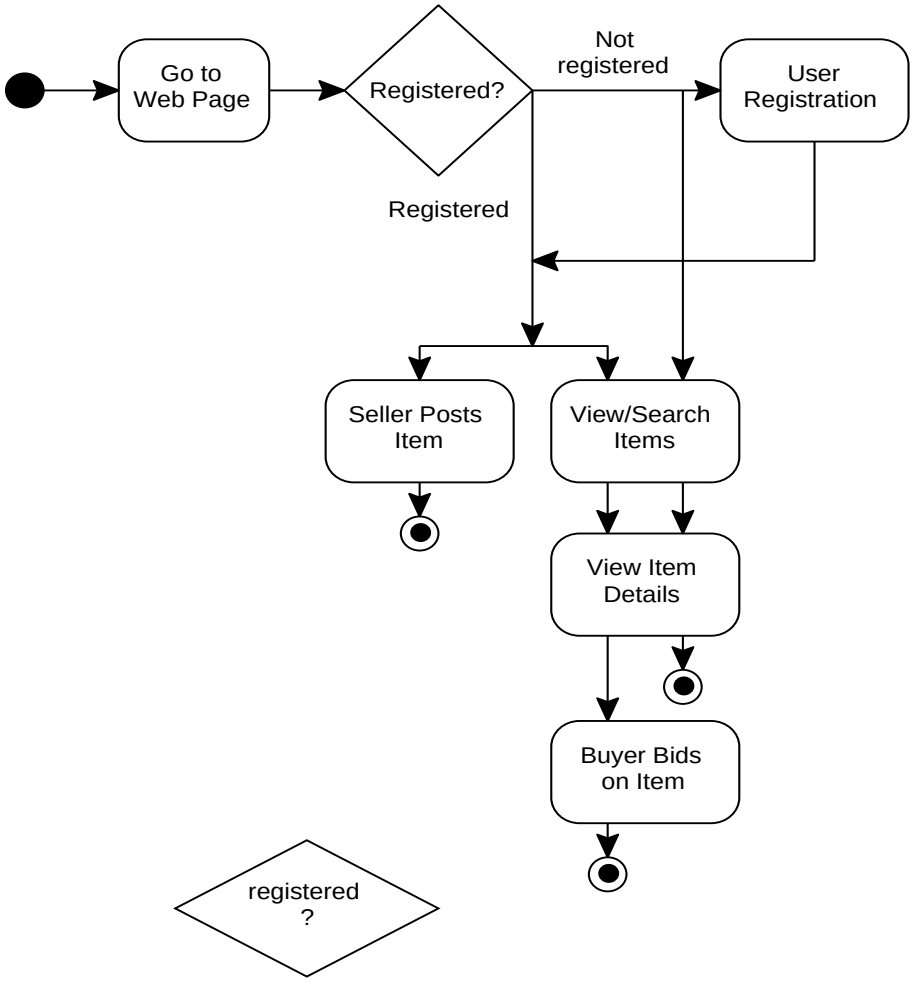


Figure 2 Activity Diagram

Choosing the Software

With the application modeled and project requirements defined, it is time to think about which Java APIs to use. The application is clearly client and server based because you will want to accommodate 1 to n buyers, sellers, and viewers at any given time. Because registration and auction item data must be stored and retrieved from somewhere, you will need an API for database access.

The core application can be created in a number of ways using any of the following APIs (to name a few):

- Servlets and HTTP
- Sockets, multithreading, and JDBC™ APIs.
- Remote Method Invocation (RMI) and JDBC APIs.
- RMI over IIOP
- Enterprise JavaBeans™ platform.
- Java Transaction Architecture (JTA)
- Common Object Request Broker Architecture (CORBA)
- Java Native Interface (JNI)

Rather than using all of these APIs as individual APIs in one application, you can use Enterprise JavaBeans. The Enterprise JavaBeans platform handles transaction and state management, multithreading, resource pooling, search functionality to handle common situations, and other complex low-level details so you can focus on creating the best business solution. You get a distributed database application without writing any socket, thread, JDBC, or RMI code. For example, simple database transactions are rolled back (not written to the database) in the event of an error, database connections are cached so there is always a connection available when the application needs one, and items in the database are retrieved and displayed to the requestor using a simple search string.

2: Auction House Application (page 13) explains the core Enterprise JavaBeans application code and how to set up and run the example. Later chapters explain how you can implement such things as your own transaction management for more complex transactions, database caching to improve performance, or a more complex or comprehensive search algorithm.

Duke's Auction Demonstration

This section shows you how the auction house application web pages look.

Home Page

The Home page introduces the auction and makes auction house features available to buyers and sellers.

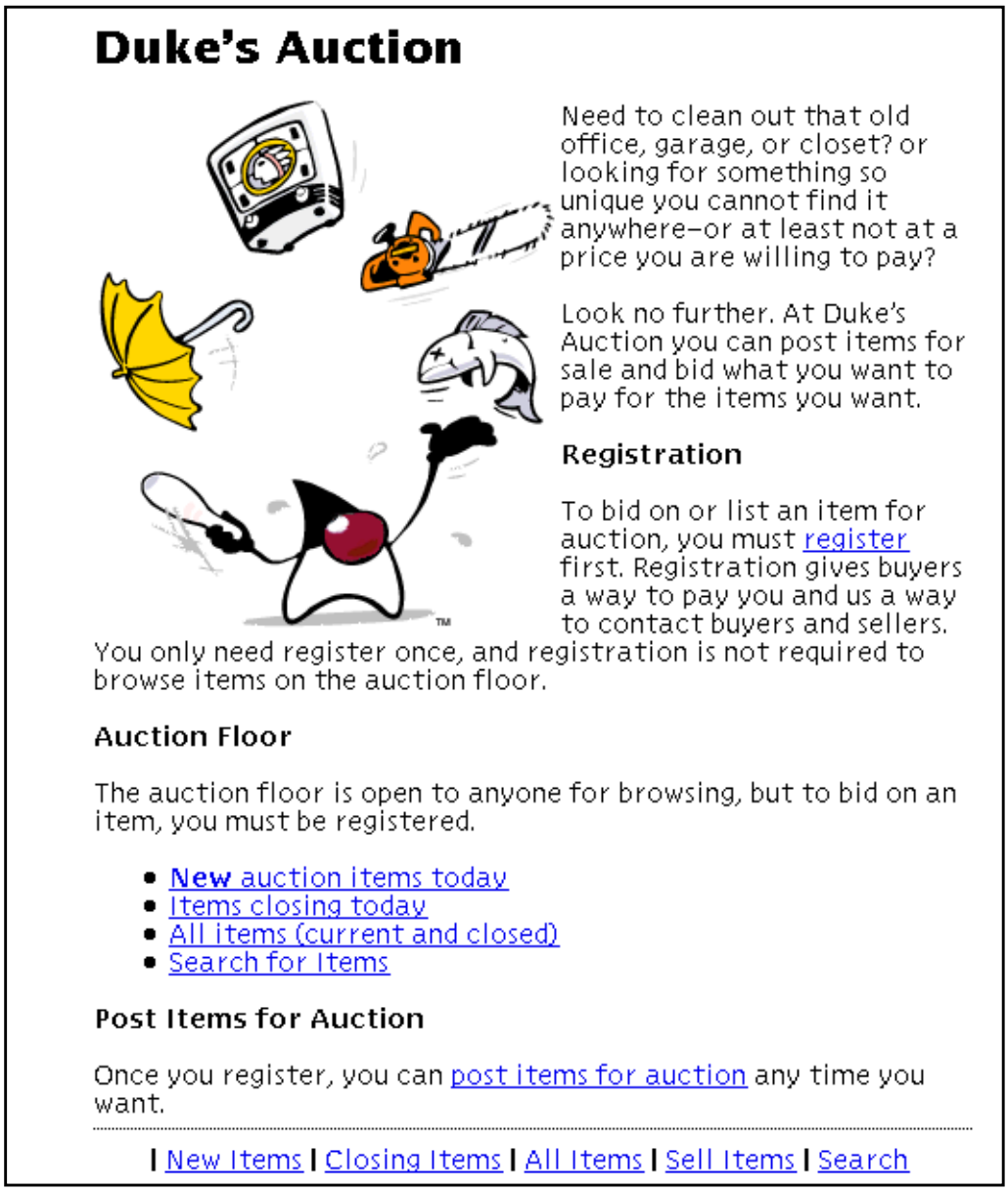


Figure 3 Duke's Auction Home Page

Registration Page

The Registration page gets information from new buyers and sellers so all individuals initiating transactions at the auction house can be identified..

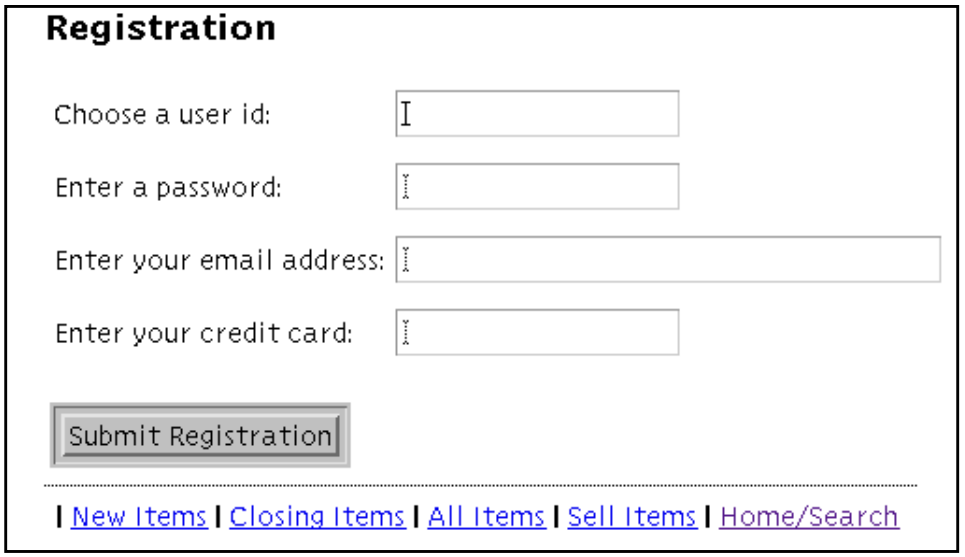
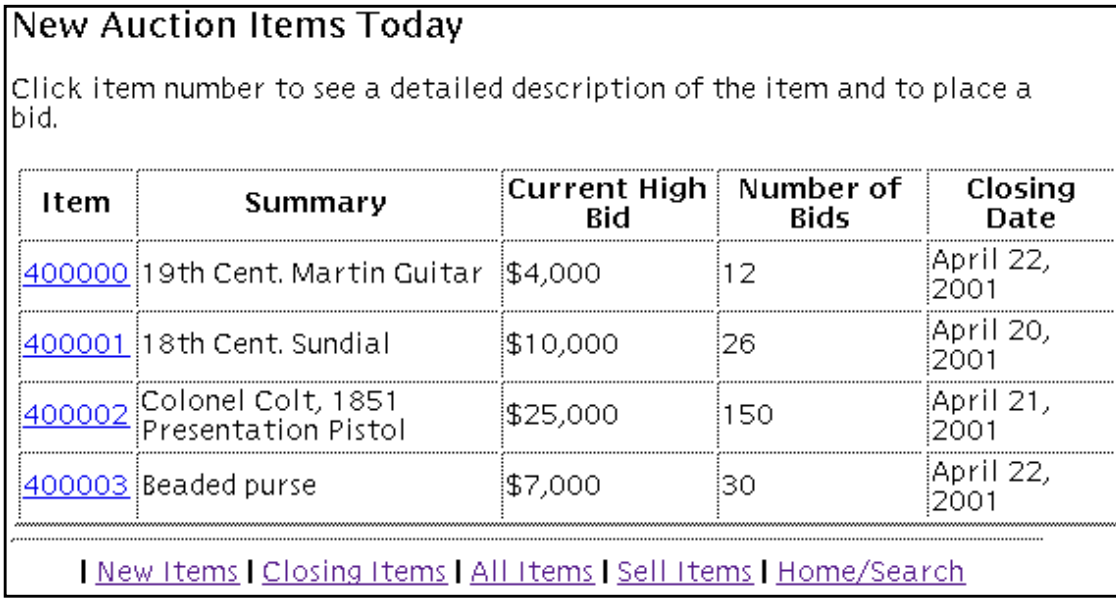


Figure 4 Duke's Auction Buyer and Seller Registration

New Auction Items Today

The New Auction Items page lets anyone view new items for sale.



New Auction Items Today

Click item number to see a detailed description of the item and to place a bid.

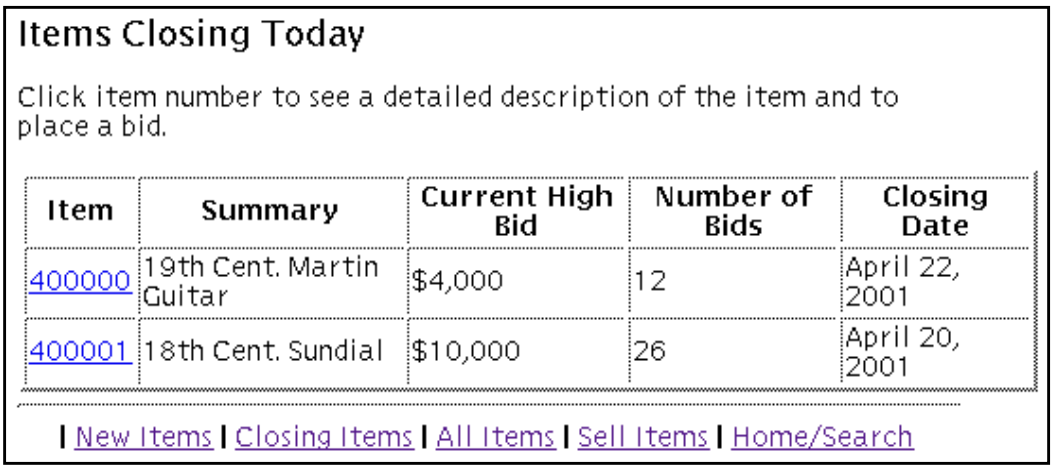
Item	Summary	Current High Bid	Number of Bids	Closing Date
400000	19th Cent. Martin Guitar	$4,000	12	April 22, 2001
400001	18th Cent. Sundial	$10,000	26	April 20, 2001
400002	Colonel Colt, 1851 Presentation Pistol	$25,000	150	April 21, 2001
400003	Beaded purse	$7,000	30	April 22, 2001

| New Items | Closing Items | All Items | Sell Items | Home/Search

Figure 5 New Items at Duke's Auction Today

Items Closing Today

The Items Closing page lets any view auction items about to be sold to the highest bidder.



Items Closing Today

Click item number to see a detailed description of the item and to place a bid.

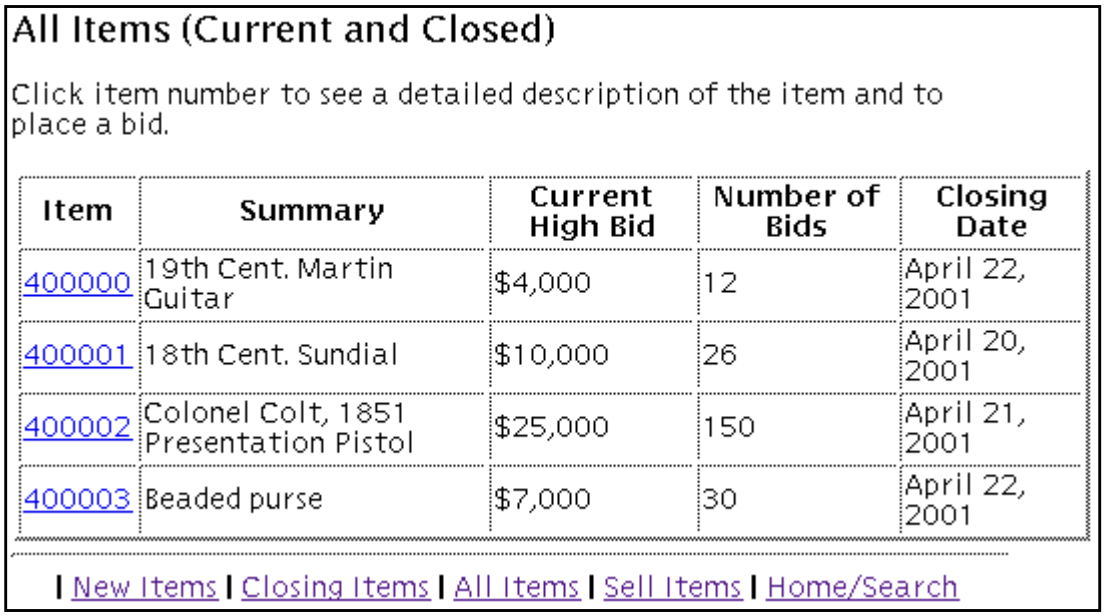
Item	Summary	Current High Bid	Number of Bids	Closing Date
400000	19th Cent. Martin Guitar	$4,000	12	April 22, 2001
400001	18th Cent. Sundial	$10,000	26	April 20, 2001

| New Items | Closing Items | All Items | Sell Items | Home/Search

Figure 6 Items Closing Today at Duke's Auction

All Items

The All Items page lets anyone view all items available for sale..



All Items (Current and Closed)

Click item number to see a detailed description of the item and to place a bid.

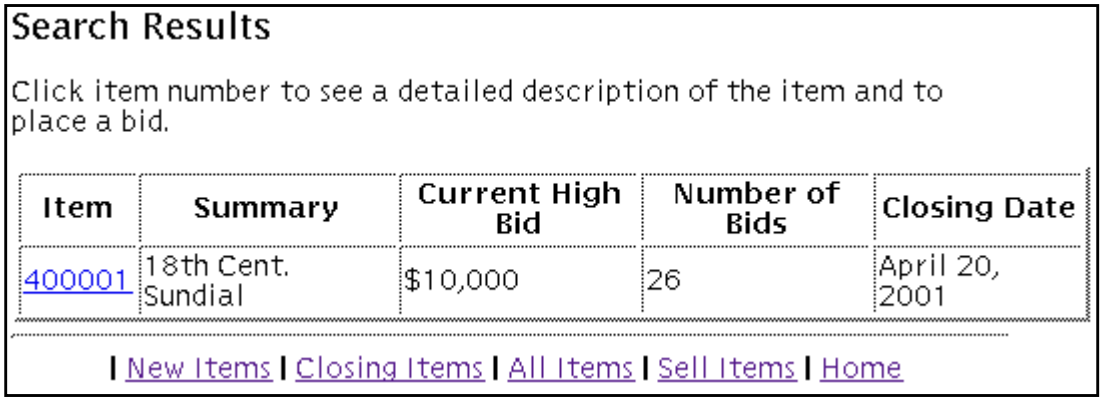
Item	Summary	Current High Bid	Number of Bids	Closing Date
400000	19th Cent. Martin Guitar	$4,000	12	April 22, 2001
400001	18th Cent. Sundial	$10,000	26	April 20, 2001
400002	Colonel Colt, 1851 Presentation Pistol	$25,000	150	April 21, 2001
400003	Beaded purse	$7,000	30	April 22, 2001

| New Items | Closing Items | All Items | Sell Items | Home/Search

Figure 7 All Items Available at Duke's Auction

Search for Items

The Search Items page lets anyone search for auction items for sale.



Search Results

Click item number to see a detailed description of the item and to place a bid.

Item	Summary	Current High Bid	Number of Bids	Closing Date
400001	18th Cent. Sundial	$10,000	26	April 20, 2001

| New Items | Closing Items | All Items | Sell Items | Home

Figure 8 Search for Items in Duke's Auction

Sell Items

The Post Items page lets registered sellers post an item for sale.



Figure 9 Sell Items at Duke's Auction

2: Auction House Application

The proliferation of internet- and intranet-based applications has created a great need for distributed transactional applications that leverage the speed, security, and reliability of server-side technology. One way to meet this need is to use a multitiered model where a thin-client application invokes business logic that executes on the server. Normally, thin-client multitiered applications are hard to write because they involve many lines of intricate code to handle transaction and state management, multithreading, resource pooling, and other complex low-level details. And to add to the difficulties, you have to rework this intricate code every time you write a new application because the code is so low-level it cannot be reused.

If you could use prebuilt and pretested transaction management code or even reuse some of your own code, you would save a lot of time and energy that you could better spend solving the business problem. Well, Enterprise JavaBeans™ technology can give you the help you need. Enterprise JavaBeans technology makes distributed transactional applications easy to write because it separates the low-level details from the business logic. You concentrate on creating the best business solution and leave the rest to the underlying architecture.

This chapter describes how to create the example auction with the services provided by the Enterprise JavaBeans platform. Later chapters show how to customize these services.

Covered in this Chapter

- A Multitiered Application with Enterprise Beans (page 14)
- How Enterprise Beans are used in the Example (page 20)
- AuctionServlet (page 20)
- Entity Bean Classes (page 22)
- Session Bean Classes (page 24)
- Container Classes (page 25)
- Examining a Container-Managed Bean (page 26)
- Container-Managed finder Methods (page 29)
- Code for this Chapter (page 33)

A Multitiered Application with Enterprise Beans

An Enterprise Bean is a small set of interfaces and classes that provide two types of methods: business logic and lifecycle. A client program calls the business logic methods to interact with the data on the server. Every Enterprise Bean has a system-level container that calls the lifecycle methods to manage the Bean on the server. In addition to these two types of methods, an Enterprise Bean has an associated configuration file, called a deployment descriptor, to configure the Bean at deployment time.

As well as being responsible for creating and deleting Beans, the Enterprise JavaBeans server also manages transactions, concurrency, security and data persistence (storing and retrieving data). Even the connections between the client and server are provided using the Remote Method Invocation (RMI) and Java Naming and Directory Interface (JNDI) APIs, and servers can optionally provide scalability through thread management and caching.

The auction house example implements a complete Enterprise JavaBeans solution by providing only the business logic and using the underlying services provided by the architecture. However, you may find that the container managed services, although providing maximum portability, do not meet all your application requirements. The next chapters show how to provide these services in your Bean.

Thin-Client Programs and Multitiered Architecture

A thin client is a client program that invokes business logic running on the server. It is called thin because most of the processing happens on the server. The auction house application shown in Figure 10 has a user interface that is a set of HTML pages. The HTML pages get input from and show information to the user in the browser. Behind the HTML pages is a servlet that passes data between the browser and the Enterprise JavaBeans server. The Enterprise JavaBeans server handles reading from and writing to the underlying database

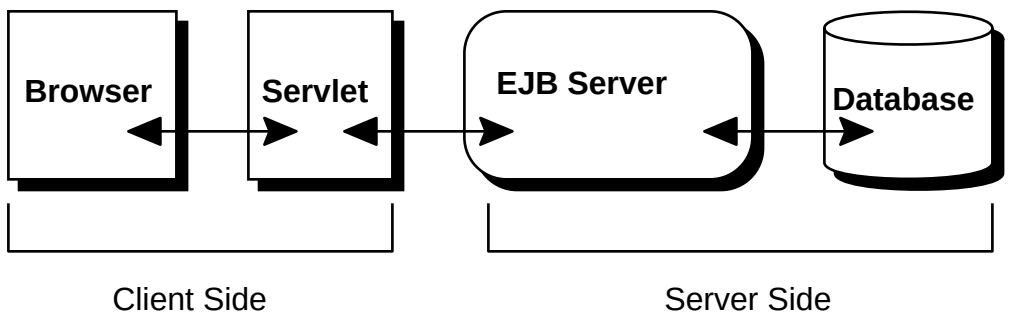


Figure 10 Client and Server Sides

Multitiered architecture or three-tier architecture extends the standard two-tier client and server model by placing a multithreaded application server between the client and the database. The application server is where the Enterprise JavaBeans reside and where the application's business logic executes.

Figure 11 shows that client programs (first tier) communicate with the database (third tier) through the application server (second tier). The application server responds to the client requests and makes database calls as needed into the underlying database.

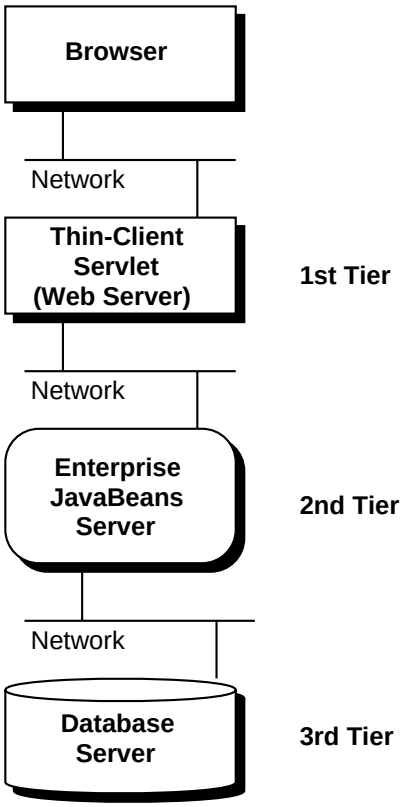


Figure 11 Multitiered Communications

Entity and Session Bean Differences

There are two types of Enterprise Beans: entity Beans and session Beans. A session Bean is created on behalf of a client and usually exists only for the duration of a single client and server session. A session Bean performs operations, such as calculations or accessing a database, for the client. The data in a session Bean is not recoverable in the event its container crashes.

An entity Bean is a persistent object that represents data maintained in a database. An entity Bean can manage its own persistence or it can delegate this function to its container. An entity Bean can live as long as the data it represents. An entity Bean is identified by a primary key. If the container in which an entity Bean is hosted crashes, the entity Bean, its primary key, and any remote references survive the crash.

Typically, an entity Bean represents one row of persistent data stored in a database table. In the auction house example, RegistrationBean is an entity Bean that represents data for one registered user, and AuctionItemBean is an entity Bean that represents the data for one auction item. Entity Beans are transactional and long-lived. As long as the data remains, the entity Bean can access and update that data. This does not mean you need a Bean running for every table row. Instead, entity Beans are loaded and saved as needed.

A session Bean might execute database reads and writes, but it is not required. A session Bean might invoke JDBC™ calls itself or it might use an entity Bean to make the call, in which case the session Bean is a client to the entity Bean. A session Bean's fields contain the state of the conversation and are transient (are not saved). If the server or client crashes, the session Bean is gone. A session Bean is often used with one or more entity Beans to perform complex operations on the data.

Session Beans	Entity Beans
Fields contain conversation state.	Represents data in a database.
Handles database access for client.	Shares access for multiple users.
Life of client is life of Bean.	Persists as long as data exists.
Can be transaction aware.	Transactional.
Does not survive server crashes.	Survives server crashes.
Not fine-grained data handling.	Fine-grained data handling.

Note: In the Enterprise JavaBeans specification, Enterprise JavaBeans server support for session Beans is mandatory. Enterprise JavaBeans server support for entity Beans was optional, but is mandatory for version 2.0 of the specification.

Auction House Workings

Figure 12 shows the Enterprise Beans for the auction house application and their relationship to the Enterprise JavaBeans server. The thin-client server invokes business logic in the four Enterprise Beans through their home and remote interfaces (described below). The Enterprise JavaBeans server in this example handles the low-level details including database read and write operations. The four Enterprise Beans in the example are:

- AuctionItemBean is an entity Bean that maintains information for an auction item.
- RegistrationBean is an entity Bean that stores user registration information.
- BidderBean is a session Bean that uses AuctionItemBean to retrieve a list of all auction items, only new items, items due to close, and items whose summary matches a search string from the database. It also checks the user ID and password when someone places a bid, and stores new bids in the database.
- SellerBean is a session Bean that uses RegistrationBean to check the user ID and password when someone posts an auction item, and AuctionItemBean to add new auction items to the database.

An entity or session Bean is really a collection of interfaces and classes. All entity and session Beans consist of a remote interface, home interface, and the Bean class. The servlet looks up the Beans's home interface running in the Enterprise JavaBeans server, uses the home interface to create the remote interface, and invokes Bean methods through the remote interface.

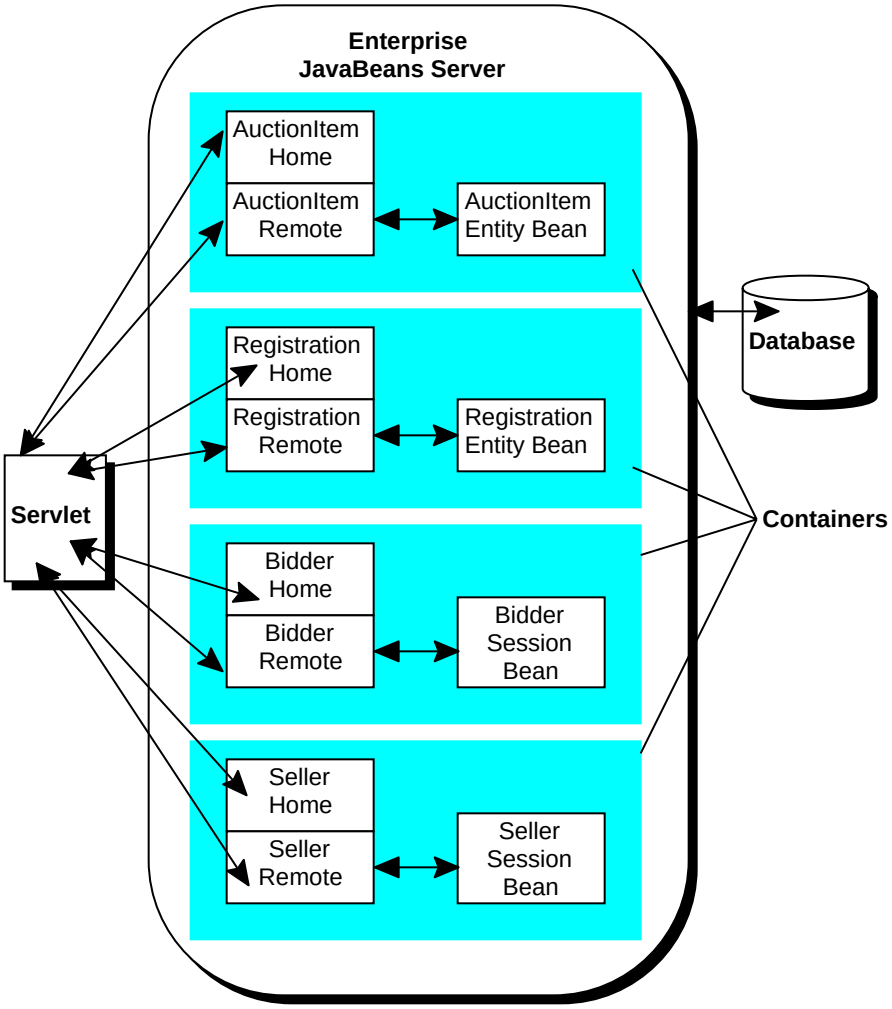


Figure 12 Parts of a Bean

- An Enterprise Bean's remote interface describes the Bean's methods, or what the Bean does. A client program or another Enterprise Bean calls the methods defined in the remote interface to invoke the business logic implemented by the Bean.
- An Enterprise Bean's home interface describes how a client program or another Enterprise Bean creates, finds (entity Beans only), and removes that Enterprise Bean from its container.
- The container provides the interface between the Enterprise Bean and the low-level platform-specific functionality that supports the Enterprise Bean.

Developing and Running Applications

Deployment tools and an Enterprise JavaBeans server are essential to running Enterprise JavaBeans applications. Deployment tools generate containers, which are classes that provide an interface to the low-level implementations in a given Enterprise JavaBeans server. The server provider can include containers and deployment tools for their server and will typically publish their low-level interfaces so other vendors can develop containers and deployment tools for their server. The auction house example uses the Enterprise JavaBeans server and deployment tools created by BEA Weblogic (`http://www.beasys.com/weblogic.html`).

Because everything is written to specification, all Enterprise Beans are interchangeable with containers, deployment tools, and servers created by other vendors. In fact, you might or might not write your own Enterprise Beans because it is possible, and sometimes desirable, to use Enterprise Beans written by one or more providers that you assemble into an Enterprise JavaBeans application.

How Multitiered Applications Work

The goal in a multitiered application is that the client be able to work on application data without knowing at build time where the data is stored in the third tier. To make this level of transparency possible, the underlying services in a multitiered architecture use lookup services to locate remote server objects (the Bean's remote interface object), and data communication services to move data from the client, through the remote server object, to its final destination in a storage medium.

Lookup Service

To find remote server objects at runtime, the client program needs a way to look them up. One way to look remote server objects up at runtime is to use the Java™ Naming and Directory Interface™ (JNDI) API. JNDI is a common interface to existing naming and directory interfaces. The Enterprise JavaBeans containers use JNDI as an interface to the Remote Method Invocation (RMI) naming service.

At deployment time, the JNDI service registers (binds) the remote interface with a name. As long as the client program uses the same naming service and asks for the remote interface by its registered name, it will be able to find it. The client program calls the lookup method on a javax.naming.Context object to ask for the remote interface by its registered name. The javax.naming.Context object is where the bindings are stored and is a different object from the Enterprise JavaBeans context, which is covered later.

Data Communication

Once the client program gets a reference to a remote server object, it makes calls on the remote server object's methods. Because the client program has a reference to the remote server object, a technique called data marshaling is used to make it appear as if the remote server object is local to the client program.

Data marshaling is where methods called on the remote server object are wrapped with their data and sent to the remote server object. The remote server object unwraps (unmarshals) the methods and data, and calls the Enterprise Bean. The results of the call to the Enterprise Bean are wrapped again, passed back to the client through the remote server object, and unmarshaled.

The Enterprise JavaBeans containers use Remote Method Invocation (RMI) services to marshal data. When the Bean is compiled, stub and skeleton files are created. The stub file provides the data wrapping and unwrapping configuration on the client, and the skeleton provides the same information for the server. The data is passed between the client program and the server using serialization. Serialization is a way to represent Java objects as bytes that can be sent over the network as a stream and reconstructed on the other side in the same state they were in when first sent.

How Enterprise Beans are used in the Example

The example uses two entity Beans and two session Beans. The entity Beans, AuctionItemBean and RegistrationBean, represent persistent items stored in a database, and the session Beans, SellerBean and BidderBean, represent short-lived operations with the client and data.

The session Beans are the client interface to the entity beans. The SellerBean processes requests to add new auction items for sale. The BidderBean processes requests to retrieve auction items and place bids on those items. Changing and adding to the database data in a container-managed Bean is left to the entity Beans.

Entity and session Beans are distributed objects that use the RMI API. This means that when an error occurs, an RMI remote exception is thrown.

AuctionServlet

AuctionServlet (page 44) is essentially the second tier in the application and the focal point for auction activities. It accepts end user input from the browser by way of hypertext transfer protocol (HTTP), passes the input to the appropriate Enterprise Bean for processing, and displays the processed results to the end user in the browser.

Figure 13 presents a Unified Modeling Language (UML) class diagram for the AuctionServlet class.

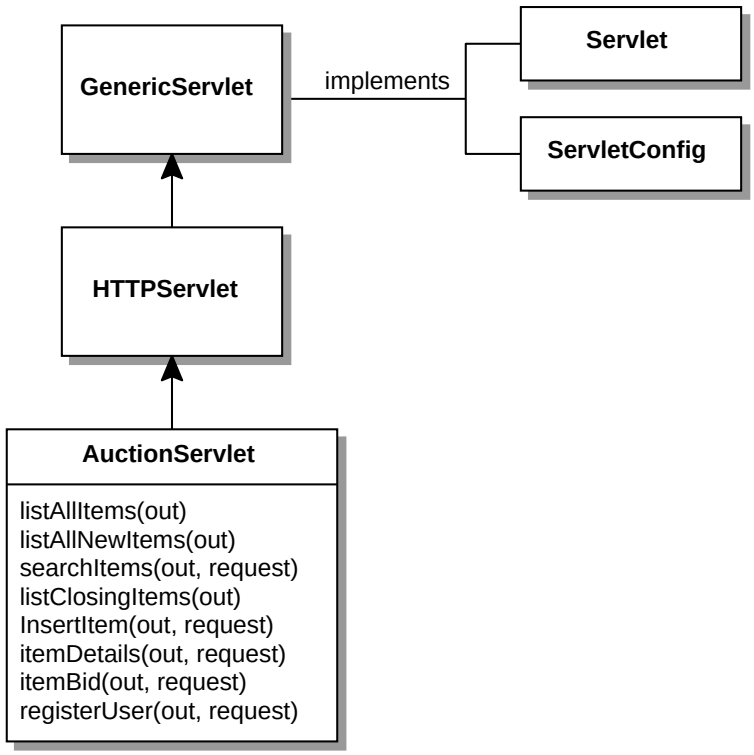


Figure 13 UML Class Diagram

The AuctionServlet methods shown above invoke business logic that executes on the server by looking up an Enterprise Bean and calling one or more of its methods. When the servlet adds HTML codes to a page for display to the user, that logic executes on the client.

For example, the listAllItems(out) method executes code on the client to dynamically generate an HTML page to be viewed by the client in a browser. The HTML page is populated with the results of a call to BidderBean that executes logic on the server to generate a list of all auction items.

```
private void listAllItems(ServletOutputStream out) throws IOException{

//Put text on HTML page
  setTitle(out, "Auction results");
  String text = "Click Item number for description and to place bid.";
  try{
     addLine("<BR>"+ text, out);
//Look up Bidder bean home interface.
```

```
     BidderHome bhome=(BidderHome) ctx.lookup("bidder");
//Create Bidder bean remote interface.
     Bidder bid=bhome.create();
//Call Bidder bean method through remote interface.
     Enumeration enum=(Enumeration)bid.getItemList();

     if(enum != null) {
//Put retrieved items on servlet page.
       displayitems(enum, out);
       addLine("", out);
     }
  } catch (Exception e) {
//Print error on servlet page.
     addLine("AuctionServlet List All Items error",out);
     System.out.println("AuctionServlet <list>:"+e);
  }
     out.flush();
}
```

Entity Bean Classes

AuctionItemBean and RegistrationBean are entity Beans. AuctionItemBean adds new auction items to the database and updates the bid amount as users bid on the item. RegistrationBean adds information to the database on registered users. Both Beans consist of the classes described here.

AuctionItem Entity Bean

These are the AuctionItemBean interfaces and classes.

- AuctionItem (page 33)
- AuctionItemHome (page 34)
- AuctionItemBean (page 34)
- AuctionItemPK (page 37)

AuctionItem is the remote interface. It describes what the Bean does by declaring the developer-defined methods that provide the business logic for this Bean. These methods are the ones used by the client to interact with the Bean over the remote connection. Its name maps to the AUCTIONITEMS table shown just below.

AuctionItemHome is the home interface. It describes how the Bean is created in, found in, and removed from its container. The Enterprise Bean server deployment tools will provide the implementation for this interface.

AuctionItemBean is the Enterprise Bean. It implements EntityBean, provides the business logic for the developer-defined methods, and implements EntityBean methods for creating the Bean and setting the session context. This is a class that the Bean developer needs to implement. Its field variables map to fields in the AUCTIONITEMS table shown just below.

AuctionItemPK is the primary key class. The Enterprise JavaBeans server requires a container-managed entity Bean to have a primary key class with a public primary key field (or fields, if using composite primary keys). The Bean developer implements this class. The ID field is the primary key in the AUCTIONITEMS table shown just below, so the ID field is a public field in this class. The id field is assigned a value when the primary key class is constructed.

You can request the container manage database persistence for an Enterprise Bean or write the code to manage the persistence yourself. In this chapter, all beans (entity and session) are container-managed. With container-managed Beans, all you do is specify which fields are container managed and let the Enterprise JavaBeans server do the rest. This is great for simple applications, but if you are coding something that is fairly complex, you might need more control. How to override the underlying Enterprise JavaBeans services to gain more control or provide similar services for non-Enterprise JavaBean applications is covered in 3: Data and Transaction Management (page 55).

Auction Items Table

This is the schema for the AUCTIONITEMS database table. Some application servers create the table for you, but others require you to create it yourself. The BEA Weblogic application server does not create the database table.

```
create table AUCTIONITEMS (SUMMARY VARCHAR(80) ,
ID                          INT ,
COUNTER                     INT ,
DESCRIPTION                 VARCHAR(1000) ,
STARTDATE                   DATE ,
ENDDATE                     DATE ,
STARTPRICE                  DOUBLE PRECISION ,
INCREMENT                   DOUBLE PRECISION ,
SELLER                      VARCHAR(30) ,
MAXBID                      DOUBLE PRECISION,
BIDCOUNT                    INT,
HIGHBIDDER                  VARCHAR(30) )
```

Registration Entity Bean

RegistrationBean consists of the same kinds of interfaces and classes and database table as the AuctionItem Bean, except the actual business logic, database table fields, and primary key are different. Rather than describe the classes, you can browse them and refer back to the AuctionItem Bean discussion if you have questions.

- Registration (page 37)
- RegistrationHome (page 37)
- RegistrationBean (page 38)
- RegistrationPK (page 39)

Registration Table

This is the schema for the REGISTRATION database table. Some application servers create the table for you, but others require you to create it yourself. The BEA Weblogic application server does not create the database table.

```
create table REGISTRATION (THEUSER VARCHAR(40) ,
PASSWORD                    VARCHAR(40) ,
EMAILADDRESS                VARCHAR(80) ,
CREDITCARD                  VARCHAR(40) ,
BALANCE                     DOUBLE PRECISION )
```

Session Bean Classes

BidderBean and SellerBean are the session Beans. BidderBean retrieves lists of auction items, searches for an item, checks the user ID and password when someone places a bid, and stores new bids in the database. SellerBean checks the user ID and password when someone posts an auction item, and adds new auction items to the database.

Both session Beans are initially deployed as stateless Beans. A stateless Bean does not keep a record of what the client did in a previous call; whereas, a stateful Bean does. Stateful Beans are very useful if the operation is more than a simple lookup and the client operation depends on something that happened in a previous call.

Bidder Session Bean

These are the BidderBean interfaces and classes. There is no primary key class because these Beans are transient and no database access is involved. To retrieve auction items from the database, BidderBean creates an instance of AuctionItemBean, and to process bids, it creates an instance of RegistrationBean.

- Bidder (page 39)
- BidderHome (page 40)
- BidderBean (page 40)

Bidder is the remote interface. It describes what the Bean does by declaring the developer-defined methods that provide the business logic for this Bean. These methods are the ones that the client calls remotely.

BidderHome is the home interface. It describes how the Bean is created in, found in, and removed from its container. BidderBean is the Enterprise Bean. It implements SessionBean, provides the business logic for the developer-defined methods, and implements SessionBean methods for creating the Bean and setting the session context.

Seller Session Bean

SellerBean consists of the same kinds of interfaces and classes as BidderBean, except the business logic is different. Rather than describe the classes, you can browse them and refer back to the BidderBean discussion if you have questions.

- Seller (page 43)
- SellerHome (page 43)
- SellerBean (page 43)

Container Classes

The container needs classes to deploy an Enterprise Bean onto a particular Enterprise JavaBeans server, and those classes are generated with a deployment tool. Container classes include *_Stub.class and *_Skel.class classes that provide the RMI hooks on the client and server sides for marshaling (moving) data between the client program and the Enterprise JavaBeans server. In addition, implementation classes are created for the interfaces and deployment rules defined for each Bean.

- The Stub object is installed on or downloaded to the client system and provides a local proxy object for the client. It implements the remote interfaces and transparently delegates all method calls across the network to the remote object.

- The Skel object is installed on or downloaded to the server system and provides a local proxy object for the server. It unwraps data received over the network from the Stub object for processing by the server.

Examining a Container-Managed Bean

This section walks through the RegistrationBean (page 38) code to show how easy it is to have the container manage persistent data storage to an underlying medium such as a database (the default). 3: Data and Transaction Management (page 55) modifies RegistrationBean to use Bean-managed persistence to handle database access and manage transactions.

Member Variables

A container-managed environment needs to know which variables are for persistent storage and which are not. In the Java programming language, the transient keyword indicates variables to not include when data in an object is serialized and written to persistent storage. In the RegistrationBean.java class, the EntityContext variable is marked transient to indicate that its data not be written to the underlying storage medium.

EntityContext data is not written to persistent storage because its purpose is to provide information on the container's runtime context. It, therefore, does not contain data on the registered user and should not be saved to the underlying storage medium. The other variables are declared public so the container can use the Reflection API to discover them.

```
protected transient EntityContext ctx;
public String theuser, password, creditcard, emailaddress;
public double balance;
```

Create Method

The Bean's ejbCreate method is called by the container after the client program calls the create method on the remote interface and passes in the registration data. This method assigns the incoming values to the member variables that represent user data. The container handles storing and loading the data, and creating new entries in the underlying storage medium.

```
public RegistrationPK ejbCreate(String theuser,String password,
                                String emailaddress,String creditcard)
                                throws CreateException, RemoteException {

  this.theuser=theuser;
  this.password=password;
  this.emailaddress=emailaddress;
  this.creditcard=creditcard;
  this.balance=0;
```

Entity Context Methods

An entity Bean has an associated EntityContext instance that gives the Bean access to container-managed runtime information such as the transaction context.

```
//API Ref :void setEntityContext(EntityContext ectx)
    public void setEntityContext(javax.ejb.EntityContext ctx)
                                        throws RemoteException {
       this.ctx = ctx;
      }

//API Ref :void unsetEntityContext(EntityContext ectx)
      public void unsetEntityContext() throws RemoteException{
        ctx = null;
      }
```

Load Method

The Bean's ejbLoad method is called by the container to load data from the underlying storage medium. This would be necessary when BidderBean or SellerBean need to check a user's ID or password against the stored values.

You do not implement the `ejbLoad` method because the Enterprise JavaBeans container seamlessly loads the data from the underlying storage medium for you.

```
//API Ref :void ejbLoad()
    public void ejbLoad() throws RemoteException {}
```

Store Method

The Bean's ejbStore method is called by the container to save user data. This method is not implemented because the Enterprise JavaBeans container seamlessly stores the data to the underlying storage medium.

```
//API Ref :void ejbStore()
    public void ejbStore() throws RemoteException {}
```

Connection Pooling

Loading data from and storing data to a database can take a lot of time and reduce an application's overall performance. To reduce database connection time, the BEA Weblogic server uses a JDBC™ connection pool to cache database connections so connections are always available when the application needs them.

However, you are not limited to the default JDBC connection pool. You can override the Bean-managed connection pooling behavior and substitute your own. 8: Performance Techniques (page 339) explains how.

Deployment Descriptor

The remaining configuration for a container-managed persistent Bean occurs at deployment time. The following is the text-based Deployment Descriptor used in a BEA Weblogic Enterprise JavaBeans server for deploying the `Registration` Bean.

```
Text Deployment Descriptor
  (environmentProperties

    (persistentStoreProperties
      persistentStoreType          jdbc

      (jdbc
        tableName                  registration
        dbIsShared                 false
        poolName                   ejbPool
        (attributeMap
          creditcard               creditcard
          emailaddress             emailaddress
          balance                  balance
          password                 password
          theuser                  theuser
        ); end attributeMap
      ); end jdbc
    ); end persistentStoreProperties
  ); end environmentProperties
```

The deployment descriptor indicates that storage is a database whose connection is held in a JDBC™ connection pool called ejbPool. The attributeMap contains the Enterprise Bean variable on the left and the associated database field on the right.

XML Deployment Descriptor

In Enterprise JavaBeans 1.1, the deployment descriptor uses XML. Below is the equivalent configuration in XML for the `Registration` Bean.

The container-managed fields map directly to their counterpart names in the database table. The container resource authorization (res-auth) means the container handles the database login for the REGISTRATION table.

```
<persistence-type>Container</persistence-type>
<cmp-field><field-name>creditcard
    </field-name></cmp-field>
<cmp-field><field-name>emailaddress
    </field-name></cmp-field>
<cmp-field><field-name>balance
    </field-name></cmp-field>
<cmp-field><field-name>password
    </field-name></cmp-field>
<cmp-field><field-name>theuser
    </field-name></cmp-field>
<resource-ref>
<res-ref-name>registration</res-ref-name>
<res-type>javax.sql.DataSource</res-type>
<res-auth>Container</res-auth>
</resource-ref>
```

Container-Managed finder Methods

The auction house search facility is implemented as a container-managed finder method. It starts when the end user types in a search string and clicks the Submit button on the home page to locate an auction item. This section walks through the different parts of the finder-based search code. 3: Data and Transaction Management (page 55) describes how to create a Bean-managed search to handle complex queries and searches that span more than one Bean type (entity and session Beans) or database tables.

Finder-Based Search

Figure 14 shows how the browser passes the search string to the AuctionServlet.searchItem method, which then passes it to the BidderBean.getMatchingItemsList method. At this point, BidderBean.getMatchingItemsList passes the search string to the findAllMatchingItems method declared in the AuctionItemHome interface.

The findAllMatchingItems method is a finder method, and container implementations vary in how they handle calls to finder methods. BEA Weblogic containers look in the Bean's deployment descriptor for information on a Bean's finder methods. In the case of the search,

the deployment descriptor maps the search string passed to AuctionItemHome.findAll-MatchingItems to the summary field in the underlying AuctionItems database table. This tells the Enterprise JavaBeans server to retrieve data for all auction items with a summary field that contains text that matches the search string.

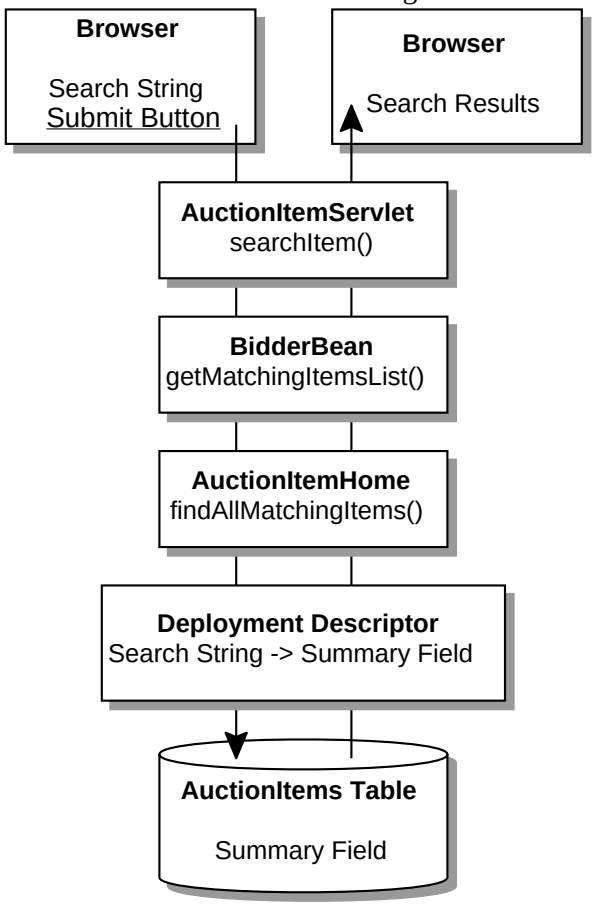


Figure 14 Searching for Auction Items

AuctionServlet.searchItems

The searchItems method retrieves the text string from the browser, creates an HTML page to display the search results, and passes the search string to the BidderBean.getMatching-ItemsList method. BidderBean is a session Bean that retrieves lists of auction items and checks the user ID and password for end users seeking to bid on auction items. The search results are returned to this method in an Enumeration variable.

```
private void searchItems(ServletOutputStream out, HttpServletRequest request)
                          throws IOException {
//Retrieve search string
  String searchString=request.getParameter(“searchString");
//Create HTML page
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Search Results");
  try {
     addLine("<BR>"+text, out);
//Look up home interface for BidderBean
     BidderHome bhome=(BidderHome) ctx.lookup("bidder");
//Create remote interface for BidderBean Bidder bid=bhome.create();
//Pass search string to BidderBean method
     Enumeration enum=(Enumeration) bid.getMatchingItemsList(searchString);
        if(enum != null) {
          displayitems(enum, out);
          addLine("", out);
        }
  } catch (Exception e) {
    addLine("AuctionServlet Search Items error", out);
    System.out.println("AuctionServlet <newlist>: "+e);
  }
    out.flush();
}
```

BidderBean.getMatchingItemsList

The BidderBean.getMatchingItemsList method calls the AuctionItemHome.findAllMatchingItems method and passes it the search string. AuctionItemBean is an entity Bean that handles auction item updates and retrievals. The search results return an Enumeration.

```
public Enumeration getMatchingItemsList(String searchString)
                                        throws RemoteException {
  Enumeration enum=null;
  try{
//Create Home interface for AuctionItemBean
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
//Pass search string to Home interface method
    enum=(Enumeration)home.findAllMatchingItems(searchString);
  }catch (Exception e) {
    System.out.println("getMatchingItemList: "+e);
    return null;
  }
  return enum;
}
```

AuctionItemHome.findAllMatchingItems

The AuctionItemHome.findAllMatchingItems method is not implemented in AuctionItemBean. Instead, the AuctionItemBean finder method implementations are defined in the AuctionItemBean deployment descriptor when BEA Weblogic containers are used.

When using these containers, even if the Bean has finder method implementations, they are ignored and the deployment descriptor settings are consulted instead.

```
//Declare method in Home interface
  public Enumeration findAllMatchingItems(String searchString)
                         throws FinderException, RemoteException;
```

AuctionItemBean Deployment Descriptor

When a Bean's finder method is called, the container consults the deployment descriptor for that Bean to find out what data the finder method needs to retrieve from the underlying database table. The container passes this information to the Enterprise JavaBeans server, which does the actual retrieval.

The deployment descriptor for AuctionItemBean provides finderDescriptors for all finder methods declared in the AuctionItemHome interface. The finderDescriptor for the findAllMatchingItems method maps the search string to the summary field in the underlying AuctionItems database table. This tells the Enterprise JavaBeans server to retrieve the data for all table rows with a summary field that matches the text in the search string.

```
(finderDescriptors
  "findAllItems()"         "(= 1 1)"
  "findAllNewItems(java.sql.Date newtoday)" "(= startdate $newtoday)"
  "findAllClosedItems(java.sql.Date closedtoday)" "(= enddate $closedtoday)"
  "findAllMatchingItems(String searchString)" "(like summary $searchString)"
); end finderDescriptors
```

Code for this Chapter

- AuctionItem (page 33)
- AuctionItemHome (page 34)
- AuctionItemBean (page 34)
- AuctionItemPK (page 37)
- Registration (page 37)
- RegistrationHome (page 37)
- RegistrationBean (page 38)
- RegistrationPK (page 39)
- Bidder (page 39)
- BidderHome (page 40)
- BidderBean (page 40)
- Seller (page 43)
- SellerHome (page 43)
- SellerBean (page 43)
- AuctionServlet (page 44)

AuctionItem

```
package auction;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that contains the methods that can be called
// in the auction bean
public interface AuctionItem extends EJBObject {
  String getDescription() throws RemoteException;
  String getSeller() throws RemoteException;
  int getId() throws RemoteException;
  String getSummary() throws RemoteException;
  double getIncrement() throws RemoteException;
```

```
  double getHighBid() throws RemoteException;
  int getBidCount() throws RemoteException;
  String getHighBidder() throws RemoteException;
  Date getStartDate() throws RemoteException;
  Date getEndDate() throws RemoteException;
  double getStartPrice() throws RemoteException;
  int setHighBid(String buyer, double amount, double increment)
                 throws RemoteException;
}
```

AuctionItemHome

```
package auction;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that is called by the client to create and return Beans
public interface AuctionItemHome extends EJBHome {
  AuctionItem create(String theseller, String thedescription,
                     int duration, double thestartprice,
                       String thesummary) throws CreateException, RemoteException;
  AuctionItem create(int id,
                     String theseller, String thedescription, int duration,
                       double thestartprice,String thesummary, double increment)
                     throws CreateException, RemoteException;
  AuctionItem findByPrimaryKey(AuctionItemPK id)
                     throws FinderException, RemoteException;
  public Enumeration findAllItems() throws FinderException, RemoteException;
  public Enumeration findAllNewItems(java.sql.Date newtoday)
                     throws FinderException, RemoteException;
  public Enumeration findAllClosedItems(java.sql.Date closedtoday)
                     throws FinderException, RemoteException;
  public Enumeration findAllMatchingItems(String searchString)
                     throws FinderException, RemoteException;
}
```

AuctionItemBean

```
package auction;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;

// Implementation of the auction bean on the server side
public class AuctionItemBean implements EntityBean {
  // one line summary of auction item
```

```
public String summary;
// username of person selling the item
public String seller;
// auction item identification number
public int id;
//auction item description
public String description;
//initial price of auction item
public double startprice=0;
// Date auction started
public java.sql.Date startdate;
// Date auction finishes
public java.sql.Date enddate;
// current maximum bid amount and bid increment
public double maxbid,increment;
// number of bids
public int bidcount;
// username of highest bidder
public String highbidder;
protected transient EntityContext ctx;
// next available auction number
public static int counter=400000;

public String getDescription() throws RemoteException {
  return description;
}
public String getSummary() throws RemoteException {
  return summary;
}
public int getId() throws RemoteException {
  return id;
}
public String getSeller() throws RemoteException {
  return seller;
}
public double getIncrement() throws RemoteException {
  return increment;
}
public double getHighBid() throws RemoteException {
  return maxbid;
}
public int getBidCount() throws RemoteException {
  return bidcount;
}
public String getHighBidder() throws RemoteException {
  return highbidder;
}
public Date getStartDate() throws RemoteException {
  return startdate;
}
public Date getEndDate() throws RemoteException {
  return enddate;
}
```

```
public double getStartPrice() throws RemoteException {
  return startprice;
}
public boolean isAuctionStillRunning() {
  Date now = (Calendar.getInstance()).getTime();
  if(enddate.getTime()>=now.getTime()) {
    return true;
  } else {
    return false;
  }
}
public int setHighBid(String buyer, double amount, double increment)
                      throws RemoteException{
  if(isAuctionStillRunning()) {
    highbidder=buyer;
    maxbid=amount;
    this.increment=increment;
    bidcount++;
    return(0);
  } else {
    return (-1);
  }
}
public void ejbCreate(String theseller, String thedescription, int duration,
                      double thestartprice, String thesummary)
                      throws CreateException, RemoteException {
  double inc=0;
  try {
    inc=Math.round((thestartprice)/10);
  } catch(Exception e) {
    System.out.println("error parsing money"+e);
  }
  ejbCreate(counter++, theseller, thedescription, duration, thestartprice,
            thesummary, inc);
  }
  public void ejbCreate(int theid, String theseller, String thedescription,
              int duration, double thestartprice, String thesummary,
                double theincrement)  throws CreateException, RemoteException {
    id=theid;
    seller=theseller;
    description=thedescription;
    Calendar currenttime=Calendar.getInstance();
    Date currentdate=currenttime.getTime();
    startdate=new java.sql.Date(currentdate.getTime());
    currenttime.add(Calendar.DATE,duration);
    enddate=new java.sql.Date((currenttime.getTime()).getTime());
    startprice=thestartprice;
    summary=thesummary;
    maxbid=startprice;
    increment=theincrement;
}
public void ejbPostCreate(int id, String theseller, String thedescription,
            int duration, double thestartprice, String thesummary,
```

```
            double increment) throws CreateException, RemoteException {
  }
  public void ejbPostCreate(String theseller, String thedescription,
            int duration, double thestartprice, String thesummary)
            throws CreateException, RemoteException {
  }
  public void setEntityContext(javax.ejb.EntityContext ctx)
            throws RemoteException {
    this.ctx = ctx;
  }
  public void unsetEntityContext() throws RemoteException {
    ctx = null;
  }
  public void ejbRemove() throws RemoteException, RemoveException { }
  public void ejbActivate() throws RemoteException { }
  public void ejbPassivate() throws RemoteException { }
  public void ejbLoad() throws RemoteException { }
  public void ejbStore() throws RemoteException { }
}
```

AuctionItemPK

```
package auction;

// Class that is used to find auction items by their unique id
public class AuctionItemPK implements java.io.Serializable {
  public int id;
}
```

Registration

```
package registration;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that contains the methods that can be called
// on the registration bean that stores users accounts
public interface Registration extends EJBObject {
  boolean verifyPassword(String password) throws RemoteException;
  String getEmailAddress() throws RemoteException;
  String getUser() throws RemoteException;
  void adjustAccount(double amount) throws RemoteException;
  double getBalance() throws RemoteException;
}
```

RegistrationHome

```
package registration;
```

```
import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that is called by the client to create and return Beans
public interface RegistrationHome extends EJBHome {
  Registration create(String theuser, String password, String emailaddress,
              String creditcard)
              throws CreateException, RemoteException;
  Registration findByPrimaryKey(RegistrationPK theuser)
              throws FinderException, RemoteException;
}
```

RegistrationBean

```
package registration;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;

// Implementation of container-managed persistence
// for the registration bean

public class RegistrationBean implements EntityBean {
  protected transient EntityContext ctx;
  public String theuser, password, creditcard, emailaddress;
 // simple information about the user including password, user identifier, etc.
  public double balance; // balance of users account

  public boolean verifyPassword(String password) throws RemoteException {
    if (this.password.equals(password)) {
       return true;
    } else {
      return false;
    }
  }

  public String getEmailAddress() throws RemoteException {
    return emailaddress;
  }
  public String getUser() throws RemoteException {
    return theuser;
  }
  public void adjustAccount(double amount) throws RemoteException {
    balance=balance+amount;
  }
  public double getBalance() throws RemoteException {
    return balance;
  }
```

```
    public void ejbCreate(String theuser, String password, String emailaddress,
              String creditcard) throws CreateException, RemoteException {
      this.theuser=theuser;
      this.password=password;
      this.emailaddress=emailaddress;
      this.creditcard=creditcard;
      this.balance=0;
    }
    public void ejbPostCreate(String theuser,String password,String emailaddress,
              String creditcard)throws CreateException, RemoteException {
    }
    public void setEntityContext(javax.ejb.EntityContext ctx)
              throws RemoteException {
      this.ctx = ctx;
    }
    public void unsetEntityContext() throws RemoteException {
      ctx = null;
    }
//API Ref :void ejbRemove()
    public void ejbRemove() throws RemoteException, RemoveException { }
//API Ref :void ejbActivate()
    public void ejbActivate() throws RemoteException { }
//API Ref :void ejbPassivate()
    public void ejbPassivate() throws RemoteException { }
    public void ejbLoad() throws RemoteException { }
    public void ejbStore() throws RemoteException { }
  }
```

RegistrationPK

```
package registration;

// Class that is used to find user by their unique user id
public class RegistrationPK implements java.io.Serializable {
  public String theuser;
  public RegistrationPK() {}
}
```

Bidder

```
package bidder;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that contains the methods that can be called
// on the bidder bean for placed auction bids
public interface Bidder extends EJBObject {
  int placeBid(int item, String user, String password, double amount)
              throws RemoteException;
```

```
  Enumeration getItemList() throws RemoteException;
  Enumeration getNewItemList() throws RemoteException;
  Enumeration getClosedItemList() throws RemoteException;
   Enumeration getMatchingItemsList(String searchString) throws RemoteException;
}
```

BidderHome

```
package bidder;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that is called by the client to create Beans
public interface BidderHome extends EJBHome {
  Bidder create() throws CreateException, RemoteException;
}
```

BidderBean

```
package bidder;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;

// Implementation of the session Bidder Bean
public class BidderBean implements SessionBean {
  protected SessionContext sctx;
  Properties p = new Properties();
  Context ctx;

  public int placeBid(int item, String buyer, String password, double amount)
                    throws RemoteException {
    double highbid, increment=0;
    int bidcount=0;

    try {
      // Find bidder details from Registration database and
      // verify that the password supplied for that bidder matches what
      // is in the database
      RegistrationHome rhome = (RegistrationHome) ctx.lookup("registration");
      RegistrationPK rpk=new RegistrationPK();
      rpk.theuser=buyer;
      Registration newbidder=rhome.findByPrimaryKey(rpk);
```

```
    if ((newbidder == null) || (!newbidder.verifyPassword(password))) {
       return(Auction.INVALID_USER);
    }
    // Now find the auction item receiving bids, and retrieve and set
    // the highest bid number of bids on that item. The next bid
    // increment is calculated based on a simple formula applied to
    // the current bid price

    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    AuctionItemPK pk=new AuctionItemPK();
    pk.id=item;
    AuctionItem aitem=home.findByPrimaryKey(pk);
    if (aitem !=null) {
      increment=aitem.getIncrement();
      highbid=aitem.getHighBid();
      bidcount=aitem.getBidCount();
        if ((amount >= (highbid+increment)) ||
                    (bidcount == 0 && amount >=highbid )) {
          if(aitem.setHighBid(buyer, amount, (amount)/10)>=0 ) {
            // bid beat other bids!
            return Auction.HIGH_BID;
          } else {
            return Auction.AUCTION_OVER;
          }
        } else { //  amount < highbid+increment
          return Auction.OUT_BID;
        }
    } else { aitem = null
      // Item does not exist
      return Auction.INVALID_ITEM;
    }
  } catch(Exception e) {
    System.out.println("placeBid: "+e);
    return Auction.INVALID_ITEM;
  }
}
public Enumeration getItemList() throws RemoteException {
  // Return list of all items in the auction database
  Enumeration enum=null;
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    enum=(Enumeration)home.findAllItems();
  } catch (Exception e) {
    System.out.println("getItemList: "+e);
    return null;
  }
  return enum;
}
public Enumeration getNewItemList() throws RemoteException {
  // Return list of all new items in the auction database
  Enumeration enum=null;
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
```

```
    Calendar currenttime=Calendar.getInstance();
    enum = (Enumeration)home.findAllNewItems(
           new java.sql.Date((currenttime.getTime()).getTime()));
   } catch (Exception e) {
     System.out.println("getNewItemList: "+e);
     return null;
   }
   return enum;
}
public Enumeration getClosedItemList() throws RemoteException {
  // Return list of all completed auction items in the auction database
  Enumeration enum=null;
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    Calendar currenttime=Calendar.getInstance();
    enum = (Enumeration)home.findAllClosedItems(
           new java.sql.Date((currenttime.getTime()).getTime()));
  } catch (Exception e) {
    System.out.println("getClosedItemList: "+e);
    return null;
  }
    return enum;
}
public Enumeration getMatchingItemsList(String searchString)
                                        throws RemoteException {
  // Return list of all items matching searchString in the auction database
  Enumeration enum=null;
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    enum=(Enumeration)home.findAllMatchingItems(searchString);
  } catch (Exception e) {
    System.out.println("getMatchingItemList: "+e);
    return null;
  }
  return enum;
}
public void ejbCreate() throws CreateException, RemoteException {
  Properties p = new Properties();
  p.put(Context.INITIAL_CONTEXT_FACTORY,
                        "weblogic.jndi.TengahInitialContextFactory");
  try {
    ctx = new InitialContext(p);
  } catch(Exception e) {
    System.out.println("create exception: "+e);
   }
}
public void setSessionContext(SessionContext sctx) throws RemoteException {
  this.sctx = sctx;
}
public void unsetSessionContext() throws RemoteException {
  sctx = null;
}
public void ejbRemove() {}
```

```
  public void ejbActivate() throws RemoteException { }
  public void ejbPassivate() throws RemoteException { }
}
```

Seller

```
package seller;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that contains the methods that can be called
// on the seller bean to create new auction items
public interface Seller extends EJBObject {int insertItem(String user,
                String password, String description, int auctiondays,
                double startprice, String summary) throws RemoteException;
}
```

SellerHome

```
package seller;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

// Interface that is called by the client to create Beans
public interface SellerHome extends EJBHome {
  Seller create() throws CreateException, RemoteException;
}
```

SellerBean

```
package seller;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;

// Implementation of the session seller bean
public class SellerBean implements SessionBean {
  protected SessionContext ctx;
  Properties p = new Properties();
  public int insertItem(String seller, String password, String description,
```

```
                        int auctiondays, double startprice, String summary)
                       throws RemoteException {
    try {
      Context ectx = new InitialContext(p);
       RegistrationHome rhome = (RegistrationHome) ectx.lookup("registration");
      RegistrationPK rpk=new RegistrationPK();
      rpk.theuser=seller;
      Registration newseller=rhome.findByPrimaryKey(rpk);
      if ((newseller == null) || (!newseller.verifyPassword(password))) {
           return(Auction.INVALID_USER);
      }
      AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
      AuctionItem aitem= home.create(seller, description, auctiondays,
                                     startprice, summary);
      if (aitem == null) {
         return Auction.INVALID_ITEM;
      } else {
        return(aitem.getId());
      }
    } catch(Exception e){
      System.out.println("insert problem="+e);
      return Auction.INVALID_ITEM;
    }
  }
  public void ejbCreate() throws CreateException, RemoteException {
    Properties p = new Properties();
    p.put(Context.INITIAL_CONTEXT_FACTORY,
                       "weblogic.jndi.TengahInitialContextFactory");
  }
  public void setSessionContext(SessionContext ctx) throws RemoteException {
    this.ctx = ctx;
  }
  public void unsetSessionContext() throws RemoteException {
    ctx = null;
  }
  public void ejbRemove() {}
  public void ejbActivate() throws RemoteException {}
  public void ejbPassivate() throws RemoteException {}
}
```

AuctionServlet

```
package auction;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.naming.*;
import javax.ejb.*;
import java.rmi.RemoteException;
import java.util.*;
import java.text.NumberFormat;
```

```
import bidder.*;
import registration.*;
import seller.*;
import pool.*;
import search.*;
// The client side interface to EJB server is the AuctionServlet that
// displays results using HTML

public class AuctionServlet extends HttpServlet {
  Context ctx=null;
  private String detailsTemplate;

  public void init(ServletConfig config) throws ServletException{
    super.init(config);
    try {
      ctx = getInitialContext();
    } catch (Exception e){
      System.err.println("failed to contact EJB server"+e);
    }
    try {
      detailsTemplate=readFile(config.getInitParameter("detailstemplate"));
    } catch(IOException e) {
       System.err.println("Error in AuctionServlet <init>"+e);
    }
  }

  public void service(HttpServletRequest request, HttpServletResponse response)
                    throws IOException {
    // main input loop, this extracts a parameter from the
    // HTML post called action.
    String cmd;
    response.setContentType("text/html");
    ServletOutputStream out = response.getOutputStream();
    if (ctx == null ) {
      try {
         ctx = getInitialContext();
      }catch (Exception e){
        System.err.println("failed to contact EJB server"+e);
      }
    }
    cmd=request.getParameter("action");
    if (cmd !=null)  {
      if (cmd.equals("list")) {
        // Return an HTML list of all auction items
        listAllItems(out);
      } else if(cmd.equals("newlist")) {
        // Return an HTML list of all new auction items
        listAllNewItems(out);
      } else if(cmd.equals("search")) {
        // Return an HTML list of auction items where the description
        // matches the search string
        searchItems(out, request);
      } else if(cmd.equals("close")) {
```

```
      // Return an HTML list of all closed auction items
      listClosingItems(out);
    } else if (cmd.equals("insert")) {
      // Add a new auction item
      insertItem(out, request);
    } else if (cmd.equals("details")) {
      // Return an HTML page describing the chosen auction item
      itemDetails(out, request );
    } else if (cmd.equals("bid")) {
      // Bid on the identified auction item
      itemBid(out, request) ;
    } else if (cmd.equals("register")) {
      // Create a new user
      registerUser(out, request);
    }
  } else {
    // No command set
    setTitle(out, "error");
  }
  setFooter(out);
  out.flush();
}

static private void addLine(String message, ServletOutputStream out)
                          throws IOException {
  // Add a line to the returned html output
  if(message !=null) {
    out.println("<BR>"+message);
  }
}

static public Context getInitialContext() throws Exception {
 Properties p = new Properties();
 p.put(Context.INITIAL_CONTEXT_FACTORY,
                "weblogic.jndi.TengahInitialContextFactory");
      return new InitialContext(p);
}
private void listAllItems(ServletOutputStream out) throws IOException{
  // Return an HTML page with a summarized list of all auction items
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Auction results");
  try {
    addLine("<BR>"+text, out);
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getItemList();
    if(enum != null) {
      displayitems(enum, out);
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List All Items error",out);
    System.out.println("AuctionServlet <list>:"+e);
```

```
  }
  out.flush();
}
private void listAllNewItems(ServletOutputStream out) throws IOException {
  setTitle(out, "New Auction Items");
  try {
    addLine("", out);
    String text = "Click Item number for description and to place bid.";
    addLine(text, out);
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getNewItemList();
    if(enum != null) {
      displayitems(enum, out);
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List New Items error", out);
    System.out.println("AuctionServlet <newlist>:"+e);
  }
  out.flush();
}
private void searchItems(ServletOutputStream out, HttpServletRequest request)
                    throws IOException {
  // Return an HTML page with a list of auction items matching
  // the search string posted in parameter searchString
  String searchString=request.getParameter("searchString");
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Search Results");
  try {
    addLine("<BR>"+text, out);
    AuctionItemHome ahome = (AuctionItemHome) ctx.lookup("auctionitems");
    SearchHome shome=(SearchHome) ctx.lookup("search");
    Search search=shome.create();
      Enumeration enum=(Enumeration)search.getMatchingItemsList(searchString);
    addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0><TR><TH>Item</TH>
             <TH>Summary</TH><TH>Current High bid</TH><TH>Number of bids</TH>
            <TH>Closing Date</TH></TR>", out);
    while ((enum != null) && (enum.hasMoreElements())) {
      while(enum.hasMoreElements()) {
           AuctionItem ai=ahome.findByPrimaryKey((
                      AuctionItemPK)enum.nextElement());
           displayLineItem(ai, out);
      }
    }
    addLine("</TABLE>", out);
  } catch (Exception e) {
    addLine("AuctionServlet Search Items error", out);
    System.out.println("AuctionServlet <searchItems>:"+e);
  }
  out.flush();
}
private void listClosingItems(ServletOutputStream out) throws IOException{
```

```
    setTitle(out, "Items Due to Close Today");
    String text = "Click Item number for description and to place bid.";
    setTitle(out, "Items Due to Close Today");
    try {
      addLine("<BR>"+text, out);
      BidderHome bhome=(BidderHome) ctx.lookup("bidder");
      Bidder bid=bhome.create();
      Enumeration enum=(Enumeration)bid.getClosedItemList();
      if(enum != null) {
        displayitems(enum, out);
        addLine("", out);
      }
    } catch (Exception e) {
      addLine("AuctionServlet List Closed Items error", out);
      System.out.println("AuctionServlet <close>:"+e);
    }
    out.flush();
  }
  private void insertItem(ServletOutputStream out, HttpServletRequest request)
                        throws IOException{
    setTitle(out, "Insert Auction Item");
    String seller=request.getParameter("seller");
    String description=request.getParameter("description");
    String summary=request.getParameter("summary");
    String password=request.getParameter("password");
    String price=request.getParameter("startprice");
    double startprice=0.00;
    try {
      startprice = ((Number)NumberFormat.getCurrencyInstance().parse(
                   "$"+price)).doubleValue();
    } catch(java.text.ParseException e){
      System.out.println("money problem"+e);
      return;
    } catch (NumberFormatException e) {
      System.out.println("money problem"+e);
      return;
    }
    int auctiondays=7;
    try {
      auctiondays=Integer.parseInt(request.getParameter("auctiondays"));
    } catch(NumberFormatException e){
      System.out.println("problem parsing auction days"+e);
      return;
    }
    if(auctiondays<=0 || auctiondays>7) {
      auctiondays=7;
    }
    try {
      SellerHome home = (SellerHome) ctx.lookup("seller");
      Seller si= home.create();
      if(si != null) {
        int result= si.insertItem(seller, password, description,
                                  auctiondays, startprice, summary);
```

```
      if(result >0) {
        addLine("Inserted item "+summary, out);
        addLine("Details available
          <A HREF=/ AuctionServlet?action=details&item="+result+">here</A>",
           out);
      } else {
        addLine("Error inserting item", out);
        return;
      }
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet Insert Item error", out);
    System.out.println("AuctionServlet <insert>:"+e);
  }
  out.flush();
}
private void itemDetails(ServletOutputStream out, HttpServletRequest request)
                         throws IOException{
  // Return an HTML page with details about the auction identified by
  // the posted parameter item
  setTitle(out, "Item Details");
  String item=request.getParameter("item");
  int itemid=0;
  try {
    itemid=Integer.parseInt(item);
  } catch (NumberFormatException e) {
    System.out.println("problem with item id "+itemid);
    return;
  }
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    AuctionItemPK pk=new AuctionItemPK();
    pk.id=itemid;
    AuctionItem ai=home.findByPrimaryKey(pk);
    displayPageItem(ai, out);
    addLine("<BR><HR><P>Do you want to bid on this item?", out);
    addLine("<FORM ACTION=\"/AuctionServlet\" METHOD=\"POST\">", out);
    addLine("<TR>Enter your user id:<INPUT TYPE=\"TEXT\"
            SIZE=20 NAME=\"buyer\"> ", out);
     addLine("and password:<INPUT TYPE=\"PASSWORD\" SIZE=20 NAME=\"password\">
            </TR>", out);
    addLine("<TR>Your bid amount:<INPUT TYPE=\"TEXT\"
            SIZE=20 NAME=\"amount\"></TR>", out);
    addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"action\" VALUE=\"bid\">", out);
    addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"item\" VALUE=\""+itemid+"\">",
            out);
     addLine("<INPUT TYPE=\"SUBMIT\" VALUE=\"Place Bid\" NAME=\"Bid\"></FORM>",
            out);
  } catch (Exception e) {
    addLine("AuctionServlet List Item error", out);
    System.out.println("AuctionServlet <details>:"+e);
  }
```

```
  out.flush();
}
private void itemBid(ServletOutputStream out, HttpServletRequest request)
                    throws IOException{
  // Place a bid on the item specified in the item posted parameter
  setTitle(out, "Item Bid");
  String item=request.getParameter("item");
  String buyer=request.getParameter("buyer");
  String password=request.getParameter("password");
  String bid=request.getParameter("amount");
  int itemid=0;
  double bidamount=0;
  try {
    itemid=Integer.parseInt(item);
  } catch (NumberFormatException e) {
    System.out.println("problem with item id " +itemid);
    return;
  }
  try {
    bidamount=Double.valueOf(bid).doubleValue();
  } catch (NumberFormatException e) {
    System.out.println("problem with bid" +bid);
    return;
  }
  try {
    BidderHome bhome = (BidderHome) ctx.lookup("bidder");
    Bidder bidbean=bhome.create();
    int result=bidbean.placeBid(itemid, buyer, password, bidamount);
    switch(result) {
      case Auction.OUT_BID:
        addLine("Your bid was not high enough", out);
        break;
      case Auction.HIGH_BID:
        addLine("You are the high bidder", out);
        break;
      case Auction.AUCTION_OVER:
        addLine("This auction has finished", out);
        break;
      case Auction.INVALID_USER:
        addLine("Invalid user or password", out);
        break;
      default:
        addLine("Problem submitting bid", out);
     }
   } catch (Exception e) {
     addLine("AuctionServlet Bid error", out);
     System.out.println("AuctionServlet <bid>:"+e);
   }
   out.flush();
}
 private void registerUser(ServletOutputStream out, HttpServletRequest request)
                         throws IOException{
  // Register a new user
```

```
  setTitle(out, "Register User");
  String user=request.getParameter("user");
  String password=request.getParameter("password");
  String creditcard=request.getParameter("creditcard");
  String emailaddress=request.getParameter("emailaddress");
  try {
    RegistrationHome home = (RegistrationHome) ctx.lookup("registration");
    Registration reguser=
              home.create(user, password, emailaddress, creditcard);
    if(reguser != null) {
       addLine("Created user: "+reguser.getUser(), out);
       addLine(" ", out);
    } else {
       addLine("Error creating user id, possibly already exists ", out);
    }
  } catch(Exception e) {
    addLine("AuctionServlet registration error", out);
    System.out.println("AuctionServlet <register>:"+e);
  }
  out.flush();
}
static private void displayitems(Enumeration e,  ServletOutputStream out)
                    throws Exception{
  // Step through an enumerated list of auction items and
  // display each summarized line
  addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0>
          <TR><TH>Item</TH><TH>Summary</TH>
          <TH>Current High bid</TH><TH>Number of bids</TH>
          <TH>Closing Date</TH></TR>", out);
  while((e !=null) && (e.hasMoreElements())) {
    while(e.hasMoreElements()) {
      displayLineItem((AuctionItem) e.nextElement(), out);
    }
  }
  addLine("</TABLE>", out);
}
static private void displayLineItem(AuctionItem auctionItem,
                    ServletOutputStream out)
                    throws RemoteException, IOException {
  // Return an HTML page with a summarized list of auction items
  StringBuffer line= new StringBuffer();
  int bidcount=auctionItem.getBidCount();
  int key=(int)auctionItem.getId();
  line.append("<TR><TD>
        <A HREF=/AuctionServlet?action=details&item="+key+">"+key+"</A></TD>");
  line.append("<TD>"+auctionItem.getSummary() +"</TD>");
  if(bidcount >0) {
    line.append("<TD>"+
    NumberFormat.getCurrencyInstance().format(auctionItem.getHighBid())
         + "</TD>");
  } else {
    line.append("<TD>-</TD>");
  }
```

```
  line.append("<TD>"+bidcount+"</TD>");
  line.append("<TD>"+auctionItem.getEndDate() +"</TD></TR>");
  addLine(line.toString(), out);
}
static private void displayPageItem(AuctionItem auctionItem,
                    ServletOutputStream out)
                    throws RemoteException, IOException {
  // Return an HTML page with one auction item on it
  int bidcount=auctionItem.getBidCount();
  addLine(auctionItem.getSummary(), out);
  addLine("Auction Item Number: "+ auctionItem.getPrimaryKey(), out);
  if(bidcount >0) {
      addLine("<P>Current price: "+NumberFormat.getCurrencyInstance().format(
            auctionItem.getHighBid()), out);
       addLine("Minimum increment: "+NumberFormat.getCurrencyInstance().format(
            auctionItem.getIncrement()), out);
  } else {
    addLine("<P>Current price: "+NumberFormat.getCurrencyInstance().format(
            auctionItem.getStartPrice()), out);
  }
  addLine("# of bids: "+bidcount, out);
  addLine("<P>Auction Started: "+auctionItem.getStartDate(), out);
  addLine("Auction Ends: "+auctionItem.getEndDate(), out);
  addLine("<P>Seller: "+auctionItem.getSeller(), out);
  if(bidcount >0) {
    addLine("High Bidder: "+auctionItem.getHighBidder(), out);
  } else {
    addLine("High Bidder: "+"-", out);
  }
  addLine("<HR><P>", out);
  addLine("Description: "+auctionItem.getDescription(), out);
}
private String readFile (String file) throws IOException {
  // Convenience method to read a file into memory
  if(file != null) {
    StringBuffer buffer = new StringBuffer();
    BufferedReader reader = new BufferedReader (new FileReader (file));
    String line;
    while( (line=reader.readLine()) != null ) {
      buffer.append (line+'\n');
    }
    reader.close();
    return buffer.toString();
  } else {
    return null;
  }
}
private void setTitle(ServletOutputStream out, String title) {
  // Set the title on the html page
  try {
    out.println("<HTML><HEAD><TITLE>"+title+"</TITLE></HEAD>");
    out.println("<BODY BGCOLOR=\"WHITE\">");
  } catch(IOException e) {
```

```
      System.out.println("Unable to set title"+e);
    }
  }
  private void setFooter(ServletOutputStream out) {
    // Finish the html page with a navigation bar to choose
    // other auction commands
    try {
      out.println("<HR><CENTER> <A HREF=\"registration.html\">Register</A>");
      out.println("<STRONG>|</STRONG>");
        out.println("<A HREF=\"/AuctionServlet?action=newlist\">New Items</A>");
      out.println("<STRONG>|</STRONG>");
       out.println("<A HREF=\"/AuctionServlet?action=close\">Closing Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"/AuctionServlet?action=list\">All Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"sell.html\">Sell Items</A>");
      out.println("</CENTER>");
    } catch (IOException e) {
      System.out.println("Unable to set footer"+e);
    }
  }
}
```

3: Data and Transaction Management

When you use the Enterprise JavaBeans™ architecture, data is written to and read from the database without your writing any SQL code to do it. But what if you want to write your own SQL commands or manage transactions? You can override the built-in container-managed persistence and implement Bean-managed persistence using your own data storage and transaction management code.

Bean-managed persistence comes in handy when you want more control than the container-managed persistence provides. For example you might want to override the default of most containers to map the data in one Bean to one row in a table, implement your own finder methods, or customize caching.

This chapter converts the RegistrationBean class from 2: Auction House Application (page 13) to provides its own SQL commands for reading from and writing to the database. It also explains how you can write your own transaction management code and implement a more complex search than you can get with the finder-based search described in 2: Auction House Application (page 13).

Covered in this Chapter

- Bean-Managed Persistence & the JDBC Platform (page 55)
- Managing Transactions (page 59)
- Bean-Managed finder Methods (page 68)
- Code for this Chapter (page 72)

Bean-Managed Persistence & the JDBC Platform

There might be times when you want to override container-managed persistence and implement entity or session Bean methods to use the SQL commands you provide. This type of Bean-managed persistence can be useful if you need to improve performance or map data in

multiple Beans to one row in a database table. This section shows you how to convert the RegistrationBean.java class to access the database with the JDBC PreparedStatement class.

Connect to Database

This version of the RegistrationBean (SQL) (page 72) class establishes a connection to the database by instantiating a static Driver class and providing the getConnection method. The getConnection method queries the static DriverManager class for a registered database driver that matches the Uniform Resource Locator (URL). In this case, the URL is weblogic.jdbc.jts.Driver.

```
//Create static instance of database driver
static {
  new weblogic.jdbc.jts.Driver();
}

//Get registered driver from static instance
public Connection getConnection() throws SQLException{
  return DriverManager.getConnection( "jdbc:weblogic:jts:ejbPool");
}
```

Create Method

The ejbCreate method assigns values to data member variables, gets a connection to the database, and creates an instance of the java.sql.PreparedStatement class to execute the SQL statement for writing the data to the registration table in the database.

A PreparedStatement object is created from a SQL statement which is sent to the database and precompiled before any data is sent. You call the appropriate setXXX statements on the PreparedStatement object to send the data. Keeping the PreparedStatement and Connection objects as private instance variables greatly reduces overhead because the SQL statement does not have to be compiled every time data is sent.

The last thing the ejbCreate method does is create a primary key class with the user ID, and return it to the container.

```
public RegistrationPK ejbCreate(String theuser, String password,
                                String emailaddress, String creditcard)
                                throws CreateException, RemoteException {
  this.theuser=theuser;
  this.password=password;
  this.emailaddress=emailaddress;
  this.creditcard=creditcard;
  this.balance=0;
  try {
    con=getConnection();
```

```
//API Ref :PrepareStatement prepareStatement(String sql)
       ps=con.prepareStatement("insert into registration (theuser, password,
                               emailaddress, creditcard, balance)
       values (  ?, ?, ?, ?, ?)");
       ps.setString(1, theuser);
       ps.setString(2, password);
       ps.setString(3, emailaddress);
       ps.setString(4, creditcard);
       ps.setDouble(5, balance);
       if (ps.executeUpdate() != 1) {
         throw new CreateException ("JDBC did not create a row");
       }
       RegistrationPK primaryKey = new RegistrationPK();
       primaryKey.theuser = theuser;
       return primaryKey;
       } catch (CreateException ce) {
         throw ce;
       } catch (SQLException sqe) {
         throw new CreateException (sqe.getMessage());
       } finally {
         try {
            ps.close();
         } catch (Exception ignore) {}
         try {
            con.close();
         } catch (Exception ignore) {}
       }
   }
```

Load Method

The `ejbLoad` method gets the primary key from the entity context and passes it to the refresh method which loads the data.

```
public void ejbLoad() throws RemoteException {
  try {
    refresh((RegistrationPK) ctx.getPrimaryKey());
  } catch (FinderException fe) {
    throw new RemoteException (fe.getMessage());
  }
}
```

Refresh Method

The `refresh` method is programmer-supplied code to load the data from the database. It checks the primary key value, gets a connection to the database, and creates a PreparedStatement object for querying the database for the user specified in the primary key. Data is read from the database into a ResultSet and assigned to the global member variables so the RegistrationBean has the most up-to-date information for the user.

```
    private void refresh(RegistrationPK pk)throws FinderException, RemoteException {
      if (pk == null) {
        throw new RemoteException ("primary key cannot be null");
      }
      Connection con = null;
      PreparedStatement ps = null;
      try {
         con=getConnection();
         ps=con.prepareStatement("select password, emailaddress, creditcard,
                                  balance from registration where theuser = ?");
         ps.setString(1, pk.theuser);
         ps.executeQuery();
//API Ref :ResultSet getResultSet()
         ResultSet rs = ps.getResultSet();
         if (rs.next()) {
           theuser = pk.theuser;
//API Ref :String getString(String columnName)
           password = rs.getString(1);
           emailaddress = rs.getString(2);
           creditcard = rs.getString(3);
           balance = rs.getDouble(4);
         } else {
            throw new FinderException
                        ("Refresh: Registration ("+ pk.theuser + ") not found");
         }
      } catch (SQLException sqe) {
         throw new RemoteException (sqe.getMessage());
      }
      finally {
         try {
            ps.close();
         } catch (Exception ignore) {}
         try {
            con.close();
         } catch (Exception ignore) {}
      }
    }
```

Store Method

This method gets a database connection and creates a PreparedStatement to update the database.

```
    public void ejbStore() throws RemoteException {
      Connection con = null;
      PreparedStatement ps = null;
      try {
         con = getConnection();
//API Ref :PrepareStatement prepareStatement(String sql)
         ps = con.prepareStatement("update registration set password = ?,
                   emailaddress = ?, creditcard = ?, balance = ? where theuser = ?");
//API Ref :void setString(int index, String s)
```

```
        ps.setString(1, password);
        ps.setString(2, emailaddress);
        ps.setString(3, creditcard);
//API Ref :void setDouble(int index, double doublevalue)
        ps.setDouble(4, balance);
        ps.setString(5, theuser);
//API Ref :int executeUpdate()
        int i = ps.executeUpdate();
        if (i == 0) {
          throw new RemoteException (
                          "ejbStore: Registration (" + theuser + ") not updated");
        }
      } catch (RemoteException re) {
          throw re;
      } catch (SQLException sqe) {
          throw new RemoteException (sqe.getMessage());
      } finally {
        try {
          ps.close();
        } catch (Exception ignore) {}
         try {
            con.close();
        } catch (Exception ignore) {}
      }
    }
```

Find Method

The ejbFindByPrimaryKey method matches the signature of the findByPrimaryKey method in the RegistrationHome (page 37) interface. It calls the refresh method to get or refresh the user data for the user specified by the primary key. The container-managed version of RegistrationBean (page 38) does not implement this method because the container handles getting and refreshing the user data.

```
    public RegistrationPK ejbFindByPrimaryKey(RegistrationPK pk)
                          throws FinderException, RemoteException {
      if ((pk == null) || (pk.theuser == null)) {
         throw new FinderException ("primary key cannot be null");
      }
      refresh(pk);
      return pk;
    }
```

Managing Transactions

Wouldn't it be great if every operation your application attempts succeeds? Unfortunately, in the multithreaded world of distributed applications and shared resources, this is not always

possible. Why? First of all, shared resources must maintain a consistent view of the data to all users. This means reads and writes have to be managed so users do not overwrite each other's changes, or transaction errors do not corrupt data integrity. Also, if you factor in intermittent network delays or dropped connections, the potential for operations to fail in a web-based application increases as the number of users increases.

If operation failures are unavoidable, the next best thing is to recover safely, and that is where transaction management fits in. Modern databases and transaction managers let you undo and restore the state of a failed sequence of operations to ensure the data is consistent for access by multiple threads.

This section adds code to the container-managed SellerBean (page 43) so it can manage its auction item insertion transaction beyond the default transaction management provided by its container.

Why Manage Transactions?

When you access databases using the JDBC API, all operations are run with an explicit auto commit by default. This means any other application viewing this data will see the updated data after each JDBC call. For simple applications this may be acceptable, but consider the auction application and the sequences that occur when SellerBean inserts an auction item. The user's account is first charged for listing the item, and the item is then added to the list of items up for auction. These operations require RegistrationBean to debit the account and AuctionItemBean to add the item to the auction list.

In auto commit mode, if the auction item insertion fails, only the listing is backed out, and you have to manually adjust the user's account to refund the listing charge. In the meantime, another thread might try to deduct from the same user's account, find no credit left, and abort when perhaps a few milliseconds later it would have completed. There are two ways to ensure the debit is backed out if the auction item insertion fails:

- Add session synchronization code to a container-managed session Bean to gain control over transaction commits and roll backs.
- Configure JDBC services to transaction commit mode and add code to start, stop, commit, and rollback the transaction. This is a Bean-managed transaction and can be used with an entity or session Bean.

Session Synchronization

A container-managed session Bean can optionally include session synchronization code to manage the default auto commit provided by the container. Session synchronization code lets the container notify the Bean when important points in the transaction are reached. Upon

receiving the notification, the Bean can take any needed actions before the transaction proceeds to the next point.

Note: A session Bean using Bean-managed transactions does not need session synchronization because the Bean is in full control of the commit.

Container-Managed Example

SellerBean is a session Bean that uses RegistrationBean and AuctionItemBean in the following ways:

- RegistrationBean checks the user ID and password when someone posts an auction item and debit the seller's account for a listing.
- AuctionItemBean adds new auction items to the database.

The transaction begins in the SellerBean.insertItem method with the account debit and ends when the entire transaction either commits or rolls back. The entire transaction including the 50 cents debit rolls back if the auction item is null (the insertion failed), or if an exception is caught. If the auction item is not null and the insertion succeeds, the entire transaction including the 50 cents debit commits.

Session Synchronization Code

To use session synchronization, a session Bean implements the SessionSynchronzation interface and its three methods, afterBegin, beforeCompletion, and afterCompletion. This example adapts the SellerBean (Session Synchronization) (page 75) code to use session synchronization.

```
public class SellerBean implements SessionBean, SessionSynchronization {
  private transient SessionContext ctx;
  private transient Properties p = new Properties();
  private transient boolean success = true;

  public void afterBegin() {}
  public void beforeCompletion() {
    if (!success ) {
      ctx.setRollbackOnly();
    }
  }
  public void afterCompletion(boolean state) {}
```

afterBegin

The container calls the `afterBegin` method before the debit to notify the session Bean a new transaction is about to begin. You can implement this method to do any preliminary database

work that might be needed for the transaction. In this example, no preliminary database work is needed so this method has no implementation.

beforeCompletion

The container calls the `beforeCompletion` method when it is ready to write the auction item and debit to the database, but before it actually does (commits). You can implement this method to write out any cached database updates or roll back the transaction. In this example, the method calls the setRollbackOnly method on its session context in the event the success variable is set to false during the transaction.

afterCompletion

The container calls the `afterCompletion` method when the transaction commits. A boolean value of true means the data committed and false means the transaction rolled back. The method uses the boolean value to determine if it needs to reset the Bean's state in the case of a rollback. In this example, there is no need to reset the state in the event of a failure. The insertItem method shown here has comments to indicate where SessionSynchronization methods are called.

```
public int insertItem(String seller, String password, String description,
    int auctiondays, double startprice, String summary) throws RemoteException {
  try {
    Context jndiCtx = new InitialContext(p);
    RegistrationHome rhome = (RegistrationHome) sCtx.lookup("registration");
    RegistrationPK rpk=new RegistrationPK();
    rpk.theuser=seller;
    Registration newseller=rhome.findByPrimaryKey(rpk);
    if((newseller == null) || (!newseller.verifyPassword(password))) {
      return(Auction.INVALID_USER);
    }
//Call to afterBegin
    newseller.adjustAccount(-0.50);
    AuctionItemHome home = (AuctionItemHome) jndiCtx.lookup("auctionitems");
    AuctionItem ai= home.create(seller, description, auctiondays,
                                startprice, summary);
    if(ai == null) {
      success=false;
      return Auction.INVALID_ITEM;
    } else {
      return(ai.getId());
    }
  } catch(Exception e){
      System.out.println("insert problem="+e);
      success=false;
      return Auction.INVALID_ITEM;
    }
```

```
//Call to beforeCompletion
//Call to afterCompletion

 }
```

Transaction Commit Mode

If you configure the JDBC services to transaction commit mode, you can have the Bean manage the transaction. To set the JDBC services to commit, call con.setAutoCommit(false) on your JDBC connection. Not all JDBC drivers support commit mode, but to have the Bean control and manage transactions, you need a JDBC driver that does.

Transaction commit mode lets you add code that creates a safety net around a sequence of dependent operations. The Java™ Transaction API (JTA) provides the hooks you need to create that safety net. But, if you are using the Enterprise JavaBeans architecture, you can do it with a lot less code. You only have to configure the Enterprise JavaBeans server, and specify where the transaction starts, stops, rolls back, and commits in your code.

Server Configuration

Configuring the Enterprise JavaBeans server involves specifying the following settings in a configuration file for each Bean:

- An isolation level to specify how exclusive a transaction's access to shared data is.
- A transaction attribute to specify how to handle Bean-managed or container-managed transactions that continue in another Bean.
- A transaction type to specify whether the transaction is managed by the container or the Bean.

For example, you would specify these settings for the BEA Weblogic server in a DeploymentDescriptor.txt file for each Bean. Here is the part of the DeploymentDescriptor.txt for SellerBean that specifies the isolation level and transaction attribute. A description of the settings follows.

```
(controlDescriptors
  (DEFAULT
     isolationLevel            TRANSACTION_SERIALIZABLE
     transactionAttribute      REQUIRED
     runAsMode                 CLIENT_IDENTITY
     runAsIdentity             guest
  ); end DEFAULT
); end controlDescriptors
```

Here is the equivalent Enterprise JavaBeans 1.1 extended markup language (XML) description that specifies the transaction type. In this example SellerBean is container managed.

```
<container-transaction>
    <method>
        <ejb-name>SellerBean</ejb-name>
        <method-name>*</method-name>
    </method>
    <transaction-type>Container</transaction-type>
    <trans-attribute>Required</trans-attribute>
</container-transaction>
```

In this example, SellerBean is Bean managed.

```
<container-transaction>
    <method>
        <ejb-name>SellerBean</ejb-name>
        <method-name>*</method-name>
    </method>
    <transaction-type>Bean</transaction-type>
    <trans-attribute>Required</trans-attribute>
</container-transaction>
```

Transaction Attribute Descriptions

An Enterprise Bean uses a transaction attribute to specify whether a Bean's transactions are managed by the Bean itself or by the container, and how to handle transactions that started in another Bean.

The Enterprise JavaBeans server can control only one transaction at a time. This model follows the example set by the OMG Object Transaction Service (OTS), and means the current Enterprise JavaBeans specification does not provide a way to nest transactions. A nested transaction is a new transaction that starts from within an existing transaction. While transaction nesting is not allowed, continuing an existing transaction in another Bean is okay.

When a Bean is entered, the server creates a transaction context to manage the transaction. When the transaction is managed by the Bean, you access the context to begin, commit, and rollback the transaction as needed.

These are the transaction attributes with a brief description for each one. The attribute names changed between the 1.0 and 1.1 versions of the Enterprise JavaBeans specification.

1.1 Specification	1.0 Specification
REQUIRED	TX_REQUIRED
Container-managed transaction. The server either starts and manages a new transaction on behalf of the user or continues using the transaction that was started by the code that called this Bean.	
REQUIRESNEW	TX_REQUIRED_NEW
Container-managed transaction. The server starts and manages a new transaction. If an existing transaction starts this transaction, it suspends until this transaction completes.	
Specified as Bean transaction-type in deployment descriptor	TX_BEAN_MANAGED
Bean-managed transaction. You access the transaction context to begin, commit, or rollback the transaction as needed.	
SUPPORTS	TX_SUPPORTS
If the code calling this Bean has a transaction running, include this Bean in that transaction.	
NEVER	TX_NOT_SUPPORTED
If the code calling a method in this Bean has a transaction running, suspend that transaction until the method called in this Bean completes. No transaction context is created for this Bean.	
MANDATORY	TX_MANDATORY
The transaction attribute for this Bean is set when another Bean calls one of its methods. In this case, this Bean gets the transaction attribute of the calling Bean. If the calling Bean has no transaction attribute, the method called in this Bean throws a TransactionRequired exception.	

Isolation Level Descriptions

An enterprise Bean uses an isolation level to negotiate its own interaction with shared data, and the interaction of other threads with the same shared data. As the name implies, there are various levels of isolation with TRANSACTION_SERIALIZABLE providing the highest level of data integrity.

Note: Be sure to verify that your database can handle the level you choose. In the Enterprise JavaBeans 1.1 specification, only Bean-managed persistence session Beans can set the isolation level. If the database cannot handle the isolation level, the Enterprise JavaBeans server will get a failure when it tries to call the setTransactionIsolation JDBC method.

TRANSACTION_SERIALIZABLE. This level provides maximum data integrity. The Bean gets what amounts to exclusive access to the data. No other transaction can read or write this data until the serializable transaction completes.

Serializable in this context means to process the data as a serial operation, and should not be confused with serializing objects to preserve and restore their states. Running transactions as a single serial operation is the slowest setting. If performance is an issue, use another isolation level that meets your application requirements, but provides better performance.

TRANSACTION_REPEATABLE_READ. At this level, data read by a transaction can be read but not modified, by another transaction. The data is guaranteed to have the same value it had when first read, unless the previous transaction changes it and writes the changed value back.

TRANSACTION_READ_COMMITTED. At this level, data read by a transaction cannot be read by other transactions until the previous transaction either commits or rolls back. However, if another transaction alters the data after this transaction starts, a second read would retrieve the modified data. To prevent this type of inconsistent data, use `TRANSACTION_REPEATABLE_READ`.

TRANSACTION_READ_UNCOMMITTED. At this level, data involved in a transaction can be read by other threads before the previous transaction either completes or rolls back. The other transactions cannot tell if the data was finally committed or rolled back.

Bean-Managed Example

SellerBean is a session Bean that uses RegistrationBean and AuctionItemBean in the following ways:

- RegistrationBean checks the user ID and password when someone posts an auction item and debit the seller's account for a listing.
- AuctionItemBean adds new auction items to the database.

The transaction begins in the SellerBean.insertItem method with the account debit and ends when the entire transaction either commits or rolls back. The entire transaction including the 50-cent debit rolls back if the auction item is null (the insertion failed), or if an exception is caught. If the auction item is not null and the insertion succeeds, the entire transaction including the 50 cents debit commits.

For this example, the isolation level is TRANSACTION_SERIALIZABLE, and the transaction attribute is TX_BEAN_MANAGED. The other Beans in the transaction, RegistrationBean and AuctionItemBean, have an isolation level of TRANSACTION_SERIALIZABLE and a transaction attribute of REQUIRED. Changes to this version of SellerBean.insertItem over the container-managed version are flagged with comments.

```
    public int insertItem(String seller, String password, String description,
                          int auctiondays, double startprice, String summary)
                          throws RemoteException {
    //Declare transaction context variable using the
    //javax.transaction.UserTransaction class
      UserTransaction uts= null;
      try {
        Context ectx = new InitialContext(p);
    //Get the transaction context
        uts=(UserTransaction)ctx.getUserTransaction();
        RegistrationHome rhome = (RegistrationHome)ectx.lookup("registration");
        RegistrationPK rpk=new RegistrationPK();
        rpk.theuser=seller;
        Registration newseller=rhome.findByPrimaryKey(rpk);
        if((newseller == null)|| (!newseller.verifyPassword(password))) {
            return(Auction.INVALID_USER);
        }
    //Start the transaction
//API Ref :void begin()
        uts.begin();
    //Deduct 50 cents from seller's account
        newseller.adjustAccount(-0.50);
        AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
        AuctionItem ai= home.create(seller, description, auctiondays,
                                    startprice, summary);
        if(ai == null) {
    //Roll transaction back
//API Ref :void rollback()
            uts.rollback();
            return Auction.INVALID_ITEM;
        } else {
    //Commit transaction
//API Ref :void commit()
```

```
            uts.commit();
            return(ai.getId());
        }
        } catch(Exception e){
          System.out.println("insert problem="+e);
    //Roll transaction back if insert fails
            uts.rollback();
            return Auction.INVALID_ITEM;
        }
    }
```

Bean-Managed finder Methods

The container-managed search described in 2: Auction House Application (page 13) is based on a finder method mechanism where the deployment descriptor, rather than the Bean, specifies the finder method behavior. While the finder mechanism works well for simple queries and searches, it cannot handle complex operations that span more than one Bean type or database table. Also, the Enterprise JavaBeans 1.1 specification currently provides no specification for putting finder rules in the deployment descriptor.

So, for more complex queries and searches, you have to write Bean-managed queries and searches. This section explains how to write a Bean-managed version of the auction house search facility from 2: Auction House Application (page 13). The Bean-managed search involves changes to the AuctionServlet.searchItems method and a new session Bean, SearchBean.

AuctionServlet.searchItems

The search begins when the end user submits a search string to the search facility on the auction house home page and clicks the Submit button. This invokes AuctionServlet, which retrieves the search string from the HTTP header and passes it to the searchItem method.

Note: The search logic for this example is fairly simple. The idea here is to show you how to move the search logic into a separate Enterprise Bean so you can create a more complex search on your own.

Figure 15 shows how the searchItem operation is in the following two parts: 1) Using the search string to retrieve primary keys, and 2) Using primary keys to retrieve auction items. Parts 1 and 2 are described in more detail below the figure.

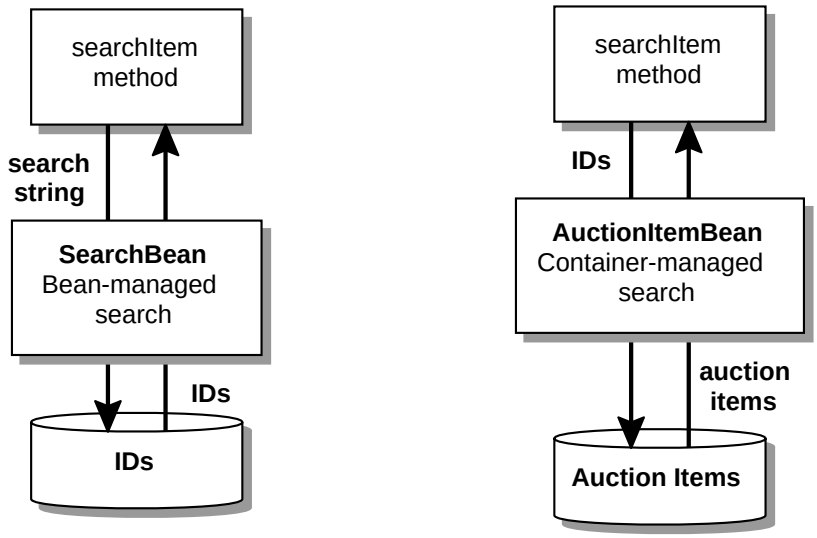


Figure 15 Search Items Operation

Part 1: The first thing the searchItems Method (page 77) does is pass the search string submitted by the end user to the SearchBean session Bean. SearchBean (page 70) implements a Bean-managed search that retrieves a list of primary keys for all auction items whose Summary fields contain characters matching the search string. This list is returned to the searchItems method in an Enumeration variable.

```
Enumeration enum = (Enumeration)search.getMatchingItemsList(searchString);
```

Part 2: The searchItems method then uses the returned Enumeration list from Part 1 and AuctionItemBean (page 34) to retrieve each Bean in turn by calling findByPrimaryKey on each primary key in the list. This is a container-managed search based on the finder mechanism described in 2: Auction House Application (page 13).

```
//Iterate through search results
while ((enum != null) && enum.hasMoreElements())) {
  while(enum.hasMoreElements(in)) {
    //Locate auction items
    AuctionItem ai=ahome.findByPrimaryKey((
    AuctionItemPK)enum.nextElement());
    displayLineItem(ai, out);
  }
}
```

SearchBean

The SearchBean (page 70) class defines a Bean-managed search for the primary keys of auction items with summary fields that contain characters matching the search string. This Bean establishes a database connection, and provides the getMatchingItemsList and EJBCreate methods. A custom search Bean is written because the default finder rules supplied by the EJB server do not allow a wild card SQL search string. This technique can also be used for more complex queries or queries that could be better optimized by the developer than those search queries generated by the EJB server.

Database Connection

Because this Bean manages its own database access and search, it has to establish its own database connection. It cannot rely on the container to do this. The database connection is established by instantiating a static Driver class and providing the getConnection method. The getConnection method queries the static DriverManager class for a registered database driver that matches the Uniform Resource Locator (URL). In this case, the URL is weblogic.jdbc.jts.Driver.

```
    //Establish database connection
      static {
        new weblogic.jdbc.jts.Driver();
      }
      public Connection getConnection() throws SQLException {
        return DriverManager.getConnection("jdbc:weblogic:jts:ejbPool");
      }
```

Get Matching Items List

The getMatchingItemsList method looks up AuctionItemBean and creates a PreparedStatement object for querying the database for summary fields that contain the search string. Data is read from the database into a ResultSet, stored in a Vector, and returned to AuctionServlet.

```
    public Enumeration getMatchingItemsList(String searchString)
                       throws RemoteException {
      ResultSet rs = null;
      PreparedStatement ps = null;
      Vector v = new Vector();
      Connection con = null;
      try {
    //Get database connection
        con=getConnection();
//API Ref :PreparedStatement prepareStatement(String sql)
```

```
    ps=con.prepareStatement("select id from auctionitems where summary like ?");
    ps.setString(1, "%"+searchString+"%");
//Execute database query
    ps.executeQuery();
//Get results set
    rs = ps.getResultSet();
//Get information from results set
    AuctionItemPK pk;
    while (rs.next()) {
      pk = new AuctionItemPK();
      pk.id = (int)rs.getInt(1);
//Store retrieved data in vector
      v.addElement(pk);
    }
    rs.close();
    return v.elements();
  } catch (Exception e) {
    System.out.println("getMatchingItemsList: "+e);
    return null;
   } finally {
     try {
     if (rs != null) {
        rs.close();
     }
     if (ps != null) {
        ps.close();
     }
     if (con != null) {
        con.close();
      }
      } catch (Exception ignore) {}
    }
  }
```

Create Method

The ejbCreate method creates an javax.naming.InitialContext object. This is a Java Naming and Directory™ (JNDI) class that lets SearchBean access the database without relying on the container.

```
public void ejbCreate() throws CreateException,
  RemoteException {
  Properties p = new Properties();
  p.put(Context.INITIAL_CONTEXT_FACTORY,
              "weblogic.jndi.TengahInitialContextFactory");
  try {
    ctx = new InitialContext(p);
  } catch(Exception e) {
    System.out.println("create exception: "+e);
  }
}
```

Code for this Chapter

- RegistrationBean (SQL) (page 72)
- SellerBean (Session Synchronization) (page 75)
- searchItems Method (page 77)
- SearchBean (page 77)

RegistrationBean (SQL)

```
package registration;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.sql.*;

// Bean-managed persistence using JDBC
public class RegistrationBean implements EntityBean {
  private transient EntityContext ctx;
  private PreparedStatement ps=null;
  private Connection con=null;
  public String theuser, password, creditcard, emailaddress;
  public double balance;
  static {
    new weblogic.jdbc.jts.Driver();
  }
  public Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:weblogic:jts:ejbPool");
  }
  public boolean verifyPassword(String password) throws RemoteException {
    if(this.password.equals(password)) {
      return true;
    } else {
      return false;
    }
  }
  public String getEmailAddress() throws RemoteException {
    return emailaddress;
  }
  public String getUser() throws RemoteException {
   return theuser;
  }
  public int adjustAccount(double amount) throws RemoteException {
    balance=balance+amount;
    return(0);
  }
  public double getBalance() throws RemoteException {
    return balance;
```

```
}
public RegistrationPK ejbCreate(String theuser, String password,
                     String emailaddress, String creditcard)
                     throws CreateException, RemoteException {
  this.theuser=theuser;
  this.password=password;
  this.emailaddress=emailaddress;
  this.creditcard=creditcard;
  this.balance=0;
  try {
    con=getConnection();
    // Create new registration entry into the database using JDBC
    ps=con.prepareStatement("insert into registration (theuser, password,
                          emailaddress, creditcard, balance)
    values (?, ?, ?, ?, ?)");
    ps.setString(1, theuser);
    ps.setString(2, password);
    ps.setString(3, emailaddress);
    ps.setString(4, creditcard);
    ps.setDouble(5, balance);
    if (ps.executeUpdate() != 1) {
        throw new CreateException ("JDBC did not create any row");
    }
    RegistrationPK primaryKey = new RegistrationPK();
    primaryKey.theuser = theuser;
    return primaryKey;
  } catch (CreateException ce) {
    throw ce;
  } catch (SQLException sqe) {
    throw new CreateException (sqe.getMessage());
  } finally {
    try {
      ps.close();
    } catch (Exception ignore) {}
    try {
      con.close();
    } catch (Exception ignore) {}
  }
}
public void ejbPostCreate(String theuser, String password, String emailaddress,
           String creditcard) throws CreateException, RemoteException {
}
public void setEntityContext(javax.ejb.EntityContext ctx)
            throws RemoteException {
  this.ctx = ctx;
}
public void unsetEntityContext() throws RemoteException {
  ctx = null;
}
public void ejbRemove() throws RemoteException, RemoveException { }
public void ejbActivate() throws RemoteException { }
public void ejbPassivate() throws RemoteException { }
public void ejbLoad() throws RemoteException {
```

```
  try {
    refresh((RegistrationPK) ctx.getPrimaryKey());
  } catch (FinderException fe) {
    throw new RemoteException (fe.getMessage());
  }
}
public void ejbStore() throws RemoteException {
  Connection con = null;
  PreparedStatement ps = null;
  try {
    con = getConnection();
    // Write the Bean contents to the database
    ps = con.prepareStatement("update registration set password = ?,
         emailaddress = ?, creditcard = ?, balance = ? where theuser = ?");
    ps.setString(1, password);
    ps.setString(2, emailaddress);
    ps.setString(3, creditcard);
    ps.setDouble(4, balance);
    ps.setString(5, theuser);
    int i = ps.executeUpdate();
    if(i == 0) {
      throw new RemoteException("ejbStore: Registration (" +
                                theuser + ") not updated");
    }
  } catch (RemoteException re) {
    throw re;
  } catch (SQLException sqe) {
    throw new RemoteException (sqe.getMessage());
  } finally {
    try {
      ps.close();
    } catch (Exception ignore) {}
    try {
      con.close();
    } catch (Exception ignore) {}
  }
}
public RegistrationPK ejbFindByPrimaryKey(RegistrationPK pk)
                      throws FinderException, RemoteException {
  // Find the Bean matching this primary key and read from database
  if((pk == null) || (pk.theuser == null)) {
    throw new FinderException ("primary key cannot be null");
  }
  refresh(pk);
  return pk;
}
private void refresh(RegistrationPK pk)
                    throws FinderException, RemoteException {
 // Read the Bean contents from the database
  if(pk == null) {
   throw new RemoteException ("primary key cannot be null");
  }
  Connection con = null;
```

```
    PreparedStatement ps = null;
    try {
      con=getConnection();
      ps=con.prepareStatement("select password, emailaddress, creditcard,
                              balance from registration where theuser = ?");
      ps.setString(1, pk.theuser);
      ps.executeQuery();
      ResultSet rs = ps.getResultSet();
      if(rs.next()) {
        theuser = pk.theuser;
        password = rs.getString(1);
        emailaddress = rs.getString(2);
        creditcard = rs.getString(3);
        balance = rs.getDouble(4);
      } else {
        throw new FinderException ("Refresh: Registration ("
                                   + pk.theuser + ") not found");
      }
    } catch (SQLException sqe) {
      throw new RemoteException (sqe.getMessage());
    } finally {
      try {
        ps.close();
      } catch (Exception ignore) {}
      try {
        con.close();
      } catch (Exception ignore) {}
    }
  }
}
```

SellerBean (Session Synchronization)

```
package seller;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import javax.jts.*;
import auction.*;
import registration.*;

public class SellerBean implements SessionBean, SessionSynchronization {
  private transient SessionContext ctx;
  private transient Properties p = new Properties();
  private transient boolean success = true;
  public void afterBegin() {}
  public void beforeCompletion() {
    // before the transaction is ready to commit
```

```
    if(!success ) {
      ctx.setRollbackOnly();
    }
  }
  public void afterCompletion(boolean state) {}

  public int insertItem(String seller, String password,
            String description, int auctiondays,
           double startprice, String summary)
           throws RemoteException {
    // Insert auction item into the auction
    try {
      Context jndiCtx = new InitialContext(p);
      RegistrationHome rhome = (RegistrationHome)
      jndiCtx.lookup("registration");
      RegistrationPK rpk=new RegistrationPK();
      rpk.theuser=seller;
      Registration newseller=rhome.findByPrimaryKey(rpk);
      if((newseller == null) ||
                (!newseller.verifyPassword(password))) {
         return(Auction.INVALID_USER);
      }
      newseller.adjustAccount(-0.50);
      AuctionItemHome home = (AuctionItemHome)
      jndiCtx.lookup("auctionitems");
      AuctionItem ai= home.create(seller, description,
      auctiondays, startprice, summary);
      if(ai == null) {
        success=false;
        return Auction.INVALID_ITEM;
      } else {
          return(ai.getId());
      }
    } catch(Exception e){
      System.out.println("insert problem="+e);
      success=false;
      return Auction.INVALID_ITEM;
    }
  }
  public void ejbCreate() throws CreateException, RemoteException {
    p.put(Context.INITIAL_CONTEXT_FACTORY,
                    "weblogic.jndi.TengahInitialContextFactory");
  }
  public void setSessionContext(SessionContext ctx) throws RemoteException {
    this.ctx = ctx;
  }
  public void unsetSessionContext() throws RemoteException {
    ctx = null;
  }
  public void ejbRemove() {}
    public void ejbActivate() throws RemoteException { }
  public void ejbPassivate() throws RemoteException { }
}
```

searchItems Method

```
private void searchItems(ServletOutputStream out, HttpServletRequest request)
                         throws IOException {
  String searchString=request.getParameter("searchString");
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Search Results");
  try {
    addLine("<BR>"+text, out);
//Look up Home interfaces
    AuctionItemHome ahome = (
    AuctionItemHome) ctx.lookup("auctionitems");
    SearchHome shome=(SearchHome) ctx.lookup("search");
//Create remote interface for search Bean
    Search search=shome.create();
//Call search method and pass the search string
    Enumeration enum=(Enumeration) search.getMatchingItemsList(searchString);
    addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0>
      <TR> <TH>Item<TH> <TH>Summary<TH><TH>Current High bid<TH>
      <TH>Number of bids<TH><TH>Closing Date<TH><TR>", out);
//Iterate through search results
    while ((enum != null) && (enum.hasMoreElements())) {
      while(enum.hasMoreElements(in)) {
//Locate auction items
       AuctionItem ai=ahome.findByPrimaryKey((AuctionItemPK)enum.nextElement());
      displayLineItem(ai, out);
    }
  }
    addLine("<TABLE>", out);
  } catch (Exception e) {
    addLine("AuctionServlet Search Items error", out);
    System.out.println("AuctionServlet <searchItems>:"+e);
  }
  out.flush();
}
```

SearchBean

```
package search;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;
import java.sql.*;

public class SearchBean implements SessionBean {
    protected SessionContext sctx;
```

```
  Properties p = new Properties();
  Context ctx;
  static {
    new weblogic.jdbc.jts.Driver();
  }

public Connection getConnection() throws SQLException {
  return DriverManager.getConnection("jdbc:weblogic:jts:ejbPool");
}
public Enumeration getMatchingItemsList(String searchString)
                                      throws RemoteException {
  // Search for matching item by comparing searchString to
  // the database summary field and returning an enumerated
  // list of the primary key auction id values.

  ResultSet rs = null;
  PreparedStatement ps = null;
  Vector v = new Vector();
  Connection con = null;
  try {
    AuctionItemHome home = (AuctionItemHome)
    ctx.lookup("auctionitems");
    con = getConnection();
    ps = con.prepareStatement(
         "select id from auctionitems where summary like ?");
    ps.setString(1, "%"+searchString+"%");
    ps.executeQuery();
    rs = ps.getResultSet();
    AuctionItemPK pk;
    while(rs.next()) {
      pk = new AuctionItemPK();
      pk.id = (int)rs.getInt(1);
      v.addElement(pk);
    }
    return v.elements();
  } catch (Exception e) {
    System.out.println("getMatchingItemsList: "+e);
    return null;
  } finally {
    try {
      if(rs != null) {
        rs.close();
      }
      if(ps != null) {
        ps.close();
      }
      if(con != null) {
        con.close();
      }
    } catch (Exception ignore) {
        // An exception was thrown trying to
       // close the used connections. The exception is ignored
       // because there is no further need for the used connection.
```

```
        }
      }
    }
    public void ejbCreate() throws CreateException, RemoteException {
      Properties p = new Properties();
      p.put(Context.INITIAL_CONTEXT_FACTORY,
                    "weblogic.jndi.TengahInitialContextFactory");
      try {
        ctx = new InitialContext(p);
      } catch(Exception e) {
        System.out.println("create exception: "+e);
      }
    }
    public void setSessionContext(SessionContext sctx) throws RemoteException {
      this.sctx = sctx;
    }
    public void unsetSessionContext() throws RemoteException {
      sctx = null;
    }
//API Ref :void ejbRemove()
    public void ejbRemove() {}
//API Ref :void ejbActivate()
    public void ejbActivate() throws RemoteException { }
//API Ref :void ejbPassivate()
    public void ejbPassivate() throws RemoteException { }
  }
```

4: Distributed Computing

As recently as ten years ago, distributed computing generally meant you had client PCs in one room with a server in another room. The problem here is if the server machine is down, the clients cannot update the payroll, sales, or whatever other distributed company databases. To prevent this kind of down time, required different distributed models. One example is the master and slave model where if the master fails, the slaves take over.

The problem with the different distributed network models is they all required some form of manual intervention and are often tied to one operating system or language. And while these approaches met some of the short-term requirements for decreasing down time, they do not apply to heterogeneous distributed systems with mixed network protocols and machines.

The Java™ platform and other advances such as Common Object Request Broker Architecture (CORBA), multi-tiered servers, and wireless networks have brought the realization of fully distributed computing another step ahead of the traditional client and server approach.

Today, you can build applications that include service redundancy by default. If one server connection fails, you can seamlessly use a service on another server. CORBA and Distributed Component Object Model (DCOM) bridges mean that objects can be transferred between virtually all machines and languages. And with the new Jini™ System software, the distributed computing environment can soon be part of everything in your home, office or school. In short, distributed computing has never been as important as it is today.

The first part of this chapter shows you how to write lookup code using JNDI, CORBA, IOR, and RMI-IIOP. The second part describes how to use lookup code with RMI and CORBA, and concludes with a discussion of JDBC™ and servlet technologies.

Covered in this Chapter

- Lookup Services (page 82)
- Remote Method Invocation (page 91)
- Common Object Request Broker Architecture (CORBA) (page 107)
- JDBC Technology (page 129)
- Servlets (page 145)

- Code for this Chapter (page 160)

Lookup Services

Lookup (naming) services enable communications over a network. A client program can use a lookup protocol to get information on remote programs or machines and use that information to establish a communication.

- One common lookup service you might already be familiar with is Directory Name Service (DNS). It maps Internet Protocol (IP) addresses to machine names. Programs use the DNS mapping to look up the IP address associated with a machine name and use the IP address to establish a communication.
- In the same way, the AuctionServlet (page 44) presented in 2: Auction House Application (page 13) uses the naming service built into the Enterprise JavaBeans™ architecture to look up and reference Enterprise Beans registered with the Enterprise JavaBeans server.

In addition to naming services, some lookup protocols provide directory services. Directory services such as Lightweight Directory Access Protocol (LDAP) and Sun's NIS+ provide other information and services beyond what is available with simple naming services. For example, NIS+ associates a workgroup attribute with a user account. This attribute can be used to restrict access to a machine so only the users in the specified workgroup have access.

This chapter describes how the Java Naming and Directory Interface™ (JNDI) is used in the auction application to look up Enterprise Beans. It also explains how to use some of the many other lookup services that have become available over time. The code to use these other services is not as simple as the lookup code in the auction application in 2: Auction House Application (page 13), but the advantages to these other services can outweigh the need for more complex code in some situations.

Java Naming and Directory Interface (JNDI)

The JNDI API makes it easy to plug lookup services from various providers into a program written in the Java programming language. As long as the client and server both use the same lookup service, the client can easily look up information registered with the server and establish communication.

The auction application session Beans use JNDI and a special JNDI naming factory from BEA Weblogic to look up entity Beans. JNDI services normally initialize the naming factory as a property on the command line or as an initialization value. First, the naming factory weblogic.jndi.TengahInitialContextFactory is put into a java.util.Property object, then the

Property object is passed as a parameter to the InitialContext constructor. Here is an example ejbCreate method.

```
Context ctx; //JNDI context
public void ejbCreate() throws CreateException, RemoteException {
  Hashtable env = new Hashtable();
  env.put(Context.INITIAL_CONTEXT_FACTORY,
                        "weblogic.jndi.TengahInitialContextFactory");
  try {
    ctx = new InitialContext(env);
  } catch(Exception e) {
    System.out.println("create exception: "+e);
  }
}
```

Once created, the JNDI context is used to look up Enterprise Bean home interfaces. In this example, a reference to the Enterprise Bean bound to the name registration is retrieved and used for further operations.

```
RegistrationHome rhome = (RegistrationHome) ctx.lookup("registration");
RegistrationPK rpk=new RegistrationPK();
rpk.theuser=buyer;
Registration newbidder = rhome.findByPrimaryKey(rpk);
```

On the server side, the deployment descriptor for the RegistrationBean has its beanhome-name value set to registration. Enterprise JavaBeans tools generate the rest of the naming code for the server.

The server calls ctx.bind to bind the name registration to the JNDI context. The `this` parameter references the _stub class that represents the RegistrationBean.

```
ctx.bind("registration", this);
```

JNDI is not the only way to look up remote objects. Lookup services are also available in the RMI, JINI, and CORBA platforms. You can use these platform-specific lookup services directly or from the JNDI API.

JNDI allows the application to change the name service with little effort. For example, here are the code changes to have the BidderBean.ejbCreate method use the org.omb.CORBA lookup services instead of the default BEA Weblogic lookup services.

```
Hashtable env = new Hashtable();
env.put("java.naming.factory.initial", “com.sun.jndi.cosnaming.CNCtxFactory");
Context ic = new InitialContext(env);
```

CORBA Naming Service

The Common Object Request Broker Architecture (CORBA) defines a specification for objects in a distributed system to communicate with each other. Objects that use the CORBA specification to communicate are called CORBA objects, and consist of client and server objects.

CORBA objects can be written in any language with Interface Definition Language (IDL) mapping. These languages include the Java programming language, C++, and many traditional non-object-orientated languages.

The naming lookup service, like all other CORBA specifications, is defined in terms of IDL. The IDL module for the CORBA lookup service is called CosNaming. Any platform with an IDL mapping can use this service to look up and discover CORBA objects. The IDL module for the CORBA lookup service is available in the Java 2 platform in the org.omg.CosNaming package and the mapping to the Java programming language is created by a tool called idltojava.

The key interface in the CosNaming module is NamingContext. The NamingContext interface defines methods to bind objects to a name, list those bidding, and retrieve bound object references.

In addition to these public interfaces are helper classes. The NameComponent helper class is used in CORBA client and server programs to build the full name for the object reference name. The full name is an array of one or more NameComponents that indicates where to find the objects. The naming scheme can be application specific.

Figure 16 shows how the full name for the auction application can be defined to use auction as the root naming context with RegistrationBean and AuctionItemBean as children of the root context. This in effect employs a similar naming scheme as that used for the application class packaging.

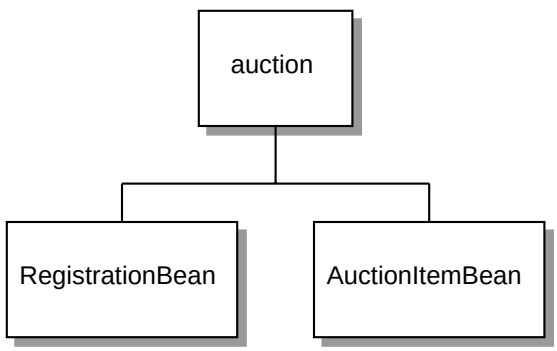


Figure 16 Full Name and Root Naming Context

In this example, the auction application has adapted SellerBean to a CORBA naming service to look up the CORBA RegistrationBean. The following code is extracted from the SellerBean class, which acts as the CORBA client, and the RegistrationServer CORBA server.

CORBA RegistrationServer

The RegistrationServer (lookup) (page 161) code creates a NameComponent object that uses the auction and RegistrationBean strings as a full name indicating where to locate the RegistrationBean.

```
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent( "RegistrationBean", "");
```

This next code binds the fullname as a new name context. The first elements in the full name (auction in this example) are placeholders for building the context naming tree. The last element of the full name (RegistrationBean in this example) is the name submitted as the binding to the object.

```
    public static void main(String args[]) {
    String[] orbargs = { "-ORBInitialPort 1050"};
//API Ref :status ORB init(String[] args, Properties props)
    ORB orb = ORB.init(orbargs, null)
    RegistrationServer rs= new RegistrationServer();
    try{
//API Ref :void connect(Object object)
      orb.connect(rs);
      org.omg.CORBA.Object nameServiceObj =
                        orb.resolve_initial_references("NameService");
      NamingContext nctx = NamingContextHelper.narrow(nameServiceObj);
//API Ref :NameComponent(String nameid, String kind)
      NameComponent[] fullname = new NameComponent[2];
      fullname[0] = new NameComponent("auction", "");
      fullname[1] = new NameComponent("RegistrationBean", "");
      NameComponent[] tempComponent = new NameComponent[1];
      for (int i=0; i < fullname.length-1; i++ ) {
        tempComponent[0]= fullname[i];
        try {
//API Ref :NamingContext bind_new_context(NameComponent[] nc)
          nctx=nctx.bind_new_context(tempComponent);
        } catch (org.omg.CosNaming.NamingContextPackage.AlreadyBound e){
          //If this part of the tree is already bound then ignore this exception
        }
      }
        tempComponent[0]=fullname[fullname.length-1];
      // finally bind the object to the full context path
//API Ref :static void rebind(String rminame, Remote obj)
        nctx.rebind(tempComponent, rs);
```

Once the RegistrationServer object is bound, it can be looked up with a JNDI lookup using a CosNaming service provider as described in Java Naming and Directory Interface (JNDI) (page 82) or using the CORBA name lookup service. Either way, the CORBA name server must be started before any lookups can take place. In the Java 2 platform, the CORBA name server is started as follows to start the CORBA RegistrationServer on the default TCP port 900:

```
tnameserv
```

If you need to use a different port, you can start the server like this:

```
tnameserv -ORBInitialPort 1050
```

CORBA SellerBean

On the client side, the CORBA lookup uses the NameComponent object to construct the name. Start the object server like this:

```
java registration.RegistrationServer
```

The difference in the client is the name is passed to the resolve method which returns the CORBA object. The following code from the SellerBean object illustrates this point.

```
    public static void main(String args[]) {
      java.util.Properties props=System.getProperties();
      props.put("org.omg.CORBA.ORBInitialPort", "1050");
      System.setProperties(props);
      ORB orb = ORB.init(args, props);

//API Ref :  Object resolve_initial_references(String servicename)
      org.omg.CORBA.Object nameServiceObj = orb.resolve_initial_references(
                                                    "NameService") ;
      nctx= NamingContextHelper.narrow(nameServiceObj);
      NameComponent[] fullname = new NameComponent[2];
      fullname[0] = new NameComponent("auction", "");
      fullname[1] = new NameComponent("RegistrationBean", "");
//API Ref :  Object resolve(NameComponent[] nc)
     org.omg.CORBA.Object cobject= nctx.resolve(fullname);

    }
```

The narrow method, from the Helper class, is generated by the IDL compiler, which provides a detailed mapping to translate each CORBA field into its respective Java programming language field. For example, the SellerBean.insertItem method looks up a registration CORBA object using the name RegistrationBean, and returns a RegistrationHome object. With the RegistrationHome object, you can return a `Registration` record by calling its findByPrimaryKey method.

```
public int insertItem(String seller, String password, String description,
                      int auctiondays, double startprice, String summary)
                      throws RemoteException {
try{
  NameComponent[] fullname = new NameComponent[2];
  fullname[0] = new NameComponent("auction", "");
  fullname[1] = new NameComponent("RegistrationBean", "");

  org.omg.CORBA.Object cobject= nctx.resolve(fullname);
  RegistrationHome regHome = RegistrationHomeHelper.narrow(cobject);
  RegistrationHome regRef =
           RegistrationHomeHelper.narrow(nctx.resolve(fullname));
  RegistrationPKImpl rpk = new RegistrationPKImpl();
  rpk.theuser(seller);
  Registration newseller =
           RegistrationHelper.narrow(regRef.findByPrimaryKey(rpk));
  if ((newseller == null) || (!newseller.verifyPassword(password))) {
      return(Auction.INVALID_USER);
  }
  } catch(Exception e) {
     System.out.println("insert problem="+e);
     return Auction.INVALID_ITEM;
  }
}
```

Interoperable Object References (IOR)

Using a CORBA name service works for most CORBA applications especially when the object request brokers (ORBs) are supplied by one vendor. However, you might find the name service is not completely compatible among all ORBs, and you could get a frustrating COMM_FAILURE message when the CORBA client tries to connect to the CORBA server.

The solution is to use an Interoperable Object Reference (IOR) instead. An IOR is available in ORBs that support the Internet Inter-ORB protocol (IIOP). It contains the information a naming service keeps for each object such as the host and port where the object resides, a unique lookup key for the object on that host, and what version of IIOP is supported.

IOR Server

To create an IOR all you do is call the object_to_string method from the ORB class and pass it an instance of the object. For example, to convert the RegistrationServer object to an IOR, you need to add the line String ref = orb.object_to_string(rs); to the main program:

```
   String[] orbargs=  {"-ORBInitialPort 1050"};
   ORB orb = ORB.init(orbargs, null);
   RegistrationServer rs = new RegistrationServer();
   //Add this line
//API Ref :String object_to_String(Object object)
```

```
    String ref = orb.object_to_string(rs);
```

So, instead of retrieving this object information from a naming service, there is another way for the server to send information to the client. You can register the returned String with a substitute name server, which can be a simple HTTP web server because the object is already in a transmittable format.

IOR Client

This example uses an HTTP connection to convert the IOR string back to an object. You call the string_to_object method from the ORB class. This method requests the IOR from the RegistrationServer and returns the IOR string. The String is passed to the ORB using the ORB.string_to_object method, and the ORB returns the remote object reference:

```
    URL iorserver = new URL("http://server.com/servlet?object=registration");
    URLConnection con = ioserver.openConnection();
    BufferedReader br = new BufferReader(new InputStreamReader(con.getInputStream));
    String ref = br.readLine();
//API Ref :Object string_to_object(String stringvalue)
    org.omg.CORBA.Object cobj = orb.string_to_object(ref);
    RegistrationHome regHome = RegistrationHomeHelper.narrow(cobj);
```

The substitute name server can keep persistent IOR records that can survive a restart if needed.

Remote Method Invocation (RMI)

The Remote Method Invocation (RMI) API originally used its own communication protocol called Java Remote Method Protocol (JRMP), which resulted in having its own lookup service. Newer releases of RMI can now use the more ubiquitous IIOP protocol, in addition to JRMP.

The JRMP RMI naming service is similar to other lookup and naming services. The actual lookup is achieved by calling Naming.lookup and passing a URL parameter to that method. The URL specifies the machine name, an optional port where the RMI naming server that knows about the object, rmiregistry, is running, and the remote object you want to reference and call methods on. For example:

```
//API Ref :static Remote lookup(String rminame)
    SellerHome shome = (SellerHome)Naming.lookup(
                       "rmi://appserver:1090/seller");
```

The above code returns the remote SellerHome reference _stub from the object bound to the name seller on the machine called appserver. The rmi part of the URL is optional and you may have seen RMI URLs without it, but if you are using JNDI or RMI-IIOP, including rmi

in the URL will save confusion later. Once you have a reference to SellerHome, you can call its methods.

In contrast to the JNDI lookup performed by AuctionServlet.java (page 82), which requires a two-stage lookup to create a context and then the actual lookup, RMI initializes the connection to the RMI name server, rmiregistry, and also gets the remote reference with one call. This remote reference is leased to the client from the rmiregistry. The lease means that unless the client informs the server it still needs a reference to the object, the lease expires and the memory is reclaimed. This leasing operation is automatic and transparent to the user, but can be tuned by setting the server property java.rmi.dgc.leaseValue value in milliseconds when starting the server as follows:

```
java -Djava.rmi.dgc.leaseValue=120000 myAppServer
```

You can find more information on leasing in Distributed Garbage Collection (page 99).

RMI Over Internet Inter-ORB Protocol (IIOP)

RMI over Internet Inter-ORB Protocol (IIOP) lets existing RMI code reference and look up an object using the CORBA CosNaming service. This gives you greater interoperability between architectures with little change to your existing RMI code.

Note: The rmic compiler provides the -iiop option to generate the stub and the classes necessary for RMI-IIOP.

IIOP Server

The RMI-IIOP protocol is implemented as a JNDI plug-in, so as before, you need to create an InitialContext:

```
Hashtable env = new Hashtable();
env.put("java.naming.factory.initial", "com.sun.jndi.cosnaming.CNCtxFactory");
env.put("java.naming.provider.url","iiop://localhost:1091");
Context ic = new InitialContext(env);
```

The naming factory should look familiar as it is the same CORBA naming service used in CORBA Naming Service (page 84). The main difference is the addition of a URL value specifying the naming service to which to connect. The naming service used here is the tnameserv program started on port 1091.

```
tnameserv -ORBInitialPort 1091
```

The other main change to the server side is to replace calls to Naming.rebind to use the JNDI rebind method in the InitialContext instance. For example:

Old RMI lookup code:

```
SellerHome shome= new SellerHome("seller");
Naming.rebind("seller", shome);
```

New RMI code:

```
Hashtable env = new Hashtable();
env.put("java.naming.factory.initial","com.sun.jndi.cosnaming.CNCtxFactory");
env.put("java.naming.provider.url", "iiop://localhost:1091");
Context ic = new InitialContext(env);
SellerHome shome= new SellerHome("seller");
ic.rebind("seller", shome);
```

IIOP Client

On the client side, the RMI lookup is changed to use an instance of the InitialContext in place of RMI Naming.lookup. The return object is mapped to the requested object by using the narrow method of the javax.rmi.PortableRemoteObject class. PortableRemoteObject replaces UnicastRemoteObject that was previously available in the RMI server code.

Old RMI code:

```
//API Ref :static Remote lookup(String rminame)
    SellerHome shome =
                (SellerHome)Naming.lookup("rmi://appserver:1090/seller");
```

New RMI code:

```
    Hashtable env = new Hashtable();
    env.put("java.naming.factory.initial", "com.sun.jndi.cosnaming.CNCtxFactory");
    env.put("java.naming.provider.url", iiop://localhost:1091");
    Context ic = new InitialContext(env);
    SellerHome shome =
//API Ref :static object narrow(Object narrowFrom, Class narrowTo
         (SellerHome)PortableRemoteObject.narrow(ic.lookup("seller"), SellerHome)
```

The PortableRemoteObject replaces UnicastRemoteObject previously available in the RMI server code. The RMI code would either extend UnicastRemoteObject or call the exportObject method from the UnicastRemoteObject class. The PortableRemoteObject also contains an equivalent exportObject method. In the current implementation, is best to explicitly remove unused objects by calling PortableRemoteObject.unexportObject.

Improving Lookup Performance

When you run your application, if you find it would be faster to walk the object to the other computer on a floppy, you have a network configuration problem. The source of the problem is how host names and IP addresses are resolved, and there is a workaround.

RMI and other naming services use the InetAddress class to obtain resolved host name and IP addresses. InetAddress caches lookup results to improve subsequent calls, but when it is passed a new IP address or host name, it performs a cross-reference between the IP address and the host name to prevent address spoofing. If you supply the host name as an IP address, InetAddress still tries to verify the name of the host. To workaround this problem, include the host name and IP address in a hosts file on the client.

Unix Systems

On Unix, the hosts file is usually /etc/hosts.

Windows

On Windows 95 or 98, the hosts file is c:\windows\hosts, (hosts.sam is a sample file). On Windows NT, the hosts file is c:\winnt\system32\drivers\etc\hosts All you do is put these lines in the hosts file. The myserver1 and myserver2 entries are the hosts running the remote server and rmiregistry.

```
127.0.0.1  localhost
129.1.1.1  myserver1
129.1.1.2  myserver2
```

Remote Method Invocation

The Remote Method Invocation (RMI) API enables client and server communications over the net between programs written in the Java programming language. The Enterprise JavaBeans server transparently implements the necessary Remote Method Invocation (RMI) code so the client program can reference the Enterprise Beans running on the server and access them as if they are running locally to the client program.

Having RMI built into the Enterprise JavaBeans server is very convenient and saves you coding time, but if you need to use advanced RMI features or integrate RMI with an existing application, you need to override the default RMI implementation and write your own RMI code.

This chapter replaces the container-managed RegistrationBean from How Enterprise Beans are used in the Example (page 20) with an RMI-based registration server. The container-

managed SellerBean (page 43) from Chapter 2 is also changed to call the new RMI registration server using a Java 2 RMI lookup call.

About RMI

The RMI API lets you access a remote server object from a client program by making simple method calls on the server object. While other distributed architectures for accessing remote server objects such as Distributed Component Object Model (DCOM) and Common Object Request Broker Architecture (CORBA) return references to the remote object, the RMI API not only returns references, but provides these additional benefits.

- The RMI API handles remote object references (call by reference) and can also return a copy of the object (call by value).
- If the client program does not have local access to the class from which a local or remote object was instantiated, RMI services can download the class file.

Serialization and Data marshaling

To transfer objects, the RMI API uses the Serialization API to wrap (marshal) and unwrap (unmarshal) the objects. To marshal an object, the Serialization API converts the object to a stream of bytes, and to unmarshal an object, the Serialization API converts a stream of bytes into an object. You can find more informationon data marshaling in Data Marshaling (page 97).

RMI over IIOP

One of the initial disadvantages to RMI was that its sole reliance on the Java platform to write the interfaces made integration into existing legacy systems difficult. However, RMI over Internet Inter-ORB Protocol (IIOP) discussed in Lookup Services (page 82) lets RMI communicate with any system or language that CORBA supports.

If you combine improved integration with the ability of RMI to burrow through firewalls using HTTP firewall proxying, you might find distributing your business logic using RMI is easier than a socket-based solution.

Note: Transferring code and data are key parts of the Jini™ System software specification. In fact, adding a discovery and join service to the RMI services would create something very similar to what you get in the Jini architecture.

RMI in the Auction Application

The RMI-based RegistrationServer (RMI) (page 165) has the following new methods:

- A new create method for creating a new user.
- A new find method for finding a user.
- A new search method for the custom search of users in the database.

The new custom search passes results back to the calling client by way of an RMI callback. The RMI callback custom search is similar to the finder methods used in the Bean- and container-managed examples from 2: Auction House Application (page 13) and 3: Data and Transaction Management (page 55), except in the RMI version, it can take more time to generate the results because the remote registration server calls a remote method exported by the RMI-based SellerBean (RMI) (page 170) client.

If the calling client is written in the Java programming language, and is not, for example, a web page, the server can update the client as soon as the results are ready. But, the HTTP protocol used in most browsers does not allow results to be pushed to the client without a request for those results. This means the resulting web page is not created until the results are ready, which can add a small delay.

Class Overview

The two main classes in the RMI-based auction implementation are SellerBean (RMI) (page 170) and RegistrationServer (RMI) (page 165). SellerBean is called from AuctionServlet (RMI) (page 173) to insert an auction item into the database, and check for low account balances.

The example models the Enterprise JavaBeans architecture in that a user's registration details are separate from the code to create and find the registration details. That is, the user's registration details provided by the Registration (RMI) (page 169) class are separate from the code to create and find a Registration object, which is in the RegistrationHome (RMI) (page 169) class.

The remote interface implementation in RegistrationHome.java is bound to the rmiregistry. When a client program wants to manipulate a user's registration details, it must first look up the reference to the RegistrationHome.java object in the rmiregistry.

File Summary

All the source code files for the RMI-based example are described in the bullet list below.

- SellerBean (RMI) (page 170): Client program that calls the RegistrationServer.verifypasswd and RegistrationServer.findLowCreditAccounts remote methods. SellerBean also exports its updateResults method that RegistrationServer calls when it completes its RegistrationServer.findLowCreditAccounts search.
- RegistrationServer (RMI) (page 165): Remote server object that implements the RegistrationHome and Registration remote interfaces.

- Registration (RMI) (page 169): Remote interface that declares the getUser, verifyPasswd, and other remote methods for managing a user's registration details.
- RegistrationHome (RMI) (page 169): Remote interface that declares the create, findByPrimaryKey, and findLowCreditAccounts remote methods that create or return instances of registration details.
- RegistrationImpl.java: The RegistrationServer (RMI) (page 165) source file includes the implementation for the Registration remote interface as class RegistrationImpl
- RegistrationPK (RMI) (page 170): Class that represents a user's registration details using just the primary key of the database record.
- ReturnResults (RMI) (page 172): Remote interface that declares the updateResults method the SellerBean class implements as a callback.
- AuctionServlet (RMI) (page 173): Modified version of the original AuctionServlet class where registration accounts are created by calling the RMI RegistrationServer directly. The auction servlet also calls the SellerBean.auditAccounts method, which returns a list of users with a low account balance.

The auditAccounts method is called with the following Uniform Resource Locator (URL), which does a simple check to verify the request came from the local host.

```
http://phoenix.eng.sun.com:7001/AuctionServlet?action=auditAccounts
```

You also need the following java.policy security policy file to grant the permissions needed to run the example on the Java 2 platform.

```
grant {
  permission java.net.SocketPermission "*:1024-65535", "connect,accept,resolve";
  permission java.net.SocketPermission "*:80", "connect";
  permission java.lang.RuntimePermission "modifyThreadGroup";
  permission java.lang.RuntimePermission "modifyThread";
};
```

Most RMI applications need the two socket permissions for socket and HTTP access to the specified ports. The two thread permissions, were listed in a stack trace as being needed for the RegistrationImpl class to create a new inner thread.

In the Java 2 platform, when a program does not have all the permissions it needs, the Java virtual machine generates a stack trace that lists the permissions that need to be added to the security policy file. See 10: Signed Applets & Security Managers (page 417) for more information on this and other security topics.

Compile the Example

Before describing the RMI-based code for the above classes, here is the command sequence to compile the example on the Unix and Win32 platforms:

Unix:

```
javac registration/Registration.java
javac registration/RegistrationPK.java
javac registration/RegistrationServer.java
javac registration/ReturnResults.java
javac seller/SellerBean.java
rmic -d . registration.RegistrationServer
rmic -d . registration.RegistrationImpl
rmic -d . seller.SellerBean
```

Win32:

```
javac registration\Registration.java
javac registration\RegistrationPK.java
javac registration\RegistrationServer.java
javac registration\ReturnResults.java
javac seller\SellerBean.java
rmic -d . registration.RegistrationServer
rmic -d . registration.RegistrationImpl
rmic -d . seller.SellerBean
```

Start the RMI Registry

Because you are using your own RMI code, you have to explicitly start the RMI Registry so the SellerBean object can find the remote Enterprise Beans. The RegistrationServer uses the RMI Registry to register or bind Enterprise Beans that can be called remotely. The SellerBean client contacts the registry to look up and get references to the remote AuctionItem and Registration Enterprise Beans.

Because RMI allows code and data to be transferred, you must be sure the system classloader does not load extra classes that could be mistakenly sent to the client. In this example, extra classes would be the Stub and Skel class files and the RegistrationSever and RegistrationImpl classes. To prevent them being mistakenly sent, they should not appear anywhere in the CLASSPATH when you start the RMI Registry, and because the current path could be included automatically, you need to start the RMI Registry away from the code workspace too.

The following commands prevent the sending of extra classes by unsetting the CLASSPATH before starting the RMI Registry on the default 1099 port. You can specify a different port by adding the port number as follows: rmiregistry 4321 &. If you specify a different port number, you must specify the same port number in both your client lookup and server rebind calls.

Unix:

```
export CLASSPATH=""
rmiregistry &
```

Win32:

```
unset CLASSPATH
start rmiregistry
```

Start the Remote Server

Once the rmiregistry is running, you can start the remote server, RegistrationServer. The RegistrationServer program registers the name registration2 with the rmiregistry name server, and any client can use this name to retrieve a reference to the remote server object, RegistrationHome.

To run the example, copy the RegistrationServer and RegistrationImpl classes and the associated stub classes to a remotely accessible area and start the server program.

Unix:

```
cp *_Stub.class /home/zelda/public_html/registration
cp RegistrationImpl.class /home/zelda/public_html/registration
cd /home/zelda/public_html/registration
java -Djava.server.hostname=phoenix.sun.com registration.RegistrationServer
```

Windows:

```
copy *_Stub.class  \home\zelda\public_html\registration
copy RegistrationImpl.class  \home\zelda\public_html\registration
cd \home\zelda\public_html\registration
java -Djava.server.hostname=phoenix.sun.com registration.RegistrationServer
```

The following key properties are used to configure RMI servers and clients. These properties can be set inside the program or supplied as command line properties to the Java virtual machine.

- The java.rmi.server.codebase property specifies where the publicly accessible classes are located. On the server this can be a simple file URL to point to the directory or JAR file that contains the classes. If the URL points to a directory, the URL must terminate with a file separator character, "/". If you are not using a file URL, you will either need an HTTP server to download the remote classes or have to manually deliver the remote client stub and remote interface classes in, for example, a JAR file.
- The java.rmi.server.hostname property is the complete host name of the server where the publicly accessible classes reside. This is only needed if the server has problems generating a fully qualified name by itself.
- The java.rmi.security.policy property specifies the policy file with the permissions needed to run the remote server object and access the remote server classes for download.

Establishing Remote Communications

Client programs communicate with each other through the server. The server program consists of three files. The Registration.java and RegistrationHome.java remote interface files define the methods that can be called remotely, and the RegistrationServer.java class file defines the RegistrationServer and RegistrationImpl classes that implement the methods.

To establish remote communications, both the client and server programs need to access the remote interface classes. The server needs the interface classes to generate the interface implementation, and the client uses the remote interface class to call the remote server method implementation. For example, SellerBean creates a reference to RegistrationHome, the interface, and not RegistrationServer, the implementation, when it needs to create a user registration.

Besides the server interfaces and classes, you need stub and skeleton classes to establish remote communications. The stub and skeleton classes needed in this example are generated when you run the rmic compiler command on the RegistrationServer and SellerBean classes.

The generated SellerBean, SellerBean_Stub.class and SellerBean_Skel.class classes are needed for the callback from the server to the SellerBean client. It is the _Stub.class file on the client that marshals data to and unmarshals it from the server, while the _Skel.class class does the same for the server.

Note: In the Java 2 platform, the server side, _Skel.class file is not used.

Data Marshaling

An example of marshaling and unmarshaling data is when you call the RegistrationHome.create method from SellerBean, this call is forwarded to the RegistrationServer_Stub.create method. The RegistrationServer_Stub.create method wraps the method arguments and sends a serialized stream of bytes to the RegistrationServer_Skel.create method.

As shown in Figure 17, the RegistrationServer_Skel.create method unwraps the serialized bytestream, re-creates the arguments to the original RegistrationHome.create call, and

returns the result of calling the real RegistrationServer.create method back along the same route, but this time wrapping the data on the server side.

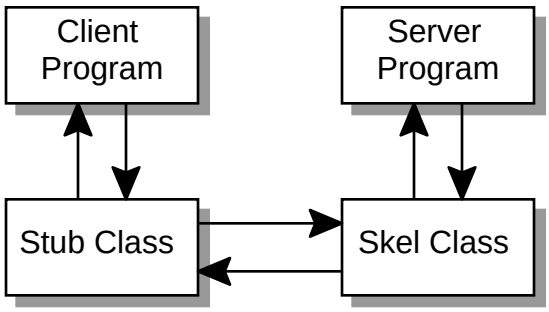


Figure 17 Data Marshaling

Marshaling and unmarshaling data is not without its complications. The first issue is that serialized objects might be incompatible across Java Development Kit (JDK™) releases. A Serialized object has an identifier stored with the object that ties the serialized object to its release. If the RMI client and server complain about incompatible serial IDs, you might need to generate backward compatible stubs and skeletons using the -vcompat option to the rmic compiler. Another issue is not all objects are serialized by default.

Fortunately, in the Java 2 platform the Collections API offers alternatives to previously unmarshalable objects. In this example, an ArrayList from the Collections API replaces the Vector. If the Collections API is not an option for you, you can create a wrapper class that extends Serializable and provides readObject and writeObject method implementations to convert the object into a bytestream.

RegistrationServer Class

The RegistrationServer (RMI) (page 165) class extends java.rmi.server.UnicastRemoteObject and implements the create, findByPrimaryKey and findLowCreditAccounts methods declared in the RegistrationHome interface. The RegistrationServer.java source file also includes the implementation for the Registration remote interface as class RegistrationImpl. RegistrationImpl also extends UnicastRemoteObject.

Exporting a Remote Object

Any object you want to be accessible remotely needs to either extend `java.rmi.server.UnicastRemoteObject` or use the `exportObject` method from the `UnicastRemoteObject` class. If you extend `UnicastRemoteObject`, you also get the `equals`, `toString` and `hashCode` methods for the exported object.

Passing by Value and Passing by Reference

Although the RegistrationImpl class is not bound to the registry, it is still referenced remotely because it is associated with the RegistrationHome return results. Because RegistrationImpl extends UnicastRemoteObject, its results are passed by reference, and so only one copy of that user's registration Bean exists in the Java virtual machine at any one time.

In the case of reporting results such as in the RegistrationServer.findLowCreditAccounts method, the RegistrationImpl class copy of the remote object could be used instead. By simply not extending UnicastRemoteObject in the RegistrationImpl class definition, a new Registration object would be returned for each request. In effect the values were passed, but not the reference to the object on the server.

Distributed Garbage Collection

Using remote references to objects on the server from a client outside the server's garbage collector introduces some potential problems with memory leaks. How does the server know it is holding onto a reference to a Registration object that is no longer being used by any clients because they aborted or a network connection was dropped?

To avoid potential memory leaks on the server from clients, RMI uses a leasing mechanism when giving out references to exported objects. When exporting an object, the Java virtual machine increases the count for the number of references to this object and sets an expiration time, or lease time, for the new reference to this object.

When the lease expires, the reference count of this object is decreased and if it reaches 0, the object is set for garbage collection by the Java virtual machine. It is up to the client that maintains this weak reference to the remote object to renew the lease if it needs the object beyond the lease time. A weak reference is a way to refer to an object in memory without keeping it from being garbage collected.

This lease time value is a configurable property measured in milliseconds. If you have a fast network, you could shorten the default value and create a large number of transient object references. The following code sets the lease timeout to 2 minutes.

```
Property prop = System.getProperties();
prop.put("java.rmi.dgc.leaseValue", 120000);
```

The create and findByPrimaryKey methods are practically identical to the other versions of the Registration Server. The main difference is that on the server side, the registration record is referenced as RegistrationImpl, which is the implementation of Registration. On the client side, Registration is used instead.

The findLowCreditAccounts method builds an ArrayList of serializable RegistrationImpl objects and calls a remote method in the SellerBean class to pass the results back. The

results are generated by an inner Thread class so the method returns before the results are complete. The SellerBean object waits for the updateAccounts method to be called before displaying the HTML page. In a client written with the Java programming language, it would not need to wait, but could display the update in real time.

```
public class RegistrationServer extends UnicastRemoteObject
                                implements RegistrationHome {
  public registration.RegistrationPK create(String theuser,
           String password, String emailaddress, String creditcard)
           throws registration.CreateException{
    double balance=0;
    Connection con = null;
    PreparedStatement ps = null;;

    try {
      con=getConnection();
      ps=con.prepareStatement("insert into registration (theuser, password,
             emailaddress, creditcard, balance) values (?, ?, ?, ?, ?)");
      ps.setString(1, theuser);
      ps.setString(2, password);
      ps.setString(3, emailaddress);
      ps.setString(4, creditcard);
      ps.setDouble(5, balance);

      if (ps.executeUpdate() != 1) {
         throw new CreateException ();//JDBC did not create any row;
      }
      RegistrationPK primaryKey = new RegistrationPK();
      primaryKey.setUser(theuser);
      return primaryKey;
    } catch (CreateException ce) {
      throw ce;
    } catch (SQLException sqe) {
      throw new CreateException ();
    } finally {
      try {
          ps.close();
          con.close();
      } catch (Exception ignore) {
        // ignore the exception as we are no longer interested
        // in this connection
      }
    }
  }
  public registration.Registration findByPrimaryKey(
                                  registration.RegistrationPK pk)
                                   throws registration.FinderException {
    // return a Registration object that is created from reading
    // values from the database by calling refresh()
    if ((pk == null) || (pk.getUser() == null)) {
      throw new FinderException ();
```

```
  }
  return(refresh(pk));
}

private Registration refresh(RegistrationPK pk) throws FinderException {
  // Read the appropriate registration fields from the database
  // based on the database item that the primary key (pk) contains
  if(pk == null) {
    throw new FinderException ();
  }
  Connection con = null;
  PreparedStatement ps = null;
  try {
    con=getConnection();
    ps=con.prepareStatement("select password, emailaddress, creditcard,
                             balance from registration where theuser = ?");
    ps.setString(1, pk.getUser());
    ps.executeQuery();
    ResultSet rs = ps.getResultSet();
    if(rs.next()) {
      RegistrationImpl reg=null;
      try{
        reg= new RegistrationImpl();
      }catch (RemoteException e) {}
        reg.theuser = pk.getUser();
        reg.password = rs.getString(1);
        reg.emailaddress = rs.getString(2);
        reg.creditcard = rs.getString(3);
        reg.balance = rs.getDouble(4);
        return reg;
    } else{
      throw new FinderException ();
    }
  } catch (SQLException sqe) {
        throw new FinderException();
  } finally {
    try {
      ps.close();
      con.close();
    } catch (Exception ignore) {}
  }
}

public void findLowCreditAccounts(
             final ReturnResults client) throws FinderException {
  // This method finds accounts with less than 3 dollars in
  // a US domain. It then creates a RegistrationImpl object and
  // calls a callback method called updateResults from the
  // ReturnResults class. It runs as a thread so the client
  // does not have to block to wait for the results to return
```

```
    Runnable bgthread = new Runnable() {
      public void run() {
        Connection con = null;
        ResultSet rs = null;
        PreparedStatement ps = null;
        ArrayList ar = new ArrayList();
        try {
          con=getConnection();
          ps=con.prepareStatement("select theuser,
                    balance from registration where balance < ?");
          ps.setDouble(1, 3.00);
          ps.executeQuery();
          rs = ps.getResultSet();
          RegistrationImpl reg=null;
          while (rs.next()) {
            try {
              reg= new RegistrationImpl();
            } catch (RemoteException e) {}
              reg.theuser = rs.getString(1);
              reg.balance = rs.getDouble(2);
              ar.add(reg);
            }
          rs.close();
          client.updateResults(ar);
          } catch (Exception e) {
            System.out.println("findLowCreditAccounts: "+e);
            return;
          }
          finally {
            try {
              if(rs != null) {
                rs.close();
              }
              if(ps != null) {
                ps.close();
                        }
              if(con != null) {
                con.close();
              }
            }catch (Exception ignore) {}
        }
      } //run
    };
    Thread t = new Thread(bgthread);
    t.start();
  }
```

The main method loads the JDBC pool driver. This version uses the Postgres database, installs the RMISecurityManager, and contacts the RMI registry to bind the RegistrationHome remote object to the name registration2. It does not need to bind the remote interface, Registration, because that class is loaded when it is referenced by RegistrationHome.

By default, the server uses port 1099. If you want to use a different port number, you can add it to the machine name with a colon as follows: phoenix:4321. If you change the port here, you must start the RMI Registry with the same port number.

The main method also installs a RMIFailureHandler. If the server fails to create a server socket then the failure handler returns true which instructs the RMI server to retry the operation.

```
      public static void main(String[] args){
        try {
          new pool.JDCConnectionDriver("postgresql.Driver",
                                "jdbc:postgresql:ejbdemo","postgres", "pass");
        } catch (Exception e){
          System.out.println("error in loading JDBC driver");
          System.exit(1);
        }
        try {
          Properties env=System.getProperties();
          env.put("java.rmi.server.codebase",
                               "http://phoenix.sun.com/registration");
          RegistrationServer rs= new RegistrationServer();
//API Ref :static SecurityManager getSecurityManager()
         if(System.getSecurityManager() == null )  {
//API Ref :static SecurityManager setSecurityManager(SecurityManager s)
            System.setSecurityManager(new RMISecurityManager());
          }
//API Ref :static void setFailureHandler(RMIFailureHandler fh)
          RMISocketFactory.setFailureHandler(new RMIFailureHandlerImpl());
//API Ref :static void rebind(String rminame, Remote obj)
          Naming.rebind("//phoenix.sun.com/registration2",rs);
        } catch (Exception e) {
          System.out.println("Exception thrown "+e);
        }
      } // End of Main
    class RMIFailureHandlerImpl implements RMIFailureHandler {
      public boolean failure(Exception ex ){
        System.out.println("exception "+ex+" caught");
        return true;
      }
    }
```

Registration Interface

The Registration (RMI) (page 169) interface declares the methods implemented by the RegistrationImpl class in the RegistrationServer.java source file.

```
package registration;

import java.rmi.*;
import java.util.*;
```

```
public interface Registration extends Remote {
  boolean verifyPassword(String password) throws RemoteException;
  String getEmailAddress() throws RemoteException;
  String getUser() throws RemoteException;
  int adjustAccount(double amount) throws RemoteException;
  double getBalance() throws RemoteException;
}
```

RegistrationHome Interface

The RegistrationHome (RMI) (page 169) interface declares the methods implemented by the RegistrationServer class. These methods mirror the Home interface defined in the Enterprise JavaBeans example. The findLowCreditAccounts method takes a remote interface as its only parameter.

```
package registration;

import java.rmi.*;
import java.util.*;

public interface RegistrationHome extends Remote {
  RegistrationPK create(String theuser, String password, String emailaddress,
                String creditcard) throws CreateException, RemoteException;
  Registration findByPrimaryKey(RegistrationPK theuser)
                throws FinderException, RemoteException;
  public void findLowCreditAccounts(ReturnResults rr)
                throws FinderException, RemoteException;
}
```

ReturnResults Interface

The ReturnResults (RMI) (page 172) interface declares the method implemented by the SellerBean class. The updateResults method is called from RegistrationServer to return data and frees the client from blocking while the results are generated.

```
package registration;
import java.rmi.*;
import java.util.*;

public interface ReturnResults extends Remote {
  public void updateResults(ArrayList results)
                 throws FinderException, RemoteException;
}
```

SellerBean Class

The SellerBean (RMI) (page 170) class includes the callback method implementation and calls the RegistrationServer object using RMI. The `updateResults` method in the `SellerBean` class is made accessible to other services in the auction application with a call to the `UnicastRemoteObject.exportObject` method.

The call to `UnicastRemoteObject` in this example makes all public methods in the `SellerBean` class available to other RMI clients. When the auction administrator wishes to retrieve the list of accounts that have a low balance, the `SellerBean.auditAccounts` method is called, which calls the remote RMI method `findLowCreditAccounts` from the `RegistrationServer` and waits on a `Boolean` object called ready.

When the remote `findLowCreditAccounts` method has finished processing the search results, it calls the updateResults method from the RMI exported `SellerBean`. The `updateResults` method updates `ArrayList` of Auction user IDs from the passed-in value and notifies all methods waiting on the Boolean object `ready` that the results have been updated. This enables the `auditAccounts` method to continue and forward the results to the Auction administrator.

```
package seller;

import java.rmi.RemoteException;
import java.rmi.*;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;
import java.rmi.server.UnicastRemoteObject;
import java.util.ArrayList;

public class SellerBean implements SessionBean, ReturnResults {
  protected SessionContext ctx;
  javax.naming.Context ectx;
  Hashtable env = new Hashtable();
  Boolean ready=new Boolean("false");
  ArrayList returned;

  public int insertItem(String seller, String password, String description,
                        int auctiondays, double startprice, String summary)
                        throws RemoteException {
    // Insert an item into the Auction items for sale. First
    // the seller details are verified with the Registration Database
    // before calling the AuctionItem Bean.
```

```
    try {
      RegistrationHome regRef = (RegistrationHome)Naming.lookup(
                                "//phoenix.sun.com/registration2");
      RegistrationPK rpk= new RegistrationPK();
      rpk.setUser(seller);
      Registration newseller = (
      Registration)regRef.findByPrimaryKey(rpk);
      if ((newseller == null) || (!newseller.verifyPassword(password))) {
        return(Auction.INVALID_USER);
      }
      AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
      AuctionItem ai = home.create(seller, description, auctiondays,
                                   startprice, summary);
      if (ai == null) {
         return Auction.INVALID_ITEM;
       } else{
         return(ai.getId());
       }
      } catch(Exception e){
        System.out.println("insert problem="+e);
        return Auction.INVALID_ITEM;
      }
    }
    public void updateResults(java.util.ArrayList ar) throws RemoteException {
      // Method called from remote rmi client as a callback
      returned=ar;
      synchronized(ready) {
        ready.notifyAll();
      }
    }

    public ArrayList auditAccounts() {
      // Call the findLowCreditAccounts method from the RegistrationServer
      // and wait for the results to be sent via the updateResults method
      try {
        RegistrationHome regRef = (RegistrationHome)Naming.lookup(
                              "//phoenix.eng.sun.com/registration2");
      regRef.findLowCreditAccounts(this);
      synchronized(ready) {
        try {
          ready.wait();
        } catch (InterruptedException e){}
      }
      return (returned);
    } catch (Exception e) {
      System.out.println("error in creditAudit "+e);
    }
    return null;
  }

  public void ejbCreate() throws javax.ejb.CreateException,
                                              RemoteException {
      // Initialize the Bean by creating the RMI connection
```

```
        // and exporting this object to the rmi registry.
        env.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
                "weblogic.jndi.TengahInitialContextFactory");
        try {
          ectx = new InitialContext(env);
        } catch (NamingException e) {
          System.out.println("problem contacting EJB server");
          throw new javax.ejb.CreateException();
        }
        Properties env=System.getProperties();
        env.put("java.rmi.server.codebase",
                "http://phoenix.sun.com/registration");
        env.put("java.security.policy","java.policy");
        UnicastRemoteObject.exportObject(this);
    }
    public void setSessionContext(SessionContext ctx) throws RemoteException {
      this.ctx = ctx;
    }
    public void unsetSessionContext() throws RemoteException {
      ctx = null;
    }
    public void ejbRemove() {}
    public void ejbActivate() throws RemoteException {
      System.out.println("activating seller bean");
    }
    public void ejbPassivate() throws RemoteException {
      System.out.println("passivating seller bean");
    }
}
```

Common Object Request Broker Architecture (CORBA)

Both the Remote Method Invocation (RMI) and Enterprise JavaBeans auction application implementations use the Java programming language to implement the different auction service tiers. However, you might need to integrate with applications written in C, C++ or other languages and running on a myriad of operating systems and machines.

One way to integrate with other applications is to transmit data in a common format such as 8 bit characters over a TCP/IP socket. The disadvantage is you have to spend a fair amount of time deriving a messaging protocol and mapping the various data structures to and from the common transmission format so the data can be sent and received over the TCP/IP connection.

This is exactly where Common Object Request Broker Architecture (CORBA) and its Interface Definition Language (IDL) can help. IDL provides a common format to represent an object that can be distributed to other applications. The other applications might not even

understand objects, but as long as they can provide a mapping between the common IDL format and their own data representations, the applications can share data.

This section describes the Java language to IDL mapping scheme, and how to replace the original container-managed RegistrationBean with its CORBA server equivalent. The SellerBean.java and AuctionServlet.java programs are changed to interoperate with the CORBA RegistrationServer program.

IDL Mapping Scheme

Many programming languages provide a mapping between their data types to the common denominator IDL format, and the Java programming language is no exception. The Java programming language can send objects defined by IDL to other CORBA distributed applications, and receive objects defined by IDL from other CORBA distributed applications.

This section describes the Java language to IDL mapping scheme and, where appropriate, discusses issues you should take into consideration.

Quick Reference

Here is a quick reference table of the Java programming language to CORBA IDL data types, and the runtime exceptions thrown when conversions fail. Data types in this table that need explanation are covered below.

Table 1 Java Data Type	**Table 2** IDL Format	**Table 3** Runtime Exception
byte	octet	
boolean	boolean	
char	char	DATA_CONVERSION
char	wchar	
double	double	
float	float	
int	long	
int	unsigned long	
long	long long	

Table 1 Java Data Type	**Table 2** IDL Format	**Table 3** Runtime Exception
long	unsigned long long	
short	short	
short	unsigned short	
java.lang.String	string	DATA_CONVERSION
java.lang.String	wstring	MARSHAL

Unsigned Values

The primitive types byte, short, int, and long are represented by 8 bit, 16 bit, 32 bit and 64 bit two's-complement integers. So, a Java short value represents the range -2^{15} to $2^{15}- 1$ or -32768 to 32767 inclusive. The equivalent signed IDL type for a short, matches that range, but the unsigned IDL short type uses the range 0 to 2^{16} or 0 to 65535.

This means that in the case of a short, if an unsigned short value greater than 32767 is passed to a program written in the Java programming language, the short value is represented in the Java programming language as a negative number. This can cause confusion in boundary tests for a value greater than 32767 or less than 0.

IDL char Types

The Java programming language uses 16-bit unicode, but the IDL char and string types are 8-bit characters. You can map a Java char to an 8-bit IDL char to transmit multi-byte characters if you use an array to do it. However, the IDL wide char type wchar is specifically designed for languages with multi-byte characters and allocates a fixed number of bytes as needed to contain that language set for each and every letter.

When mapping between the Java programming language char type and the IDL char type, a DATA_CONVERSION exception is thrown if the character does not fit into 8 bits.

IDL string Types

The IDL string type can be thought of as a sequence of IDL char types, and also raises the DATA_CONVERSION exception. The IDL wstring type is equivalent to a sequence of wchars terminated by a wchar NULL.

An IDL string and wstring type can either have a fixed size or no maximum defined sized. If you try to map a java.lang.String to a fixed size or bounded IDL string and the java.lang.String is too large, a MARSHAL exception is raised.

Setting up IDL Mappings

Java programming language to IDL mappings are placed in a file with an .idl extension. The file is compiled so it can be accessed by CORBA programs that need to send and receive data. This section explains how to construct the mappings for package statements and the Java data types. CORBA in the Auction Application (page 115) describes how to use this information to set up an IDL mapping file for the CORBA Registration server.

Java packages and interfaces. Java package statements are equivalent to the module type in IDL. The module types can be nested, which results in generated Java classes being created in nested sub-directories. IDL is compiled and the compiler maps IDL interfaces to Java interfaces.

For example, if a CORBA program contains this package statement:

```
package registration;
```

the mappings file would have this IDL module mapping for it:

```
module registration {
};
```

If a CORBA program contains a package hierarchy like this

```
package registration.corba;
```

the equivalent IDL module mapping is this:

```
module registration {
  module corba {
  };
};
```

Distributed classes are defined as Java interfaces and map to the IDL interface type. IDL does not define access such as public or private like you find in the Java programming language. It does, however, allow inheritance from other interfaces. This example adds the Java Registration interface to an IDL registration module.

```
module registration {
   interface Registration {
   };
}
```

This example adds the Java Registration interface to an IDL registration module, and indicates the Registration interface inherits from the User interface.

```
module registration {
   interface Registration: User {
```

```
    };
  }
```

Java methods. IDL operations map to Java methods. The IDL operation looks similar to a Java method except there is no concept of access control. You also have to tell the IDL compiler which parameters are in, inout or out. An inout parameter is marshaled 4 times, and in and out parameters are marshaled 2 times. Because marshling takes time, you want to specify the parameters exactly. Specifying parameters to be inout when they eally are not will only generate a lot of time consuming code.

- in - parameter is passed into the method but not changed.
- inout - parameter is passed into the method and might be returned changed.
- out - parameter might be returned changed.

This IDL mapping includes the Registration and RegistrationHome interface methods to IDL operations using one IDL module type.

```
module registration {
  interface Registration {
    boolean verifyPassword(in string password);
    string getEmailAddress();
    string getUser();
    long adjustAccount(in double amount);
    double getBalance();
  };
  interface RegistrationHome {
    Registration findByPrimaryKey( in RegistrationPK theuser) raises (FinderExce
ption);
  }
}
```

Java Arrays. Arrays in the Java programming language are mapped to the IDL array or IDL sequence type using a type definition. This example maps the Java array double balances[10] to an IDL array type of the same size.

```
typedef double balances[10];
```

These examples map the Java array double balances[10] to an IDL sequence type. The first typedef sequence is an example of an unbounded sequence, and the second typedef sequence has the same size as the array.

```
typedef sequence<double> balances;
typedef sequence<double,10> balances;
```

Java Exception. Java exceptions are mapped to IDL exceptions. Operations use IDL exceptions by including the raises keyword. This example maps the CreateException from the auction application to the IDL exception type, and adds the IDL raises keyword to the

operation as follows. IDL exceptions follow C++ syntax, so instead of throwing an exception (as you would in the Java language), the operation raises an exception.

```
exception CreateException {
};
interface RegistrationHome {
  RegistrationPK create(in string theuser, in string password,
        in string emailaddress, in string creditcard) raises (CreateException);
}
```

Other IDL keywords and types

These other basic IDL types do not have an exact equivalent in the Java programming language. Many of these should be familiar if you have used C or C++. The Java programming language provides a mapping for these types so a program written in the Java language can receive data from programs written in C or C++.

- IDL attribute keyword
- IDL enum
- IDL struct
- IDL union
- IDL Any
- IDL Principal
- IDL Object

IDL attribute

The required IDL attribute keyword is similar to the get and set methods used to access fields in the JavaBeans software. In the case of a value declared as an IDL attribute, the IDL compiler generates two methods of the same name as the IDL attribute. One method returns the field and the other method sets it. For example, this attribute definition:

```
interface RegistrationPK {
  attribute string theuser;
};
```

defines these methods

```
//return user
  String theuser();
//set user
  void theuser(String arg);
```

IDL enum

The Java programming language has an Enumeration class for representing a collection of data. The IDL enum type is different because it is declared as a data type and not a data collection. The IDL enum type is a list of values that can be referenced by name instead of by their position in the list. In the example, you can see that referring to an IDL enum status code by name is more readable than referring to it by its number. This line maps static final int values in the final class LoginError. You can reference the values as you would reference a static field: LoginError.INVALID_USER.

```
enum LoginError { INVALID_USER, WRONG_PASSWORD, TIMEOUT};
```

Here is a version of the enum type that includes a preceding underscore that can be used in switch statements:

```
switch (problem) {
  case LoginError.INVALID_USER:
    System.out.println("please login again");
    break;
}
```

IDL struct

An IDL struct type can be compared to a Java class that has only fields, which is how it is mapped by the IDL compiler. This example declares an IDL struct. Note that IDL types can reference other IDL types. In this example LoginError is from the enum type declared above.

```
struct ErrorHandler {
  LoginError errortype;
  short retries;
};
```

IDL union

An IDL union can represent one type from a list of types defined for that union. The IDL union maps to a Java class of the same name with a discriminator method used for determining the type of this union.

This example maps the GlobalErrors union to a Java class by the name of GlobalErrors. A default case: DEFAULT could be added to handle any elements that might be in the LoginErrors enum type, and not specified with a case statement here.

```
union GlobalErrors switch (LoginErrors) {
   case: INVALID_USER: string message;
   case: WRONG_PASSWORD: long attempts;
   case: TIMEOUT: long timeout;
};
```

In a program written in the Java programming language, the GlobalErrors union class is created as follows:

```
GlobalErrors ge = new GlobalErrors();
ge.message("please login again");
// The INVALID_USER value is retrieved like this:
switch (ge.discriminator().value()) {
  case: LoginError.INVALID_USER
    System.out.println(ge.message());
    break;
}
```

Any type

If you do not know what type is going to be passed or returned to an operation, you can use the Any type mapping, which can represent any IDL type. The following operation returns and passes an unknown type:

```
interface RegistrationHome {
  Any customSearch(Any searchField, out count);
};
```

To create a type of Any, first request the type from the Object Request Broker (ORB). To set a value in a type of Any, use an insert_<type> method. To retrieve a value, use the extract_<type> method. This example requests an object of type Any, and uses the insert_type method to set a value.

```
Any sfield = orb.create_any();
sfield.insert_long(34);
```

The Any type has an assigned TypeCode value that you can query using type().kind().value() on the object. The following example shows a test for the TypeCode double. This example includes a reference to the IDL TypeCode to find out which type the Any object contains. The TypeCode is used for all objects. You can analyze the type of a CORBA object using the _type or type methods as shown here.

```
public Any customSearch(Any searchField, IntHolder count){
  if(searchField.type().kind().value() == TCKind._tk_double){
//return number of balances greater than supplied amount
    double findBalance=searchField.extract_double();
```

Principal

The Principal type identifies the owner of a CORBA object, for example, a user name. The value can be interrogated from the request_principal field of the CORBA RequestHeader class to make the identification. More comprehensive security and authorization is available in the CORBA security service.

Object

The Object type is a CORBA object. If you need to send Java objects, you have to either translate them into an IDL type or use a mechanism to serialize them when they are transferred.

CORBA in the Auction Application

The container-managed RegistrationBean from the auction application in RegistrationBean (page 38) is completely replaced with a standalone CORBA RegistrationServer (CORBA) (page 181) that implements the registration service. The CORBA RegistrationServer is built by creating and compiling an IDL mappings file so client programs can communicate with the registration server.

The SellerBean (CORBA) (page 187) and AuctionServlet (CORBA) (page 191) sources are updated to look up the CORBA registration server.

CORBA RegistrationServer Implementation

This section describes the Registration.idl (CORBA) (page 187) file, which maps the RegistrationHome and Registration remote interfaces from the Enterprise JavaBean auction application to their IDL equivalents and shows how to compile the Registration.idl file into CORBA registration server classes.

The CORBA registration server implements the create and findByPrimaryKey methods from the original RegistrationBean.java file, and is enhanced with the following two new methods to help illustrate CORBA callbacks and how to use the Any type.

- findLowCreditAccounts(in ReturnResults rr), which uses a callback to return a list of accounts with a low balance.
- Any customSearch(in Any searchfield, out long count), which returns a different search result depending on the search field type submitted.

IDL Mappings File

Here is the Registration.idl (CORBA) (page 187) file that maps the data types and methods used in the RegistrationHome and Registration programs to their IDL equivalents.

```
module registration {
  interface Registration {
    boolean verifyPassword(in string password);
    string getEmailAddress();
    string getUser();
    long adjustAccount(in double amount);
    double getBalance();
  };
```

```
  interface RegistrationPK {
   attribute string theuser;
  };
  enum LoginError {INVALIDUSER, WRONGPASSWORD, TIMEOUT};
  exception CreateException {
  };
  exception FinderException {
  };
  typedef sequence<Registration> IDLArrayList;
  interface ReturnResults  {
    void updateResults(in IDLArrayList results) raises (FinderException);
  };
  interface RegistrationHome {
    RegistrationPK create(in string theuser, in string password,
                          in string emailaddress, in string creditcard)
                          raises (CreateException);
    Registration findByPrimaryKey(in RegistrationPK theuser)
                          raises (FinderException);
    void findLowCreditAccounts(in ReturnResults rr) raises (FinderException);
    Any customSearch(in Any searchfield, out long count);
  };
};
```

Compiling the IDL Mappings File

The IDL file has to be converted into Java classes that can be used in the CORBA distributed network. The Java 2 platform compiles .idl files using the program idltojava. This program will be replaced with the idltoj command. The -fno-cpp arguments indicate there is no C++ compiler installed.

```
idltojava -fno-cpp Registration.idl
```

Other Java IDL compilers should also work, for example, jidl from ORBacus can generate classes that can be used by the Java 2 ORB.

Stubs and Skeletons

CORBA and RMI are similar in that compilation generates a stub file for the client and a skeleton file for the server. As shown in Figure 18, the stub (or proxy) and skeleton (or servant) are used to marshal and unmarshal data between the client and server. The skeleton is used by the server. In this example, the IDL RegistrationHome interface mapping generates

a _RegistrationHomeImplBase class (the skeleton or servant class) that the generated RegistrationServer class extends.

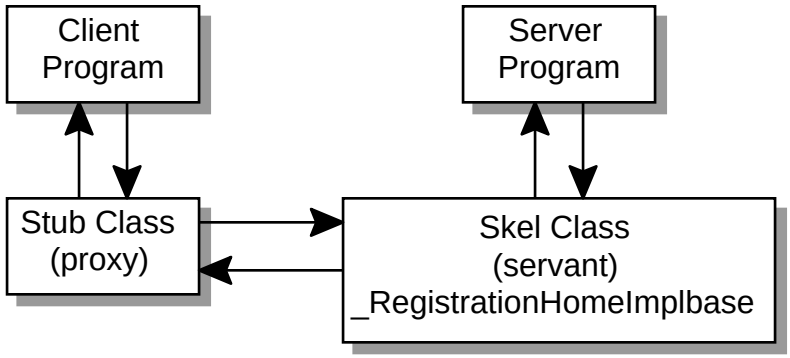


Figure 18 Marshaling Data

When requesting a remote CORBA object or calling a remote method, the client call passes through the stub class before reaching the server. This proxy class invokes CORBA requests for the client program. The following example is the code automatically generated for the RegistrationHomeStub.java class.

```
org.omg.CORBA.Request r = _request("create");
r.set_return_type(registration.RegistrationPKHelper.type());
org.omg.CORBA.Any _theuser = r.add_in_arg();
```

Object Request Broker

The center of the CORBA distributed network is the Object Request Broker or ORB. The ORB is involved in marshaling and unmarshaling objects between the client and server. Other services such as the Naming Service and Event Service work with the ORB.

The Java 2 platform includes an ORB in the distribution called the IDL ORB. This ORB is different from many other ORBs because it does not include a distinct Basic Object Adapter (BOA) or Portable Object Adapter (POA).

An object adapter manages the creation and lifecycle of objects in the CORBA distributed space. This can be compared to the container in the Enterprise JavaBeans server managing the lifecycle of the session and entity beans.

The AuctionServlet (CORBA) (page 191) and Seller (CORBA) (page 191) programs create and initialize a Java 2 ORB like this:

```
//API Ref :status ORB init(String{} args, Properties props)
    ORB orb = ORB.init(args, null);
```

In the RegistrationServer (CORBA) (page 181) program, the server object to be distributed is bound to the ORB using the connect method:

```
    RegistrationServer rs = new RegistrationServer();
//API Ref :void connect(Object object)
    orb.connect(rs);
```

An object connected to an ORB can be removed with the disconnect method:

```
//API Ref :void disconnect(Object object)
    orb.disconnect(rs);
```

Once connected to a CORBA server object, the Java 2 ORB keeps the server alive and waits for client requests to the CORBA server.

```
    java.lang.Object sync = new java.lang.Object();
    synchronized(sync) {
      sync.wait();
    }
```

Making the CORBA Server Accessible

Although this object is now being managed by the ORB, the clients do not yet have a mechanism to find the remote object. This can be solved by binding the CORBA server object to a naming service.

The Java 2 naming service is called tnameserv. The naming service by default uses port 900; however, this value can be changed by setting the argument -ORBInitialPort portnumber when starting tnameserv or setting the property org.omg.CORBA.ORBInitialPort when starting the client and server processes.

```
    java.util.Properties props=System.getProperties();
    props.put("org.omg.CORBA.ORBInitialPort", "1050");
    System.setProperties(props);
    ORB orb = ORB.init(args, props);
```

The next lines from the main method in the RegistrationServer (CORBA) (page 181) class show how this naming reference is created. The first step is to request the service called NameService by calling the resolve_initial_references method from the Object Request Broker with the value NameService. This technique is used to find other services from the ORB, not just the NameService.

The NamingContext is retrieved by narrowing the returned Object to the `NameComponent` type. The name is built up and bound to the naming service as a list of `NameComponent` elements. The name in this example has a root called auction with this object being bound at the next level of the name tree as `RegistrationBean`. The naming scheme could be used to mirror the package and class that the object came from, in this example it could be used to describe that the object was also the class `auction.RegistrationBean`.

```
    org.omg.CORBA.Object nameServiceObj = orb.resolve_initial_references(
                                          NameService");
    NamingContext nctx = NamingContextHelper.narrow(nameServiceObj);
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent("RegistrationBean", "");
//API Ref :NameComponent(String nameid, String kind)
    NameComponent[] tempComponent = new NameComponent[1];
    for(int i=0; i < fullname.length-1; i++ ) {
        tempComponent[0]= fullname[i];
        try {
          nctx=nctx.bind_new_context(tempComponent);
        } catch (org.omg.CosNaming.NamingContextPackage.AlreadyBound e){
          // It may already be bound so ignore
    }
    tempComponent[0]=fullname[fullname.length-1];
    try {
      nctx.rebind(tempComponent, rs);
    } catch (Exception e){
      System.out.println("rebind failed"+e);
    }
```

Plugging in a new ORB

The Java 2 IDL ORB does not currently include some of the services available in many other commercial ORBs such as security or event notification services. However, you can use another ORB in the Java 2 runtime by configuring two properties and including any necessary object adapter code.

Using a new ORB in the registration server requires the org.omg.CORBA.ORBClass and org.omg.CORBA.ORBSingletonClass properties point to the appropriate ORB classes. In this example the ORBacus ORB is used instead of the Java 2 IDL ORB. To use another ORB, the code below should be plugged into the RegistrationServer.main method.

In the example code, a SingletonClass ORB is used. The SingletonClass ORB is not a full ORB, and is primarily used as a factory for TypeCodes. The call to ORB.init() in the last line creates the Singleton ORB.

```
Properties props= System.getProperties();
props.put("org.omg.CORBA.ORBClass", "com.ooc.CORBA.ORB");
props.put("org.omg.CORBA.ORBSingletonClass","com.ooc.CORBA.ORBSingleton");
System.setProperties(props);
ORB orb = ORB.init(args, props) ;
```

In the Java 2 IDL, there is no distinct object adapter. As shown in the example code segment below, using the Basic Object Adapter from ORBacus requires an explicit cast to the ORBacus ORB. The Broker Object Architecture (BOA) is notified that the object is ready to be distributed by calling the impl_is_ready(null) method.

```
BOA boa = ((com.ooc.CORBA.ORB)orb).BOA_init(args, props);
 ...
boa.impl_is_ready(null);
```

Although both the ORBSingletonClass and ORBClass ORBs build the object name using NameComponent, you have to use a different ORBacus Naming Service. The CosNaming.Server service is started as follows where the -OAhost parameter is optional:

```
java com.ooc.CosNaming.Server -OAhost localhost -OAport 1060
```

Once the naming service is started, the server and client programs find the naming service using the IIOP protocol to the host and port named when starting the Naming service:

```
java registration.RegistrationServer -ORBservice NameService
                            iiop://localhost:1060/DefaultNamingContext
```

Naming Service Access by CORBA Clients

CORBA clients access the naming service in a similar way to the server, except that instead of binding a name, the client resolves the name built from the NameComponents. The AuctionServlet and SellerBean classes use the following code to look up the CORBA server:

```
//API Ref :NameComponent(String nameid, String kind)
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent("RegistrationBean", "");

    RegistrationHome regRef = RegistrationHomeHelper.narrow(nctx.resolve(fullname));
```

In the case of the ORBacus ORB, the clients also need a Basic Object Adapter if callbacks are used as in the SellerBean.auditAccounts method. The naming context helper is also configured differently for the ORBacus server started earlier:

```
Object obj = ((com.ooc.CORBA.ORB)orb).get_inet_object ("localhost",
                                    1060, "DefaultNamingContext");
NamingContext nctx = NamingContextHelper.narrow(obj);
```

Helper and Holder classes

References to remote objects in CORBA use a Helper class to retrieve a value from that object. A commonly used method is the Helper narrow method, which ensures the object is cast correctly.

`Holder` classes hold values returned when using `inout` or `out` parameters in a method. The caller first instantiates the appropriate Holder class for that type and passes the Holder class to the method call as one of the parameters. On return from the method call, the application retrieves the value that has been changed in the Holder.

In the next example, the count of how many occurrences of the search term is held in a instance of an `int` type `Holder` object. The `Holder` object, count is passed to the `customSearch` method where the field called value in the `Holder` object is changed on the remote end. On return from the `customSearch` method, the count value is retrieved by accessing the `value` field.

```
//API Ref :IntHolder()
    IntHolder count= new IntHolder();
    sfield=regRef.customSearch(sfield,count);
    System.out.println("count now set to "+count.value);
```

Garbage Collection

Unlike RMI, CORBA does not have a distributed garbage collection mechanism. References to an object are local to the client proxy and the server servant. This means each Java virtual machine is free to reclaim that object and garbage collect it if there are no longer references to it. If an object is no longer needed on the server, the orb.disconnect(object) needs to be called to allow the object to be garbage collected.

CORBA Callbacks

The new `findLowCreditAccounts` method is called from the `AuctionServlet` when the user uses the Uniform Resource Locator (URL) http://localhost:7001/AuctionServlet?action=auditAccounts. The AuctionServlet.auditAccounts method calls the `SellerBean.auditAccounts` method, which returns an `ArrayList` of `Registration` records.

```
    // From file AuctionServlet.java
      private void auditAccounts(ServletOutputStream out,HttpServletRequest request)
                                 throws IOException{
    //   ...
       SellerHome home = (SellerHome) ctx.lookup("seller");
       Seller si= home.create();
       if(si != null) {
         // Call the auditAccounts method in SellerBean and return
         // an ArrayList of Registration records. Call the getUser
         // and getBalance methods on each Registration Object.
         ArrayList ar=si.auditAccounts();
//API Ref :Interator iterator()
//API Ref :boolean hasNext()
         for(Iterator i=ar.iterator(); i.hasNext();) {
           Registration user=(Registration)(i.next());
           addLine("<TD>"+user.getUser() + "<TD><TD>"+user.getBalance() +
                   "<TD><TR>", out);
         }
           addLine("<TABLE>", out);
        }
```

The SellerBean object calls the CORBA RegistrationHome.findLowCreditAccounts method implemented in the RegistrationServer class, and passes a reference to itself as a parameter in the findLowCreditAccounts method call. The reference can be passed because the SellerBean class implements the ReturnResults interface declared in the Registration.idl file.

```
    // From file SellerBean.java
      public ArrayList auditAccounts() {
        try {
//API Ref :NameComponent(String nameid, String kind)
          NameComponent[] fullname = new NameComponent[2];
          fullname[0] = new NameComponent("auction", "");
          fullname[1] = new NameComponent("RegistrationBean", "");
          RegistrationHome regRef = RegistrationHomeHelper.narrow(
                                    nctx.resolve(fullname));
          // Pass a reference to this SellerBean object to the
          // Registration CORBA method findLowCreditAccounts
          regRef.findLowCreditAccounts(this);
          synchronized(ready) {
            try {
              ready.wait();
            } catch (InterruptedException e){}
          }
            return (returned);
          } catch (Exception e) {
            System.out.println("error in auditAccounts "+e);
          }
          return null;
      }
```

The RegistrationServer.findLowCreditAccounts method retrieves user records from the database registration table that have a credit value less than three. It then returns the list of Registration records in an ArrayList by calling the SellerBean.updateResults method to which it has a reference.

```
    // From file RegistrationServer.java
      public void findLowCreditAccounts(final ReturnResults client)
                                        throws FinderException {
        // Start a new thread to do the search because it might
        // take a while to complete
        Runnable bgthread = new Runnable() {
          public void run() {
            Connection con = null;
            ResultSet rs = null;
            PreparedStatement ps = null;
            ArrayList ar = new ArrayList();
            try {
              con=getConnection();
              // Search for the records we are interested in using
              // a simple JDBC call
//API Ref :PrepareStatement prepareStatement(String sql)
```

```
            ps=con.prepareStatement("select theuser, balance from registration
                                     where balance < ?");
//API Ref :void setDouble(int index, double doublevalue)
            ps.setDouble(1, 3.00);
            //Execute the prepared statement query
//API Ref :ResultSet executeQuery()
            ps.executeQuery();
            rs = ps.getResultSet();
            RegistrationImpl reg=null;
            while(rs.next()) {
              // Create a Registration object for each row and add each
              // Registration to an ArrayList
              try {
                reg= new RegistrationImpl();
              } catch (Exception e) {
                System.out.println("Problem creating registration record"+e);
              }
//API Ref :String getString(String columnName)
                reg.theuser = rs.getString(1);
//API Ref :double getDouble(String columnName)
                reg.balance = rs.getDouble(2);
//API Ref :boolean add(Object object)
                ar.add(reg);
              }
              rs.close();
//API Ref :Object[] toArray(Object[] objarray)
              RegistrationImpl[] regarray = (RegistrationImpl [])ar.toArray(
                                            new RegistrationImpl[0]);
              // Convert the ArrayList to an array of RegistrationImpl objects
              // and pass the results back by calling the updateResults method
              //  from the SellerBean client
              client.updateResults(regarray);
            } catch (Exception e) {
              System.out.println("findLowCreditAccounts: "+e);
              return;
            } finally {
              try {
                if(rs != null) {
                  rs.close();
                }
                if(ps != null) {
                  ps.close();
                }
                if(con != null) {
                  con.close();
                }
              } catch (Exception ignore) {}
            }
          }//run
        };
      Thread t = new Thread(bgthread);
      t.start();
      }
```

The SellerBean.updateResults method updates the global ArrayList of Registration records returned by the RegistrationServer object and notifies the SellerBean.auditAccounts method that it can return that ArrayList of Registration records to the AuctionServlet.

```
// From file SellerBean.java
ArrayList returned =new ArrayList();

 public void updateResults(Registration[] ar)
                              throws registration.FinderException {
    if (ar == null) {
       throw new registration.FinderException();
    }
    try {
      for(int i=0; i< ar.length; i++) {
        returned.add(ar[i]);
      }
    } catch (Exception e) {
      System.out.println("updateResults error:"+e);
      throw new registration.FinderException();
    }
    synchronized(ready) {
      ready.notifyAll();
    }
  }
```

Using the Any type

The RegistrationServer.customSearch method uses the IDL `Any` type to pass in and return results. Users can call the `customSearch` method from the `AuctionServlet` as follows:

```
http://phoenix.sun.com:7001/
AuctionServlet?action=customSearch&searchfield=2
```

The searchfield parameter can be set to a number or a string. The AuctionServlet.customFind method passes the search field directly to the SellerBean.customFind method and retrieves a String that is then displayed to the user.

```
// From file AuctionServlet.java

private void customSearch(ServletOutputStream out,HttpServletRequest request)
                           throws IOException{
   String text = "Custom Search";
   String searchField=request.getParameter("searchfield");
   setTitle(out, "Custom Search");
   if(searchField == null ) {
     addLine("Error: SearchField was empty", out);
     out.flush();
     return;
   }
   try {
```

```
          addLine("<BR>"+text, out);
          SellerHome home = (SellerHome)ctx.lookup("seller");
          Seller si= home.create();
          if(si != null) {
            // Call customFind method from SellerBean and display results
            String displayMessage=si.customFind(searchField);
            if(displayMessage != null ) {
              addLine(displayMessage+"<BR>", out);
            }
          }
        } catch (Exception e) {
          addLine("AuctionServlet customFind error",out);
          System.out.println("AuctionServlet " + "<customFind>:"+e);
        }
        out.flush();
      }
```

The `SellerBean.customFind` method calls the `RegistrationHome` object implemented in the RegistrationServer (CORBA) (page 181) class, and because the `searchField` might be a number or a string, an object of the CORBA type `Any` is used to transfer the value.

The convenience of not needing to specify the type of the object incurs an extra cost because special methods are required to set and retrieve values from an object of type `Any`. The `Any` object is created by a call to the ORB, `orb.create_any` method.

The `customFind` method also uses an out parameter, `count`, of type `int` that returns the number of records found. The number of records found is retrieved from `count.value` when the `customFind` method returns. The return type of the `customFind` method also is type `String` and is described after the code sample.

```
     // From file SellerBean.java
      public String customFind(String searchField)
                               throws javax.ejb.FinderException, RemoteException{
      int total=-1;
      IntHolder count= new IntHolder();
      try {
//API Ref :NameComponent(String nameid, String kind)
        NameComponent[] fullname = new NameComponent[2];
        fullname[0] = new NameComponent("auction", "");
        fullname[1] = new NameComponent(
                              "RegistrationBean", "");
        RegistrationHome regRef =
                        RegistrationHomeHelper.narrow(nctx.resolve(fullname));
          if(regRef == null ) {
            System.out.println("cannot contact RegistrationHome");
            throw new javax.ejb.FinderException();
          }
//API Ref :Any create_any()
          Any sfield=orb.create_any();
          Double balance;
          try {
           // Treat the search value as an account balance. If it cannot
```

```
          // be converted to a number then its an email address so send it as a String
//API Ref :static Double valueOf(double doublevalue)
            balance=Double.valueOf(searchField);
            try {
              // Insert the double value held as balance into the Any object
//API Ref :void insert_double(double doublevalue)
              sfield.insert_double(balance.doubleValue());
            } catch (Exception e) {
              return("Problem with search value"+balance);
            }
           sfield=regRef.customSearch(sfield,count);
           // The number of records is available in the count out parameter
           // and in the return object.
           if(sfield != null ) {
             total=sfield.extract_long();
           }
           return(total+" accounts are below optimal level from" +
                  count.value+" records");
         } catch (NumberFormatException e) {
           // The value could not be converted to a double so search
           // with the value of type String
//API Ref :void insert_string(String stringvalue)
           sfield.insert_string(searchField);
           Registration reg;
           // The returned object contains a Registration record stored
           // as a type Any.
           if((reg=RegistrationHelper.extract(regRef.customSearch(sfield,count)))
                                                    != null ) {
            return("Found user "+reg.getUser() +" who has email address "+
                   reg.getEmailAddress());
          } else {
            return("No users found who have email address " + searchField);
          }
         }
        } catch(Exception e){
          System.out.println("customFind problem="+e);
          throw new javax.ejb.FinderException();
        }
      }
```

The return value from the call to `customFind` is extracted into an object of type `Any`, the results of the search are then displayed to the user in the `AuctionServlet`. For simple types, the `extract_<type>` method of the `Any` object can be used. However, for the `Registration` type, the `RegistrationHelper` class is used.

```
Registration reg = RegistrationHelper.extract(
                                regRef.customSearch(sfield,count));
```

The RegistrationServer.customSearch method determines the type of Object being passed in the searchField parameter by checking the .type().kind().value() of the Any object.

```
if(searchField.type().kind().value() == TCKind._tk_double)
```

Finally, because the customSearch method returns an object of type Any, a call to orb.create_any() is required. For simple types like double, the insert_<type> method is used. For a Registration record, the RegistrationHelper class is used like this: RegistrationHelper.insert(returnResults, regarray[0]).

```
    // From file RegistrationServer.java
      public Any customSearch(Any searchField, IntHolder count){
        Any returnResults= orb.create_any();
        int tmpcount=count.value;
        if(searchField.type().kind().value() == TCKind._tk_double){
          // Search database for number of accounts where the balance is
          // less than supplied amount
          double findBalance=searchField.extract_double();
          Connection con = null;
          ResultSet rs = null;
          PreparedStatement ps = null;
          try {
            con=getConnection();
//API Ref :PrepareStatement prepareStatement(String sql)
            ps=con.prepareStatement("select count(*) from
                                registration where balance < ?");
//API Ref :void setDouble(int index, double doublevalue)
            ps.setDouble(1, findBalance);
//API Ref :ResultSet executeQuery()
            ps.executeQuery();
            rs = ps.getResultSet();
//API Ref :boolean next()
            if(rs.next()) {
//API Ref :int getInt(String ColumnName)
              tmpcount = rs.getInt(1);
            }
            count.value=tmpcount;
            rs.close();
           } catch (Exception e) {
             System.out.println("custom search: "+e);
             returnResults.insert_long(-1);
             return(returnResults);
           } finally {
             try {
               if(rs != null) { rs.close(); }
               if(ps != null) { ps.close(); }
               if(con != null) { con.close(); }
             } catch (Exception ignore) {}
           }
             returnResults.insert_long(tmpcount);
             return(returnResults);
         } else if(searchField.type().kind().value() == TCKind._tk_string) {
           // If the any value is of type string then the admin was
           // looking for an email address so return the
```

```
        // email addresses that match supplied address
        String findEmail=searchField.extract_string();
        Connection con = null;
        ResultSet rs = null;
        PreparedStatement ps = null;
        ArrayList ar = new ArrayList();
        RegistrationImpl reg=null;
        try {
          con=getConnection();
          ps=con.prepareStatement("select theuser, emailaddress from
                    registration where emailaddress like ?");
          ps.setString(1, findEmail);
          ps.executeQuery();
          rs = ps.getResultSet();
          while (rs.next()) {
            reg= new RegistrationImpl();
            reg.theuser = rs.getString(1);
            reg.emailaddress = rs.getString(2);
            // Add each entry to an ArrayList in readiness to convert
            // an array of RegistrationImpl records
            ar.add(reg);
          }
          rs.close();
         RegistrationImpl[] regarray = (RegistrationImpl [])ar.toArray(
         new RegistrationImpl[0]);
         RegistrationHelper.insert(returnResults, regarray[0]);
         return(returnResults);
        } catch (Exception e) {
          System.out.println("custom search: "+e);
          return(returnResults);
        }
        finally {
          try {
            if(rs != null) { rs.close(); }
            if(ps != null) { ps.close(); }
            if(con != null) { con.close(); }
          } catch (Exception ignore) {}
        }
      }
      return(returnResults);
    }
```

In Conclusion

As you have seen, converting the application to use RMI or CORBA requires very little change to core programs. The main difference has been the initialization and naming service. By abstracting these two areas in your application away from the business logic you ease migration between different distributed object architectures for the Java platform.

JDBC Technology

The Bean-managed Enterprise JavaBeans auction application with its Remote Method Invocation (RMI) and Common Object Request Broker (CORBA) variants has used simple JDBC calls to retrieve and update information from a database using a JDBC connection pool. By default, JDBC database access involves opening a database connection, running SQL commands in a statement, processing the returned results, and closing the database connection.

Overall, the default approach works well for low volume database access, but how do you manage a large number of requests that update many related tables at once and still ensure data integrity? This section explains how.

JDBC Drivers

The connection to the database is handled by the JDBC Driver class. The Java SDK contains only one JDBC driver, a jdbc-odbc bridge that can communicate with an existing Open DataBase Connectivity (ODBC) driver. Other databases need a JDBC driver specific to that database.

To get a general idea of what the JDBC driver does, you can examine the JDCConnectionDriver (page 391) class. The JDCConnectionDriver class implements the java.sql.Driver class and acts as a pass-through driver by forwarding JDBC requests to the real database JDBC Driver. The JDBC driver class is loaded with a call to Class.forName(drivername).

These next code lines show how to load three different JDBC driver classes:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Class.forName("postgresql.Driver");
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Each JDBC driver is configured to understand a specific URL so multiple JDBC drivers can be loaded at any one time. When you specify a URL at connect time, the first matching JDBC driver is selected.

The jdbc-odbc bridge accepts Uniform Resource Locators (URLs) starting with jdbc:odbc: and uses the next field in that URL to specify the data source name. The data source name identifies the particular database scheme you wish to access. The URL can also include more details on how to contact the database and enter the account.

```
//access the ejbdemo tables
  String url = "jdbc:odbc:ejbdemo";
```

This next example contains the Oracle SQL*net information on the particular database called ejbdemo on machine dbmachine.

```
String url = "jdbc:oracle:thin:user/password@(description=(address_list=(
       address=(protocol=tcp (host=dbmachine)(port=1521)))(source_route=yes)
       (connect_data=(sid=ejbdemo)))";
```

This next example uses mysql to connect to the ejbdemo database on the local machine. The login user name and password details are also included.

```
String url = "jdbc:mysql://localhost/ejbdemo?user=user;
                                                password=pass";
```

JDBC drivers are divided into four types. Drivers may also be categorized as pure Java or thin drivers to indicate if they are used for client applications (pure Java drivers) or applets (thin drivers). Newer drivers are usually Type 3 or 4. The four types are as follows:

Type 1 Drivers

Type 1 JDBC drivers are the bridge drivers such as the jdbc-odbc bridge. These drivers rely on an intermediary such as ODBC to transfer the SQL calls to the database. Bridge drivers often rely on native code, although the jdbc-odbc library native code is part of the Java 2 virtual machine.

Type 2 Drivers

Type 2 Drivers use the existing database API to communicate with the database on the client. Although Type 2 drivers are faster than Type 1 drivers, Type 2 drivers use native code and require additional permissions to work in an applet. A Type 2 driver might need client-side database code to connect over the network.

Type 3 Drivers

Type 3 Drivers call the database API on the server. JDBC requests from the client are first proxied to the JDBC Driver on the server to run. Type 3 and 4 drivers can be used by thin clients as they need no native code.

Type 4 Drivers

The highest level of driver reimplements the database network API in the Java programming language. Type 4 drivers can also be used on thin clients as they also have no native code.

Database Connections

A database connection can be established with a call to the DriverManager.getConnection method. The call takes a URL that identifies the database, and optionally, the database login user name and password.

```
//API Ref :static Connection getConnection(String url)
```

```
    Connection con = DriverManager.getConnection(url);
//API Ref :static Connection getConnection(String url, String user, String password)
    Connection con = DriverManager.getConnection(url, "user", "password");
```

After a connection is established, a statement can be run against the database. The results of the statement can be retrieved and the connection closed.

One useful feature of the DriverManager class is the setLogStream method. You can use this method to generate tracing information to help you diagnose connection problems that would normally not be visible. To generate tracing information, just call the method like this:

```
//API Ref :static void setLogStream(PrintStream out)
    DriverManager.setLogStream(System.out);
```

Connection Pooling (page 344) shows you how to improve the throughput of JDBC connections by not closing the connection once the statement completes. Each JDBC connection to a database incurs overhead in opening a new socket and using the username and password to log into the database. Reusing the connections reduces the overhead. The Connection Pool keeps a list of open connections and clears any connections that cannot be reused.

Statements

There are three basic types of SQL statements used in the JDBC API: CallableStatement, Statement, and PreparedStatement. When a Statement or PreparedStatement is sent to the database, the database driver translates it into a format the underlying database can recognize.

Callable Statements

Once you have established a connection to a database, you can use the Connection.prepareCall method to create a callable statement. A callable statement lets you execute SQL stored procedures. This next example creates a CallableStatement object with three parameters for storing account login information.

```
//API Ref :CallableStatement prepareCall(String sql)
      CallableStatement cs = con.prepareCall("{call accountlogin(?,?,?)}");
//API Ref :void setString(int index, String s)
      cs.setString(1,theuser);
      cs.setString(2,password);
//API Ref :void registerOutParameter(int index, int sqltype)
      cs.registerOutParameter(3,Types.DATE);
      cs.executeQuery();
//API Ref :Date getDate(int index)
      Date lastLogin = cs.getDate(3);
```

Statements

The Statement interface lets you execute a simple SQL statement with no parameters. The SQL instructions are inserted into the Statement object when the Statement.execute<*type*> method is called.

Query Statement. This code segment creates a Statement object and calls the Statement.executeQuery method to select text from the dba database table. The results of the query are returned in a ResultSet object. How to retrieve results from a ResultSet object is explained in Result Sets (page 133) below.

```
//API Ref :Statement createStatement()
     Statement stmt = con.createStatement();
//API Ref :int executeQuery(String sql)
     ResultSet results = stmt.executeQuery("SELECT TEXT FROM dba ");
```

Update Statement. This next code segment creates a Statement object and calls the Statement.executeUpdate method to add an email address to a table in the dba database table.

```
    String updateString = "INSERT INTO dba VALUES ('some text')";
//API Ref :int executeUpdate(String sql)
    int count = stmt.executeUpdate(updateString);
```

Prepared Statements

The PreparedStatement interface descends from the Statement interface and uses a template to create a SQL request. Use a PreparedStatement to make multiple database operations where only the values change. The advantage to using a PreparedStatement is that the statement is compiled the first time and subsequent calls are optimized by inserting only the changed data.

Note: Not all database drivers compile prepared statements.

Query PreparedStatement. You create a PreparedStatement object by specifying the template definition and parameter placeholders. The parameter data is inserted into the PreparedStatement object by calling its set<*type*> methods and specifying the parameter and its data. The SQL instructions and parameters are sent to the database when the execute<*type*> method is called.

This next code segment creates a PreparedStatement object to select user data based on the user's email address. The question mark ("?") indicates this statement has one parameter.

```
//API Ref :PrepareStatement prepareStatement(String sql)
```

```
    PreparedStatement pstmt = con.prepareStatement(select theuser from
                              registration where emailaddress like ?");
    //Initialize first parameter with email address
//API Ref :void setString(int index, String s)
    pstmt.setString(1, emailAddress);
//API Ref :ResultSet executeQuery()
    ResultSet results = ps.executeQuery();
```

Once the PreparedStatement template is initialized, only the changed values are inserted for each call.

```
    pstmt.setString(1, anotherEmailAddress);
```

Update PreparedStatement. This code segment creates a PreparedStatement object to update a seller's registration record. The template has five parameters, which are set with five calls to the appropriate PreparedStatement.set<*type*>X methods.

```
    PreparedStatement ps = con.prepareStatement(
                       "insert into registration(theuser, password,
                       emailaddress, creditcard, balance) values (?, ?, ?, ?, ?)");
    ps.setString(1, theuser);
    ps.setString(2, password);
    ps.setString(3, emailaddress);
    ps.setString(4, creditcard);
    ps.setDouble(5, balance);
    ps.executeUpdate();
```

Caching Database results

The PreparedStatement concept of reusing requests can be extended to caching the results of a JDBC call. For example, an auction item description remains the same until the seller changes it. If the item receives thousands of requests, the results of the query statement "select description from auctionitems where item_id='4000343'" might be stored more efficiently in a hash table.

Storing results in a hash table requires the JDBC call be intercepted before creating a real statement to return the cached results, and the cache entry be cleared if there is a corresponding update to that item_id.

Result Sets

The ResultSet interface manages access to data returned from a query. The data returned equals one row in a database table. Some queries return one row of data while many queries return multiple rows of data.

You use getType methods to retrieve data from specific columns for each row returned by the query. The SELECT TEXT FROM dba query selects the TEXT column from the daba table.

```
    Statement stmt = con.createStatement();
    ResultSet results = stmt.executeQuery("SELECT TEXT FROM dba ");
//API Ref :boolean next()
      while(results.next()){
//API Ref :String getString(String columnName)
        String s = results.getString("TEXT");
        displayText.append(s + "\n");
      }
    stmt.close();
```

Scrolling Result Sets

Before JDBC 2.0, JDBC drivers returned read-only result sets with cursors that moved in one direction, forwards. Each element was retrieved by calling the next method on the result set. JDBC 2.0 introduces scrollable results sets whose values can be read and updated if reading and updating is supported by the underlying database. With scrollable result sets, any row can be selected at random, and the result set can be traversed forwards and backwards.

One advantage to the new result set is you can update a set of matching rows without having to issue an additional executeUpdate call. The updates are made using JDBC calls so no custom SQL commands need to be generated. This improves the portability of the database code you create.

Both Statements and PreparedStatements have an additional constructor that accepts a scroll and an update type parameter. The scroll type value can be any one of the following:

- ResultSet.TYPE_FORWARD_ONLY: Default behavior in JDBC 1.0, application can only call next() on the result set.
- ResultSet.SCROLL_SENSITIVE: ResultSet is fully navigable and updates are reflected in the result set as they occur.
- ResultSet.SCROLL_INSENSITIVE: Result set is fully navigable, but updates are only visible after the result set is closed. You need to create a new result set to see the results.

The update type parameter can be one of the following two values:

- ResultSet.CONCUR_READ_ONLY: The result set is read only.
- ResultSet.CONCUR_UPDATABLE: The result set can be updated.

You can verify that your database supports these types by calling con.getMetaData().supportsResultSetConcurrency() method as shown here.

```
    Connection con = getConnection();
//API Ref :DatabaseMetaData getMetaData()
```

```
//API Ref :boolean supportsResultSetConcurrency(int type, int concurrency)
    if(con.getMetaData().supportsResultSetConcurrency(
      ResultSet.SCROLL_INSENSITIVE,
      ResultSet.CONCUR_UPDATABLE)) {
      PreparedStatement pstmt = con.prepareStatement("select password, emailaddres,
                          creditcard, balance from registration where theuser = ?",
                          ResultSet.SCROLL_INSENSITIVE,ResultSet.CONCUR_UPDATABLE);
     }
```

Navigating the ResultSet

The fully scrollable result set returns a cursor that can be moved using simple commands. By default, the result set cursor points to the row before the first row of the result set. A call to the next method retrieves the first result set row. The cursor can also be moved by calling one of the following ResultSet methods:

- beforeFirst(): Default position. Puts the cursor before the first row of the result set.
- first(): Puts the cursor on the first row of the result set.
- last(): Puts the cursor before the last row of the result set.
- afterLast() Puts the cursor beyond last row of the result set. Calls to previous moves backwards through the ResultSet.
- absolute(pos): Puts the cursor at the row number position where absolute(1) is the first row and absolute(-1) is the last row.
- relative(pos): Puts the cursor at a row relative to its current position where relative(1) moves row cursor one row forward.

Updating the Result Set

You can update a value in a result set by calling the ResultSet.update<type> method on the row where the cursor is positioned. The type value here is the same value used when retrieving a value from the result set. For example, updateString updates a String value in the result set.

These next code lines update the balance for a user from the result set created earlier. The update applies only to the result set until the call to rs.updateRow(), which updates the underlying database. Closing the result set before calling updateRow will lose any edits applied to the result set.

```
//API Ref :boolean first()
    rs.first();
//API Ref :double updateDouble(String columnName)
//API Ref :double getDouble(String columnName, double d)
    updateDouble("balance", rs.getDouble("balance") - 5.00);
```

Inserting a new row uses the same update<type> methods. The only difference being that the method rs.moveToInsertRow is called before, and rs.insertRow is called after the fields have been initialized. You can delete the current row with a call to rs.deleteRow.

Batch Jobs

By default, every JDBC statement is sent to the database individually. Apart from the additional network requests, this process incurs additional delays if a transaction spans several statements. JDBC 2.0 lets you submit multiple statements at one time with the addBatch method.

This next code segment shows how to use the addBatch statement. The calls to stmt.addBatch append statements to the original Statement, and the call to executeBatch submits the entire statement with all the appends to the database.

```
    Statement stmt = con.createStatement();
    stmt.addBatch("update registration set balance=balance-5.00
           where theuser="+theuser);
    stmt.addBatch("insert into auctionitems(description, startprice)
           values("+description+","+startprice+")");
//API Ref :int executeBatch()
    int[] results = stmt.executeBatch();
```

The addBatch method return result is an array of row counts affected for each statement executed in the batch job. If a problem occurs, a java.sql.BatchUpdateException is thrown. An incomplete array of row counts can be obtained from BatchUpdateException by calling its getUpdateCounts method.

Storing Classes, Images and Other Large Objects

Many databases can store binary data as part of a row if the database field is assigned a long raw, longvarbinary, or other similar type. These fields can accommodate up to two Gigabytes of data. This means if you can convert the data into a binary stream or array of bytes, it can be stored and retrieved from the database in the same way you would store a string or double. This technique can be used to store and retrieve images and Java objects.

Storing and retrieving an image. It is very easy to store an object that can be serialized or converted to a byte array. Unfortunately, java.awt.Image is not Serializable. However, as shown in this next code example, you can store the image data to a file and store the information in the file as bytes in a database binary field.

```
      int itemnumber=400456;
      // Each image is assumed to be in the format <auctionitem>.jpg
      // 400456.jpg
      File file = new File(itemnumber+".jpg");
      FileInputStream fis = new FileInputStream(file);
      PreparedStatement pstmt = con.prepareStatement("update auctionitems
            set theimage=? where id= ?");
      // The FileInputStream fis and the length of that stream are used
      // as parameters to the setBinaryStream method
//API Ref :void setBinaryStream(int index, InputStream stream, int length)
      pstmt.setBinaryStream(1, fis, (int)file.length()):
      pstmt.setInt(2, itemnumber);
//API Ref :int executeUpdate()
      pstmt.executeUpdate();
//API Ref :void close()
      pstmt.close();
      fis.close();
```

To retrieve this image and create a byte array that can be passed to createImage, do the following:

```
      int itemnumber=400456;
      byte[] imageBytes;
      PreparedStatement pstmt = con.prepareStatement(
                                "select theimage from auctionitems where id= ?");

//API Ref :void setInt(int index, int intvalue)
      pstmt.setInt(1, itemnumber);
      ResultSet rs = pstmt.executeQuery();
      if(rs.next()) {
        // Retrieve the image as bytes and convert to an image using the
        // method createImage from the awt Toolkit
//API Ref :byte[] getBytes(String ColumnName)
        imageBytes = rs.getBytes(1);
      }
      pstmt.close();
      rs.close();
      Image auctionimage = Toolkit.getDefaultToolkit().createImage(imageBytes);
```

Storing and retrieving an object. A class can be serialized to a binary database field in much the same way as the image was in the previous example. In this example, the RegistrationImpl class is changed to support default serialization by adding implements Serializable to the Class declaration.

Next, a ByteArrayInputStream is created to be passed as the JDBC Binary Stream. To create the ByteArrayInputStream, RegistrationImpl is first piped through an ObjectOutputStream to an underlying ByteArrayInputStream with a call to RegistrationImpl.writeObject The ByteArrayInputStream is then converted to a byte array, which can then be used to create the ByteArrayInputStream. The create method in RegistrationServer.java is changed as follows:

```
public registration.RegistrationPK create(String theuser, String password,
                                  String emailaddress, String creditcard)
                                  throws registration.CreateException{
   double balance=0;
   Connection con = null;
   PreparedStatement ps = null;;
   try {
     con=getConnection();
     // Create a Registration Bean from the parameters supplied
     RegistrationImpl reg= new RegistrationImpl();
     reg.theuser = theuser;
     reg.password = password;
     reg.emailaddress = emailaddress;
     reg.creditcard = creditcard;
     reg.balance = balance;
     // Getting this Bean into the database requires an input stream,
     // which in this case is a ByteArrayInputStream that is created from
     // a byte array. The byte array is created from a bytearrayoutput
     // stream of the object that is to be stored.
     // Create a ByteArray stream and write the object to that stream
     ByteArrayOutputStream regStore = new ByteArrayOutputStream();
     ObjectOutputStream regObjectStream = new ObjectOutputStream(regStore);
     regObjectStream.writeObject(reg);
     // Create an array of bytes from the bytearray stream
     byte[] regBytes=regStore.toByteArray();
     regObjectStream.close();
     regStore.close();

     // Finally, create the byte array input stream from the array of bytes
     ByteArrayInputStream regArrayStream = new ByteArrayInputStream(regBytes);
     ps=con.prepareStatement("insert into registration (theuser, theclass)
                              values (?, ?)");
     ps.setString(1, theuser);
     ps.setBinaryStream(2, regArrayStream, regBytes.length);
     if(ps.executeUpdate() != 1) {
       throw new CreateException ();
     }
     RegistrationPK primaryKey = new RegistrationPKImpl();
     primaryKey.theuser(theuser);
     return primaryKey;
   } catch (IOException ioe) {
     throw new CreateException ();
   } catch (CreateException ce) {
     throw ce;
   } catch (SQLException sqe) {
     System.out.println("sqe="+sqe);
     throw new CreateException ();
   } finally {
     try {
       ps.close();
       con.close();
```

```
        } catch (Exception ignore) {
        }
      }
    }
```

The object is retrieved and reconstructed by extracting the bytes from the database, creating a ByteArrayInputStream from those bytes to be read from an ObjectInputStream, and calling readObject to create the instance again.

This next example shows the changes needed to the RegistrationServer.refresh method to retrieve the registration instance from the database.

```
private Registration refresh(RegistrationPK pk) throws FinderException {
    if(pk == null) {
        throw new FinderException ();
    }
    ResultSet rs = null;
    Connection con = null;
    PreparedStatement ps = null;
    try {
       con=getConnection();
       ps=con.prepareStatement("select theclass from registration
                               where theuser = ?");
       ps.setString(1, pk.theuser());
       ps.executeQuery();
       rs = ps.getResultSet();
       if(rs.next()){
          byte[] regBytes = rs.getBytes(1);
          ByteArrayInputStream regArrayStream =
              new ByteArrayInputStream(regBytes);
          ObjectInputStream regObjectStream =
              new ObjectInputStream(regArrayStream);
          RegistrationImpl reg=
              (RegistrationImpl)regObjectStream.readObject();
          return reg;
       } else {
         throw new FinderException ();
       }
   } catch (Exception sqe) {
     System.out.println("Exception when loading image: "+sqe);
     throw new FinderException ();
   } finally {
     try {
       rs.close();
       ps.close();
       con.close();
     } catch (Exception ignore) {}
   }
}
```

BLOBs and CLOBs . Storing large fields in a table with the other data is not necessarily optimum especially if the data has a variable size. One way to handle large, variable sized

objects is with the Large Objects (LOBs) type. LOBs use a locator, essentially a pointer, in the database record that points to the real database field.

There are two types of LOBs: Binary Large Objects (BLOBs) and Character Large Objects (CLOBs). When you access a BLOB or CLOB, the data is not copied to the client. To retrieve the actual data from a result set, you have to retrieve the pointer with a call to BLOB blob = getBlob(1) or CLOB clob = getClob(1), and then retrieve the data with a call to blob.getBinaryStream() or clob.getBinaryStream(). Both `getBlob` and `getClob` are in the `java.sql` package.

Controlling Transactions

By default, JDBC statements are processed in full auto-commit mode. This mode works well for a single database query, but if an operation depends on several database statements that all have to complete successfully or the entire operation is cancelled, a finer transaction is needed.

A description of transaction isolation levels is covered in more detail in 3: Data and Transaction Management (page 55). To use transaction management in the JDBC platform, you first need to disable the full auto-commit mode by calling:

```
Connection con= getConnection();
con.setAutoCommit(false);
```

At this point, you can either commit any following JDBC statements or undo any updates by calling the Connection.rollback method. The rollback call is commonly placed in the Exception handler, although it can be placed anywhere in the transaction flow.

This next example inserts an auction item and decrements the user's balance. If the balance is less than zero, the entire transaction is rolled back and the auction item is removed.

```
// New insertItem code to add to file SellerBean.java
static {
   try{
        new pool.JDCConnectionDriver("COM.cloudscape.core.JDBCDriver", "jdbc:clo
udscape:ejbdemo","none", "none");
   } catch(Exception e) {
      System.out.println("new pool error"+e);
   }
}

public Connection getConnection() throws SQLException {
   return DriverManager.getConnection("jdbc:jdc:jdcpool");
}
```

```
    public int insertItem(String seller, String password, String description,
                          int auctiondays, double startprice, String summary) {
        Connection con = null;
        int count=0;
        double balance=0;
        java.sql.Date enddate, startdate;
        Statement stmt=null;
//API Ref :PrepareStatement prepareStatement(String sql)
        PreparedStatement ps = null;

        try {
          con = getConnection();
          // Disable auto commit of jdbc transactions for this connection
//API Ref :void setAutoCommit(boolean autoCommit)
          con.setAutoCommit(false);
//API Ref :Statement createStatement()
          stmt = con.createStatement();
          stmt.executeQuery("select counter from auctionitems");
//API Ref :ResultSet getResultSet()
          ResultSet rs = stmt.getResultSet();
          if(rs.next()) {
            count=rs.getInt(1);
          }
          // Calculate the end date of the auction
//API Ref :static Calendar getInstance()
          Calendar currenttime = Calendar.getInstance();
//API Ref :long getTime()
          java.util.Date currentdate = currenttime.getTime();
          startdate=new java.sql.Date(currentdate.getTime());
//API Ref :void add(int partofdate, int dateamount)
          currenttime.add(Calendar.DATE, auctiondays);
          enddate = new java.sql.Date((currenttime.getTime()).getTime());

          // Insert auction item
          ps = con.prepareStatement("insert into auctionitems(id, description,
                    startdate, enddate, startprice, summary) values (?,?,?,?,?,?)");
          ps.setInt(1, count);
          ps.setString(2, description);
//API Ref :void setDate(int index, Date datavalue)
          ps.setDate(3, startdate);
          ps.setDate(4, enddate);
//API Ref :void setDouble(int index, double doublevalue)
          ps.setDouble(5, startprice);
          ps.setString(6, summary);
          ps.executeUpdate();
          ps.close();

          // Update users balance to deduct a listing fee
//API Ref :PrepareStatement prepareStatement(String sql)
          ps = con.prepareStatement("update registration set balance=balance -0.50
                                    where theuser= ?");
          ps.setString(1, seller);
          executeUpdate(ps);
```

```
          ps.close();
          stmt = con.createStatement();
          stmt.executeQuery("select balance from registration
                             where theuser='"+seller+"'");
//API Ref :ResultSet getResultSet()
          rs = stmt.getResultSet();
          if(rs.next()) {
             balance=rs.getDouble(1);
           }
           stmt.close();
           // Finally, check the users balance, its is less than 0 rollback
           // the whole transaction else update the auction id number and
           // commit everything to the database
           if(balance <0) {
//API Ref :void rollback()
             con.rollback();
             con.close();
             return (-1);
           }
           stmt= con.createStatement();
           // Counter is the next auction item id number that can be
           // used for a auction listing
//API Ref :int executeUpdate(String sql)
           stmt.executeUpdate("update auctionitems set counter=counter+1");
           stmt.close();
           con.commit();
           con.close();
           return(0);
         } catch(SQLException e) {
           try {
             // Always roll the transaction back if something unexpected happens
             con.rollback();
             con.close();
             stmt.close();
             ps.close();
           } catch (Exception ignore){}
         }
         return (0);
      }
```

Escaping Characters

The JDBC API provides the escape keyword so you can specify the character you want to use to escape characters. For example, if you want to use the percent sign (%) as the percent sign and not have it interpreted as the SQL wildcard used in SQL LIKE queries, you have to escape it with the escape character you specify with the escape keyword. This next statement shows how you would use the escape keyword to look for the value 10%.

```
stmt.executeQuery("select tax from sales where tax like '10\%' {escape '\'}");
```

If your program stores names and addresses entered from the command line or by way of a user interface to the database, the single quotes (') symbol might appear in the data. Passing single quotes directly into a SQL string causes problems when the SQL statement is parsed because SQL gives this symbol another meaning unless it is escaped.

To solve this problem, the following method escapes any ' symbol found in the input line. This method can be extended to escape any other characters such as commas that the database or database driver might interpret another way.

```
static public String escapeLine(String s) {
  String retvalue = s;
  if(s.indexOf ("'") != -1 ) {
    StringBuffer hold = new StringBuffer();
    char c;
    for(int i=0; i < s.length(); i++ ) {
      if((c=s.charAt(i)) == '\'' ) {
      hold.append ("''");
    } else {
      hold.append(c);
    }
  }
  retvalue = hold.toString();
  }
  return retvalue;
}
```

However, if you use a PreparedStatement instead of a simple Statement, most of these escape problems go away. For example, instead of this line with the escape sequence:

```
stmt.executeQuery("select tax from sales where tax like '10\%' {escape '\'}");
```

You could use this line:

```
preparedstmt = C.prepareStatement("update tax set tax = ?");
```

Mapping Database Types

Apart from a few JDBC types such as INTEGER that are represented as an INTEGER in most popular databases, you might find that the JDBC type for a table column does not match the type as it is represented in the database. This means calls to ResultSet.getObject, PreparedStatement.setObject and CallableStatement.getObject() will very likely fail.

Your program can determine the database column type from the database meta data and use that information to check the value before retrieving it. This next code checks that the value is in fact type INTEGER before retrieving its value.

```
int count=0;
Connection con=getConnection();
Statement stmt= con.createStatement();
stmt.executeQuery("select counter from auctionitems");
ResultSet rs = stmt.getResultSet();
if(rs.next()) {
  if(rs.getMetaData().getColumnType(1) == Types.INTEGER) {
    Integer i=(Integer)rs.getObject(1);
    count=i.intValue();
  }
}
rs.close();
```

Mapping Date types

The DATE type is where most mismatches occur. This is because the java.util.Date class represents both Date and Time, but SQL has the following three types to represent data and time information:

- A DATE type that represents the date only (03/23/99).
- A TIME type that specifies the time only (12:03:59)
- A TIMESTAMP that represents time value in nanoseconds.

These three additional types are provided in the java.sql package as java.sql.Date, java.sql.Time and java.sql.Timestamp and are all subclasses of java.util.Date. This means you can use convert java.util.Date values to the type you need to be compatible with the database type.

Note: The Timestamp class loses precision when it is converted to a java.util.Date because java.util.Date does not contain a nanosecond field. It is better to not convert a Timestamp instance if the value will be written back to the database.

This example uses the java.sql.Date class to convert the java.util.Date value returned by the call to Calendar.getTime to a java.sql.Date.

```
Calendar currenttime = Calendar.getInstance();
java.sql.Date startdate = new java.sql.Date((currenttime.getTime()).getTime());
```

You can also use the java.text.SimpleDateFormat class to do the conversion. This example uses the java.text.SimpleDateFormat class to convert a java.util.Date object to a java.sql.Date object:

```
SimpleDateFormat template = new SimpleDateFormat("yyyy-MM-dd");
java.util.Date enddate = new java.util.Date("10/31/99");
java.sql.Date sqlDate = java.sql.Date.valueOf(template.format(enddate));
```

If you find a database date representation cannot be mapped to a Java type with a call to getObject or getDate, retrieve the value with a call to getString and format the string as a Date value using the SimpleDateFormat class shown above.

Servlets

A servlet is a server-side program written in the Java programming language that interacts with clients and is usually tied to a HyperText Transfer Protocol (HTTP) server. One common use for a servlet is to extend a web server by providing dynamic web content.

Servlets have an advantage over other technologies such as Perl scripts in that they are compiled, have threading capability built in, and provide a secure programming environment. Even web sites that previously did not provide servlet support can do so now by using programs such as JRun or the Java module for the Apache web server.

The web-based auction application described in Duke's Auction Demonstration (page 7) uses a servlet to accept and process buyer and seller input through the browser and dynamically return auction item information to the browser. The AuctionServlet program is created by extending the HttpServlet class. The HttpServlet class provides a framework for handling HTTP requests and responses.

This section examines the AuctionServlet and includes information on how to use Cookie and Session objects in a servlet.

HttpServlet

The AuctionServlet (page 44) class extends HttpServlet, which is an abstract class.

```
public class AuctionServlet extends HttpServlet {
```

A servlet can be either loaded when the web server starts up or when requested by way of an HTTP URL that specifies the servlet. The servlet is usually loaded by a separate classloader in the web server because this allows the servlet to be reloaded by unloading the class loader that loaded the servlet class. However, if the servlet depends on other classes and one of those classes changes, you will need to update the date stamp on the servlet for it to reload.

After a servlet loads, the first stage in its lifecycle is the web server calls the servlet's init method. Once loaded and initialized, the next stage in the servlet's lifecycle is to serve requests. The servlet serves requests through its service, doGet, or doPost method implementations.

The servlet can optionally implement a destroy method to perform clean-up operations before the web server unloads the servlet.

The init Method

The init method is only called once by the web server when the servlet is first started. The init method is passed a ServletConfig object containing initialization information pertaining to the web server where the application is running.

The ServletConfig object is used to access information maintained by the web server including values from the initArgs parameter in the servlet properties file. Code in the init method uses the ServletConfig object to retrieve the initArgs values by calling the config.getInitParameter("parameter") method.

The AuctionServlet.init method also contacts the Enterprise JavaBeans server to create a context (ctx) object. The ctx object is used in the service method to establish a connection with the Enterprise JavaBeans server.

```
public class AuctionServlet extends HttpServlet {
Context ctx=null;
private String pageTemplate;

public void init(ServletConfig config) throws ServletException{
  super.init(config);
  try {
    ctx = getInitialContext();
  } catch (Exception e){
    System.err.println("failed to contact EJB server"+e);
  }
  try {
    // The pagetemplate parameter contains the name of the file
    // that contains the default HTML page layout.
    // The readFile method is a convenience method in AuctionServlet
    // to read a file as into a string.
    pageTemplate=readFile(config.getInitParameter("pagetemplate"));
  } catch(IOException e) {
    System.err.println("Error in AuctionServlet <init>"+e);
  }
}
```

The destroy Method

The destroy method is a lifecycle method implemented by servlets that need to save their state between servlet loading and unloading. For example, the destroy method would save the current servlet state, and the next time the servlet is loaded, that saved state would be retrieved by the init method. You should be aware that the destroy method might not be called if the server machine crashes.

```
public void destroy() {
  saveServletState();
}
```

The service Method

The AuctionServlet is an HTTP servlet that handles client requests and generates responses through its service method. It accepts as parameters the HttpServletRequest and HttpServletResponse request and response objects.

- HttpServletRequest contains the headers and input streams sent from the client to the server.
- HttpServletResponse is the output stream that is used to send information from the servlet back to the client.

The service method handles standard HTTP client requests received by way of its HttpServletRequest parameter by delegating the request to one of the following methods designed to handle that request. The different types of requests are described in HTTP Requests (page 148).

- doGet for GET, conditional GET, and HEAD requests.
- doPost for POST requests.
- doPut for PUT requests.
- doDelete for DELETE requests.
- doOptions for OPTIONS requests.
- doTrace for TRACE requests.

The AuctionServlet program provides its own service method implementation that calls one of the following methods based on the value returned by the call to cmd = request.getParameter(“action”). These method implementations match the default implementations provided in the doGet and doPost methods called by the default service method, but add some auction application-specific functionality for looking up Enterprise Beans.

- listAllItems(out)
- listAllNewItems(out)
- listClosingItems(out)
- insertItem(out, request)
- itemDetails(out, request)
- itemBid(out, request)
- registerUser(out, request)

```
public void service(HttpServletRequest request, HttpServletResponse response)
                    throws IOException {
  String cmd;
  response.setContentType("text/html");
  ServletOutputStream out = response.getOutputStream();
  if(ctx == null ) {
```

```
      try {
        ctx = getInitialContext();
      } catch (Exception e){
        System.err.println("failed to contact EJB server"+e);
      }
    }

    // The action parameter is part of the posted URL. For example
    // http://phoenix.sun.com/AuctionServlet?action=details&item=4000001
    cmd =request.getParameter("action");
    if(cmd !=null)  {
      if(cmd.equals("list")) {
        listAllItems(out);
      } else if(cmd.equals("newlist")) {
        listAllNewItems(out);
      } else if(cmd.equals("search")) {
        searchItems(out, request);
      } else if(cmd.equals("close")) {
        listClosingItems(out);
      } else if(cmd.equals("insert")) {
        insertItem(out, request);
      } else if (cmd.equals("details")) {
        itemDetails(out, request );
      } else if (cmd.equals("bid")) {
        itemBid(out, request) ;
      } else if (cmd.equals("register")) {
        registerUser(out, request);
      }
    } else{
      // no command set
      setTitle(out, "error");
    }
    setFooter(out);
    out.flush();
  }
```

HTTP Requests

A request is a message sent from a client program such as a browser to a server program. The first line of the request message contains a method that indicates the action to perform on the incoming Uniform Resource Locator (URL). The two commonly used mechanisms for sending information to the server are GET and POST.

- GET requests might pass parameters to a URL by appending them to the URL. GET requests can be bookmarked and emailed and add the information to the URL of the response.
- POST requests might pass additional data to a URL by directly sending it to the server separately from the URL. POST requests cannot be bookmarked or emailed and do not change the URL of the response.

- PUT requests are the reverse of GET requests. Instead of reading the page, PUT requests write (or store) the page.
- DELETE requests are for removing web pages.
- OPTIONS requests are for getting information about the communication options available on the request/response chain.
- TRACE requests are for testing or diagnostic purposes because they let the client see what is being received at the other end of the request chain.

Using Cookies in servlets

HTTP cookies are essentially custom HTTP headers passed between a client and a server. Although cookies are not overwhelmingly popular, they do enable state to be shared between the two machines. For example, when a user logs into a site, a cookie can maintain a reference verifying the user has passed the password check and can use that reference to identify that same user on future visits.

Cookies are normally associated with a server. If you set the domain to .java.sun.com, then the cookie is associated with the domain. If no domain is set, the cookie is only associated with the server that created the cookie.

Setting a Cookie

The Java Servlet API includes a Cookie class that you can use to set or retrieve the cookie from the HTTP header. HTTP cookies include a name and value pair. The `startSession` method shown here is in the Login Servlet (page 200) program. In this method, the name in the name and value pair used to create the Cookie is `JDCAUCTION`, and a unique identifier generated by the server is the value.

```
      protected Session startSession(String theuser, String password,
                                     HttpServletResponse response) {
        Session session = null;
        if( verifyPassword(theuser, password) ) {
          // The user was validated in the user database,
          // create a session
          session = new Session (theuser);
          // Set a timeout for this session that we control
          session.setExpires (sessionTimeout + System.currentTimeMillis());
          sessionCache.put (session);
        // Create a client cookie
//API Ref :Cookie(String name, String value)
          Cookie c = new Cookie("JDCAUCTION", String.valueOf(session.getId()));
//API Ref :void setPath(String path)
          c.setPath ("/");
//API Ref :void setMaxAge(int expire)
          c.setMaxAge (-1);
//API Ref :void setDomain(String domainstring)
```

```
          c.setDomain (domain);
//API Ref :void addCookie(Cookie cookie)
          response.addCookie (c);
        }
        return session;
      }
```

Later versions of the Servlet API include a Session API, to create a session using the Servlet API in the previous example you can use the getSession method.

```
HttpSession session = new Session (true);
```

The startSession method is called by requesting the login action from a POST to the LoginServlet as follows:

```
<FORM ACTION="/LoginServlet" METHOD="POST">
<TABLE>
<INPUT TYPE="HIDDEN" NAME="action" VALUE="login">
<TR><TD>Enter your user id:</TD>
<TD><INPUT TYPE="TEXT" SIZE=20 NAME="theuser"></TD>
</TR>
<TR><TD>Enter your password:<TD>
<TD><INPUT TYPE="PASSWORD" SIZE=20 NAME="password"></TD>
</TR>
</TABLE>
<INPUT TYPE="SUBMIT" VALUE="Login" NAME="Enter">
</FORM>
```

The cookie is created with an maximum age of -1, which means the cookie is not stored but remains alive while the browser runs. The value is set in seconds, although when using values smaller than a few minutes you need to be careful of machine times being slightly out of sync.

The path value can be used to specify that the cookie only applies to files and directories under the path set on that machine. In this example the root path / means the cookie is applicable to all directories.

The domain value in the example is read from the initialization parameters for the servlet. If the domain is null, the cookie is applied to that machines domain only.

Retrieving a Cookie

The cookie is retrieved from the HTTP headers with a call to the getCookies method on the request:

```
//API Ref :Cookie[] getCookies()
   Cookie c[] = request.getCookies();
```

You can later retrieve the name and value pair settings by calling the Cookie.getName method to retrieve the name, and the Cookie.getValue method to retrieve the value. Login-

Servlet has a validateSession method that checks the user's cookies to find a JDCAUCTION cookie that was set in this domain:

```
private Session validateSession(HttpServletRequest request,
                                HttpServletResponse response) {
  // Request the users cookies for this domain
  Cookie c[] = request.getCookies();
  Session session = null;
  if( c != null ) {
    for(int i=0; i < c.length && session == null; i++ ) {
      // Look for a cookie whose name is JDCAUCTION in this
      // domain. This cookie was previously set by us.
      // The cookie value is the session id number stored in the
      // session cache.
      if(c[i].getName().equals("JDCAUCTION")) {
        String key = String.valueOf (c[i].getValue());
        session=sessionCache.get(key);
      }
    }
  }
  return session;
}
```

If you use the Servlet session API, you can use the following method. Note that the parameter is false to specify that the session value is returned and a new session not created.

```
HttpSession session = request.getSession(false);
```

Generating Sessions

The LoginServlet.validateSession method returns a Session object represented by the Session (Servlets) (page 204) class. The Session class uses an identifier generated from a numeric sequence. This numbered session identifier is the value part of the name and value pair stored in the cookie.

The only way to reference the user name on the server is with this session identifier, which is stored in a simple memory cache with the other session IDs. When a user terminates a session, the LoginServlet logout action is called like this:

```
http://phoenix.sun.com:7001/LoginServlet?action=logout
```

The session cache implemented in the SessionCache (Servlets) (page 205) program includes a reaper thread to remove sessions older than a preset time. The preset timeout could be measured in hours or days depending on how many visitors visit the site.

Preventing Page Caching

The LoginServlet.setNoCache method sets the Cache-Control or Pragma values (depending on which version of the HTTP protocol is being used) in the response header to no-cache.

The Expires expiration header is also set to 0. Alternately, you can set the time to be the current system time. Even if the client does not cache the page, there are often proxy servers in a corporate network that would. Only pages using Secure Socket Layer (SSL) are not cached by default.

```
    private void setNoCache (HttpServletRequest request,
                             HttpServletResponse response) {
//API Ref :String getProtocol()
      if(request.getProtocol().compareTo ("HTTP/1.0") == 0) {
//API Ref :void setHeader(String name, String value)
        response.setHeader ("Pragma", "no-cache");
      } else if (request.getProtocol().compareTo("HTTP/1.1") == 0) {
        response.setHeader ("Cache-Control", "no-cache");
      }
//API Ref :void setDateHeader(String name, long datevalue)
      response.setDateHeader ("Expires", 0);
    }
```

Restricting Access and Redirections

If you install the Login Servlet (page 200) as the default servlet or servlet to run when serving any page under the document root, you can use cookies to restrict users to certain sections of the site. For example, you can allow users who have cookies that state they have logged in to access sections of the site that require a login password and keep all others out.

The Login servlet checks for a restricted directory in its in it method. The init method shown below sets the protectedDir variable to true if the config variable passed to it specifies a protected directory. The web server configuration file provides the settings passed to a servlet in the config variable.

```
    public void init(ServletConfig config) throws ServletException  {
      super.init(config);
//API Ref :String getInitParameter(String parametername)
      domain = config.getInitParameter("domain");
      restricted = config.getInitParameter("restricted");
      if(restricted != null) {
        protectedDir=true;
      }
    // ..
```

Later on in the validateSession and service methods, the protectedDir variable is checked and the HttpResponse.sendRedirect method is called to send the user to the correct page based on their login and session status.

```
//API Ref :void sendRedirect(String urlstring)
    if(protectedDir) {
        response.sendRedirect (restricted+"/index.html");
      } else{
        response.sendRedirect (defaultPage);
      }
```

The init method also retrieves the servlet context for the FileServlet servlet so methods can be called on the FileServlet in the validateSession method. The advantage to calling methods on FileServlet to serve the files rather than serving the files from within the LoginServlet, is you get the full advantage of all the functionality added into FileServlet such as memory mapping or file caching. The downside is that the code may not be portable to other servers that do not have FileServlet. This code retrieves the FileServlet context.

```
    FileServlet fileServlet = (FileServlet)
//API Ref :ServletContext getServletContext()
                              config.getServletContext().getServlet("file");
```

The validateSession method prevents users without a logon session from accessing the restricted directory.

HTTP Error Codes

You can return an HTTP error code using the sendError method. For example, two common HTTP error codes are error 500 which indicates an internal server error, and the 404 error code indicates page not found.

This next code segment returns the HTTP 500 error code.

```
protected void service(HttpServletRequest request,HttpServletResponse response)
                        throws ServletException {
  response.sendError (500);
}
```

Reading GET and POST Values

The Servlet API has a getParameter method in the HttpServletRequest class that returns the GET or POST value for the name you supply.

- The HTTP GET request handles name and value pairs as part of the URL. The getParameter method parses the URL passed in, retrieves the name=value pairs delimited by the ampersand (&) character, and returns the value.
- The HTTP POST request reads the name and value pairs from the input stream from the client. The getParameter method parses the input stream for the name and value pairs.

The getParameter method works well for simple servlets, but if you need to retrieve the POST parameters in the order they were placed on the web page or handle multi-part posts, you can write your own code to parse the input stream.

The next example returns POST parameters in the order they were received from the web page. Normally, the parameters are stored in a Hashtable that does not maintain the sequence

order of elements stored in it. The example keeps a reference to each name and value pair in a vector that can be traversed to return the values in the order they were received by the server.

```
    // File auction.PostServlet.java
    package auction;

    import java.io.*;
    import java.util.*;
    import javax.servlet.*;
    import javax.servlet.http.*;

    public class PostServlet extends HttpServlet {

      public void init(ServletConfig config) throws ServletException  {
        super.init(config);
      }

      public void service(HttpServletRequest request, HttpServletResponse response)
                          throws IOException {
        private Vector paramOrder;
        private Hashtable parameters;
//API Ref :  void setContentType(String type)
        response.setContentType("text/html");
//API Ref :PrintWriter getWriter()
        PrintWriter out = response.getWriter();
        if(request.getMethod().equals("POST") && request.getContentType().equals(
                           "application/x-www-form-urlencoded")) {
          // If this is a POST request and the content type from the post is
          // a url encoded form, build a hash table with the extracted parameter
          // values and generate an associated vector that just records the
          // name of the parameters in ascending order
          //
          parameters = parsePostData(request.getContentLength(),
                       request.getInputStream(), paramOrder);
        }
        // Parse through the list of posted parameters by getting the name
        // of each parameter from the vector and extracting the value
        // of that parameter by calling getParameter on the Hashtable of
        // value pairs
        for(int i=0;i<paramOrder.size();i++) {
          String name = (String)paramOrder.elementAt(i);
          String value = getParameter((String)paramOrder.elementAt(i), parameters);
          out.println("name="+name+" value="+value);
        }
        out.println("</body></html>");
        out.close();
      }

      private Hashtable parsePostData(int length, ServletInputStream instream,
    Vector paramOrder) {
        String valArray[] = null;
        int inputLen, offset;
```

```
byte[] postedBytes = null;
boolean dataRemaining=true;
String postedBody;
Hashtable ht = new Hashtable();
paramOrder= new Vector(10);
StringBuffer sb = new StringBuffer();
if(length <=0) {
  return null;
}
postedBytes = new byte[length];
// Read the posted form into a byte array

try {
  offset = 0;
  while(dataRemaining) {
    inputLen = instream.read (postedBytes, offset, length - offset);
    if(inputLen <= 0) {
      throw new IOException ("read error");
    }
    offset += inputLen;
    if((length-offset) ==0) {
      dataRemaining=false;
    }
  }
} catch (IOException e) {
  System.out.println("Exception ="+e);
  return null;
}
// Create a string from the byte array and parse the name/value
// pairs.
postedBody = new String (postedBytes);
StringTokenizer st = new StringTokenizer(postedBody, "&");
String key=null;
String val=null;
while (st.hasMoreTokens()) {
  String pair = (String)st.nextToken();
  int pos = pair.indexOf('=');
  if(pos == -1) {
    throw new IllegalArgumentException();
  }
  try {
    // Spaces and other http sensitive characters are url encoded
    // before posting so convert the posted string into its original
    // form
    key = java.net.URLDecoder.decode(pair.substring(0, pos));
    val = java.net.URLDecoder.decode(pair.substring(pos+1, pair.length()));
  } catch (Exception e) {
    throw new IllegalArgumentException();
  }
  if(ht.containsKey(key)) {
    String oldVals[] = (String []) ht.get(key);
    valArray = new String[oldVals.length + 1];
    for(int i = 0; i < oldVals.length; i++) {
```

```
            valArray[i] = oldVals[i];
          }
          valArray[oldVals.length] = val;
        } else {
          valArray = new String[1];
          valArray[0] = val;
        }
        ht.put(key, valArray);
        paramOrder.addElement(key);
      }
      return ht;
    }

    public String getParameter(String name, HashTable parameters) {
      String vals[] = (String []) parameters.get(name);
      if(vals == null) {
        return null;
      }
      String vallist = vals[0];
      for(int i = 1; i < vals.length; i++) {
        vallist = vallist + "," + vals[i];
      }
      return vallist;
    }
  }
```

To find out whether the request is POST or GET, call the getMethod in the HttpServletRequest class. To determine the format of the data being posted, call the getContentType method in the HttpServletRequest class. For simple HTML web pages, the type returned by this call will be application/x-www-form-urlencoded.

If you need to create a post with more than one part such as the one created by the following HTML form, the servlet will need to read the input stream from the post to reach individual sections. Each section is distinguished by a boundary defined in the post header.

```
<FORM ACTION="/PostMultiServlet" METHOD="POST" ENCTYPE="multipart/form-data">
<INPUT TYPE="TEXT" NAME="desc" value="">
<INPUT TYPE="FILE" NAME="filecontents" value="">
<INPUT TYPE="SUBMIT" VALUE="Submit" NAME="Submit">
</FORM>
```

The next example extracts a description and a file from the client browsers. It reads the input stream looking for a line matching the boundary string, reads the content line, skips a line and then reads the data associated with that part. The uploaded file is simply displayed, but could also be written to disk.

```
// File auction.PostMultiServlet.java
package auction;

import java.io.*;
import java.util.*;
```

```
import javax.servlet.*;
import javax.servlet.http.*;

public class PostMultiServlet extends HttpServlet {
   public void init(ServletConfig config) throws ServletException  {
     super.init(config);
   }

   public void service(HttpServletRequest request, HttpServletResponse response)
                       throws ServletException, IOException {
     response.setContentType("text/html");
     PrintWriter out = response.getWriter();
     if(request.getMethod().equals("POST") &&
                  request.getContentType().startsWith("multipart/form-data")) {
       int index = request.getContentType().indexOf("boundary=");
       if(index < 0) {
         System.out.println("can't find boundary type");
         return;
       }
       // Look for the data past the identifying string boundary=
       String boundary = request.getContentType().substring(index+9);
       ServletInputStream instream = request.getInputStream();
       byte[] tmpbuffer = new byte[8192];
       int length=0;
       String inputLine=null;
       boolean moreData=true;
//Skip http header fields until form data is reached
       length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
       inputLine = new String (tmpbuffer, 0, 0, length);
       while(inputLine.indexOf(boundary)  >0 && moreData) {
         length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
         inputLine = new String (tmpbuffer, 0, 0, length);
         if(inputLine !=null)
           System.out.println("input name="+inputLine);
           if(length<0) {
             moreData=false;
           }
         }
         if(moreData) {
           length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
           inputLine = new String (tmpbuffer, 0, 0, length);
           if(inputLine.indexOf("desc") >=0) {
             length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
             inputLine = new String (tmpbuffer, 0, 0, length);
             length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
             inputLine = new String (tmpbuffer, 0, 0, length);
             System.out.println("description ="+inputLine);
         }
     }
         while(inputLine.indexOf(boundary)  >0 && moreData) {
           length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
           inputLine = new String (tmpbuffer, 0, 0, length);
         }
```

```
            if(moreData) {
              length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);
              inputLine = new String (tmpbuffer, 0, 0, length);
              if(inputLine.indexOf("filename") >=0) {
                // We know what the file name is now
                int startindex=inputLine.indexOf("filename");
                System.out.println("file name="+ inputLine.substring(startindex+10,
                                    inputLine.indexOf("\"", startindex+10)));
                length = instream.readLine(tmpbuffer, 0, tmpbuffer.length);

                inputLine = new String (tmpbuffer, 0, 0, length);
              }
            }
            byte fileBytes[]=new byte[50000];
            int offset=0;
            if(moreData) {
              // Read the contents of the posted file into fileBytes
              while(inputLine.indexOf(boundary) >0 && moreData) {
                length = instream.readLine(tmpbuffer,0, tmpbuffer.length);
                inputLine = new String (tmpbuffer, 0, 0, length);
                if(length >0 && (inputLine.indexOf(boundary) <0)) {
                  System.arraycopy(tmpbuffer, 0, fileBytes, offset, length);
                  offset+=length;
                } else {
                 moreData=false;
                }
              }
            }
    // Trim last two newline/return characters before using data
            for(int i=0;i<offset-2;i++) {
              System.out.print((char)fileBytes[i]);
            }
        }
        out.println("</body></html>");
        out.close();
      }
    }
```

Threading

A servlet must be able to handle multiple concurrent requests. Any number of end users at any given time could invoke the servlet, and while the init method is always run single-threaded, the service method is multi-threaded to handle multiple requests.

This means any fields accessed by the service method should be restricted to simple thread access. The example below uses the synchronized keyword to restrict access to a counter so it can only be updated by one thread at a time:

```
static int counter;
Boolean lock = new Boolean(true);
synchronized(lock){
  counter++;
}
```

HTTPS

Many servers, browsers, and the Java Plug-In can support the secure HTTP protocol called HTTPS. HTTPS is similar to HTTP except the data is transmitted over a secure socket layer (SSL) instead of a normal socket connection. Web servers often listen for HTTP requests on one port while listening for HTTPS requests on another.

SSL has checks to find out if encrypted data sent over the network has been tampered with in transit. SSL also authenticates the web server to its clients by providing a public key certificate. In SSL 3.0 the client can also authenticate itself with the server using a public key certificate.

Public key cryptography (also called asymmetric key encryption) uses a public and private key pair. Any message encrypted with the private key in the pair can only be decrypted with the corresponding public key. Certificates are digitally signed statements generated from a trusted third party Certificate Authority. The Certificate Authority needs proof that you are who you say you are because clients will be trusting the certificate they receive.

It is this certificate that contains the public key in the public and private key pair. The certificate is signed by the private key of the Certificate Authority, and most browsers know the public key for the main Certificate Authorities.

While public key encryption is good for authentication purposes, it is not as fast as symmetric key encryption and so the SSL protocol uses both types of keys in the lifecycle of an SSL connection. The client and server begin an HTTPS transaction with a connection initialization or handshaking phase.

It is in the handshaking stage that the server is authenticated using the certificate that the client has received. The client uses the server's public key to encrypt messages sent to the server. After the client has been authenticated and the encryption algorithm or cipher has been agreed between the two parties, new symmetric session keys are used to encrypt and decrypt any further communication.

The encryption algorithm or cipher can be one of many popular algorithms like Rivest Shamir and Adleman (RSA) or Data Encryption Standard (DES). The greater the number of bits used to make the key, the more difficult it is to break into using brute force search techniques.

HTTPS using public key cryptography and certificates lets you provide the amount of privacy your application needs for safe and secure transactions. Servers, browsers, and Java Plug-In have their own setup for enabling HTTPS using SSL communications. In general, the steps involve the following:

- Get a private key and a digitally-signed certificate with the matching public key.
- Install the certificate in a location specified by the software you are using (server, browser, or Java Plug-In).
- Enable SSL features and specify your certificate and private key files as instructed in your documentation.

You should enable SSL features according to your specific application requirements depending on the level of security you need. For example, you do not need to verify the identity of customers browsing auction items, but you will want to encrypt credit card and other personal information supplied when buyers and sellers register to participate.

HTTPS can be used for any data not just HTTP web pages. Programs written in the Java programming language can be downloaded over an HTTPS connection, and you can open a connection to a HTTPS server in the Java Plug-in. To write a program in the Java programming language that uses SSL, you need an SSL library and a detailed knowledge of the HTTPS handshaking process. Your SSL library should provide all the functionality you need because this information is restricted by export security control.

Code for this Chapter

- RegistrationServer (lookup) (page 161)
- SellerBean (lookup) (page 163)
- RegistrationServer (RMI) (page 165)
- Registration (RMI) (page 169)
- RegistrationHome (RMI) (page 169)
- RegistrationPK (RMI) (page 170)
- SellerBean (RMI) (page 170)
- AuctionServlet (RMI) (page 173)
- RegistrationServer (CORBA) (page 181)
- Registration.idl (CORBA) (page 187)
- SellerBean (CORBA) (page 187)
- Seller (CORBA) (page 191)
- AuctionServlet (CORBA) (page 191)

- Login Servlet (page 200)
- Session (Servlets) (page 204)
- SessionCache (Servlets) (page 205)

RegistrationServer (lookup)

```
package registration;

import java.sql.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class RegistrationServer extends _RegistrationHomeImplBase{
  static {
    new pool.JDCConnectionDriver("postgresql.Driver",
            "jdbc:postgresql:ejbdemo","postgres", "pass");
  }
  public Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:jdc:jdcpool");
  }
  public registration.RegistrationPK create(String theuser, String password,
                String emailaddress, String creditcard)
                throws registration.CreateException{
    double balance=0;
    Connection con = null;
    PreparedStatement ps = null;
    try {
      con=getConnection();
      ps=con.prepareStatement("insert into registration(theuser, password,
            emailaddress, creditcard, balance) values (?, ?, ?, ?, ?)");
      ps.setString(1, theuser);
      ps.setString(2, password);
      ps.setString(3, emailaddress);
      ps.setString(4, creditcard);
      ps.setDouble(5, balance);
      if(ps.executeUpdate() != 1) { //JDBC did not create a row
        throw new CreateException ();
      }
      RegistrationPK primaryKey = new RegistrationPKImpl();
      primaryKey.theuser(theuser);
      return primaryKey;
    } catch (CreateException ce) {
      throw ce;
    } catch (SQLException sqe) {
      throw new CreateException ();
    } finally {
      try {
        ps.close();
        con.close();
      } catch (Exception ignore) {
    }
```

```
}
public registration.Registration findByPrimaryKey(
                   registration.RegistrationPK pk)
                  throws registration.FinderException {
  //Primary key cannot be nul
  if((pk == null) || (pk.getUser() == null)) {l
    throw new FinderException ();
  }
  return(refresh(pk));
}
private Registration refresh(RegistrationPK pk) throws FinderException {
  if(pk == null) {//Primary key cannot be null
    throw new FinderException ();
  }
  Connection con = null;
 PreparedStatement ps = null;
 try {
   con=getConnection();
   ps=con.prepareStatement("select password, emailaddress, creditcard,
          balance from registration where theuser = ?");
   ps.setString(1, pk.getUser());
   ps.executeQuery();
   ResultSet rs = ps.getResultSet();
   if (rs.next()) {
       RegistrationImpl reg= new RegistrationImpl();
       reg.theuser = pk.getUser();
       reg.password = rs.getString(1);
       reg.emailaddress = rs.getString(2);
       reg.creditcard = rs.getString(3);
       reg.balance = rs.getDouble(4);
       return reg;
    } else {
      throw new FinderException ();
    }
  } catch (SQLException sqe) {
     throw new FinderException();
  } finally {
    try {
      ps.close();
      con.close();
    } catch (Exception ignore) {}
  }
  return null;
}
public static void main(String args[]) {
  String[] orbargs = { "-ORBInitialPort 1050"};
  ORB orb = ORB.init(orbargs, null) ;
  RegistrationServer rs= new RegistrationServer();
  try {
    org.omg.CORBA.Object nameServiceObj =
                orb.resolve_initial_references("NameService") ;
    NamingContext nctx= NamingContextHelper.narrow(nameServiceObj);
    NameComponent[] fullname = new NameComponent[2];
```

```
        fullname[0] = new NameComponent("auction", "");
        fullname[1] = new NameComponent("RegistrationBean", "");
        NameComponent[] tempComponent = new NameComponent[1];
        for(int i=0; i < fullname.length-1; i++ ) {
           tempComponent[0]= fullname[i];
           try {
             nctx=nctx.bind_new_context(tempComponent);
           }catch (Exception e){}
        }
        tempComponent[0]=fullname[fullname.length-1];
        nctx.bind(tempComponent, rs);
        java.lang.Object sync= new java.lang.Object();
        synchronized(sync) {
          sync.wait();
        } catch (Exception e) {
          System.out.println("Exception occured="+e);
        }
      }
    }
  class RegistrationImpl extends _RegistrationImplBase {
    public String theuser, password, creditcard, emailaddress;
    public double balance;
    public boolean verifyPassword(String password) {
      if(this.password.equals(password)) {
        return true;
      } else {
        return false;
      }
    }
    public String getEmailAddress() {
      return emailaddress;
    }
    public String getUser() {
      return theuser;
    }
    public int adjustAccount(double amount) {
      balance=balance+amount;
      return(0);
    }
    public double getBalance() {
      return balance;
    }
  }
```

SellerBean (lookup)

```
package seller;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
```

```
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class SellerBean implements SessionBean {
  protected SessionContext ctx;
  javax.naming.Context ectx;
  Hashtable env = new Hashtable();
  ORB orb = null;
  NamingContext nctx= null;

  public int insertItem(String seller, String password, String description,
                        int auctiondays, double startprice, String summary)
                       throws RemoteException {
    try {
      NameComponent[] fullname = new NameComponent[2];
      fullname[0] = new NameComponent("auction", "");
      fullname[1] = new NameComponent("RegistrationBean", "");
      RegistrationHome regRef =
                     RegistrationHomeHelper.narrow(nctx.resolve(fullname));
      RegistrationPKImpl rpk= new RegistrationPKImpl();
      rpk.theuser(seller);
      Registration newseller =
                     RegistrationHelper.narrow(regRef.findByPrimaryKey(rpk));
      if((newseller == null)|| (!newseller.verifyPassword(password))) {
          return(Auction.INVALID_USER);
      }
      AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
      AuctionItem ai= home.create(seller, description, auctiondays,
                                  startprice, summary);
      if(ai == null) {
        return Auction.INVALID_ITEM;
      } else {
        return(ai.getId());
      }
    } catch(Exception e){
      System.out.println("insert problem="+e);
      return Auction.INVALID_ITEM;
    }
  }
  public void ejbCreate() throws javax.ejb.CreateException, RemoteException {
    env.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
          "weblogic.jndi.TengahInitialContextFactory");
    try {
      ectx = new InitialContext(env);
    } catch (NamingException e) {
      System.out.println("problem contacting EJB server");
      throw new javax.ejb.CreateException();
    }
    try {
```

```
      String[] args = { "-ORBInitialPort 1050"};
      orb = ORB.init(args, null) ;
      org.omg.CORBA.Object nameServiceObj =
                          orb.resolve_initial_references("NameService") ;
      nctx= NamingContextHelper.narrow(nameServiceObj);
    } catch(org.omg.CORBA.SystemException e) {
      throw new javax.ejb.CreateException("CORBA Error in SellerCreate "+e);
    } catch ( org.omg.CORBA.ORBPackage.InvalidName e) {
      throw new javax.ejb.CreateException("CORBA NameError in SellerCreate " + e);
    }
  }
  public void setSessionContext(SessionContext ctx) throws RemoteException {
    this.ctx = ctx;
  }
  public void unsetSessionContext() throws RemoteException {
    ctx = null;
  }
  public void ejbRemove() {}
  public void ejbActivate() throws RemoteException {
    System.out.println("activating seller bean");
  }
  public void ejbPassivate() throws RemoteException {
    System.out.println("passivating seller bean");
  }
}
```

RegistrationServer (RMI)

```
package registration;

import java.sql.*;
import java.rmi.*;
import java.rmi.server.*;
import java.util.Properties;
import java.util.*;

public class RegistrationServer extends UnicastRemoteObject
             implements RegistrationHome {
  RegistrationServer() throws RemoteException {
    super();
  }
  public Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:jdc:jdcpool");
  }
  public registration.RegistrationPK create(String theuser, String password,
                                    String emailaddress, String creditcard)
                                    throws registration.CreateException{
    double balance=0;
    Connection con = null;
    PreparedStatement ps = null;
    try {
      con = getConnection();
```

```
    ps = con.prepareStatement("insert into registration(theuser, password,
             emailaddress,creditcard, balance) values (?, ?, ?, ?, ?)");
    ps.setString(1, theuser);
    ps.setString(2, password);
    ps.setString(3, emailaddress);
    ps.setString(4, creditcard);
    ps.setDouble(5, balance);
    if(ps.executeUpdate() != 1) {
      throw new CreateException ();
    }
    RegistrationPK primaryKey = new RegistrationPK();
    primaryKey.setUser(theuser);
    return primaryKey;
  } catch (CreateException ce) {
    throw ce;
  } catch (SQLException sqe) {
    throw new CreateException ();
  } finally {
    try {
      ps.close();
      con.close();
    } catch (Exception ignore) {}
  }
}
public registration.Registration findByPrimaryKey(
                                  registration.RegistrationPK pk)
                                     throws registration.FinderException {
  // Return a Registration object that is created from reading
  // values from the database by calling refresh()
  if ((pk == null) || (pk.getUser() == null)) {
      throw new FinderException ();
  }
  return(refresh(pk));
}

private Registration refresh(RegistrationPK pk) throws FinderException {
  // Read the appropriate registration fields from the database
  // based on the database item that the primary key (pk) contains
  if(pk == null) {
    throw new FinderException ();
  }
  Connection con = null;
  PreparedStatement ps = null;
  try {
    con=getConnection();
    ps=con.prepareStatement("select password, emailaddress, creditcard,
                            balance from registration where theuser = ?");
    ps.setString(1, pk.getUser());
    ps.executeQuery();
    ResultSet rs = ps.getResultSet();
    if(rs.next()) {
      RegistrationImpl reg=null;
      try{
```

```
        reg= new RegistrationImpl();
      }catch (RemoteException e) {}
        reg.theuser = pk.getUser();
        reg.password = rs.getString(1);
        reg.emailaddress = rs.getString(2);
        reg.creditcard = rs.getString(3);
        reg.balance = rs.getDouble(4);
        return reg;
    } else{
      throw new FinderException ();
    }
  } catch (SQLException sqe) {
        throw new FinderException();
  } finally {
    try {
      ps.close();
      con.close();
    } catch (Exception ignore) {}
    }
}

public void findLowCreditAccounts(
             final ReturnResults client) throws FinderException {
  // This method finds accounts with less than 3 dollars in
  // a US domain. It then creates a RegistrationImpl object and
  // calls a callback method called updateResults from the
  // ReturnResults class. It runs as a thread so that the client
  // does not have to block to wait for the results to return
  Runnable bgthread = new Runnable() {
    public void run() {
      Connection con = null;
      ResultSet rs = null;
      PreparedStatement ps = null;
      ArrayList ar = new ArrayList();
      try {
        con=getConnection();
        ps=con.prepareStatement("select theuser,
                 balance from registration where balance < ?");
        ps.setDouble(1, 3.00);
        ps.executeQuery();
        rs = ps.getResultSet();
        RegistrationImpl reg=null;
        while (rs.next()) {
          try {
            reg= new RegistrationImpl();
          } catch (RemoteException e) {}
            reg.theuser = rs.getString(1);
            reg.balance = rs.getDouble(2);
            ar.add(reg);
          }
        rs.close();
        client.updateResults(ar);
        } catch (Exception e) {
```

```
            System.out.println("findLowCreditAccounts: "+e);
            return;
          }
          finally {
            try {
              if(rs != null) {
                rs.close();
              }
              if(ps != null) {
                ps.close();
                       }
              if(con != null) {
                con.close();
              }
            }catch (Exception ignore) {}
        }
      } //run
    };
    Thread t = new Thread(bgthread);
    t.start();
  }


  public static void main(String[] args){
    try {
      new pool.JDCConnectionDriver("postgresql.Driver",
                           "jdbc:postgresql:ejbdemo","postgres", "pass");
    } catch (Exception e){
      System.out.println("error in loading JDBC driver");
      System.exit(1);
    }
    try {
      Properties env=System.getProperties();
      env.put("java.rmi.server.codebase",
                          "http://phoenix.sun.com/registration");
      RegistrationServer rs= new RegistrationServer();
     if(System.getSecurityManager() == null )  {
        System.setSecurityManager(new RMISecurityManager());
      }
      RMISocketFactory.setFailureHandler(new RMIFailureHandlerImpl());
      Naming.rebind("//phoenix.sun.com/registration2",rs);
    } catch (Exception e) {
      System.out.println("Exception thrown "+e);
    }
  } // End of Main

class RMIFailureHandlerImpl implements RMIFailureHandler {
  public boolean failure(Exception ex ){
    System.out.println("exception "+ex+" caught");
    return true;
  }
}
```

```
class RegistrationImpl extends UnicastRemoteObject implements Registration {
   public String theuser, password, creditcard, emailaddress;
   public double balance;

   RegistrationImpl() throws RemoteException {
     super();
   }
   public boolean verifyPassword(String password) {
     if(this.password.equals(password)) {
       return true;
      } else {
        return false;
      }
    }
    public String getEmailAddress() {
      return emailaddress;
    }
    public String getUser() {
      return theuser;
    }
    public int adjustAccount(double amount) {
      balance=balance+amount;
      return(0);
    }
    public double getBalance() {
      return balance;
    }
}
```

Registration (RMI)

```
package registration;

import java.rmi.*;
import java.util.*;

public interface Registration extends Remote {
   boolean verifyPassword(String password) throws RemoteException;
   String getEmailAddress() throws RemoteException;
   String getUser() throws RemoteException;
   int adjustAccount(double amount) throws RemoteException;
   double getBalance() throws RemoteException;
}
```

RegistrationHome (RMI)

```
package registration;

import java.rmi.*;
import java.util.*;
```

```
public interface RegistrationHome extends Remote {
  RegistrationPK create(String theuser, String password,
                        String emailaddress, String creditcard)
                        throws CreateException, RemoteException;
   Registration findByPrimaryKey(RegistrationPK theuser)
                        throws FinderException, RemoteException;
   public void findLowCreditAccounts(ReturnResults rr)
                        throws FinderException, RemoteException;
}
```

RegistrationPK (RMI)

```
package registration;

public class RegistrationPK implements java.io.Serializable {
  public String theuser;
  public RegistrationPK() {}
  public String getUser() {
    return theuser;
  }
  public void setUser(String user) {
    theuser=user;
  }
}
```

SellerBean (RMI)

```
package seller;

import java.rmi.RemoteException;
import java.rmi.*;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;
import java.rmi.server.UnicastRemoteObject;
import java.util.ArrayList;

public class SellerBean implements SessionBean, ReturnResults {
  protected SessionContext ctx;
  javax.naming.Context ectx;
  Hashtable env = new Hashtable();
  Boolean ready=new Boolean("false");
  ArrayList returned;
```

```
public int insertItem(String seller, String password, String description,
                      int auctiondays, double startprice, String summary)
                      throws RemoteException {
  // Insert an item into the Auction items for sale. First,
  // the seller details are verified with the Registration Database
  // before calling the AuctionItem Bean.
  try {
    RegistrationHome regRef = (RegistrationHome)Naming.lookup(
                              "//phoenix.sun.com/registration2");
    RegistrationPK rpk= new RegistrationPK();
    rpk.setUser(seller);
    Registration newseller = (
    Registration)regRef.findByPrimaryKey(rpk);
    if ((newseller == null) || (!newseller.verifyPassword(password))) {
      return(Auction.INVALID_USER);
    }
    AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
    AuctionItem ai = home.create(seller, description, auctiondays,
                                 startprice, summary);
    if (ai == null) {
       return Auction.INVALID_ITEM;
     } else{
       return(ai.getId());
     }
    } catch(Exception e){
      System.out.println("insert problem="+e);
      return Auction.INVALID_ITEM;
    }
  }
  public void updateResults(java.util.ArrayList ar) throws RemoteException {
    // method called from remote rmi client as a callback
    returned=ar;
    synchronized(ready) {
      ready.notifyAll();
    }
  }

  public ArrayList auditAccounts() {
    // Call the findLowCreditAccounts method from the RegistrationServer
    // and wait for the results to be sent via the updateResults method
    try {
      RegistrationHome regRef = (RegistrationHome)Naming.lookup(
                            "//phoenix.eng.sun.com/registration2");
    regRef.findLowCreditAccounts(this);
    synchronized(ready) {
      try {
        ready.wait();
      } catch (InterruptedException e){}
    }
    return (returned);
  } catch (Exception e) {
```

```
      System.out.println("error in creditAudit "+e);
    }
    return null;
  }

  public void ejbCreate() throws javax.ejb.CreateException,
                                                RemoteException {
      // Initialize the Bean by creating the RMI connection
      // and exporting this object to the rmi registry.
      env.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
              "weblogic.jndi.TengahInitialContextFactory");
      try {
        ectx = new InitialContext(env);
      } catch (NamingException e) {
        System.out.println("problem contacting EJB server");
        throw new javax.ejb.CreateException();
      }
      Properties env=System.getProperties();
      env.put("java.rmi.server.codebase",
              "http://phoenix.sun.com/registration");
      env.put("java.security.policy","java.policy");
      UnicastRemoteObject.exportObject(this);
   }
   public void setSessionContext(SessionContext ctx) throws RemoteException {
     this.ctx = ctx;
   }
   public void unsetSessionContext() throws RemoteException {
     ctx = null;
   }
   public void ejbRemove() {}
   public void ejbActivate() throws RemoteException {
     System.out.println("activating seller bean");
   }
   public void ejbPassivate() throws RemoteException {
     System.out.println("passivating seller bean");
   }
}
```

ReturnResults (RMI)

```
package registration;

import java.rmi.*;
import java.util.*;

public interface ReturnResults extends Remote {
  public void updateResults(ArrayList results)
    throws FinderException, RemoteException;
}
```

AuctionServlet (RMI)

```
package auction;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.naming.*;
import javax.ejb.*;
import java.rmi.RemoteException;
import java.util.*;
import java.text.NumberFormat;
import bidder.*;
import registration.*;
import seller.*;
import pool.*;
import search.*;

public class AuctionServlet extends HttpServlet {
  javax.naming.Context ctx=null;
  public void init(ServletConfig config) throws ServletException{
    super.init(config);
    try {
      ctx = getInitialContext();
    } catch (Exception e){
      System.err.println("failed to contact EJB server"+e);
    }
      Properties env=System.getProperties();
      env.put("java.rmi.server.codebase", "file:/root/book/");
      env.put("java.security.policy","/root/book/policy");
  }
  public void service(HttpServletRequest request, HttpServletResponse response)
                      throws IOException {
    String cmd;
    response.setContentType("text/html");
    ServletOutputStream out = response.getOutputStream();
    if (ctx == null ) {
      try {
        ctx = getInitialContext();
      } catch (Exception e){
        System.err.println("failed to contact EJB server"+e);
      }
    }
    cmd=request.getParameter("action");
    if(cmd !=null)  {
      if(cmd.equals("list")) {
        listAllItems(out);
      } else if(cmd.equals("newlist")) {
        listAllNewItems(out);
      } else if(cmd.equals("search")) {
        searchItems(out, request);
      } else if(cmd.equals("close")) {
        listClosingItems(out);
```

```
    } else if(cmd.equals("insert")) {
      insertItem(out, request);
    } else if (cmd.equals("details")) {
      itemDetails(out, request );
    } else if (cmd.equals("bid")) {
      itemBid(out, request) ;
    } else if (cmd.equals("auditAccounts")) {
      auditAccounts(out, request);
    } else if (cmd.equals("register")) {
      registerUser(out, request);
    }
  } else {// no command set
    setTitle(out, "error");
  }
  setFooter(out);
  out.flush();
}
static private void addLine(String message, ServletOutputStream out)
                            throws IOException {
  if(message !=null) {
    out.println("<BR>"+message);
  }
}
static public javax.naming.Context getInitialContext() throws Exception {
   Properties p = new Properties();
   p.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
                  "weblogic.jndi.TengahInitialContextFactory");
   return new InitialContext(p);
}
private void listAllItems(ServletOutputStream out) throws IOException{
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Auction results");
  try {
    addLine("<BR>"+text, out);
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getItemList();
    if(enum != null) {
      displayitems(enum, out);
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List All Items error",out);
    System.out.println("AuctionServlet <list>:"+e);
  }
  out.flush();
}
private void listAllNewItems(ServletOutputStream out) throws IOException {
  setTitle(out, "New Auction Items");
  try {
    addLine("", out);
    String text = "Click Item number for description and to place bid.";
    addLine(text, out);
```

```
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getNewItemList();
    if(enum != null) {
      displayitems(enum, out);
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List New Items error", out);
    System.out.println("AuctionServlet <newlist>:"+e);
  }
  out.flush();
}
private void searchItems(ServletOutputStream out,
                         HttpServletRequest request) throws IOException {
  String searchString=request.getParameter("searchString");
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Search Results");
  try {
    addLine("<BR>"+text, out);
    AuctionItemHome ahome = (AuctionItemHome) ctx.lookup("auctionitems");
    SearchHome shome=(SearchHome) ctx.lookup("search");
    Search search=shome.create();
    Enumeration enum=(Enumeration)search.getMatchingItemsList(searchString);
    addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0><TR>
             <TH>Item</TH><TH>Summary</TH>
             <TH>Current High bid</TH><TH>Number of bids</TH>
            <TH>Closing Date</TH></TR>", out);
    while((enum != null) && (enum.hasMoreElements())) {
      while(enum.hasMoreElements()) {
          AuctionItem ai = ahome.findByPrimaryKey(
                                      (AuctionItemPK)enum.nextElement());
        displayLineItem(ai, out);
      }
    }
    addLine("</TABLE>", out);
  } catch (Exception e) {
    addLine("AuctionServlet Search Items error", out);
    System.out.println("AuctionServlet <searchItems>:"+e);
  }
  out.flush();
}
private void listClosingItems(ServletOutputStream out)  throws IOException{
  setTitle(out, "Items Due to Close Today");
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Items Due to Close Today");
  try {
    addLine("<BR>"+text, out);
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getClosedItemList();
    if(enum != null) {
      displayitems(enum, out);
```

```
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List Closed Items error", out);
    System.out.println("AuctionServlet <close>:"+e);
  }
  out.flush();
}
private void insertItem(ServletOutputStream out, HttpServletRequest request)
                        throws IOException{
  setTitle(out, "Insert Auction Item");
  String seller=request.getParameter("seller");
  String description=request.getParameter("description");
  String summary=request.getParameter("summary");
  String password=request.getParameter("password");
  String price=request.getParameter("startprice");
  double startprice=0.00;
  try {
    startprice=((Number)NumberFormat.getCurrencyInstance(
    Locale.US).parse("$"+price)).doubleValue();
  } catch(java.text.ParseException e) {
    System.out.println("money problem"+e);
    return;
  } catch (NumberFormatException e) {
    System.out.println("money problem"+e);
    return;
  }
  int auctiondays=7;
  try {
    auctiondays=Integer.parseInt(request.getParameter("auctiondays"));
  } catch(NumberFormatException e){
    System.out.println("problem parsing auction days"+e);
    return;
  }
  if(auctiondays<=0 || auctiondays>7) {
    auctiondays=7;
  }
  try {
    SellerHome home = (SellerHome) ctx.lookup("seller");
    Seller si= home.create();
    if(si != null) {
      int result= si.insertItem(seller, password, description,
                                auctiondays, startprice, summary);
      if(result >0) {
        addLine("Inserted item "+summary, out);
        addLine("Details available <A HREF=/AuctionServlet?action=details&item
                = " + resul + ">here</A>", out);
      } else {
        addLine("Error inserting item", out);
        return;
      }
      addLine("", out);
     }
```

```
    } catch (Exception e) {
      addLine("AuctionServlet Insert Item error", out);
      System.out.println("AuctionServlet <insert>:"+e);
    }
    out.flush();
  }
  private void itemDetails(ServletOutputStream out, HttpServletRequest request)
                          throws IOException{
    setTitle(out, "Item Details");
    String item=request.getParameter("item");
    int itemid=0;
    try {
      itemid=Integer.parseInt(item);
    } catch (NumberFormatException e) {
      System.out.println("problem with item id "+itemid);
      return;
    }
    try {
      AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
      AuctionItemPK pk=new AuctionItemPK();
      pk.id=itemid;
      AuctionItem ai=home.findByPrimaryKey(pk);
      displayPageItem(ai, out);
      addLine("<BR><HR><P>Do you want to bid on this item?", out);
      addLine("<FORM ACTION=\"/AuctionServlet\" METHOD=\"POST\">", out);
      addLine("<TR>Enter your user id:<INPUT TYPE=\"TEXT\"
              SIZE=20 NAME=\"buyer\"> ", out);
      addLine("and password:<INPUT TYPE=\"PASSWORD\"
              SIZE=20 NAME=\"password\"></TR>", out);
      addLine("<TR>Your bid amount:<INPUT TYPE=\"TEXT\"
              SIZE=20 NAME=\"amount\"></TR>", out);
      addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"action\" VALUE=\"bid\">", out);
      addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"item\" VALUE=\""+itemid+"\">", out);
        addLine("<INPUT TYPE=\"SUBMIT\" VALUE=\"Place Bid\" NAME=\"Bid\">
                </FORM>", out);
    } catch (Exception e) {
      addLine("AuctionServlet List Item error", out);
      System.out.println("AuctionServlet <details>:"+e);
    }
    out.flush();
  }
private void itemBid(ServletOutputStream out, HttpServletRequest request)
                    throws IOException{
  setTitle(out, "Item Bid");
  String item=request.getParameter("item");
  String buyer=request.getParameter("buyer");
  String password=request.getParameter("password");
  String bid=request.getParameter("amount");
  int itemid=0;
  double bidamount=0;
  try {
    itemid=Integer.parseInt(item);
  } catch (NumberFormatException e) {
```

```
    System.out.println("problem with item id " +itemid);
    return;
  }
  try {
    bidamount=Double.valueOf(bid).doubleValue();
  } catch (NumberFormatException e) {
    System.out.println("problem with bid" +bid);
    return;
  }
  try {
    BidderHome bhome = (BidderHome) ctx.lookup("bidder");
    Bidder bidbean=bhome.create();
    int result=bidbean.placeBid(itemid, buyer, password, bidamount);
    switch(result) {
      case Auction.OUT_BID:
        addLine("Your bid was not high enough", out);
        break;
      case Auction.HIGH_BID:
        addLine("You are the high bidder", out);
        break;
      case Auction.AUCTION_OVER:
        addLine("This auction has finished", out);
        break;
      case Auction.INVALID_USER:
        addLine("Invalid user or password", out);
        break;
      default:
        addLine("Problem submitting bid", out);
      }
    } catch (Exception e) {
      addLine("AuctionServlet Bid error", out);
      System.out.println("AuctionServlet <bid>:"+e);
    }
    out.flush();
  }
  private void registerUser(ServletOutputStream out, HttpServletRequest request)
                            throws IOException{
    String user=request.getParameter("user");
    String password=request.getParameter("password");
    String creditcard=request.getParameter("creditcard");
    String emailaddress=request.getParameter("emailaddress");
    try {
      RegistrationHome regRef = (RegistrationHome)java.rmi.Naming.lookup(
                                "//localhost/registration2");
      RegistrationPK reguser = regRef.create(user, password,
                                emailaddress, creditcard);
      if(reguser != null) {
        addLine("Created user: "+reguser.getUser(),out);
        addLine("", out);
      } else {
        addLine("Error creating user id, possibly already exists ", out);
      }
    } catch(Exception e) {
```

```
    addLine("AuctionServlet registration error", out);
    System.out.println("AuctionServlet <register>:"+e.toString());
  }
  out.flush();
}
private void auditAccounts(ServletOutputStream out, HttpServletRequest request)
                          throws IOException{
  String text = "Audit of users with low credit.";
  String remoteaddr =request.getHeader("REMOTE_ADDR");
  setTitle(out, "Audit Accounts");
  if(!remoteaddr.equals("127.0.0.1")) {
     addLine("service not available", out);
  }
  try {
    addLine("<BR>"+text, out);
    addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0>
           <TR><TH>User</TH><TH>Accounts Balance</TH></TR>", out);
    SellerHome home = (SellerHome) ctx.lookup("seller");
    Seller si= home.create();
    if(si != null) {
      ArrayList ar=si.auditAccounts();
      for(Iterator i=ar.iterator(); i.hasNext();) {
         Registration user=(Registration)(i.next());
         addLine("<TD>"+user.getUser()+"</TD>
                 <TD>"+user.getBalance()+"</TD></TR>", out);
       }
       addLine("</TABLE>", out);
     }
   } catch (Exception e) {
     addLine("AuctionServlet auditAccounts error",out);
     System.out.println("AuctionServlet <auditAccounts>:"+e);
   }
   out.flush();
}
static private void displayitems(Enumeration e, ServletOutputStream out)
                                 throws Exception{
  addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0>
          <TR><TH>Item</TH><TH>Summary</TH>
          <TH>Current High bid</TH><TH>Number of bids</TH>
         <TH>Closing Date</TH></TR>", out);
  while((e !=null) && (e.hasMoreElements())) {
    while(e.hasMoreElements()) {
      displayLineItem((AuctionItem) e.nextElement(), out);
    }
  }
  addLine("</TABLE>", out);
}
static private void displayLineItem(AuctionItem auctionItem,
                                    ServletOutputStream out)
                                    throws RemoteException, IOException {
  StringBuffer line= new StringBuffer();
  int bidcount=auctionItem.getBidCount();
  int key=(int)auctionItem.getId();
```

```
  line.append("<TR><TD><A HREF=/AuctionServlet?action=details&item="+
              key+">"+key+"</A></TD>");
  line.append("<TD>"+auctionItem.getSummary() +"</TD>");
  if(bidcount >0) {
    line.append("<TD>"+
           NumberFormat.getCurrencyInstance().format(auctionItem.getHighBid())
          +"</TD>");
  } else {
    line.append("<TD>-</TD>");
  }
  line.append("<TD>"+bidcount+"</TD>");
  line.append("<TD>"+auctionItem.getEndDate() +"</TD></TR>");
  addLine(line.toString(), out);
}
static private void displayPageItem(AuctionItem auctionItem,
                                    ServletOutputStream out)
                                    throws RemoteException, IOException {
  int bidcount=auctionItem.getBidCount();
  addLine(auctionItem.getSummary(), out);
  addLine("Auction Item Number: "+auctionItem.getPrimaryKey(), out);
  if(bidcount >0) {
    addLine("<P>Current price: " +
         NumberFormat.getCurrencyInstance().format(auctionItem.getHighBid()),
         out);
    addLine("Minimum increment: "+
           NumberFormat.getCurrencyInstance().format(
          auctionItem.getIncrement()), out);
  } else {
    addLine("<P>Current price: "+ NumberFormat.getCurrencyInstance().format(
           auctionItem.getStartPrice()), out);
  }
  addLine("# of bids: "+bidcount, out);
  addLine("<P>Auction Started: "+auctionItem.getStartDate(), out);
  addLine("Auction Ends: "+auctionItem.getEndDate(), out);
  addLine("<P>Seller: "+auctionItem.getSeller(), out);
  if(bidcount >0) {
    addLine("High Bidder: "+auctionItem.getHighBidder(), out);
  } else {
    addLine("High Bidder: "+"-", out);
  }
  addLine("<HR><P>", out);
  addLine("Description: "+auctionItem.getDescription(), out);
}
private String readFile (String file) throws IOException {
  if(file != null) {
    StringBuffer buffer = new StringBuffer();
    BufferedReader reader = new BufferedReader (new FileReader (file));
    String line;
    while( (line=reader.readLine()) != null ) {
      buffer.append (line+'\n');
    }
    reader.close();
    return buffer.toString();
```

```
      } else {
      return null;
    }
  }
  private void setTitle(ServletOutputStream out, String title) {
    try {
      out.println("<HTML><HEAD><TITLE>"+title+"</TITLE></HEAD>");
      out.println("<BODY BGCOLOR=\"WHITE\">");
     } catch(IOException e) {
       System.out.println("Unable to set title"+e);
     }
  }
  private void setFooter(ServletOutputStream out) {
    try {
      out.println("<HR><CENTER> <A HREF=\"registration.html\">Register</A>");
      out.println("<STRONG>|</STRONG>");
        out.println("<A HREF=\"/AuctionServlet?action=newlist\">New Items</A>");
      out.println("<STRONG>|</STRONG>");
       out.println("<A HREF=\"/AuctionServlet?action=close\">Closing Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"/AuctionServlet?action=list\">All Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"sell.html\">Sell Items</A>");
      out.println("</CENTER>");
    } catch (IOException e) {
      System.out.println("Unable to set footer"+e);
    }
  }
}
```

RegistrationServer (CORBA)

```
//This file contains the RegistrationServer
//and RegistrationImpl implementations

package registration;

import java.sql.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;
import java.util.ArrayList;

public class RegistrationServer extends _RegistrationHomeImplBase{
   ORB orb=null;
    static {
      try {
        new pool.JDCConnectionDriver("COM.cloudscape.core.JDBCDriver",
                          "jdbc:cloudscape:ejbdemo","none", "none");
      }catch(Exception e){ System.out.println("new pool error"+e);}
    }
    public Connection getConnection() throws SQLException {
        return DriverManager.getConnection("jdbc:jdc:jdcpool");
```

```
  }
public RegistrationServer(ORB orb)  {
  super();
  this.orb=orb;
}
public registration.RegistrationPK create(String theuser, String password,
                                String emailaddress, String creditcard)
                                throws registration.CreateException{
  double balance=0;
  Connection con = null;
  PreparedStatement ps = null;;
  try {
    con=getConnection();
    ps=con.prepareStatement("insert into registration(theuser, password,
                    emailaddress, creditcard, balance) values (?, ?, ?, ?, ?)");
    ps.setString(1, theuser);
    ps.setString(2, password);
    ps.setString(3, emailaddress);
    ps.setString(4, creditcard);
    ps.setDouble(5, balance);

    if(ps.executeUpdate() != 1) {
       throw new CreateException ();
    }
    RegistrationPK primaryKey = new RegistrationPKImpl();
    primaryKey.theuser(theuser);
    return primaryKey;
  } catch (CreateException ce) {
    throw ce;
  } catch (SQLException sqe) {
    System.out.println("sqe="+sqe);
    throw new CreateException ();
  } finally {
    try {
      ps.close();
      con.close();
    } catch (Exception ignore) {
  }
}
public registration.Registration findByPrimaryKey(
                                    registration.RegistrationPK pk)
  if((pk == null) || (pk.theuser() == null)) {
    throw new FinderException ();
  }
  return(refresh(pk));
}
private Registration refresh(RegistrationPK pk) throws FinderException {
  if(pk == null) {
    throw new FinderException ();
  }
  Connection con = null;
  PreparedStatement ps = null;
  try {
```

```
    con=getConnection();
    ps=con.prepareStatement("select password, emailaddress, creditcard,
                            balance from registration where theuser = ?");
    ps.setString(1, pk.theuser());
    ps.executeQuery();
    ResultSet rs = ps.getResultSet();
    if(rs.next()) {
      RegistrationImpl reg= new RegistrationImpl();
      reg.theuser = pk.theuser();
      reg.password = rs.getString(1);
      reg.emailaddress = rs.getString(2);
      reg.creditcard = rs.getString(3);
      reg.balance = rs.getDouble(4);
      return reg;
    } else {
       throw new FinderException ();
    }
  } catch (SQLException sqe) {
    throw new FinderException ();
  } finally {
    try {
      ps.close();
      con.close();
     } catch (Exception ignore) {}
   }
}
public void findLowCreditAccounts(final ReturnResults client)
                                  throws FinderException {
  Runnable bgthread = new Runnable() {
    public void run() {
      Connection con = null;
      ResultSet rs = null;
      PreparedStatement ps = null;
      ArrayList ar = new ArrayList();
      try {
        con=getConnection();
        ps=con.prepareStatement("select theuser,
                            balance from registration where balance < ?");
        ps.setDouble(1, 3.00);
        ps.executeQuery();
        rs = ps.getResultSet();
        RegistrationImpl reg=null;
        while (rs.next()) {
          try {
            reg= new RegistrationImpl();
          } catch (Exception e) {
            System.out.println("creating reg"+e);
          }
          reg.theuser = rs.getString(1);
          reg.balance = rs.getDouble(2);
          ar.add(reg);
        }
        rs.close();
```

```
            RegistrationImpl[] regarray = (RegistrationImpl [])
            ar.toArray( new RegistrationImpl[0]);
            client.updateResults(regarray);
          } catch (Exception e) {
            System.out.println("findLowCreditAccounts: "+e);
            return;
          } finally {
            try {
              if(rs != null) {
                rs.close();
              }
              if(ps != null) {
                ps.close();
              }
              if(con != null) {
                con.close();
              }
            } catch (Exception ignore) {}
          }
        } //run
      };
      Thread t = new Thread(bgthread);
      t.start();
  }
  public Any customSearch(Any searchField, IntHolder count) {
    Any returnResults= orb.create_any();
    int tmpcount=count.value;
    if(searchField.type().kind().value() == TCKind._tk_double) {
      // return number of balances greater than supplied amount
      double findBalance=searchField.extract_double();
      Connection con = null;
      ResultSet rs = null;
      PreparedStatement ps = null;
      try {
        con = getConnection();
        ps =c on.prepareStatement("select count(*)
                                  from registration where balance < ?");
        ps.setDouble(1, findBalance);
        ps.executeQuery();
        rs = ps.getResultSet();
        if(rs.next()) {
          tmpcount = rs.getInt(1);
        }
        count.value=tmpcount;
        rs.close();
      } catch (Exception e) {
        System.out.println("custom search: "+e);
        returnResults.insert_long(-1);
        return(returnResults);
      } finally {
        try {
          if(rs != null) {
             rs.close();
```

```
        }
        if(ps != null) {
          ps.close();
        }
        if(con != null) {
           con.close();
        }
      } catch (Exception ignore) {}
    }
    returnResults.insert_long(tmpcount);
    return(returnResults);
  } else if(searchField.type().kind().value() ==
    TCKind._tk_string) {
    // return email addresses that match supplied address
    String findEmail=searchField.extract_string();
    Connection con = null;
    ResultSet rs = null;
    PreparedStatement ps = null;
    ArrayList ar = new ArrayList();
    RegistrationImpl reg=null;
    try {
      con = getConnection();
      ps = con.prepareStatement("select theuser,
                  emailaddress from registration where emailaddress like ?");
      ps.setString(1, findEmail);
      ps.executeQuery();
      rs = ps.getResultSet();
      while(rs.next()) {
        reg= new RegistrationImpl();
        reg.theuser = rs.getString(1);
        reg.emailaddress = rs.getString(2);
        ar.add(reg);
      }
      rs.close();
      RegistrationImpl[] regarray = (RegistrationImpl
                                [])ar.toArray( new RegistrationImpl[0]);
      RegistrationHelper.insert(returnResults, regarray[0]);
      return(returnResults);
    } catch (Exception e) {
      System.out.println("custom search: "+e);
      return(returnResults);
    } finally {
      try {
        if(rs != null) {
          rs.close();
        }
        if(ps != null) {
           ps.close();
        }
        if(con != null) {
          con.close();
        }
      } catch (Exception ignore) {}
```

```
        }
      }
      return(returnResults);
    }
    public static void main(String args[]) {
      java.util.Properties props=System.getProperties();
      props.put("org.omg.CORBA.ORBInitialPort", "1050");
      System.setProperties(props);
      ORB orb = ORB.init(args, props);
      RegistrationServer rs= new RegistrationServer(orb);
      try {
        orb.connect(rs);
        org.omg.CORBA.Object nameServiceObj =
                    orb.resolve_initial_references("NameService") ;
        NamingContext nctx= NamingContextHelper.narrow(nameServiceObj);
        NameComponent[] fullname = new NameComponent[2];
        fullname[0] = new NameComponent("auction", "");
        fullname[1] = new NameComponent("RegistrationBean", "");
        NameComponent[] tempComponent = new NameComponent[1];
        for(int i=0; i < fullname.length-1; i++ ) {
           tempComponent[0]= fullname[i];
           try {
             nctx=nctx.bind_new_context(tempComponent);
           } catch (Exception e){ System.out.println("bind new"+e);}
        }
        tempComponent[0]=fullname[fullname.length-1];
        try {
          nctx.rebind(tempComponent, rs);
        } catch (Exception e){
          nctx.unbind(tempComponent);
          nctx.rebind(tempComponent, rs);
          System.out.println("rebind"+e);
        }
        java.lang.Object sync= new java.lang.Object();
        synchronized(sync) {
          sync.wait();
        }
      } catch (Exception e) {
        System.out.println("e="+e);
      }
    }
}
class RegistrationImpl extends _RegistrationImplBase {
    public String theuser, password, creditcard, emailaddress;
    public double balance;

    public boolean verifyPassword(String password) {
      if(this.password.equals(password)) {
        return true;
      } else {
        return false;
      }
    }
```

```
  public String getEmailAddress() {
    return emailaddress;
  }
  public String getUser() {
    return theuser;
  }
  public int adjustAccount(double amount) {
    balance=balance+amount;
    return(0);
  }
  public double getBalance() {
    return balance;
  }
}
```

Registration.idl (CORBA)

```
module registration {

  interface Registration {
    boolean verifyPassword(in string password);
    string getEmailAddress();
    string getUser();
    long adjustAccount(in double amount);
    double getBalance();
   };

  interface RegistrationPK {
    attribute string theuser;
  };
  enum LoginError {INVALIDUSER, WRONGPASSWORD, TIMEOUT};
  exception CreateException {};
  exception FinderException {};
  typedef sequence<Registration> IDLArrayList;
  interface ReturnResults {
   void updateResults(in IDLArrayList results) raises (FinderException);
  };
  interface RegistrationHome {
    RegistrationPK create(in string theuser, in string password,
                          in string emailaddress, in string creditcard)
                          raises (CreateException);
    Registration findByPrimaryKey(in RegistrationPK theuser)
                          raises (FinderException);
   void findLowCreditAccounts(in ReturnResults rr) raises (FinderException);
    any customSearch(in any searchfield, out long count);
  };
};
```

SellerBean (CORBA)

```
package seller;
```

```
import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.io.Serializable;
import javax.naming.*;
import auction.*;
import registration.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class SellerBean extends _ReturnResultsImplBase implements SessionBean {
  protected SessionContext ctx;
  javax.naming.Context ectx;
  Hashtable env = new Hashtable();
  ORB orb = null;
  NamingContext nctx= null;
  Boolean ready=new Boolean("false");
  ArrayList returned =new ArrayList();
  public int insertItem(String seller, String password, String description,
                        int auctiondays, double startprice, String summary)
                        throws RemoteException {
    try {
      NameComponent[] fullname = new NameComponent[2];
      fullname[0] = new NameComponent("auction", "");
      fullname[1] = new NameComponent("RegistrationBean", "");
      RegistrationHome regRef = RegistrationHomeHelper.narrow(
                                nctx.resolve(fullname));
      RegistrationPK rpk= new RegistrationPKImpl();
      rpk.theuser(seller);
      Registration newseller = RegistrationHelper.narrow(
      regRef.findByPrimaryKey(rpk));
      if((newseller == null)|| (!newseller.verifyPassword(password))) {
          return(Auction.INVALID_USER);
      }
      AuctionItemHome home = (AuctionItemHome) ectx.lookup("auctionitems");
      AuctionItem ai= home.create(seller, description,  auctiondays,
                                  startprice, summary);
      if(ai == null) {
        return Auction.INVALID_ITEM;
       } else {
         return(ai.getId());
       }
     } catch(Exception e){
       System.out.println("insert problem="+e);
       return Auction.INVALID_ITEM;
     }
  }
  public void updateResults(Registration[] ar)
                          throws registration.FinderException {
    if(ar == null) {
      throw new registration.FinderException();
```

```
  }
  try {
    for(int i=0; i< ar.length; i++) {
      returned.add(ar[i]);
    }
  } catch (Exception e) {
    System.out.println("updateResults="+e);
    throw new registration.FinderException();
  }
  synchronized(ready) {
    ready.notifyAll();
  }
}
public ArrayList auditAccounts() {
  try {
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent("RegistrationBean", "");
    RegistrationHome regRef = RegistrationHomeHelper.narrow(
                              nctx.resolve(fullname));
    regRef.findLowCreditAccounts(this);
    synchronized(ready) {
    try {
      ready.wait();
    } catch (InterruptedException e){
    }
    return (returned);
  } catch (Exception e) {
    System.out.println("error in auditAccounts "+e);
  }
    return null;
}
public String customFind(String searchField)
             throws javax.ejb.FinderException, RemoteException{
  int total=-1;
  IntHolder count= new IntHolder();
  try {
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent("RegistrationBean", "");
    RegistrationHome regRef = RegistrationHomeHelper.narrow(
                                nctx.resolve(fullname));
    if(regRef == null ) {
      System.out.println("cannot contact RegistrationHome");
      throw new javax.ejb.FinderException();
    }
    Any sfield=orb.create_any();
    Double balance;
    try {
      balance=Double.valueOf(searchField);
      try {
        sfield.insert_double(balance.doubleValue());
      } catch (Exception e) {
```

```
        return("Problem with search value"+balance);
      }
      sfield=regRef.customSearch(sfield,count);
      if(sfield != null ) {
        total=sfield.extract_long();
      }
      return(total+" accounts are below optimal level");
    } catch (NumberFormatException e) {
      sfield.insert_string(searchField);
      Registration reg;
      if((reg = RegistrationHelper.extract(
               regRef.customSearch(sfield,count))) != null ) {
        return("Found user "+ reg.getUser() +
               " who has email address "+reg.getEmailAddress());
      } else {
        return("No users found who have email address "+searchField);
      }
    }
    } catch(Exception e){
      System.out.println("customFind problem="+e);
      throw new javax.ejb.FinderException();
    }
}
public void ejbCreate() throws javax.ejb.CreateException, RemoteException {
  env.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
          "weblogic.jndi.TengahInitialContextFactory");
  try {
    ectx = new InitialContext(env);
  } catch (NamingException e) {
    System.out.println("problem contacting EJB server");
    throw new javax.ejb.CreateException();
  }
  try {
    java.util.Properties props=System.getProperties();
    props.put("org.omg.CORBA.ORBInitialPort", "1050");
    System.setProperties(props);
    String[] args={};
    orb = ORB.init(args, props);
    org.omg.CORBA.Object nameServiceObj = orb.resolve_initial_references(
                                          "NameService") ;
    nctx= NamingContextHelper.narrow(nameServiceObj);
  } catch(org.omg.CORBA.SystemException e) {
    throw new javax.ejb.CreateException("CORBA Error in SellerCreate "+e);
  } catch ( org.omg.CORBA.ORBPackage.InvalidName e) {
     throw new javax.ejb.CreateException("CORBA NameError in SellerCreate "+e);
  }
}
public void setSessionContext(SessionContext ctx) throws RemoteException {
  this.ctx = ctx;
}
public void unsetSessionContext() throws RemoteException {
  ctx = null;
}
```

```
  public void ejbRemove() {}
  public void ejbActivate() throws RemoteException {
    System.out.println("activating seller bean");
  }
  public void ejbPassivate() throws RemoteException {
    System.out.println("passivating seller bean");
  }
}
```

Seller (CORBA)

```
package seller;

import javax.ejb.*;
import java.rmi.*;
import java.util.*;

public interface Seller extends EJBObject {
  int insertItem(String user, String password, String description,
                 int auctiondays,double startprice, String summary)
                 throws RemoteException;
   ArrayList auditAccounts() throws FinderException, RemoteException;
    String customFind(String searchField) throws FinderException, RemoteException;
}
```

AuctionServlet (CORBA)

```
package auction;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.naming.*;
import javax.ejb.*;
import java.rmi.RemoteException;
import java.util.*;
import java.text.NumberFormat;
import bidder.*;
import registration.*;
import seller.*;
import pool.*;
import search.*;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;

public class AuctionServlet extends HttpServlet {
  javax.naming.Context ctx=null;
  ORB orb = null;
  NamingContext nctx= null;
  public void init(ServletConfig config) throws ServletException{
    super.init(config);
```

```
  try {
    ctx = getInitialContext();
  } catch (Exception e) {
    System.err.println("failed to contact EJB server"+e);
  }
  try {
    String[] args = {};
    Properties props= System.getProperties();
    props.put("org.omg.CORBA.ORBInitialPort", "1050");
    System.setProperties(props);
    orb = ORB.init(args, props) ;
    org.omg.CORBA.Object nameServiceObj =
                          orb.resolve_initial_references("NameService") ;
    nctx= NamingContextHelper.narrow(nameServiceObj);
  } catch(org.omg.CORBA.SystemException e) {
      System.err.println("CORBA Error in AuctionServlet <init>"+e);
  } catch(Exception e) {
      System.err.println("Error in AuctionServlet <init>"+e);
  }
}
public void service(HttpServletRequest request,
                    HttpServletResponse response) throws IOException {
  String cmd;
  response.setContentType("text/html");
  ServletOutputStream out = response.getOutputStream();
  if(ctx == null ) {
    try {
      ctx = getInitialContext();
    } catch (Exception e){
      System.err.println("failed to contact EJB server"+e);
    }
  }
  cmd=request.getParameter("action");
  if(cmd !=null)  {
    if(cmd.equals("list")) {
      listAllItems(out);
    } else if(cmd.equals("newlist")) {
      listAllNewItems(out);
    } else if(cmd.equals("search")) {
      searchItems(out, request);
    } else if(cmd.equals("close")) {
      listClosingItems(out);
    } else if(cmd.equals("insert")) {
      insertItem(out, request);
    } else if (cmd.equals("details")) {
      itemDetails(out, request );
    } else if (cmd.equals("bid")) {
      itemBid(out, request) ;
    } else if (cmd.equals("auditAccounts")) {
      auditAccounts(out, request);
    } else if (cmd.equals("register")) {
      registerUser(out, request);
    } else if (cmd.equals("customSearch")) {
```

```
      customSearch(out, request);
    } else {
      addLine("unknown command "+cmd, out);
    }
  } else {
    // no command set
    setTitle(out, "error");
  }
  setFooter(out);
  out.flush();
}
static private void addLine(String message, ServletOutputStream out)
           throws IOException {
  if(message !=null) {
    out.println("<BR>"+message);
  }
}
static public javax.naming.Context getInitialContext() throws Exception {
  Properties p = new Properties();
  p.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
                        "weblogic.jndi.TengahInitialContextFactory");
  return new InitialContext(p);
}
private void listAllItems(ServletOutputStream out) throws IOException{
  String text = "Click Item number for description and to place bid.";
  setTitle(out, "Auction results");
  try {
    addLine("<BR>"+text, out);
    BidderHome bhome=(BidderHome) ctx.lookup("bidder");
    Bidder bid=bhome.create();
    Enumeration enum=(Enumeration)bid.getItemList();
    if(enum != null) {
      displayitems(enum, out);
      addLine("", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet List All Items error",out);
    System.out.println("AuctionServlet <list>:"+e);
  }
  out.flush();
}
private void listAllNewItems(ServletOutputStream out) throws IOException {
      setTitle(out, "New Auction Items");
      try {
          addLine("", out);
          String text = "Click Item number for description and to place bid.";
          addLine(text, out);
          BidderHome bhome = (BidderHome) ctx.lookup("bidder");
          Bidder bid = bhome.create();
          Enumeration enum = (Enumeration)bid.getNewItemList();
          if(enum != null) {
              displayitems(enum, out);
              addLine("", out);
```

```
            }
          } catch (Exception e) {
              addLine("AuctionServlet List New Items error", out);
              System.out.println("AuctionServlet <newlist>:"+e);
          }
          out.flush();
      }
      private void searchItems(ServletOutputStream out,
                  HttpServletRequest request) throws IOException {
        String searchString = request.getParameter("searchString");
        String text = "Click Item number for description and to place bid.";
        setTitle(out, "Search Results");
        try {
          addLine("<BR>"+text, out);
          AuctionItemHome ahome = (AuctionItemHome) ctx.lookup("auctionitems");
          SearchHome shome = (SearchHome) ctx.lookup("search");
          Search search = shome.create();
          Enumeration enum = (Enumeration)
                               search.getMatchingItemsList(searchString);
          addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0>
                  <TR><TH>Item</TH><TH>Summary</TH>
                  <TH>Current High bid</TH><TH>Number of bids</TH>
                  <TH>Closing Date</TH></TR>", out);
            while ((enum != null) && (enum.hasMoreElements())) {
              while(enum.hasMoreElements()) {
                AuctionItem ai = ahome.findByPrimaryKey((
                                  AuctionItemPK)enum.nextElement());
                displayLineItem(ai, out);
              }
            }
            addLine("</TABLE>", out);
          } catch (Exception e) {
            addLine("AuctionServlet Search Items error", out);
            System.out.println("AuctionServlet <searchItems>:"+e);
          }
          out.flush();
      }
      private void listClosingItems(ServletOutputStream out)  throws IOException{
        setTitle(out, "Items Due to Close Today");
        String text = "Click Item number for description and to place bid.";
        setTitle(out, "Items Due to Close Today");
        try {
          addLine("<BR>"+text, out);
          BidderHome bhome=(BidderHome) ctx.lookup("bidder");
          Bidder bid=bhome.create();
          Enumeration enum=(Enumeration)bid.getClosedItemList();
          if(enum != null) {
            displayitems(enum, out);
            addLine("", out);
          }
        } catch (Exception e) {
          addLine("AuctionServlet List Closed Items error", out);
          System.out.println("AuctionServlet <close>:"+e);
```

```
  }
  out.flush();
}
private void insertItem(ServletOutputStream out, HttpServletRequest request)
                        throws IOException{
  setTitle(out, "Insert Auction Item");
  String seller=request.getParameter("seller");
  String description=request.getParameter("description");
  String summary=request.getParameter("summary");
  String password=request.getParameter("password");
  String price=request.getParameter("startprice");
  double startprice=0.00;
  try {
    startprice=((Number)NumberFormat.
    getCurrencyInstance(Locale.US).parse("$"+price)).
    doubleValue();
  } catch(java.text.ParseException e){
    System.out.println("money problem"+e);
    return;
  } catch (NumberFormatException e) {
    System.out.println("money problem"+e);
    return;
  }
  int auctiondays=7;
  try {
     auctiondays=Integer.parseInt(request.getParameter("auctiondays"));
  } catch(NumberFormatException e){
    System.out.println("problem parsing auction days"+e);
    return;
  }
  if(auctiondays<=0 || auctiondays>7) {
    auctiondays=7;
  }
  try {
    SellerHome home = (SellerHome) ctx.lookup("seller");
    Seller si= home.create();
    if(si != null) {
      int result= si.insertItem(seller, password, description,
                                auctiondays, startprice, summary);
      if(result >0) {
        addLine("Inserted item "+summary, out);
        addLine("Details available
                 <A HREF=/AuctionServlet?action=details&item="
                 + result + ">here</A>", out);
      } else {
        addLine("Error inserting item", out);
        return;
      }
      addLine("", out);
    }
  } catch (Exception e) {
     addLine("AuctionServlet Insert Item error", out);
     System.out.println("AuctionServlet <insert>:"+e);
```

```
  }
  out.flush();
}
private void itemDetails(ServletOutputStream out, HttpServletRequest request)
                        throws IOException{
  setTitle(out, "Item Details");
  String item=request.getParameter("item");
  int itemid=0;
  try {
    itemid=Integer.parseInt(item);
  } catch (NumberFormatException e) {
    System.out.println("problem with item id "+itemid);
    return;
  }
  try {
    AuctionItemHome home = (AuctionItemHome) ctx.lookup("auctionitems");
    AuctionItemPK pk=new AuctionItemPK();
    pk.id=itemid;
    AuctionItem ai=home.findByPrimaryKey(pk);
    displayPageItem(ai, out);
    addLine("<BR><HR><P>Do you want to bid on this item?", out);
    addLine("<FORM ACTION=\"/AuctionServlet\" METHOD=\"POST\">", out);
    addLine("<TR>Enter your user id:<INPUT TYPE=\"TEXT\"
           SIZE=20 NAME=\"buyer\"> ", out);
    addLine("and password:<INPUT TYPE=\"PASSWORD\"
           SIZE=20 NAME=\"password\"></TR>", out);
    addLine("<TR>Your bid amount:<INPUT TYPE=\"TEXT\"
            SIZE=20 NAME=\"amount\"></TR>", out);
    addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"action\" VALUE=\"bid\">", out);
    addLine("<INPUT TYPE=\"HIDDEN\" NAME=\"item\" VALUE=\""+itemid+"\">", out);
    addLine("<INPUT TYPE=\"SUBMIT\" VALUE=\"Place Bid\" NAME=\"Bid\">
            </FORM>", out);
  } catch (Exception e) {
    addLine("AuctionServlet List Item error", out);
    System.out.println("AuctionServlet <details>:"+e);
  }
  out.flush();
}
private void itemBid(ServletOutputStream out, HttpServletRequest request)
                     throws IOException{
  setTitle(out, "Item Bid");
  String item=request.getParameter("item");
  String buyer=request.getParameter("buyer");
  String password=request.getParameter("password");
  String bid=request.getParameter("amount");
  int itemid=0;
  double bidamount=0;
  try {
    itemid=Integer.parseInt(item);
  } catch (NumberFormatException e) {
    System.out.println("problem with item id " +itemid);
    return;
  }
```

```
  try {
    bidamount=Double.valueOf(bid).doubleValue();
  } catch (NumberFormatException e) {
    return;
  }
  try {
    BidderHome bhome = (BidderHome) ctx.lookup("bidder");
    Bidder bidbean=bhome.create();
    int result=bidbean.placeBid(itemid, buyer, password, bidamount);
    switch(result) {
      case Auction.OUT_BID:
        addLine("Your bid was not high enough", out);
        break;
      case Auction.HIGH_BID:
        addLine("You are the high bidder", out);
        break;
      case Auction.AUCTION_OVER:
        addLine("This auction has finished", out);
        break;
      case Auction.INVALID_USER:
        addLine("Invalid user or password", out);
        break;
     default:
       addLine("Problem submitting bid", out);
    }
  } catch (Exception e) {
    addLine("AuctionServlet Bid error", out);
    System.out.println("AuctionServlet <bid>:"+e);
  }
  out.flush();
}
private void registerUser(ServletOutputStream out,
                          HttpServletRequest request) throws IOException{
  String user=request.getParameter("user");
  String password=request.getParameter("password");
  String creditcard=request.getParameter("creditcard");
  String emailaddress=request.getParameter("emailaddress");
  try {
    NameComponent[] fullname = new NameComponent[2];
    fullname[0] = new NameComponent("auction", "");
    fullname[1] = new NameComponent("RegistrationBean", "");
    RegistrationHome regRef = RegistrationHomeHelper.narrow(
                              nctx.resolve(fullname));
    RegistrationPK reguser = RegistrationPKHelper.narrow(regRef.create(
                             user, password, emailaddress, creditcard));
    if(reguser != null) {
      addLine("Created user: "+reguser.theuser(),out);
    } else {
      addLine("Error creating user id, possibly already exists ", out);
    }
  } catch(org.omg.CORBA.SystemException e) {
    addLine("AuctionServlet registration error", out);
    System.out.println("AuctionServlet CORBA <register>:"+e.toString());
```

```
    } catch (Exception e) {
      System.out.println("AuctionServlet <register>:"+e.toString());
    }
    out.flush();
  }
  private void auditAccounts(ServletOutputStream out,
              HttpServletRequest request) throws IOException {
    String text = "Audit of users with low credit.";
    setTitle(out, "Audit Accounts");
    try {
      addLine("<BR>"+text, out);
      addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0><TR>
              <TH>User</TH><TH>Accounts Balance</TH></TR>", out);
      SellerHome home = (SellerHome) ctx.lookup("seller");
      Seller si= home.create();
      if(si != null) {
        ArrayList ar=si.auditAccounts();
        for(Iterator i=ar.iterator(); i.hasNext();) {
          Registration user=(Registration)(i.next());
          addLine("<TD>"+user.getUser()+"</TD>
                <TD>"+user.getBalance()+"</TD></TR>", out);
        }
        addLine("</TABLE>", out);
      }
    } catch (Exception e) {
      addLine("AuctionServlet auditAccounts error",out);
      System.out.println("AuctionServlet <auditAccounts>:"+e);
    }
    out.flush();
  }
  private void customSearch(ServletOutputStream out,
              HttpServletRequest request) throws IOException{
    String text = "Custom Search";
    String searchField=request.getParameter("searchfield");
    setTitle(out, "Custom Search");
    if(searchField == null ) {
      addLine("Error: SearchField was empty", out);
      out.flush();
      return;
    }
    try {
      addLine("<BR>"+text, out);
      SellerHome home = (SellerHome) ctx.lookup("seller");
      Seller si= home.create();
      if(si != null) {
        String displayMessage=si.customFind(searchField);
        if(displayMessage != null ) {
          addLine(displayMessage+"<BR>", out);
        }
      }
    } catch (Exception e) {
      addLine("AuctionServlet customFind error",out);
      System.out.println("AuctionServlet <customFind>:"+e);
```

```
  }
  out.flush();
}
static private void displayitems(Enumeration e,
                    ServletOutputStream out) throws Exception{
   addLine("<TABLE BORDER=1 CELLPADDING=1 CELLSPACING=0><TR>
           <TH>Item</TH><TH>Summary</TH><TH>Current High bid</TH>
           <TH>Number of bids</TH><TH>Closing Date</TH></TR>", out);
   while((e !=null) && (e.hasMoreElements())) {
     while(e.hasMoreElements()) {
       displayLineItem((AuctionItem) e.nextElement(), out);
     }
   }
   addLine("</TABLE>", out);
}
static private void displayLineItem(AuctionItem auctionItem,
                                    ServletOutputStream out)
                                    throws RemoteException, IOException {
  StringBuffer line= new StringBuffer();
  int bidcount=auctionItem.getBidCount();
  int key=(int)auctionItem.getId();
  line.append("<TR><TD><A HREF=/AuctionServlet?action=details&item="
              + key + ">" + key + "</A></TD>");
  line.append("<TD>"+auctionItem.getSummary() +"</TD>");
  if(bidcount >0) {
    line.append("<TD>" + NumberFormat.getCurrencyInstance().format(
                auctionItem.getHighBid()) +"</TD>");
  } else {
    line.append("<TD>-</TD>");
  }
  line.append("<TD>"+bidcount+"</TD>");
  line.append("<TD>"+auctionItem.getEndDate() +"</TD></TR>");
  addLine(line.toString(), out);
}
static private void displayPageItem(AuctionItem auctionItem,
                                    ServletOutputStream out)
                                    throws RemoteException, IOException {
  int bidcount=auctionItem.getBidCount();
  addLine(auctionItem.getSummary(), out);
  addLine("Auction Item Number: "+auctionItem.getPrimaryKey(), out);
  if(bidcount >0) {
    addLine("<P>Current price: "+  NumberFormat.getCurrencyInstance().format(
                                   auctionItem.getHighBid()), out);
    addLine("Minimum increment: "+ NumberFormat.getCurrencyInstance().format(
                                   auctionItem.getIncrement()), out);
  } else {
    addLine("<P>Current price:  "+ NumberFormat.getCurrencyInstance().format(
                                   auctionItem.getStartPrice()), out);
  }
  addLine("# of bids: "+bidcount, out);
  addLine("<P>Auction Started: "+auctionItem.getStartDate(), out);
  addLine("Auction Ends: "+auctionItem.getEndDate(), out);
  addLine("<P>Seller: "+auctionItem.getSeller(), out);
```

```
    if(bidcount >0) {
      addLine("High Bidder: "+auctionItem.getHighBidder(), out);
    } else {
      addLine("High Bidder: "+"-", out);
   }
    addLine("<HR><P>", out);
    addLine("Description: "+auctionItem.getDescription(), out);
  }
  private String readFile (String file) throws IOException {
    if(file != null) {
      StringBuffer buffer = new StringBuffer();
      BufferedReader reader = new BufferedReader (new FileReader (file));
      String line;
      while( (line=reader.readLine()) != null ) {
        buffer.append (line+'\n');
      }
      reader.close();
      return buffer.toString();
    } else {
      return null;
    }
  }
  private void setTitle(ServletOutputStream out, String title) {
    try {
      out.println("<HTML><HEAD><TITLE>"+title+"</TITLE></HEAD>");
      out.println("<BODY BGCOLOR=\"WHITE\">");
    } catch(IOException e) {
      System.out.println("Unable to set title"+e);
    }
  }
  private void setFooter(ServletOutputStream out) {
    try {
      out.println("<HR><CENTER> <A HREF=\"registration.html\">Register</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"/AuctionServlet?action=newlist\">New Items</A>");
      out.println("<STRONG>|</STRONG>");
       out.println("<A HREF=\"/AuctionServlet?action=close\">Closing Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"/AuctionServlet?action=list\">All Items</A>");
      out.println("<STRONG>|</STRONG>");
      out.println("<A HREF=\"sell.html\">Sell Items</A>");
      out.println("</CENTER>");
    } catch (IOException e) {
      System.out.println("Unable to set footer"+e);
    }
  }
}
```

Login Servlet

```
package login;
```

```
import java.io.*;
import java.util.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
  protected String site;
  protected String domain, restricted;
  protected SessionCache sessionCache;
  protected long flushTimeout=600000;
  protected long sessionTimeout=7200000;
  protected boolean protectedDir=false;
  protected String defaultPage="/index.html";
  weblogic.servlet.FileServlet fileServlet;
  static {
    try {
      new pool.JDCConnectionDriver("postgresql.Driver",
                          "jdbc:postgresql:ejbdemo","postgres", "pass");
    } catch(Exception e){
      System.out.println("new pool error"+e);
    }
  }
  public Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:jdc:jdcpool");
  }
  public void init(ServletConfig config) throws ServletException  {
    super.init(config);
    domain = config.getInitParameter("domain");
    restricted = config.getInitParameter("restricted");
    if(restricted != null) {
      protectedDir=true;
    }
    sessionCache = new SessionCache (flushTimeout);
    if((fileServlet=(weblogic.servlet.FileServlet)
      config.getServletContext().getServlet("file")) == null ) {
      throw new ServletException ("No FileServlet found installed ");
    }
  }
  public void service(HttpServletRequest request,
                      HttpServletResponse response) throws IOException {
    Cookie thisCookie=null;
    boolean activeSession;
    String cmd;
    Session session = validateSession (request, response);
    if((cmd=request.getParameter ("action")) != null ) {
      setNoCache (request, response);
      if(cmd.equals ("login") && session == null ) {
        // create a new session
        session = startSession(request.getParameter("theuser"),
                               request.getParameter ("password"), response);
        if(protectedDir) {
          response.sendRedirect (restricted+"/index.html");
```

```
        } else {
          response.sendRedirect (defaultPage);
        }
      } else if (cmd.equals (“displayDetails”))  {
        if(session != null) {
          response.setContentType(“text/html”);
          ServletOutputStream out = response.getOutputStream();
          out.println(“User id is “+session.getUser());
          out.flush();
        } else {
          response.sendRedirect (“/login.html”);
        }
      } else if (cmd.equals (“logout”)) {
        if(session != null ) {
          endSession (session);
        }
        response.sendRedirect (defaultPage);
      } else {
        response.sendRedirect (defaultPage);
      }
    } else {
      if (session != null) {
      // already logged in
      if( response.containsHeader (“Expires”) == false ) {
        response.setDateHeader (“Expires”, session.getExpires());
      }
      try {
        fileServlet.doGet (request, response);
      } catch (Exception e) {
        response.sendRedirect (“/login.html”);
      }
    } else {
      // no existing session
      if(protectedDir &&
        request.getRequestURI().indexOf(restricted)>=0) {
        // restricted directory. Dont go in there!
           response.sendRedirect (“/login.html”);
         } else {
        // this file looks ok to serve dispense
           try {
             fileServlet.doGet (request, response);
           } catch (Exception e) {
             response.sendRedirect (“/login.html”);
             System.out.println(“error”+e);
           }
         }
       }
    }
  }
  protected boolean verifyPassword(String theuser, String password) {
    String originalPassword=null;
    try {
      Connection con=getConnection();
```

```
    Statement stmt= con.createStatement();
    stmt.executeQuery("select password from registration
                                    where theuser='"+theuser+"'");
    ResultSet rs = stmt.getResultSet();
    if(rs.next()) {
       originalPassword=rs.getString(1);
    }
    stmt.close();
    if(originalPassword.equals(password)) {
      return true;
    } else {
      return false;
    }
  } catch (Exception e){
    System.out.println("Exception: verifyPassword="+e);
    return false;
  }
}
protected Session startSession (String theuser, String password,
                                HttpServletResponse response) {
  Session session = null;
  if( verifyPassword(theuser, password) ) {
    // Create a session
    session = new Session (theuser);
    session.setExpires (sessionTimeout+System.currentTimeMillis());
    sessionCache.put (session);
    // Create a client cookie
    Cookie c = new Cookie("JDCAUCTION", String.valueOf (session.getId()));
    c.setPath ("/");
    c.setMaxAge (-1);
    c.setDomain (domain);
    response.addCookie (c);
  }
  return session;
}
private Session validateSession (HttpServletRequest request,
                                 HttpServletResponse response) {
  Cookie c[] = request.getCookies();
  Session session = null;
  if(c != null ) {
     for(int i=0; i < c.length && session == null; i++ ) {
       if(c[i].getName().equals("JDCAUCTION")) {
         String key = String.valueOf (c[i].getValue());
         session=sessionCache.get (key);
       }
     }
   }
   return session;
}
protected void endSession (Session session) {
  synchronized (sessionCache) {
    SessionCache.remove (session);
  }
```

```
  }
  private void setNoCache (HttpServletRequest request,
                           HttpServletResponse response) {
    if(request.getProtocol().compareTo ("HTTP/1.0") == 0) {
      response.setHeader ("Pragma", "no-cache");
    } else if (request.getProtocol().compareTo ("HTTP/1.1") == 0) {
      response.setHeader ("Cache-Control", "no-cache");
    }
    response.setDateHeader ("Expires", 0);
  }
}
```

Session (Servlets)

```
package login;
import java.util.Random;
import java.net.*;

public class Session {
  private String theuser;
  private int id;
  private long expires;
  public Session (String theuser) {
    this.theuser = URLEncoder.encode (theuser);
    // Choose a quasi unique number for this session
    id = Math.abs(new Random (System.currentTimeMillis()).nextInt());
    expires = 0;
  }
  synchronized public void setExpires (long e) {
    expires = e;
  }
  public long getExpires() {
   return expires;
  }
  public String key() {
    return String.valueOf (id);
    }
    public String getEncodedUser() {
      return theuser;
    }
    public String getUser() {
      try {
        return URLDecoder.decode (theuser);
      } catch (Exception e) {
        return null;
      }
   }
   public int getId() {
      return id;
   }
}
```

SessionCache (Servlets)

```
package login;
import java.util.Hashtable;
import java.util.Enumeration;

// This class keeps a copy in memory of everyone logged
// onto the system. It has a background threads that removes sessions
// whose timeout has expired. The timeout is in milliseconds
// since 1970

class SessionCache implements  Runnable {
  private Hashtable sessionCache;
  private long flush;
  private Thread reaper;
  SessionCache (long flush) {
    this.flush=flush;
    sessionCache=new Hashtable (100);
    reaper=new Thread (this);
    reaper.setPriority (Thread.MIN_PRIORITY);
    reaper.start();
  }
  public void run() {
    while(true) {
      try {
        Thread.sleep (flush);
        Enumeration sessions;
        Session s;
        long expire;
        expire=System.currentTimeMillis();
        sessions=sessionCache.elements();
        while(sessions.hasMoreElements() ) {
          s=(Session)sessions.nextElement();
          if(expire >= s.getExpires()) {
             sessionCache.remove (s.key());
           }
         }
       } catch (Exception e) {
         return;
       }
     }
    }
    Session put (Session s) {
      return (Session)sessionCache.put (s.key(), s);
    }
    Session get (String key) {
      return (Session)sessionCache.get (key);
    }
    Enumeration elements () {
      return sessionCache.elements();
    }
    Enumeration keys () {
      return sessionCache.keys();
```

```
    }
    void remove (Session s) {
      sessionCache.remove (s.key());
    }
}
```

5: JNI Technology

The Java™ platform is relatively new, which means there could be times when you will need to integrate programs written in the Java programming language with existing non-Java language services, API toolkits, and programs. The Java platform provides the Java Native Interface (JNI) to help ease this type of integration.

The JNI defines a standard naming and calling convention so the Java virtual machine can locate and invoke native methods. In fact, JNI is built into the Java virtual machine so the Java virtual machine can invoke local system calls to perform input and output, graphics, networking, and threading operations on the host operating system.

This chapter explains how to use JNI in programs written in the Java programming language to call any libraries on the local machine, call Java methods from inside native code, and explains how to create and run a Java virtual machine instance. To show how you can put JNI to use, the examples in this chapter include integrating JNI with the Xbase C++ database API, and how you can call a mathematical function. Xbase (`http://www.startech.keller.tx.us/xbase/xbase.html`) has sources you can download.

Covered in this Chapter

- JNI Example (page 207)
- Strings and Arrays (page 212)
- Other Programming Issues (page 220)

JNI Example

The ReadFile example program shows how you can use the Java Native Interface (JNI) to invoke a native method that makes C function calls to map a file into memory.

About the Example

You can call code written in any programming language from a program written in the Java programming language by declaring a native Java method, loading the library that contains

the native code, and calling the native method. The ReadFile source code below does exactly that.

However, successfully running the program requires a few additional steps beyond compiling the Java programming language source file. After you compile, but before you run the example, you have to generate a header file. The native code implements the function definitions contained in the generated header file and implements the business logic as well. The following sections walk through all the steps.

```
import java.util.*;

class ReadFile {
//Native method declaration
  native byte[] loadFile(String name);
//Load the library
  static {
    System.loadLibrary("nativelib");
  }

  public static void main(String args[]) {
    byte buf[];
//Create class instance
    ReadFile mappedFile=new ReadFile();
//Call native method to load ReadFile.java
    buf=mappedFile.loadFile("ReadFile.java");
//Print contents of ReadFile.java
    for(int i=0;i<buf.length;i++) {
      System.out.print((char)buf[i]);
    }
  }
}
```

Native Method Declaration

The native declaration provides the bridge to run the native function in the Java virtual machine. In this example, the loadFile function maps to a C function called Java_ReadFile_loadFile. The function implementation accepts a String that represents a file name and returns the contents of that file in the byte array.

```
native byte[] loadFile(String name);
```

Load the Library

The library containing the native code implementation is loaded by a call to System.loadLibrary(). Placing this call in a static initializer ensures this library is only loaded once per class. The library can be loaded outside of the static block if your application requires it.

Note: You might need to configure your environment so the loadLibrary method can find your native code library. See Compile the Dynamic or Shared Object Library (page 211) for this information.

```
//API Ref :static void loadLibrary(String libraryname)
    static {
      System.loadLibrary("nativelib");
    }
```

Compile the Program

To compile the program, run the javac compiler command as you normally would:

```
javac ReadFile.java
```

Next, you need to generate a header file with the native method declaration and implement the native method to call the C functions for loading and reading a file.

Generate the Header File

To generate a header file, run the javah command on the ReadFile class. In this example, the generated header file is named ReadFile.h. It provides a method signature that you have to use when you implement the loadfile native function.

```
javah -jni ReadFile
```

Method Signature

The ReadFile.h header file defines the interface to map the Java method to the native C function. It uses a method signature to map the arguments and return value of the Java mappedfile.loadFile method to the loadFile native method in the nativelib library. Here is the loadFile native method mapping (method signature):

```
/*
 * Class:     ReadFile
 * Method:    loadFile
 * Signature: (Ljava/lang/String;)[B
 */
JNIEXPORT jbyteArray JNICALL Java_ReadFile_loadFile
  (JNIEnv *, jobject, jstring);
```

The method signature parameters function as follows:

- JNIEnv *: A pointer to the JNI environment. This pointer is a handle to the current thread in the Java virtual machine, and contains mapping and other housekeeping information.
- jobject: A reference to the object that called this native code. If the calling method is static, this parameter would be type jclass instead of jobject.
- jstring: The parameter supplied to the native method. In this example, it is the name of the file to be read.

Implement the Native Method

In this native C source file, the loadFile definition is a copy and paste of the C declaration contained in ReadFile.h. The definition is followed by the native method implementation. JNI provides a mapping for both C and C++ by default.

```
    /* file nativelib.c */
    #include <jni.h>
    #include <sys/types.h>
    #include <sys/ipc.h>
    #include <sys/shm.h>
    #include <sys/mman.h>
    #include <sys/stat.h>
    #include <fcntl.h>
    #include <unistd.h>

    JNIEXPORT jbyteArray JNICALL Java_ReadFile_loadFile
      (JNIEnv * env, jobject jobj, jstring name) {
        caddr_t m;
        jbyteArray jb;
        jboolean iscopy;
        struct stat finfo;
//API Ref :const char* GetStringUTFChars(JNIEnv *env, jstring string, jboolean
          *iscopy)
        const char *mfile = (*env)->GetStringUTFChars(env, name, &iscopy);
        int fd = open(mfile, O_RDONLY);
        if (fd == -1) {
          printf("Could not open %s\n", mfile);
        }
        lstat(mfile, &finfo);
        m = mmap((caddr_t) 0, finfo.st_size,
                    PROT_READ, MAP_PRIVATE, fd, 0);
        if (m == (caddr_t)-1) {
          printf("Could not mmap %s\n", mfile);
          return(0);
        }
//API Ref :jbyteArray NewByteArray(JNIEnv *env, jsize length)
        jb=(*env)->NewByteArray(env, finfo.st_size);
//API Ref :SetByteArrayRegion(JNIEnv *env, jbyteArray array, jsize startelement,
          jsize length, jbyte *buffer)
```

```
    (*env)->SetByteArrayRegion(env, jb, 0, finfo.st_size, (jbyte *)m);
     close(fd);
     (*env)->ReleaseStringUTFChars(env, name, mfile);
     return (jb);
}
```

You can approach calling an existing C function instead of implementing one in one of two ways:

1. Map the name generated by JNI to the existing C function name. The Other Programming Issues (page 220) section shows how to map between Xbase database functions and Java programming language code
2. Use the shared stubs code available from the JNI page (`http://java.sun.com/products/jdk/faq/jnifaq.html`) on the java.sun.com web site.

Compile the Dynamic or Shared Object Library

The library needs to be compiled as a dynamic or shared object library so it can be loaded at runtime. Static or archive libraries are compiled into an executable and cannot be loaded at runtime. The shared object or dynamic library for the loadFile example is compiled on various platforms as follows:

GNU C/Linux

```
gcc  -o libnativelib.so -shared -Wl,-soname,libnative.so  -I/export/home/jdk1.2/
include -I/export/home/jdk1.2/include/linux nativelib.c -static -lc
```

SunPro C/Solaris

```
cc -G -so libnativelib.so -I/export/home/jdk1.2/include -I/export/home/jdk1.2/
include/solaris nativelib.c
```

GNU C++/Linux with Xbase

```
g++ -o libdbmaplib.so -shared -Wl,-soname,libdbmap.so  -I/export/home/jdk1.2/
include -I/export/home/jdk1.2/include/linux dbmaplib.cc -static -lc -lxbase
```

Win32/WinNT/Win2000

```
cl -Ic:/jdk1.2/include -Ic:/jdk1.2/include/win32 -LD nativelib.c -
Felibnative.dll
```

Run the Example

To run the example, the Java virtual machine needs to find the native JNI library that was created. To do this, set the library path to the current directory as follows:

Unix or Linux:

```
LD_LIBRARY_PATH=.
export LD_LIBRARY_PATH
```

Windows NT/2000/95:

```
set PATH=%path%;.
```

With the library path properly specified for your platform, invoke the program as you normally would with the interpreter command:

```
java ReadFile
```

Strings and Arrays

This section explains how to pass string and array data between a program written in the Java programming language and a program written in another languages.

Passing Strings

The String object in the Java programming language, which is represented as jstring in Java Native Interface (JNI), is a 16 bit unicode string. In C a string is by default constructed from 8 bit characters. So, to access a Java String object passed to a C or C++ function or return a C or C++ string to a Java method, you need to use JNI conversion functions in your native method implementation.

The GetStringUTFChar function retrieves 8-bit characters from a 16-bit jstring using the Unicode Transformation Format (UTF). UTF represents Unicode as a string of 8 or 16 bit characters without losing any information. The third parameter GetStringUTFChar returns the result JNI_TRUE if it made a local copy of the jstring or JNI_FALSE otherwise.

C Version:

```
(*env)->GetStringUTFChars(env, name, iscopy)
```

C++ Version:

```
env->GetStringUTFChars(name, iscopy)
```

The following C JNI function converts an array of C characters to a jstring:

```
//API Ref :jstring NewStringUTF(JNIEnv *env, const char *bytes)
```

```
    (*env)->NewStringUTF(env, lastfile)
```

The example below converts the lastfile[80] C character array to a jstring, which is returned to the calling Java method:

```
    static char lastfile[80];

    JNIEXPORT jstring JNICALL Java_ReadFile_lastFile
      (JNIEnv *env, jobject jobj) {
//API Ref :jstring NewStringUTF(JNIEnv *env, const char *bytes)
       return((*env)->NewStringUTF(env, lastfile));
    }
```

To let the Java virtual machine know you are finished with the UTF representation, call the ReleaseStringUTFChars conversion function as shown below. The second argument is the original jstring value used to construct the UTF representation, and the third argument is the reference to the local representation of that String.

```
    (*env)->ReleaseStringUTFChars(env, name, mfile);
```

If your native code can work with Unicode, without needing the intermediate UTF representation, call the GetStringChars function to retrieve the unicode string, and release the reference with a call to ReleaseStringChars:

```
    JNIEXPORT jbyteArray JNICALL Java_ReadFile_loadFile
      (JNIEnv * env, jobject jobj, jstring name) {
        caddr_t m;
        jbyteArray jb;
        struct stat finfo;
        jboolean iscopy;
//API Ref :const jchar *GetStringChars(JNIEnv *env,  jstring string, jboolean
          *iscopy)
        const jchar *mfile = (*env)->GetStringChars(env, name, &iscopy);
      //...
//API Ref :void ReleaseStringChars(JNIEnv *env, jstring string, const jchar *chars)
        (*env)->ReleaseStringChars(env, name, mfile);
```

Passing Arrays

In the example presented in the last section, the loadFile native method returns the contents of a file in a byte array, which is a primitive type in the Java programming language. You can retrieve and create primitive types in the Java programming language by calling the appropriate TypeArray function.

For example, to create a new array of floats, call NewFloatArray, or to create a new array of bytes, call NewByteArray. This naming scheme extends to retrieving elements from, adding elements to, and changing elements in the array. To get elements from an array of bytes, call

GetByteArrayElements. To add elements to or change elements in the array, call Set<type>ArrayElements.

The GetByteArrayElements function affects the entire array. To work on a portion of the array, call GetByteArrayRegion instead. There is only a Set<type>ArrayRegion function for changing array elements. However the region could be of size `array.length`, which is equivalent to the non-existent Sete<type>ArrayElements.

Native Code Type	**Functions Used**

`jboolean`	`NewBooleanArray`
	`GetBooleanArrayElements`
	`GetBooleanArrayRegion/SetBooleanArrayRegion`
	`ReleaseBooleanArrayRegion`

`jbyte`	`NewByteArray`
	`GetByteArrayElements`
	`GetByteArrayRegion/SetByteArrayRegion`
	`ReleaseByteArrayRegion`

`jchar`	`NewCharArray`
	`GetCharArrayElements`
	`GetCharArrayRegion/SetCharArrayRegion`
	`ReleaseCharArrayRegion`

`jdouble`	`NewDoubleArray`
	`GetDoubleArrayElements`
	`GetDoubleArrayRegion/SetDoubleArrayRegion`
	`ReleaseDoubleArrayRegion`

`jfloat`	`NewFloatArray`
	`GetFloatArrayElements`
	`GetFloatArrayRegion/SetFloatArrayRegion`
	`ReleaseFloatArrayRegion`

`jint`	`NewIntArray`
	`GetIntArrayElements`
	`GetIntArrayRegion/SetIntArrayRegion`
	`ReleaseIntArrayRegion`

`jlong`	`NewLongArray`
	`GetLongArrayElements`
	`GetLongArrayRegion/SetLongArrayRegion`
	`ReleaseLongArrayRegion`

`jobject`	`NewObjectArray` `GetObjectArrayElement/SetObjectArrayElement`

Native Code Type	Functions Used
jshort	NewShortArray GetShortArrayElements GetShortArrayRegion/SetShortArrayRegion ReleaseShortArrayRegion

In the loadFile native method from the example in the previous section, the entire array is updated by specifying a region that is the size of the file being read in:

```
    jbyteArray jb;
//API Ref :jbyteArray NewByteArray(JNIEnv *env, jsize length)
    jb=(*env)->NewByteArray(env, finfo.st_size);
//API Ref :SetByteArrayRegion(JNIEnv *env, jbyteArray array, jsize startelement,
          jsize length, jbyte *buffer)
    (*env)->SetByteArrayRegion(env, jb, 0, finfo.st_size, (jbyte *)m);
    close(fd);
```

The array is returned to the calling Java programming language method, which in turn, garbage collects the reference to the array when it is no longer used. The array can be explicitly freed with the following call.

```
//API Ref :void ReleaseByteArrayElements(JNIEnv *env, jbyteArray array, jbyte
          *elems, jint mode)
    (*env)-> ReleaseByteArrayElements(env, jb, (jbyte *)m, 0);
```

The last argument to the ReleaseByteArrayElements function above can have the following values:

- 0: Updates to the array from within the C code are reflected in the Java programming language copy.
- JNI_COMMIT: The Java programming language copy is updated, but the local jbyteArray is not freed.
- JNI_ABORT: Changes are not copied back, but the jbyteArray is freed. The value is used only if the array is obtained with a get mode of JNI_TRUE meaning the array is a copy.

See Memory Issues (page 227) for more information on memory management.

Pinning Array

When retrieving an array, you can specify if this is a copy (JNI_TRUE) or a reference to the array residing in your Java program (JNI_FALSE). If you use a reference to the array, you will want the array to stay where it is in the Java heap and not get moved by the garbage col-

lector when it compacts heap memory. To prevent the array references from being moved, the Java virtual machine pins the array into memory. Pinning the array ensures that when the array is released, the correct elements are updated in the Java virtual machine.

In the loadfile native method example from the previous section, the array is not explicitly released. One way to ensure the array is garbage collected when it is no longer needed is to call a Java method, pass the byte array instead, and free the local array copy. This technique is shown in the section on MultiDimensional Arrays (page 217).

Object Arrays

You can store any Java object in an array with the NewObjectArray and SetObjectArrayElement function calls. The main difference between an object array and an array of primitive types is that when constructing a jobjectArray, the Java class is used as a parameter.

This next C++ example shows how to call NewObjectArray to create an array of String objects. The size of the array is set to five, the class definition is returned from a call to FindClass, and the elements of the array are initialized with an empty string. The elements of the array are updated by calling SetObjectArrayElement with the position and value to put in the array.

```
/*  ReturnArray.C  */
#include <jni.h>
#include "ArrayHandler.h"

  JNIEXPORT jobjectArray JNICALL Java_ArrayHandler_returnArray
                                    (JNIEnv *env, jobject jobj){
    jobjectArray ret;
    int i;
    char *message[5]= {"first", "second", "third", "fourth", "fifth"};
    ret = (jobjectArray)env->NewObjectArray(5,
//API Ref :jclass FindClass(JNIenv *env, const char *name)
          env->FindClass("java/lang/String"),
//API Ref :jstring NewStringUTF(JNIEnv *env, const char *bytes)
          env->NewStringUTF(""));
    for(i=0;i<5;i++) {
//API Ref :void SetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize index,
       jobject value)
         env->SetObjectArrayElement(ret,i,env->NewStringUTF(message[i]));
    }
    return(ret);
  }
```

The Java class that calls this native method is as follows:

```
// ArrayHandler.java
public class ArrayHandler {
    public native String[] returnArray();
    static{
//API Ref :static void loadLibrary(String libraryname)
```

```
        System.loadLibrary("nativelib");
    }
    public static void main(String args[]) {
      String ar[];
      ArrayHandler ah= new ArrayHandler();
      ar = ah.returnArray();
      for(int i=0; i<5; i++) {
         System.out.println("array element"+i+ "=" + ar[i]);
      }
    }
}
```

MultiDimensional Arrays

You might need to call existing numerical and mathematical libraries such as the linear algebra library CLAPACK/LAPACK or other matrix crunching programs from your Java program using native methods. Many of these libraries and programs use two-dimensional and higher order arrays.

In the Java programming language, any array that has more than one dimension is treated as an array of arrays. For example, a two-dimensional integer array is handled as an array of integer arrays. The array is read horizontally, or in what is also termed as row order.

Other languages such as FORTRAN use column ordering so extra care is needed if your program hands a Java array to a FORTRAN function. Also, the array elements in an application written in the Java programming language are not guaranteed to be contiguous in memory. Some numerical libraries use the knowledge that the array elements are stored next to each other in memory to perform speed optimizations, so you might need to make an additional local copy of the array to pass to those functions.

The next example passes a two-dimensional array to a native method which then extracts the elements, performs a calculation, and calls a Java method to return the results. The array is passed as an object array that contains an array of jints. The individual elements are extracted by first retrieving a jintArray instance from the object array by calling GetObjectArrayElement, and then extracting the elements from the jintArray row.

The example uses a fixed-size matrix. If you do not know the size of the array being used, the GetArrayLength(array) function returns the size of the outermost array. You will need to call the GetArrayLength(array) function on each dimension of the array to discover the total size of the array. The new array sent back to the program written in the Java language is built in reverse. First, a jintArray instance is created and that instance is set in the object array by calling SetObjectArrayElement.

```
    public class ArrayManipulation {
      private int arrayResults[][];
      Boolean lock=new Boolean(true);
      int arraySize=-1;

      public native void manipulateArray(int[][] multiplier, Boolean lock);
      static{
//API Ref :static void loadLibrary(String libraryname)
        System.loadLibrary("nativelib");
      }
      public void sendArrayResults(int results[][]) {
        arraySize=results.length;
        arrayResults=new int[results.length][];
        System.arraycopy(results,0,arrayResults, 0, arraySize);
      }
      public void displayArray() {
        for (int i=0; i<arraySize; i++) {
          for(int j=0; j <arrayResults[i].length;j++) {
            System.out.println("array element "+i+","+j+ "= " + arrayResults[i][j]);
          }
        }
      }
      public static void main(String args[]) {
        int[][] ar = new int[3][3];
        int count=3;
        for(int i=0;i<3;i++) {
          for(int j=0;j<3;j++) {
            ar[i][j]=count;
          }
          count++;
        }
        ArrayManipulation am= new ArrayManipulation();
        am.manipulateArray(ar, am.lock);
        am.displayArray();
      }
    }

    #include <jni.h>
    #include <iostream.h>
    #include "ArrayManipulation.h"

    JNIEXPORT void
         JNICALL Java_ArrayManipulation_manipulateArray
                (JNIEnv *env, jobject jobj, jobjectArray elements, jobject lock){
      jobjectArray ret;
      int i,j;
      jint arraysize;
      int asize;
      jclass cls;
      jmethodID mid;
      jfieldID fid;
```

```
      long localArrayCopy[3][3];
      long localMatrix[3]={4,4,4};

      for(i=0; i<3; i++) {
//API Ref :jobject GetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize
          index)
         jintArray oneDim= (jintArray)env->GetObjectArrayElement(elements, i);
//API Ref :jint* GetIntArrayElements(JNIEnv *env, jintArray array, jboolean *iscopy)
         jint *element=env->GetIntArrayElements(oneDim, 0);
         for(j=0; j<3; j++) {
            localArrayCopy[i][j]= element[j];
         }
      }
    // With the C++ copy of the array,
    // process the array with LAPACK, BLAS, etc.

      for (i=0;i<3;i++) {
        for (j=0; j<3 ; j++) {
          localArrayCopy[i][j]=
            localArrayCopy[i][j]*localMatrix[i];
        }
      }
    // Create array to send back
//API Ref :jintArray NewIntArray(JNIEnv *env, jsize length)
      jintArray row= (jintArray)env->NewIntArray(3);
//API Ref :jclass GetObjectClass(JNIEnv *env, jobject obj)
      ret=(jobjectArray)env->NewObjectArray(3, env->GetObjectClass(row), 0);

      for(i=0;i<3;i++) {
        row= (jintArray)env->NewIntArray(3);
//API Ref :SetIntArrayRegion(JNIEnv *env, jintArray array, jsize startelement, jsize
          length, jint *buffer)
        env->SetIntArrayRegion((jintArray)row,(jsize)0,3,(jint *)localArrayCopy[i]);
        env->SetObjectArrayElement(ret,i,row);
      }
      cls=env->GetObjectClass(jobj);
//API Ref :jmethodID GetMethodId(JNIEnv *env, jclass class, const char *methodname,
          const char *methodsig)
      mid=env->GetMethodID(cls, "sendArrayResults",
                                "([[I)V");
      if (mid == 0) {
        cout <<"Can't find method sendArrayResults";
        return;
      }
//API Ref :void ExceptionClear(JNIEnv *env)
      env->ExceptionClear();
//API Ref :jint MonitorEnter(JNIEnv *env, jobject object)
      env->MonitorEnter(lock);
//API Ref :CallVoidMethod(JNIEnv *env, jobject object, jmethodId methodid, object
          arg1)
      env->CallVoidMethod(jobj, mid, ret);
//API Ref :jint MonitorExit(JNIEnv *env, jobject object)
      env->MonitorExit(lock);
```

```
//API Ref :jthrowable ExceptionOccurred(JNIEnv *env)
      if(env->ExceptionOccurred()) {
        cout << "error occured copying array back" << endl;
//API Ref :void ExceptionDescribe(JNIEnv *env)
        env->ExceptionDescribe();
        env->ExceptionClear();
      }
//API Ref :jfieldID GetFieldID(JNIEnv *env, jclass class, const char *fieldname,
          const char *fieldsig)
      fid=env->GetFieldID(cls, "arraySize",  "I");
      if (fid == 0) {
        cout <<"Can't find field arraySize";
        return;
      }
      asize=env->GetIntField(jobj,fid);
      if(!env->ExceptionOccurred()) {
        cout<< "Java array size=" << asize << endl;
      } else {
        env->ExceptionClear();
      }
      return;
    }
```

Other Programming Issues

This section presents information on accessing classes, methods, and fields, and covers threading, memory, and Java virtual machine issues.

Language issues

So far, the native method examples have covered calling standalone C and C++ functions that either return a result or modify parameters passed into the function. However, C++ like the Java programming language, uses instances of classes. If you create a class in one native method, the reference to this class does not have an equivalent class in the Java programming language. This makes it difficult to call functions on the C++ class that was first created.

One way to handle this situation is to keep a record of the C++ class reference and pass that back to a proxy or to the calling program. To ensure the C++ class persists across native method calls, use the C++ new operator to create a reference to the C++ object on the heap.

The following code provides a mapping between the Xbase database and Java code. The Xbase database has a C++ API and uses an initialization class to perform subsequent database operations. When the class object is created, a pointer to this object is returned as a Java int value. You can use a long or larger value for machines with greater than 32 bits.

```
    public class CallDB {
      public native int initdb();
      public native short opendb(String name, int ptr);
      public native short GetFieldNo(String fieldname, int ptr);

      static {
//API Ref :static void loadLibrary(String libraryname)
        System.loadLibrary("dbmaplib");
      }
      public static void main(String args[]) {
        String prefix=null;
        CallDB db=new CallDB();
        int res=db.initdb();
        if(args.length>=1) {
          prefix=args[0];
        }
        System.out.println(db.opendb("MYFILE.DBF", res));
        System.out.println(db.GetFieldNo("LASTNAME", res));
        System.out.println(db.GetFieldNo("FIRSTNAME", res));
       }
    }
```

The return result from the call to the initdb native method, the int value, is passed to subsequent native method calls. The native code included in the dbmaplib.cc library dereferences the Java object passed in as a parameter and retrieves the object pointer. The line xbDbf* Myfile=(xbDbf*)ptr; casts the int pointer value to be a pointer of Xbase type xbDbf.

```
    #include <jni.h>
    #include <xbase/xbase.h>
    #include "CallDB.h"

    JNIEXPORT jint JNICALL Java_CallDB_initdb(JNIEnv *env, jobject jobj) {
      xbXBase* x;
      x= new xbXBase();
      xbDbf* Myfile;
      Myfile =new xbDbf(x);
      return ((jint)Myfile);
    }
    JNIEXPORT jshort JNICALL Java_CallDB_opendb(JNIEnv *env, jobject jobj,
                                                jstring dbname, jint ptr) {
      xbDbf* Myfile=(xbDbf*)ptr;
      return((*Myfile).OpenDatabase( "MYFILE.DBF"));
    }
    JNIEXPORT jshort JNICALL Java_CallDB_GetFieldNo(JNIEnv *env, jobject jobj,
                                                    jstring fieldname, jint ptr) {
      xbDbf* Myfile=(xbDbf*)ptr;
      return((*Myfile).GetFieldNo(env->GetStringUTFChars(fieldname,0)));
    }
```

Calling Methods

The section on arrays highlighted some reasons for calling Java programming language methods from within native code; for example, when you need to free the result you intend to return. Other uses for calling Java native methods from within your native code are if you need to return more than one result or you just simply want to modify Java programming language values from within native code. Calling a Java programming language method from within native code involves the following three steps:

1. Retrieve a class reference
2. Retrieve a method identifier
3. Call the Methods

Retrieve a Class Reference

The first step is to retrieve a reference to the class that contains the methods you want to access. To retrieve a reference, you can either use the FindClass method or access the jobject or jclass argument to the native method.

Use the FindClass method:

```
JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jobject jobj){
  jclass cls = (*env)->FindClass(env, "ClassName");
}
```

Use the jobject argument:

```
JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jobject jobj){
  jclass cls=(*env)->GetObjectClass(env, jobj);
}
```

or

Use the jclass argument:

```
JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jclass jcls){
  jclass cls=jcls;
}
```

Retrieve a Method Identifier

Once the class has been obtained, the second step is to call the GetMethodID function to retrieve an identifier for a method you select in the class. The identifier is needed when calling the method of that class instance. Because the Java programming language supports method overloading, you also need to specify the method signature you want to call. To find out what signature your Java method uses, run the javap command as follows:

```
javap -s Class
```

The method signature used is displayed as a comment after each method declaration as shown here:

```
bash# javap -s ArrayHandler
Compiled from ArrayHandler.java
public class ArrayHandler extends java.lang.Object {
  java.lang.String arrayResults[];
   /*   [Ljava/lang/String;   */
  static {};
   /*   ()V   */
  public ArrayHandler();
   /*   ()V   */
  public void displayArray();
   /*   ()V   */
  public static void main(java.lang.String[]);
   /*   ([Ljava/lang/String;)V   */
  public native void returnArray();
   /*   ()V   */
  public void sendArrayResults(java.lang.String[]);
   /*   ([Ljava/lang/String;)V   */
}
```

Use the GetMethodID function to call instance methods in an object instance, or use the GetStaticMethodID function to call static method. Their argument lists are the same.

Call the Methods

Third, the matching instance method is called using a Call<type>Method function. The type value can be Void, Object, Boolean, Byte, Char, Short, Int, Long, Float, or Double.

The parameters to the method can be passed as a comma-separated list, an array of values to the Call<type>MethodA function, or as a va_list. The va_list is a construct often used for variable argument lists in C. Call<*type*>MethodV is the function used to pass a va_list ().

Static methods are called in a similar way except the method naming includes an additional Static identifier, CallStaticByteMethodA, and the jclass value is used instead of jobject.

The next example returns the object array by calling the sendArrayResults method from the ArrayHandler class.

```
// ArrayHandler.java
public class ArrayHandler {
  private String arrayResults[];
  int arraySize=-1;

  public native void returnArray();
  static{
//API Ref :static void loadLibrary(String libraryname)
```

```
      System.loadLibrary("nativelib");
    }
    public void sendArrayResults(String results[]) {
      arraySize=results.length;
      arrayResults=new String[arraySize];
      System.arraycopy(results,0,arrayResults,0,arraySize);
    }
    public void displayArray() {
      for (int i=0; i<arraySize; i++) {
        System.out.println("array element "+i+ "= " + arrayResults[i]);
      }
    }
    public static void main(String args[]) {
      String ar[];
      ArrayHandler ah= new ArrayHandler();
      ah.returnArray();
      ah.displayArray();
    }
  }
```

The native C++ code is defined as follows:

```
  // file: nativelib.cc
  #include <jni.h>
  #include <iostream.h>
  #include "ArrayHandler.h"

  JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jobject jobj){
    jobjectArray ret;
    int i;
    jclass cls;
    jmethodID mid;
    char *message[5]= {"first", "second", "third", "fourth", "fifth"};
//API Ref :jarray NewObjectArray(JNIEnv *env, jsize length, jclass elementClass,
        jobject initialElement)
    ret=(jobjectArray)env->NewObjectArray(5,
        env->FindClass("java/lang/String"),
        env->NewStringUTF(""));
    for(i=0;i<5;i++) {
      env->SetObjectArrayElement(ret,i,env->NewStringUTF(message[i]));
    }
    cls=env->GetObjectClass(jobj);
    mid=env->GetMethodID(cls, "sendArrayResults", "([Ljava/lang/String;)V");
    if (mid == 0) {
      cout <<“Can't find method sendArrayResults";
      return;
    }
    env->ExceptionClear();
    env->CallVoidMethod(jobj, mid, ret);
    if(env->ExceptionOccurred()) {
      cout << "error occured copying array back" <<endl;
```

```
    env->ExceptionDescribe();
    env->ExceptionClear();
  }
  return;
}
```

To build this on Linux, run the following commands:

```
javac ArrayHandler.java
javah -jni ArrayHandler

g++  -o libnativelib.so -shared -Wl,-soname,libnative.so -I/export/home/jdk1.2/
include -I/export/home/jdk1.2/include/linux nativelib.cc  -lc
```

If you want to specify a super class method to, for example, call the parent constructor, you can do so by calling the CallNonvirtual<*type*>Method functions. One important point when calling Java methods or fields from within native code is you need to catch any raised exceptions. The ExceptionClear function clears any pending exceptions while the ExceptionOccured function checks to see if an exception has been raised in the current JNI session.

Accessing Fields

Accessing Java fields from within native code is similar to calling Java methods. However, the set or field is retrieved with a field ID, instead of a method ID.

The first thing you need to do is retrieve a field ID. You can use the GetFieldID function, but specify the field name and signature in place of the method name and signature. Once you have the field ID, call a Get<type>Field function to set the field value. The <type> is the same as the native type being returned except the *j* is dropped and the first letter is capitalized. For example, the <type> value is Int for native type jint, and Byte for native type jbyte.

The Get<type>Field function result is returned as the native type. For example, to retrieve the arraySize field in the ArrayHandler class, call GetIntField as shown in the following example.

The field can be set by calling the env->SetIntField(jobj, fid, arraysize) functions. Static fields can be set by calling SetStaticIntField(jclass, fid, arraysize) and retrieved by calling GetStaticIntField(jclass, fid).

```
#include <jni.h>
#include <iostream.h>
#include "ArrayHandler.h"

JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jobject jobj){
    jobjectArray ret;
    int i;
    jint arraysize;
    jclass cls;
```

```
    jmethodID mid;
    jfieldID fid;
    char *message[5]= {"first", "second", "third", "fourth", "fifth"};

    ret=(jobjectArray)env->NewObjectArray(5,
        env->FindClass("java/lang/String"),
        env->NewStringUTF(""));
    for(i=0;i<5;i++) {
      env->SetObjectArrayElement(ret,i,env->NewStringUTF(message[i]));
    }
    cls=env->GetObjectClass(jobj);
    mid=env->GetMethodID(cls, "sendArrayResults", "([Ljava/lang/String;)V");
    if (mid == 0) {
        cout <<“Can't find method sendArrayResults";
        return;
    }
    env->ExceptionClear();
    env->CallVoidMethod(jobj, mid, ret);
    if(env->ExceptionOccurred()) {
       cout << "error occured copying array back" << endl;
       env->ExceptionDescribe();
       env->ExceptionClear();
    }
    fid=env->GetFieldID(cls, "arraySize",  "I");
    if (fid == 0) {
        cout <<“Can't find field arraySize";
        return;
    }
    arraysize=env->GetIntField(jobj, fid);
    if(!env->ExceptionOccurred()) {
       cout<< "size=" << arraysize << endl;
    } else {
       env->ExceptionClear();
    }
    return;
}
```

Threads and Synchronization

Although the native library is loaded once per class, individual threads in an application written in the Java programming language use their own interface pointer when calling the native method. If you need to restrict access to a Java object from within native code, you can either ensure that the Java methods you call have explicit synchronization or you can use the JNI MonitorEnter and MonitorExit functions.

In the Java programming language, code is protected by a monitor whenever you specify the synchronized keyword, and the monitor enter and exit routines are normally hidden from the application developer. In JNI, you need to explicitly delineate the entry and exit points of thread safe code.

The following example uses a Boolean object to restrict access to the CallVoidMethod function.

```
env->ExceptionClear();
env->MonitorEnter(lock);
env->CallVoidMethod(jobj, mid, ret);
env->MonitorExit(lock);
if(env->ExceptionOccurred()) {
  cout << "error occured copying array back" << endl;
  env->ExceptionDescribe();
  env->ExceptionClear();
}
```

You may find that in cases where you want access to a local system resource like an MFC window handle or message queue, it is better to use one `Java.lang.Thread` and access the local threaded native event queue or messaging system from within the native code.

Memory Issues

By default, JNI uses local references when creating objects inside a native method. This means when the method returns, the references are eligible to be garbage collected. If you want an object to persist across native method calls, use a global reference instead. A global reference is created from a local reference by calling NewGlobalReference on the local reference.

You can explicitly mark a reference for garbage collection by calling DeleteGlobalRef on the reference. You can also create a weak style Global reference that is accessible outside the method, but can be garbage collected. To create one of these references, call NewWeakGlobalRef and DeleteWeakGlobalRef to mark the reference for garbage collection.

You can even explicitly mark a local reference for garbage collection by calling the env->DeleteLocalRef(localobject) method. This is useful if you are using a large amount of temporary data.

```
static jobject stringarray=0;

JNIEXPORT void JNICALL Java_ArrayHandler_returnArray(JNIEnv *env, jobject jobj){
    jobjectArray ret;
    int i;
    jint arraysize;
    int asize;
    jclass cls, tmpcls;
    jmethodID mid;
    jfieldID fid;
    char *message[5]= {"first", "second", "third", "fourth", "fifth"};
```

```
        ret=(jobjectArray)env->NewObjectArray(5,
            env->FindClass("java/lang/String"),
            env->NewStringUTF(""));
      //Make the array available globally
//API Ref :jobject NewGlobalRef(JNIEnv *env, jobject object)
        stringarray=env->NewGlobalRef(ret);
      //Process array
      // ...
      //clear local reference when finished..
//API Ref :void DeleteLocalRef(JNIEnv *env, jobject localref)
        env->DeleteLocalRef(ret);
      }
```

Invocation

The section on calling methods showed you how to call a method or field in a Java program using the JNI interface and a class loaded using the FindClass function. With a little more code, you can create a standalone program that invokes a Java virtual machine and includes its own JNI interface pointer that can be used to create instances of Java classes. In the Java 2 release, the runtime program named java is a small JNI application that does exactly that.

You can create a Java virtual machine with a call to JNI_CreateJavaVM, and shut the created Java virtual machine down with a call to JNI_DestroyJavaVM. A Java virtual machine might also need some additional environment properties. These properties can be passed to the JNI_CreateJavaVM function in a JavaVMInitArgs structure.

The JavaVMInitArgs structure contains a pointer to a JavaVMOption value used to store environment information such as the classpath and Java virtual machine version, or system properties that would normally be passed on the command line to the program.

When the JNI_CreateJavaVM function returns, you can call methods and create instances of classes using the FindClass and NewObject functions the same way you would for embedded native code.

Note: The Java virtual machine invocation was only used for native thread Java virtual machines. Some older Java virtual machines have a green threads option that is stable for invocation use. On a Unix platform, you may also need to explicitly link with -lthread or -lpthread.

This next program invokes a Java virtual machine, loads the ArrayHandler class, and retrieves the arraySize field which should contain the value minus one. The Java virtual machine options include the current path in the classpath and turning the Just-In-Time (JIT) compiler off with the option -Djava.compiler=NONE.

```
    #include <jni.h>

    void main(int argc, char *argv[], char **envp) {
      JavaVMOption options[2];
      JavaVMInitArgs vm_args;
      JavaVM *jvm;
      JNIEnv *env;
      long result;
      jmethodID mid;
      jfieldID fid;
      jobject jobj;
      jclass cls;
      int i, asize;

      options[0].optionString = ".";
      options[1].optionString = "-Djava.compiler=NONE";
      vm_args.version = JNI_VERSION_1_2;
      vm_args.options = options;
      vm_args.nOptions = 2;
      vm_args.ignoreUnrecognized = JNI_FALSE;
//API Ref :jint JNI_CreateJavaVM(JavaVM **pvm, void **penv, void *args)
      result = JNI_CreateJavaVM(&jvm,(void **)&env, &vm_args);
      if(result == JNI_ERR ) {
        printf("Error invoking the JVM");
        exit (-1);
      }
      cls = (*env)->FindClass(env,"ArrayHandler");
      if( cls == NULL ) {
        printf("can't find class ArrayHandler\n");
        exit (-1);
      }
      (*env)->ExceptionClear(env);
      mid=(*env)->GetMethodID(env, cls, "<init>", "()V");
      jobj=(*env)->NewObject(env, cls, mid);
      fid=(*env)->GetFieldID(env, cls, "arraySize", "I");
      asize=(*env)->GetIntField(env, jobj, fid);
      printf("size of array is %d",asize);
      (*jvm)->DestroyJavaVM(jvm);
    }
```

Attaching Threads

After the Java virtual machine is invoked, there is one local thread running the Java virtual machine. You can create more threads in the local operating system and attach the Java virtual machine to those new threads. You might want to do this if your native application is multithreaded.

Attach the local thread to the Java virtual machine with a call to AttachCurrentThread. You need to supply pointers to the Java virtual machine instance and JNI environment. In the Java 2 platform, you can also specify in the third parameter the thread name and/or group you

want this new thread to live under. It is important to detach any thread that has been previously attached; otherwise, the program will not exit when you call DestroyJavaVM.

```
#include <jni.h>
#include <pthread.h>

JavaVM *jvm;

void *native_thread(void *arg) {
  JNIEnv *env;
  jclass cls;
  jmethodID mid;
  jfieldID fid;
  jint result;
  jobject jobj;
  JavaVMAttachArgs args;
  jint asize;

  args.version= JNI_VERSION_1_2;
  args.name="user";
  args.group=NULL;
  result=(*jvm)->AttachCurrentThread(jvm, (void **)&env, &args);
  cls = (*env)->FindClass(env,"ArrayHandler");
  if( cls == NULL ) {
    printf("can't find class ArrayHandler\n");
    exit (-1);
  }
  (*env)->ExceptionClear(env);
  mid=(*env)->GetMethodID(env, cls, "<init>", "()V");
  jobj=(*env)->NewObject(env, cls, mid);
  fid=(*env)->GetFieldID(env, cls, "arraySize", "I");
  asize=(*env)->GetIntField(env, jobj, fid);
  printf("size of array is %d\n",asize);
  (*jvm)->DetachCurrentThread(jvm);
}
void main(int argc, char *argv[], char **envp) {
  JavaVMOption *options;
  JavaVMInitArgs vm_args;
  JNIEnv *env;
  jint result;
  pthread_t tid;
  int thr_id;
  int i;
  options = (void *)malloc(3 * sizeof(JavaVMOption));
  options[0].optionString = "-Djava.class.path=.";
  options[1].optionString = "-Djava.compiler=NONE";
  vm_args.version = JNI_VERSION_1_2;
  vm_args.options = options;
  vm_args.nOptions = 2;
  vm_args.ignoreUnrecognized = JNI_FALSE;
  result = JNI_CreateJavaVM(&jvm,(void **)&env, &vm_args);
  if(result == JNI_ERR ) {
    printf("Error invoking the JVM");
```

```
    exit (-1);
  }
  thr_id=pthread_create(&tid, NULL, native_thread, NULL);
// If you don't have join, sleep instead
// sleep(1000);
  pthread_join(tid, NULL);
  (*jvm)->DestroyJavaVM(jvm);
  exit(0);
}
```

6: Project Swing: Building a User Interface

The Java™ Foundation Classes (JFC) Project Swing and Enterprise JavaBeans™ architectures share one key design element: the separation of data from the display or manipulation of that data. In Enterprise JavaBeans applications, the entity bean provides a view of the data. The underlying data storage mechanism can be swapped out and replaced without changing the entity bean view or recompiling any code that uses the view.

Project Swing separates the view and control of a visual component from its contents, or data model. However, although Project Swing does have the components that make up a Model-View-Controller (MVC) architecture, it is more accurately described as a model-delegate architecture. This is because the controller part of the Project Swing interface, often the mouse and keyboard events the component responds to, is combined with the physical view in one User Interface delegate (UI delegate) object.

Each component, for example a JButton or a JScrollBar, has a separate UI delegate class that inherits from the ComponentUI class and is under the control of a separate UI manager. While each component has a basic UI delegate, it is no longer tied to the underlying data so a new set of delegates -- a set of metal-styled components, for example -- can be swapped in while the application is still running. The ability to change the look and behavior reflects the pluggable look and feel (PLAF) feature available in Project Swing.

This chapter describes Project Swing user interface components in terms of the AuctionClient example application.

Covered in this Chapter

- Components and Data Models (page 234)
- Printing API (page 256)
- Advanced Printing (page 263)
- Code for this Chapter (page 270)

Components and Data Models

This section explains Project Swing in terms of the AuctionClient (page 270) program shown in Figure 19. It is a simple GUI application that lets auction administrators list and browse auction items.

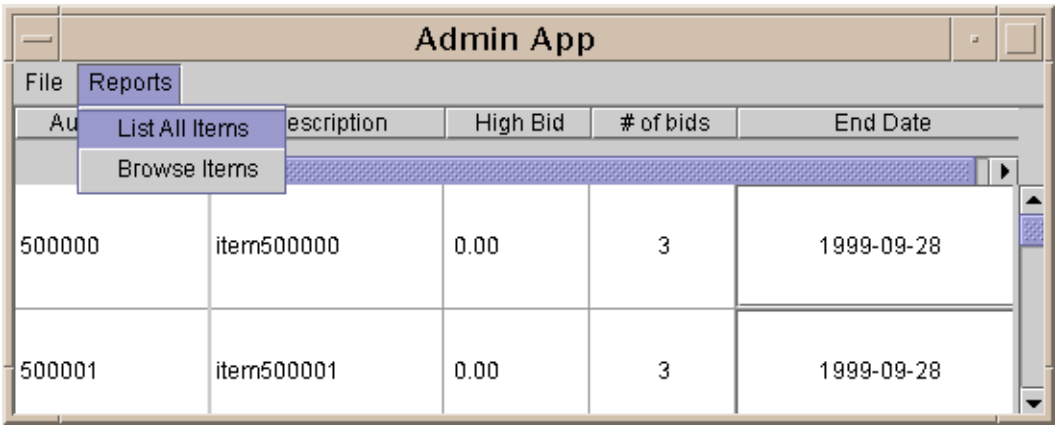


Figure 19 Administration Applet

Lightweight Components

All components in Project Swing, except JApplet, JDialog, JFrame and JWindow are lightweight components. Lightweight components, unlike their Abstract Window Toolkit (AWT) counterparts, do not depend on the local windowing toolkit.

For example, a heavyweight java.awt.Button running on the Java platform for the Unix platform maps to a real Motif button. In this relationship, the Motif button is called the peer to the java.awt.Button. If you create two java.awt.Buttons in an application, two peers and hence two Motif Buttons are also created. The Java platform communicates with the Motif Buttons using the Java Native Interface (JNI). For each and every component added to the application, there is additional overhead tied to the local windowing system, which is why these components are called heavyweight.

Lightweight components are termed peerless components and emulate the local window system components. A lightweight button is represented as a rectangle with a label inside that accepts mouse events. Adding more lightweight buttons means drawing more rectangles.

A lightweight component needs to be drawn on something, and an application written in the Java programming language needs to interact with the local window manager so the main application window can be closed or minimized. This is why the top-level parent compo-

nents mentioned above (JFrame, JApplet, and others) are implemented as heavyweight components -- they need to be mapped to a component in the local window toolkit.

A JButton is a very simple shape to draw. For more complex components like JList or JTable, the elements or cells of the list or table are drawn by a CellRenderer object. A CellRenderer object provides flexibility because it makes it possible for any type of object to be displayed in any row or column.

For example, a JTable can use a different CellRenderer for each column. This code segment sets the second column, which is referenced as index 1, to use a CustomRenderer object to create the cells for that column.

```
    JTable scrollTable=new JTable(rm);
    TableColumnModel scrollColumnModel = scrollTable.getColumnModel();
    CustomRenderer custom = new CustomRenderer();
//API Ref :TableColumn getColumn(int index)
    scrollColumnModel.getColumn(1).setCellRenderer(custom);
```

Ordering Components

Each Project Swing applet or application needs at least one heavyweight container component (a JFrame, JWindow, JApplet, or JDialog). Each of these containers with JFrame's lightweight multiple document interface (MDI) counterpart, JInternalFrame, contains a component called a root pane. The JRootPane manages the additional layers used in the container such as the JLayeredPane, JContentPane, GlassPane and the optional JMenuBar. It also lets all emulated (lightweight) components interact with the AWT event queue to send and receive events. Interacting with the event queue gives emulated components indirect interaction with the local window manager.

JLayeredPane

The JLayeredPane sits on top of the JRootPane, and as its name implies, controls the layers of the components contained within the boundary of the heavyweight container. The components are not added to the JLayeredPane, but to the JContentPane instead. The JLayeredPane determines the Z-ordering of the components in the JRootPane. The Z-order can be thought of as the order of overlay among the various components. If you drag-and-drop a component or request a dialog to pop up, you want that component to appear in front of the others in the application window. The JLayeredPane lets you layer components.

The JLayeredPane divides the depth of the container into different bands that can be used to assign a component to a type-appropriate level. The DRAG_LAYER band, value 400, appears above all other defined component layers. The lowermost level of JLayeredpane, the

DEFAULT_FRAME_LAYER band, has value -3000 and is the level of the heavyweight containers, including the MenuBar. The bands are as follows:

Value	Layer Name	Component Types
-3000	DEFAULT_FRAME_LAYER	JMenubar
0	DEFAULT_LAYER	JButton, JTable, ..
	PALETTE_LAYER	Floating components such as JToolBar
	MODAL_LAYER	Modal Dialogs
400	DRAG_LAYER	Drag-and-drop over all layers

Within these general depth bands, components can be further arranged with another numbering system to order the components in a particular band, but this system reverses the numbering priority.

For example, in a specific band such as DEFAULT_LAYER, components with a value of 0 appear in front of others in that band; whereas, components with a higher number or -1 appear behind them. The highest number in this scheme is the number of components minus 1, so one way to visualize it is shown in Figure 20, which shows a vector of components that steps through painting the components with a higher number first finishing with the one at position 0.

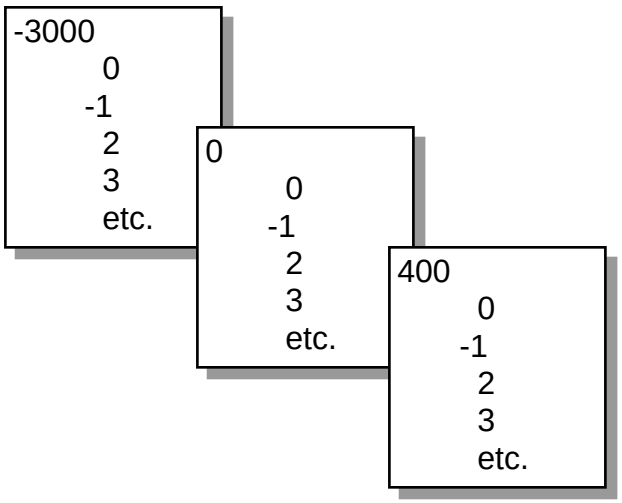


Figure 20 Painting Components

For example, the following code adds a JButton to the default layer and specifies that it appear in front of the other components in that same layer:

```
JButton enterButton = new JButton("Enter");
layeredPane.add(enterButton, LayeredPane.Default_Layer, 0);
```

You can achieve the same effect by calling the LayeredPane.moveToFront method within a layer or using the LayeredPane.setLayer method to move to a different layer.

JContentPane

The JContentPane manages adding components to heavyweight containers. So, you have to call the getContentPane method to add a component to the ContentPane of the RootPane. By default, a ContentPane is initialized with a BorderLayout layout manager. There are two ways to change the layout manager. You can call the setLayout method like this:

```
getContentPane()).setLayout(new BoxLayout())
```

Or you can replace the default ContentPane with your own ContentPane, such as a JPanel, like this:

```
JPanel pane= new JPanel();
pane.setLayout(new BoxLayout());
setContentPane(pane);
```

GlassPane

The GlassPane is usually completely transparent and just acts as a sheet of glass in front of the components. You can implement your own GlassPane by using a component like JPanel and installing it as the GlassPane by calling the setGlassPane method. The RootPane is configured with a GlassPane that can be retrieved by calling getGlassPane.

One way to use a GlassPane is to implement a component that invisibly handles all mouse and keyboard events, effectively blocking user input until an event completes. The GlassPane can block the events, but currently the cursor will not return to its default state if you have set the cursor to be a busy cursor in the GlassPane. An additional mouse event is required for the refresh.

```
MyGlassPane  glassPane = new MyGlassPane();
setGlassPane(glassPane);
glassPane.setVisible(true); //before worker thread
  ..
glassPane.setVisible(false); //after worker thread

private class MyGlassPane extends JPanel {

  public MyGlassPane() {
    addKeyListener(new KeyAdapter() { });
    addMouseListener(new MouseAdapter() { });
    super.setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
  }
}
```

Data Models

Numerous model layers are combined to form the tables of the AuctionClient GUI. At a foundational level, the TableModel interface and its two implementations AbstractTableModel and DefaultTableModel provide the most basic means for storage, retrieval and modification of the underlying data.

The TableModel defines and categorizes the data by its class. It also determines if the data can be edited and how the data is grouped into columns and rows. It is important to note, however, that while the TableModel interface is used most often in the construction of a JTable, it is not fundamentally tied to their display. Implementations could just as easily form the basis of a spreadsheet component, or even a non-GUI class that calls for the organization of data in tabular format.

The ResultsModel class is at the heart of the AuctionClient tables. It defines a dynamic data set, dictates whether class users can edit the data through its ResultsModel.isCellEditable method, and provides the update method to keep the data current. The model underlies the scrolling and fixed tables, and lets modifications be reflected in each view.

At a higher level and representing an intermediate layer between data and its graphical representation, is the TableColumnModel. At this level the data is grouped by column in anticipation of its ultimate display in the table. The visibility and size of these columns, their headers, and the component types of their cell renderers and editors are all managed by the TableColumnModel class.

For example, freezing the left-most columns in the AuctionClient GUI is possible because column data is easily exchanged among multiple TableColumnModel and JTable objects. This translates to the fixedTable and scrollTable objects of the AuctionClient program.

Higher still lie the various renderers, editors, and header components whose combination define the look and organization of the JTable component. This level is where the fundamental layout and display decisions of the JTable are made.

The creation of the inner classes CustomRenderer and CustomButtonRenderer within the AuctionClient application allows users of those classes to redefine the components upon which the appearance of table cells are based. Likewise, the CustomButtonEditor class takes the place of the table's default editor. In true object-oriented style, the default editors and renderers are easily replaced, affecting neither the data they represent nor the function of the component in which they reside.

Finally, the various component user interfaces are responsible for the ultimate appearance of the JTable. It is here the look-and-feel-specific representation of the AuctionClient tables and their data are rendered in final form to the user. The end result is that adding a Project Swing front-end to existing services requires little additional code. In fact, coding the model is one of the easier tasks in building a Project Swing application.

Table Model

The JTable class has an associated DefaultTableModel class that internally uses a Vector of vectors to store data. The data for each row is stored in a single Vector object while another Vector object stores each of those rows as its constituent elements. The DefaultTableModel object can be initialized with data in several different ways. This code shows the DefaultTableModel created with a two-dimensional array and a second array representing column headings. The DefaultTableModel in turn converts the Object arrays into the appropriate Vector objects:

```
      Object[][] data = new Object[][]{{"row 1 col1", "row 1 col2" },
                                       {"row 2 col 1", "row 2 col 2"}};
      Object[] headers = new Object[] {"first header","second header"};
      DefaultTableModel model = new DefaultTableModel(data, headers);
      table = new JTable(model);
//API Ref :void setAutoResizeMode(int mode)
      table.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
```

Creating a custom table model is nearly as easy as using DefaultTableModel, and requires little additional coding. You can implement a table model by implementing a method to return the number of entries in the model, and a method to retrieve an element at a specific position in that model. For example, the JTable model can be implemented from javax.swing.table.AbstractTableModel by implementing the methods getColumnCount, getRowCount and getValueAt as shown here:

```
final Object[][] data = new Object[][]{ {"row 1 col1","row 1 col2" },
                                        {"row 2 col 1","row 2 col 2"} };
final Object[] headers = new Object[] {"first header","second header"};
TableModel model = new AbstractTableModel(){
  public int getColumnCount() {
    return data[0].length;
  }
  public int getRowCount() {
    return data.length;
  }
  public String getColumnName(int col) {
    return (String)headers[col];
  }
  public Object getValueAt(int row,int col) {
     return data[row][col];
  }
};
table = new JTable(model);
table.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
```

This table is read-only and its data values are already known. In fact, the data is even declared final so it can be retrieved by the inner TableModel class. This is not normally the situation when working with live data.

You can create an editable table by adding the isCellEditable verification method, which is used by the default cell editor and the AbstractTableModel class for setting a value at a position. Up until this change, the AbstractTableModel has been handling the repainting and resizing of the table by firing different table changed events. Because the AbtractTableModel does not know that something has occurred to the table data, you need to inform it by calling the fireTableCellUpdated method. The following lines are added to the AbstractTableModel inner class to allow editing of the data:

```
      public void setValueAt (Object value, int row, int col) {
        data[row][col] = value;
        fireTableCellUpdated (row, col);
      }
//API Ref :boolean isCellEditable(int row, int column)
      public boolean isCellEditable(int row, int col) {
        return true;
      }
```

More Table Models

A common requirement for the display of tabular data is the inclusion of a non-scrolling column. This column provides a set of anchor data that remains stationary and visible while its neighboring columns are scrolled horizontally (and often out of view). This is particularly important in cases where row data can be identified by a unique value in the fixed column, such as a name or identification number. The next code example uses a fixed table column to display a list of the auction items.

The base table model in this example implements the AbstractTableModel class. Its update method dynamically populates the table data from a call to the database. It sends an event that the table has been updated by calling the fireTableStructureChanged method to indicate the number of rows or columns in the table have changed.

```
package auction;

import javax.swing.table.AbstractTableModel;
import javax.swing.event.TableModelEvent;
import java.text.NumberFormat;
import java.util.*;
import java.awt.*;

public class ResultsModel extends AbstractTableModel{
  String[]  columnNames={};
  Vector rows = new Vector();

  public String getColumnName(int column) {
    if(columnNames[column] != null) {
      return columnNames[column];
    } else {
      return "";
    }
  }
  public boolean isCellEditable(int row, int column){
    return false;
  }
  public int getColumnCount() {
    return columnNames.length;
  }
  public int getRowCount() {
    return rows.size();
  }
  public Object getValueAt(int row, int column){
    Vector tmprow = (Vector)rows.elementAt(row);
    return tmprow.elementAt(column);
  }
  public  void update(Enumeration enum) {
    try {
      // Create the column headers which are later accessed through
      // the getColumnName method
      columnNames = new String[5];
```

```
            columnNames[0]=new String("Auction Id #");
            columnNames[1]=new String("Description");
            columnNames[2]=new String("High Bid");
            columnNames[3]=new String("# of bids");
            columnNames[4]=new String("End Date");
            // Create each vector row from the AuctionItem data
            while((enum !=null) && (enum.hasMoreElements())) {
                AuctionItem auctionItem=(AuctionItem)enum.nextElement();
                Vector items=new Vector();
                items.addElement(new Integer(auctionItem.getId()));
                items.addElement(auctionItem.getSummary());
                int bidcount= auctionItem.getBidCount();
                if(bidcount >0) {
                  items.addElement(NumberFormat.getCurrencyInstance().format(
                                  auctionItem.getHighBid()));
                } else {
                  items.addElement("-");
                }
                items.addElement(new Integer(bidcount));
                items.addElement(auctionItem.getEndDate());
                rows.addElement(items);
            }
//API Ref :void fireTableStructureChanged()
            fireTableStructureChanged();
          } catch (Exception e) {
            System.out.println("Exception e"+e);
          }
        }
      }
```

The table is created from the ResultsModel model. Then, the first table column is removed from that table and added to a new table. Because there are now two tables, the only way the selections can be kept in sync is to use a ListSelectionModel object to set the selection on the table row in the other tables that were not selected by calling the setRowSelectionInterval method. The full example can be found in the AuctionClient (page 270) source file:

```
      private void listAllItems() throws IOException{
        ResultsModel rm=new ResultsModel();
         try {
            BidderHome bhome=(BidderHome)ctx.lookup("bidder");
            Bidder bid=bhome.create();
            Enumeration enum=(Enumeration)bid.getItemList();
            if (enum != null) {
              rm.update(enum);
            }
         } catch (Exception e) {
             System.out.println("AuctionServlet <list>:"+e);
         }
        // Create a new table using the ResultsModel object as the table model
        scrollTable=new JTable(rm);
        // Force the End Date and Description columns to have
        // a set width
```

```
        adjustColumnWidth(scrollTable.getColumn("End Date"), 150);
        adjustColumnWidth(scrollTable.getColumn("Description"), 120);
        // The scroll column model is initially mapped to
        // display all the table data. The fixed column model
        // is initially empty
        scrollColumnModel = scrollTable.getColumnModel();
        fixedColumnModel = new DefaultTableColumnModel();

        // Get the first column, remove it from the scroll column, and
        // add it to the fixed table
        TableColumn col = scrollColumnModel.getColumn(0);
        scrollColumnModel.removeColumn(col);
        fixedColumnModel.addColumn(col);

        // Create a new table based on the fixed column model (1 column)
        fixedTable = new JTable(rm,fixedColumnModel);

        // Keep the heights of the fixed and scroll tables the same
        fixedTable.setRowHeight(scrollTable.getRowHeight());

        headers = new JViewport();
        // Get the selection model used in this table and override it
        // so we can also update the scroll table if the fixed table
        // is selected and vice-versa
        ListSelectionModel fixedSelection = fixedTable.getSelectionModel();
        fixedSelection.addListSelectionListener(
            new ListSelectionListener() {
                public void valueChanged(ListSelectionEvent e) {
                  ListSelectionModel lsm = (ListSelectionModel)e.getSource();
                  if (!lsm.isSelectionEmpty())  {
                     setScrollableRow();
                  }
                }
         });
//API Ref :ListSelectionMode getSelectionModel()
         ListSelectionModel scrollSelection = scrollTable.getSelectionModel();
         scrollSelection.addListSelectionListener(new ListSelectionListener() {
           public void valueChanged(ListSelectionEvent e) {
             ListSelectionModel lsm = (ListSelectionModel)e.getSource();
             if (!lsm.isSelectionEmpty())  {
               setFixedRow();
             }
           }
         });
         // Create a custom renderer and use it on column 2
         CustomRenderer custom = new CustomRenderer();
         custom.setHorizontalAlignment(JLabel.CENTER);
//API Ref :void setCellRenderer(TableCellRenderer renderer)
         scrollColumnModel.getColumn(2).setCellRenderer(custom);
         // Use the custom button renderer on column 3
         scrollColumnModel.getColumn(3).setCellRenderer(new CustomButtonRenderer());
         // Add the custom button editor to column 3
         CustomButtonEditor customEdit=new CustomButtonEditor(frame);
```

```
        scrollColumnModel.getColumn(3).setCellEditor(customEdit);

        // Add the scrolling column headers to the header viewport
        // the headers viewport is later added to the headerPanel with
        // an associated scroll bar
//API Ref :JTableHeader getTableHeader()
        headers.add(scrollTable.getTableHeader());

        // Create a panel for the top left of the display that contains
        // the headers for the column that does not move
        JPanel topPanel = new JPanel();
        topPanel.setLayout(new BoxLayout(topPanel, BoxLayout.X_AXIS));
        adjustColumnWidth(fixedColumnModel.getColumn(0), 100);
        JTableHeader fixedHeader=fixedTable.getTableHeader();
        fixedHeader.setAlignmentY(Component.TOP_ALIGNMENT);
        topPanel.add(fixedHeader);
//API Ref :Component createRigidArea(Dimension d)
        topPanel.add(Box.createRigidArea(new Dimension(2, 0)));
        topPanel.setPreferredSize(new Dimension(400, 40));

        // Create a panel to contain the scroll headers that will
        // be displayed at the top right
        JPanel headerPanel = new JPanel();
        headerPanel.setAlignmentY(Component.TOP_ALIGNMENT);
        headerPanel.setLayout(new BorderLayout());
        JScrollPane scrollpane = new JScrollPane();
//API Ref :JScrollBar getHorizontalScrollBar()
        scrollBar = scrollpane.getHorizontalScrollBar();
        headerPanel.add(headers, "North");
        headerPanel.add(scrollBar, "South");
        topPanel.add(headerPanel);
        // Set the scrollable part of both the table views
        scrollTable.setPreferredScrollableViewportSize(new Dimension(300,180));
        fixedTable.setPreferredScrollableViewportSize( new Dimension(100,180));
        fixedTable.setPreferredSize(new Dimension(100,180));
        innerPort = new JViewport();

        // If the user moves the scrollbar at the top
        // of the scrollable data update the header and table
        // data positions so they are viewable
//API Ref :void setView(Component view)
        innerPort.setView(scrollTable);
        scrollpane.setViewport(innerPort);
         scrollBar.getModel().addChangeListener(new ChangeListener()  {
          public void stateChanged(ChangeEvent e) {
//API Ref :Point getViewPosition()
            Point q = headers.getViewPosition();
            Point p = innerPort.getViewPosition();
            int val = scrollBar.getModel().getValue();
            p.x = val;
            q.x = val;

//API Ref :void setView(Component view)
```

```
            headers.setViewPosition(p);
            headers.repaint(headers.getViewRect());
            innerPort.setViewPosition(p);
            innerPort.repaint(innerPort.getViewRect());
          }
        });

        // Disable continuous update of the scroll table. Needed to
        // improve performance of a scrolling table in SDK 1.2
        scrollTable.getTableHeader().setUpdateTableInRealTime(false);

        // The bottom panel is where both tables data is positioned
        JPanel bottomPanel = new JPanel();
        bottomPanel.setLayout(new BoxLayout(bottomPanel, BoxLayout.X_AXIS));
        fixedTable.setAlignmentY(Component.TOP_ALIGNMENT);
        bottomPanel.add(fixedTable);
        bottomPanel.add(Box.createRigidArea(new Dimension(2, 0)));
        innerPort.setAlignmentY(Component.TOP_ALIGNMENT);
        bottomPanel.add(innerPort);
        bottomPanel.add(Box.createRigidArea(new Dimension(2, 0)));

        // Only allow vertical scrollbars as we are managing the
        // horizontal scroll ourselves
        scrollPane = new JScrollPane(bottomPanel,
                        JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
                        JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        JViewport outerPort = new JViewport();
        outerPort.add(bottomPanel);
        // Workaround to affix the headers to the top of the scrollpane
        // uses setColumnHeaderView
//API Ref :void setColumnHeaderView(Component view)
         scrollPane.setColumnHeaderView(topPanel);
//API Ref :void setViewport(JViewport viewport)
        scrollPane.setViewport(outerPort);
        scrollTable.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
        frame.getContentPane().add(scrollPane);
        scrollTable.validate();
        frame.setSize(450,200);
      }
      void setFixedRow() {
        int index=scrollTable.getSelectedRow();
        fixedTable.setRowSelectionInterval(index, index);
      }
      void setScrollableRow() {
        // Highlight the row by calling setRowSelectionInterval with a
        // range of 1
        int index=fixedTable.getSelectedRow();
        scrollTable.setRowSelectionInterval(index, index);
      }
      void adjustColumnWidth(TableColumn c, int size) {
```

```
    c.setPreferredWidth(size);
    c.setMaxWidth(size);
    c.setMinWidth(size);
  }
```

JList Model

The JList component displays a vertical list of data elements and uses a ListModel to hold and manipulate the data. It also uses a ListSelectionModel object to enable selection and subsequent retrieval of elements in the list.

Default implementations of the AbstractListModel and AbstractListSelectionModel classes are provided in the Project Swing API in the form of the DefaultListModel and DefaultListSelectionModel classes. If you use these two default models and the default cell renderer, you get a list that displays model elements by calling the toString method on each object. The list uses the MULTIPLE_INTERVAL_SELECTION list selection model to select each element from the list.

Three selection modes are available to DefaultListSelectionModel: SINGLE_SELECTION, where only one item is selected at a time; SINGLE_INTERVAL_SELECTION in which a range of sequential items can be selected; and MULTIPLE_INTERVAL_SELECTION, which allows any or all elements to be selected. The selection mode can be changed by calling the setSelectionMode method in the JList class.

```
public SimpleList() {
  JList list;
  DefaultListModel deflist;

  deflist= new DefaultListModel();
  deflist.addElement("element 1");
  deflist.addElement("element 2");
  list = new JList(deflist);
  JScrollPane scroll = new JScrollPane(list);
  getContentPane().add(scroll, BorderLayout.CENTER);
 }
```

JTree Model

The JTree class models and displays a vertical list of elements or nodes arranged in a tree-based hierarchy as shown in Figure 21.

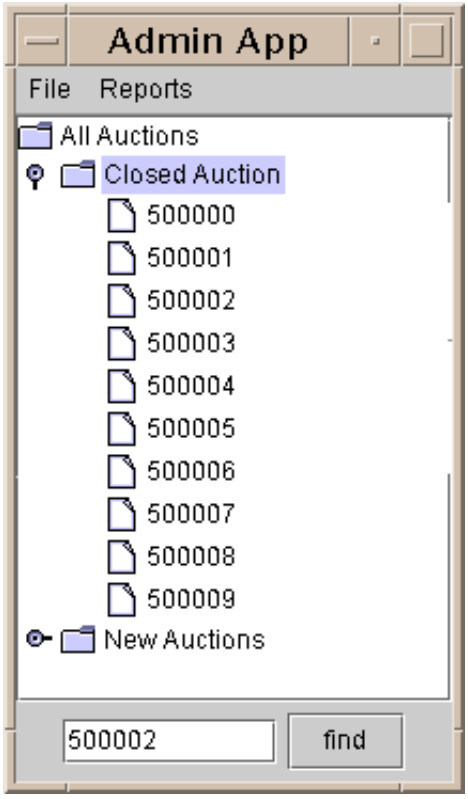


Figure 21 Vertical list of elements in a tree

A JTree object has one root node and one or more child nodes, which can contain further child nodes. Each parent node can be expanded to show all its children similar to directory trees familiar to Windows users.

Like the JList and JTable components, the JTree consists of more than one model. The selection model is similar to the one detailed for the JList model. The selection modes have the following slightly different names: SINGLE_TREE_SELECTION, DISCONTIGUOUS_TREE_SELECTION, and CONTIGUOUS_TREE_SELECTION.

While DefaultTreeModel maintains the data in the tree and is responsible for adding and removing nodes, it is the DefaultTreeMutableTreeNode class that defines the methods used for node traversal. The DefaultTreeModel is often used to implement custom models

because there is no AbstractTreeModel in the JTree package. However, if you use custom objects, you must implement TreeModel. This code example creates a JTree using the DefaultTreeModel so the adminstrator can search and browse the auction numbers and details for reporting.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.tree.*;

public class SimpleTree extends JFrame {
  public SimpleTree() {
    String[] treelabels =   {"All Auctions", "Closed Auction", "Open Auctions"};
    Integer[] closedItems = {new Integer(500144), new Integer(500146),
                             new Integer(500147)};
    Integer[] openItems = { new Integer(500148), new Integer(500149)};
    // Create an array of nodes based on the size of the labels and
    // open and closed items
    DefaultMutableTreeNode[] nodes = new
                             DefaultMutableTreeNode[treelabels.length];
    DefaultMutableTreeNode[] closednodes = new
                             DefaultMutableTreeNode[closedItems.length];
    DefaultMutableTreeNode[] opennodes = new
                             DefaultMutableTreeNode[openItems.length];
    // Populate the labels with the data label elements
    for (int i=0; i < treelabels.length; i++) {
      nodes[i] = new DefaultMutableTreeNode(treelabels[i]);
    }

    // Arrange the tree so that from the top level All Auctions
    // closed and open auctions are children from that top level
    nodes[0].add(nodes[1]);
    nodes[0].add(nodes[2]);

   for (int i=0; i < closedItems.length; i++) {
       // Populate the closed items nodes with the closed item data
       // ie auction ids
       closednodes[i] = new DefaultMutableTreeNode(closedItems[i]);
       // Add the closed items under the Closed Auction parent node
       nodes[1].add(closednodes[i]);
    }

    for (int i=0; i < openItems.length; i++) {
       // Populate the open items nodes with the open item data
       // ie auction ids
       opennodes[i] = new DefaultMutableTreeNode(openItems[i]);
       // Add the open items under the Open Auction parent node
       nodes[2].add(opennodes[i]);
    }
    // Finally create the tree model from the nodes created above
```

```
    // and then create the tree
    DefaultTreeModel model=new DefaultTreeModel(nodes[0]);
    JTree tree = new JTree(model);
    JScrollPane scroll = new JScrollPane(tree);
    getContentPane().add(scroll, BorderLayout.CENTER);
  }

  public static void main(String[] args) {
    SimpleTree frame = new SimpleTree();
    frame.addWindowListener( new WindowAdapter() {
      public void windowClosing( WindowEvent e ) {
        System.exit(0);
      }
    });
    frame.setVisible(true);
    frame.pack();
    frame.setSize(150,150);
  }
}
```

The toString method is used to retrieve the value for the Integer objects in the tree. And although the DefaultTreeModel is used to maintain the data in the tree and to add or remove nodes, the DefaultMutableTreeNode class defines the methods used to traverse through the nodes in the tree.

A primitive search of the nodes in a JTree is accomplished with the depthFirstEnumeration method, which is the same as the postorderEnumeration method and works its way from the end points of the tree first. You can also call the preorderEnumeration method, the reverse of the postorderEnumeration method, which starts from the root and descends each tree in turn. Or you can call the breadthFirstEnumeration method, which starts from the root and visits all the child nodes in one level before visiting the child nodes at a lower depth.

The following code expands the parent node if it contains a child node that matches the search field entered. It uses a call to Enumeration e = nodes[0].depthFirstEnumeration(); to return a list of all the nodes in the tree. Once it has found a match, it builds the TreePath from the root node to the node that matched the search to pass to the makeVisible method in the JTree class that ensures the node is expanded in the tree.

```
import java.awt.*;
import java.util.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.tree.*;

public class SimpleSearchTree extends JFrame {
  JPanel findPanel;
  JTextField findField;
  JTree tree;
  JButton findButton;
  DefaultMutableTreeNode[] nodes;
```

```
      public SimpleSearchTree() {
        String[] treelabels =  { "All Auctions", "Closed Auction", "Open Auctions"};
        Integer[] closedItems = { new Integer(500144), new Integer(500146),
                                  new Integer(500147) };
        Integer[] openItems ={ new Integer(500148), new Integer(500149)};
        // Create an array of nodes based on the size of the labels and
        // open and closed items

        nodes = new DefaultMutableTreeNode[treelabels.length];
//API Ref :DefaultMutableTreeNode(Object node)
        DefaultMutableTreeNode[] closednodes = new
                                  DefaultMutableTreeNode[closedItems.length];
        DefaultMutableTreeNode[] opennodes = new
                                  DefaultMutableTreeNode[openItems.length];

        // Populate the labels with the data label elements
        for (int i=0; i < treelabels.length; i++) {
           nodes[i] = new DefaultMutableTreeNode(treelabels[i]);
        }

        // Arrange the tree so that from the top level, all
        // closed and open auctions are children from that top level
        nodes[0].add(nodes[1]);
        nodes[0].add(nodes[2]);
        for (int i=0; i < closedItems.length; i++) {
           // Populate the closed items nodes with the closed item data,
           // i.e., auction ids
           closednodes[i] = new DefaultMutableTreeNode(closedItems[i]);
           // Add the closed items under the Closed Auction parent node
           nodes[1].add(closednodes[i]);
        }

        for (int i=0; i < openItems.length; i++) {
           // Populate the open item nodes with the open item data,
           // i.e., auction ids
           opennodes[i] = new DefaultMutableTreeNode(openItems[i]);
           // Add the open items under the Open Auction parent node
           nodes[2].add(opennodes[i]);
        }
        // Finally, create the tree model from the nodes created above
        // and then create the tree

        DefaultTreeModel model=new DefaultTreeModel(nodes[0]);
        tree = new JTree(model);
        JScrollPane scroll = new JScrollPane(tree);
        getContentPane().add(scroll, BorderLayout.CENTER);
        findPanel= new JPanel();
        findField= new JTextField(10);
        findButton= new JButton("find");
//API Ref :void addActionListener(ActionListener l)
        findButton.addActionListener (new ActionListener() {
//API Ref :void actionPerformed(ActionEvent e)
```

```
         public void actionPerformed (ActionEvent e) {
            String field=findField.getText();
            if (field != null) {
              // Search for the node whose label matches the search string
              findNode(findField.getText());
            } else {
              return;
            }
          }
        });
        findPanel.add(findField);
        findPanel.add(findButton);
        getContentPane().add(findPanel, BorderLayout.SOUTH);
      }
      public void findNode(String field) {
        // Get an enumeration list of all the nodes.
        // The ordering is generated by starting from the root
        // to the end of each branch, and to the end of
        // the next branch including any nodes not visited before.
//API Ref :Enumeration depthFirstEnumeration()
        Enumeration e = nodes[0].depthFirstEnumeration();
        Object currNode;
        while (e.hasMoreElements()) {
          currNode = e.nextElement();
          if (currNode.toString().equals(field)) {
           // Does the node match our search string? If so, generate
           // a path object to highlight the matching node
           // with a call to setSelectionRow.
//API Ref :TreePath(Object singlePath)
           TreePath path=new TreePath(((DefaultMutableTreeNode)currNode).getPath());
//API Ref :void makeVisible(TreePath path)
           tree.makeVisible(path);
//API Ref :void setSelectionRow(int row)
           tree.setSelectionRow(tree.getRowForPath(path));
           return;
          }
        }
      }

      public static void main(String[] args) {
        SimpleSearchTree frame = new SimpleSearchTree();
//API Ref :void addWindowListener(WindowListener l)
        frame.addWindowListener( new WindowAdapter() {
//API Ref :void windowClosing(WindowEvent e)
          public void windowClosing( WindowEvent e ) {
            System.exit(0);
          }
        });
        frame.setVisible(true);
        frame.pack();
        frame.setSize(300,150);
      }
    }
```

JTree, JTable and JList are probably the most common components you will want to customize. But you can use models such as SingleSelectionModel for general data manipulation. The SingleSelectionModel class lets you specify how data is selected in a component.

Custom Cell Rendering

As you learned above, many components have a default cell renderer to paint each element in a table, tree or list. The default cell renderer is usually a JLabel and displays a String representation of the data element.

A simple custom cell renderer can extend the DefaultXXXCellRenderer class to provide additional customization in the getXXXCellRenderer. The DefaultTableCellRenderer and DefaultTreeCellRenderer Components both use a JLabel to render the cell. This means any customization that can be applied to a JLabel can also be used in the JTable or JTree cell.

For example, the following renderer sets the background color of the component if the auction item has received a high number of bids:

```
    class CustomRenderer extends DefaultTableCellRenderer {
//API Ref :Component getTableCellRendererComponent(JTable table, Object value,
          boolean isSelected, boolean hasFocus, int row, int column)
      public Component getTableCellRendererComponent(JTable table,Object value,
                       boolean isSelected, boolean hasFocus, int row, int column) {
        Component comp = super.getTableCellRendererComponent(table,value,isSelected,
                         hasFocus, row,column);
          JLabel label = (JLabel)comp;
           // Does the auction item have 30 or more bids. If so display
           // a hot item image
          if(((Integer)value).intValue() >= 30) {
//API Ref :void setIcon(Icon icon)
            label.setIcon(new ImageIcon("Hot.gif"));
          } else {
            label.setIcon(new ImageIcon("Normal.gif"));
          }
          return label;
       }
    }
```

The renderer is set on a column like this:

```
      CustomRenderer custom = new CustomRenderer();
      custom.setHorizontalAlignment(JLabel.CENTER);
      scrollColumnModel.getColumn(2).setCellRenderer(custom);
```

If the component being displayed inside the JTable column requires more functionality than is available using a JLabel, you can create your own TableCellRenderer. This next code example uses a JButton as the renderer cell.

```
    class CustomButtonRenderer extends JButton implements TableCellRenderer {
       public CustomButtonRenderer() {
         setOpaque(true);
       }

//API Ref :Component getTableCellRendererComponent(JTable table, Object value,
          boolean isSelected, boolean hasFocus, int row, int column)
      public Component getTableCellRendererComponent(JTable table, Object value,
                       boolean isSelected, boolean hasFocus, int row, int column) {
      if(isSelected) {
        ((JButton)value).setForeground(table.getSelectionForeground());
        ((JButton)value).setBackground(table.getSelectionBackground());
      } else {
        ((JButton)value).setForeground(table.getForeground());
        ((JButton)value).setBackground(table.getBackground());
      }
      return (JButton)value;
    }
```

Like the default JLabel cell renderer, this class relies on an underlying component (in this case, JButton) to do the painting. Selection of the cell toggles the button colors. As before, the cell renderer is secured to the appropriate column of the auction table with the setCellRenderer method:

```
//API Ref :void setCellRenderer(TableCellRenderer renderer)
    scrollColumnModel.getColumn(3).setCellRenderer(new CustomButtonRenderer());
```

Alternately, all JButton components can be configured to use the CustomButtonRenderer in the table with a call to setDefaultRenderer as follows:

```
//API Ref :void setDefaultRenderer(Class columnClass, TableCellRenderer renderer)
    table.setDefaultRenderer(JButton.class, new CustomButtonRenderer());
```

Custom Cell Editing

In the same way that you can configure how a cell is painted in a JTable or JTree component, you can also configure how an editable cell responds to edits. One difference between using cell editors and cell renderers is that there is a DefaultCellEditor for all components, but no DefaultTableCellEditor for table cells.

While separate renderers exist for JTree and JTable, a single DefaultCellEditor class implements both the TableCellEditor and TreeCellEditor interfaces. However, the DefaultCellEditor class has constructors for only the JComboBox, JCheckBox, and JTextField components. The JButton class does not map to any of these constructors so a dummy JCheckBox is created to satisfy the requirements of the DefaultCellEditor class.

This next example from AuctionClient.java contains code to create a custom button editor that displays the number of days left in the auction when the button is double clicked. The double click to trigger the action is specified by setting the value clickCountToStart to two.

An exact copy of the getTableCellEditorComponent method paints the button in edit mode. A JDialog component that displays the number of days left appears when the getCellEditor-Value method is called. The value for the number of days left is calculated by moving the current calendar date towards the end date. The Calendar class does not have a method that expresses a difference in two dates in anything other than the milliseconds between those two dates.

```
    // From AuctionClient.java
    class CustomButtonEditor extends DefaultCellEditor {
      final JButton mybutton;
      JFrame frame;

      CustomButtonEditor(JFrame frame) {
        super(new JCheckBox());
        mybutton = new JButton();
        this.editorComponent = mybutton;
        // To activate this button requires to mouse clicks (double click)
        this.clickCountToStart = 2;
        this.frame=frame;
        mybutton.setOpaque(true);
        mybutton.addActionListener(new ActionListener() {
          public void actionPerformed(ActionEvent e) {
//API Ref :void fireEditingStopped()
            fireEditingStopped();
          }
        });
      }

      protected void fireEditingStopped() {
        super.fireEditingStopped();
      }

//API Ref :Object getCellEditorValue()
      public Object getCellEditorValue() {
        JDialog jd= new JDialog(frame, "Time left");
        Calendar today=Calendar.getInstance();
        Calendar end=Calendar.getInstance();
        SimpleDateFormat in=new SimpleDateFormat("yyyy-MM-dd");
        try {
          // Get the button text from the table row. This value is the end date
          // of the auction and was previously initialized when the table
          // was populated with data.
//API Ref : void setTime(Date date)
          end.setTime(in.parse(mybutton.getText()));
        } catch (Exception e){
          System.out.println("Error in date"+mybutton.getText()+e);
        }
        int days = 0;
        // To work out if the date in the button is later or earlier than
        // today, roll the date forward a day at a time until it is later
        // than today.
//API Ref :boolean before(Object when)
```

```
        while(today.before(end)) {
//API Ref :void roll(int datefield, boolean up)
        today.roll(Calendar.DATE, true);
        days++;
        }
        jd.setSize(200,100);
        // If the end date was originally after today, mark the auction as
        // completed. Otherwise, display the number of days remaining.
        if (today.after(end)) {
           jd.getContentPane().add(new JLabel("Auction completed"));
        } else {
          jd.getContentPane().add(new JLabel("Days left="+days));
        }
        jd.setVisible(true);
         return new String(mybutton.getText());
      }
//API Ref :Component getTableCellEditorComponent(JTable table, Object value, boolean
          isSelected, int row, int column)
     public Component getTableCellEditorComponent(JTable table, Object value,
                    boolean isSelected, int row, int column) {
        ((JButton) editorComponent).setText(((JButton)value).getText());
        if (isSelected) {
          ((JButton) editorComponent).setForeground(table.getSelectionForeground());
          ((JButton) editorComponent).setBackground(table.getSelectionBackground());
        } else {
          ((JButton) editorComponent).setForeground(table.getForeground());
          ((JButton) editorComponent).setBackground(table.getBackground());
        }
        return editorComponent;
        }
    }
```

Specialized Event Handling

Project Swing uses the event handling classes available in the AWT API since JDK 1.1. However, some new APIs are available in the SwingUtilities class that are used to add some control over the event queue. The two new event handling methods are invokeLater and invokeAndWait. The invokeAndWait method waits for the event to be processed in the event queue.

These methods are often used to request focus on a component after another event has occurred that might affect the component focus. You can return the focus by calling the invokeLater method and passing a Thread:

```
   JButton button =new JButton();
//API Ref :static void invokeLater(Runnable run)
```

```
SwingUtilities.invokeLater(new Runnable() {
  public void run() {
    button.requestFocus();
  }
});
```

Project Swing Directions

While the basic architecture of Project Swing has stayed true to its original design, many optimizations and improvements have been made to components like JTable and in areas such as scrolling. Add to this the Java HotSpot™ Performance Engine, which greatly reduces the cost of object creation, and Project Swing can boast its best performance to date.

However, as seen in the Analyze a Program (page 355) section, a simple 700 x 300 table requires nearly half a megabyte of memory when double buffered. The creation of ten tables would probably require swapping memory to disk, severely affecting performance on low-end machines.

Printing API

The Java 2 platform java.awt.print package lets you print anything that can be rendered to a Graphics or Graphics2D context--including AWT components, Project Swing components, and 2D graphics.

The Printing API is easy to use. Your application tells the printing system what to print, and the printing system determines when each page is rendered. This callback printing model enables printing support on a wide range of printers and systems. The callback model also lets users print to a bitmap printer from a computer that does not have enough memory or disk space to hold the bitmap for an entire page.

A graphics context lets a program paint to a rendering device such as a screen, printer, or off screen image. Because Project Swing components are rendered through a Graphics object using AWT graphics support, it is easy to print Project Swing components with the new printing API. However, AWT components are not rendered to a graphics device, so you must extend the AWT component class and implement the AWT component paint method.

What is in the Package?

The java.awt.print consists of the following interfaces, classes, and exceptions.

- Interfaces
  - •Pageable
  - •Printable

- PrinterGraphics
- Classes
  - Book
  - PageFormat
  - Paper
  - PrinterJob
- Exceptions
  - PrinterAbortException
  - PrinterException
  - PrinterIOException

Printing an AWT Component

The PrintButton (AWT) (page 280) application displays the panel with MyButton on it shown in Figure 22. When you click the button, the application prints the MyButton component.

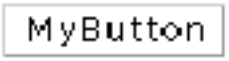


Figure 22 AWT Button

In the code, the Button class is extended to implement Printable and includes the paint and print method implementations. The print method is required because the class implements Printable, and the paint method is needed to describe how the button shape and label text looks when printed.

To see the button, the printer graphics context is translated into the imageable area of the printer, and to see the label text, a font is set on the printer graphics context.

In this example, the button is printed at a 164/72 inches inset from the left imageable margin (there are 72 pixels per inch) and 5/72 inches from the top imageable margin. This is where the button is positioned in the frame by the layout manager and those same numbers are returned by the following calls:

```
int X = (int)this.getLocation().getX();
int Y = (int)this.getLocation().getY();
```

And here is the MyButton class code:

```
    class MyButton extends Button implements Printable {
      public MyButton() {
        super("MyButton");
      }
      public void paint(Graphics g) {
      //To see the label text, you must specify a font for
      //the printer graphics context
        Font  f = new Font("Monospaced", Font.PLAIN,12);
        g.setFont (f);
      //Using "g" render anything you want.
      //Get the button's location, width, and height
        int X = (int)this.getLocation().getX();
        int Y = (int)this.getLocation().getY();
        int W = (int)this.getSize().getWidth();
        int H = (int)this.getSize().getHeight();
      //Draw the button shape
//API Ref :void drawRect(int x, int y, int width, int height)
        g.drawRect(X, Y, W, H);
      //Draw the button label
      //For simplicity code to center the label inside the
      //button shape is replaced by integer offset values
        g.drawString(this.getLabel(), X+10, Y+15);
      }
//API Ref :int print(Graphics graphics, PageFormat pageFormat, int pageIndex)
      public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
        if (pi >= 1) {
          return Printable.NO_SUCH_PAGE;
        }
        Graphics2D g2 = (Graphics2D) g;
      //To see the button on the printed page,translate the printer graphics
      //context into the imageable area
//API Ref :double getImageableX()
//API Ref :double getImageableY()
        g2.translate(pf.getImageableX(), pf.getImageableY());
        g2.setColor(Color.black);
        paint(g2);
        return Printable.PAGE_EXISTS;
       }
    }
```

Note: The printing Graphics2D is based on the BufferedImage class and on some platforms does not default to a foreground color of black. If this is the case on your platform, you have to add g2.setColor(Color.black) to the print method before the paint invocation.

The `PrintButton` class creates a `MyButton` instance and implements an `actionPerformed` method to create a printer job for `MyButton` when it is clicked.

Printing a Project Swing Component

Printing the Project Swing component shown in Figure 23 is almost the same as printing an AWT component, except the MyButton subclass does not need a paint method implementation. It does, however, have a print method that calls the paint method for the component. The paint method implementation is not needed because Project Swing components know how to paint themselves.



Figure 23 Project Swing Button

The PrintButton (Project Swing) (page 281) application for Project Swing displays a panel with a `MyButton` on it.

```
class MyButton extends JButton implements Printable {
  public MyButton() {
    super("MyButton");
  }
  public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
    if (pi >= 1) {
      return Printable.NO_SUCH_PAGE;
    }
    Graphics2D g2 = (Graphics2D) g;
    g2.translate(pf.getImageableX(),  pf.getImageableY());
    Font  f = new Font("Monospaced", Font.PLAIN,12);
    g2.setFont (f);
    paint(g2);
    return Printable.PAGE_EXISTS;
   }

//.. }
```

If you extend a JPanel and implement Printable, you can print a panel component and all of its contents.

```
public class printpanel extends JPanel implements ActionListener, Printable {
```

The PrintPanel (page 282) code prints a JPanel object and the JButton it contains, and the ComponentPrinterFrame (page 283) code prints a JFrame object and the JButton, JList, JCheckBox, and JComboBox components it contains.

Printing Graphics in Project Swing

In the same way the AWT example subclassed a Button component and implemented the paint method to draw the button, you can subclass an AWT or Project Swing component and implement the paint method to render 2D graphics to the screen or printer. The ShapesPrint

(page 284) Project Swing application borrowed from *The Java Tutorial* (`http://java.sun.com/docs/books/tutorial`) shows how this is done. It is modified for this article to include a TextLayout object.

The paintComponent method calls the drawShapes method to render the 2D graphics to the screen when the application starts. When you click the Print button, a printer graphics context is created and passed to the drawShapes method for the printing shown in Figure 24.

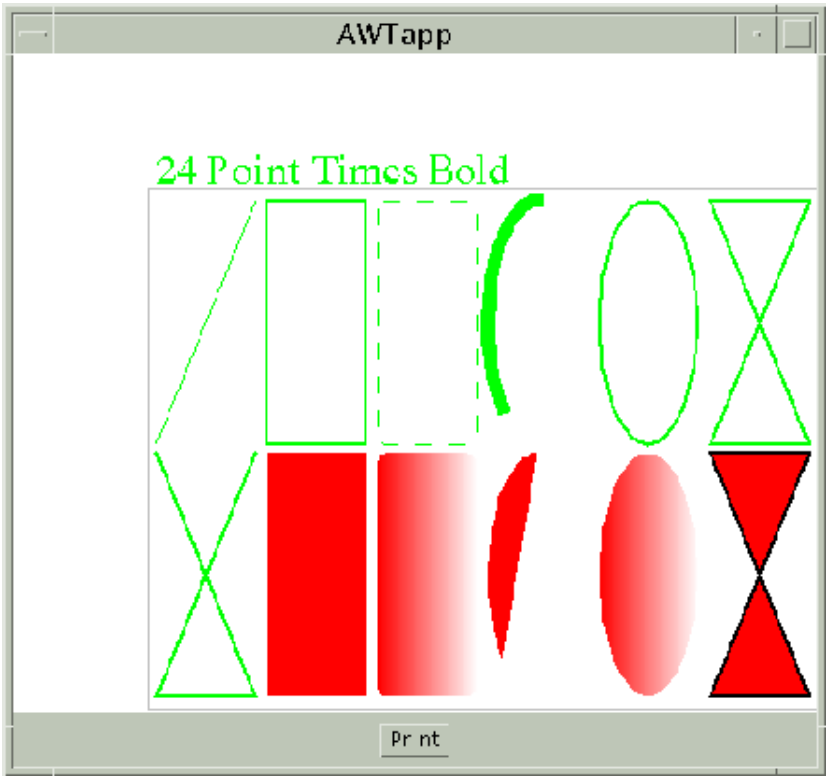


Figure 24 Printing Graphical Shapes

Print Dialog

It is easy to display a Print dialog (Figure 25) so the end user can interactively change the print job properties. The actionPerformed method of the previous Project Swing example is modified here to do just that.

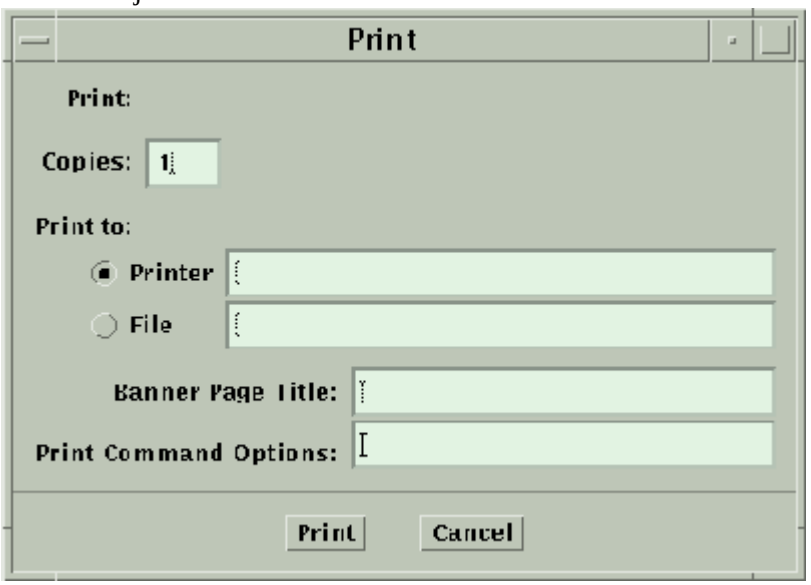


Figure 25 Print Dialog Box

```
    public void actionPerformed(ActionEvent e) {
      PrinterJob printJob = PrinterJob.getPrinterJob();
      printJob.setPrintable((MyButton) e.getSource());
//API Ref :boolean printDialog()
     if(printJob.printDialog()){
        try { printJob.print(); }
        catch (Exception PrinterExeption) { }
      }
    }
```

Note: In Project Swing, the printJob.setPageable((MyButton) e.getSource()); statement can be written as printJob.setPrintable((MyButton) e.getSource());. The difference is setPrintable is for applications that do not know the number of pages they are printing. If you use setPrintable, you need to add if(pi >= 1){return Printable.NO_SUCH_PAGE;} to the beginning of the print method. Pages are numbered starting with zero, so `pi >= 1` tests if the page index has two or more pages .

Page Setup Dialog

You can add a line of code that tells the PrinterJob object to display a Page dialog so the end user can interactively modify the page format for printing in portrait, landscape, or reverse landscape mode. The actionPerformed method of the previous Project Swing example is modified here to display Page and Print dialogs.

Note: Some platforms do not support a page dialog. On those platforms, the pageDialog call simply returns the passed-in PageFormat object and no dialog appears.

```
   public void actionPerformed(ActionEvent e) {
//API Ref :static PrinterJob getPrinterJob()
    static PrinterJob getPrinterJob()
     PrinterJob printJob = PrinterJob.getPrinterJob();
//API Ref :void setPrintable(Printable painter)
     printJob.setPrintable((MyButton) e.getSource());
//API Ref :PageFormat defaultPage()
//API Ref :PageFormat pageDialog(PageFormat page)
      PageFormat pf = printJob.pageDialog(printJob.defaultPage());
      if(printJob.printDialog()){
        try {
//API Ref :void print()
         printJob.print();
        } catch (Exception ex) {}
      }
    }
```

Printing a Collection of Pages

You can use the Book class to print a collection of pages that you append to the book. The pages can be in any order and have different page formats.

The Print2Button (page 287) example puts the `Print` and `Print 2` buttons of type `MyButton` on the panel shown in Figure 26. It creates a book that contains the pages to print. When you click either button, the book prints one copy of the Print button in landscape mode and two

copies of the Print 2 button in portrait more, as specified in the actionPerformed method implementation shown below.

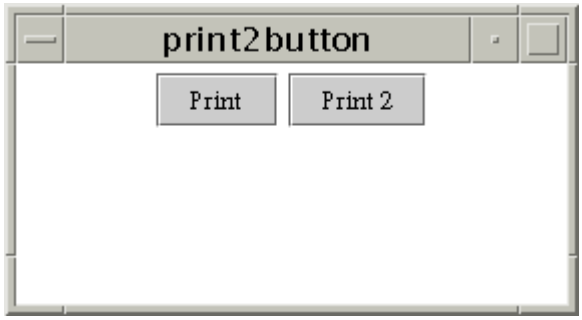


Figure 26 Printing a Collection of Pages

Note: Currently a bug restricts the Solaris platform to only print in portrait mode.

```
    public void actionPerformed(ActionEvent e) {
      PrinterJob printJob = PrinterJob.getPrinterJob();
    /* Set up Book */
      PageFormat landscape = printJob.defaultPage();
      PageFormat portrait = printJob.defaultPage();
//API Ref :void setOrientation(int orientation)
      landscape.setOrientation(PageFormat.LANDSCAPE);
      portrait.setOrientation(PageFormat.PORTRAIT);
      Book bk = new Book();
//API Ref :void append(Printable painter, PageFormat page)
      bk.append((Printable)b, landscape);
//API Ref :void append(Printable painter, PageFormat page, int numPages)
      bk.append((Printable)b2, portrait, 2);
//API Ref :void setPageable(Pageable document)
     printJob.setPageable(bk);
      try {
        printJob.print();
      } catch (Exception ex) { }
    }
```

Advanced Printing

The previous section explained how to print simple components and covered techniques that can be used to print screen captures. However, if you want to print more than one component per page, or if your component is larger than one page size, you need to do some additional work inside your print method. This section explains what you need to do and concludes with an example of how to print the contents of a JTable component.

Multiple Components Per Page

There are times when printing one component on a page does not meet your printing needs. For example, you might want to include a header on each page or print a footer with the page number--something that isn't necessarily displayed on the screen. Unfortunately, printing multiple customized components on a page is not as easy as adding additional paint calls because each paint call overwrites the output of the previous call.

The key to printing more than one component on a page is to use the translate(double, double) and setClip methods in the Graphics2D class. The translate method moves an imaginary pen to the next position of the print output where the component can be painted and then printed. There are two translate methods in the Graphics2D class. To print multiple components you need the one that takes double arguments because this translate method allows relative positioning. Be sure to cast any integer values to double or float. Relative positioning in this context means that previous calls to translate are taken into account when calculating the new translated point.

The setClip method is used to restrict the component to only be painted, and therefore printed, in the area specified. This lets you print multiple components on a page by moving the imaginary pen to different points on the page and painting each component in the clip area.

Example

You can replace the print method in the PrintButton (AWT) (page 280) and PrintButton (Project Swing) (page 281) examples with the following code to add the footer message Company Confidential to the page.

```
    public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
         if (pi >= 1) {
            return Printable.NO_SUCH_PAGE;
         }
         Graphics2D g2 = (Graphics2D) g;
         Font f= Font.getFont("Courier");
//API Ref :double getImageableWidth()
//API Ref :double getImageableHeight()
         double height=pf.getImageableHeight();
         double width=pf.getImageableWidth();
         g2.translate(pf.getImageableX(), pf.getImageableY());
         g2.setColor(Color.black);
         g2.drawString("Company Confidential", (int)width/2,
                              (int)height-g2.getFontMetrics().getHeight());
         g2.translate(0f,0f);
//API Ref :void setClip(int x, int y, int width, int height)
          g2.setClip(0,0,(int)width,(int)(height-g2.getFontMetrics().getHeight()*2));
         paint (g2);
         return Printable.PAGE_EXISTS;
    }
```

In the new print method, the Graphics2D context is clipped before calling the parent JButton paint method. This prevents the JButton paint method from overwriting the bottom of the page. The translate method is used to point the JButton paint method to start the paint at offset 0,0 from the visible part of the page. The visible area was already calculated by the previous translate call:

```
//API Ref :void translate(double x, double y)
    g2.translate(pf.getImageableX(), pf.getImageableY());
```

For some components, you might need to set the foreground color to see your results. In this example the text color is printed in black.

Useful Methods To Call In the print Method

The following methods are useful for calculating the number of pages required and for shrinking components to fit on a page:

PageFormat methods:

```
getImageableHeight()
```

Returns the page height you can use for printing your output.

```
getImageableWidth()
```

Returns the page width you can use for printing your output.

Graphics2D methods:

```
//API Ref :void scale(double sx, double sy)
   scale(xratio, yratio)
```

Scales the 2D graphics context by this size. A ratio of one maintains the size, less than one shrinks the graphics context.

Components Larger Than One Page

The Java 2 Printing API has a Book API that provides the concept of pages. However, the Book API only adds printable objects to a collection of printable objects. It does not calculate page breaks or split components over multiple pages.

When printing a simple component on a page, you only have to check for the index value being greater or equal to one and return NO_SUCH_PAGE when this value is reached. To print multiple pages, you have to calculate the number of pages needed to contain the component. You can calculate the total number of pages needed by subtracting the space taken

by the component from the value returned by getImageableHeight. Once the total number of pages is calculated, you can run the following check inside the print method:

```
if(pageIndex >=TotalPages) {
  return NO_SUCH_PAGE;
}
```

The Printing framework calls the print method multiple times until pageIndex is less than or equal to TotalPages. All you need to do is create a new page from the same component on each print loop. This is done by treating the printed page like a sliding window over the component.

The part of the component that is to be printed is selected by a translate call to mark the top of the page and a setClip call to mark the bottom of the page. Figure 27 illustrates this process. The left side of the diagram represents the page sent to the printer. The right side contains the long component being printed in the print method. The first page can be represented as follows:

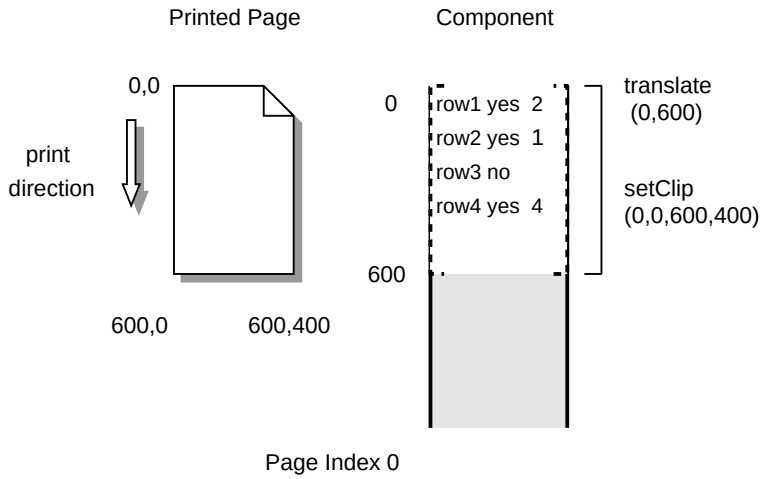


Figure 27 Translate Calls

Figure 28 shows how the printed page window slides along the component to print the second page, page index one. This process continues until the last page is reached.

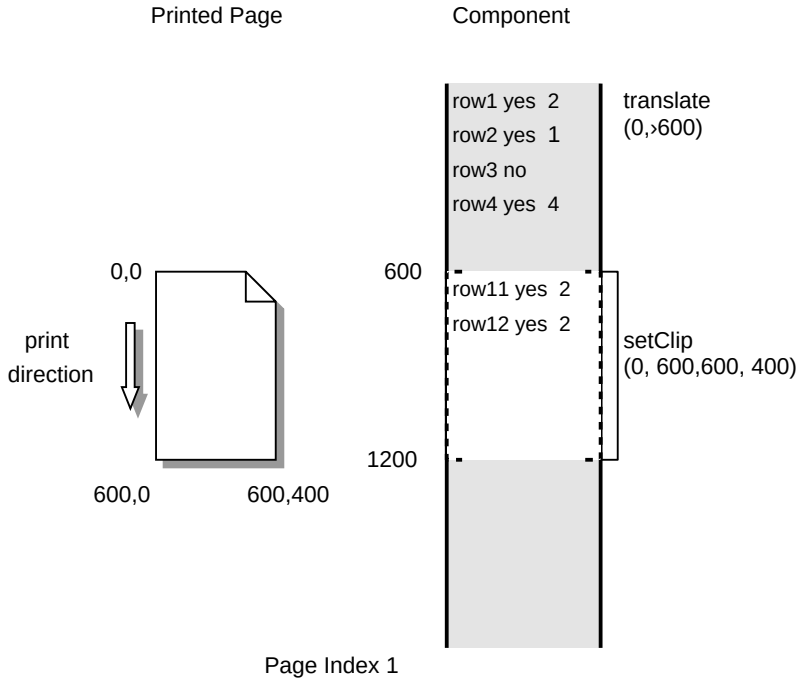


Figure 28 Printed Page Window Sliding

Printing A JTable Component

The Report (page 288) class uses many of the advanced techniques covered in this section to print out the data and header of a JTable component (Figure 29) that can span many pages. The printed output also includes a footer at the bottom with the page number.



Figure 29 Sales Report

Figure 30 shows how the report looks when it prints:

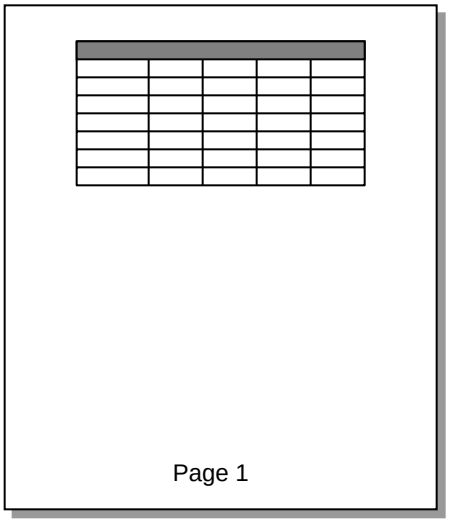


Figure 30 Printed Page

Print a Sales Report

The SalesReport (page 291) Applet class prints a sales report with the rows split over multiple pages with numbers at the bottom of each page. Figure 31 shows how the application looks when launched:

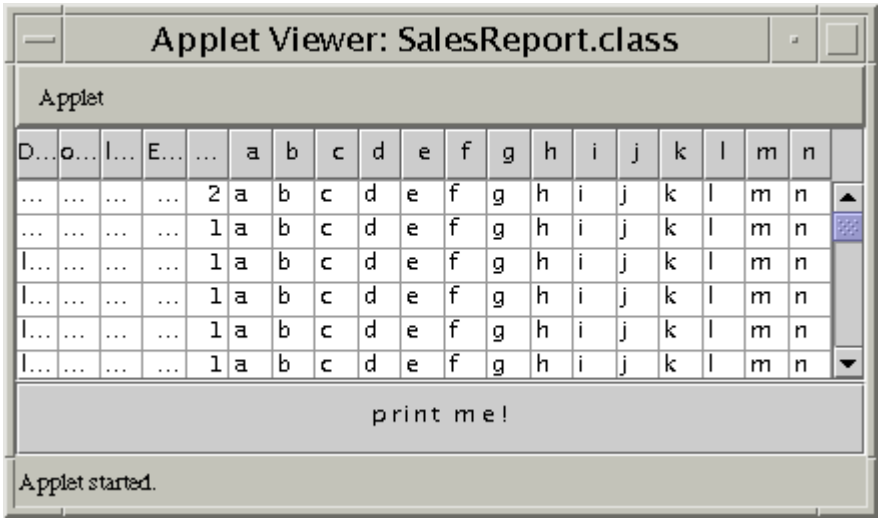


Figure 31 Sales Report

You need this policy file to launch the applet:

```
grant {
  permission java.lang.RuntimePermission "queuePrintJob";
};
```

To launch the applet assuming a policy file named printpol and an HTML file named SalesReport.html, you would type:

```
appletviewer -J-Djava.security.policy=printpol SalesReport.html
```

Figure 32 shows how the report prints:



Figure 32 Printed Sales Report

Code for this Chapter

- AuctionClient (page 270)
- PrintButton (AWT) (page 280)
- PrintButton (Project Swing) (page 281)
- PrintPanel (page 282)
- ComponentPrinterFrame (page 283)
- ShapesPrint (page 284)
- Print2Button (page 287)
- Report (page 288)
- SalesReport (page 291)

AuctionClient

```
package auction;

import java.io.*;
import javax.naming.*;
import javax.ejb.*;
import java.rmi.RemoteException;
import java.util.*;
import java.text.NumberFormat;
import bidder.*;
import registration.*;
import seller.*;
import search.*;
```

```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;
import java.awt.*;
import java.text.*;
import javax.swing.table.*;
import javax.swing.tree.*;

public class AuctionClient implements ActionListener{
  javax.naming.Context ctx=null;
  JMenuBar mb = new JMenuBar();
  JMenu file = new JMenu("File");
  JMenu reports = new JMenu("Reports");
  JMenuItem filePrint = new JMenuItem("Print");
  JSeparator separator = new JSeparator();
  JMenuItem fileQuit = new JMenuItem("Quit");
  JMenuItem itemList = new JMenuItem("List All Items");
  JMenuItem browseItems = new JMenuItem("Browse Items");
  JFrame frame=new JFrame();
  JViewport innerPort, headers;
  JScrollBar scrollBar;
  JScrollPane scrollPane;
  JTable fixedTable, scrollTable;
  TableColumnModel fixedColumnModel, scrollColumnModel;
  int rowHeight=64;
  JTree tree;
  DefaultMutableTreeNode[] nodes;
  JTextField findField;
  JPanel homePanel=null;
  JPanel browsePanel=null;
  JPanel scrollPanel=null;

  public AuctionClient() {
    try {
      ctx = getInitialContext();
    } catch (Exception e) {
      System.out.println("error contacting EJB server"+e);
    }

    // Add menu items to menus
    file.add(filePrint);
    file.add(separator);
    file.add(fileQuit);
    reports.add(itemList);
    reports.add(browseItems);
    mb.add(file);
    mb.add(reports);
    frame.setJMenuBar(mb);

    filePrint.addActionListener(this);
    fileQuit.addActionListener(this);
    itemList.addActionListener(this);
    browseItems.addActionListener(this);
```

```
  homePanel = new JPanel();
  homePanel.setLayout(new CardLayout());
  browsePanel = new JPanel();
  browsePanel.setLayout(new BorderLayout());
  scrollPanel = new JPanel();
  scrollPanel.setLayout(new BorderLayout());
  homePanel.add(scrollPanel, "List All Items");
  homePanel.add(browsePanel, "Browse Items");
  frame.getContentPane().add(homePanel, BorderLayout.CENTER);
  frame.setSize(200,100);
  frame.setVisible(true);
}
public void actionPerformed(ActionEvent evt) {
  Object obj = evt.getSource();
  if (obj == itemList) {
    try {
      listAllItems();
      ((CardLayout)homePanel.getLayout()).show(homePanel,
                                          ((JMenuItem)obj).getText());
    } catch (Exception e) {
      System.out.println("Exception: "+e);
    }
  } else if (obj == browseItems) {
    try {
      browseItems();
      ((CardLayout)homePanel.getLayout()).show(homePanel,
                                          ((JMenuItem)obj).getText());
    } catch (Exception e) {
      System.out.println("Exception: "+e);
    }
  } else if (obj == fileQuit) {
    frame.dispose();
    System.exit(0);
  }
}
static public javax.naming.Context getInitialContext() throws Exception {
  Properties p = new Properties();
  p.put(javax.naming.Context.INITIAL_CONTEXT_FACTORY,
                            "weblogic.jndi.TengahInitialContextFactory");
  return new InitialContext(p);
}
private void listAllItems() throws IOException{
  ResultsModel rm=new ResultsModel();
   try {
      BidderHome bhome=(BidderHome)ctx.lookup("bidder");
      Bidder bid=bhome.create();
      Enumeration enum=(Enumeration)bid.getItemList();
      if (enum != null) {
        rm.update(enum);
      }
```

```
         } catch (Exception e) {
             System.out.println("AuctionServlet <list>:"+e);
         }
        // Create a new table using the ResultsModel object as the table model
        scrollTable=new JTable(rm);
        // Force the End Date columns and Description columns to have
        // a set width
        adjustColumnWidth(scrollTable.getColumn("End Date"), 150);
        adjustColumnWidth(scrollTable.getColumn("Description"), 120);
        // The scroll column model is initially mapped to
        // display all the table data. The fixed column model
        // is initially empty
        scrollColumnModel = scrollTable.getColumnModel();
        fixedColumnModel = new DefaultTableColumnModel();

        // Get the first column, remove it from the scroll column and
        // add it to the fixed table
        TableColumn col = scrollColumnModel.getColumn(0);
        scrollColumnModel.removeColumn(col);
        fixedColumnModel.addColumn(col);

        // Create a new table based on the fixed column model (1 column)
        fixedTable = new JTable(rm,fixedColumnModel);

        // Keep the heights of the fixed and scroll tables the same
        fixedTable.setRowHeight(scrollTable.getRowHeight());

        headers = new JViewport();
        // Get the selection model used in this table and override it
        // so we can also update the scroll table if the fixed table
        // is selected and vice-versa
        ListSelectionModel fixedSelection = fixedTable.getSelectionModel();
//API Ref :void addListSelectionListener(ListSelectionListener l)
        fixedSelection.addListSelectionListener(
            new ListSelectionListener() {
                public void valueChanged(ListSelectionEvent e) {
                  ListSelectionModel lsm = (ListSelectionModel)e.getSource();
                  if (!lsm.isSelectionEmpty())  {
                     setScrollableRow();
                  }
                }
         });
//API Ref :ListSelectionMode getSelectionModel()
         ListSelectionModel scrollSelection = scrollTable.getSelectionModel();
         scrollSelection.addListSelectionListener(new ListSelectionListener() {
           public void valueChanged(ListSelectionEvent e) {
             ListSelectionModel lsm = (ListSelectionModel)e.getSource();
             if (!lsm.isSelectionEmpty())  {
               setFixedRow();
             }
           }
```

```
        });
        // Create a custom renderer and use it on column 2
        CustomRenderer custom = new CustomRenderer();
        custom.setHorizontalAlignment(JLabel.CENTER);
//API Ref :void setCellRenderer(TableCellRenderer renderer)
        scrollColumnModel.getColumn(2).setCellRenderer(custom);
        // Use the custom button renderer on column 3
        scrollColumnModel.getColumn(3).setCellRenderer(new CustomButtonRenderer());
        // Add the custom button editor to column 3
        CustomButtonEditor customEdit=new CustomButtonEditor(frame);
        scrollColumnModel.getColumn(3).setCellEditor(customEdit);

        // Add the scrolling column headers to the header viewport
        // the headers viewport is later added to the headerPanel with
        // an associated scroll bar
//API Ref :JTableHeader getTableHeader()
        headers.add(scrollTable.getTableHeader());

        // Create a panel for the top left of the display that contains
        // the headers for the column that does not move
        JPanel topPanel = new JPanel();
        topPanel.setLayout(new BoxLayout(topPanel, BoxLayout.X_AXIS));
        adjustColumnWidth(fixedColumnModel.getColumn(0), 100);
        JTableHeader fixedHeader=fixedTable.getTableHeader();
//API Ref :void setAlignmentY()
        fixedHeader.setAlignmentY(Component.TOP_ALIGNMENT);
        topPanel.add(fixedHeader);
//API Ref :Component createRigidArea(Dimension d)
        topPanel.add(Box.createRigidArea(new Dimension(2, 0)));
        topPanel.setPreferredSize(new Dimension(400, 40));

        // Create a panel to contain the scroll headers that will
        // displayed at the top right
        JPanel headerPanel = new JPanel();
        headerPanel.setAlignmentY(Component.TOP_ALIGNMENT);
        headerPanel.setLayout(new BorderLayout());
        JScrollPane scrollpane = new JScrollPane();
//API Ref :JScrollBar getHorizontalScrollBar()
        scrollBar = scrollpane.getHorizontalScrollBar();
        headerPanel.add(headers, "North");
        headerPanel.add(scrollBar, "South");
        topPanel.add(headerPanel);
        // set the scrollable part of both the table views
        scrollTable.setPreferredScrollableViewportSize(new Dimension(300,180));
        fixedTable.setPreferredScrollableViewportSize( new Dimension(100,180));
        fixedTable.setPreferredSize(new Dimension(100,180));
        innerPort = new JViewport();

        // If the user moves the scrollbar at the top
        // of the scrollable data update the header and table
        // data positions so they are viewable
//API Ref :void setView(Component view)
```

```
        innerPort.setView(scrollTable);
        scrollpane.setViewport(innerPort);
         scrollBar.getModel().addChangeListener(new ChangeListener()  {
          public void stateChanged(ChangeEvent e) {
//API Ref :Point getViewPosition()
            Point q = headers.getViewPosition();
            Point p = innerPort.getViewPosition();
            int val = scrollBar.getModel().getValue();
            p.x = val;
            q.x = val;

//API Ref :void setView(Component view)
            headers.setViewPosition(p);
            headers.repaint(headers.getViewRect());
            innerPort.setViewPosition(p);
            innerPort.repaint(innerPort.getViewRect());
          }
        });

        // Disable continuous update of the scroll table. Needed to
        // improve performance of a scrolling table in SDK 1.2
        scrollTable.getTableHeader().setUpdateTableInRealTime(false);

        // The bottom panel is where both tables data is positioned
        JPanel bottomPanel = new JPanel();
        bottomPanel.setLayout(new BoxLayout(bottomPanel, BoxLayout.X_AXIS));
        fixedTable.setAlignmentY(Component.TOP_ALIGNMENT);
        bottomPanel.add(fixedTable);
        bottomPanel.add(Box.createRigidArea(new Dimension(2, 0)));
        innerPort.setAlignmentY(Component.TOP_ALIGNMENT);
        bottomPanel.add(innerPort);
        bottomPanel.add(Box.createRigidArea(new Dimension(2, 0)));

        // Only allow vertical scrollbars as we are managing the
        // horizontal scroll ourselves
        scrollPane = new JScrollPane(bottomPanel,
                        JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
                        JScrollPane.HORIZONTAL_SCROLLBAR_NEVER);
        JViewport outerPort = new JViewport();
        outerPort.add(bottomPanel);
        // Workaround to affix the headers to the top of the scrollpane
        // uses setColumnHeaderView
//API Ref :void setColumnHeaderView(Component view)
         scrollPane.setColumnHeaderView(topPanel);
//API Ref :void setViewport(JViewport viewport)
        scrollPane.setViewport(outerPort);
        scrollTable.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
        frame.getContentPane().add(scrollPane);
        scrollTable.validate();
        frame.setSize(450,200);
     }
     void setFixedRow() {
       int index=scrollTable.getSelectedRow();
```

```
  fixedTable.setRowSelectionInterval(index, index);
}
void setScrollableRow() {
  // Highlight the row by calling setRowSelectionInterval with a
  // range of 1
  int index=fixedTable.getSelectedRow();
  scrollTable.setRowSelectionInterval(index, index);
}
void adjustColumnWidth(TableColumn c, int size) {
  c.setPreferredWidth(size);
  c.setMaxWidth(size);
  c.setMinWidth(size);
}

private void browseItems() throws IOException {
  JPanel findPanel;
  JButton findButton;
  Vector newItems=new Vector();
  Vector closedItems=new Vector();
  Bidder bid=null;
  Enumeration enum;
    String[] treelabels = { “All Auctions”, “Closed Auction”, “New Auctions”};
    try {
      BidderHome bhome=(BidderHome) ctx.lookup(“bidder”);
      bid=bhome.create();
      enum=(Enumeration)bid.getNewItemList();
      while((enum !=null) && (enum.hasMoreElements())) {
        while(enum.hasMoreElements()) {
          AuctionItem auctionItem=(AuctionItem)enum.nextElement();
          newItems.addElement(new Integer(auctionItem.getId()));
        }
      }
      enum=(Enumeration)bid.getClosedItemList();
      while((enum !=null) && (enum.hasMoreElements())) {
        while(enum.hasMoreElements()) {
          AuctionItem auctionItem=(AuctionItem)enum.nextElement();
          closedItems.addElement(new Integer(auctionItem.getId()));
         }
      }
   } catch (Exception e) {
     System.out.println(“AuctionServlet <list>:”+e);
   }
   // Create an array of nodes based on the size of the labels and
   // open and closed items
   nodes = new DefaultMutableTreeNode[treelabels.length];
   DefaultMutableTreeNode[] closednodes = new
                                  DefaultMutableTreeNode[closedItems.size()];
   DefaultMutableTreeNode[] newnodes = new
                                 DefaultMutableTreeNode[newItems.size()];

   for(int i=0; i<treelabels.length; i++) {
     nodes[i] = new DefaultMutableTreeNode(treelabels[i]);
   }
```

```
    // Arrange the tree so that from the top level, all
    // closed and open auctions are children from that top level
    nodes[0].add(nodes[1]);
    nodes[0].add(nodes[2]);
    for(int i=0; i<closedItems.size(); i++) {
       closednodes[i] = new DefaultMutableTreeNode(closedItems.elementAt(i));
       nodes[1].add(closednodes[i]);
    }
    for(int i=0; i<newItems.size(); i++) {
       newnodes[i] = new DefaultMutableTreeNode(newItems.elementAt(i));
       nodes[2].add(newnodes[i]);
    }
    DefaultTreeModel model=new DefaultTreeModel(nodes[0]);
    tree = new JTree(model);
    JScrollPane scroll = new JScrollPane(tree);
    browsePanel.add(scroll, BorderLayout.CENTER);
    findPanel= new JPanel();
    findField= new JTextField(10);
    findButton= new JButton("find");
    findButton.addActionListener (new ActionListener() {
      public void actionPerformed (ActionEvent e) {
        String field=findField.getText();
        if(field != null) {
          findNode(findField.getText());
        } else {
          return;
        }
      }
    });
    findPanel.add(findField);
    findPanel.add(findButton);
    browsePanel.add(findPanel, BorderLayout.SOUTH);
    frame.setSize(300,350);
 }
 private void findNode(String field) {
   // Get an enumeration list of all the nodes.
   // The ordering is generated by starting from the root
   // to the end of each branch, and to the end of
   // the next branch including any nodes not visited before

   Enumeration e = nodes[0].depthFirstEnumeration();
   Object currNode;
   while(e.hasMoreElements()) {
     currNode = e.nextElement();
     if(currNode.toString().equals(field)) {
       TreePath path = new TreePath(((
                          DefaultMutableTreeNode)currNode).getPath());
       tree.makeVisible(path);
       tree.setSelectionRow(tree.getRowForPath(path));
       return;
     }
   }
}
```

```
  public static void main(String args[]) {
    AuctionClient ac= new AuctionClient();
  }
}
class CustomRenderer extends DefaultTableCellRenderer {
  public Component getTableCellRendererComponent(JTable table,
                                Object value, boolean isSelected,
                               boolean hasFocus, int row, int column) {
   Component comp = super.getTableCellRendererComponent(table, value, isSelected,
                               hasFocus, row,column);
     JLabel label = (JLabel)comp;
     // Does the auction item have 30 or more bids? If so, display
     // a hot item image
       if(((Integer)value).intValue() >= 30) {
         label.setIcon(new ImageIcon("Hot.gif"));
        } else {
          label.setIcon(new ImageIcon("Normal.gif"));
        }
        return (JLabel)label;
     }
  }
class CustomButtonEditor extends DefaultCellEditor {
    final JButton mybutton;
    JFrame frame;
    CustomButtonEditor(JFrame frame) {
      super(new JCheckBox());
      mybutton = new JButton();
      this.editorComponent = mybutton;
      // To activate this button requires to mouse clicks (double click)
      this.clickCountToStart = 2;
      this.frame=frame;
      mybutton.setOpaque(true);
      mybutton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
          fireEditingStopped();
        }
    });
  }
  protected void fireEditingStopped() {
    super.fireEditingStopped();
  }
  public Object getCellEditorValue() {
    JDialog jd= new JDialog(frame, "Time left");
    Calendar today=Calendar.getInstance();
    Calendar end=Calendar.getInstance();
    SimpleDateFormat in=new SimpleDateFormat("yyyy-MM-dd");
    try {
      // Get the button text from the table row. This value is the end date
      // of the auction and was previously initialised when the table
      // was populated with data.
      end.setTime(in.parse(mybutton.getText()));
    } catch (Exception e){
      System.out.println("Error in date"+mybutton.getText()+e);
```

```
    }
    int days = 0;
    // To work out if the date in the button is later or earlier than
    // today, roll the date forward a day at a time until it is later
    // than today
    while(today.before(end)) {
      today.roll(Calendar.DATE,true);
      days++;
    }
    jd.setSize(200,100);
    // If the end date was originally after today, mark the auction as
    // completed. Otherwise, display the number of days remaining.
    if(today.after(end)) {
      jd.getContentPane().add(new JLabel("Auction completed"));
    } else {
      jd.getContentPane().add(new JLabel("Days left="+days));
    }
    jd.setVisible(true);
    return new String(mybutton.getText());
  }
  public Component getTableCellEditorComponent(JTable table, Object value,
                                boolean isSelected, int row, int column) {
    ((JButton) editorComponent).setText(((JButton)value).getText());
    if(isSelected) {
       ((JButton) editorComponent).setForeground(table.getSelectionForeground());
      ((JButton) editorComponent).setBackground(table.getSelectionBackground());
    } else {
      ((JButton) editorComponent).setForeground(table.getForeground());
      ((JButton) editorComponent).setBackground(table.getBackground());
    }
    return editorComponent;
  }
}
class CustomButtonRenderer extends JButton implements TableCellRenderer {
  public CustomButtonRenderer() {
    setOpaque(true);
  }
  public Component getTableCellRendererComponent(JTable table,
    Object value, boolean isSelected, boolean hasFocus,
    int row, int column) {
    if(isSelected) {
      ((JButton)value).setForeground(table.getSelectionForeground());
      ((JButton)value).setBackground(table.getSelectionBackground());
    } else {
      ((JButton)value).setForeground(table.getForeground());
      ((JButton)value).setBackground(table.getBackground());
    }
    return (JButton)value;
  }
}
```

PrintButton (AWT)

```
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;

class MyButton extends Button implements Printable {
  public MyButton() {
    super("MyButton");
  }
  public void paint(Graphics g) {
    Font  f = new Font("Monospaced",Font.PLAIN,12);
    g.setFont (f);
  //Using "g" render anything you want.
  //Get the button's location, width, and height
    int X = (int)this.getLocation().getX();
    int Y = (int)this.getLocation().getY();
    int W = (int)this.getSize().getWidth();
    int H = (int)this.getSize().getHeight();
  //Draw the button shape
    g.drawRect(X, Y, W, H);
  //Draw the button label
  //For simplicity code to center the label inside the
  //button shape is replaced by integer offset values
    g.drawString(this.getLabel(), X+3, Y+15);
  }
  public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
    if (pi >= 1) {
      return Printable.NO_SUCH_PAGE;
    }
    Graphics2D g2 = (Graphics2D) g;
    g2.translate(pf.getImageableX(), pf.getImageableY());
    g2.setColor(Color.black);
    paint(g2);
    return Printable.PAGE_EXISTS;
  }
}
public class PrintButton extends Panel implements ActionListener {
    public PrintButton() {setBackground(Color.white);
        MyButton b = new MyButton();
        b.addActionListener(this);
        add(b);
    }
    public void actionPerformed(ActionEvent e) {
        PrinterJob printJob = PrinterJob.getPrinterJob();
        printJob.setPrintable((MyButton) e.getSource());
        try { printJob.print(); } catch (Exception PrintException) { }
    }
    public static void main(String s[]) {WindowListener l = new WindowAdapter() {
     public void windowClosing(WindowEvent e) {System.exit(0);}};
       Frame f = new Frame("printbutton");
       f.addWindowListener(l);
       f.add("Center", new printbutton());
```

```
        f.pack();
        f.setSize(new Dimension(400,300));
        f.show();
      }
    }
```

PrintButton (Project Swing)

```
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;
import javax.swing.*;

class MyButton extends JButton implements Printable {
  public MyButton(String label) {
    super(label);
  }
   public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
//There is only one page so this check is ok
    if(pi >= 1) {
      return Printable.NO_SUCH_PAGE;
    }
    Graphics2D g2 = (Graphics2D) g;
    g2.translate(pf.getImageableX(), pf.getImageableY());
    Font f = new Font("Courier", Font.PLAIN, 12);
    g2.setFont (f);
    paint(g2);
    return Printable.PAGE_EXISTS;
  }
}
public class PrintButton extends JPanel implements ActionListener {
  public PrintButton() {
    setBackground(Color.white);
    MyButton b = new MyButton("MyButton");
    b.addActionListener(this);
    add(b);
  }
  public void actionPerformed(ActionEvent e) {
    PrinterJob printJob = PrinterJob.getPrinterJob();
    printJob.setPrintable((MyButton) e.getSource());
//Page dialoh
    PageFormat pf = printJob.pageDialog(printJob.defaultPage());
//Print dialog
    if(printJob.printDialog()){
      try { printJob.print(); } catch (Exception PrintException) { }
    }
//No dialogs
    try { printJob.print(); } catch (Exception PrintException) { }
  }
  public static void main(String s[]) {
    WindowListener l = new WindowAdapter() {
      public void windowClosing(WindowEvent e) {System.exit(0);}
```

```
    };
    Frame f = new Frame("printbutton");
    f.addWindowListener(l);
    f.add("Center", new printbutton());
    f.pack();
    f.setSize(new Dimension(400,300));
    f.show();
  }
}
```

PrintPanel

```
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;
import javax.swing.*;

public class PrintPanel extends JPanel implements ActionListener, Printable {
  public PrintPanel() {
    setBackground(Color.white);
    JButton b = new JButton("MyButton");
    b.addActionListener(this);
    add(b);
  }
  public void actionPerformed(ActionEvent e) {
    PrinterJob printJob = PrinterJob.getPrinterJob();
    printJob.setPrintable(this);
    try { printJob.print(); } catch (Exception PrintException) { }
  }
  public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
     if(pi >= 1) {
       return Printable.NO_SUCH_PAGE;
      }
      g.translate(100, 100);
      Font  f = new Font("Monospaced",Font.PLAIN,12);
      g.setFont (f);
      paint (g);
      return Printable.PAGE_EXISTS;
  }
  public static void main(String s[]) {
    WindowListener l = new WindowAdapter() {
      public void windowClosing(WindowEvent e) {System.exit(0);}
    };
    Frame f = new Frame("printbutton");
    f.addWindowListener(l);
    f.add("Center", new PrintPanel());
    f.pack();
    f.setSize(new Dimension(400,300));
    f.show();
  }
}
```

ComponentPrinterFrame

```
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;
import javax.swing.*;

public class ComponentPrinterFrame
  extends JFrame
  implements Printable {
  public static void main(String[] args) {
    ComponentPrinterFrame cpf = new ComponentPrinterFrame();
    cpf.setVisible(true);
  }
  public ComponentPrinterFrame() {
    super("ComponentPrinterFrame v1.0");
    createUI();
  }
  protected void createUI() {
    JPanel panel = new JPanel();
    JButton printButton = new JButton("Print");
    panel.add(printButton);
    panel.add(new JList(new Object[] { "One", "Two", "Three" }));
    panel.add(new JButton("Push me"));
    panel.add(new JCheckBox("Chess", true));
    panel.add(new JComboBox(new Object[] { "Eins", "Zwei", "Drei" }));

    printButton.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent ae) {
        PrinterJob pj = PrinterJob.getPrinterJob();
        pj.setPrintable(ComponentPrinterFrame.this);
        if(pj.printDialog()) {
          try { pj.print(); }
          catch (PrinterException pe) {
            System.out.println(pe);
          }
        }
      }
    });
    setContentPane(panel);
    setSize(400, 400);
    // Center.
    Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
    Dimension frameSize = getSize();
    int x = (screenSize.width - frameSize.width) / 2;
    int y = (screenSize.height - frameSize.height) / 2;
    setLocation(x, y);
     addWindowListener(new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        dispose();
        System.exit(0);
      }
    });
```

```
  }
  public int print(Graphics g, PageFormat pf, int pageIndex) {
    if (pageIndex != 0) return NO_SUCH_PAGE;
    Graphics2D g2 = (Graphics2D)g;
    g2.translate(pf.getImageableX(), pf.getImageableY());
    getContentPane().paint(g2);
    return PAGE_EXISTS;
  }
}
```

ShapesPrint

```
import java.awt.geom.*;
import java.awt.font.*;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.print.PrinterJob;
import java.awt.event.*;
import java.awt.*;
import java.awt.print.*;

public class ShapesPrint extends Panel implements Printable, ActionListener {
  final static Color bg = Color.white;
  final static Color fg = Color.black;
  final static Color red = Color.red;
  final static Color white = Color.white;
  final static BasicStroke stroke = new BasicStroke(2.0f);
  final static BasicStroke wideStroke = new BasicStroke(8.0f);
  final static float dash1[] = {10.0f};
  final static BasicStroke dashed = new BasicStroke(1.0f,
              BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dash1, 0.0f);
  final static Button button = new Button("Print");
  public ShapesPrint() {
    setBackground(bg);
    button.addActionListener(this);
  }
   public void actionPerformed(ActionEvent e) {
     if(e.getSource() instanceof Button) {
       PrinterJob printJob = PrinterJob.getPrinterJob();
       printJob.setPrintable(this);
       if(printJob.printDialog()) {
         try {
           printJob.print();
         } catch (Exception PrintException) {
           PrintException.printStackTrace();
         }
       }
     }
    }
    public void paint(Graphics g) {
      super.paint(g);
      Graphics2D g2 = (Graphics2D) g;
```

```
    drawShapes(g2);
}
public void drawShapes(Graphics2D g2){
  Dimension d = getSize();
  int gridWidth = 400 / 6;
  int gridHeight = 300 / 2;
  int rowspacing = 5;
  int columnspacing = 7;
  int rectWidth = gridWidth - columnspacing;
  int rectHeight = gridHeight - rowspacing;
  Color fg3D = Color.lightGray;
  g2.setPaint(fg3D);
  g2.drawRect(80, 80, 400 - 1, 310);
  g2.setPaint(fg);
  int x = 85;
  int y = 87;
  //draw Text Layout
  FontRenderContext frc = g2.getFontRenderContext();
  Font f = new Font("Times",Font.BOLD, 24);
  String s = new String("24 Point Times Bold");
  TextLayout tl = new TextLayout(s, f, frc);
  g2.setColor(Color.green);
  tl.draw(g2, x, y-10);
  // draw Line2D.Double
  g2.draw(new Line2D.Double(x, y+rectHeight-1, x + rectWidth, y));
  x += gridWidth;
 // draw Rectangle2D.Double
 g2.setStroke(stroke);
 g2.draw(new Rectangle2D.Double(x, y, rectWidth, rectHeight));
 x += gridWidth;
 // draw  RoundRectangle2D.Double
 g2.setStroke(dashed);
 g2.draw(new RoundRectangle2D.Double(x, y, rectWidth, rectHeight, 10, 10));
 x += gridWidth;
 // draw Arc2D.Double
 g2.setStroke(wideStroke);
  g2.draw(new Arc2D.Double(x, y, rectWidth, rectHeight, 90, 135, Arc2D.OPEN));
 x += gridWidth;
 // draw Ellipse2D.Double
 g2.setStroke(stroke);
 g2.draw(new Ellipse2D.Double(x, y, rectWidth, rectHeight));
 x += gridWidth;
 // draw GeneralPath (polygon)
 int x1Points[] = {x, x+rectWidth, x, x+rectWidth};
 int y1Points[] = {y, y+rectHeight, y+rectHeight, y};
 GeneralPath polygon = new GeneralPath(GeneralPath.WIND_EVEN_ODD,
                                       x1Points.length);
 polygon.moveTo(x1Points[0], y1Points[0]);
 for(int index = 1; index < x1Points.length; index++ ) {
    polygon.lineTo(x1Points[index], y1Points[index]);
 };
 polygon.closePath();
 g2.draw(polygon);
```

```
        // NEW ROW
        x = 85;
        y += gridHeight;
        // draw GeneralPath (polyline)
        int x2Points[] = {x, x+rectWidth, x, x+rectWidth};
        int y2Points[] = {y, y+rectHeight, y+rectHeight, y};
        GeneralPath polyline = new GeneralPath(GeneralPath.WIND_EVEN_ODD,
                                               x2Points.length);
        polyline.moveTo (x2Points[0], y2Points[0]);
        for(int index = 1; index < x2Points.length; index++ ) {
           polyline.lineTo(x2Points[index], y2Points[index]);
        };
        g2.draw(polyline);
        x += gridWidth;
        // fill Rectangle2D.Double (red)
        g2.setPaint(red);
        g2.fill(new Rectangle2D.Double(x, y, rectWidth, rectHeight));
        g2.setPaint(fg);
        x += gridWidth;
        // fill RoundRectangle2D.Double
        GradientPaint redtowhite = new GradientPaint(x,y,red,x+rectWidth, y,white);
        g2.setPaint(redtowhite);
        g2.fill(new RoundRectangle2D.Double(x, y, rectWidth, rectHeight, 10, 10));
        g2.setPaint(fg);
        x += gridWidth;
        // fill Arc2D
        g2.setPaint(red);
        g2.fill(new Arc2D.Double(x, y, rectWidth, rectHeight, 90,135, Arc2D.OPEN));
        g2.setPaint(fg);
        x += gridWidth;
        // fill Ellipse2D.Double
        redtowhite = new GradientPaint(x,y,red,x+rectWidth, y,white);
        g2.setPaint(redtowhite);
        g2.fill (new Ellipse2D.Double(x, y, rectWidth, rectHeight));
        g2.setPaint(fg);
        x += gridWidth;
        // fill and stroke GeneralPath
        int x3Points[] = {x, x+rectWidth, x, x+rectWidth};
        int y3Points[] = {y, y+rectHeight, y+rectHeight, y};
        GeneralPath filledPolygon = new GeneralPath(GeneralPath.WIND_EVEN_ODD,
                                                    x3Points.length);
        filledPolygon.moveTo(x3Points[0], y3Points[0]);
        for (int index = 1; index < x3Points.length; index++ ) {
           filledPolygon.lineTo(x3Points[index], y3Points[index]);
        }
        filledPolygon.closePath();
        g2.setPaint(red);
        g2.fill(filledPolygon);
        g2.setPaint(fg);
        g2.draw(filledPolygon);
    }
    public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
      if(pi >= 1) {
```

```
      return Printable.NO_SUCH_PAGE;
    }
    Graphics g2 = button.getGraphics();
    button.printAll(g2);
    drawShapes((Graphics2D) g);
    return Printable.PAGE_EXISTS;
   }
   public static void main(String s[]){
     WindowListener l = new WindowAdapter() {
       public void windowClosing(WindowEvent e) {System.exit(0);}
       public void windowClosed(WindowEvent e) {System.exit(0);}
     };
     Frame f = new Frame();
     f.addWindowListener(l);
     Panel panel = new Panel();
     f.add(BorderLayout.SOUTH, panel);
     f.add(BorderLayout.CENTER, new ShapesPrint());
     panel.add(button);
     f.add(BorderLayout.SOUTH, panel);
     f.setSize(580, 500);
     f.show();
   }
}
```

Print2Button

```
import java.awt.*;
import java.awt.event.*;
import java.awt.print.*;
import javax.swing.*;

class MyButton extends JButton implements Printable {
  public MyButton() {
    super("Print");
  }
  public int print(Graphics g, PageFormat pf, int pi) throws PrinterException {
    Graphics2D g2 = (Graphics2D) g;
    g2.translate(pf.getImageableX(), pf.getImageableY());
    Font  f = new Font("Monospaced",Font.PLAIN,12);
    g2.setFont (f);
    paint (g2);
    return Printable.PAGE_EXISTS;
  }
}
public class Print2Button extends JPanel implements ActionListener {
  private JButton b, b2;
  public Print2Button() {
    setBackground(Color.white);
    b = new MyButton();
    b2 = new MyButton();
    b2.setLabel("Print 2");
    b.addActionListener(this);
```

```
    b2.addActionListener(this);
    add(b);
    add(b2);
  }
  public void actionPerformed(ActionEvent e) {
    PrinterJob printJob = PrinterJob.getPrinterJob();
/* Set up Book */
    PageFormat landscape = printJob.defaultPage();
    PageFormat portrait = printJob.defaultPage();
    landscape.setOrientation(PageFormat.LANDSCAPE);
    landscape.setOrientation(PageFormat.PORTRAIT);
    Book bk = new Book();
    bk.append((Printable)b, landscape);
    bk.append((Printable)b2, portrait, 2);
    printJob.setPageable(bk);
//Page dialog
    PageFormat pf = printJob.pageDialog(printJob.defaultPage());
//Print dialog
    if(printJob.printDialog()){
      try { printJob.print(); } catch (Exception PrintException) { }
    }
/*
    try { printJob.print(); } catch (Exception PrintException) { }
*/
  }
  public static void main(String s[]) {
    WindowListener l = new WindowAdapter() {
      public void windowClosing(WindowEvent e) {System.exit(0);}
    };
    Frame f = new Frame("print2button");
    f.addWindowListener(l);
    f.add("Center", new print2button());
    f.pack();
    f.setSize(new Dimension(400,300));
    f.show();
  }
}
```

Report

```
import javax.swing.*;
import javax.swing.table.*;
import java.awt.print.*;
import java.util.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import java.awt.Dimension;

public class Report implements Printable{
  JFrame frame;
  JTable tableView;
```

```
public Report() {
  frame = new JFrame("Sales Report");
  frame.addWindowListener(new WindowAdapter() {
  public void windowClosing(WindowEvent e) {System.exit(0);}});
  final String[] headers = {"Description", "open price",
                      "latest price", "End Date", "Quantity"};
  final Object[][] data = {
    {"Box of Biros", "1.00", "4.99", new Date(), new Integer(2)},
    {"Blue Biro", "0.10", "0.14", new Date(), new Integer(1)},
    {"legal pad", "1.00", "2.49", new Date(), new Integer(1)},
    {"tape", "1.00", "1.49", new Date(), new Integer(1)},
    {"stapler", "4.00", "4.49", new Date(), new Integer(1)},
    {"legal pad", "1.00", "2.29", new Date(), new Integer(5)}};
  TableModel dataModel = new AbstractTableModel() {
    public int getColumnCount() {
      return headers.length;
    }
    public int getRowCount() {
      return data.length;
    }
    public Object getValueAt(int row, int col) {
      return data[row][col];
    }
    public String getColumnName(int column) {
      return headers[column];
    }
    public Class getColumnClass(int col) {
      return getValueAt(0,col).getClass();
    }
    public boolean isCellEditable(int row, int col) {
      return (col==1);
    }
    public void setValueAt(Object aValue, int row, int column) {
      data[row][column] = aValue;
    }};
  tableView = new JTable(dataModel);
  JScrollPane scrollpane = new JScrollPane(tableView);
  scrollpane.setPreferredSize(new Dimension(500, 80));
  frame.getContentPane().setLayout(new BorderLayout());
  frame.getContentPane().add(BorderLayout.CENTER,scrollpane);
  frame.pack();
  JButton printButton= new JButton();
  printButton.setText("print me!");
  frame.getContentPane().add(BorderLayout.SOUTH,printButton);
  // for faster printing turn double buffering off
  RepaintManager.currentManager(frame).setDoubleBufferingEnabled(false);
  printButton.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent evt) {
      PrinterJob pj=PrinterJob.getPrinterJob();
      pj.setPrintable(Report.this);
      pj.printDialog();
      try {
        pj.print();
```

```
      } catch (Exception PrintException) {}
    }});
    frame.setVisible(true);
  }
  public int print(Graphics g, PageFormat pageFormat, int pageIndex)
                   throws PrinterException {
    Graphics2D  g2 = (Graphics2D) g;
    g2.setColor(Color.black);
    int fontHeight=g2.getFontMetrics().getHeight();
    int fontDesent=g2.getFontMetrics().getDescent();
//Leave room for page number
    double pageHeight = pageFormat.getImageableHeight()-fontHeight;
    double pageWidth = pageFormat.getImageableWidth();
    double tableWidth = (double)
                  tableView.getColumnModel().getTotalColumnWidth();
    double scale = 1;
    if(tableWidth >= pageWidth) {
      scale =  pageWidth / tableWidth;
    }
    double headerHeightOnPage=tableView.getTableHeader().getHeight()*scale;
    double tableWidthOnPage=tableWidth*scale;
    double oneRowHeight=(tableView.getRowHeight() +
                                          tableView.getRowMargin())*scale;
    int numRowsOnAPage =
                    (int)((pageHeight-headerHeightOnPage)/oneRowHeight);
    double pageHeightForTable=oneRowHeight*numRowsOnAPage;
    int totalNumPages= (int)Math.ceil((
                    (double)tableView.getRowCount())/numRowsOnAPage);
    if(pageIndex>=totalNumPages) {
      return NO_SUCH_PAGE;
    }
    g2.translate(pageFormat.getImageableX(), pageFormat.getImageableY());
    g2.drawString("Page: "+(pageIndex+1),(int)pageWidth/2-35,
                    (int)(pageHeight+fontHeight-fontDesent));//bottom center
    g2.translate(0f,headerHeightOnPage);
    g2.translate(0f,-pageIndex*pageHeightForTable);
    //If this piece of the table is smaller than the size available,
    //clip to the appropriate bounds.
    if(pageIndex + 1 == totalNumPages) {
      int lastRowPrinted = numRowsOnAPage * pageIndex;
      int numRowsLeft = tableView.getRowCount() - lastRowPrinted;
      g2.setClip(0, (int)(pageHeightForTable * pageIndex),
      (int) Math.ceil(tableWidthOnPage),
      (int) Math.ceil(oneRowHeight * numRowsLeft));
    } else { //else clip to the entire area available.
      g2.setClip(0, (int)(pageHeightForTable*pageIndex),
      (int) Math.ceil(tableWidthOnPage),
      (int) Math.ceil(pageHeightForTable));
    }
    g2.scale(scale,scale);
    tableView.paint(g2);
    g2.scale(1/scale,1/scale);
    g2.translate(0f,pageIndex*pageHeightForTable);
```

```
      g2.translate(0f, -headerHeightOnPage);
      g2.setClip(0, 0,(int) Math.ceil(tableWidthOnPage),
                              (int)Math.ceil(headerHeightOnPage));
      g2.scale(scale,scale);
      tableView.getTableHeader().paint(g2);//paint header at top
      return Printable.PAGE_EXISTS;
    }
    public static void main(String[] args) {
      new Report();
    }
}
```

SalesReport

```
import javax.swing.*;
import javax.swing.table.*;
import java.awt.print.*;
import java.util.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import java.awt.Dimension;

public class SalesReport extends JApplet implements Printable {
  JTable ppTable;
  JTableHeader tableHeader;
  int [] subTableSplit = null;
  boolean pageinfoCalculated=false;
  int totalNumPages=0;
  int prevPageIndex = 0;
  int subPageIndex = 0;
  int subTableSplitSize = 0;
  double tableHeightOnFullPage, headerHeight;
  double pageWidth, pageHeight;
  int fontHeight, fontDesent;
  double tableHeight, rowHeight;

  public void init() {
    final String[] headers = {
          "Description", "open price", "latest price", "End Date", "Quantity",
          "a","b" "c","d","e","f","g","h","i","j","k","l","m","n"};
    // Make the data array larger if you want to see the table split
    // over more than one page
    final Object[][] data = { {
          "Box of Biros", "1.00", "4.99", new Date(), new Integer(2),
          "a","b","c","d","e","f","g" "h" "i" "j" "k","l","m ","n"},
          {"Blue Biro", "0.10", "0.14", new Date(), new Integer(1),
          "a","b", "c", "d", "e", "f", "g", "h", "i","j","k","l","m","n"},
          {"legal pad", "1.00", "2.49", new Date(),new Integer(1),
          "a","b", "c", "d", "e", "f", "g", "h", "i","j","k","l","m","n"},
          {"legal pad", "1.00", "2.49", new Date(), new Integer(1),
          "a","b", "c", "d", "e", "f", "g", "h", "i","j","k","l","m","n"},
```

```
      {“legal pad”, “1.00”, “2.49”, new Date(), new Integer(1),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”},
      {“legal pad”, “1.00”, “2.49”, new Date(), new Integer(1),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”},
      {“tape”, “1.00”, “1.49”, new Date(), new Integer(1),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”},
      {“stapler”, “4.00”, “4.49”, new Date(), new Integer(1),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”},
      {“stapler”, “4.00”, “4.49”, new Date(), new Integer(1),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”},
      {“legal pad”, “1.00”, “2.29”, new Date(), new Integer(5),
      ”a”,”b”, “c”, “d”, “e”, “f”, “g”, “h”, “i”,”j”,”k”,”l”,”m”,”n”}
    };
    TableModel dataModel = new AbstractTableModel() {
       public int getColumnCount() {
         return headers.length;
       }
       public int getRowCount() {
         return data.length;
       }
        public Object getValueAt(int row, int col) {
          return data[row][col]
       }
       public String getColumnName(int column) {
         return headers[column];
       }
       public Class getColumnClass(int col) {
         return getValueAt(0,col).getClass();
       }
       public boolean isCellEditable(int row, int col) {
         return (col==1);
       }
       public void setValueAt(Object aValue, int row, int column) {
         data[row][column] = aValue;
       }};
    ppTable = new JTable(dataModel);
    JScrollPane scrollpane = new JScrollPane(ppTable);
    scrollpane.setPreferredSize(new Dimension(500, 80));
    getContentPane().setLayout(new BorderLayout());
    getContentPane().add(BorderLayout.CENTER,scrollpane);
    JButton printButton= new JButton();
    printButton.setText(“print me!”);
    getContentPane().add(BorderLayout.SOUTH,printButton);

  // For faster printing turn double buffering off
    RepaintManager.currentManager(this).setDoubleBufferingEnabled(false);
    printButton.addActionListener( new ActionListener(){
    public void actionPerformed(ActionEvent evt) {
    PrinterJob pj=PrinterJob.getPrinterJob();
    pj.setPrintable(SalesReport.this);
    pj.printDialog();
    try {
      pj.print();
```

```
            } catch (Exception PrintException) {}
         }});
        setVisible(true);
      }
      public int print(Graphics g, PageFormat pageFormat, int pageIndex)
                                  throws PrinterException {
        Graphics2D g2=(Graphics2D)g;
        if(!pageinfoCalculated) {
          getPageInfo(g, pageFormat);
        }
        g2.setColor(Color.black);
        if(pageIndex>=totalNumPages) {
          return NO_SUCH_PAGE;
        }
        if(prevPageIndex != pageIndex) {
          subPageIndex++;
          if ( subPageIndex == subTableSplitSize -1) {
           subPageIndex=0;
          }
        }
        g2.translate(pageFormat.getImageableX(), pageFormat.getImageableY());
        int rowIndex = pageIndex/ (subTableSplitSize - 1);
        printTablePart(g2, pageFormat, rowIndex, subPageIndex);
        prevPageIndex= pageIndex;
        return Printable.PAGE_EXISTS;
      }

      public void getPageInfo(Graphics g, PageFormat pageFormat) {
        subTableSplit = null;
        subTableSplitSize = 0;
        subPageIndex = 0;
        prevPageIndex = 0;
//API Ref :FontMetrics getFontMetrics()
        fontHeight=g.getFontMetrics().getHeight();
        fontDesent=g.getFontMetrics().getDescent();
        tableHeader = ppTable.getTableHeader();
        double headerWidth = tableHeader.getWidth();
        headerHeight = tableHeader.getHeight() + ppTable.getRowMargin();
        pageHeight = pageFormat.getImageableHeight();
        pageWidth =  pageFormat.getImageableWidth();
        double tableWidth = ppTable.getColumnModel().getTotalColumnWidth();
        tableHeight = ppTable.getHeight();
//API Ref :int getRowHeight()
//API Ref :int getRowMargin()
        rowHeight = ppTable.getRowHeight() + ppTable.getRowMargin();
        tableHeightOnFullPage = (int)(pageHeight - headerHeight - fontHeight*2);
        tableHeightOnFullPage = tableHeightOnFullPage/rowHeight * rowHeight;
        TableColumnModel tableColumnModel = tableHeader.getColumnModel();
//API Ref :int getColumnCount()
        int columns = tableColumnModel.getColumnCount();
//API Ref :int getColumnMargin()
        int columnMargin = tableColumnModel.getColumnMargin();
        int [] temp = new int[columns];
```

```
    int columnIndex = 0;
    temp[0] = 0;
    int columnWidth;
    int length = 0;
    subTableSplitSize = 0;
    while( columnIndex < columns ) {
      columnWidth = tableColumnModel.getColumn(columnIndex).getWidth();
      if(length + columnWidth + columnMargin > pageWidth ) {
        temp[subTableSplitSize+1] = temp[subTableSplitSize] + length;
        length = columnWidth;
        subTableSplitSize++;
      } else {
        length += columnWidth + columnMargin;
      }
        columnIndex++;
    } //while
    if(length > 0 )  {  // if are more columns left, part page
      temp[subTableSplitSize+1] = temp[subTableSplitSize] + length;
      subTableSplitSize++;
    }
    subTableSplitSize++;
    subTableSplit = new int[subTableSplitSize];
    for(int i=0; i < subTableSplitSize; i++ ) {
      subTableSplit[i]= temp[i];
    }
    totalNumPages = (int)(tableHeight/tableHeightOnFullPage);
    // at least 1 more row left
    if(tableHeight%tableHeightOnFullPage >= rowHeight ) {
      totalNumPages++;
    }
    totalNumPages *= (subTableSplitSize-1);
    pageinfoCalculated = true;
  }
  public void printTablePart(Graphics2D g2, PageFormat pageFormat,
                             int rowIndex,int columnIndex) {
    String pageNumber = "Page: "+(rowIndex+1);
    if(subTableSplitSize > 1 ) {
      pageNumber += "-" + (columnIndex+1);
    }
    int pageLeft = subTableSplit[columnIndex];
    int pageRight = subTableSplit[columnIndex + 1];
    int pageWidth =  pageRight-pageLeft;
    // page number message
     g2.drawString(pageNumber, pageWidth/2-35, (int)(pageHeight - fontHeight));
    double clipHeight = Math.min(tableHeightOnFullPage,
                                 tableHeight - rowIndex*tableHeightOnFullPage);
    g2.translate(-subTableSplit[columnIndex], 0);
    g2.setClip(pageLeft ,0, pageWidth, (int)headerHeight);
    tableHeader.paint(g2);   // draw the header on every page
    g2.translate(0, headerHeight);
    g2.translate(0,  -tableHeightOnFullPage*rowIndex);
    // cut table image and draw on the page
```

```
    g2.setClip(pageLeft, (int)tableHeightOnFullPage*rowIndex, pageWidth,
                         (int)clipHeight);
    ppTable.paint(g2);
    double pageTop =  tableHeightOnFullPage*rowIndex - headerHeight;
    double pageBottom = pageTop +  clipHeight + headerHeight;
    g2.drawRect(pageLeft, (int)pageTop, pageWidth,
                          (int)(clipHeight+ headerHeight));
  }
}
```

7: Debugging Applets, Applications, and Servlets

An unwritten law of programming states you will spend 10 percent of your time on the first 90 percent of a project, and the other 90 percent of your time on the remaining 10 percent. If this sounds like any of your projects, you are probably spending 90 percent of your time on debugging and integration. While there are plenty of books and people to help you start a project, there are far fewer resources available to help you finish it.

The good news is this chapter focuses completely on debugging and fixing to get your project out on time. This chapter and 8: Performance Techniques (page 339) depart from the auction application and use simple examples to walk you through the steps to debugging, fixing, and tuning your programs.

By the time you finish this chapter, you should be an expert at troubleshooting programs written in the Java™ programming language--applets, applications, and servlets--of all shapes and sizes.

Covered in this Chapter

- In a Rush? (page 298)
- Collecting Evidence (page 298)
- Running Tests and Analyzing (page 304)
- Servlet Debugging (page 314)
- Abstract Window Toolkit Debugging (page 318)
- Analyzing Stack Traces (page 320)
- Stack Trace Examples (page 331)
- Version Issues (page 334)

In a Rush?

If you have a pressing problem you need an answer to right now, this table might help. It tells you where to find answers for common problems so you can go directly to the information.

Problem	Section
Program hangs or crashes	Analyzing Stack Traces (page 320)
Problem in a running program	Getting Behind the Seat with jdb (page 304)
Java Web Server problems	Servlet Debugging (page 314) and Analyzing Stack Traces (page 320)

Collecting Evidence

The first step in trying to solve any problem is to gather as much evidence and information as possible. If you can picture a crime scene, you know that everything is checked, cataloged and analyzed before any conclusions are reached. When debugging a program, you do not have weapons, hair samples, or fingerprints, but there is plenty of evidence you can gather that might contain or ultimately lead to the solution. This section explains how to gather that evidence.

Installation and Environment

The Java platform is a fast-moving and changing technology. You might have more than one release installed on your system, and those releases might have been installed as part of another product installation. In an environment with mixed releases, a program can experience problems due to changes to the platform in a new version or release.

For example, if classes, libraries, or Windows registry entries from previous installations remain on your system after an upgrade, there is a chance the new software mix is causing your problems and needs to be investigated and ruled out. Opportunities for problems related to mixed software releases have increased with the use of different release tools to deliver the Java platform software.

The section on Version Issues (page 334) at the end of this chapter provides a complete list of major Java platform release and version information to help you rule out software release issues. This next section highlights the most common problems you are likely to encounter.

Class Path

In the Java 2 platform, the CLASSPATH environment variable is needed to specify the application's own classes only, and not the Java platform classes as was required in earlier releases. So it is possible your CLASSPATH environment variable is pointing at Java platform classes from earlier releases and causing problems. To examine the CLASSPATH, type the following at the command line for your operating system.

Windows 95/98/NT

```
echo %CLASSPATH%
```

Unix Systems

```
echo $CLASSPATH
```

Java classes are loaded on a first come, first served basis from the CLASSPATH list. If the CLASSPATH variable contains a reference to a lib/classes.zip file, which in turn points to a different Java platform installation, this can cause incompatible classes to be loaded.

Note: In the Java 2 platform, the system classes are chosen before any class on the CLASSPATH list to minimize the possibility of any old broken Java classes being loaded instead of a Java 2 class of the same name.

The CLASSPATH variable can get its settings from the command line or from configuration settings such as those specified in the User Environment on Windows NT, an autoexec.bat file, or a shell startup file like .cshrc on Unix.

You can control the classes the Java virtual machine uses by compiling your program with a special command-line option that lets you supply the CLASSPATH you want. The Java 2 platform option and parameter is -Xbootclasspath classpath, and earlier releases use -classpath classpath and -sysclasspath classpath. Regardless of which release you are running, the classpath parameter specifies the system and user classpath, and zip or Java Archive (JAR) files to be used in the compilation.

As an example, to compile and run the Myapp.java program with a system CLASSPATH supplied on the command line, use the following instructions for your operating system.

Windows 95/98/NT

In this example, the Java platform is installed in the C:\java directory. Type everything on one line. The -J option lets you pass extra conditions to the compiler.

```
javac -J -Xbootclasspath:c\javaıib\tools.jar;c:\java\jreıibt.jar;
c:\java\jreıib\i18n.jar;.Myapp.java
```

You do not need the -J runtime flag to run the compiled Myapp program, just type the following on one line:

```
java -Xbootclasspath:c:\java\jreıibt.jar;c:\java\jreıib\i18n.jar;.
Myapp
```

Unix Systems:

In this example, the Java platform is installed in the /usr/local/java directory. Type everything on one line:

```
javac -J-Xbootclasspath:/usr/local/java/lib/tools.jar:/usr/local/java/
jre/lib/rt.jar:/usr/local/java/jre/lib/i18n.jar:. Myapp.java
```

You do not need the -J runtime flag to run the compiled Myapp program, just type the following on one line:

```
java -Xbootclasspath:/usr/local/java/jre/lib/rt.jar:/usr/local/java/
jre/lib/i18n.jar:. Myapp
```

Class Loading

Another way to analyze `CLASSPATH` problem is to locate where your application is loading its classes. The `verbose` option to the `java` interpreter command shows which `.zip` or `.jar` file a class comes from when it is loaded. This way, you will be able to tell if it came from the Java platform zip file or from some other application's JAR file.

For example, an application might be using the `Password` class you wrote for it or it might be loading a `Password` class from an installed integrated development environment (IDE) tool. You should see each jar and zip file named as in the example below:

```
$ java -verbose SalesReport
[Opened /usr/local/java/jdk1.2/solaris/jre/lib/rt.jar in 498 ms]
[Opened /usr/local/java/jdk1.2/solaris/jre/lib/i18n.jar in 60 ms]
[Loaded java.lang.NoClassDefFoundError from /usr/local/java/jdk1.2/solaris/jre/
lib/rt.jar]
[Loaded java.lang.Class from /usr/local/java/jdk1.2/solaris/jre/lib/rt.jar]
[Loaded java.lang.Object from /usr/local/java/jdk1.2/solaris/jre/lib/rt.jar]
```

Including Debug Code

A common way to add diagnostic code to an application is to use `System.out.println` statements at strategic locations in the application. This technique is fine during development, providing you remember to remove them all when you release your product.

However, there are other approaches that are just as simple, do not affect the performance of your application, and do not display messages that you do not want your customers to see. The following are two techniques that overcome the problems with `System.out.println` statements.

Turning Debug Information on at Runtime

The first technique is to turn debugging information on at runtime. One advantage to this is you do not need to recompile any code if problems appear at the testing stage or on a customer site. Another advantage is that sometimes software problems can be attributed to race conditions where the same segment of code behaves unpredictably due to timing between other program interactions.

If you control your debug code from the command line instead of adding `println` debug statements, you can rule out sequence problems caused by race conditions coming from the `println` code. This technique also saves you adding and removing `println` debug statements and having to recompile your code.

To use this technique, you have to set the debug flag to `true` and include application code to test that system property value. So for example, you can set the debug flag to `true` and specify the `TestRuntime` program as follows:

```
java -Ddebug=true TestRuntime
```

The source code for `theTestRuntime` class needs to examine this property and set the debug boolean flag as follows:

```
public class TestRuntime {
  boolean debugmode; //global flag that we test
  public TestRuntime () {
    String dprop=System.getProperty("debug");
    if((dprop !=null) && (dprop.equals("yes"))){
      debugmode=true;
    }
    if(debugmode) {
      System.err.println("debug mode!");
    }
  }
}
```

Creating Debug and Production Releases at Runtime

As mentioned earlier, one problem with adding `System.out.println` debug statements to your code is finding and removing them before you release the product. Apart from adding unnecessary code, `println` debug statements can contain information you do not want your customers to see.

One way to remove `System.out.println` debug statements from your code is to use the following compiler optimization to remove pre-determined branches from your code at compile time and archive something similar to a debug pre-processor.

This example uses a static `dmode` boolean flag that when set to `false` results in the debug code and the debug test statement being removed. When the `dmode` value is set to `true`, the code is included in the compiled class file and is available to the application for debugging purposes.

```
class Debug {
  //set dmode to false to compile out debug code
  public static final boolean dmode=true;
}

public class TestCompiletime {
  if (Debug.dmode) {                        // These
    System.err.println("Debug message");   // are
    }                                       // removed
}
```

Using Diagnostic Methods

You can use diagnostic methods to request debug information from the Java virtual machine. The following two methods from the `Runtime` class trace the method calls and Java virtual machine byte codes your application uses. As both these methods produce a lot of output, it is best to trace very small amounts of code, even as little as one line at a time.

To enable trace calls so you will see the output, you have to start the Java virtual machine with the `java_g` or `java -Xdebug` interpreter commands. To list every method as it is invoked at runtime, add the following line before the code you wish to start tracing and add a matching `traceMethodCalls` line with the argument set to false to turn the tracing off. The tracing information is displayed on the standard output.

```
// set boolean argument to false to disable
  Runtime.getRuntime().traceMethodCalls(true);
  callMyCode();
  Runtime.getRuntime().traceMethodCalls(false);
```

You can also add the following line to your application to dump your own stack trace. You can find out how to read a stack trace in Analyzing Stack Traces (page 320), but for now you

can think of a stack trace as a snapshot of the current thread running in the Java virtual machine.

```
Thread.currentThread().dumpStack();
```

Adding Debug Information

Local variable information is not included in the core Java platform system classes. So, if you use a debug tool to list local variables for system classes where you place `stop` commands, you will get the following output, even when you compile with the `-g` flag as suggested by the output. This output is from a `jdb` session:

```
main[1] locals
No local variables: try compiling with -g
```

To get access to the local variable information, you have to obtain the source (`src.zip` or `src.jar`) and recompile it with a `debug` flag. You can get the source for most java.* classes with the binary downloads from java.sun.com(`http://java.sun.com`).

Once you download the `src.zip` or `src.jar` file, extract only the files you need. For example, to extract the `String` class, type the following at the command line:

```
unzip /tmp/src.zip src/java/lang/String.java
```

or

```
jar -xf /tmp/src.jar src/java/lang/String.java
```

Recompile the extracted class or classes with the `-g` option. You could also add your own additional diagnostics to the source file at this point.

```
javac -g src/java/lang/String.java
```

Note: The Java 2 `javac` compiler gives you more options than just the original `-g` option for debug code, and you can reduce the size of your classes by using `-g:none`, which gives you on average about a 10 percent reduction in size.

To run the application with the newly compiled debug class or classes, you need to use the boot classpath option so these new classes are picked up first. Type the following on one line with a space before `myapp`.

Win95/NT Java 2 Platform

This example assumes the Java platform is installed in `c:\java`, and the source files are in `c:\java\src`:

```
jdb -Xbootclasspath:c:\java\src;c:\java\jre\lib\rt.jar;c:
\java\jre\i18n.jar;.  myapp
```

Unix Systems:

This example assumes the Java platform is installed in `c:\java`, and the source files are in `c:\java\src`.

```
jdb -Xbootclasspath:/usr/java/src;/usr/java/jre/lib/rt.jar;/usr/java/
jre/i18n.jar;.  myapp
```

Running Tests and Analyzing

If you are still having problems even after you have ruled out installation and environment problems and included debugging code, it is time to use tools to test and analyze your program.

Getting Behind the Seat with jdb

Although there are some very good integrated development environment (IDE) tools on the market, the Java debugger tool, jdb and its successors, have an important role to play in testing and debugging programs. Some advantages of jdb over IDE tools are it is free, it is platform independent (some IDE tools are not), and it runs as a separate process to the program being debugged. The benefit to jdb running as a separate process is you can attach a debug session to a running program.

The downsides to using jdb are it has only a command-line interface, and it relies on the same code you are trying to debug. This means if there is a bug in the Java virtual machine, jdb could break attempting to diagnose that same bug!

The new JBUG architecture was created to solve these problems in jdb. JBUG, amongst other things, provides a debugger helper API in the Java virtual machine called the Java virtual machine Debug Interface (JVMDI). This helper communicates with the debugging front end using the Java Debug Wire Protocol (JDWP). The debugging front end uses the remote Java Debug Interface (JDI) to send and receive commands over the Java Debug Wire Protocol. JBug is available for Java 2 platforms, and has a jdb style front end that you will learn more about below.

Simple jdb Test Drive

Back to the classic jdb tool. Here are some simple steps to analyze a program using jdb. This first example debugs a program from application startup. The Remote Debugging (page 310) example explains how to connect to a running program.

Start the Session

To begin the debug session, compile the SimpleJdbTest program below with full debugging information using javac and the -g debug flag. In this example, the SimpleJdbTest.java program is an application, but it could just as well be an applet. The procedures for debugging applications with jdb are the same for debugging applets once the debug session has started.

SimpleJdbTest Program:

```
import java.awt.*;
import java.awt.event.*;

public class SimpleJdbTest extends Frame{
  Panel p;
  Button b[]=new Button[2];
  int counter=0;

  SimpleJdbTest() {
    setSize(100,200);
    setup();
  }
  void setup (){
    p=new Panel();
    b[0]= new Button("press");
    p.add(b[0]);
    add(p);
    b[0].addActionListener( new ActionListener() {
      public void actionPerformed(ActionEvent e) {
      counter++;
    }
  });}
  public static void main(String args[]) {
    SimpleJdbTest sjb=new SimpleJdbTest();
    sjb.setVisible(true);
  }
}
```

Compile SimpleJdbTest:

```
javac -g SimpleJdbTest.java
```

Start the jdb Tool:

Next, start the jdb tool with the program class name as a parameter:

```
jdb SimpleJdbTest
Initializing jdb...
0xad:class(SimpleJdbTest)
```

To debug an applet in appletviewer use the -debug parameter as in this example:

```
$ appletviewer -debug MyApplet.html
Initializing jdb...
0xee2f9808:class(sun.applet.AppletViewer)
>
```

Setting a Breakpoint and Listing Methods

At this point, the SimpleJdbTest class has only been loaded; the class constructor has not been called. To make jdb stop when the program is first instantiated, put a stop, or breakpoint, at the constructor using the stop in command. When the breakpoints has been set, instruct jdb to run your program using the run command as follows:

```
> stop in SimpleJdbTest.<init>
Breakpoint set in SimpleJdbTest.<init>
run
run SimpleJdbTest
running ...
main[1]
Breakpoint hit: SimpleJdbTest.<init> (SimpleJdbTest:10)
```

The jdb tool stops at the first line in the constructor. To list the methods that were called to get to this breakpoint, enter the where command:

```
main[1] where
[1] SimpleJdbTest.<init> (SimpleJdbTest:10)
[2] SimpleJdbTest.main (SimpleJdbTest:29)
```

The numbered method in the list is the last stack frame that the Java virtual machine has reached. In this case the last stack frame is the SimpleJdbTest constructor that was called from SimpleJdbTest main.

Whenever a new method is called, it is placed on this stack list. Java Hotspot Performance Engine achieves some of its speed gains by eliminating a new stack frame when a new method is called.

To get a general appreciation of where the code has stopped, enter the list command.

```
main[1] list
6   Panel p;
7   Button b;
8   int counter=0;
9
10  SimpleJdbTest() {
```

```
11       setSize(100,200);
12       setup();
13  }
14  void setup (){
```

Locating the Source

If the source to the class file stopped in is not available on the current path, you can tell jdb to find the source with the use command by giving it the source directory as a parameter. In the following example the source is in a subdirectory or folder called book.

```
main[1] list
Unable to find SimpleJdbTest.java
main[1] use book
main[1] list
6   Panel p;
7   Button b[];
8   int counter=0;
9
10 => SimpleJdbTest() {
```

Looking at a Method

To see what happens in the setup method for SimpleJdbText, use the step command to step through the 4 lines to get to it.

```
main[1] step
main[1]
Breakpoint hit: java.awt.Frame.<init> (Frame:222)
```

This is now the Frame class constructor! If you keep stepping you follow the Frame Constructor and not the SimpleJdbText class. Because SimpleJdbTest extends the Frame class, the parent constructor, which in this case is Frame, is called on your behalf.

The step up Command

You could continue stepping and eventually you will return to the SimpleJdbTest constructor, but to return immediately, you can use the step up command to go back to the SimpleJdbTest constructor.

```
main[1] step up
main[1]
Breakpoint hit: SimpleJdbTest.<init> (SimpleJdbTest:8)
```

The next Command

You can also use the next command to get to the setup method. In this next example, the jdb tool has approximated that the source line is outside the constructor when it processed the last step up command. To return to the constructor, use another step command, and to get to

the setup method, use a next command. To debug the setup method, you can step through the setup method.

```
main[1] step
Breakpoint hit: SimpleJdbTest.<init>
                  (SimpleJdbTest:11)
main[1] list
7   Button b[]=new Button[2];
8   int counter=0;
9
10  SimpleJdbTest() {
11        setSize(100,200);<
12        setup();
13  }
14  void setup (){
15        p=new Panel();
16  }
main[1] next
Breakpoint hit: SimpleJdbTest.<init>
                  (SimpleJdbTest:12)
main[1] step
Breakpoint hit: SimpleJdbTest.setup (SimpleJdbTest:15)
```

The list and print Commands

Another way to get to the setup method is to use the stop in SimpleJdbTest.setup command. Before using stop in, list the source to check where you are:

```
main[1] list
11        setSize(100,200);
12        setup();
13  }
14  void setup (){
15=>      p=new Panel();
16        b[0]= new Button("press");
17        p.add(b[0]);
18        add(p);
19
```

The first thing the setup method does is create a Panel p. If you try to display the value of p with the print p command, you will find that the value is null.

```
main[1] print p
p = null
```

This occurred because the line has not been executed and so field p has not been assigned a value. You need to step over that assignment operation with the next command and then use the print p command again.

```
main[1] next

Breakpoint hit: SimpleJdbTest.setup (SimpleJdbTest:16)
```

```
main[1] print p
p = java.awt.Panel[panel0,0,0,0x0,invalid, layout=java.awt.FlowLayout]
```

Using the Stop In Command to Set Breakpoints on Overloaded Methods

Although stepping through small classes is fast, as a general rule on larger applications, it is often a lot faster to set breakpoints. This is partly because jdb has a very simple command set and no shortcuts, so each command has to be pasted or typed in full.

To set a breakpoint in the Button class, use stop in java.awt.Button.<init>

```
main[1] stop in java.awt.Button.<init>
java.awt.Button.<init> is overloaded,use one of the following:
void <init>
void <init>(java.lang.String)
```

The message explains why jdb cannot stop in this method without more information, but the message is slightly misleading as you do not need to specify the return type for overloaded methods, you just need to be explicit about exactly which one of the overloaded methods you want to stop in. To stop in the Button constructor that creates this Button, use stop in java.awt.Button.<init>(java.lang.String).

Later releases of `jdb` let you choose the desired method as a numbered option.

The cont Command

To continue the jdb session, use the cont command. The next time the program creates a Button with a String as the constructor, jdb stops so you can examine the output.

```
main[1] cont
main[1]
Breakpoint hit: java.awt.Button.<init> (Button:130)
```

If the Button class had not been recompiled with debug information as described earlier, you would not see the internal fields from the print command.

Clearing Breakpoints

To clear this breakpoint and not stop every time a Button is created use the clear command. This example uses the clear command with no arguments to display the list of current breakpoints, and the clear command with the java.awt.Button:130 argument to clear the java.awt.Button:130 breakpoint.

```
main[1] clear
Current breakpoints set:
SimpleJdbTest:10
java.awt.Button:130
main[1] clear java.awt.Button:130
Breakpoint cleared at java.awt.Button: 130
```

Displaying Object Details

To display details about an object, use the print command to call the object's toString method, or use the dump command to display the object's fields and values.

This example puts a breakpoint at line 17 and uses the print and dump commands to print and dump the first Button object in the array of Button objects. The dump command output has been abbreviated.

```
main[1] stop at SimpleJdbTest:17
Breakpoint set at SimpleJdbTest:17
main[1] cont
main[1]
Breakpoint hit: SimpleJdbTest.setup (SimpleJdbTest:17)

main[1] print b[0]
b[0] = java.awt.Button[button1,0,0,0x0,invalid,label=press]
main[1] dump b[0]
b[0] = (java.awt.Button)0x163 {
private int componentSerializedDataVersion = 2
boolean isPacked = false
private java.beans.PropertyChangeSupport changeSupport = null
long eventMask = 4096
transient java.awt.event.InputMethodListener
        inputMethodListener = null
....
java.lang.String actionCommand = null
java.lang.String label = press
}
```

Ending the Session

That finishes the simple jdb examples. To terminate the jdb session, use the quit command:

```
0xee2f9820:class(SimpleJdbTest)
> quit
```

Remote Debugging

The jdb tool is an external process debugger, which means it debugs the program by sending messages to and from a helper inside the Java virtual machine. This makes it is easy to debug a running program, and helps you debug a program that interacts with the end user. A remote debug session from the command-line does not interfere with the normal operation of the application.

Starting the Session

Before the Java 2 release, the only thing required to enable remote debugging was to start the program with the -debug flag as the first argument, and if the application uses native librar-

ies, make the library name end in _g. For example, you would need to copy nativelib.dll to nativelib_g.dll to debug with that library.

In Java 2, things are a little more complicated. You need to tell the Java virtual machine where the tools.jar file is with the CLASSPATH variable. The tools.jar file contains non-core class files to support tools and utilities in the SDK. It is normally found in the lib directory of the Java platform installation.

You also need to disable the Just In Time (JIT) compiler if one exists. The JIT compiler is disabled by setting the java.compiler property to NONE or to an empty string. Finally, as the -classpath option overrides any previously set user classpath, you also need to add the CLASSPATH needed by your application.

Putting all of this together, here is the command line needed to start a program in remote debug mode. Put this all on one line and include all the classes you need on the command line.

Windows

```
$ java -debug -classpath C:\java\lib\tools.jar;.
-Djava.compiler=NONE SimpleJdbTest
Agent password=4gk5hm
```

Unix:

```
$ java -debug -classpath /usr/java/lib/tools.jar:.
-Djava.compiler=NONE SimpleJdbTest
Agent password=5ufhic
```

The output is the agent password (in this case, 4gk5hm) if the program was successfully started. The agent password is supplied when starting jdb so jdb can find the corresponding application started in debug mode on that machine.

To start jdb in remote debug mode, supply a host name, which can be either the machine where the remote program was started or localhost if you are debugging on the same machine as the remote program, and the agent password.

```
jdb -host localhost -password 4gk5hm
```

Listing Threads

Once inside the jdb session, you can list the currently active threads with the threads command, and use the thread <threadnumber> command, for example, thread 7 to select the thread to analyze. Once the thread is selected, use the where command to see which methods have been called for this thread.

```
$ jdb -host arsenal -password 5ufhic
Initializing jdb...
> threads
Group system:
1. (java.lang.Thread)0x9        Signal dispatcher
                                cond. waiting
2. (java.lang.ref.Reference     0xb Reference Handler
      $ReferenceHandler)        cond. waiting
3. (java.lang.ref.              Finalizer
      Finalizer                 cond. waiting
      $FinalizerThread)0xd

4. (java.lang.Thread)0xe        Debugger agent
                                running
5. (sun.tools.agent.            Breakpoint handler
      Handler)0x10              cond. waiting
6. (sun.tools.agent.            Step handler
      StepHandler)0x12          cond. waiting
Group main:
7. (java.awt.                   AWT-EventQueue-0
       EventDispatchThread)     cond. waiting
       0x19
8. (sun.awt.                     PostEventQueue-0
      PostEventQueue)0x1b       cond. waiting
9. (java.lang.Thread)0x1c       AWT-Motif
                                running
10. (java.lang.Thread)0x1d      TimerQueue
                                cond. waiting
11. (sun.awt.                   Screen Updater
       ScreenUpdater)0x1f       cond. waiting
12. (java.lang.Thread)0x20      Thread-0
                                cond. waiting
> thread 7
AWT-EventQueue-0[1] where
  [1] java.lang.Object.wait (native method)
  [2] java.lang.Object.wait (Object:424)
  [3] java.awt.EventQueue.getNextEvent (EventQueue:179)
  [4] java.awt.EventDispatchThread.run (EventDispatchThread:67)
```

Listing Source

To list the source, the thread needs to be suspended using the suspend command. To let this thread continue use the resume command. The example uses resume 7.

```
AWT-EventQueue-0[1] suspend 7
AWT-EventQueue-0[1] list
Current method is native
AWT-EventQueue-0[1] where
   [1] java.lang.Object.wait (native method)
   [2] java.lang.Object.wait (Object:424)
   [3] java.awt.EventQueue.getNextEvent   (EventQueue:179)
   [4] java.awt.EventDispatchThread.run (EventDispatchThread:67)
AWT-EventQueue-0[1] resume 7
```

Ending the Session

When you finish debugging this program remotely, clear any remaining breakpoints before quitting the debug session. To get a list of remaining breakpoints use the clear command, and to remove them enter clear class:linenumber as follows:

```
main[1] clear
Current breakpoints set:
SimpleJdbTest:10

main[1] clear SimpleJdbTest:10
main[1] quit
```

Using Auto-Pilot

One little known trick with jdb is the jdb startup file. jdb automatically looks for a file called jdb.ini in the user.home directory. If you have multiple projects, it is a good idea to set a different user.home property for each project when you start jdb. To start jdb with a jdb.ini file in the current directory, type the following:

```
jdb -J-Duser.home=.
```

The jdb.ini file lets you set up jdb configuration commands, such as use, without having to enter the details each time a jdb runs. The following example jdb.ini file starts a jdb session for the FacTest class. It includes the Java platform sources on the source path list and passes the parameter 6 to the program. It then runs and stops at line 13, displays the free memory, and waits for further input.

```
load FacTest
stop at FacTest:13
use /home/calvin/java:/home/calvin/jdk/src/
run FacTest 6
memory
```

Here is the output from the jdb.ini file execution:

```
$ jdb -J-Duser.home=/home/calvin/java
Initializing jdb...
0xad:class(FacTest)
Breakpoint set at FacTest:13
running ...
Free: 662384, total: 1048568
main[1]
Breakpoint hit: FacTest.compute (FacTest:13)
main[1]
```

You might wonder if jdb.ini files can be used to control an entire jdb session. Unfortunately, commands in a jdb.ini startup file are executed synchronously, and jdb does not wait until a breakpoint is reached before executing the next command. This makes printing variables awkward. You can add artificial delays with repeated help commands, but there is still no guarantee the thread will be suspended when you need it to be.

Creating a Session Log

You can use a little-known jdb feature to obtain a record of your debug session. The output is similar to what you see when you run jdb -dbgtrace. To enable jdb logging, create a file called .agentLog in the directory where you are running jdb or java -debug. In the .agentLog file, put the file name that you want the session information to be written to on the first line. For example, an .agentLog file would have these contents:

```
jdblog
```

When you next run jdb or java -debug, you will see jdb session information as shown below. You can use this information to retrieve the breakpoint hits and the commands entered if you need to reproduce this debug session.

```
---- debug agent message log ----
[debug agent: adding Debugger agent to system thread list]
[debug agent: adding Breakpoint handler to system thread list]
[debug agent: adding Step handler to system thread list]
[debug agent: adding Finalizer to system thread list]
[debug agent: adding Reference Handler to system thread list]
[debug agent: adding Signal dispatcher to system thread list]
[debug agent: Awaiting new step request]
[debug agent: cmd socket: Socket[addr=localhost/127.0.0.1, port=38986,localport=3
8985]]
[debug agent: connection accepted]
[debug agent: dumpClasses()]
[debug agent: no such class: HelloWorldApp.main]
[debug agent: Adding breakpoint bkpt:main(0)]
[debug agent: no last suspended to resume]
[debug agent: Getting threads for HelloWorldApp.main]
```

Servlet Debugging

You can debug servlets with the same jdb commands you use to debug an applet or an application. The JavaServer™ Web Development Kit (JSWDK) provides a standalone program called servletrunner that lets you run a servlet without a web browser. On most systems, this program simply runs the java sun.servlet.http.HttpServer command. You can, therefore, start a jdb session with the HttpServer class.

A key point to remember when debugging servlets is that Java WebServer™ and servletrunner achieve servlet loading and unloading by not including the servlets directory on the CLASSPATH. This means the servlets are loaded using a custom classloader and not the default system classloader.

Running servletrunner in Debug Mode

In this example, the servlets examples directory is included on the CLASSPATH. You can configure the CLASSPATH for debug mode as follows:

Unix

```
ksh> export CLASSPATH=./lib/jsdk.jar:./examples:$CLASSPATH
```

Windows

```
$ set CLASSPATH=lib\jsdk.jar;examples;%classpath%
```

To start the servletrunner program to debug `SnoopServlet`, either run the supplied startup script called servletrunner or just supply the servletrunner classes as a parameter to jdb. This example uses the parameter to servletrunner.

```
$ jdb sun.servlet.http.HttpServer
Initializing jdb...
0xee2fa2f8:class(sun.servlet.http.HttpServer)
> stop in SnoopServlet.doGet
Breakpoint set in SnoopServlet.doGet
> run
run sun.servlet.http.HttpServer
running ...
main[1] servletrunner starting with settings:
port = 8080
backlog = 50
max handlers = 100
timeout = 5000
servlet dir = ./examples
document dir = ./examples
servlet propfile = ./examples/servlet.properties
```

To run SnoopServlet in debug mode, enter the following URL in a browser where your machine is the machine where you started servlet runner and 8080 is the port number displayed in the settings output.

```
http://yourmachine:8080/servlet/SnoopServlet
```

In this example jdb stops at the first line of the servlet's doGet method. The browser will wait for a response from your servlet until a timeout is reached.

```
main[1] SnoopServlet: init

Breakpoint hit: SnoopServlet.doGet (SnoopServlet:45)
Thread-105[1]
```

We can use the list command to work out where jdb has stopped in the source.

```
Thread-105[1] list
41        throws ServletException, IOException
42        {
43      PrintWriter     out;
44
45 =>   res.setContentType("text/html");
46      out = res.getWriter ();
47
48      out.println("<html>");
49      out.println("<head>
                    <title>Snoop Servlet
                    </title></head>");
Thread-105[1]
```

The servlet can continue using the cont command.

```
Thread-105[1] cont
```

Running Java WebServer in Debug Mode

The JWSDK release does not contain classes available in the Java WebServer and it also has its own special servlet configuration. If you cannot run your servlet from servletrunner, then the other option is to run the Java WebServer in debug mode.

To do this add the -debug flag for the first parameter after the java program. For example in the script bin/js change the JAVA line to look like the following. In releases prior to the Java 2 platform release, you will also need to change the program pointed to by the variable $JAVA to java_g instead of java.

Before:

```
exec $JAVA $THREADS $JITCOMPILER $COMPILER $MS $MX \
```

After:

```
exec $JAVA -debug $THREADS $JITCOMPILER $COMPILER $MS $MX \
```

Here is how to remotely connect to Java WebServer. The agent password is generated on the standard output from the Java WebServer so it can be redirected into a file somewhere. You can find out where by checking the Java WebServer startup scripts.

```
jdb -host localhost -password <the agent password>
```

The servlets are loaded by a separate classloader if they are contained in the servlets directory, which is not on the CLASSPATH used when starting Java WebServer. Unfortunately, when debugging remotely with jdb, you cannot control the custom classloader and request it to load the servlet, so you have to either include the servlets directory on the CLASSPATH for debugging or load the servlet by requesting it through a web browser and placing a breakpoint once the servlet has run.

In this next example, the jdc.WebServer.PasswordServlet is included on the CLASSPATH when Java WebServer starts. The example sets a breakpoint to stop in the service method of this servlet, which is the main processing method of this servlet. The Java WebServer standard output produces this message, which lets you proceed with the remote jdb session:

```
Agent password=3yg23k

$ jdb -host localhost -password 3yg23k
Initializing jdb...
> stop in jdc.WebServer.PasswordServlet:service
Breakpoint set in jdc.WebServer.PasswordServlet.service
> stop
Current breakpoints set:
        jdc.WebServer.PasswordServlet:111
```

The second stop lists the current breakpoints in this session and shows the line number where the breakpoint is set. You can now call the servlet through your HTML page. In this example, the servlet is run as a POST operation

```
<FORM METHOD="post" action="/servlet/PasswordServlet">
<INPUT TYPE=TEXT SIZE=15 Name="user" Value="">
<INPUT TYPE=SUBMIT Name="Submit" Value="Submit">
</FORM>
```

You get control of the Java Web Server thread when the breakpoint is reached, and you can continue debugging using the same techniques as used in Remote Debugging (page 310).

```
Breakpoint hit: jdc.WebServer.PasswordServlet.service
(PasswordServlet:111) webpageservice Handler[1] where
 [1] jdc.WebServer.PasswordServlet.service
(PasswordServlet:111)
```

```
[2] javax.servlet.http.HttpServlet.service
(HttpServlet:588)
 [3] com.sun.server.ServletState.callService
(ServletState:204)
 [4] com.sun.server.ServletManager.callServletService
(ServletManager:940)
 [5] com.sun.server.http.InvokerServlet.service
(InvokerServlet:101)
```

A common problem when using the Java WebServer and other servlet environments is that Exceptions are thrown but are caught and handled outside the scope of your servlet. The catch command allows you to trap all these exceptions.

```
webpageservice Handler[1] catch java.io.IOException
webpageservice Handler[1]
Exception: java.io.FileNotFoundException
 at com.sun.server.http.FileServlet.sendResponse(FileServlet.java:153)
 at com.sun.server.http.FileServlet.service(FileServlet.java:114)
 at com.sun.server.webserver.FileServlet.service(FileServlet.java:202)
 at javax.servlet.http.HttpServlet.service(HttpServlet.java:588)
 at com.sun.server.ServletManager.callServletService(ServletManager.java:936)

 at com.sun.server.webserver.HttpServiceHandler.handleRequest(
                                                HttpServiceHandler.java:416)
 at com.sun.server.webserver.HttpServiceHandler.handleRequest(
                                                HttpServiceHandler.java:246)
 at com.sun.server.HandlerThread.run(HandlerThread.java:154)
```

This simple example was generated when the file was not found, but this technique can be used for problems with posted data. Remember to use cont to allow the web server to proceed. To clear this trap use the ignore command.

```
webpageservice Handler[1] ignore java.io.IOException
webpageservice Handler[1] cont
webpageservice Handler[1]
```

Abstract Window Toolkit Debugging

Before the new Abstract Window Toolkit (AWT) Event mechanism introduced in JDK 1.1, events were received by a component such as a TextField, and propagated upwards to its parent components. This meant you could simply add some diagnostic code to the component's handleEvent or action method to monitor the events as they arrived.

With the introduction of JDK 1.1 and the new system event queue, events are delivered to an event queue instead of the component itself. The events are then dispatched from the System Event queue to event listeners that register to be notified when an event has been dispatched for that object.

Using AWTEventListener

You can use an AWTEventListener to monitor the AWT events from a system event queue. This listener takes an event mask built from an OR operation of the AWTEvents you want to monitor.

Note: Do not use AWTEventListener in a shipping product because it will degrade system performance

---

```
//EventTest.java
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class EventTest extends JFrame {
   public EventTest() {
     JButton jb1=new JButton("hello");
     getContentPane().add(jb1);
//API Ref : void addAWTEventListener(AWTEventListener l, long eventmask)
     getToolkit().addAWTEventListener(new AWTEventListener() {
         public void eventDispatched(AWTEvent e) {
           System.out.println(e + "\n");
         }
       }, AWTEvent.MOUSE_EVENT_MASK | AWTEvent.FOCUS_EVENT_MASK
        );
   }
   public static void main (String args[]) {
     EventTest et=new EventTest();
     et.setSize(300,300);
     et.pack();
     et.show();
   }
}
```

At runtime, the `EventTest` program tracks and outputs mouse and focus events. For example, when you click the button, you get this output:

```
java.awt.event.MouseEvent[MOUSE_CLICKED,(58,14),mods=16,clickCount=1] on
javax.swing.JButton[,0,0,96x27,layout=javax.swing.OverlayLayout,alignmentX=0.0,a
lignmentY=0.5,border=javax.swing.plaf.BorderUIResource$CompoundBorderUIResource@
e466ea9c,flags=48,maximumSize=,minimumSize=,preferredSize=,defaultIcon=,disabled
Icon=,disabledSelectedIcon=,margin=javax.swing.plaf.InsetsUIResource[top=2,left=
14,bottom=2,right=14],paintBorder=true,paintFocus=true,pressedIcon=,rolloverEnab
led=false,rolloverIcon=,rolloverSelectedIcon=,selectedIcon=,text=hello,defaultCa
pable=true]
```

To obtain a simple list of the AWTEvent events, use the javap -public java.awt.AWTEvent command.

```
Compiled from AWTEvent.java
public abstract class java.awt.AWTEvent extends java.util.EventObject {
    public static final long COMPONENT_EVENT_MASK;
    public static final long CONTAINER_EVENT_MASK;
    public static final long FOCUS_EVENT_MASK;
    public static final long KEY_EVENT_MASK;
    public static final long MOUSE_EVENT_MASK;
    public static final long MOUSE_MOTION_EVENT_MASK;
    public static final long WINDOW_EVENT_MASK;
    public static final long ACTION_EVENT_MASK;
    public static final long ADJUSTMENT_EVENT_MASK;
    public static final long ITEM_EVENT_MASK;
    public static final long TEXT_EVENT_MASK;
    public static final long INPUT_METHOD_EVENT_MASK;
    public static final int RESERVED_ID_MAX;
    public java.awt.AWTEvent(java.awt.Event);
    public java.awt.AWTEvent(java.lang.Object,int);
    public int getID();
    public java.lang.String paramString();
    public java.lang.String toString();
}
```

Analyzing Stack Traces

Stack traces have often been considered a mystery to developers. There is little or no documentation available, and when you get one or need to generate one, time is always at a premium. The next sections uncover the secrets to debugging stack traces, and by the end, you might consider a stack trace to be a helpful tool for analyzing other programs--not just broken ones!

A stack trace produced by the Java platform is a user friendly snapshot of the threads and monitors in a Java virtual machine. Depending on how complex your application or applet is, a stack trace can range from fifty lines to thousands of lines of diagnostics. Regardless of the size of the stack trace, there are a few key things that anyone can find to help diagnose most software problems, whether you are an expert or very new to the Java platform.

There are three popular ways to generate a stack trace: sending a signal to the Java virtual machine; the Java virtual machine generates a stack trace for you; or using debugging tools or API calls.

Sending a signal to the Java VM

On UNIX platforms you can send a signal to a program with the kill command. This is the quit signal, which is handled by the Java virtual machine.

Unix Systems

For example, on the Solaris™ and other Unix platforms, you can use the kill -QUIT process_id command, where process_id is the process number of your program. Alternately, you can enter the key sequence <ctrl>\ in the window where the program started. Sending this signal instructs a signal handler in the Java virtual machine to recursively print out all the information on the threads and monitor inside the Java virtual machine.

Windows 95/NT

To generate a stack trace on the Windows 95 or Windows NT platforms, enter the key sequence <ctrl><break> in the window where the program is running.

The Java VM Generates a Stack Trace

If the Java virtual machine experienced an internal error such as a segmentation violation or an illegal page fault, it calls its own signal handler to print out the threads and monitor information.

Core Files

If the JVM generated the stack trace because of an internal error then some native code in your own application or the JVM is probably to blame. If you are using UNIX, and you find a core file, run the following command to find out which JDK™ software it came from:

```
strings core | grep JAVA_HOME
```

Using Debugging Tools or API Calls

You can generate a partial stack trace, (which in this case is only the threads information) by using the Thread.dumpStack method, or the printStackTrace method of the Throwable class. You can also obtain similar information by entering where inside the Java debugger.

If you are successful at generating a stack trace, you should see something similar to Stack Trace 1 (page 331).

In the Java 2 software release, threads that called methods resulting in a call to native code are indicated in the stack trace.

Which Release Generated The Stack Trace?

In the Java 2 release the stack trace contains the Java virtual machine version string, the same information you see when using the -version parameter.

However, if there is no version string, you can still take a pretty good guess at which release this stack trace came from. Obviously, if you generated the stack trace yourself this should not be much of an issue, but you may see a stack trace posted on a news group or in an email.

First identify where the Registered Monitor Dump section is in the stack trace:

- If you see a utf8 hash table lock in the Registered Monitor Dump, this is a Java 2 platform stack trace. The final release of the Java 2 platform also contains a version string so if a version string is missing this stack trace may be from a Java 2 beta release.
- If you see a JNI pinning lock and no utf8 hash lock, this is a JDK 1.1+ release.

If neither of these appears in the Registered Monitor Dump, it is probably a JDK 1.0.2 release.

Which Platform Generated the Stack Trace?

You can also find out if the stack trace came from a Windows 95, an NT, or UNIX machine by looking for any waiting threads. On a UNIX machine the waiting threads are named explicitly. On a Windows 95, or NT machine only a count of the waiting threads is displayed:

- Windows 95/NT: Finalize me queue lock: <unowned> Writer: 1
- UNIX: Finalize me queue lock: <unowned> waiting to be notified “Finalizer Thread”

Which Thread Package was Used?

Windows 95, Windows NT, and Solaris Java virtual machines use native threads by default. There also exists a pseudo thread implementation called green threads. To make your Java virtual machine use either green or native threads, specify the `-green` or -native parameter. For example, java -native MyClass.

By verifying the existence of an Alarm monitor in the stack trace output you can identify that this stack trace came from a green threads Java virtual machine.

What are the Thread States?

You will see many different threads in many different states in a snapshot from a Java VM stack trace. This table describes the various keys and their meanings.

Key	Meaning
R	Running or runnable thread

Key	Meaning
S	Suspended thread
CW	Thread waiting on a condition variable
MW	Thread waiting on a monitor lock. Could indicate a deadlock.
MS	Thread suspended waiting on a monitor lock

Normally, only threads in R, S, CW or MW should appear in the stack trace. If you see a thread in state MS, report it to Sun Microsystems, through the Java Developer Connection (JDC) Bug Parade feature, because there is a good chance it is a bug. The reason being that most of the time a thread in Monitor Wait (MW) state will appear in the S state when it is suspended.

Monitors let you manage access to code that should only be run by one thread at a time. See Examining Monitors (page 324) for more information on monitors.

The other two common thread states you may see are R, runnable threads and CW, threads in a condition wait state. Runnable threads by definition are threads that could be running or are running at that instance of time. On a multi-processor machine running a true multi-processing Operating System, it is possible for all the runnable threads to be running at one time. However, it is more likely for the other runnable threads to be waiting on the thread scheduler to have their turn to run.

Threads in a condition wait state can be thought of as waiting for an event to occur. Often a thread will appear in state CW if it is in a Thread.sleep or in a synchronized wait. In our earlier stack trace our main method was waiting for a thread to complete and to be notified of its completion. In the stack trace this appears as

```
"main" (TID:0xebc981e0, sys_thread_t:0x26bb0, state:CW) prio=5
 at java.lang.Object.wait(Native Method)
 at java.lang.Object.wait(Object.java:424)
 at HangingProgram.main(HangingProgram.java:33)
```

The code that created this stack trace is as follows:

```
synchronized(t1) {
  try {
    t1.wait();    //line 33
  }catch (InterruptedException e){}
}
```

In the Java 2 release, monitor operations, including our wait here, are handled by the Java virtual machine through a JNI call to sysMonitor. The condition wait thread is kept on a special monitor wait queue on the object it is waiting on. This explains why even though you are only waiting on an object that the code still needs to be synchronized on that object as it is in fact using the monitor for that object.

Examining Monitors

This brings us to the other part of the stack trace: the monitor dump. If you consider that the threads section of a stack trace identifies the multithreaded part of your application, then the monitors section represents the parts of your application that are single threaded.

It may be easier to imagine a monitor as a car wash. In most car washes, only one car can be in the wash at a time. In your Java code only one thread at a time can have the lock to a synchronized piece of code. All the other threads queue up to enter the synchronized code just as cars queue up to enter the car wash.

A monitor can be thought of as a lock on an object, and every object has a monitor. When you generate a stack trace, monitors are either listed as being registered or not. In the majority of cases these registered monitors, or system monitors, should not be the cause of your software problems, but it helps to be able to understand and recognize them. The following table describes the common registered monitors:

Monitor	Description
utf8 hash table	Locks the hashtable of defined i18N Strings that were loaded from the class constant pool.
JNI pinning lock	Protects block copies of arrays to native method code.
JNI global reference lock	Locks the global reference table which holds values that need to be explicitly freed, and will outlive the lifetime of the native method call.
BinClass lock	Locks access to the loaded and resolved classes list. The global table list of classes.
Class linking lock	Protects a classes data when loading native libraries to resolve symbolic references.
System class loader lock	Ensures that only one thread is loading a system class at a time.
Code rewrite lock	Protects code when an optimization is attempted.
Heap lock	Protects the Java heap during heap memory management.
Monitor cache lock	Only one thread can have access to the monitor cache at a time this lock ensures the integrity of the monitor cache.
Dynamic loading lock	Protects Unix green threads JVMs from loading the shared library stub libdl.so more than once at a time.
Monitor IO lock	Protects physical I/O, for example, open and read.
User signal monitor	Controls access to the signal handler if a user signal USRSIG in green thread Java virtual machine.

Monitor	Description
Child death monitor	Controls access to the process wait information when using the runtime system calls to run locals commands in a green threads Java virtual machine.
I/O Monitor	Controls access to the threads file descriptors for poll/select events.
Alarm Monitor	Controls access to a clock handler used in green threads Java virtual machines to handle time-outs.
Thread queue lock	Protects the queue of active threads.
Monitor registry	Only one thread can have access to the monitor registry at a time this lock ensures the integrity of that registry.
Has finalization queue lock *	Protects the list of queue lock objects that have been garbage-collected, and deemed to need finalization. They are copied to the Finalize me queue.
Finalize me queue lock *	Protects a list of objects that can be finalized at leisure.
Name and type hash table lock *	Protects the Java virtual machine hash tables of constants and their types.
String intern lock *	Locks the hashtable of defined strings that were loaded from the class constant pool.
Class loading lock *	Ensures only one thread loads a class at a time.
Java stack lock *	Protects the free stack segments list.

Note: * Lock only appeared in pre-Java 2 stack traces

The monitor registry itself is protected by a monitor. This means the thread that owns the lock is the last thread to use a monitor. It is very likely this thread is also the current thread. Because only one thread can enter a synchronized block at a time, other threads queue up at

the start of the synchronized code and appear as thread state MW. In the monitor cache dump, they are denoted as "waiting to enter" threads. In user code a monitor is called into action wherever a synchronized block or method is used.

Any code waiting on an object or event (a wait method) also has to be inside a synchronized block. However, once the wait method is called, the lock on the synchronized object is given up.

When the thread in the wait state is notified of an event to the object, it has to compete for exclusive access to that object, and it has to obtain the monitor. Even when a thread has sent a "notify event" to the waiting threads, none of the waiting threads can actually gain control of the monitor lock until the notifying thread has left its synchronized code block. You will see "Waiting to be notified" for threads at the wait method

Putting the Steps Into Practice

Example 1

Consider a real-life problem such as Bug ID 4098756, for example. You can find details on this bug in JDC Bug Parade. This bug documents a problem that occurs when using a Choice Component on Windows 95.

When the user selects one of the choices from the Choice Component using the mouse, everything is fine. However, when the user tries to use an Arrow key to move up or down the list of choices, the application freezes.

Fortunately, this problem is reproducible and produces a stack trace to help track down the problem. The full stack trace is in the bug report page, but you only need to focus on the following two key threads:

```
"AWT-Windows" (TID:0xf54b70, sys_thread_t:0x875a80,Win32ID:0x67, state:MW) prio=5
 java.awt.Choice.select(Choice.java:293)
 sun.awt.windows.WChoicePeer.handleAction(WChoicePeer.java:86)
 "AWT-EventQueue-0" (TID:0xf54a98,sys_thread_t:0x875c20,
 Win32ID:0x8f, state:R) prio=5
 java.awt.Choice.remove(Choice.java:228)
 java.awt.Choice.removeAll(Choice.java:246)
```

The AWT-EventQueue-0 thread is in a runnable state inside the remove method. Remove is synchronized, which explains why the AWT-Windows thread cannot enter the select method. The AWT-Windows thread is in MW state (monitor wait); however, if you keep taking stack traces, this situation does not change and the graphical user interface (GUI) appears to have frozen.

This indicates that the remove call never returned. By following the code path to the ChoicePeer class, you can see this is making a native MFC call that does not return. That is where the real problem lies and is a bug in the Java core classes. The user's code was okay.

Example 2

In this second example you will investigate a bug that on initial outset appears to be a fault in Project Swing, but as you will discover is due to the fact that Project Swing is not thread safe. Again the bug report is available to view on the JDC site, the bug number this time is 4098525.

Here is a cut down sample of the code used to reproduce this problem. The modal dialog is being created from within the JPanel paint method.

```
import java.awt.event.*;
import java.awt.*;
import java.util.*;
import javax.swing.*;

class MyDialog extends Dialog implements ActionListener {
  MyDialog(Frame parent) {
    super(parent, "My Dialog", true);
    Button okButton = new Button("OK");
    okButton.addActionListener(this);
    add(okButton);
    pack();
  }
  public void actionPerformed(ActionEvent event) {
    dispose();
  }
}
public class Tester extends JPanel {
  MyDialog myDialog;
  boolean firstTime = true;
  public Tester (JFrame frame) throws Exception {
    super();
    myDialog = new MyDialog(frame);
  }
  void showDialogs() {
    myDialog.show();
  }
  public void paint(Graphics g) {
    super.paint(g);
    if (firstTime) {
        firstTime = false;
        showDialogs();
     }
   }
   public static void main(String args[]) throws Exception {
     JFrame frame = new JFrame ("Test");
     Tester gui = new Tester(frame);
```

```
        frame.getContentPane().add(gui);
        frame.setSize(800, 600);
       frame.pack();
       frame.setVisible(true);
    }
}
```

When you run this program, you find it deadlocks straight away. By taking a stack trace you see the key threads shown in Stack Trace 2 (page 332). This stack trace is slightly different from the stack trace that appears in the bug report, but is caused by the same problem.

Stack Trace 2 is produced by the Java 2 platform release using the -Djava.compiler=NONE option to the `java` interpreter command so you can see the source line numbers. The thread to look for is the thread in monitor wait, which in this case is thread AWT-EventQueue-1.

```
"AWT-EventQueue-1" (TID:0xebca8c20, sys_thread_t:0x376660, state:MW) prio=6
at java.awt.Component.invalidate(Component.java:1664)
at java.awt.Container.invalidate(Container.java:507)
at java.awt.Window.dispatchEventImpl(Window.java:696)
at java.awt.Component.dispatchEvent(Component.java:2289)
at java.awt.EventQueue.dispatchEvent(EventQueue.java:258)
at java.awt.EventDispatchThread.run(EventDispatchThread.java:68)
```

If you look for that line in file java/awt/Component.java which is contained in the src.jar archive, you see the following:

```
public void invalidate() {
  synchronized (getTreeLock()) { //line 1664
```

This is where the application is stuck. It is waiting for the getTreeLock monitor lock to become free. The next task is to find out which thread has this getTreeLock monitor lock held. To see who is holding this monitor lock you look at the Monitor cache dump and in this example you can see the following:

```
Monitor Cache Dump:
java.awt.Component$AWTTreeLock@EBC9C228/EBCF2408:
                      owner "AWT-EventQueue-0" (0x263850) 3 entries
Waiting to enter: "AWT-EventQueue-1" (0x376660)
```

The method getTreeLock monitor is actually a lock on a specially created inner class object of AWTTreeLock. This is the code used to create that lock in file Component.java.

```
static final Object LOCK = new AWTTreeLock();
static class AWTTreeLock {}
```

The current owner is AWT-EventQueue-0. This thread called the paint method to create the modal Dialog with a call to the paintComponent method. The paintComponent method was called from an update call of JFrame.

So where was the lock set? Well there is no simple way to find out which stack frame actually held the lock, but on a simple search of javax.swing.JComponent, you see that getTreeLock is called inside the method paintChildren which you left at line 388.

```
at Tester.paint(Tester.java:39)
at javax.swing.JComponent.paintChildren(JComponent.java:388)
```

The rest of the puzzle is pieced together by analyzing the `MDialogPeer` show method. The Dialog code creates a new ModalThread which is why you see an `AWT-Modal` thread in the stack trace output, this thread is used to post the Dialog. It is when this event is dispatched using `AWT-EventQueue-1`, which used to be the AWT Dispatch proxy, that `getTreeLock` monitor access is required and so you have a deadlock.

Unfortunately, Project Swing code is not designed to be thread safe, and so the workaround in this example is to not create modal dialogs inside Project Swing paint methods. Since Swing has to do a lot of locking and calculations as to which parts of a lightweight component needs to be painted, it is strongly recommended to not include synchronized code or code that will result in a synchronized call such as in a modal dialog, inside `paint` method.

You should now know what to look for the next time you see a stack trace. To save time, you should make full use of the JDC bug search to see if the problem you are having has already been reported by someone else.

Expert's Checklist

To summarize, these are the steps to take the next time you come across a problem in a Java program:

- **Hanging, deadlocked or frozen programs:** If you think your program is hanging, generate a stack trace. Examine the threads in states `MW` or `CW`. If the program is deadlocked, some of the system threads will probably show up as the current thread because there is nothing else for the Java virtual machine to do.
- **Crashed or aborted programs:** On UNIX look for a core file. You can analyze this file in a native debugging tool such as `gdb` or `dbx`. Look for threads that have called native methods. Because Java technology uses a safe memory model, any corruption probably occurred in the native code. Remember that the Java virtual machine also uses native code so it might not be a bug in your application.
- **Busy programs:** The best course of action you can take for busy programs is to generate frequent stack traces. This will narrow down the code path that is causing the errors, and you can start your investigation from there.

Stack Trace Examples

- Stack Trace 1 (page 331)
- Stack Trace 2 (page 332)

Stack Trace 1

```
$ java -Djava.compiler=NONE HangingProgram
^\SIGQUIT

Full thread dump Classic VM (JDK-1.2-V, green threads):
 "Thread-1" (TID:0xebc9c0f0, sys_thread_t:0x130180, state:MW) prio=5
    at mythread.stopper(HangingProgram.java:9)
    at mythread.run(HangingProgram.java:19)
 "Thread-0" (TID:0xebc9c150, sys_thread_t:0x12f960, state:CW) prio=5
    at java.lang.Thread.sleep(Native Method)
    at mythread.stopper(HangingProgram.java:12)
    at mythread.run(HangingProgram.java:19)
 "Finalizer" (TID:0xebc98320, sys_thread_t:0x69418, state:CW) prio=8
    at java.lang.Object.wait(Native Method)
    at java.lang.ref.ReferenceQueue.remove(ReferenceQueue.java:112)
    at java.lang.ref.ReferenceQueue.remove(ReferenceQueue.java:127)
    at java.lang.ref.Finalizer$FinalizerThread.run(Finalizer.java:174)
 "Reference Handler" (TID:0xebc983b0, sys_thread_t:0x64f68, state:CW) prio=10
    at java.lang.Object.wait(Native Method)
    at java.lang.Object.wait(Object.java:424)
    at java.lang.ref.Reference$ReferenceHandler.run(Reference.java:114)
 "Signal dispatcher" (TID:0xebc983e0, sys_thread_t:0x5e1e8, state:R) prio=5
 "main" (TID:0xebc981e0, sys_thread_t:0x26bb0, state:CW) prio=5
    at java.lang.Object.wait(Native Method)
    at java.lang.Object.wait(Object.java:424)
    at HangingProgram.main(HangingProgram.java:33)
Monitor Cache Dump:
  java.lang.Class@EBC9BEB8/EBCFC538: owner "Thread-0" (0x12f960) 1 entry
     Waiting to enter:
            "Thread-1" (0x130180)
  java.lang.ref.ReferenceQueue$Lock@EBC98338/EBCCE028: <unowned>
     Waiting to be notified:
            "Finalizer" (0x69418)
  mythread@EBC9C150/EBCFC608: <unowned>
     Waiting to be notified:
            "main" (0x26bb0)
  java.lang.ref.Reference$Lock@EBC983C0/EBCCDB20: <unowned>
     Waiting to be notified:
            "Reference Handler" (0x64f68)
Registered Monitor Dump:
  utf8 hash table: <unowned>
  JNI pinning lock: <unowned>
  JNI global reference lock: <unowned>
  BinClass lock: <unowned>
```

```
  Class linking lock: <unowned>
  System class loader lock: <unowned>
  Code rewrite lock: <unowned>
  Heap lock: <unowned>
  Monitor cache lock: owner "Signal dispatcher" (0x5e1e8) 1 entry
  Dynamic loading lock: <unowned>
  Monitor IO lock: <unowned>
  User signal monitor: <unowned>
  Child death monitor: <unowned>
  I/O monitor: <unowned>
  Alarm monitor: <unowned>
     Waiting to be notified:
        <unknown thread> (0x2beb8)
  Thread queue lock: owner "Signal dispatcher" (0x5e1e8) 1 entry
  Monitor registry: owner "Signal dispatcher" (0x5e1e8) 1 entry
```

Stack Trace 2

```
Full thread dump Classic VM (JDK-1.2-V, green threads):
    "AWT-Modal" (TID:0xebca8a40, sys_thread_t:0x376d50, state:CW) prio=6
        at java.lang.Object.wait(Native Method)
        at sun.awt.motif.MDialogPeer.pShow(Native Method)
        at sun.awt.motif.ModalThread.run(MDialogPeer.java:247)
    "AWT-EventQueue-1" (TID:0xebca8c20, sys_thread_t:0x376660, state:MW) prio=6
        at java.awt.Component.invalidate(Component.java:1664)
        at java.awt.Container.invalidate(Container.java:507)
        at java.awt.Window.dispatchEventImpl(Window.java:696)
        at java.awt.Component.dispatchEvent(Component.java:2289)
        at java.awt.EventQueue.dispatchEvent(EventQueue.java:258)
        at java.awt.EventDispatchThread.run(EventDispatchThread.java:68)
    "Screen Updater" (TID:0xebca9a08, sys_thread_t:0x3707c8, state:CW) prio=4
        at java.lang.Object.wait(Native Method)
        at java.lang.Object.wait(Object.java:424)
        at sun.awt.ScreenUpdater.nextEntry(ScreenUpdater.java:79)
        at sun.awt.ScreenUpdater.run(ScreenUpdater.java:99)
    "AWT-Motif" (TID:0xebcafa30, sys_thread_t:0x285370, state:CW) prio=5
        at sun.awt.motif.MToolkit.run(Native Method)
        at java.lang.Thread.run(Thread.java:479)
    "SunToolkit.PostEventQueue-0" (TID:0xebcafc58, sys_thread_t:0x263988,
state:CW) prio=5
        at java.lang.Object.wait(Native Method)
        at java.lang.Object.wait(Object.java:424)
        at sun.awt.PostEventQueue.run(SunToolkit.java:363)
    "AWT-EventQueue-0" (TID:0xebcafc28, sys_thread_t:0x263850, state:CW) prio=6
        at java.lang.Object.wait(Native Method)
        at java.lang.Object.wait(Object.java:424)
        at sun.awt.motif.MDialogPeer.show(MDialogPeer.java:181)
        at java.awt.Dialog.show(Dialog.java:368)
        at Tester.showDialogs(Tester.java:32)
        at Tester.paint(Tester.java:39)
        at javax.swing.JComponent.paintChildren(JComponent.java:388)
        at javax.swing.JComponent.paint(JComponent.java:550)
```

```
        at javax.swing.JComponent.paintChildren(JComponent.java:388)
        at javax.swing.JComponent.paint(JComponent.java:550)
        at javax.swing.JLayeredPane.paint(JLayeredPane.java:547)
        at javax.swing.JComponent.paintChildren(JComponent.java:388)
        at javax.swing.JComponent.paint(JComponent.java:535)
        at java.awt.Container.paint(Container.java:770)
        at javax.swing.JFrame.update(JFrame.java:255)
        at sun.awt.motif.MComponentPeer.handleEvent(MComponentPeer.java:248)
        at java.awt.Component.dispatchEventImpl(Component.java:2429)
        at java.awt.Container.dispatchEventImpl(Container.java:1032)
        at java.awt.Window.dispatchEventImpl(Window.java:714)
        at java.awt.Component.dispatchEvent(Component.java:2289)
        ... (more frames not shown)
    “Finalizer” (TID:0xebc98320, sys_thread_t:0x69418, state:CW) prio=8
        at java.lang.Object.wait(Native Method)
        at java.lang.ref.ReferenceQueue.remove(ReferenceQueue.java:112)
        at java.lang.ref.ReferenceQueue.remove(ReferenceQueue.java:127)
        at java.lang.ref.Finalizer$FinalizerThread.run(Finalizer.java:174)
    “Reference Handler” (TID:0xebc983b0, sys_thread_t:0x64f68, state:CW) prio=10
        at java.lang.Object.wait(Native Method)
        at java.lang.Object.wait(Object.java:424)
        at java.lang.ref.Reference$ReferenceHandler.run(Reference.java:114)
    “Signal dispatcher” (TID:0xebc983e0, sys_thread_t:0x5e1e8, state:R) prio=5
    “Thread-0” (TID:0xebcaa000, sys_thread_t:0x26bb0, state:CW) prio=5
Monitor Cache Dump:
    java.awt.Component$AWTTreeLock@EBC9C228/EBCF2408: owner “AWT-EventQueue-0”
                (0x263850) 3 entries
        Waiting to enter:
            “AWT-EventQueue-1” (0x376660)
    java.lang.Class@EBCA5148/EBD11A48: <unowned>
        Waiting to be notified:
            “AWT-Modal” (0x376d50)
    sun.awt.motif.ModalThread@EBCA8A40/EBD67998: owner “AWT-Modal” (0x376d50) 1
entry
        Waiting to be notified:
            “AWT-EventQueue-0” (0x263850)
    java.lang.ref.ReferenceQueue$Lock@EBC98338/EBCCDCD8: <unowned>
        Waiting to be notified:
            “Finalizer” (0x69418)
    sun.awt.PostEventQueue@EBCAFC58/EBD405B0: <unowned>
        Waiting to be notified:
            “SunToolkit.PostEventQueue-0” (0x263988)
    sun.awt.ScreenUpdater@EBCA9A08/EBD633F8: <unowned>
        Waiting to be notified:
            “Screen Updater” (0x3707c8)
    java.lang.ref.Reference$Lock@EBC983C0/EBCCD8A8: <unowned>
        Waiting to be notified:
            “Reference Handler” (0x64f68)
Registered Monitor Dump:
    utf8 hash table: <unowned>
    JNI pinning lock: <unowned>
    JNI global reference lock: <unowned>
    BinClass lock: <unowned>
```

```
Class linking lock: <unowned>
System class loader lock: <unowned>
Code rewrite lock: <unowned>
Heap lock: <unowned>
Monitor cache lock: owner “Signal dispatcher” (0x5e1e8) 1 entry
Dynamic loading lock: <unowned>
Monitor IO lock: <unowned>
User signal monitor: <unowned>
Child death monitor: <unowned>
I/O monitor: <unowned>
    Waiting to be notified:
        “AWT-Motif” (0x285370)
Alarm monitor: <unowned>
    Waiting to be notified:
        <unknown thread> (0x2beb8)
Thread queue lock: owner “Signal dispatcher” (0x5e1e8) 1 entry
    Waiting to be notified:
        “Thread-0” (0x26bb0)
Monitor registry: owner “Signal dispatcher” (0x5e1e8) 1 entry
```

Version Issues

This section summarizes problems and solutions related to having different versions of the Java platform installed on your system.

JDK 1.0.2

Deployment

Uses CLASSPATH to find and load the core system classes.

On Windows 95:

```
CLASSPATH=c:\java\lib\classes.zip:.
```

On Unix:

```
CLASSPATH=/usr/java/lib/classes.zip:.
```

Unix dynamic libraries, .dll files, shared objects, and .so files are located by the PATH variable.

Side Effects

The Win95 Autoexec.bat file contains an outdated CLASSPATH variable set by a user or the installation of other applications.

The WinNT User Environment contains an old CLASSPATH variable.

The Unix .cshrc, .profile, or .login scripts contains wrong CLASSPATH.

The JAVA_HOME environment variable is also used by programs so check this is not set. You can clear this field in the Bourne shell (sh) as follows: unset JAVA_HOME

Diagnostics

Use the -classpath option to force the Java VM to use the command-line CLASSPATH only: java -classpath c:\java\lib\classes.zip;. myapp

JDK 1.1

Deployment

Uses relative paths to find the classes.zip file from the Java platform installation. The CLASSPATH environment variable is used to load application classes.

Side Effects

Other Java releases found on the application path might be picked up if the new JDK bin directory is not explicitly set at the front of the PATH environment variable.

Diagnostics

Use the -sysclasspath option to force the Java VM to use the CLASSPATH supplied on the command line only: java -sysclasspath c:\java\lib\classes.zip;. myapp

Java 2 Platform

Deployment

The platform is split into a Java Runtime Environment (JRE) and Java compiler. The JRE is included as a subdirectory in the release, and the traditional java and javac programs in the bin directory invoke the real program in the jre/bin directory. The separate jre launcher is no longer provided, and the java program is solely used instead.

The Java Archive (JAR) files containing the core Java platform system classes, rt.jar and i18.jar, are located in the jre/lib directory with a relative search path.

Side Effects

If applications previously used the classes.zip file to load the core Java platform systems, they might still try to load an additional set of classes in error.

Diagnostics

Use the -Xbootclasspath option to force the Java VM to use the CLASSPATH supplied on the command line only: java -Xbootclasspath:c:\java\jre\lib\rt.jar;c:\java\jre\lib\i18n.jar;. myapp

You might need to supply this as a runtime option as follows: javac -J-Xbootclasspath:c\java\lib\tools.jar;c: \java\jre\lib\rt.jar;c:\java\jre\lib\i18n.jar;. myapp.java

Java Plug-In

Deployment

On Windows 95 and Windows NT uses the registry to find installed plug-in Java platform releases.

Side Effects

Registry can become corrupted, or plug-in removed physically but not from the registry.

Diagnostics

Display the java.version and java.class.path property in your code and display it on the Java Plug-in Console

```
System.out.println("version="+System.getProperty("java.version"));
System.out.println("class path="+System.getProperty("java.class.path");
```

If there is a conflict, check the registry with the regedit command, search for the word VM and if it exists, delete it and reinstall the plug-in

Netscape

Deployment

Uses .jar files such as java40.jar in the netscape directory.

Side Effects

Not all Netscape releases are fully JDK 1.1 compliant. You can get upgrade patches at `http://www.netscape.com`.

Diagnostics

Start the browser on the command line with the -classes option.

Internet Explorer

Deployment

Uses .cab files to contain system classes. Also uses system registry on Windows 95/NT.

Side Effects

Use the regedit command to search for the word VM. There is a CLASSPATH entry to which you can add your own classes.

Diagnostics

The registry can become corrupted. Search for CLASSPATH using the regedit program and edit the value that CLASSPATH points to.

8: Performance Techniques

One of the biggest challenges in developing large applications for the Java platform is to make the application meet its performance criteria. This chapter departs from the auction application and uses simple and targeted examples to show you how to track down performance bottlenecks to improve application performance.

Covered in this Chapter

- Connection Pooling (page 344)
- Performance Features and Tools (page 348)
- Performance Analysis (page 354)
- Caching Client/Server Applications (page 363)
- Code for this Chapter (page 369)

Improving Applet Download Speed

Applet download performance refers to the time it takes for the browser to download all the files and resources it needs to start the applet. An important factor affecting any applet's download performance is the number of times it has to request data from the server. You can reduce the number of requests by packaging the applet images into one class file, or using Java Archive (JAR) files.

Packaging Images into One Class

Normally, if an applet has, for example, six image buttons, that translates to six additional requests sent back to the web server to load those image files. Six additional requests might not seem like much on an internal network, but given connections of lesser speed and reliability, those additional requests can have a significant negative impact on performance. So, your ultimate goal should be to load the applet as quickly as possible.

One way to store images in a class file is to use an ASCII encoding scheme such as X-PixMap (XPM) (`http://www.inria.fr/koala/lehors/xpm.html`). This way, rather than maintain-

ing the images as GIF files on the server, the files are encoded as Strings and stored in a single class file.

This code sample uses packages from the JavaCup winner at JavaOne 1996, which contains the XImageSource and XpmParser classes. These classes provide all you need to read a standard XPM file. You can see these files at SunSite (`http://sunsite.utk.edu/winners_circle/developer_tools/DESVS7NU/applet.html`).

For the initial encoding process, there are a number of graphics tools you can use to create XPM files. On Solaris you can use ImageTool or a variety of other GNU image packages (`http://www.gnu.ai.mit.edu/software/software.html`). Go to the Download.com web site (`http://download.cnet.com`) to get the encoding software for Windows platforms.

The following code excerpted from the MyApplet (page 369) sample class loads the images shown in Figure 33. You can see the coded String form for the images in the XPM Definition (page 375) of the images.



Figure 33 Images

The Toolkit class creates an Image object for each image from the XPM Image Source object. The parameters to `XImageSource` represent a coded `String` form from the XPM Definition (page 375).

```
Toolkit kit = Toolkit.getDefaultToolkit();
Image image;
mage = kit.createImage (new XImageSource (_reply));
image = kit.createImage (new XImageSource (_post));
image = kit.createImage (new XImageSource (_reload));
image = kit.createImage (new XImageSource (_catchup));
image = kit.createImage (new XImageSource (_back10));
image = kit.createImage (new XImageSource (_reset));
image = kit.createImage (new XImageSource (_faq));
```

The alternative technique below uses GIF files. It requires a request back to the web server for each image loaded.

```
Image image;
image = getImage (reply.gif);
image = getImage (post.gif);
image = getImage (reload.gif);
image = getImage (catchup.gif);
```

```
image = getImage (back10.gif);
image = getImage (reset.gif);
image = getImage (faq.gif);
```

This technique reduces network traffic because all images are available in a single class file.

- Using XPM encoded images makes the class size larger, but the number of network requests fewer.
- Making the XPM image definitions part of your applet class file makes the image loading process part of the regular loading of the applet class file with no extra classes.

Once loaded, you can use the images to create buttons or other user interface components. This next code segment shows how to use the images with the javax.swing.JButton class.

```
ImageIcon icon = new ImageIcon(kit.createImage(new XImageSource(_reply)));
JButton button = new JButton (icon, Reply);
```

Using JAR Files

When an applet consists of more than one file, you can improve download performance with JAR files. A JAR file contains all of an applet's related files in one single file for a faster download. Much of the time saved comes from reducing the number of HTTP connections the browser must make. 9: Deploying the Auction Application (page 399) has information on creating and signing JAR files.

The HTML code below uses the CODE tag to specify the executable for the MyApplet applet, and the ARCHIVE tag to specify the JAR file that contains all of MyApplet's related files. The executable specified by the CODE tag is sometimes called the code base.

For security reasons the JAR files listed by the archive parameter must be in the same directory or a sub-directory as the applet’s codebase. If no codebase parameter is supplied, the directory from where the applet was loaded is used as the codebase. The following example specifies jarfile as the JAR file that contains the related files for the MyApplet.class executable.

```
<APPLET CODE=MyApplet.class ARCHIVE=jarfile WIDTH=100 HEIGHT=200>
</APPLET>
```

The applet downloads the entire JAR file, regardless of whether or not the JAR file includes infrequently used files.

```
<APPLET CODE=MyApplet.class ARCHIVE=jarfile1, jarfile2 WIDTH=100 HEIGHT=200>
</APPLET>
```

To improve performance when an applet has infrequently used files, put the frequently used files into the JAR file and the infrequently used files into the applet class directory. Infrequently used files are then located and downloaded by the browser only when needed.

Thread Pooling

Bandwidth restrictions imposed on networks around the world make network-based operations potential bottlenecks that can have a significant impact on an application's performance. Many network-based applications are designed to use connection pools so they can reuse existing network connections and save on the time and overhead invested in opening and closing network connections.

The Java Developer Connection (JDC) applet servers and the Java WebServer™ make extensive use of thread pooling to improve performance. Thread pooling is creating a ready supply of sleeping threads at the beginning of execution. Thread creation and their statup process is expensive, so rutime performance is improved with thread pooling because threads are not created during runtime; they are simply reused.

This code sample taken from the Pool (page 380) class shows one way to implement thread pooling. In the pool's constructor (shown below), the WorkerThreads are initialized and started. The call to the start method executes the run method of the WorkerThread, and the call to wait in the run method suspends the Thread while the Thread waits for work to arrive. The last line of the constructor pushes the sleeping Thread onto the stack.

```
public Pool (int max, Class workerClass)   throws Exception {
  _max = max;
  _waiting = new Stack();
  _workerClass = workerClass;
  Worker worker;
  WorkerThread w;
  for ( int i = 0; i < _max; i++ ) {
     worker = (Worker)_workerClass.newInstance();
     w = new WorkerThread (Worker#+i, worker);
     w.start();
     _waiting.push(w);
   }
 }
```

Besides the run method, the WorkerThread class has a wake method. When work comes in, the wake method is called, which assigns the data and notifies the sleeping WorkerThread (the one initialized by the Pool) to resume running. The wake method's call to notify causes the blocked WorkerThread to fall out of its wait state, and the run method of the HttpServerWorker (page 383) class is executed. Once the work is done, the WorkerThread is either put back onto the Stack (assuming the thread pool is not full) or terminates.

```
synchronized void wake (Object data) {
  _data = data;
  notify();
}
synchronized public void run(){
  boolean stop = false;
  while (!stop){
    if ( _data == null ){
    try {
      wait();
    } catch (InterruptedException e){
      e.printStackTrace(); continue;
    }
    if( _data != null ){
      _worker.run(_data);
    }
    _data = null;
    stop = !(_waiting.push(this));
  }
}
```

At its highest level, incoming work is handled by the performWork method in the Pool class (shown below). As work comes in, an existing WorkerThread is popped off of the Stack (or a new one is created if the Pool is empty). The sleeping WorkerThread is then activated by a call to its wake method.

```
public void performWork (Object data)   throws InstantiationException{
  WorkerThread w = null;
  synchronized (_waiting){
    if( _waiting.empty() ){
      try {
        w = new WorkerThread (additional worker,
                              (Worker)_workerClass.newInstance());
        w.start();
      } catch (Exception e){
        throw new InstantiationException(
                  Problem creating instance of Worker.class: + e.getMessage());
      } else{
        w = (WorkerThread)_waiting.pop();
      }
    }
    w.wake (data);
 }
```

The HttpServer (page 384) class constructor creates a new Pool instance to service HttpServerWorker (page 383) instances. HttpServerWorker instances are created and stored as part of the WorkerThread data. When a WorkerThread is activated by a call to its wake method, the HttpServerWorker instance is invoked by way of its run method.

```
  try{
    _pool = new Pool (poolSize, HttpServerWorker.class);
```

```
  } catch (Exception e){
    e.printStackTrace();
    throw new InternalError (e.getMessage());
  }
```

This next code is in the run method of the HttpServer (page 384) class. Every time a request comes in, the data is initialized and the Thread starts work.

Note: If creating a new Hashtable for each WorkerThread presents too much overhead, just modify the code so it does not use the Worker abstraction.

```
try {
   Socket s = _serverSocket.accept();
  Hashtable data = new Hashtable();
   data.put (Socket, s);
   data.put (HttpServer, this);
   _pool.performWork (data);
} catch (Exception e){
  e.printStackTrace();
}
```

Thread pooling is an effective performance-tuning technique that puts the expensive thread startup process at the startup of an application. This way, the negative impact on performance occurs once at program startup where it is least likely to be noticed.

Connection Pooling

If you have used an SQL or other similar tool to connect to a database and act on the data, you probably know that getting the connection and logging in is the part that takes the most time. An application can easily spend several seconds every time it needs to establish a connection.

In releases prior to JDBC™ 2.0 every database session requires a new connection and login even if the previous connection and login used the same table and user account. If you are using a JDBC release prior to 2.0 and want to improve performance, you can cache JDBC connections instead of creating a new connection and login.

Cached connections are kept in a runtime object pool and can be used and reused as needed by the application. One way to implement the object pool is to make a simple hashtable of connection objects. However, a more flexible way to do it is to write a wrapper JDBC Driver that is an intermediary between the client application and database.

The wrapper approach works particularly well in an Enterprise Bean that uses Bean-managed persistence for two reasons: 1) Only one Driver class is loaded per Bean, and 2) spe-

cific connection details are handled outside the Bean. This section explains how to write a wrapper JDBC Driver class.

Wrapper Classes

The wrapper JDBC Driver created for this example consists of the following three classes:

- JDCConnectionDriver
- JDCConnectionPool
- JDCConnectio

Connection Driver

The JDCConnectionDriver (page 391) class implements the java.sql.Driver interface, which provides methods to load drivers and create new database connections. A JDCConnectionManager object is created by the application seeking a database connection. The application provides the database Uniform Resource Locator (URL) for the database, login user ID, and login password.

The JDCConnectionManager constructor does the following:

- Registers the JDCConnectionManager object with the DriverManager.
- Loads the Driver class passed to the constructor by the calling program.
- Initializes a JDCConnectionPool object for the connections with the database URL, login user ID, and login password passed to the constructor by the calling program.

```
public JDCConnectionDriver(String driver, string url, String user,
                           String password)
                           throws ClassNotFoundException,
                           InstantiationException,
                           IllegalAccessException, SQLException {

  DriverManager.registerDriver(this);
  Class.forName(driver).newInstance();
  pool = new JDCConnectionPool(url, user, password);
}
```

When the calling program needs a database connection, it calls the JDCConnectionDriver.connect method, which in turn, calls the JDCConnectionPool.getConnection method.

Connection Pool

The JDCConnectionPool (page 391) class makes connections available to a calling program in its getConnection method. This method searches for an available connection in the connection pool. If no connection is available from the pool, a new connection is created. If a

connection is available from the pool, the getConnection method leases the connection and returns it to the calling program.

```
public synchronized Connection getConnection() throws SQLException {

  JDCConnection c;
  for(int i = 0; i < connections.size(); i++) {
     c = (JDCConnection)connections.elementAt(i);
     if (c.lease()) {
        return c;
     }
  }

  Connection conn = DriverManager.getConnection(url, user, password);
  c = new JDCConnection(conn, this);
  c.lease();
  connections.addElement(c);
  return c;
}
```

The JDCConnection (page 393) class represents a JDBC connection in the connection pool, and is essentially a wrapper around a real JDBC connection. The JDCConnection object maintains a state flag to indicate if the connection is in use and the time the connection was taken from the pool. This time is used by the ConnectionReaper class to identify hanging connections.

Deadlocks and Hangs

While many client and server databases have graceful ways to handle deadlocks and hangs so you do not have to write code to handle these situations, many of the newer, lightweight distributed databases are not so well equipped. The connection pool class provides a dead connection reaper to handle these situations.

The ConnectionReaper class in the JDCConnectionPool (page 391) class file decides a connection is dead if the following conditions are met.

- The connection is flagged as being in use.
- The connection is older than a preset connection time out.
- The connection fails a validation check.

The validation check runs a simple SQL query over the connection to see if it throws an exception. In this example, the validation method requests the high-level description of the database tables. If a connection fails the validation test, it is closed, a new connection is initiated to the database, and added to the connection pool.

```
public boolean validate() {
  try {
     conn.getMetaData();
  }catch (Exception e) {
     return false;
  }
  return true;
}
```

Closing Connections

The connection is returned to the connection pool when the calling program calls the JDCConnection.close method in its finally clause.

```
public void close() throws SQLException {
  pool.returnConnection(this);
}
```

Example Application

You use a connection pool in an application in a similar way to how you would use any other JDBC driver. Here is the code for a Bean-managed RegistrationBean (page 395). This RegistrationBean is adapted from the auction house Enterprise JavaBeans™ example described in Chapters 1 - 3.

When the first RegistrationBean object is created, it creates one static instance of the JDCConnectionDriver class. This static driver object registers itself with the DriverManager in the JDCConnectionDriver constructor making it available for connection requests to all RegistrationBean objects created by the client application.

Passing the URL as jdbc:jdc:jdcpool in the getConnection method, lets the DriverManager match the getConnection request to the registered driver. The DriverManager uses simple String matching to find an available driver that can handle URLs in that format.

```
public class RegistrationBean implements EntityBean{

  private transient EntityContext ctx;
  public String theuser, password;
  public Stringcreditcard, emailaddress;
  public double balance;

//Static class instantiation
  static {
    try {
      new pool.JDCConnectionDriver("COM.cloudscape.core.JDBCDriver",
                                   "none", "none");
    } catch(Exception e){}
  }
```

```
  public Connection getConnection() throws SQLException{
    return DriverManager.getConnection(jdbc:jdc:jdcpool");
  }
}
```

Performance Features and Tools

The new Java virtual machines have features to increase performance, and you can use a number of tools to increase application performance or reduce the size of generated class files. Such features and tools improve the performance of your application with little or no change required to your application.

Java Virtual Machine Features

The Java 2 Platform release has introduced many performance improvements over previous releases, including faster memory allocation, reduction of class sizes, improved garbage collection, streamlined monitors and a built-in Just-In-Time compiler (JIT) as standard.

When using the new Java 2 virtual machine straight out of the box, you will see an improvement; however, by understanding how the speed-ups work you can tune your application to squeeze out every last bit of performance.

Method Inlining

The Java 2 release of the Java virtual machine automatically inlines simple methods at runtime. In an un-optimized Java virtual machine, every time a new method is called, a new stack frame is created. The creation of a new stack frame requires additional resources as well as some remapping of the stack. The end result is that creating new stack frames incurs a small overhead.

Method inlining increases performance by reducing the number of method calls your program makes. The Java virtual machine inlining code inlines methods that return constants or only access internal fields. To take advantage of method inlining you can do one of two things: You can either make a method look attractive to the virtual machine to inline, or manually inline a method if it does not break your object model. Manual inlining in this context means moving the code from a method into the method that is calling it. Automatic virtual machine inlining is illustrated in this next example:

```
public class InlineMe {
  int counter=0;

  public void method1() {
    for(int i=0;i<1000;i++) {
      addCount();
      System.out.println("counter="+counter);
```

```
  }

  public int addCount() {
    counter=counter+1;
    return counter;
  }

  public static void main(String args[]) {
    InlineMe im=new InlineMe();
    im.method1();
  }
}
```

In its current state the addCount method does not look very attractive to the inline detector in the virtual machine because the addCount method returns a value. To find out if this method is inlined, run the compiled example with profiling enabled:

```
java -Xrunhprof:cpu=times InlineMe
```

This generates a java.hprof.txt output file. The top ten methods will look similar to this:

```
CPU TIME (ms) BEGIN (total = 510)   Thu Jan 28 16:56:15 1999
rank self accum  count trace method
  1  5.88%  5.88%    1  25 java/lang/Character.  <clinit>
  2  3.92%  9.80% 5808  13 java/lang/String.charAt
  3  3.92% 13.73%    1  33 sun/misc/Launcher$AppClassLoader.getPermissions
  4  3.92% 17.65%    3  31 sun/misc/URLClassPath.getLoader
  5  1.96% 19.61%    1  39 java/net/URLClassLoader.access$1
  6  1.96% 21.57% 1000  46 InlineMe.addCount
  7  1.96% 23.53%    1  21 sun/io/Converters.newConverter
  8  1.96% 25.49%    1  17 sun/misc/Launcher$ExtClassLoader.getExtDirs
  9  1.96% 27.45%    1  49 java/util/Stack.peek
 10  1.96% 29.41%    1  24 sun/misc/Launcher.<init>
```

If you change the addCount method to no longer return a value, the virtual machine will inline it for you at runtime. To make the code friendly to inlining, replace the addCount method with the following code:

```
public void addCount() {
  counter=counter+1;
}
```

And run the profiler again:

```
java -Xrunhprof:cpu=times InlineMe
```

This time the java.hprof.txt output should look different. The addCount method is gone because it has been inlined!

```
CPU TIME (ms) BEGIN (total = 560)  Thu Jan 28 16:57:02 1999
rank self  accum  count trace method
  1  5.36%  5.36%    1  27 java/lang/Character.<clinit>
  2  3.57%  8.93%    1  23 java/lang/System.initializeSystemClass
  3  3.57% 12.50%    2  47 java/io/PrintStream.<init>
  4  3.57% 16.07% 5808  15 java/lang/String.charAt
  5  3.57% 19.64%    1  42 sun/net/www/protocol/file/Handler.openConnection
  6  1.79% 21.43%    2  21 java/io/InputStreamReader.fill
  7  1.79% 23.21%    1  54 java/lang/Thread.<init>
  8  1.79% 25.00%    1  39 java/io/PrintStream.write
  9  1.79% 26.79%    1  40 java/util/jar/JarFile.getJarEntry
 10  1.79% 28.57%    1  38 java/lang/Class.forName0
```

Streamlined synchronization

Up until Java 2 synchronized methods and objects have always incurred an additional performance hit. This is because the mechanism used to implement the locking used a global monitor registry, which was only single-threaded in some areas such as when searching for existing monitors.

In the Java 2 release, each thread has a monitor registry and so many of the existing bottlenecks have been removed. If you have previously used other locking mechanisms because of the performance hit with synchronized methods, it is now worthwhile to revisit this code and incorporate the new Java 2 streamlined locks.

This next example creates monitors for the synchronized block and achieves a 40 percent increase in speed. Time was 14ms using JDK 1.1.7 and only 10ms with Java 2 on a Sun Ultra 1.

```
class MyLock {
  static Integer count=new Integer(5);
  int test=0;

  public void letslock() {
    synchronized(count) {
      test++;
    }
  }
}

public class LockTest {
  public static void main(String args[]) {
    MyLock ml=new MyLock();
    long time = System.currentTimeMillis();
    for(int i=0;i<5000;i++ ) {
       ml.letslock();
    }
    System.out.println("Time taken=" + (System.currentTimeMillis()-time));
  }
}
```

Java Hotspot Performance Engine

The Java HotSpot™ Performance Engine is Sun Microsystem's next-generation virtual machine implementation. HotSpot adheres to the same specification as the Java 2 virtual machine, and runs the same byte codes, but it has been re-engineered to leverage new technologies like adaptive optimization and improved garbage collection models to dramatically improve the speed of the Java virtual machine.

Adaptive optimization

Hotspot does not include a plug-in JIT compiler, but instead compiles and inlines methods it determines as being the most used in the application. This means that on the first pass through, Java bytecodes are interpreted as if you did not have a JIT compiler present. If the code then appears to be a hot spot in your application, the Hotspot compiler compiles the bytecodes into native code that is stored in a cache and inlines methods at the same time. See Method Inlining (page 348) for details on the advantages to inlining code.

One advantage to selective compilation over a JIT compiler is the byte compiler can spend more time generating highly optimized code for the areas that would benefit from the optimization most. The compiler can also avoid compiling code that may best run in interpreted mode.

Earlier versions of HotSpot did not optimize code not currently in use. The downside to this is if the application is in a big busy loop, the optimizer is unable to compile the code for that area until the loop finishes. Later Hotspot releases use on-stack replacement, meaning that code can be compiled into native code even if it is in use by the interpreter.

Improved Garbage Collection

The garbage collector used in HotSpot introduces several improvements over existing garbage collectors. The first is the garbage collector is termed a fully accurate collector. What this means is the garbage collector knows exactly what is an object reference and what is just data. HotSpot uses direct references to objects on the heap instead of object handles. This design means memory fragmentation can be reduced and results in a more compact memory footprint.

The second improvement is in the use of generational copying. The Java platform creates a large number of objects on the heap and often those objects are short lived. By placing newly created objects in a memory bucket, waiting for the bucket to fill up, and only copying the remaining live objects to a new area, the block of memory the bucket used can be freed in one block. This means the virtual machine does not have to search for a hole to fit each new object in the heap, and smaller sections of memory must be manipulated at one time.

For older objects the garbage collector sweeps through the heap and compacts holes from dead objects directly, which removes the need for a free list used in earlier garbage collection algorithms.

The third area of improvement is to remove the perception of garbage collection pauses by staggering the compaction of large free object spaces into smaller groups and compacting them incrementally.

Fast Thread Synchronization

HotSpot also improves existing synchronized code. Synchronized methods and code blocks have always had a performance overhead when run in a Java virtual machine. HotSpot implements the monitor entry and exit synchronization points itself and does not depend on the local operating system to provide this synchronization. This results in a large speed improvement especially to often heavily synchronized GUI applications.

Just-In-Time Compilers

The simplest tool used to increase the performance of your application is the Just-In-Time (JIT) compiler. A JIT is a code generator that converts Java bytecodes into native machine code. Java programs invoked with a JIT generally run much faster than when the bytecode is executed by the interpreter. Hotspot removes the need for a JIT compiler in most cases; however, you might still find the JIT compiler being used in earlier releases.

The JIT compiler was first made available as a performance update in the Java Development Kit (JDK™) 1.1.6 software release and is now a standard tool invoked whenever you use the java interpreter command in the Java 2 platform release. You can disable the JIT compiler with the -Djava.compiler=NONE option to the Java virtual machine. This is covered in more detail at the end of the JIT section.

How do JIT Compilers work?

JIT compilers are supplied as standalone platform-dependent native libraries. If the JIT Compiler library exists, the Java virtual machine initializes Java Native Interface (JNI) native code hooks to call JIT functions available in that library instead of the equivalent function in the interpreter.

The java.lang.Compiler class loads the native library and starts the initialization inside the JIT compiler. When the Java virtual machine invokes a Java method, it uses an invoker method as specified in the method block of the loaded class object. The Java virtual machine has several invoker methods, for example, a different invoker is used if the method is synchronized or if it is a native method.

The JIT compiler uses its own invoker. Sun production releases check the method access bit for value ACC_MACHINE_COMPILED to notify the interpreter that the code for this method has already been compiled and stored in the loaded class.

When does the code become JIT compiled code?

When a method is called the first time, the JIT compiler compiles the method block into native code for this method and stores it in the code block for the method. Once the code is compiled, the ACC_MACHINE_COMPILED bit, which is used on the Sun platform, is set.

How can I see what the JIT compiler is doing?

The _JIT_ARGS environment variable allows simple control of the Sun Solaris JIT compiler. Two useful values are trace and exclude(list). To exclude the methods from the InlineMe example and show a trace set _JIT_ARGS as follows:

Unix:

```
export _JIT_ARGS="trace exclude(InlineMe.addCount  InlineMe.method1)"
$ java InlineMe
Initializing the JIT library ...
DYNAMICALLY COMPILING java/lang/System.getProperty
mb=0x63e74
DYNAMICALLY COMPILING java/util/Properties.getProperty
mb=0x6de74
DYNAMICALLY COMPILING java/util/Hashtable.get
mb=0x714ec DYNAMICALLY COMPILING java/lang/String.hashCode
mb=0x44aec DYNAMICALLY COMPILING java/lang/String.equals
mb=0x447f8 DYNAMICALLY COMPILING java/lang/String.valueOf
mb=0x454c4 DYNAMICALLY COMPILING java/lang/String.toString
mb=0x451d0 DYNAMICALLY COMPILING java/lang/StringBuffer.<init>
mb=0x7d690
<<<< Inlined java/lang/String.length (4)
```

Notice that inlined methods such as String.length are exempt. The String.length is also a special method because it is normally compiled into an internal shortcut bytecode by the Java Interpreter. When using the JIT compiler these optimizations provided by the Java Interpreter are disabled to enable the JIT compiler to understand which method is being called.

How to use the JIT to your advantage

The first thing to remember is that the JIT compiler achieves most of its speed improvements the second time it calls a method. The JIT compiler compiles the whole method instead of interpreting it line by line, which can also be a performance gain when running an application with the JIT enabled. This means that if code is only called once, you will not see a significant performance gain. The JIT compiler also ignores class constructors, so if possible, keep constructor code to a minimum.

The JIT compiler also achieves a minor performance gain by not prechecking certain boundary conditions in the Java programming language such as null pointer or array out of bounds exceptions. The only way the JIT compiler knows it has a null pointer exception is by a signal raised by the operating system. Because the signal comes from the operating system and not the Java virtual machine, your program takes a performance hit. To ensure the best performance when running an application with the JIT, make sure your code is very clean with no errors like null pointer or array out of bounds exceptions.

You might want to disable the JIT compiler if you are running the Java virtual machine in remote debug mode, or if you want to see source line numbers instead of the label (Compiled Code) in your Java stack traces. To disable the JIT compiler, supply a blank or invalid name for the name of the JIT compiler when you invoke the interpreter command. The following examples show the javac command to compile the source code into bytecodes, and two forms of the java interpreter command to invoke the interpreter without the JIT compiler.

```
javac MyClass.java
java -Djava.compiler=NONE MyClass
```

or

```
javac MyClass.java
java -Djava.compiler="" MyClass
```

Third-Party Tools

Some of the other tools available include those that reduce the size of the generated Java class files. A Java class file contains an area called a constant pool. The constant pool keeps a list of strings and other information for the class file in one place for reference. One of the pieces of information available in the constant pool are the method and field name.

The class file refers to a field in the class as a reference to an entry in the constant pool. This means as long as the references stay the same, it does not matter what the values stored in the constant pool are. This knowledge is exploited by several tools that rewrite the names of the field and methods in the constant pool into shortened names. This technique can reduce the class file by a significant percentage with the benefit that a smaller class file means a shorter network download.

Performance Analysis

Another way to improve performance is with performance analysis. Performance analysis is looking at program execution to pinpoint where bottlenecks or other performance problems

such as memory leaks might occur. Once you know where potential trouble spots are, you can change your code to remove or reduce their impact.

Profiling

Java virtual machines have been able to provide simple profile reports since the introduction of Java Developer Kit (JDK) 1.0.2. However, the information provided is limited to a sorted list of methods called by a program.

The Java® 2 platform software provides much better profiling capabilities than previously available, and analysis of this generated data is made easier by the Heap Analysis Tool (HAT). The heap analysis tool, as its name implies, lets you analyze heap profile reports. The heap is a block of memory the Java virtual machine uses at run time. You can locate the Heap Analysis Tool using the search engine on the `http://java.sun.com` web site.

The heap analysis tool lets you generate reports on objects that were used to run your application. Not only can you get a listing of the most frequently called methods and the memory used in calling those methods, but you can also track down memory leaks. Memory leaks can have a significant impact on performance.

Analyze a Program

This section shows you how to analyze the TableExample3 program included in the demo/jfc/Table directory in the Java 2 platform download. To do this, you need to generate a profile report. The simplest report to generate is a text profile.

To generate a text profile, run the application with the -Xhprof parameter. In the final release of the Java 2 platform software, this option was renamed -Xrunhprof. To see a list of the currently available options run the `java` interpreter command as follows:

```
java -Xrunhprof:help
Hprof usage: -Xrunhprof[:help]|[<option>=<value>, ...]
Option Name and Value     Description               Default
---------------------     -----------               --------
heap=dump|sites|all       heap profiling            all
cpu=samples|times|old     CPU usage                 off
monitor=y|n               monitor contention        n
format=a|b                ascii or binary output    a
file=<file>               write data to file        java.hprof(.txt for ascii)
net=<host>:<port>         send data over a socket   write to file
depth=<size>              stack trace depth         4
cutoff=<value>            output cutoff poin        0.0001
lineno=y|n                line number in traces     y
thread=y|n                thread in traces?         n
doe=y|n                   dump on exit?             y
Example: java -Xrunhprof:cpu=samples,file=log.txt, depth=3 FooClass
```

The following invocation creates a text output file that you can view without the java.hprof.txt heap analysis tool, which is called when the program generates a stack trace or exits. A different invocation is used to create a binary file to use with the heap analysis tool.

```
java -Xrunhprof TableExample3

d:\jdk12\demo\jfc\Table> java -Xrunhprof TableExample3
Dumping Java heap ... allocation sites ... done.
```

The profile option literally logs every object created on the heap, so even just starting and stopping the small TableExample3 program results in a four megabyte report file. Although the heap analysis tool uses a binary version of this file and provides a summary, there are some quick and easy things you can learn from the text file without using the heap analysis tool.

View the Text File

Choose an editor that can handle large files and go to the end of this file. There could be hundreds of thousands of lines, so use a shortcut instead of scrolling, or search for the words SITES BEGIN. You should see a list of lines that start with an increasing rank number followed by two percentage numbers. The first entry in this list, should look similar to the example below:

```
SITES BEGIN (ordered by live bytes)  Sun Dec 20 16:33:28 1998
      percent              live          alloc'ed    stack    class
rank   self   accum    bytes  objs    bytes   objs    trace    name
1    55.86%  55.86%   826516   5     826516    5      3981     [S
```

The [S notation at the end of the last line above indicates the first entry is an array of short, which is a primitive type. This notation is expected with Project Swing or Abstract Window Toolkit (AWT) applications.

The five count under the objs header means there are currently five of these arrays, there have only been five in the lifetime of this application, and they take up 826516 bytes.

The reference key to this object is the value listed under stack trace. To find where this object was created in this example, search for TRACE 3981. You will see the following:

```
TRACE 3981:
java/awt/image/DataBufferUShort.<init>(DataBufferUShort.java:50)
java/awt/image/Raster.createPackedRaster(Raster.java:400)
java/awt/image/DirectColorModel.createCompatibleWritableRaster(
                                      DirectColorModel.java:641)
sun/awt/windows/WComponentPeer.createImage(WComponentPeer.java:186)
```

The TableExample3 code sets a scrollpane that is 700 by 300. When you look at the source of Raster.java, which is in the src.jar file, you find these statements at line 400:

```
case DataBuffer.TYPE_USHORT:d = new DataBufferUShort(w*h);
break;
```

The values w and h above are the width and height from the createImage call at the start of TRACE 3981. The DataBufferUShort constructor creates and array of shorts as follows:

```
data = new short[size];
```

where size is w*h. So, in theory there should be an entry for an array of 210000 elements. You look for each instantiation of this class by searching for trace=3981. One of the five entries will look like this:

```
OBJ 5ca1fc0 (sz=28, trace=3979,class=java/awt/image/DataBufferUShort@9a2570)
data 5ca1670
bankdata 5ca1f90
offsets 5ca1340
ARR 5ca1340 (sz=4, trace=3980, nelems=1, elem type=int)
ARR 5ca1670 (sz=420004, trace=3981, nelems=210000,elem type=short)
ARR 5ca1f90 (sz=12, trace=3982, nelems=1, elem type=[S@9a2d90)
[0] 5ca1670
```

You can see that the data value of this raster image references an array 5ca1670 which in turns lists 210000 elements of a short of size 2. This means 420004 bytes of memory are used in this array.

From this data you can conclude that the TableExample3 program uses nearly 0.5Mb to map each table. If the example application is running on a small memory machine, you should make sure you do not keep unnecessary references to large tables or images that are built by the createImage method.

The Heap Analysis Tool

The Heap Analysis Tool can analyze the same data for you, but requires a binary report file as input. You can generate a binary report file as follows:

```
java -Xrunhprof:file=TableExample3.hprof,format=b TableExample3
```

To generate the binary report, close the TableExample3 window. The binary report file TableExample3.hprof is created when the program exits. The Heap Analysis tool starts an HTTP Server that analyzes the binary profile file and displays the results in HTML that you can view with a browser.

You can get a copy of the Heap Analysis Tool from the java.sun.com web site. Once you install it, you can run shell and batch scripts in the installed bin directory to start the Heap Analysis Tool server as follows:

```
>hat TableExample3.hprof
```

```
Started HCODEP server on port 7000
Reading from /tmp/TableExample3.hprof...
Dump file created Tue Jan 05 13:28:59 PST 1999
Snapshot read, resolving...
Resolving 17854 objects...
Chasing references, expect 35 dots.....................
Eliminating duplicate references.....................
Snapshot resolved.
Server is ready.
```

The above output tells you an HTTP server is started on port 7000 by default. To view this report enter the url http://localhost:7000 or http://your_machine_name:7000 in your web browser. If you have problems starting the server using the scripts, you can alternately run the application by including the hat.zip classes file on your CLASSPATH and use the following command:

```
java hat.Main TableExample3.hprof
```

The default report view contains a list of all the classes. At the bottom of this initial page are the following two key report options:

```
Show all members of the rootset
Show instance counts for all classes
```

If you select the Show all members of the rootset link, you see a list of the following references because these reference are likely targets for potential memory leaks.

```
Java Static References
Busy Monitor References
JNI Global References
JNI Local References
System Class References
```

What you look for here are instances in the application that have references to objects that have a risk of not being garbage collected. This can sometimes occur in the case of JNI if memory is allocated for an object, the memory is left to the garbage collector to free up, and the garbage collector does not have the information it needs to do it. In this list of references, you are mainly interested in a large number of references to objects or objects of a large size.

The other key report is the Show instance counts for all classes. This lists the number of calls to a particular method. The String and Character array objects, [S and [C, are always going to be high on this list, but some objects are a bit more intriguing. Why are there 323 instances of java.util.SimpleTimeZone for example?

```
5109 instances of class java.lang.String
5095 instances of class [C
2210 instances of class java.util.Hashtable$Entry
968 instances of class java.lang.Class
407 instances of class [Ljava.lang.String;
```

```
323 instances of class java.util.SimpleTimeZone
305 instances of class sun.java2d.loops.GraphicsPrimitiveProxy
304 instances of class java.util.HashMap$Entry
269 instances of class [I
182 instances of class [Ljava.util.Hashtable$Entry;
170 instances of class java.util.Hashtable
138 instances of class java.util.jar.Attributes$Name
131 instances of class java.util.HashMap
131 instances of class [Ljava.util.HashMap$Entry;
130 instances of class [Ljava.lang.Object;
105 instances of class java.util.jar.Attributes
```

To get more information on the SimpleTimeZone instances, click on the link (the line beginning with 323). This will list all 323 references and calculate how much memory has been used. In this example, 21964 bytes have been used.

```
Instances of java.util.SimpleTimeZone

class java.util.SimpleTimeZone

java.util.SimpleTimeZone@0x004f48c0 (68 bytes)
java.util.SimpleTimeZone@0x003d5ad8 (68 bytes)
java.util.SimpleTimeZone@0x004fae88 (68 bytes)
.....
Total of 323 instances occupying 21964 bytes.
```

If you click on one of these SimpleTimeZone instances, you see where this object was allocated.

```
Object allocated from:

java.util.TimeZoneData.<clinit>(()V) : TimeZone.java line 1222
java.util.TimeZone.getTimeZone((Ljava/lang/String;)
             Ljava/util/TimeZone;) : TimeZone.java line (compiled method)
java.util.TimeZone.getDefault(()Ljava/util/TimeZone;) :
            TimeZone.java line (compiled method)
java.text.SimpleDateFormat.initialize((Ljava/util/Locale;)V) :
             SimpleDateFormat.java line (compiled method)
```

In this example the object was allocated from TimeZone.java. The source to this file is in the standard src.jar file, and on examining this file, you can see that indeed there are nearly 300 of these objects in memory.

```
static SimpleTimeZone zones[] = {
// Total Unix zones: 343
// Total Java zones: 289
// Not all Unix zones become Java zones due to
// duplication and overlap.
//----------------------------------------
new SimpleTimeZone(-11*ONE_HOUR,  "Pacific/Niue" /*NUT*/),
```

Unfortunately, you have no control over the memory used in this example because it is allocated when the program first requests a default time zone. However, this same technique can be applied to analyzing your own application where you may be able to make some improvements

Where the Application Spends its Time

Again, you can use the -Xrunhprof parameter to get information about the time the application spent processing a particular method.

You can use one of two CPU profiling options to achieve this. The first option is cpu=samples. This option reports the result of a sampling of the running threads of the Java virtual machine to which a statistical count of the frequency of the occurrence of a particular method is used to find busy sections of the applications. The second option is cpu=times, which measures the time taken by individual methods and generates a sorted list ranked as a total percentage of the CPU time taken by the application. By using the cpu=times option, you should see something similar to this at the end of the output file:

```
CPU TIME (ms) BEGIN (total = 11080) Fri Jan  8 16:40:59 1999
rank self  accum count trace   method
1  13.81%  13.81%  1  437  sun/awt/X11GraphicsEnvironment.initDisplay
2  2.35%   16.16%  4  456  java/lang/ClassLoader$NativeLibrary.load
3  0.99%    17.15%  46  401  java/lang/ClassLoader.findBootstrapClass
```

If you contrast this with the cpu=samples output, you see the difference between how often a method appears during the runtime of the application in the samples output compared to how long that method took in the times output.

```
CPU SAMPLES BEGIN (total = 14520) Sat Jan 09 17:14:47 1999
rank self  accum count trace  method
1  2.93%  2.93%  425 2532 sun/awt/windows/WGraphics.W32LockViewResources
2  1.63%  4.56%  237  763 sun/awt/windows/WToolkit.eventLoop
3  1.35%   5.91%   96  1347 java/text/DecimalFormat.<init>
```

The W32LockView method, which calls a native windows lock routine, is called 425 times. So when it is sampled, it appears in the active runnings threads because 425 calls take time to complete. However although the initDisplay method is called only once, it takes the longest time to complete in real time.

Operating System Performance Tools

Sometimes the performance bottleneck occurs at the system or operating system level. This is because Java virtual machine depends on many operating system libraries for functionality

such as disk access or networking. However, what occurs in these libraries after the Java virtual machine calls them is beyond the reach of most profiling tools for the Java platform.

The next sections describe tools you can use to analyze performance problems on some common operating systems.

Solaris Platform Tools

- sar
- truss

The System Accounting Reports tool, sar, reports the activity of the system in terms of disk IO, user program activity, and system level activity. If your application is using excessive amounts of memory, it might require disk swap space, which shows up as high percentage values in the WIO column. User programs that get stuck in a busy loop show a high percentage in the user column:

```
developer$ sar 1 10

SunOS developer 5.6 Generic_105181-09 sun4u  02/05/99

11:20:29    %usr    %sys    %wio   %idle
11:20:30      30       6       9      55
11:20:31      27       0       3      70
11:20:32      25       1       1      73
11:20:33      25       1       0      74
11:20:34      27       0       1      72
```

The truss command traces and records the details of every system call called by the Java virtual machine to the Solaris kernel. A common way to run truss is:

```
truss -f -o /tmp/output -p <process id>
```

The -f parameter follows any child processes that are created, the -o parameter writes the output to the named file, and the -p parameter traces an already running program from its process ID. Alternately, you can replace -p <process id> with the Java virtual machine, for example:

```
truss -f -o /tmp/output java MyDaemon
```

The /tmp/output is used to store the truss output, which should look similar to the following:

```
15573:  execve("/usr/local/java/jdk1.2/solaris/bin/java", 0xEFFFF2DC,
        0xEFFFF2E8)  argc  = 4
15573:  open("/dev/zero", O_RDONLY) = 3
15573:  mmap(0x00000000, 8192, PROT_READ|PROT_WRITE|PROT_EXEC,
        MAP_PRIVATE, 3, 0) = 0xEF7C0000
15573: open("/home/calvin/java/native4/libsocket.so.1",  O_RDONLY)
       Err#2 ENOENT
```

```
15573: open("/usr/lib/libsocket.so.1", O_RDONLY) = 4
15573: fstat(4, 0xEFFFEF6C)                          = 0
15573: mmap(0x00000000, 8192, PROT_READ|PROT_EXEC,
       MAP_SHARED, 4, 0) = 0xEF7B00 00
15573: mmap(0x00000000, 122880, PROT_READ|PROT_EXEC,
       MAP_PRIVATE, 4, 0) = 0xEF7 80000
15573: munmap(0xEF78E000, 57344) = 0
15573: mmap(0xEF79C000, 5393, PROT_READ|PROT_WRITE|PROT_EXEC,
       MAP_PRIVATE|MAP_FIXED, 4, 49152) = 0xEF79C000
15573:  close(4) = 0
```

In the truss output, look for files that fail when opened due to access problems, such as error ENOPERM, or a missing file error ENOENT. You can also track data read or written with the truss parameters -rall to log all data read, or -wall to log all data written by the program. With these parameters, it is possible to analyze data sent over a network or to a local disk.

Linux Platform

Linux has a trace command called strace. It traces system calls to the underlying Linux kernel. This example traces the SpreadSheet example in the JDK demo directory.

```
$ strace -f -o /tmp/output java sun.applet.AppletViewer example1.html
$ cat /tmp/output

639   execve("/root/java/jdk117_v1at/java/jdk117_v1a/bin/java"
    , ["java","sun.applet.AppletViewer ", "example1.html"], [/* 21 vars */]) = 0
639   brk(0) = 0x809355c
639   open("/etc/ld.so.preload", O_RDONLY)  = -1 ENOENT
      (No such file or directory)
639   open("/etc/ld.so.cache", O_RDONLY) = 4
639   fstat(4, {st_mode=0, st_size=0, ...}) = 0
639   mmap(0, 14773, PROT_READ, MAP_PRIVATE, 4, 0) = 0x4000b000
639   close(4)                                    = 0
639   open("/lib/libtermcap.so.2", O_RDONLY)      = 4
639   mmap(0, 4096, PROT_READ, MAP_PRIVATE, 4, 0) = 0x4000f000
```

To obtain system information similar to the Solaris sar command, read the contents of the file /proc/stat. The format of this file is described in the proc man page. Look at the cpu line to get the user and system time.

```
cpu  4827 4 1636 168329
```

In the above example, the cpu line indicates 48.27 seconds in user space, 0.04 at nice priority, 16.36 seconds processing system calls, and 168 seconds idle. This is a running total; individual entries for each process are available in /proc/<process_id>/stat.

Windows95/98/NT Platforms

There are no standard performance analysis tools included on this platform, but the following tools are available by way of freeware or shareware resources such as `http://www.download.com`.

- Runtime memory analysis: Memory meter
- Network Analysis:Ttraceplus

Caching Client/Server Applications

Caching is one of the first techniques used to improve the performance of web browsers and web servers. The browser cache makes network lookup operations unnecessary because a recent copy of the file is kept in the local cache, and the web server cache reduces the cost of loading the file from disk for each request. This section explains how you can use caching in a similar way to improve performance in many client/server applications written in the Java programming language.

The java.util.Collections API available in the Java® 2 Software Development Kit (SDK) software makes implementing a cache simple. This API provides the HashMap class, which works well for caching one object, and the LinkedList class, which works well in combination with the HashMap class for caching many objects.

Caching One Object

A HashMap object stores data in key and value pairs. When you put a data value in the HashMap, you assign it a key and later use that key to retrieve the data.

A HashMap object is very similar to a Hashtable and can be used to keep a temporary copy of previously generated results. Objects kept in the HashMap cache could, for example, be a list of completed auction results.

In this case, the results of a JDBC query might be requested hundreds of times a second by persons wanting to know who was the highest bidder, but the completed results lists only actual changes once a minute as each auction completes. You can write your program to retrieve unchanged objects from the results cache instead of querying the database every time and gain a significant performance improvement.

This code example runs a database query once a minute, and returns cached copies for requests that come between the queries.

```
// File DBCache.java
import java.util.*;
```

```
import java.io.*;

class DBCacheRecord {
  Object data;
  long time;

  public DBCacheRecord(Object results, long when) {
    time=when;
    data=results;
  }
  public Object getResults() {
    return data;
  }
  public long getLastModified() {
     return time;
  }
}

public class DBCache {
  Map cache;

  public DBCache() {
    cache = new HashMap();
  }

  public Object getDBData(String dbcommand) {
    if(!cache.containsKey(dbcommand)) {
       synchronized(cache) {
        cache.put(dbcommand, readDBData(dbcommand));
      }
    } else {
      if((new Date().getTime() ) -((DBCacheRecord)cache.get(
                       dbcommand)).getLastModified()>=1000){
        synchronized(cache) {
         cache.put(dbcommand, readDBData(dbcommand));
        }
      }
   }
   return ((DBCacheRecord)cache.get(dbcommand)).getResults();
  }

  public Object readDBData(String dbcommand) {
  /*Insert your JDBC code here For Example:
    ResultSet results=stmt.executeQuery(dbcommand);
  */
    String results="example results";
    return(new DBCacheRecord(results,new Date().getTime()));
  }
  public static void main(String args[]) {
    DBCache d1=new DBCache();
    for(int i=1;i<=20;i++) {
      d1.getDBData("select count(*) from results where TO_DATE(
```

```
                    results.completed) <=SYSDATE");
      }
    }
  }
```

Caching Many Objects

Sometimes you will want to cache more than one object. For example, you might want to keep the most recently accessed files on a web server in a cache. If you use a HashMap object for this, it will continue to grow and use a lot of memory.

If your machine has large amounts of memory and only a small number of objects to cache, a growing HashMap may not be a problem. However, if you intend to cache a lot of objects, you might find that keeping only the most recent objects in the cache provides the best use of the machine's memory. You can combine a HashMap object with a LinkedList to create what is called a Most Recently Used (MRU) cache.

Note: There are other techniques to constrain cache size besides MRU. MRU is one of the simpler algorithms.

With an MRU cache, you can place a constraint on which objects remain in cache, and thereby, control the size of the cache. There are three main operations that the MRU cache has to perform:

- If the cache is not full, new objects not already in the cache are inserted at the head of the list.
- If the cache is not full and the object to be inserted already exists in the cache, it is moved to the head of the list.
- If the cache is full and a new object is to be inserted, the last object in the cache is removed and the new object is inserted at the head of the list.

This diagram shows how the LinkedList and HashMap work together to implement the operations described above. A discussion of the diagram follows.

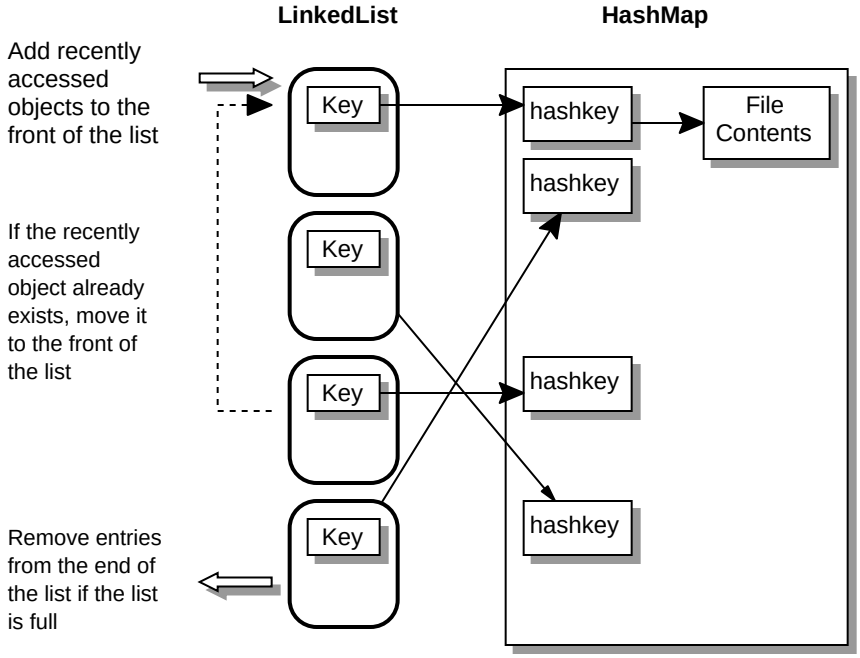


Figure 34 MRU Cache with LinkedList and HashMap

The LinkedList provides the queue mechanism, and the entries in the LinkedList contain the key to the data in the HashMap. To add a new entry to the front of the list, the addFirst method is called.

- If the list is already full, the removeLast method is called and the data entry is also removed from the HashMap.
- If an entry is already in the list, it is removed with a call to the remove method and inserted at the front of the list with a call to the addFirst method.

The Collections API does not implement locking, so if you remove entries from or add entries to LinkedList or HashMap objects, you need to lock access to these objects. You can also use a Vector or ArrayList to get the same results as shown in the code below with the LinkedList.

This code example uses an MRU cache to keep a cache of files loaded from disk. When a file is requested, the program checks to see if the file is in the cache. If the file is not in the cache, the program reads the file from disk and places the cache copy at the beginning of the list.

- If the file is in cache, the program compares the modification times of the file and cache entry.
- If the cache entry time is older, the program reads the file from disk, removes the cache copy, and places a new copy in the cache at the front of the LinkedList.
- If the file time is older, the program gets the file from the cache and moves the cache copy to the front of the list.

```
// File MRUCache.java
import java.util.*;
import java.io.*;

class myFile {
  long lastmodified;
  String contents;

  public myFile(long last, String data) {
    lastmodified=last;
    contents=data;
  }
  public long getLastModified() {
    return lastmodified;
  }
  public String getContents() {
    return contents;
  }
}

public class MRUCache {

  Map cache;
  LinkedList mrulist;
  int cachesize;

  public MRUCache(int max) {
    cache = new HashMap();
    mrulist= new LinkedList();
    cachesize=max;
  }

  public String getFile(String fname) {
//API Ref :boolean containsKey(Object key
   synchronized (cache) {
       if(!cache.containsKey(fname)) {
        if(mrulist.size() >=cachesize) {
//API Ref :Object remove(Object key)
          cache.remove(mrulist.getLast());
//API Ref :Object removeLast()
          mrulist.removeLast();
        }
//API Ref :Object put(Object key, Object value)
        cache.put(fname, readFile(fname));
```

```
//API Ref :void addFirst(Object value)
            mrulist.addFirst(fname);
        } else {
          if((new File(fname).lastModified())>
//API Ref :Object get(Object key)
            ((myFile)cache.get(fname)).getLastModified()) {
              cache.put(fname, readFile(fname));
           }
         boolean remove(Object value)
            mrulist.remove(fname);
            mrulist.addFirst(fname);
        }
        return ((myFile)cache.get(fname)).getContents();
      }
      public myFile readFile(String name) {
        File f = new File(name);
        StringBuffer filecontents= new StringBuffer();

        try {
          BufferedReader br=new BufferedReader(new FileReader(f));
          String line;
          while((line =br.readLine()) != null) {
            filecontents.append(line);
          }
        } catch (FileNotFoundException fnfe){
          return (null);
        } catch ( IOException ioe) {
           return (null);
        }
          return (new myFile(f.lastModified(),
           filecontents.toString()));
      }

      public void printList() {
        for(int i=0;i<mrulist.size();i++) {
          System.out.println("item "+i+"="+mrulist.get(i));
        }
      }

      public static void main(String args[]) {
        // Number of entries in MRU cache is set to 10
        MRUCache h1=new MRUCache(10);
        for(int i=1;i<=20;i++) {
          // files are stored in a subdirectory called data
          h1.getFile("data"+File.separatorChar+i);
        }
          h1.printList();
      }
    }
```

Code for this Chapter

- MyApplet (page 369)
- XPM Definition (page 375)
- Pool (page 380)
- Worker (page 382)
- HttpServerWorker (page 383)
- HttpServer (page 384)
- JDCConnectionDriver (page 391)
- JDCConnectionPool (page 391)
- JDCConnection (page 393)
- RegistrationBean (page 395)

MyApplet

```
import java.applet.*;
// import XPM Parser classes
import java.awt.*;
import javax.swing.*;

public class MyApplet extends Applet
{
  public void init() {
    JButton button;
    Toolkit kit = Toolkit.getDefaultToolkit();
    button = new JButton(new ImageIcon(kit.createImage(new
                                       XImageSource(reply))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(
                                        new XImageSource(post))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(
                                      new XImageSource(reload))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(
                                      new XImageSource(catchup))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(
                                     new XImageSource (back10))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(
                                      new XImageSource (reset))));
    add (button);
    button = new JButton(new ImageIcon(kit.createImage(new
                                     XImageSource(faq))));
    add (button);
```

```
  }
static final public String post[] = {"32 32 6 1",
"       c #C0C0C0C0C0C0",
".      c #FFFFFFFFFFFF",
"X      c #000000000000",
"o      c #FFFF99990000",
"O      c #333333333333",
"+      c #00009999FFFF",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
" .............................  ",
" .XXXXXX.............oooo..O ",
" ......................++++..O ",
" .XXXXXX.............++++..O ",
" ......................++++..O ",
" ..........................O ",
" ..........................O ",
" ..........................O ",
" ..........................O ",
" ..........................O ",
" ............XXXXXXXXXXX..O ",
" ..........................O ",
" ..........................O ",
" ............XXXXXXXXXXX..O ",
" ..........................O ",
" ..........................O ",
" ............XXXXXXXXXXX..O ",
" ..........................O ",
"  OOOOOOOOOOOOOOOOOOOOOOOOOO ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                               "};
static final public String reply[] = {"32 32 7 1",
"       c #C0C0C0C0C0C0",
".      c #FFFFFFFFFFFF",
"X      c #000000000000",
"o      c #FFFF99990000",
"O      c #333333333333",
"+      c #00009999FFFF",
"@      c #FFFF00000000",
"                                ",
"                                ",
"                                ",
" .............................  ",
" .XXXXXX.............oooo..O ",
```

```
" ......................++++..O ",
" .XXXXXX..............++++..O ",
" ......................++++..O ",
" ....@.......................O ",
" ....@.......................O ",
" ...@@@......................O ",
" ...@@@......................O ",
" ..@@@@@.....................O ",
" ..@@@@@........XXXXXXXXXXX..O ",
" ....@.......................O ",
" ....@.......................O ",
" ....@..........XXXXXXXXXXX..O ",
" ....@.......................O ",
" ....@.......................O ",
" ....@..........XXXXXXXXXXX..O ",
" ....@.......................O ",
"  OOO@OOOOOOOOOOOOOOOOOOOOOOO ",
"     @                         ",
"     @             @@@@@@@@@@  ",
"     @                 @       ",
"     @                 @       ",
"     @                 @       ",
"     @                 @       ",
"     @                 @       ",
"     @@@@@@@@@@@@@@@@@@@       ",
"                               ",
"                               "};
static final public String reload[] = {"32 32 4 1",
"       c #C0C0C0C0C0C0",
".      c #9999CCCCFFFF",
"X      c #000099999999",
"o      c #00009999FFFF",
"                               ",
"                ..             ",
"                ..X            ",
"            .......X   ooooooo  ",
"          .........X   oooooooX ",
"        ...........X   oooooooX ",
"       .....XXXXX..X   oooooooX ",
"      ....XXX    ..X   oooooooX ",
"     ....XX      ..X   oooooooX ",
"     ...XX        XX   oooooooX ",
"    ...XX               XXXXXXX ",
"    ..XX                        ",
"   ...X                         ",
"   ..XX                         ",
"   ..X                          ",
"   ..X                   .X     ",
"   ..X                  ...X    ",
"   ..X                 .....X   ",
"   ..X                .......X  ",
"   ...               .........X ",
"    ..X                ...XXXXX ",
```

```
"   ...                  ...X     ",
"    ...               ...XX     ",
"    ....             ....X      ",
"     ....           ....XX      ",
"      .....       .....XX       ",
"       ..............XX         ",
"        X...........XXX         ",
"          X.......XXX           ",
"            XXXXXXX             ",
"                                ",
"                                "};
static final public String catchup[] = {"32 32 9 1",
"       c #C0C0C0C0C0C0",
".      c #000000000000",
"X      c #FFFFCCCC0000",
"o      c #00009999FFFF",
"O      c #333333333333",
"+      c #000099999999",
"@      c #999999999999",
"#      c #9999CCCCFFFF",
"$      c #00009999FFFF",
"                                ",
"                                ",
"                                ",
"                .               ",
"               ...              ",
"              XX.XX             ",
"              XXXXXXX           ",
"              oXXXXXoO          ",
"              o+XXX+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              o+o+o+oO          ",
"              @@@@@@@O          ",
"              #######O          ",
"              $$$$$$$O          ",
"              $$$$$$$O          ",
"              $$$$$$$O          ",
"              $$$$$$$O          ",
"               OOOOOOO          ",
"                                "};
```

```
static final public String back10[] = {"32 32 3 1",
"       c #C0C0C0C0C0C0",
".      c #9999CCCCFFFF",
"X      c #000099999999",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"             .           .     ",
"           ...X          ...X  ",
"         .....X        .....X  ",
"       .......X      .......X  ",
"     .........X    .........X  ",
"   ...........X  ...........X  ",
"  ............X ............X  ",
"   ...........X  ...........X  ",
"    X.........X   X.........X  ",
"      X.......X     X.......X  ",
"        X.....X       X.....X  ",
"          X...X         X...X  ",
"            X.X           X.X  ",
"              X             X  ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               ",
"                               "};
static final public String reset[] = {"32 32 5 1",
"       c #C0C0C0C0C0C0",
".      c #99999999FFFF",
"X      c #000099999999",
"o      c #9999CCCCFFFF",
"O      c #00009999FFFF",
"                               ",
"                               ",
"                               ",
"                               ",
"             .......           ",
"           ............        ",
"         ..............        ",
"       ......XXXXXXX......    ",
"      .....XXooooooXX.....     ",
"      ....XoooooooooooX....    ",
"     ....XooooooooooooooX....  ",
"    ....XoooooooooooooooX....  ",
```

```
"    ...XooooooooooooooooX...    ",
"    ...XooooooooooooooooX...    ",
"   ...XooooooooXXXoooooooooX...  ",
"   ...XooooooooX000XooooooooX...  ",
"   ...XoooooooX00000XoooooooX...  ",
"   ...XoooooooX00000XoooooooX...  ",
"   ...XoooooooX00000XoooooooX...  ",
"   ...XooooooooX000XooooooooX...  ",
"   ...XooooooooXXXooooooooooX...  ",
"    ...XooooooooooooooooX...    ",
"    ...XooooooooooooooooX...    ",
"    ....XoooooooooooooooX....    ",
"     ....XooooooooooooooX....     ",
"      ....XoooooooooooX....      ",
"      .....XXoooooooXX.....      ",
"       ......XXXXXXX......       ",
"         ................        ",
"          ..............         ",
"             .......             ",
"                                 ",
"                                 "};
  static String faq[] = {
/* width height ncolors cpp [x_hot y_hot] */
"32 32 12 1 0 0",
/* colors */
"       s none  m none  c none",
".      c #5D5D60606969",
"X      s iconGray8     m black c #212121212121",
"o      c #DCDCDEDEE5E5",
"O      s iconGray4     m white c #949494949494",
"+      s iconGray1     m white c #dedededededede",
"@      s iconColor2    m white c white",
"#      s iconGray2     m white c #bdbdbdbdbdbd",
"$      s iconGray5     m black c #737373737373",
"%      s iconColor1    m black c black",
"&      s iconGray7     m black c #424242424242",
"*      s iconGray6     m black c #636363636363",
/* pixels */
"                                 ",
"                                 ",
"                                 ",
"  ............................Xo",
"  .O+@@@@@@@@@@@@@@@@@@@@@@@@@+Xoo",
"  .#@@@@@@@@@@@@@@@@@@@@@@@@@@OXo ",
" .O+@@@@@@@@@@@@@@@@@@@@@@@@@+Xoo ",
" .#@@@@@@@@@@@@@@@@@@@@@@@@@@#$o  ",
" .@@@@@XX%@%&&@@&&&&@%%X@@@OOo  ",
".O@@@@@@@@@@@@@@@@@@@@@@@@@@*oo  ",
".#@@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
".+@@@@XX%%@@%%%%%%%%@%%X@@@Xoo  ",
".@@@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
".+@@@XX%%@%%%@%%@%%%%%%X@@@Xoo  ",
".#@@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
```

```
".O@@@@XX%%@%%%@@%%%%@%%X@@@Xoo  ",
" .+@@@@@@@@@@@@@@@@@@@@@@@@@O#o  ",
" .#@@@@XX&@&%%@%&&@@&&XX@@@#Xo  ",
" .O@@@@@@@@@@@@@@@@@@@@@@@@+Xoo ",
"  .+@@@@XX%%&%@&%%%%&&%%X@@@OOo ",
"  .#@@@@@@@@@@@@@@@@@@@@@@@@#Xo ",
"  .O@@@@XX%&@&&%%@&&&@%%X@@@+Xoo",
"   .@@@@@@@@@@@@@@@@@@@@@@@@@OXo",
"  .O@@@@@@@@@@@@@@@@@@@@@@@@@X+o",
"   .@@@@XX%&&@%%%@@@&@&%XX@@@OXo",
"  .O@@@@@@@@@@@@@@@@@@@@@@@@@X+o",
"  .#@@@@XX&@&&&@%%@@&&&%XX@@+X+o",
"  .+@@@@@@@@@@@@@@@@@@@@@@@@#$oo",
"  .@@@@@XX&%&&@&&@&&%@%@X@@@O$o ",
" .O@@@@@@@@@@@@@@@@@@@@@@@@+$oo ",
" .#@@@@XXX&%@%%@@@@@@@@@@@@#Xoo ",
" .+@@@@@@@@@@@@@@@@@@@@@@@@O$o  ",
".O@@@@@@@@@@@@@@@@@@@@@@@@#X+o  ",
"%%%%%%%%%%%%%%%%%%%%%%%%%%o    "};
}
```

XPM Definition

```
static final public String _post[] = {"32 32 6 1",
"       c #C0C0C0C0C0C0",
".      c #FFFFFFFFFFFF",
"X      c #000000000000",
"o      c #FFFF99990000",
"O      c #333333333333",
"+      c #00009999FFFF",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
" ..............................  ",
" .XXXXXX................oooo..O ",
" ........................++++..O ",
" .XXXXXX................++++..O ",
" ........................++++..O ",
" ..............................O ",
" ..............................O ",
" ..............................O ",
" ..............................O ",
" ..............................O ",
" ................XXXXXXXXXXXX..O ",
" ..............................O ",
" ..............................O ",
" ................XXXXXXXXXXXX..O ",
" ..............................O ",
" ..............................O ",
```

```
" ..............XXXXXXXXXXX..O ",
" ............................O ",
"  OOOOOOOOOOOOOOOOOOOOOOOOOOOO ",
"                              ",
"                              ",
"                              ",
"                              ",
"                              ",
"                              ",
"                              "};
static final public String _reply[] = {"32 32 7 1",
"       c #C0C0C0C0C0C0",
".      c #FFFFFFFFFFFF",
"X      c #000000000000",
"o      c #FFFF99990000",
"O      c #333333333333",
"+      c #00009999FFFF",
"@      c #FFFF00000000",
"                              ",
"                              ",
"                              ",
" ............................  ",
" .XXXXXX.................oooo..O ",
" ........................++++..O ",
" .XXXXXX.................++++..O ",
" ........................++++..O ",
" ....@.........................O ",
" ....@.........................O ",
" ...@@@........................O ",
" ...@@@........................O ",
" ..@@@@@.......................O ",
" ..@@@@@........XXXXXXXXXXX..O ",
" ....@.........................O ",
" ....@.........................O ",
" ....@..........XXXXXXXXXXX..O ",
" ....@.........................O ",
" ....@.........................O ",
" ....@..........XXXXXXXXXXX..O ",
" ....@.........................O ",
"  OOO@OOOOOOOOOOOOOOOOOOOOOOOO ",
"     @                        ",
"     @             @@@@@@@@@@ ",
"     @                 @      ",
"     @                 @      ",
"     @                 @      ",
"     @                 @      ",
"     @                 @      ",
"     @@@@@@@@@@@@@@@@@@@      ",
"                              ",
"                              "};
static final public String _reload[] = {"32 32 4 1",
"       c #C0C0C0C0C0C0",
".      c #9999CCCCFFFF",
```

```
"X      c #000099999999",
"o      c #00009999FFFF",
"                              ",
"                              ",
"                ..            ",
"                ..X           ",
"            .......X   ooooooo   ",
"          .........X   oooooooX  ",
"        ...........X   oooooooX  ",
"       .....XXXXX..X   oooooooX  ",
"      ....XXX    ..X   oooooooX  ",
"     ....XX      ..X   oooooooX  ",
"     ...XX        XX   oooooooX  ",
"    ...XX                XXXXXXX  ",
"    ..XX                        ",
"   ...X                         ",
"   ..XX                         ",
"   ..X                          ",
"   ..X                   .X     ",
"   ..X                  ...X    ",
"   ..X                 .....X   ",
"   ..X                .......X  ",
"   ...               .........X ",
"    ..X                ...XXXXX ",
"    ...                ...X     ",
"     ...               ...XX    ",
"     ....              ....X    ",
"      ....           ....XX     ",
"       .....       .....XX      ",
"        ..............XX       ",
"         X...........XXX        ",
"           X.......XXX          ",
"             XXXXXXX            ",
"                              ",
"                              "};
static final public String _catchup[] = {"32 32 9 1",
"       c #C0C0C0C0C0C0",
".      c #000000000000",
"X      c #FFFFCCCC0000",
"o      c #00009999FFFF",
"O      c #333333333333",
"+      c #000099999999",
"@      c #999999999999",
"#      c #9999CCCCFFFF",
"$      c #00009999FFFF",
"                              ",
"                              ",
"                .             ",
"               ...            ",
"              XX.XX           ",
"             XXXXXXX          ",
"             oXXXXXoO         ",
"             o+XXX+oO         ",
"             o+o+o+oO         ",
```

```
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             o+o+o+o0           ",
"             @@@@@@@0           ",
"             #######0           ",
"             $$$$$$$0           ",
"             $$$$$$$0           ",
"             $$$$$$$0           ",
"             $$$$$$$0           ",
"              0000000           ",
"                                ",
"                                "};
static final public String _back10[] = {"32 32 3 1",
"       c #C0C0C0C0C0C0",
".      c #9999CCCCFFFF",
"X      c #000099999999",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"            .           .       ",
"          ...X         ...X     ",
"        .....X       .....X     ",
"      .......X     .......X     ",
"    .........X   .........X     ",
"  ...........X ...........X     ",
" ............X............X     ",
"  ...........X ...........X     ",
"   X.........X  X.........X     ",
"     X.......X    X.......X     ",
"       X.....X      X.....X     ",
"         X...X        X...X     ",
"           X.X          X.X     ",
"             X            X     ",
"                                ",
"                                ",
```

```
"                                ",
"                                ",
"                                ",
"                                ",
"                                ",
"                                "};
static final public String _reset[] = {"32 32 5 1",
"       c #C0C0C0C0C0C0",
".       c #99999999FFFF",
"X       c #000099999999",
"o       c #9999CCCCFFFF",
"O       c #00009999FFFF",
"                                ",
"                                ",
"                                ",
"              .......           ",
"           ............         ",
"         ..............         ",
"       ......XXXXXXX......     ",
"      .....XXooooooXX.....      ",
"      ....XoooooooooooX....     ",
"     ....XoooooooooooooX....    ",
"    ....XooooooooooooooX....    ",
"    ...XoooooooooooooooX...     ",
"    ...XoooooooooooooooX...     ",
"   ...XooooooooXXXooooooooX...  ",
"   ...XooooooooX000Xooooooo X...",
"   ...XooooooX00000Xooooooo X...",
"   ...XooooooX00000Xooooooo X...",
"   ...XooooooX00000Xooooooo X...",
"   ...XooooooooX000Xooooooo X...",
"   ...XooooooooXXXooooooooX...  ",
"    ...XooooooooooooooooX...    ",
"    ...XooooooooooooooooX...    ",
"    ....XooooooooooooooX....    ",
"     ....XooooooooooooX....     ",
"      ....XoooooooooooX....     ",
"      .....XXooooooXX.....      ",
"       ......XXXXXX......       ",
"         ..............         ",
"          ............          ",
"             .......            ",
"                                ",
"                                "};

static String _faq[] = {
/* width height ncolors cpp [x_hot y_hot] */
"32 32 12 1 0 0",
/* colors */
"       s none  m none  c none",
".      c #5D5D60606969",
"X      s iconGray8     m black c #212121212121",
"o      c #DCDCDEDEE5E5",
```

```
"O      s iconGray4     m white c #949494949494",
"+      s iconGray1     m white c #dedededededede",
"@      s iconColor2    m white c white",
"#      s iconGray2     m white c #bdbdbdbdbdbd",
"$      s iconGray5     m black c #737373737373",
"%      s iconColor1    m black c black",
"&      s iconGray7     m black c #424242424242",
"*      s iconGray6     m black c #636363636363",
/* pixels */
"                                ",
"                                ",
"                                ",
"   .............................Xo",
"   .O+@@@@@@@@@@@@@@@@@@@@@@@@@+Xoo",
"   .#@@@@@@@@@@@@@@@@@@@@@@@@@@OXo ",
"  .O+@@@@@@@@@@@@@@@@@@@@@@@@@+Xoo ",
"  .#@@@@@@@@@@@@@@@@@@@@@@@@@#$o  ",
"  .@@@@@XX%@%&&@@&&&&@%%X@@@OOo  ",
".O@@@@@@@@@@@@@@@@@@@@@@@@@*oo  ",
".#@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
".+@@@@XX%%@@%%%%%%%%@%%X@@@Xoo  ",
".@@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
".+@@@XX%%@%%%@%%@%%%%%%X@@@Xoo  ",
".#@@@@@@@@@@@@@@@@@@@@@@@@@Xoo  ",
".O@@@@XX%%@%%%@@%%%@%%X@@@Xoo  ",
"  .+@@@@@@@@@@@@@@@@@@@@@@@@O#o  ",
"  .#@@@@XX&@&%%@%&&@@&&XX@@@#Xo  ",
"  .O@@@@@@@@@@@@@@@@@@@@@@@@@+Xoo ",
"   .+@@@@XX%%&%@&%%%%&&%%X@@@OOo ",
"   .#@@@@@@@@@@@@@@@@@@@@@@@@@#Xo ",
"   .O@@@@XX%&@&&%%@&&&@%%X@@@+Xoo",
"    .@@@@@@@@@@@@@@@@@@@@@@@@@@OXo",
"   .O@@@@@@@@@@@@@@@@@@@@@@@@@@X+o",
"    .@@@@XX%&&@%%%@@@&@&%XX@@@OXo",
"   .O@@@@@@@@@@@@@@@@@@@@@@@@@@X+o",
"   .#@@@@XX&@&&&@%%@@&&&%XX@@+X+o",
"   .+@@@@@@@@@@@@@@@@@@@@@@@@@#$oo",
"   .@@@@@XX&%&&@&&@&&%@%@X@@@O$o ",
"  .O@@@@@@@@@@@@@@@@@@@@@@@@@+$oo ",
"  .#@@@@XXX&%@%%@@@@@@@@@@@@@#Xoo ",
"  .+@@@@@@@@@@@@@@@@@@@@@@@@O$o  ",
".O@@@@@@@@@@@@@@@@@@@@@@@@@#X+o  ",
"%%%%%%%%%%%%%%%%%%%%%%%%%%o   "};
```

Pool

```
import java.util.*;

//Implements Thread Pooling. Thread Pool simply keeps a
//bunch of suspended threads around to do some work.
public class Pool {
//Handler class to perform work requested by the Pool.
```

```
  class WorkerThread extends Thread {
    private Worker _worker;
    private Object _data;
    //Create a new WorkerThread @param id Thread ID
    //@param worker Worker instance associated with the WorkerThread
    WorkerThread(String id, Worker worker) {
      super(id);
      _worker = worker;
      _data = null;
    }
  //Wake the thread and does some work @param data Data to send
  //to the Worker. @return void
  synchronized void wake (Object data) {
     _data = data;
     notify();
  }
  //WorkerThread's thread routine
  synchronized public void run() {
    boolean stop = false;
      while (!stop) {
        if( _data == null ) {
          try {
            wait();
          } catch (InterruptedException e) {
            e.printStackTrace();
            continue;
          }
        }
        if( _data != null ) {
          _worker.run(_data);
        }
        _data = null;
        stop = !(_waiting.push (this));
      }
   }
};

private Stack _waiting;
private int _max;
private Class _workerClass;

// Creates a new Pool instance
// @param max Max number of handler threads
// @param workerClass Name of Worker implementation
// @throws Exception
public Pool (int max, Class workerClass) throws Exception {
  _max = max;
  _waiting = new Stack();
  _workerClass = workerClass;
  Worker worker;
  WorkerThread w;
  for( int i = 0; i < _max; i++ ) {
    worker = (Worker)_workerClass.newInstance();
```

```
    w = new WorkerThread ("Worker#"+i, worker);
    w.start();
    _waiting.push(w);
  }
}

// Request the Pool to perform some work.
// @param data Data to give to the Worker
// @return void
// @throws InstantiationException Thrown if additional worker can't be created
public void performWork (Object data) throws InstantiationException {
  WorkerThread w = null;
  synchronized (_waiting) {
    if( _waiting.empty()) {
      try {
        w = new WorkerThread("additional worker",
                             (Worker)_workerClass.newInstance());
        w.start();
      } catch (Exception e) {
        throw new InstantiationException(
                 "Problem creating instance of Worker.class: " + e.getMessage());
      }
    } else {
      w = (WorkerThread)_waiting.pop();
    }
  }
  w.wake (data);
}

//Convience method used by WorkerThread to put Thread back on the stack
//@param w WorkerThread to push
//@return boolean True if pushed, false otherwise
  private boolean _push (WorkerThread w) {
    boolean stayAround = false;
    synchronized (_waiting) {
      if( _waiting.size() < _max ) {
        stayAround = true;
        _waiting.push(w);
      }
    }
    return stayAround;
  }
}
```

Worker

```
public interface Worker {
//Method invoked to request worker to perform task
  public void run(Object data);
}
```

HttpServerWorker

```
import java.io.*;
import java.net.*;
import java.util.*;

//Implements the Worker interface for the HttpServer
public class HttpServerWorker implements Worker {
//Invoked by the Pool when a job comes in for the Worker
// @param data Worker data
// @return void
public void run(Object data) {
  Socket socket = (Socket)((Hashtable)data).get ("Socket");
  HttpServer server = (HttpServer)((Hashtable)data).get ("HttpServer");
  try {
    DataInputStream input = new DataInputStream(
                          new BufferedInputStream(socket.getInputStream()));
    String line = input.readLine();
    if(line.toUpperCase().startsWith ("POST") ) {
      for( ; (line=input.readLine()).length() > 0; )
         ;
      int type = input.readInt();
      switch (type) {
        case HttpClient.DATA : {
          int length = input.readInt();
          byte buffer[] = new byte[length];
          input.readFully (buffer);
          ByteArrayOutputStream dataOut = new ByteArrayOutputStream();
          server.notifyListener(new ByteArrayInputStream (buffer), dataOut);
          DataOutputStream output = new DataOutputStream(
                    new BufferedOutputStream(socket.getOutputStream()));
          server.writeResponse (output);
//API Ref :void writeInt(int value)
           output.writeInt (HttpClient.DATA);
//API Ref :byte[] toByteArray()
          output.writeInt (dataOut.toByteArray().length);
          output.write (dataOut.toByteArray());
          output.flush();
          input.close();
          output.close();
          socket.close();
          break;
        } case HttpClient.PENDING : {
        //DON'T CLOSE THE SOCKET!
          server.addClient (socket);
          break;
        } default : {
          System.err.println ("Invalid type: " + type);
        }
      }
    } else {
      System.err.println ("Invalid HTTP request: " + line);
    }
```

```
  } catch (IOException e) {
    e.printStackTrace();
  }
 }
}
```

HttpServer

```
import java.io.*;
import java.net.*;
import java.util.*;

//Implementation of a basic HTTP server for Firewall tunneling.
//The server supports both Client->Server and Server->Client communication.
public class HttpServer implements Runnable {
// Accept socket
  private ServerSocket _serverSocket;
// Server Listener
  private HttpServerListener _listener;
// Thread accepting connections
  private Thread _acceptTID;
// Handler threads
  private Pool _pool;
// Client sockets
  private Vector _clients;
// Default HTTP Response
  private String _httpResponse;
//Create a new HttpServer instance
//@param port Port to listen on
//@param poolSize Number of handler threads
//@throw IOException Thrown if the accept socket cannot be opended
  public HttpServer (int port, int poolSize) throws IOException {
    _serverSocket = new ServerSocket (port);
    _httpResponse = "HTTP/1.0 200 MyServer \nCache-Control:
                  no-cache\nPragma: no-cache \r\n\r\n";
    try {
      _pool = new Pool (poolSize, HttpServerWorker.class);
    } catch (Exception e) {
      e.printStackTrace();
      throw new InternalError (e.getMessage());
    }
    _clients = new Vector();
    _acceptTID = new Thread(this);
    _acceptTID.start();
  }

//Adds a new client
//@param s Socket
  synchronized void addClient (Socket s) {
    _clients.addElement(s);
  }
```

```
//Adds a new HttpServerListener. Only one listener can be added
//@param l HttpServerListener
//@throws TooManyListenersException Thrown if more then one listener is added
  public void addHttpServerListener (HttpServerListener l)
                                    throws TooManyListenersException {
    if( _listener == null ) {
      _listener = l;
    } else {
      throw new TooManyListenersException();
    }
  }
//Removes a new HttpServerListener.
//@param l HttpServerListener
public void removeHttpServerListener (HttpServerListener l){
  _listener = null;
}
//Notifies the listener when a message arrives
//@param data Message data
//@param out Stream to write results too
  synchronized void notifyListener (InputStream data, OutputStream out){
    if( _listener != null ) {
      _listener.service (data, out);
    }
  }
//Simple implementation that sends data to all clients
//@param data Array of bytes containing data to send
  synchronized public void send (byte data[]) {
    Enumeration elements = _clients.elements();
    while( elements.hasMoreElements() ) {
      Socket s = (Socket)elements.nextElement();
      try {
        DataOutputStream output = new DataOutputStream(
                            new BufferedOutputStream (s.getOutputStream()));
        int length;
        writeResponse (output);
        output.writeInt (data.length);
        output.write (data);
        output.flush();
        output.close();
      } catch (IOException e) {
        e.printStackTrace();
      } finally {
        try {
          s.close();
        } catch (IOException e) {
      }
    }
  }
  _clients.removeAllElements();
}
//Thread to accept connections
  public void run() {
    while(true) {
```

```
      try {
        Socket s = _serverSocket.accept();
        Hashtable data = new Hashtable();
        data.put ("Socket", s);
        data.put ("HttpServer", this);
        _pool.performWork (data);
      } catch (Exception e) {
        e.printStackTrace();
      }
   }
}
//Convience method to write the HTTP Response header
  * @param out Stream to write the response too
  * @throws IOException Thrown if response can't be written
  void writeResponse (DataOutputStream out) throws IOException {
    out.writeBytes (_httpResponse);
  }
}
```

HttpServerListener

```
/**
 * Used by the server to listen for messages from the HttpClient
 */
public interface HttpServerListener extends java.util.EventListener
{
   /*
    * Method invoked when a message is received by the HttpServer
    * @param data Message data
    * @param results Stream to write response too
    * @return void
    */
    public void service (java.io.InputStream data,
                                java.io.OutputStream results);
}
```

HttpListener

```
/**
 * Used by the client's to listen for messages from the HttpServer
 */
public interface HttpListener extends java.util.EventListener
{
        /**
         * Method invoked when a message is sent from the HttpServer
         * @param data Message data
         * @return void
         */
        public void service (java.io.InputStream data);
```

```
}
```

HttpClient

```
import java.net.*;
import java.io.*;
import java.util.*;


/**
 * Implementation of a basic HTTP client for Firewall
 * tunneling. The client supports both Client->Server and
 * Server->Client communication.
 */
public class HttpClient implements Runnable {
  // Type of message: also used in HttpServerWorker
  final static public int PENDING = 1;
  final static public int DATA = 2;

  // The Server URL
  private String _url;

  // Pending connection Thread
  private Thread _pendingTID;
  private boolean _stop;

  // Listeners for incoming messages
  private HttpListener _listener;

  /**
   * Creates a new HttpClient instance configured
   * to connect to the server specified in the url
   * parameter.
   * @param url URL of the server, example http://bogus:9000
   */
  public HttpClient (String url) {
    _url = url;
    _pendingTID = new Thread(this);
    _pendingTID.setPriority (Thread.MAX_PRIORITY);
    _listener = null;
  }

  /**
   * Removes the HttpListener. The pending connection
   * is stopped, so the client won't receive any more
   * messages from the server
   * @param l HttpListener to remove
   * @return void
   */
  public void removeHttpListener (HttpListener l) {
    if ( _listener != null ) {
      synchronized (this) {
```

```
    _stop = true;
  }
  _pendingTID.destroy();
  _listener = null;
  }
}

/**
 * Adds the HttpListener. The pending connection
 * is started, so the client will receive
 * messages from the server.
 * @param l HttpListener to remove
 * @return void
 * @throws TooManyListenersException Thrown if more then one
 * HttpListener is added.
 */
public void addHttpListener (HttpListener l)
                          throws TooManyListenersException {
  if ( _listener == null ) {
    _listener = l;
    synchronized (this) {
      _stop = false;
    }
    _pendingTID.start();
  } else {
    throw new TooManyListenersException();
  }
}

/**
 * Convience method to send the PENDING connection to
 * the server
 * @return void
 * @throws IOException Thrown if a problem occurs while setting
 * up the pending message.
 */
private URLConnection _sendPending () throws IOException {
  URL url = new URL (_url);
  URLConnection connection = url.openConnection();
  connection.setUseCaches(false);
  connection.setDoOutput (true);
  return connection;
}

/**
 * Thread used to maintain the pending connection
 * @return void
 */
public void run() {
  boolean s;
  do {
    URLConnection connection;
    int length;
```

```
try {
  connection = _sendPending();
  DataOutputStream output = new DataOutputStream
                            (connection.getOutputStream());
  output.writeInt (PENDING);
  output.flush();
  output.close();
  output = null;

  // Check our response code
  int code = ((HttpURLConnection)connection).getResponseCode();
  switch (code) {
      case 200: // HTTP_OK
                {
                DataInputStream input =
                    new DataInputStream (connection.getInputStream());
                length = input.readInt();
                byte buffer[] = new byte[length];
                if ( length >= 0 ) {
                  input.readFully (buffer);
                  if ( buffer.length == length ) {
                    if ( _listener != null ) {
                       _listener.service (
                          new ByteArrayInputStream (buffer));
                    }
                  } else {
                    System.err.println ("Short read");
                  }
                  buffer = null;
                } else {
                  System.err.println ("Invalid length: " + length);
                }
                input.close();
                input = null;
               }
      case 504: //HTTP_GATEWAY_TIMEOUT
                {
                connection = _sendPending();
                break;
                }
      default: //OTHER HTTP_SERVER ERRORS
                {
                  System.out.println ("Invalid code:"+code);
                  synchronized (this) {
                    _stop = true;
                  }
                }
  }
} catch (IOException e) {
  e.printStackTrace();
  synchronized (this) {
    _stop = true;
  }
```

```
        }
        synchronized (this) {
          s = _stop;
        }
      }while ( s == false );

    }

    /**
     * Sends a message to the server
     * @param data Array of bytes to send
     * @return void
     * @throws IOException Thrown if a problem occurs while sending
     * the message
     */
    synchronized public byte[] send (byte data[])
                  throws IOException {
      byte buffer[];

      // Establish a connection
      URL url = new URL (_url);
      URLConnection connection = url.openConnection();
      connection.setUseCaches (false);
      connection.setDoOutput(true);

      // Write out the data
      DataOutputStream dataOut =
                new DataOutputStream (
                    new BufferedOutputStream (connection.getOutputStream()));
      dataOut.writeInt (HttpClient.DATA);
      dataOut.writeInt (data.length);
      dataOut.write (data);
      dataOut.flush();
      dataOut.close();

      int length;
      DataInputStream input =
                 new DataInputStream (
                     new BufferedInputStream (connection.getInputStream()));
      int type = input.readInt();
      if ( type == HttpClient.DATA ) {
        length = input.readInt();
        buffer = new byte[length];
        input.readFully (buffer);
      } else {
        buffer = null;
        throw new IOException ("Unknown Response Type");
      }
      input.close();
      return buffer;
    }
  }
```

JDCConnectionDriver

```
package pool;

import java.sql.*;
import java.util.*;

public class JDCConnectionDriver implements Driver {
  public static final String URL_PREFIX = “jdbc:jdc:”;
  private static final int MAJOR_VERSION = 1;
  private static final int MINOR_VERSION = 0;
  private JDCConnectionPool pool;
  public JDCConnectionDriver(String driver, String url,
                             String user, String password)
                             throws ClassNotFoundException,
                             InstantiationException, IllegalAccessException,
                             SQLException {
   DriverManager.registerDriver(this);
   Class.forName(driver).newInstance();
   pool = new JDCConnectionPool(url, user, password);
}
  public Connection connect(String url, Properties props) throws SQLException {
    if(!url.startsWith(URL_PREFIX) {
      return null;
    }
    return pool.getConnection();
  }
  public boolean acceptsURL(String url) {
    return url.startsWith(URL_PREFIX);
  }
  public int getMajorVersion() {
    return MAJOR_VERSION;
  }
  public int getMinorVersion() {
    return MINOR_VERSION;
  }
  public DriverPropertyInfo[] getPropertyInfo(String str, Properties props) {
    return new DriverPropertyInfo[0];
  }
  public boolean jdbcCompliant() {
    return false;
  }
}
```

JDCConnectionPool

```
package pool;

import java.sql.*;
import java.util.*;
import java.io.*;
```

```
    class ConnectionReaper extends Thread {
      private JDCConnectionPool pool;
      private final long delay=300000;
      ConnectionReaper(JDCConnectionPool pool) {
        this.pool=pool;
      }
      public void run() {
        while(true) {
          try {
            sleep(delay);
          } catch( InterruptedException e) { }
            pool.reapConnections();
          }
        }
    }
    public class JDCConnectionPool {
      private Vector connections;
      private String url, user, password;
      final private long timeout=60000;
      private ConnectionReaper reaper;
      final private int poolsize=10;

      public JDCConnectionPool(String url, String user, String password) {
        this.url = url;
        this.user = user;
        this.password = password;
        connections = new Vector(poolsize);
        reaper = new ConnectionReaper(this);
        reaper.start();
      }
      public synchronized void reapConnections() {
        long stale = System.currentTimeMillis() - timeout;
        Enumeration connlist = connections.elements();
//API Ref :boolean hasMoreElements()
         while((connlist != null) && (connlist.hasMoreElements())) {
//API Ref :Object nextElement()
          JDCConnection conn = (JDCConnection)connlist.nextElement();
            if((conn.inUse()) && (stale >conn.getLastUse()) && (!conn.validate())) {
             removeConnection(conn);
           }
         }
       }

      public synchronized void closeConnections() {
        Enumeration connlist = connections.elements();
        while((connlist != null) && (connlist.hasMoreElements())) {
          JDCConnection conn = (JDCConnection)connlist.nextElement();
           removeConnection(conn);
         }
      }
      private synchronized void removeConnection(JDCConnection conn) {
        connections.removeElement(conn);
      }
```

```
  public synchronized Connection getConnection() throws SQLException {
    JDCConnection c;
    for(int i = 0; i < connections.size(); i++) {
       c = (JDCConnection)connections.elementAt(i);
       if(c.lease()) {
         return c;
       }
     }
     Connection conn = DriverManager.getConnection(url, user, password);
     c = new JDCConnection(conn, this);
     c.lease();
     connections.addElement(c);
     return c;
  }

  public synchronized void returnConnection(JDCConnection conn) {
    conn.expireLease();
  }
}
```

JDCConnection

```
package pool;

import java.sql.*;
import java.util.*;
import java.io.*;

public class JDCConnection implements Connection {
  private JDCConnectionPool pool;
  private Connection conn;
  private boolean inuse;
  private long timestamp;

  public JDCConnection(Connection conn, JDCConnectionPool pool) {
    this.conn=conn;
    this.pool=pool;
    this.inuse=false;
    this.timestamp=0;
  }
  public synchronized boolean lease() {
    if(inuse)  {
      return false;
    } else {
      inuse=true;
      timestamp=System.currentTimeMillis();
      return true;
    }
  }
  public boolean validate() {
    try {
      conn.getMetaData();
```

```
    } catch (Exception e) {
      return false;
    }
    return true;
  }
  public boolean inUse() {
    return inuse;
  }
  public long getLastUse() {
    return timestamp;
  }
  public void close() throws SQLException {
    pool.returnConnection(this);
  }
  protected void expireLease() {
    inuse=false;
  }
  protected Connection getConnection() {
    return conn;
  }
  public PreparedStatement prepareStatement(String sql) throws SQLException {
    return conn.prepareStatement(sql);
  }
  public CallableStatement prepareCall(String sql) throws SQLException {
    return conn.prepareCall(sql);
  }
  public Statement createStatement() throws SQLException {
    return conn.createStatement();
  }
  public String nativeSQL(String sql) throws SQLException {
    return conn.nativeSQL(sql);
  }
  public void setAutoCommit(boolean autoCommit) throws SQLException {
    conn.setAutoCommit(autoCommit);
  }
  public boolean getAutoCommit() throws SQLException {
    return conn.getAutoCommit();
  }
  public void commit() throws SQLException {
    conn.commit();
  }
  public void rollback() throws SQLException {
    conn.rollback();
  }
  public boolean isClosed() throws SQLException {
    return conn.isClosed();
  }
  public DatabaseMetaData getMetaData() throws SQLException {
    return conn.getMetaData();
  }
  public void setReadOnly(boolean readOnly) throws SQLException {
    conn.setReadOnly(readOnly);
  }
```

```
  public boolean isReadOnly() throws SQLException {
    return conn.isReadOnly();
  }
  public void setCatalog(String catalog) throws SQLException {
    conn.setCatalog(catalog);
  }
  public String getCatalog() throws SQLException {
    return conn.getCatalog();
  }
  public void setTransactionIsolation(int level) throws SQLException {
    conn.setTransactionIsolation(level);
  }
  public int getTransactionIsolation() throws SQLException {
    return conn.getTransactionIsolation();
  }
  public SQLWarning getWarnings() throws SQLException {
    return conn.getWarnings();
  }
  public void clearWarnings() throws SQLException {
    conn.clearWarnings();
  }
}
```

RegistrationBean

```
package registration;

import java.rmi.RemoteException;
import javax.ejb.*;
import java.util.*;
import java.text.NumberFormat;
import java.sql.*;

// uses our connection pool
public class RegistrationBean implements EntityBean {
  protected transient EntityContext ctx;
  public String theuser, password, creditcard, emailaddress;
  public double balance;
  static {
    try {
      new pool.JDCConnectionDriver("COM.cloudscape.core.JDBCDriver",
                                   "jdbc:cloudscape:ejbdemo","none", "none");
    } catch(Exception e){}
  }
  public Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:jdc:jdcpool");
  }
  public boolean verifyPassword(String password) throws RemoteException {
    if(this.password.equals(password)) {
      return true;
    } else {
      return false;
```

```
  }
}
public String getEmailAddress() throws RemoteException {
  return emailaddress;
}
public String getUser() throws RemoteException {
  return theuser;
}
public int adjustAccount(double amount) throws RemoteException {
  balance=balance+amount;
  return(0);
}
public double getBalance() throws RemoteException {
  return balance;
}
public RegistrationPK ejbCreate(String theuser, String password,
                                String emailaddress, String creditcard)
                                throws CreateException, RemoteException {
  System.out.println("registration create");
  this.theuser=theuser;
  this.password=password;
  this.emailaddress=emailaddress;
  this.creditcard=creditcard;
  this.balance=0;
  Connection con = null;
  PreparedStatement ps = null;;
  try {
    con=getConnection();
    ps=con.prepareStatement("insert into registration (theuser,
    password, emailaddress, creditcard, balance) values (?, ?, ?, ?, ?)");
    ps.setString(1, theuser);
    ps.setString(2, password);
    ps.setString(3, emailaddress);
    ps.setString(4, creditcard);
    ps.setDouble(5, balance);
    if(ps.executeUpdate() != 1) {
       System.out.println("registration create failed");
       throw new CreateException ("JDBC did not create any row");
     }
     RegistrationPK primaryKey = new RegistrationPK();
     primaryKey.theuser = theuser;
     return primaryKey;
   } catch (CreateException ce) {
     throw ce;
   } catch (SQLException sqe) {
     throw new CreateException (sqe.getMessage());
   } finally {
    try {
       ps.close();
       con.close();
     } catch (Exception ignore) {}
   }
  }
```

```
  public void ejbPostCreate(String theuser, String password,
    String emailaddress, String creditcard)
    throws CreateException, RemoteException {
  }
  public void setEntityContext(javax.ejb.EntityContext ctx)
                              throws RemoteException {
    this.ctx = ctx;
  }
  public void unsetEntityContext() throws RemoteException {
    ctx = null;
  }
  public void ejbRemove() throws RemoteException, RemoveException { }
  public void ejbActivate() throws RemoteException { }
  public void ejbPassivate() throws RemoteException { }
  public void ejbLoad() throws RemoteException {
    System.out.println("registration load");
    try {
      refresh((RegistrationPK) ctx.getPrimaryKey());
    } catch (FinderException fe) {
      throw new RemoteException (fe.getMessage());
    }
  }
  public void ejbStore() throws RemoteException {
    System.out.println("registration store");
    Connection con = null;
    PreparedStatement ps = null;
    try {
      con = getConnection();
      ps = con.prepareStatement("update registration set password = ?,
         emailaddress = ?, creditcard = ?, balance = ? where theuser = ?");
      ps.setString(1, password);
      ps.setString(2, emailaddress);
      ps.setString(3, creditcard);
      ps.setDouble(4, balance);
      ps.setString(5, theuser);
      int i = ps.executeUpdate();
      if(i == 0) {
        throw new RemoteException ("ejbStore: Registration (" + theuser
                                   + ") not updated");
      }
    } catch (RemoteException re) {
      throw re;
    } catch (SQLException sqe) {
      throw new RemoteException (sqe.getMessage());
    } finally {
      try {
        ps.close();
      con.close();
    } catch (Exception ignore) {}
  }
}
public RegistrationPK ejbFindByPrimaryKey(RegistrationPK pk)
                                throws FinderException, RemoteException {
```

```
    if((pk == null) || (pk.theuser == null)) {
      throw new FinderException ("primary key cannot be null");
    }
    refresh(pk);
    return pk;
  }
  private void refresh(RegistrationPK pk)
                       throws FinderException, RemoteException {
    if(pk == null) {
      throw new RemoteException ("primary key cannot be null");
    }
    Connection con = null;
    PreparedStatement ps = null;
    try {
      con=getConnection();
      ps=con.prepareStatement("select password, emailaddress, creditcard,
                              balance from registration where theuser = ?");
      ps.setString(1, pk.theuser);
      ps.executeQuery();
      ResultSet rs = ps.getResultSet();
      if(rs.next()) {
         theuser = pk.theuser;
         password = rs.getString(1);
         emailaddress = rs.getString(2);
         creditcard = rs.getString(3);
         balance = rs.getDouble(4);
       } else {
         throw new FinderException ("Refresh: Registration ("
                                    + pk.theuser + ") not found");
       }
     } catch (SQLException sqe) {
       throw new RemoteException (sqe.getMessage());
     } finally {
       try {
         ps.close();
         con.close();
       } catch (Exception ignore) {}
     }
   }
}
```

9: Deploying the Auction Application

With the auction application tested, debugged, and tuned, you are ready to deploy it. In this chapter, Administration applet replaces the AuctionClient (page 270) application so you can see how to deploy an applet, which is a little more involved than deploying an application.

To deploy the complete application, you bundle the application files, move the application files to their production locations, install Java™ Plug-In so auction administrators can run the Administration applet from their browsers, and install the Administration applet policy file. Java Plug-In is needed because the Administration applet is written with JDK 1.2 APIs, but the administrators' browsers will likely run an earlier version of the Java Runtime Environment™ (JRE) software.

This chapter explains how to use the Java Archive (JAR) file format to bundle and deploy the application files, and how to install Java Plug-In and a security policy file for the Solaris™ and Win32 platforms to run the Administration applet.

Covered in this Chapter

- JAR File Format (page 399)
- Deploy to Solaris Operating System (page 406)
- Deploy to Win32 Platform (page 411)

JAR File Format

Java Archive (JAR) file format is a compression and file packaging format and tool for bundling executable files with other related application files so they can be deployed as a single unit. The auction application has the following three sets of files to deploy to three different locations:

1. The HTML files that make up the auction application user interface deploy to a publicly accessible location under the web server.

2. The Enterprise Beans deploy to an internal location accessible to your production installation of the Enterprise JavaBeans™ server.
3. The Administration applet deploys to an internal location accessible to auction administrators where it is run from their browsers.

Bundle and Deploy the HTML Files

Here is the list of HTML files that make up the auction application user interface:

- all.html
- close.html
- details.html
- index.html
- juggler.med.gif
- new.html
- registration.html
- search.html
- sell.html

And here is the jar command to bundle them. Everything goes on one line. This command is executed in the same directory with the files. If you execute the command from a directory other than where the files are, specify the full or relative pathname as appropriate.

```
jar cvf HTML.jar all.html close.html details.html index.html
juggler.med.gif new.html registration.html search.html sell.html
```

jar is the Java Archive command. If you type jar with no options, you get the following help screen. You can see from the help screen that the cf options to the jar command mean create a new JAR file named HTML.jar and put the list of files that follows into it. The new JAR file is placed in the current directory.

```
kq6py% jar
Usage: jar {ctxu}[vfm0M][jar-file][manifest-file][-C dir] files ...
Options:
  -c  create new archive
  -t  list table of contents for archive
  -x  extract named (or all) files from archive
  -u  update existing archive
  -v  generate verbose output on standard output
  -f  specify archive file name
  -m  include manifest information from specified  manifest file
  -0  store only; use no ZIP compression
  -M  Do not create a manifest file for the entries
  -C  change to the specified directory and include the following file
```

```
If any file is a directory then it is processed recursively. The manifest file name
and the archive file name needs to be specified in the same order the 'm' and 'f'
flags are specified.

Example 1: to archive two class files into an archive  called classes.jar: jar cvf
classes.jar Foo.class Bar.class
Example 2: use an existing manifest file 'mymanifest' and archive all the files in
the foo/ director into 'classes.jar': jar cvfm classes.jar mymanifest -C foo/ .
```

To deploy the HTML files, all you have to do is move the HTML.jar file to a publicly accessible directory under the web server and decompress the JAR file:

```
jar xvf HTML.jar
```

Note: If you included a full or relative pathname when you added the files to the JAR file, the files are placed in the same directory structure when they are unpacked.

The JAR file contains a manifest file, `META-INF/MANIFEST.MF` that the jar tool automatically placed there. A manifest file contains information about the files packaged in the JAR file. You can tailor this information to enable JAR file functionality such as electronic signing, version control, package sealing, and more. You can find more information on manifest files on the web at `http://java.sun.com/docs/books/tutorial/jar/basics/manifest.html`.

Bundle and Deploy the Enterprise Beans

Some Enterprise JavaBeans servers create the JAR file for you. However, if yours does not or if you just wonder how it's done, this section describes the steps.

These are the server-side files you need to deploy the Enterprise Beans. This list is taken from the original auction application described in 2: Auction House Application (page 13) before any modifications were made. Note the inclusion of the deployment descriptor, and the container-generated stub and skel classes.

Note: Because there are so many files, these steps bundle the example code into four JAR files, one each for the `auction`, `registration`, `bidder`, and `seller` packages. You could also bundle all the packages into one JAR file and deploy that instead.

---

auction Package

These are the application files in the auction package that make up the AuctionServlet servlet and AuctionItemBean Enterprise Bean. Because they are all to be installed in an auction directory accessible to the production Enterprise JavaBeans server, bundle them together so

they can be unpacked in one step in the destination directory and placed in the auction subdirectory.

- auction.AuctionServlet.class
- auction.AuctionItem.class
- auction.AuctionItemBean.class
- auction.AuctionItemHome.class
- auction.AuctionItemPK.class
- auction.DeploymentDescriptor.txt
- AuctionItemBeanHomeImpl_ServiceStub.class
- WLStub1h1153e3h2r4x3t5w6e82e6jd412c.class
- WLStub364c363d622h2j1j422a4oo2gm5o.class
- WLSkel1h1153e3h2r4x3t5w6e82e6jd412c.class
- WLSkel364c363d622h2j1j422a4oo2gm5o.class

Here is how to bundle them. Everything goes on one line, and the command is executed one directory above where the class files are located.

Unix:

```
jar cvf auction.jar  auction/*.class auction/*.txt
```

Win32:

```
jar cvf auction.jar  auction\*.class auction\*.txt
```

Once the JAR file is copied to the destination directory for the Enterprise beans, unpack it as follows. The extraction creates an auction directory with the class files in it.

```
jar xvf auction.jar
```

registration Package

Here are the application files in the registration package that make up the Registration Enterprise Bean.

- registration.Registration.class
- registration.RegistrationBean.class
- registration.RegistrationHome.class
- registration.RegistrationPK.class
- registration.DeploymentDescriptor.txt

- RegistrationBeanHomeImpl_ServiceStub.class
- WLStub183w4u1f4e70p6j1r4k6z1x3f6yc21.class
- WLStub4z67s6n4k3sx131y4fi6w4x616p28.class
- WLSkel183w4u1f4e70p6j1r4k6z1x3f6yc21.class
- WLSkel4z67s6n4k3sx131y4fi6w4x616p28.class

Here is how to bundle them. Everything goes on one line, and the command is executed one directory above where the class files are located.

Unix:

```
jar cvf registration.jar  registration/*.class registration/*.txt
```

Win32:

```
jar cvf registration.jar  registration\*.class registration\*.txt
```

Once the JAR file is copied to the destination directory for the Enterprise beans, unpack it as follows. The extraction creates a registration directory with the class files in it.

```
jar xvf registration.jar
```

bidder Package

Here are the application files in the bidder package that make up the Bidder Enterprise Bean.

- bidder.Bidder.class
- bidder.BidderHome.class
- bidder.BidderBean.class
- bidder.DeploymentDescriptor.txt
- BidderBeanEOImpl_ServiceStub.class
- BidderBeanHomeImpl_ServiceStub.class
- WLStub1z35502726376oa1m4m395m4w5j1j5t.class
- WLStub5g4v1dm3m271tr4i5s4b4k6p376d5x.class
- WLSkel1z35502726376oa1m4m395m4w5j1j5t.class
- WLSkel5g4v1dm3m271tr4i5s4b4k6p376d5x.class

Here is how to bundle them. Everything goes on one line, and the command is executed one directory above where the class files are located.

Unix:

```
jar cvf bidder.jar  bidder/*.class bidder/*.txt
```

Win32:

```
jar cvf bidder.jar  bidder\*.class bidder\*.txt
```

Once the JAR file is copied to the destination directory for the Enterprise beans, unpack it as follows. The extraction creates a bidder directory with the class files in it.

```
jar xvf bidder.jar
```

seller Package

Here are the application files in the seller package that make up the Seller Enterprise Bean.

- seller.Seller.class
- seller.SellerHome.class
- seller.SellerBean.class
- seller.DeploymentDescriptor.txt
- SellerBeanEOImpl_ServiceStub.class
- SellerBeanHomeImpl_ServiceStub.class
- WLStub3xr4e731e6d2x3b3w5b693833v304q.class
- WLStub86w3x4p2x6m4b696q4kjp4p4p3b33.class
- WLSkel3xr4e731e6d2x3b3w5b693833v304q.class
- WLSkel86w3x4p2x6m4b696q4kjp4p4p3b33.class

Here is how to bundle them. Everything goes on one line, and the command is executed one directory above where the class files are located.

Unix:

```
jar cvf seller.jar  seller/*.class seller/*.txt
```

Win32:

```
jar cvf seller.jar  seller\*.class seller\*.txt
```

Once the JAR file is copied to the destination directory for the Enterprise beans, unpack it as follows. The extraction creates a seller directory with the class files in it.

```
jar xvf seller.jar
```

Bundle and Deploy the Applet Program

For this chapter, the Administration applet family of files consists of two source files: `AdminApplet.java` and `java.policy`. For reference, Figure 35 shows the Admin applet.

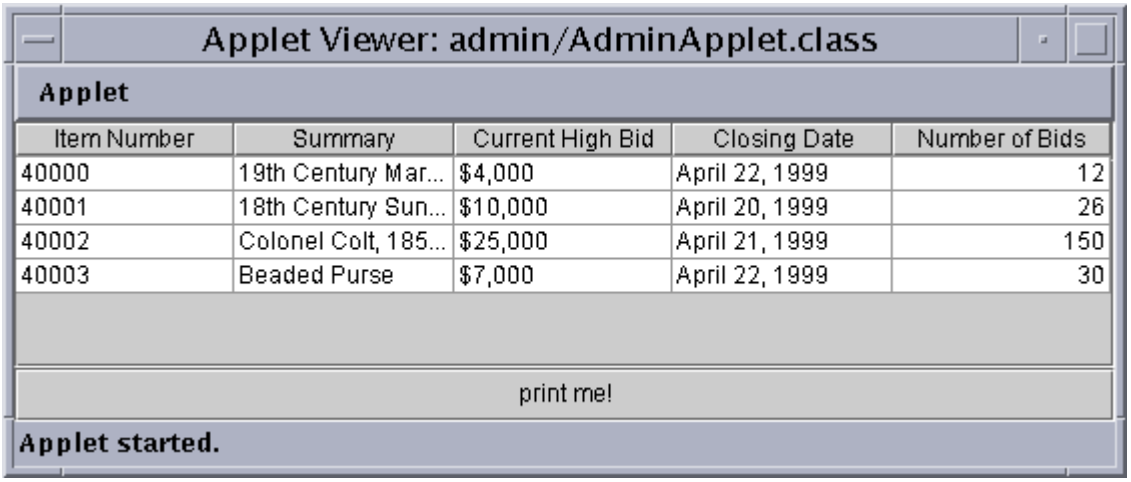


Figure 35 Administration Applet

```
grant {
  permission java.lang.RuntimePermission "queuePrintJob";
};
```

Here is the jar command to bundle them. Everything goes on one line, and the command is executed where the policy file is located which is one directory above where the class files are located.

Unix:

```
jar cvf applet.jar admin/*.class java.policy
```

Win32:

```
jar cvf applet.jar admin\*.class java.policy
```

To deploy the applet, copy the applet.jar file to the destination applet directory and extract it as follows. The extraction creates an admin directory with the Administration applet class files in it.

```
jar xvf applet.jar
```

Deploy to Solaris Operating System

Java Plug-in software lets you direct applets or JavaBeans components on intranet web pages to run using the Java Runtime Environment (JRE) instead of the web browser's default virtual machine. The Java Plug-In works with Netscape Communicator and Microsoft Internet Explorer. Free downloads of all the software you need to install and use Java Plug-In are available from the download page (`http://java.sun.com/products/plugin/1.2/download/all.html`).

This section explains how to install Java Plug-In with Netscape Communicator on the Solaris operating system.

Get Downloads

To install and use Java Plug-In on Solaris 2.6 or Solaris 7, you need the following downloads. Put the downloads in a directory anywhere you want.

- Java Plug-In for Solaris operating systems. It is available for either Intel or Sparc platforms.
- Java Plug-In patches for either Solaris 2.6 or Solaris 7, depending on which one you have.
- Netscape Communicator 4.5.1 or newer (webstart version).
- Java Plug-In HTML Converter

These instructions were tested on a Sun Microsystems Ultra 2 running Solaris 2.6 with Netscape Communicator 4.5.1.

Extract Downloaded Files

Go to the directory where you downloaded the files and extract each one.

Extract Java Plug-In Files:

```
zcat plugin-12-webstart-sparc.tar.Z | tar -xf -
```

Extract Patch Files for Solaris 2.6:

```
zcat JPI1.2-Patches-Solaris2.6-sparc.tar.Z | tar -xf -
```

Extract Netscape Navigator 4.5.1:

```
zcat NSCPcom_webstart_sparc.tar.Z | tar -xf -
```

Install Java Plug-In

The Java Plug-In download includes a user guide that you can view in your browser from the following directory:

```
plugin-12-webstart-sparc/Java_Plug-in_1.2.2/common/Docs/en/
Users_Guide_Java_Plug-in.html
```

The user guide explains how to install Java Plug-In. There are several easy ways to do it, and the command sequence below is one quick way that installs Java Plug-In in the default /opt/NSCPcom directory using the pkgadd command:

```
su
<root password>
cd ~/plugin-12-webstart-sparc
pkgadd -d ./Java_Plug-in_1.2.2/sparc/Product
```

Install Java Plug-In Patches

Before you can run Java Plug-In, you have to install the patches. You install the patches one at a time as root. The following command sequence goes to the patch directory, lists the files, and issues the command to install the first patch:

```
cd ~/JPI1.2-Patches-Solaris2.6-sparC
su
<password>
kq6py#ls
105210-19  105490-07  105568-13
kq6py#./105210-19/installpatch 105210-19
```

You will see this output when the patch is successfully installed:

```
Patch number 105210-19 has beenZ successfully installed.
See /var/sadm/patch/105210-19/log for details

Patch packages installed:
  SUNWarc
  SUNWcsu
```

Continue installing the patches one-by-one until all patches have successfully installed. The user's guide provides a list of required and suggested patches and links to where you can download additional suggested patches if you want to install them.

Install Netscape Communicator

The extracted Netscape Communicator 4.5.1 files provide a user's guide in the /home/monicap/NETSCAPE/Netscape_Communicator_4.51/common/Docs/en directory that explains the installation. The following command sequence is one easy way to do it with the pkgadd command. By default, the installation puts Netscape Communicator in the /opt/NSCPcom directory where your Java Plug-In and patches are also installed.

When you extracted the NSCPcom_webstart_sparc.tar.Z download, it placed the files in a NETSCAPE directory. From the NETSCAPE directory, execute the following command sequence:

```
cd ~/NETSCAPE/Netscape_Communicator_4.51/sparc/Product
su
<password>
pkgadd -d .
```

Check the Installation

There are two ways to check your Java Plug-In, patch, and Netscape Communicator installation.

1. Open the Netscape Help menu and select About Plug_Ins. You will see a list of Mime types. Check this list against the list presented in the user's guide. If your mime types are correct, the installation is correct and complete.
1. Start the control panel applet by loading the /opt/NSCPcom/j2pi/ControlPanel.html file, which contains the Control Panel applet. If the Control Panel applet starts, the installation is correct and complete. The control panel applet lets you change the default settings used by Java Plug-In at startup. All applets running inside Java Plug-In use these settings.

You can also run the Control Panel as an application like this:

```
cd /opt/NSCPcom/j2pi
ControlPanel &
```

Install the HTML Converter

Your browser will not automatically use the Java Plug-In when you load an HTML file with an applet. You have to download and run the Java Plug-In HTML Converter on the HTML page that invokes the applet to direct the applet ro run using the plug-in instead of the browser's default runtime.

Unzip the Java Plug-In HTML Converter download:

```
unzip htmlconv12.zip
```

Add the HTMLConverter.java program or its directory to your CLASSPATH.

Security Policy File

In the Java 2 platform, applets are restricted to a sandbox-like environment and need permission to access system resources outside their restricted environment. Applets are restricted to access operations within their local directory. All other access operations require permission.

Types of Policy Files

You need a policy file to grant access permissions to the Administration applet. If the applet runs on a disk other than the disk where the browser is running, the applet will also need to be signed. See Signed Applets (page 417) for information on signing and deploying applets.

There are three kinds of policy files: system, user, and program. The system policy file is located in jdk1.2/jre/lib/security/java.policy or jre1.2/lib/security/java.policy and contains permissions for everyone on the system.

The user policy file is located in the user's home directory. The user policy file provides a way to give certain users additional permissions over those granted to everyone on the system. The permissions in the system file are combined with the permissions in the user file.

A program policy file can be located anywhere. It is specifically named when an application is invoked with the java interpreter command or when an applet is invoked with applet viewer. When an application or applet is invoked with a specific policy file, the permissions in that policy file take the place of (are not combined with) permissions specified in the system or user policy file. Program policy files are used for program testing or intranet deployment of applets and applications.

Install the Policy File

Place the security policy file in your home directory and name it .java.policy. When the applet tries to perform an action that requires a policy file with a permission, the policy file is loaded from this directory and remains in effect until you exit and restart the browser. If an applet tries to perform an access operation without the right permission, it quietly quits without raising either an applet or browser error.

Change the Name or Location

You can change the name and/or location of the default system or user policy file. Edit the jdk1.2/jre/lib/security/java.security or jre1.2/lib/security/java.security file and add a third entry specifying the name and location of an alternative policy file.

```
policy.url.1=file:${java.home}/lib/security/java.policy
policy.url.2=file:${user.home}/.java.policy
policy.url.3=file:/<mypolicyfile path and name>
```

Run the Administration Applet

Copy the Java Archive (JAR) file with the Administration applet and policy file to its final location. In this example, that location is the /home/zelda/public_html directory. Next, extract the applet class file and policy file from the JAR file:

```
cp admin.jar /home/zelda/public_html/
jar xvf applet.jar
```

The extraction places the policy file under the public_html directory and creates an admin directory under the public_html directory with the applet class file in it. Make sure the policy file in the public_html directory is named .java.policy and copy it to your home directory.

In the public_html directory, create an HTML file that invokes the Administration applet class. Be sure to include the admin directory when you specify the applet class to the CODE option. Note that when using Java Plug-In, you cannot have the browser load the class file from the Java Archive (JAR) file.

```
<HTML>
<BODY>
<APPLET CODE=admin/AdminApplet.class
  WIDTH=550
  HEIGHT=150>
</APPLET>
</BODY>
</HTML>
```

Start the HTML Converter.

```
java HTMLConverter
```

In the HTML Converter graphical user interface shown in Figure 36, select One File, specify the path to the admin.html file, and click the Convert button. After the conversion completes, load the admin.html file in your browser.

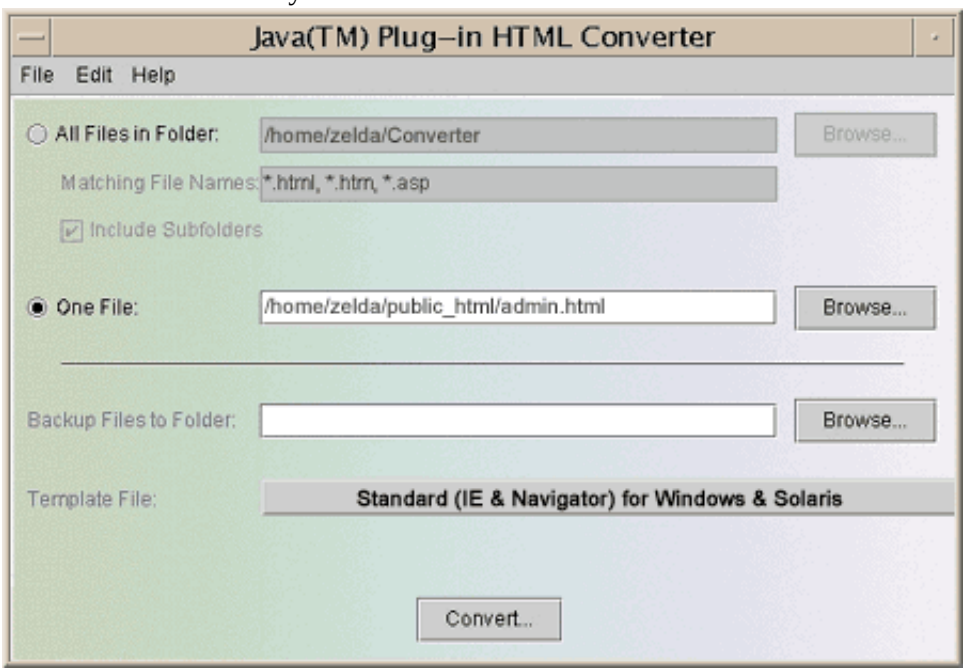


Figure 36 HTML Converter User Interface

Deploy to Win32 Platform

On Win32 platforms, Java Plug-In software is bundled with the Java 2 Runtime Environment. Java Plug-In lets web browsers use the Java 2 Runtime Environment to run 1.2-based applets and JavaBeans components instead of the web browser's default virtual machine. The Java Plug-In works with Netscape Communicator and Microsoft Internet Explorer.

This section explains how to install Java Plug-In with Netscape Communicator on the Win 32 operating system.

Get Downloads

To install and use the Java Runtime Environment with Java Plug-In, you need the following downloads. Put the downloads in a temporary directory.

- Java Runtime Environment with Java Plug-In for Win32 Platforms.

- Java Plug-In HTML Converter

Install JRE with Java Plug-In

An optionally installable version of the Java 2 Runtime Environment with Java Plug-In is included with the Java 2 SDK download. You can also download and install Java 2 Runtime Environment with Java Plug-In separately (`http://java.sun.com/products/jdk/1.2/jre/download-windows.html`).

Either way, install the Java 2 Runtime Environment with Java Plug-In by double-clicking its icon and following the installation instructions. When the installation completes, you will see the Java Plug-In control panel on your Windows Start menu under Programs.

Install the HTML Converter

Your browser will not automatically use the Java Plug-In when you load an HTML file with an applet. You have to download and run the Java Plug-In HTML Converter on the HTML page that invokes the applet to direct the applet ro run using the plug-in instead of the browser's default runtime.

Unzip the Java Plug-In HTML Converter download:

```
unzip htmlconv12.zip
```

Add the HTMLConverter.java program or its directory to your CLASSPATH.

Security Policy File

In the Java 2 platform, applets are restricted to a sandbox-like environment and need permission to access system resources outside their restricted environment. Applets are restricted to resource access operations within their local directory. All other access operations require permission.

Types of Policy Files

You need a policy file to grant access permissions to the Administration applet. If the applet runs on a disk other than the disk where the browser is running, the applet will also need to be signed. See Signed Applets (page 417) for information on signing and deploying applets.

There are three kinds of policy files: system, user, and program. The system policy file is located in jdk1.2\jre\lib\security\java.policy or jre1.2\lib\security\java.policy and contains permissions for everyone on the system.

The user policy file is located in the user's home directory. The user policy file provides a way to give certain users additional permissions over those granted to everyone on the system. The permissions in the system file are combined with the permissions in the user file.

A program policy file can be located anywhere. It is specifically named when an application is invoked with the java command or when an applet is invoked with applet viewer.

When an application or applet is invoked with a specific policy file, the permissions in that policy file take the place of (are not combined with) permissions specified in the system or user policy file. Program policy files are used for program testing or intranet deployment of applets and applications.

Install the Security Policy File

Place the security policy file in your home directory and make sure it is named java.policy. When the applet tries to perform an action that requires a policy file with a permission, the policy file is loaded from this directory and remains in effect until you exit and restart the browser. If an applet tries to perform an access operation without the right permission, it quietly quits without raising either an applet or browser error.

Change the Name or Location

You can change the name and/or location of the default system or user policy file. Edit the jdk1.2\jre\lib\security\java.security or jre1.2\lib\security\java.security file and add a third entry specifying the name and location of an alternative policy file.

```
policy.url.1=file:${java.home}\lib\security\java.policy
policy.url.2=file:${user.home}\java.policy
policy.url.3=file:\<mypolicyfile path and name>
```

Note: On Windows/NT machines, you might place the policy file in the C:\Winnt\Profiles\<userid>\java.policy directory.

Run the Administration Applet

Copy the Java Archive (JAR) file with the Administration applet and policy file to its final location. In this example, that location is the \home\zelda\public_html directory. Next, extract the applet class file and policy file from the JAR file:

```
copy admin.jar \home\zelda\public_html
jar xf applet.jar
```

The extraction places the policy file under public_html and creates an admin directory under the public_html directory with the applet class file in it. Rename the policy file in the public_html directory to java.policy and copy it to your home directory.

In the public_html directory, create an HTML file that invokes the Administration applet class. Be sure to include the admin directory when you specify the applet class to the CODE option. Note that when using Java Plug-In, you cannot have the browser load the class file from the Java Archive (JAR) file.

```
<HTML>
<BODY>
<APPLET CODE=admin/AdminApplet.class
  WIDTH=550
  HEIGHT=150>
</APPLET>
</BODY>
</HTML>
```

Start the HTML Converter.

```
java HTMLConverter
```

In the HTML Converter graphical user interface shown in Figure 37, select One File, specify the path to the admin.html file, and click the Convert button. After the conversion completes, load the admin.html file in your browser.

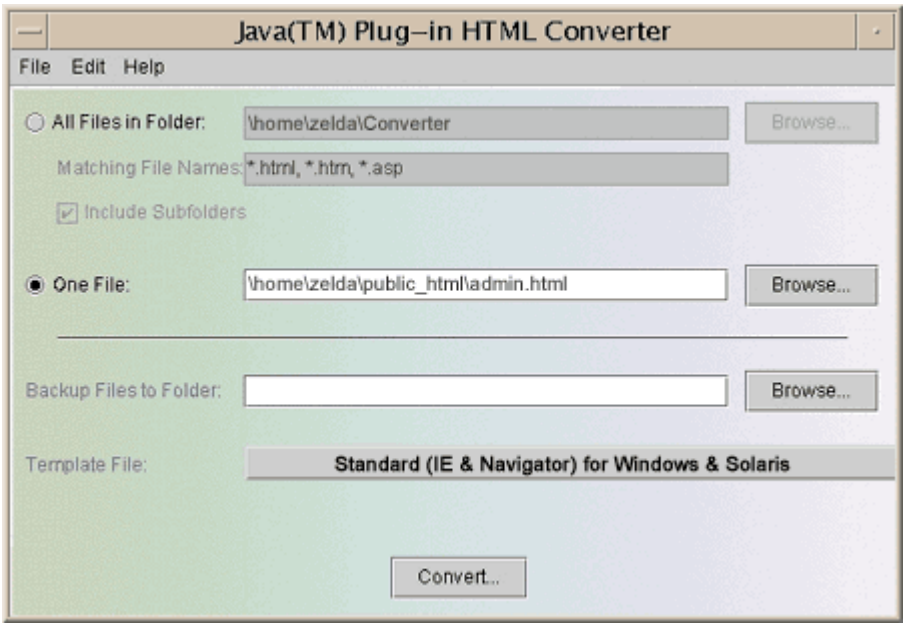


Figure 37 HTML Converter User Interface

How Does It Work?

On Windows machines, the Java Plug-In finds the Java Runtime Environment (JRE) by running the OLE custom control file beans.ocx installed by default in the \Program Files\JavaSoft\1.2\bin web browser directory. The OLE control examines the Windows registry to find the Java Plug-In key and uses the value associated with that key to find the installed JRE.

If you find that the wrong JRE is being loaded, use regedit to check the Java Plug-In registry values for the current user. If no JRE is installed, the control checks the Java Plug-in values for HKEY_LOCAL_MACHINE. You should see a value for Java Runtime Environment under Software\JavaSoft.

10: Signed Applets & Security Managers

This chapter concludes the book with security topics you should find useful--signing applets and writing a security manager. The examples do not relate directly to the auction house, but are simple and targeted to illustrate these concepts.

Covered in this Chapter

- Signed Applets (page 417)
- Writing a Security Manager (page 422)
- Code for this Chapter (page 426)

Signed Applets

A policy file can be defined to require a signature on all applets or applications that attempt to run with the policy file. The signature is a way to verify that the applet or application is from a reliable source and can be trusted to run with the permissions granted in the policy file.

If a policy file requires a signature, an applet or application can get the access granted by the policy file only if it has the correct signature. If the applet or application has the wrong signature or no signature, it will not get access to the file.

This section walks through an example of signing an applet, verifying the signature, and running the applet with a policy file.

Signed Applet Example

The policy file granting access can be set up to require or not require a signature. If a signature is required, the applet has to be bundled into a Java Archive (JAR) file before it can be

signed. This example shows you how to sign and grant permission to an applet so it can create demo.ini in the user's home directory when it executes in Applet Viewer.

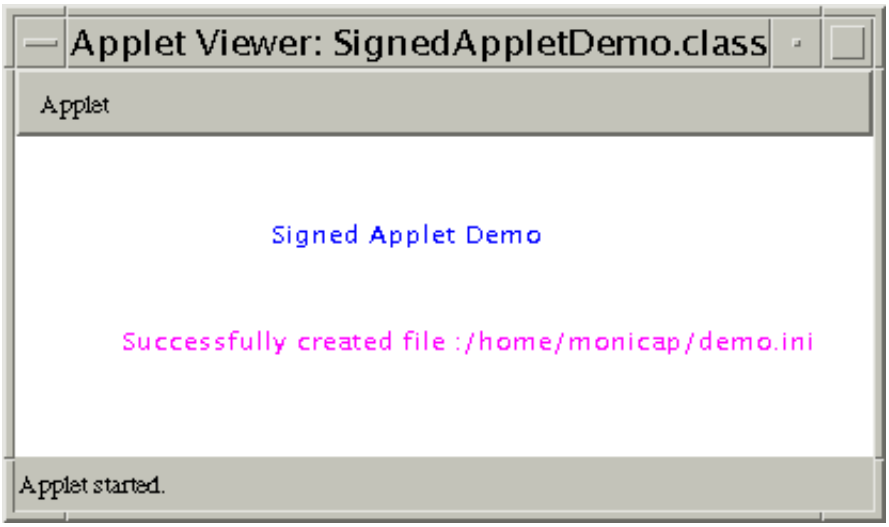


Figure 38 Signed Applet

These files are used for the example. You can copy them to or create them in your working directory.

- SignedAppletDemo (page 426) source file containing the applet code
- `Write.jp` policy file granting access to the user's home directory

```
/* AUTOMATICALLY GENERATED ON Mon Sep 14 09:55:03 PDT 1998*/
/* DO NOT EDIT */

keystore "raystore";

grant signedBy "susan" {
  permission java.util.PropertyPermission "user.home", "read";
  permission java.io.FilePermission "${user.home}/newfile", "write";
};
```

- Applet tag embedded in the SignedApplet.html file:

```
<applet code="SignedAppletDemo.class"
     archive="SSignedApplet.jar"
     width=400 height=400>
     <param name=file value="/etc/inet/hosts">
</applet>
```

Usually an applet is bundled and signed by an intranet developer and handed off to the end user who verifies the signature and runs the applet. In this example, the intranet developer performs Steps 1 through 5 and Ray, the end user, performs Steps 6 through 8. But, to keep things simple for this example, all steps occur in the same working directory.

1. Compile the applet
2. Create a JAR file
3. Generate Keys
4. Sign the JAR file
5. Export the Public Key Certificate
6. Import the Certificate as a Trusted Certificate
7. Create the policy file
8. Run the applet

Intranet Developer

Susan, the intranet developer, bundles the applet executable in a JAR file, signs the JAR file, and exports the public key certificate.

1: Compile the Applet

In her working directory, Susan uses the javac command to compile the SignedAppletDemo.java class. The output from the javac command is the SignedAppletDemo.class.

```
javac SignedAppletDemo.java
```

2: Make a JAR File

Susan then stores the compiled SignedAppletDemo.class file into a JAR file. The cvf option to the jar command creates a new archive (c), using verbose mode (v), and specifies the archive file name (f). The archive file name is SignedApplet.jar.

```
jar cvf SignedApplet.jar SignedAppletDemo.class
```

3: Generate Keys

A JAR file is signed with the private key of the creator of the JAR file and the signature is verified by the recipient of the JAR file with the public key in the pair. The certificate is a statement from the owner of the private key that the public key in the pair has a particular value so the person using the public key can be assured the public key is authentic. Public and private keys must already exist in the keystore database before jarsigner can be used to sign or verify the signature on a JAR file.

Susan creates a keystore database named compstore that has an entry for a newly generated public and private key pair with the public key in a certificate using the keytool command. In her working directory, Susan creates a keystore database and generates the keys:

```
keytool -genkey -alias signFiles -keystore compstore
        -keypass kpi135 -dname "cn=jones" -storepass ab987c
```

The above keytool -genkey command invocation generates a key pair that is identified by the alias signFiles. Subsequent keytool command invocations use this alias and the key password (-keypass kpi135) to access the private key in the generated pair.

The generated key pair is stored in a keystore database called compstore (-keystore compstore) in the current directory, and accessed with the compstore password (-storepass ab987c).

The -dname “cn=jones” option specifies an X.500 Distinguished Name with a commonName (cn) value. X.500 Distinguished Names identify entities for X.509 certificates. In this example, Susan uses her last name, Jones, for the common name. She could use any common name that suits her purposes. You can view all keytool options and parameters by typing:

```
keytool -help
```

4: Sign the JAR File

JAR Signer is a command line tool for signing and verifying the signature on JAR files. In her working directory, Susan uses jarsigner to make a signed copy of the SignedApplet.jar file.

```
jarsigner -keystore compstore -storepass ab987c -keypass kpi135
          -signedjar SSignedApplet.jar SignedApplet.jar signFiles
```

The -storepass ab987c and -keystore compstore options specify the keystore database and password where the private key for signing the JAR file is stored. The -keypass kpi135 option is the password to the private key, SSignedApplet.jar is the name of the signed JAR file, and signFiles is the alias to the private key. jarsigner extracts the certificate from the keystore whose entry is signFiles and attaches it to the generated signature of the signed JAR file.

5: Export the Public Key Certificate

The public key certificate is sent with the JAR file to the end user who will be using the applet. That person uses the certificate to authenticate the signature on the JAR file. A certificate is sent by exporting it from the compstore database.

In her working directory, Susan uses keytool to copy the certificate from compstore to a file named CompanyCer.cer as follows:

```
keytool -export -keystore compstore -storepass ab987c -
alias signFiles -file CompanyCer.cer
```

As the last step, Susan posts the JAR and certificate files to a distribution directory on a web page.

End User

Ray, the end user, downloads the JAR file from the distribution directory, imports the certificate, creates a policy file granting the applet access, and runs the applet.

6: Import Certificate as a Trusted Certificate

Ray downloads SSignedApplet.jar and CompanyCer.cer to his home directory. Ray must now create a keystore database (raystore) and import the certificate into it using the alias company. Ray uses keytool in his home directory to do this:

```
keytool -import -alias company -file CompanyCer.cer
                      -keystore raystore -storepass abcdefgh
```

7: Create the Policy File

The policy file grants the SSignedApplet.jar file signed by the alias company permission to create demo.ini (and no other file) in the user's home directory.

Ray creates the policy file in his home directory using either policytool or an ASCII editor.

```
keystore "/home/ray/raystore";
// A sample policy file that lets a program
// create demo.ini in user's home directory
// Satya N Dodda
grant SignedBy "company"
{
   permission java.util.PropertyPermission "user.home", "read";
   permission java.io.FilePermission "${user.home}/demo.ini", "write";
};
```

8: Run the Applet in Appletviewer

The appletviewer tool connects to the HTML documents and resources specified in the call to appletviewer, and displays the applet in its own window. To run the example, Ray copies the signed JAR file and HTML file to /home/aURL/public_html and invokes Applet viewer from his home directory as follows:

```
appletviewer -J-Djava.security.policy=Write.jp http://aURL.com/
SignedApplet.html
```

Note: Type everything on one line and put a space after Write.jp

---

The -J-Djava.security.policy=Write.jp option tells Applet Viewer to run the applet referenced in the SignedApplet.html file with the Write.jp policy file.

Note: The Policy file can be stored on a server and specified in the appletviewer invocation as a URL.

Running an Application with a Policy File

This application invocation restricts MyProgram to a sandbox-like environment the same way applets are restricted, but allows access as specified in the polfile policy file.

```
java -Djava.security.manager -Djava.security.policy=polfile MyProgram
```

Signed Applets in JDK 1.1

JDK 1.1 signed applets can access local system resources if the local system is properly set up to allow it. See the JDK 1.1 Signed Applet Example page (`http://java.sun.com/security/signExample/index.html`) for details.

Writing a Security Manager

A security manager is a Java virtual machine object that implements a security policy. By default, the Java 2® platform software provides a security manager that disallows all access to local system resources apart from read access to the directory and its subdirectories where the program is invoked.

You can extend the default security manager to implement customized verifications and approvals for applets and applications, but the implementation must include the appropriate access verification code for every check<*type*> method you override. If you do not include this code, no access verification check happens, and your code breaches system security policy.

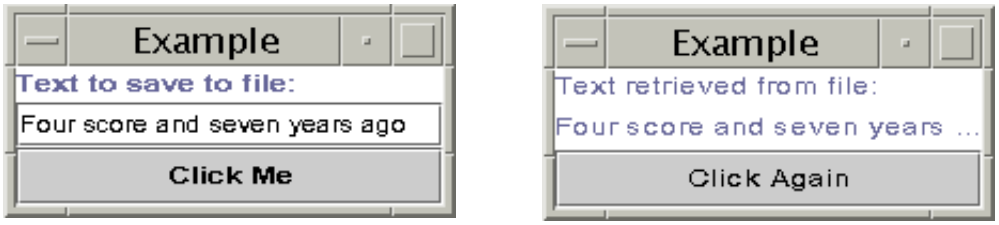
This section uses an example application to explain how to write a custom security manager that prompts the end user for password identification before reading from and writing to spe-

cific files. The implementation includes access verification code so once the end user makes it through the password check, he or she still needs the file read and write permissions in his or her policy file.

The example consists of the FileIO application, and the PasswordSecurityManager program that provides the custom security manager implementation.

The FileIO Program

The FileIO (page 428) program displays a simple user interface and asks the end user to enter some text. When the end user clicks the Click Me button, the text is saved to a file in the end user's home directory, and a second file is opened and read. The text read from the second file is displayed to the end user.



Before Button Click After Button Click

Figure 39 FileIO Program

The custom security manager for this program prompts the end user to enter a password before it allows `FileIO` to write text to or read text from a file. The `main` method of `FileIO` creates a custom security manager called `PasswordSecurityManager`.

```
public static void main(String[] args){
  BufferedReader buffy = new BufferedReader(
      new InputStreamReader(System.in));
  try {
    System.setSecurityManager(
      new PasswordSecurityManager("pwd", buffy));
  } catch (SecurityException se) {
    System.err.println("SecurityManager already set!");
  }
```

The PasswordSecurityManager Class

The PasswordSecurityManager (page 427) class declares two private instance variables, which are initialized by the constructor when the custom security manager is installed. The

password instance variable contains the actual password, and the buffy instance variable is an input buffer that stores the end user's password input.

```
public class PasswordSecurityManager extends SecurityManager{
 private String password;
  private BufferedReader buffy;

  public PasswordSecurityManager(String p, BufferedReader b){
    super();
    this.password = p;
    this.buffy = b;
 }
```

The accessOK method prompts the end user for a password, verifies the password, and returns true if the password is correct and false if it is not.

```
private boolean accessOK() {
  int c;
  String response;

  System.out.println("Password, please:");
  try {
    response = buffy.readLine();
    if (response.equals(password))
      return true;
    else
      return false;
  } catch (IOException e) {
    return false;
  }
}
```

Verify Access

The SecurityManager parent class provides methods to verify file system read and write access. The checkRead and checkWrite methods each have a version that accepts a String and another version that accepts a file descriptor. This example overrides only the String versions to keep the example simple, and because the FileIO program accesses directories and files as Strings.

```
//API Ref :void checkRead(String filename)
  public void checkRead(String filename) {
    if((filename.equals(File.separatorChar + "home" +
                        File.separatorChar + "monicap" +
                        File.separatorChar + "text2.txt"))){
    if(!accessOK()){
      super.checkRead(filename);
      throw new SecurityException("No Way!");
    } else {
      FilePermission perm = new FilePermission(
                            File.separatorChar + "home" +
```

```
                                File.separatorChar + "monicap" +
                                File.separatorChar + "text2.txt", "read");
                                checkPermission(perm);
            }
          }
      }
//API Ref :void checkWrite(String filename)
      public void checkWrite(String filename) {
        if((filename.equals(File.separatorChar + "home" +
          File.separatorChar + "monicap" +
          File.separatorChar + "text.txt"))){
          if(!accessOK()){
            super.checkWrite(filename);
            throw new SecurityException("No Way!");
          } else {
            FilePermission perm = new FilePermission(
            File.separatorChar + "home" +
            File.separatorChar + "monicap" +
            File.separatorChar + "text.txt" , "write");
//API Ref :void checkPermission(Permission perm)
          checkPermission(perm);
          }
        }
       }
      }
```

The checkWrite method is called before the end user input is written to the output file. This is because the `FileOutputStream` class calls SecurityManager.checkWrite first.

The custom implementation for `SecurityManager.checkWrite` tests for the pathname `/home/monicap/text.txt`, and if `true` prompts the end user for the password. If the password is correct, the `checkWrite` method performs the access check by creating an instance of the required permission and passing it to the `SecurityManager.checkPermission` method. This check will succeed if the security manager finds a system, user, or program policy file with the specified permission. Once the write operation completes, the end user is prompted for the password two more times. The first time to read the `/home/monicap` directory, and the second time to read the `text2.txt` file. An access check is performed before the read operation takes place.

Policy File

Here is the policy file the `FileIO` program needs for its read and write operations. It also grants permission to the custom security manager to access the event queue on behalf of the application and show the application window without the warning banner.

```
grant {
  permission java.io.FilePermission "${user.home}/text.txt", "write";
  permission java.util.PropertyPermission "user.home", "read";
  permission java.io.FilePermission "${user.home}/text2.txt", "read";
```

```
  permission java.awt.AWTPermission "accessEventQueue";
  permission java.awt.AWTPermission "showWindowWithoutWarningBanner";
};
```

Run the FileIO Program

Here is how to run the `FileIO` program with the policy file:

```
java -Djava.security.policy=polfile FileIO
```

Reference Information

A: Security and Permissions (page 431) describes the available permissions and explains the consequences of granting permissions. One way to use this information is to help you limit what permissions a given applet or application might need to successfully execute. Another way to use this information is to educate yourself on the ways in which a particular permission can be exploited by malicious code.

B: Classes, Methods, and Permissions (page 447) provides lists of Java 2 platform software methods that are implemented to perform security access checks, the permission each requires, and the `java.security.SecurityManager` method called to perform the access check. You can use this reference to write your own security manager implementations or when you implement abstract methods that perform security-related tasks.

C: Security Manager Methods (page 465) lists the permissions checked for by the `SecurityManager` methods.

Code for this Chapter

- SignedAppletDemo (page 426)
- PasswordSecurityManager (page 427)
- FileIO (page 428)

SignedAppletDemo

```
//File: @(#)SignedAppletDemo.java
//(#)author:  Satya Dodda
import java.applet.Applet;
import java.awt.Graphics;
import java.io.*;
import java.awt.Color;
public class SignedAppletDemo extends Applet {
    public String test() {
      setBackground(Color.white);
```

```
        String fileName = System.getProperty("user.home") +
             System.getProperty("file.separator") + "demo.ini";
        String msg  = "This message was written by a signed applet!!!\n";
        String s ;
        try {
          FileWriter fos = new FileWriter(fileName);
          fos.write(msg, 0, msg.length());
          fos.close();
          s = new String("Successfully created file :" + fileName);
        } catch (Exception e) {
          System.out.println("Exception e = " + e);
          e.printStackTrace();
          s = new String("Unable to create file :  " + fileName);
        }
      return s;
    }
     public void paint(Graphics g) {
         g.setColor(Color.blue);
         g.drawString("Signed Applet Demo", 120, 50);
         g.setColor(Color.magenta);
         g.drawString(test(), 50, 100);
     }
}
```

PasswordSecurityManager

```
import java.io.*;
import java.security.AccessController;

public class PasswordSecurityManager extends SecurityManager {
  private String password;
  private BufferedReader buffy;
  public PasswordSecurityManager(String p, BufferedReader b) {
    super();
    this.password = p;
    this.buffy = b;
  }
  private boolean accessOK() {
    int c;
    String response;
    System.out.println("What's the secret password?");
    try {
      response = buffy.readLine();
      if(response.equals(password)) {
         return true;
      } else {
        return false;
      }
    } catch (IOException e) {
        return false;
    }
  }
```

```
  public void checkRead(String filename) {
//Mention file by name so don't get prompted for password
//for everything the application loads to create itself
    if((filename.equals(File.separatorChar +
     "home" + File.separatorChar + "monicap" + File.separatorChar + "text2.txt"))){
       if(!accessOK()){
         super.checkRead(filename);
         throw new SecurityException("No Way!");
       } else {
         FilePermission perm = new FilePermission(File.separatorChar +
                                "home" + File.separatorChar +
                               "monicap" + File.separatorChar +
                               "text2.txt", "read");
         checkPermission(perm);
       }
    }
  }
  public void checkWrite(String filename) {
//Mention file by name so don't get prompted for password
//for everything the application loads to create itself
    if((filename.equals(File.separatorChar + "home" +
                        File.separatorChar + "monicap"
                        + File.separatorChar + "text.txt"))){
      if(!accessOK()){
        super.checkWrite(filename);
        throw new SecurityException("No Way!");
      } else {
        FilePermission perm = new FilePermission(File.separatorChar +
                                 "home" + File.separatorChar + "monicap" +
                                 File.separatorChar + "text.txt" , "write");
        checkPermission(perm);
      }
    }
  }
}
```

FileIO

```
import java.awt.Color;
import java.awt.BorderLayout;
import java.awt.event.*;
import javax.swing.*;
import java.io.*;

class FileIO extends JFrame implements ActionListener {
  JLabel text;
  JButton button;
  JPanel panel;
  JTextField textField;
  private boolean _clickMeMode = true;

  FileIO() { //Begin Constructor
```

```
    text = new JLabel("Text to save to file:");
    button = new JButton("Click Me");
    button.addActionListener(this);
    textField = new JTextField(20);
    panel = new JPanel();
    panel.setLayout(new BorderLayout());
    panel.setBackground(Color.white);
    getContentPane().add(panel);
    panel.add(BorderLayout.NORTH, text);
    panel.add(BorderLayout.CENTER, textField);
    panel.add(BorderLayout.SOUTH, button);
  } //End Constructor

  public void actionPerformed(ActionEvent event){
    Object source = event.getSource();
//The equals operator (==) is one of the few operators
//allowed on an object in the Java programming language
    if(source == button) {
      JLabel label = new JLabel();
      //Write to file
      if(_clickMeMode){
        try {
          String text = textField.getText();
          byte b[] = text.getBytes();
          String outputFileName = System.getProperty("user.home",
                                  File.separatorChar + "home" +
                                  File.separatorChar + "zelda") +
                                  File.separatorChar + "text.txt";
          FileOutputStream out = new FileOutputStream(outputFileName);
          out.write(b);
          out.close();
        } catch(java.io.IOException e) {
          System.out.println("Cannot write to text.txt");
        }
      //Read from file
        try {
          String inputFileName = System.getProperty("user.home",
                          File.separatorChar + "home" +
                          File.separatorChar + "zelda") +
                          File.separatorChar + "text.txt";
          FileInputStream in = new FileInputStream(inputFileName);
          byte bt[] = new byte[(int)inputFileName.length()];
          in.read(bt);
          String s = new String(bt);
          label.setText(s);
          in.close();
        } catch(java.io.IOException e) {
          System.out.println("Cannot read from text.txt");
        }
//Display text read from file
        text.setText("Text retrieved from file:");
        button.setText("Click Again");
        _clickMeMode = false;
```

```
      } else {
//Save text to file
        text.setText("Text to save to file:");
        textField.setText("");
        button.setText("Click Me");
        _clickMeMode = true;
      }
    }
  }
  public static void main(String[] args){
    FileIO frame = new FileIO();
    frame.setTitle("Example");
    WindowListener l = new WindowAdapter() {
      public void windowClosing(WindowEvent e) {
        System.exit(0);
      }
    };
    frame.addWindowListener(l);
    frame.pack();
    frame.setVisible(true);
  }
}
```

A: Security and Permissions

All applets and any applications invoked with a security manager must be granted explicit permission to access local system resources apart from accessing the directory where the program is invoked and its subdirectories. The Java™ platform provides permissions to allow various levels of access to different types of local information. Because permissions let an applet or application override the default security policy, you should be very careful when you assign permissions to not create an opening for malicious code to attack your system.

This appendix describes the available permissions and explains how each permission can create an opening for malicious attacks. One way to use this information is to help you limit what permissions a given applet or application might need to successfully execute. Another way to use this information is to educate yourself on the ways in which a particular permission can be exploited by malicious code so you can take steps against that.

As a safeguard, never trust an unknown applet or application. Always check the code carefully against the information in this appendix to be sure you are not giving malicious code permission to cause serious problems on the local system.

Covered in this Appendix

- Overview (page 432)
- Knowing Which Permissions (page 432)
- AllPermission (page 433)
- AWTPermission (page 433)
- FilePermission (page 434)
- NetPermission (page 435)
- PropertyPermission (page 436)
- ReflectPermission (page 437)
- RuntimePermission (page 437)
- SecurityPermission (page 440)
- SerializablePermission (page 443)
- SocketPermission (page 444)

Overview

Permissions are granted to a program with a policy file. A policy file contains permissions for specific access. A permission consists of the permission name, a target, and in some cases, a comma-separated list of actions.

For example, the following policy file entry specifies a java.io.FilePermission permission that grants read access (the action) to the ${user.home}/text2.txt target.

```
grant {
  permission java.io.FilePermission "${user.home}/text2.txt", "read";
};
```

There is one policy file for the Java platform installation (system) and an optional policy file for each user. The system policy file is in {java.home}/lib/security/java.policy, and the user policy file is in each user's home directory. The system and user policy files are combined. So, for example, there could be a system policy file with very few permissions granted to all users on the system, and individual policy files granting additional permissions to certain users.

To run an application with the security manager and a policy file named polfile in the user's home directory, type:

```
java -Djava.security.main  -DJava.security.policy=polfile  FileIO
```

To run an applet in appletviewer with a policy file named polfile in the user's home directory, type:

```
appletviewer -J-Djava.security.policy=polfile fileIO.html
```

When running an applet in a browser, the browser looks for the user and system policy files to find the permissions the applet needs to access local system resources on behalf of the user who downloaded the applet.

Knowing Which Permissions

When you run an applet or invoke an application with a security manager that needs permissions, you will get a stack trace if you did not provide a policy file with all the needed permissions. The stack trace contains the information you need to add the permission to the policy file. If the program needs additional permissions, you will keep getting stack traces until all required permissions are added to the policy file. The only drawback to this approach is you have to try every possible code path in your application.

Another way to determine which permission your program needs is to browse B: Classes, Methods, and Permissions (page 447). This appendix tells you which Java 2 platform software methods are prevented from executing without the listed permission. The information is also useful for developers who want to write their own security manager to customize the verifications and approvals needed in a program.

Here is a short example to show how to translate the first couple of lines in a stack trace to a policy file entry. The first line tells you access is denied. This means this stack trace was generated because the program tried to access a system resource without the proper permission. The second line means you need a java.net.SocketPermission that gives the program permission to connect to and resolve the host name for Internet Protocol (IP) address 129.144.176.176, port 1521.

```
java.security.AccessControlException: access denied
     (java.net.SocketPermission 129.144.176.176:1521 connect,resolve)
```

To turn this into a policy file entry, list the permission name, a target, and an action list as follows where java.net.SocketPermission is the permission name, 129.144.176.176:1521 is the target, and connect,resolve is the action list.

```
grant {
permission java.net.SocketPermission  "129.144.176.176:1521",
                                   "connect,resolve";
};
```

AllPermission

java.security.AllPermission specifies all permissions in the system for all possible targets and actions. This permission should be used only during testing because it grants permission to run with all security restrictions disabled as if there were no security manager.

```
grant {
  permission java.security.AllPermission;
};
```

AWTPermission

java.awt.AWTPermission grants access to the following Abstract Window Toolkit (AWT) targets. The possible targets are listed by name with no action list.

```
grant {
  permission java.awt.AWTPermission "accessClipboard";
  permission java.awt.AWTPermission "accessEventQueue";
  permission java.awt.AWTPermission "showWindowWithoutWarningBanner";
};
```

accessClipboard: This target grants permission to post information to and retrieve information from the AWT clipboard. Granting this permission could allow malicious code to share potentially sensitive or confidential information.

accessEventQueue: This target grants permission to access the AWT event queue. Granting this permission could allow malicious code to peek at and remove existing events from the system, or post bogus events that could cause the application or applet to perform malicious actions.

listenToAllAWTEvents: This target grants permission to listen to all AWT events throughout the system. Granting this permission could allow malicious code to read and exploit confidential user input such as passwords.

Each AWT event listener is called from within the context of that event queue's EventDispatchThread, so if the accessEventQueue permission is also enabled, malicious code could modify the contents of AWT event queues throughout the system, which can cause the application or applet to perform unintended and malicious actions.

readDisplayPixels: This target grants permission to read pixels back from the display screen. Granting this permission could allow interfaces such as java.awt.Composite that allow arbitrary code to examine pixels on the display to include malicious code that snoops on user activities.

showWindowWithoutWarningBanner: This target grants permission to display a window without also displaying a banner warning that the window was created by an applet. Without this warning, an applet might pop up windows without the user knowing they belong to an applet. This could be a problem in environments where users make security-sensitive decisions based on whether the window belongs to an applet or an application. For example, disabling the banner warning might trick the end user into entering sensitive user name and password information.

FilePermission

java.io.FilePermission grants access to a file or directory. The targets consist of the target pathname and a comma-separated list of actions.

This policy file grants read, write, delete, and execute permission to all files.

```
grant {
  permission java.io.FilePermission
          "<<ALL FILES>>", "read, write, delete, execute";
};
```

This policy file grants read and write permission to text.txt in the user's home directory.

```
grant {
  permission java.io.FilePermission
                  "${user.home}/text.txt", "read, write";
};
```

You can use the following wild cards to specify the target pathname.

- A pathname that ends in /* where /* is the file separator character, indicates a directory and all the files contained in that directory.
- A pathname that ends with /- indicates a directory, and recursively, all files and subdirectories contained in that directory.
- A pathname consisting of a single asterisk (*) indicates all files in the current directory.
- A pathname consisting of a single dash (-) indicates all files in the current directory, and recursively, all files and subdirectories contained in the current directory.

The actions are specified in a list of comma-separated keywords and have the following meanings:

- read: Permission to read a file or directory.
- write: Permission to write to and create a file or directory.
- execute: Permission to execute a file or search a directory.
- delete: Permission to delete a file or directory.

When granting file permissions, always think about the implications of granting read and especially write access to various files and directories. The <<ALL FILES>> permission with write action is especially dangerous because it grants permission to write to the entire file system. This means the system binary can be replaced, which includes the Java1 Virtual Machine (VM) runtime environment.

NetPermission

java.net.NetPermission grants access to various network targets. The possible targets are listed by name with no action list.

```
grant {
  permission java.net.NetPermission "setDefaultAuthenticator";
  permission java.net.NetPermission "requestPasswordAuthentication";
};
```

setDefaultAuthenticator: This target grants permission to set the way authentication information is retrieved when a proxy or HTTP server asks for authentication. Granting this permission could mean malicious code can set an authenticator that monitors and steals user authentication input as it retrieves the input from the user.

requestPasswordAuthentication: This target grants permission to ask the authenticator registered with the system for a password. Granting this permission could mean malicious code might steal the password.

specifyStreamHandler: This target grants permission to specify a stream handler when constructing a Uniform Resource Locator (URL). Granting this permission could mean malicious code might create a URL with resources to which it would not normally have access, or specify a stream handler that gets the actual bytes from somewhere to which it does have access. This means the malicious code could trick the system into creating a ProtectionDomain/CodeSource for a class even though the class really did not come from that location.

PropertyPermission

java.util.PropertyPermission grants access to system properties. The java.util.Properties class represents persistent settings such as the location of the installation directory, the user name, or the user's home directory.

```
grant {
  permission java.util.PropertyPermission "java.home", "read";
  permission java.util.PropertyPermission "os.name", "write";
  permission java.util.PropertyPermission "user.name", "read, write";
};
```

The target list contains the name of the property; for example, java.home or os.name. The naming convention for the properties follows the hierarchical property naming convention, and includes wild cards. An asterisk at the end of the property name, after a dot (.), or alone signifies a wild card match. For example, java.* or * are valid, but *java or a*b are invalid.

The actions are specified in a list of comma-separated keywords, and have the following meanings:

- read: Permission to read (get) a property.

- write: Permission to write (set) a property.

Granting property permissions can leave your system open to intrusion. For example, granting permission to access the java.home property makes the installation directory vulnerable to attack, and granting permission to access the user.name and user.home properties might reveal the user's account name and home directory to code that might misuse the information.

ReflectPermission

java.lang.reflect.ReflectPermission grants permission for various reflective operations. The possible targets are listed by name with no action list.

```
grant {
 permission java.lang.reflect.ReflectPermission "suppressAccessChecks";
};
```

suppressAccessChecks: This target grants permission to access fields and invoke methods in a class. This includes public, protected, and private fields and methods. Granting this permission could reveal confidential information and make normally unavailable methods accessible to malicious code.

RuntimePermission

java.lang.RuntimePermission grants access to various runtime targets such as the class loader, Java VM, and thread. The possible targets are listed by name with no action list.

```
grant {
  permission java.lang.RuntimePermission "createClassLoader";
  permission java.lang.RuntimePermission "getClassLoader";
  permission java.lang.RuntimePermission "exitVM";
  permission java.lang.RuntimePermission "setFactory";
  permission java.lang.RuntimePermission "setIO";
  permission java.lang.RuntimePermission "modifyThread";
  permission java.lang.RuntimePermission "modifyThreadGroup";
  permission java.lang.RuntimePermission "getProtectionDomain";
  permission java.lang.RuntimePermission "setProtectionDomain";
  permission java.lang.RuntimePermission "readFileDescriptor";
  permission java.lang.RuntimePermission "writeFileDescriptor";
  permission java.lang.RuntimePermission "loadLibrary.<library name>";
  permission java.lang.RuntimePermission
                          "accessClassInPackage.<package name>";
  permission java.lang.RuntimePermission
```

```
                              "defineClassInPackage.<package name>";
    permission java.lang.RuntimePermission
                               "accessDeclaredMembers.<class name>";
    permission java.lang.RuntimePermission "queuePrintJob";
};
```

The naming convention for target information where a library, package, or class name is added follows the hierarchical property naming convention, and includes wild cards. An asterisk at the end of the target name, after a dot (.), or alone signifies a wild card match. For example, loadLibrary.* or * are valid, but *loadLibrary or a*b are not.

createClassLoader: This target grants permission to create a class loader. Granting this permission might allow a malicious application to instantiate its own class loader and load harmful classes into the system. Once loaded, the class loader could place these classes into any protection domain and give them full permissions for that domain.

getClassLoader: This target grants permission to retrieve the class loader for the calling class. Granting this permission could enable malicious code to get the class loader for a particular class and load additional classes.

setContextClassLoader: This target grants permission to set the context class loader used by a thread. System code and extensions use the context class loader to look up resources that might not exist in the system class loader. Granting this permission allows code to change which context class loader is used for a particular thread, including system threads. This can cause problems if the context class loader has malicious code.

setSecurityManager: This target grants permission to set or replace the security manager. The security manager is a class that allows applications to implement a security policy. Granting this permission could enable malicious code to install a less restrictive manager, and thereby, bypass checks that would have been enforced by the original security manager.

createSecurityManager: This target grants permission to create a new security manager. Granting this permission could give malicious code access to protected and sensitive methods that might disclose information about other classes or the execution stack. It could also allow the introduction of a weakened security manager.

exitVM: This target grants permission to halt the Java VM. Granting this permission could allow malicious code to mount a denial-of-service attack by automatically forcing the VM to stop.

setFactory: This target grants permission to set the socket factory used by the ServerSocket or Socket class, or the stream handler factory used by the URL class. Granting this permission allows code to set the actual implementation for the socket, server socket, stream handler, or Remote Method Invocation (RMI) socket factory. An attacker might set a faulty implementation that mangles the data stream.

setIO: This target grants permission to change the value of the System.out, System.in, and System.err standard system streams. Granting this permission could allow an attacker to change System.in to steal user input, or set System.err to a null output stream, which would hide any error messages sent to System.err.

modifyThread: This target grants permission to modify threads by calls to the stop, suspend, resume, setPriority, and setName methods in the Thread class. Granting this permission could allow an attacker to start or suspend any thread in the system.

stopThread: This target grants permission to stop threads. Granting this permission allows code to stop any thread in the system provided the code already has permission to access that thread. Malicious code could corrupt the system by killing existing threads.

modifyThreadGroup: This target grants permission to modify threads by calls to the destroy, resume, setDaemon, setmaxPriority, stop, and suspend methods of the ThreadGroup class. Granting this permission could allow an attacker to create thread groups and set their run priority.

getProtectionDomain: This target grants permission to retrieve the ProtectionDomain instance for a class. Granting this permission allows code to obtain policy information for that code source. While obtaining policy information does not compromise the security of the system, it does give attackers additional information, such as local file names for example, to better aim an attack.

readFileDescriptor: This target grants permission to read file descriptors. Granting this permission allows code to read the particular file associated with the file descriptor, which is dangerous if the file contains confidential data.

writeFileDescriptor: This target grants permission to write file descriptors. Granting this permission allows code to write to the file associated with the descriptor, which is dangerous if the file descriptor points to a local file.

loadLibrary.{library name}: This target grants permission to dynamically link the specified library. Granting this permission could be dangerous because the security architecture is not designed to and does not extend to native code loaded by way of the java.lang.System.loadLibrary method.

accessClassInPackage.{package name}: This target grants permission to access the specified package by way of a class loader's loadClass method when that class loader calls the SecurityManager.checkPackageAcesss method. Granting this permission gives code access to classes in packages to which it normally does not have access. Malicious code may use these classes to help in its attempt to compromise security in the system.

defineClassInPackage.{package name}: This target grants permission to define classes in the specified package by way of a class loader's defineClass method when that class loader

calls the SecurityManager.checkPackageDefinition method. Granting this permission allows code to define a class in a particular package, which can be dangerous because malicious code with this permission might define rogue classes in trusted packages like java.security or java.lang, for example.

accessDeclaredMembers: This target grants permission to access the declared members of a class. Granting this permission allows code to query a class for its public, protected, default (package), and private fields and methods. Although the code would have access to the private and protected field and method names, it would not have access to the private and protected field data and would not be able to invoke any private methods. Nevertheless, malicious code may use this information to better aim an attack. Additionally, malicious code might invoke any public methods or access public fields in the class, which could be dangerous if the code would normally not be able to invoke those methods or access the fields because it cannot cast the object to the class or interface with those methods and fields.

queuePrintJob: This target grants permission to initiate a print job request. Granting this permission could allow code to print sensitive information to a printer or maliciously waste paper.

SecurityPermission

java.security.SecurityPermission grants access to various security configuration parameters. The possible targets are listed by name with no action list. Security permissions currently apply to methods called on the following objects:

- java.security.Policy, which represents the system security policy for applications.
- java.security.Security, which centralizes all security properties and common security methods. It manages providers.
- java.security.Provider, which represents an implementation of such things as security algorithms (DSA, RSA, MD5, or SHA-1) and key generation.
- java.security.Signer, which manages private keys. Even though, Signer is deprecated, the related permissions are available for backwards compatibility.
- java.security.Identity, which manages real-world objects such as people, companies, or organizations whose identities can be authenticated using their public keys.

```
grant {
  permission java.security.SecurityPermission "getPolicy";
  permission java.security.SecurityPermission "setPolicy";
  permission java.security.SecurityPermission "getProperty.os.name";
  permission java.security.SecurityPermission "setProperty.os.name";
  permission java.security.SecurityPermission “insertProvider.SUN";
  permission java.security.SecurityPermission "removeProvider.SUN";
```

```
    permission java.security.SecurityPermission "setSystemScope";
    permission java.security.SecurityPermission "setIdentityPublicKey";
    permission java.security.SecurityPermission "setIdentityInfo";
    permission java.security.SecurityPermission "addIdentityCertificate";
    permission java.security.SecurityPermission
                              "removeIdentityCertificate";
    permission java.security.SecurityPermission
                               "clearProviderProperties.SUN";
    permission java.security.SecurityPermission
                              "putProviderProperty.<provider name>";
    permission java.security.SecurityPermission
                              "removeProviderProperty.SUN";
    permission java.security.SecurityPermission "getSignerPrivateKey";
    permission java.security.SecurityPermission "setSignerKeyPair";
  };
```

getPolicy: This target grants permission to retrieve the system-wide security policy. Granting this permission discloses which permissions would be granted to a given application or applet. While revealing the policy does not compromise the security of the system, it does provide malicious code with additional information it could use to better aim an attack.

setPolicy: This target grants permission to set the system-wide security policy. Granting this permission could allow malicious code to grant itself all the necessary permissions to successfully mount an attack an attack on the system.

getProperty.{key}: This target grants permission to retrieve the security property specified by {key}. Depending on the particular key for which access has been granted, the code may have access to the list of security providers and the location of the system-wide and user security policies. While revealing this information does not compromise the security of the system, it does provide malicious code with additional information which it may use to better aim an attack.

setProperty.{key}: This target grants permission to set the security property specified by {key}. This could include setting a security provider or defining the location of the system-wide security policy. Malicious code that has permission to set a new security provider may set a rogue provider that steals confidential information such as cryptographic private keys. In addition, malicious code with permission to set the location of the system-wide security policy may point it to a security policy that grants the attacker all the necessary permissions it requires to successfully mount an attack on the system.

insertProvider.{provider name}: This target grants permission to add the new security provider specified by {provider name}. Granting this permission allows the introduction of a possibly malicious provider that could do such things as disclose the private keys passed to it. This is possible because the Security object, which manages the installed providers, does not currently check the integrity or authenticity of a provider before attaching it.

removeProvider.{provider name}: This target grants permission to remove the provider specified by {provider name}. Granting this permission could change the behavior or disable execution of other parts of the program. If a provider requested by the program has been removed, execution might fail.

setSystemScope: This target grants permission to set the system identity scope. Granting this permission could allow an attacker to configure the system identity scope with certificates that should not be trusted. This could grant code signed with those certificates privileges that would be denied by the original identity scope.

setIdentityPublicKey: This target grants permission to set the public key for an Identity object. If the identity is marked trusted, this allows an attacker to introduce its own public key that is not trusted by the system's identity scope.This could grant code signed with that public key privileges that would be otherwise denied.

SetIdentityInfo: This target grants permission to set a general information string for an Identity object. Granting this permission allows attackers to set the general description for an identity. Doing so could trick applications into using a different identity than intended or prevent applications from finding a particular identity.

addIdentityCertificate: This target grants permission to add a certificate for an Identity object. Granting this permission allows attackers to set a certificate for an identity's public key making the public key trusted to a wider audience than originally intended.

removeIdentityCertificate: This target grants permission to remove a certificate for an Identity object. Granting this permission allows attackers to remove a certificate for an identity's public key. This could be dangerous because public key suddenly becomes considered less trustworthy than it otherwise would be.

printIdentity: This target grants permission to print out the name of a principal, the scope in which the principal is used, and whether the principal is considered trusted in that scope. The printed scope could be a filename, in which case it might convey local system information. For example, here is a sample printout of an identity named carol, who is marked not trusted in the user's identity database:

```
carol[/home/luehe/identitydb.obj][not trusted].
```

clearProviderProperties.{provider name}: This target grants permission to clear a Provider object so it no longer contains the properties used to look up services implemented by the provider. Granting this permission disables the lookup of services implemented by the provider. This could change the behavior or disable execution of other parts of the program that would normally utilize the Provider, as described under the removeProvider.{provider name} permission above.

putProviderProperty.{provider name}: This target grants permission to set properties for the specified provider. The provider properties each specify the name and location of a particular service implemented by the provider. Granting this permission allows code to replace the service specification with another one with a different implementation and could be dangerous if the new implementation has malicious code.

removeProviderProperty.{provider name}: This target grants permission to remove properties from the specified provider. Granting this permission disables the lookup of services implemented by the provider making them inaccessible. Granting this permission to malicious code could allow the malicious code to change the behavior or disable execution of other parts of the program that would normally utilize the Provider object, as described under the removeProvider.{provider name} permission above.

getSignerPrivateKey: This target grants permission to retrieve the private key of a Signer object. Private keys should always be kept secret. Granting this permission could allow malicious code to use the private key to sign files and claim the signature came from the Signer object.

setSignerKeyPair: This target grants permission to set the public and private key pair for a Signer object. Granting this permission could allow an attacker to replace the target's key pair with a possibly weaker (smaller) key pair. This would also allow an attacker to listen in on encrypted communication between the target and its peers. The target's peers might wrap an encryption session key under the target's new public key, which would allow the attacker (who possesses the corresponding private key) to unwrap the session key and decipher the communication data encrypted under that session key.

SerializablePermission

java.io.SerializablePermission grants access to serialization operations. The possible targets are listed by name with no action list.

```
grant {
  permission java.io.SerializablePermission
     "enableSubclassImplementation";
  permission java.io.SerializablePermission
     "enableSubstitution";
};
```

enableSubclassImplementation: This target grants permission to implement a subclass of ObjectOutputStream or ObjectInputStream to override the default serialization or deserialization of objects. Granting this permission could allow code to use this to serialize or deserialize classes in a malicious way. For example, during serialization, malicious code could

store confidential private field data in a way easily accessible to attackers; or, during deserialization malicious code could deserialize a class with all its private fields zeroed out.

enableSubstitution: This target grants permission to substitute one object for another during serialization or deserialization. Granting this permission could allow malicious code to replace the actual object with one that has incorrect or malignant data.

SocketPermission

The java.net.SocketPermission permission grants access to a network by way of sockets. The target is a host name and port address, and the action list specifies ways to connect to that host. Possible connections are accept, connect, listen, and resolve.

This policy file entry allows a connection to and accepts connections on port 7777 on the host puffin.eng.sun.com.

```
grant {
  permission java.net.SocketPermission
                  "puffin.eng.sun.com:7777","connect, accept";
};
```

This policy file entry allows connections to, accepts connections on, and listens on any port between 1024 and 65535 on the local host.

```
grant {
  permission java.net.SocketPermission
                  "localhost:1024-","accept, connect, listen";
};
```

The host is expressed with the following syntax as a DNS name, as a numerical IP address, or as localhost (for the local machine). The asterisk (*) wild card can be included once in a DNS name host specification. If included, it must be in the left-most position, as in *.sun.com.

```
host = (hostname | IPaddress)[:portrange]
portrange = portnumber | -portnumber | portnumber-[portnumber]
```

The port or port range is optional. A port specification of the form N-, where N is a port number, means all ports numbered N and above, while a specification of the form -N indicates all ports numbered N and below.

The listen action is only meaningful when used with localhost, and the resolve (resolve host/ip name service lookups) action is implied when any of the other actions are present.

Granting code permission to accept or make connections to remote hosts may be dangerous because malevolent code can more easily transfer and share confidential data among parties that might not otherwise have access to the data.

Note: On Unix platforms, only root is normally allowed access to ports lower than 1024.

B: Classes, Methods, and Permissions

A number of Java™ 2 platform methods are implemented to verify access permissions. This means that before they execute, they verify that the system, user, or program has a policy file with the required permissions for execution to continue. If no such permission is found, execution stops with an error condition.

The access verification code passes the required permissions to the security manager, and the security manager checks that permission against the policy file permissions to determine whether to allow access. This means that Java 2 platform API methods are associated with specific permissions, and specific permissions are associated with specific java.security.SecurityManager methods.

This appendix lists the Java 2 platform methods, the permission associated with each method, and the java.security.SecurityManager method called to verify the existence of that permission. You need this information when you implement certain abstract methods or create your own security manager so you can include access verification code to keep your implementations in line with Java 2 platform security policy. If you do not include access verification code, your implementations will bypass the built-in Java 2 platform security checks.

Covered in this Appendix

- java.awt.Graphics2D (page 448)
- java.awt.Toolkit (page 449)
- java.awt.Window (page 449)
- java.beans.Beans (page 449)
- java.beans.Introspector (page 449)
- java.beans.PropertyEditorManager (page 450)
- java.io.File (page 450)
- java.io.FileInputStream (page 450)
- java.io.FileOutputStream (page 451)
- java.io.ObjectInputStream (page 451)

- java.io.ObjectOutputStream (page 451)
- java.io.RandomAccessFile (page 451)
- java.lang.Class (page 452)
- java.lang.ClassLoader (page 453)
- java.lang.Runtime (page 454)
- java.lang.SecurityManager (page 454)
- java.lang.System (page 454)
- java.lang.Thread (page 455)
- java.lang.ThreadGroup (page 456)
- java.lang.reflect.AccessibleObject (page 457)
- java.net.Authenticator (page 457)
- java.net.DatagramSocket (page 458)
- java.net.HttpURLConnection (page 459)
- java.net.InetAddress (page 459)
- java.net.MulticastSocket (page 459)
- java.net.ServerSocket (page 460)
- java.net.Socket (page 460)
- java.net.URL (page 460)
- java.net.URLConnection (page 461)
- java.net.URLClassLoader (page 461)
- java.rmi.activation.ActivationGroup (page 461)
- java.rmi.server.RMISocketFactory (page 461)
- java.security.Identity (page 461)
- java.security.IdentityScope (page 462)
- java.security.Permission (page 462)
- java.security.Policy (page 462)
- java.security.Provider (page 462)
- java.security.SecureClassLoader (page 463)
- java.security.Security (page 463)
- java.security.Signer (page 463)
- java.util.Locale (page 463)
- java.util.zip.ZipFile (page 464)

java.awt.Graphics2D

```
public abstract void setComposite(Composite comp)
java.Security.SecurityManager.checkPermission
java.awt.AWTPermission "readDisplayPixels"
```

The access verification code for setComposite should call java.Security.SecurityManager.checkPermission and pass it java.awt.AWTPermission “readDisplayPixels” when a

Graphics2D context draws to a Component on the display screen and the Composite is a custom object rather than an AlphaComposite object.

java.awt.Toolkit

```
public void addAWTEventListener(AWTEventListener listener, long eventMask)
public void removeAWTEventListener(AWTEventListener listener)
checkPermission
java.awt.AWTPermission "listenToAllAWTEvents"

public abstract PrintJob getPrintJob(Frame frame, String jobtitle,
                                     Properties props)
checkPrintJobAccess
java.lang.RuntimePermission "queuePrintJob"

public abstract Clipboard getSystemClipboard()
checkSystemClipboardAccess
java.awt.AWTPermission "accessClipboard"

public final EventQueue getSystemEventQueue()
checkAwtEventQueueAccess
java.awt.AWTPermission "accessEventQueue"
```

java.awt.Window

```
Window()
checkTopLevelWindow
java.awt.AWTPermission "showWindowWithoutWarningBanner"
```

java.beans.Beans

```
public static void setDesignTime(boolean isDesignTime)
public static void setGuiAvailable(boolean isGuiAvailable)
checkPropertiesAccess
java.util.PropertyPermissions "*", "read,write"
```

java.beans.Introspector

```
public static synchronized void setBeanInfoSearchPath(String path[])
checkPropertiesAccess
java.util.PropertyPermissions "*", "read,write"
```

java.beans.PropertyEditorManager

```
public static void registerEditor(Class targetType, Class editorClass)
public static synchronized void setEditorSearchPath(String path[])
checkPropertiesAccess
java.util.PropertyPermissions "*", "read,write"
```

java.io.File

```
public boolean delete()
public void deleteOnExit()
checkDelete(String)
java.io.FilePermission "{name}", "delete"

public boolean exists()
public boolean canRead()
public boolean isFile()
public boolean isDirectory()
public boolean isHidden()
public long lastModified()
public long length()
public String[] list()
public String[] list(FilenameFilter filter)
public File[] listFiles()
public File[] listFiles(FilenameFilter filter)
public File[] listFiles(FileFilter filter)
checkRead(String)
java.io.FilePermission "{name}", "read"

public boolean canWrite()
public boolean createNewFile()
public static File createTempFile(String prefix, String suffix)
public static File createTempFile(String prefix, String suffix,File directory)
public boolean mkdir()
public boolean mkdirs()
public boolean renameTo(File dest)
public boolean setLastModified(long time)
public boolean setReadOnly()
checkWrite(String)
java.io.FilePermission "{name}", "write"
```

java.io.FileInputStream

```
FileInputStream(FileDescriptor fdObj)
checkRead(FileDescriptor)
java.lang.RuntimePermission "readFileDescriptor"
```

```
FileInputStream(String name)
FileInputStream(File file)
checkRead(String)
java.io.FilePermission "{name}", "read"
```

java.io.FileOutputStream

```
FileOutputStream(FileDescriptor fdObj)
checkWrite(FileDescriptor)
java.lang.RuntimePermission "writeFileDescriptor"

FileOutputStream(File file)
FileOutputStream(String name)
FileOutputStream(String name, boolean append)
checkWrite(String)
java.io.FilePermission "{name}", "write"
```

java.io.ObjectInputStream

```
protected final boolean enableResolveObject(boolean enable);
checkPermission
java.io.SerializablePermission "enableSubstitution"

protected ObjectInputStream()
protected ObjectOutputStream()
checkPermission
java.io.SerializablePermission "enableSubclassImplementation"
```

java.io.ObjectOutputStream

```
protected final boolean enableReplaceObject(boolean enable)
checkPermission
java.io.SerializablePermission "enableSubstitution"
```

java.io.RandomAccessFile

```
RandomAccessFile(String name, String mode)
RandomAccessFile(File file, String mode)
checkRead(String)
java.io.FilePermission "{name}", "read"
```

In both these constructors the mode is read.

```
RandomAccessFile(String name, String mode)
checkRead(String) and checkWrite(String)
java.io.FilePermission "{name}", "read,write"
```

In this constructor the mode is read-write.

java.lang.Class

```
public static Class forName(String name, boolean initialize, ClassLoader loader)
checkPermission
java.lang.RuntimePermission("getClassLoader")
```

The access verification code for this method calls checkPermission and pass it java.lang.RuntimePermission(“getClassLoader”) when loader is null and the caller's class loader is not null.

```
public Class[] getClasses()
checkMemberAccess(this, Member.DECLARED)
java.lang.RuntimePermission "accessDeclaredMembers"
java.lang.RuntimePermission "accessClassInPackage.{pkgName}
```

The access verification code for this class and each of its superclasses calls checkMemberAccess(this, Member.DECLARED). If the class is in a package, checkPackageAccess({pkgName}) is also called. By default, checkMemberAccess does not require permission if this class's classloader is the same as that of the caller. Otherwise, it requires java.lang.RuntimePermission “accessDeclaredMembers”. If the class is in a package, java.lang.RuntimePermission “accessClassInPackage.{pkgName}” is also required.

```
public ClassLoader getClassLoader()
checkPermission
java.lang.RuntimePermission "getClassLoader"
```

If the caller's class loader is null, or is the same as or an ancestor of the class loader for the class whose class loader is being requested, no permission is needed. Otherwise, java.lang.RuntimePermission “getClassLoader” is required.

```
public Class[] getDeclaredClasses()
public Field[] getDeclaredFields()
public Method[] getDeclaredMethods()
public Constructor[] getDeclaredConstructors()
public Field getDeclaredField(String name)
public Method getDeclaredMethod(...)
public Constructor getDeclaredConstructor(...)
checkMemberAccess(this, Member.DECLARED)
checkPackageAccess({pkgName})
java.lang.RuntimePermission "accessDeclaredMembers”
java.lang.RuntimePermission "accessClassInPackage.{pkgName}”
```

If Class is in a package, the access verification code should call checkPackageAccess({pkgName}) and pass it java.lang.RuntimePermission “accessClassInPackage.{pkgName}”.

If Class is not in a package, the access verification code for these methods should call checkMemberAccess(this, Member.DECLARED) and pass it java.lang.RuntimePermission “accessClassInPackage.{pkgName}”.

```
public Field[] getFields()
public Method[] getMethods()
public Constructor[] getConstructors()
public Field getField(String name)
public Method getMethod(...)
public Constructor getConstructor(...)
checkMemberAccess(this, Member.PUBLIC)
checkPackageAccess({pkgName})
java.lang.RuntimePermission "accessClassInPackage.{pkgName}"
```

If Class is not in a package, the access verification code for these methods calls checkMemberAccess(this, Member.PUBLIC), but no permission is passed.

If Class is in a package, the access verification code for these methods should call checkPackageAccess({pkgName}) and pass it checkPackageAccess({pkgName}).

```
public ProtectionDomain getProtectionDomain()
checkPermission
java.lang.RuntimePermission "getProtectionDomain"
```

java.lang.ClassLoader

```
ClassLoader()
ClassLoader(ClassLoader parent)
checkCreateClassLoader
java.lang.RuntimePermission "createClassLoader"

public static ClassLoader getSystemClassLoader()
public ClassLoader getParent()
checkPermission
java.lang.RuntimePermission "getClassLoader"
```

If the caller's class loader is null or is the same as or an ancestor of the class loader for the class whose class loader is being requested, no permission is needed. Otherwise, java.lang.RuntimePermission “getClassLoader” is required.

java.lang.Runtime

```
public Process exec(String command)
public Process exec(String command, String envp[])
public Process exec(String cmdarray[])
public Process exec(String cmdarray[], String envp[])
checkExec
java.io.FilePermission "{command}", "execute"

public void exit(int status)
public static void runFinalizersOnExit(boolean value)
checkExit(status) where status is 0 for runFinalizersOnExit
java.lang.RuntimePermission "exitVM"

public void load(String lib)
public void loadLibrary(String lib)
checkLink({libName})
java.lang.RuntimePermission "loadLibrary.{libName}"
```

In these methods {libName} is the lib, filename or libname argument.

java.lang.SecurityManager

```
<all methods>
checkPermission
```
See C: Security Manager Methods (page 465).

java.lang.System

```
public static void exit(int status)
public static void runFinalizersOnExit(boolean value)
checkExit(status) where status is 0 for runFinalizersOnExit
java.lang.RuntimePermission "exitVM"

public static void load(String filename)
public static void loadLibrary(String libname)
checkLink({libName})
java.lang.RuntimePermission "loadLibrary.{libName}"
```

In these methods {libName} is the lib, filename or libname argument.

```
public static Properties getProperties()
public static void setProperties(Properties props)
checkPropertiesAccess
java.util.PropertyPermission "*", "read,write"
```

```
public static String getProperty(String key)
public static String getProperty(String key, String def)
checkPropertyAccess
java.util.PropertyPermission "{key}", "read"

public static void setIn(InputStream in)
public static void setOut(PrintStream out)
public static void setErr(PrintStream err)
checkPermission
java.lang.RuntimePermission "setIO"

public static String setProperty(String key, String value)
checkPermission
java.util.PropertyPermission "{key}", "write"

public static synchronized void setSecurityManager(SecurityManager s)
checkPermission
java.lang.RuntimePermission "setSecurityManager"
```

java.lang.Thread

```
public ClassLoader getContextClassLoader()
checkPermission
java.lang.RuntimePermission "getClassLoader"
```

If the caller's class loader is null or is the same as or an ancestor of the context class loader for the thread whose context class loader is being requested, no permission is needed. Otherwise, java.lang.RuntimePermission “getClassLoader” is required.

```
public void setContextClassLoader(ClassLoader cl)
checkPermission
java.lang.RuntimePermission "setContextClassLoader"

public final void checkAccess()
public void interrupt()
public final void suspend()
public final void resume()
public final void setPriority(int newPriority)
public final void setName(String name)
public final void setDaemon(boolean on)
checkAccess(this)
java.lang.RuntimePermission "modifyThread"

public static int enumerate(Thread tarray[])
checkAccess({threadGroup})
java.lang.RuntimePermission "modifyThreadGroup"
```

```
public final void stop()
checkAccess(this).
checkPermission
java.lang.RuntimePermission "modifyThread"
java.lang.RuntimePermission "stopThread"
```

The access verification code should call checkAccess and pass it java.lang.RuntimePermission “modifyThread”, unless the current thread is trying to stop a thread other than itself. In this case, the access verification code should call checkPermission and pass it java.lang.RuntimePermission “stopThread”.

```
public final synchronized void stop(Throwable obj)
checkAccess(this).
checkPermission
java.lang.RuntimePermission "modifyThread"
java.lang.RuntimePermission "stopThread"
```

The access verification code should call checkAccess and pass it java.lang.RuntimePermission “modifyThread” unless the current thread is trying to stop a thread other than itself or obj is not an instance of ThreadDeath. In this case, the access verification code should call checkPermission and pass it java.lang.RuntimePermission “stopThread”.

```
Thread()
Thread(Runnable target)
Thread(String name)
Thread(Runnable target, String name)
checkAccess({parentThreadGroup})
java.lang.RuntimePermission "modifyThreadGroup"

Thread(ThreadGroup group, ...)
checkAccess(this) for ThreadGroup methods, or
checkAccess(group) for Thread methods
java.lang.RuntimePermission "modifyThreadGroup"
```

java.lang.ThreadGroup

```
public final void checkAccess()
public int enumerate(Thread list[])
public int enumerate(Thread list[],boolean recurse)
public int enumerate(ThreadGroup list[])
public int enumerate(ThreadGroup list[],boolean recurse)
public final ThreadGroup getParent()
public final void setDaemon(boolean daemon)
public final void setMaxPriority(int pri)
public final void suspend()
public final void resume()
```

```
public final void destroy()
checkAccess(this) for ThreadGroup methods, or
checkAccess(group) for Thread methods
java.lang.RuntimePermission "modifyThreadGroup"

ThreadGroup(String name)
ThreadGroup(ThreadGroup parent,
String name)
checkAccess({parentThreadGroup})
java.lang.RuntimePermission "modifyThreadGroup"

public final void interrupt()
checkAccess(this)
java.lang.RuntimePermission "modifyThreadGroup"
java.lang.RuntimePermission "modifyThread"
```

The access verification code for this method also requires java.lang.RuntimePermission "modifyThread" because the java.lang.Thread interrupt() method is called for each thread in the thread group and in all of its subgroups.

```
public final void stop()
checkAccess(this)
java.lang.RuntimePermission "modifyThreadGroup"
java.lang.RuntimePermission "modifyThread"
java.lang.RuntimePermission "stopThread"
```

The access verification code for this method also requires java.lang.RuntimePermission "modifyThread" and possibly java.lang.RuntimePermission "stopThread" because the java.lang.Thread stop() method is called for each thread in the thread group and in all of its subgroups.

java.lang.reflect.AccessibleObject

```
public static void setAccessible(...)
public void setAccessible(...)
checkPermission
java.lang.reflect.ReflectPermission "suppressAccessChecks"
```

java.net.Authenticator

```
public static PasswordAuthentication
requestPasswordAuthentication(InetAddress addr,int port,
        String protocol, String prompt, String scheme)
checkPermission
java.net.NetPermission "requestPasswordAuthentication"
```

```
public static void setDefault(Authenticator a)
checkPermission
java.net.NetPermission "setDefaultAuthenticator"
```

java.net.DatagramSocket

```
public void send(DatagramPacket p)
checkMulticast(p.getAddress())
checkConnect(p.getAddress().getHostAddress(), p.getPort())
java.net.SocketPermission((p.getAddress()).getHostAddress(), "accept,connect")
java.net.SocketPermission "{host}","resolve"
```

The access verification code for send calls checkMulticast in the following case:

```
if(p.getAddress().isMulticastAddress()) {
  java.net.SocketPermission((p.getAddress()).getHostAddress(),"accept,connect")
}
```

The access verification code for send calls checkConnect in the following case:

```
else {
  port = p.getPort();
  host = p.getAddress().getHostAddress();
  if (port == -1) {
    java.net.SocketPermission
   "{host}","resolve";
} else {
  java.net.SocketPermission
  "{host}:{port}","connect"
}

public InetAddress getLocalAddress()
checkConnect({host}, -1)
java.net.SocketPermission "{host}", "resolve"

DatagramSocket(...)
checkListen({port})
```

The access verification code for this method calls checkListen and passes in socket permissions as follows:

```
if (port == 0){
  java.net.SocketPermission "localhost:1024-", "listen";
} else {
  java.net.SocketPermission "localhost:{port}", "listen";
}

public synchronized void receive(DatagramPacket p)
checkAccept({host}, {port})
java.net.SocketPermission "{host}:{port}", "accept"
```

java.net.HttpURLConnection

```
public static void setFollowRedirects(boolean set)
checkSetFactory
java.lang.RuntimePermission "setFactory"
```

java.net.InetAddress

```
public String getHostName()
public static InetAddress[] getAllByName(String host)
public static InetAddress getLocalHost()
checkConnect({host}, -1)
java.net.SocketPermission "{host}", "resolve"
```

java.net.MulticastSocket

```
public void joinGroup(InetAddress mcastaddr)
public void leaveGroup(InetAddress mcastaddr)
checkMulticast(InetAddress)
java.net.SocketPermission(mcastaddr.getHostAddress(),"accept,connect")

public synchronized void send(DatagramPacket p, byte ttl)
checkMulticast(p.getAddress(), ttl)
checkConnect(p.getAddress().getHostAddress(), p.getPort())
java.net.SocketPermission((p.getAddress()).getHostAddress(),"accept,connect")
java.net.SocketPermission "{host}","resolve"
```

The access verification code for send calls checkMulticast in the following case:

```
if(p.getAddress().isMulticastAddress()) {
  java.net.SocketPermission((p.getAddress()).getHostAddress(), "accept,connect")
}
```

The access verification code for this method calls checkConnect in the following case:

```
} else {
  port = p.getPort();
  host = p.getAddress().getHostAddress();
  if (port == -1) {
     java.net.SocketPermission "{host}","resolve"
  } else {
    java.net.SocketPermission "{host}:{port}","connect"
}

MulticastSocket(...)
checkListen({port})
```

The access verification code for this method calls checkListen in the following case:

```
if (port == 0) {
  java.net.SocketPermission "localhost:1024-", "listen";
} else {
  java.net.SocketPermission "localhost:{port}","listen";
}
```

java.net.ServerSocket

```
ServerSocket(...)
checkListen({port})
```

The access verification code for this method calls checkListen in the following case:

```
if (port == 0) {
  java.net.SocketPermission "localhost:1024-","listen";
} else {
  java.net.SocketPermission "localhost:{port}","listen";
}

public Socket accept()
protected final void implAccept(Socket s)
checkAccept({host}, {port})
java.net.SocketPermission "{host}:{port}", "accept"

public static synchronized void setSocketFactory(...)
checkSetFactory
java.lang.RuntimePermission "setFactory"
```

java.net.Socket

```
public static synchronized void setSocketImplFactory(...)
checkSetFactory
java.lang.RuntimePermission "setFactory"

Socket(...)
checkConnect({host}, {port})
java.net.SocketPermission "{host}:{port}", "connect"
```

java.net.URL

```
public static synchronized void setURLStreamHandlerFactory(...)
checkSetFactory
java.lang.RuntimePermission "setFactory"

URL(...)
checkPermission
java.net.NetPermission "specifyStreamHandler"
```

java.net.URLConnection

```
public static synchronized void setContentHandlerFactory(...)
public static void setFileNameMap(FileNameMap map)
checkSetFactory
java.lang.RuntimePermission "setFactory"
```

java.net.URLClassLoader

```
URLClassLoader(...)
checkCreateClassLoader
java.lang.RuntimePermission "createClassLoader"
```

java.rmi.activation.ActivationGroup

```
public static synchronized ActivationGroup createGroup(...)
public static synchronized void setSystem(ActivationSystem system)
checkSetFactory
java.lang.RuntimePermission "setFactory"
```

java.rmi.server.RMISocketFactory

```
public synchronized static void setSocketFactory(...)
checkSetFactory
java.lang.RuntimePermission "setFactory"
```

java.security.Identity

```
public void addCertificate(...)
checkSecurityAccess("addIdentityCertificate")
java.security.SecurityPermission "addIdentityCertificate"

public void removeCertificate(...)
checkSecurityAccess("removeIdentityCertificate")
java.security.SecurityPermission "removeIdentityCertificate"

public void setInfo(String info)
checkSecurityAccess("setIdentityInfo")
java.security.SecurityPermission "setIdentityInfo"

public void setPublicKey(PublicKey key)
checkSecurityAccess("setIdentityPublicKey")
java.security.SecurityPermission "setIdentityPublicKey"
```

```
public String toString(...)
checkSecurityAccess("printIdentity")
java.security.SecurityPermission "printIdentity"
```

java.security.IdentityScope

```
protected static void setSystemScope()
checkSecurityAccess("setSystemScope")
java.security.SecurityPermission "setSystemScope"
```

java.security.Permission

```
public void checkGuard(Object object)
checkPermission(this)
```

This Permission object is the permission checked.

java.security.Policy

```
public static Policy getPolicy()
checkPermission
java.security.SecurityPermission "getPolicy"

public static void setPolicy(Policy policy);
checkPermission
java.security.SecurityPermission "setPolicy"
```

java.security.Provider

In the following methods, the parameter `name` represents the provider name.

```
public synchronized void clear()
checkSecurityAccess("clearProviderProperties."+{name})
java.security.SecurityPermission "clearProviderProperties.{name}"

public synchronized Object put(Object key, Object value)
checkSecurityAccess("putProviderProperty."+{name})
java.security.SecurityPermission "putProviderProperty.{name}"

public synchronized Object remove(Object key)
checkSecurityAccess("removeProviderProperty."+{name})
java.security.SecurityPermission "removeProviderProperty.{name}"
```

java.security.SecureClassLoader

```
SecureClassLoader(...)
checkCreateClassLoader
java.lang.RuntimePermission "createClassLoader"
```

java.security.Security

```
public static void getProperty(String key)
checkPermission
java.security.SecurityPermission "getProperty.{key}"

public static int addProvider(Provider provider)
public static int insertProviderAt(Provider provider, int position);
checkSecurityAccess("insertProvider." + provider.getName())
java.security.SecurityPermission "insertProvider.{name}"

public static void removeProvider(String name)
checkSecurityAccess("removeProvider."+name)
java.security.SecurityPermission "removeProvider.{name}"

public static void setProperty( String key, String datum)
checkSecurityAccess("setProperty."+key)
java.security.SecurityPermission "setProperty.{key}"
```

java.security.Signer

```
public PrivateKey getPrivateKey()
checkSecurityAccess("getSignerPrivateKey")
java.security.SecurityPermission "getSignerPrivateKey"

public final void setKeyPair(KeyPair pair)
checkSecurityAccess("setSignerKeypair")
java.security.SecurityPermission "setSignerKeypair"
```

java.util.Locale

```
public static synchronized void setDefault(Locale newLocale)
checkPermission
java.util.PropertyPermission "user.language","write"
```

java.util.zip.ZipFile

```
ZipFile(String name)
checkRead
java.io.FilePermission "{name}","read"
```

C: Security Manager Methods

This appendix lists the java.lang.SecurityManager methods and the permissions their default implementations check for. Each check method calls the SecurityManager.checkPermission method with the indicated permission, except the checkConnect and checkRead methods, which take a context argument instead. The checkConnect and checkRead methods expect the context to be an AccessControlContext and call the context's checkPermission method with the specified permission.

```
public void checkAccept(String host, int port);
java.net.SocketPermission "{host}:{port}", "accept";

public void checkAccess(Thread g);
java.lang.RuntimePermission "modifyThread");

public void checkAccess(ThreadGroup g);
java.lang.RuntimePermission "modifyThreadGroup");

public void checkAwtEventQueueAccess();
java.awt.AWTPermission "accessEventQueue";

public void checkConnect(String host, int port);
  if(port == -1) {
    java.net.SocketPermission "{host}","resolve";
  } else
    java.net.SocketPermission "{host}:{port}","connect";
  }

public void checkConnect(String host, int port, Object context);
  if(port == -1){
    java.net.SocketPermission "{host}","resolve";
  } else {
    java.net.SocketPermission "{host}:{port}","connect";
  }

public void checkCreateClassLoader();
java.lang.RuntimePermission "createClassLoader";

public void checkDelete(String file);
java.io.FilePermission "{file}", "delete";
```

```
public void checkExec(String cmd);
  if(cmd is an absolute path) {
    java.io.FilePermission "{cmd}", "execute";
  } else {
    java.io.FilePermission "-", "execute";
  }

public void checkExit(int status);
java.lang.RuntimePermission "exitVM");

public void checkLink(String lib);
java.lang.RuntimePermission "loadLibrary.{lib}";

public void checkListen(int port);
  if(port == 0) {
    java.net.SocketPermission "localhost:1024-","listen";
  } else {
    java.net.SocketPermission "localhost:{port}","listen";
  }

public void checkMemberAccess(Class clazz, int which);
  if(which != Member.PUBLIC) {
    if(currentClassLoader() != clazz.getClassLoader()) {
     checkPermission(
     new java.lang.RuntimePermission("accessDeclaredMembers"));
    }
  }

public void checkMulticast(InetAddress maddr);
java.net.SocketPermission(maddr.getHostAddress(),"accept,connect");

public void checkMulticast(InetAddress maddr, byte ttl);
java.net.SocketPermission(maddr.getHostAddress(),"accept,connect");

public void checkPackageAccess(String pkg);
java.lang.RuntimePermission "accessClassInPackage.{pkg}";

public void checkPackageDefinition(String pkg);
java.lang.RuntimePermission "defineClassInPackage.{pkg}";

public void checkPrintJobAccess();
java.lang.RuntimePermission "queuePrintJob";

public void checkPropertiesAccess();
java.util.PropertyPermission "*", "read,write";

public void checkPropertyAccess(String key);
java.util.PropertyPermission "{key}", "read,write";

public void checkRead(FileDescriptor fd);
java.lang.RuntimePermission "readFileDescriptor";
```

```
public void checkRead(String file);
java.io.FilePermission "{file}", "read";

public void checkRead(String file, Object context);
java.io.FilePermission "{file}", "read";

public void checkSecurityAccess(String action);
java.security.SecurityPermission "{action}";

public void checkSetFactory();
java.lang.RuntimePermission "setFactory";

public void checkSystemClipboardAccess();
java.awt.AWTPermission "accessClipboard";

public boolean checkTopLevelWindow(Object window);
java.awt.AWTPermission "showWindowWithoutWarningBanner";

public void checkWrite(FileDescriptor fd);
java.lang.RuntimePermission "writeFileDescriptor";

public void checkWrite(String file);
java.io.FilePermission "{file}", "write";

public SecurityManager();
java.lang.RuntimePermission "createSecurityManager";
```

D: API Reference

This appendix lists Java™ class and interface methods used in the examples for this book. The methods are grouped by the class or interface to which they belong. Each method listing provides a page number where the method is used. You can find information on the method in the surrounding text or earlier in the chapter. You can also use the Index to locate these same methods using the method names or the fully qualified class or interface names to which the methods belong.

ActionListener Interface

Interface java.awt.event.ActionListener

void actionPerformed(ActionEvent e) (page 250)

WindowListener Interface

Interface java.awt.event.WindowListener

void windowClosing(WindowEvent e) (page 251)

Graphics Class

Class java.awt.Graphics

void drawRect(int x, int y, int width, int height) (page 258) page 261

FontMetrics getFontMetrics() (page 293)

void setClip(int x, int y, int width, int height) (page 264)

Graphics2D class

Class java.awt.Graphics2D

void translate(double x, double y) (page 265)

void scale(double sx, double sy) (page 265)

Book Class

Class java.awt.print.Book

void append(Printable painter, PageFormat page) (page 263)

void append(Printable painter, PageFormat page, int numPages) (page 263)

PageFormat Class

Class java.awt.print.PageFormat

double getImageableX() (page 258)

double getImageableY() (page 258)

double getImageableWidth() (page 264)

double getImageableHeight() (page 264)

void setOrientation(int orientation) (page 263)

Printable Interface

Interface java.awt.print.Printable

int print(Graphics graphics, PageFormat pageFormat, int pageIndex) (page 258)

PrinterJob Class

Class java.awt.print.PrinterJob

PageFormat defaultPage() (page 262)

static PrinterJob getPrinterJob() (page 262)

PageFormat pageDialog(PageFormat page) (page 262)

void print() (page 262)

boolean printDialog() (page 261)

void setPageable(Pageable document) (page 263)

void setPrintable(Printable painter) (page 262)

Toolkit Class

Class java.awt.Toolkit

void addAWTEventListener(AWTEventListener l, long eventmask) (page 319)

ByteArrayOutputStream Class

Class java.io.ByteArrayOutputStream

byte[] toByteArray() (page 383)

DataOutputStream Class

Class java.io.DataOutputStream

byte[] toByteArray() (page 383)

Double Class

Class java.lang.Double

static Double valueOf(double doublevalue) (page 126)

SecurityManager Class

Class java.lang.SecurityManager

void checkRead(String filename) (page 424)

void checkWrite(String filename) (page 425)

void checkPermission(Permission perm) (page 425)

System Class

Class java.lang.System

static SecurityManager getSecurityManager() (page 103)

static SecurityManager setSecurityManager(SecurityManager s) (page 103)

static void loadLibrary(String libraryname) (page 209), (page 216), (page 218), (page 221)

static void loadLibrary(String libraryname) (page 223)

Naming Class

Class java.rmi.Naming

static void rebind(String rminame, Remote obj) (page 85)

static Remote lookup(String rminame) (page 88), (page 90)

static void rebind(String rminame, Remote obj) (page 103)

RMISocketFactory Class

Class java.rmi.server.RMISocketFactory

static void setFailureHandler(RMIFailureHandler fh) (page 103)

CallableStatement Interface

Class java.sql.CallableStatement

void setString(int index, String s) (page 131)

void registerOutParameter(int index, int sqltype) (page 131)

Date getDate(int index) (page 131)

Connection Interface

Class java.sql.Connection

CallableStatement prepareCall(String sql) (page 131)

Statement createStatement() (page 132), (page 141)

DatabaseMetaData getMetaData() (page 134) page 145

PrepareStatement prepareStatement(String sql) (page 57), (page 58), (page 70), (page 122), (page 127), (page 132), (page 141), (page 141)

void rollback() (page 142)

void setAutoCommit(boolean autoCommit) (page 141)

DatabaseMetaData Interface

Class java.sql.DatabaseMetaData

boolean supportsResultSetConcurrency(int type, int concurrency) (page 135)

DriverManager Class

Class java.sql.DriverManager

static Connection getConnection(String url) (page 130)

static Connection getConnection(String url, String user, String password) (page 131)

static void setLogStream(PrintStream out) (page 131)

PreparedStatement Interface

Class java.sql.PreparedStatement

ResultSet executeQuery() (page 123), (page 127), (page 133)

int executeUpdate() (page 59), (page 137)

void setDate(int index, Date datavalue) (page 141)

void setDouble(int index, double doublevalue) (page 59), (page 123), (page 127), (page 141)

void setInt(int index, int intvalue) (page 137)

void setString(int index, String s) (page 58), (page 131)

void setBinaryStream(int index, InputStream stream, int length) (page 137)

ResultSet Interface

Interface java.sql.ResultSet

void close() (page 137)

boolean first() (page 135)

byte[] getBytes(String ColumnName) (page 137)

double getDouble(String columnName) (page 123), (page 135)

int getInt(String ColumnName) (page 127)

boolean next() (page 127), (page 134)

String getString(String columnName) (page 58), (page 123), (page 134)

double updateDouble(String columnName) (page 135)

Statement Interface

Class java.sql.Statement

void close() (page 137)

int executeQuery(String sql) (page 132)

int executeBatch() (page 136)

int executeUpdate(String sql) (page 132), (page 142)

ResultSet getResultSet() (page 58), (page 141), (page 142)

ArrayList Class

Class java.util.ArrayList

boolean add(Object object) (page 123)

Object[] toArray(Object[] objarray) (page 123)

Calendar Class

Class java.util.Calendar

void add(int partofdate, int dateamount) (page 141)

boolean before(Object when) (page 254)

static Calendar getInstance() (page 141)

void roll(int datefield, boolean up) (page 255)

void setTime(Date date) (page 254)

Date Class

Class java.util.Date

long getTime() (page 141)

Enumeration Interface

Interface java.util.Enumeration

boolean hasMoreElements() (page 392)

Object nextElement() (page 392)

HashMap Class

Class java.util.HashMap

boolean containsKey(Object key (page 367)

Object get(Object key) (page 368)

Object put(Object key, Object value) (page 367)

Object remove(Object key) (page 367)

Iterator Interface

Class java.util.Iterator

boolean hasNext() (page 121)

LinkedList Class

Class java.util.LinkedList

void addFirst(Object value) (page 368)

((myFile)cache.get(fname)).getLastModified()) { cache.put(fname, readFile(fname)); } boolean remove(Object value) (page 368)

Object removeLast() (page 367)

List Class

Class java.util.List

Interator iterator() (page 121)

EntityBean Interface

Interface javax.ejb.EntityBean

void ejbRemove() (page 39)

void ejbActivate() (page 39)

void ejbPassivate() (page 39)

void ejbLoad() (page 27)

void ejbStore() (page 27)

void setEntityContext(EntityContext ectx) (page 27)

void unsetEntityContext(EntityContext ectx) (page 27)

SessionBean Interface

Interface javax.ejb.SessionBean

void ejbRemove() (page 79)

void ejbActivate() (page 79)

void ejbPassivate() (page 79)

UserTransaction Interface

Interface javax.jts.UserTransaction

void begin() (page 67)

void commit() (page 67)

void rollback() (page 67)

RemoteObject Class

Class javax.rmi.PortableRemoteObject

static object narrow(Object narrowFrom, Class narrowTo (page 90)

Cookie Class

Class javax.servlet.http.Cookie

void setPath(String path) (page 149)

void setMaxAge(int expire) (page 149)

void setDomain(String domainstring) (page 149)

Cookie(String name, String value) (page 149)

HttpServlet Class

Class javax.servlet.http.HttpServlet

void service(HttpServletRequest request, HttpServletResponse response) page 145

HttpServletRequest Interface

Class javax.servlet.http.HttpServletRequest

Cookie[] getCookies() (page 150)

HttpServletResponse Interface

Class javax.servlet.http.HttpServletResponse

void addCookie(Cookie cookie) (page 150)

void sendRedirect(String urlstring) (page 152)

void setDateHeader(String name, long datevalue) (page 152)

void setHeader(String name, String value) (page 152)

ServletConfig Interface

Interface javax.servlet.ServletConfig

String getInitParameter(String parametername) (page 152)

ServletContext getServletContext() (page 153)

ServletRequest Interface

Interface javax.servlet.ServletRequest

String getProtocol() (page 152)

ServletResponse Interface

Interface javax.servlet.ServletResponse

PrintWriter getWriter() (page 154)

void setContentType(String type) (page 154)

Box Class

Class javax.swing.Box

Component createRigidArea(Dimension d) (page 244)

DefaultCellEditor Class

Class javax.swing.DefaultCellEditor

void fireEditingStopped() (page 254)

Object getCellEditorValue() (page 254)

Component getTableCellEditorComponent(JTable table, Object value, boolean isSelected, int row, int column) (page 255)

JButton Class

Class javax.swing.JButton

void addActionListener(ActionListener l) (page 250)

JComponent Class

Class javax.swing.JComponent

void setAlignmentY() (page 274)

JFrame Class

Class javax.swing.JFrame

void addWindowListener(WindowListener l) (page 251)

JLabel Class

Class javax.swing.JLabel

void setIcon(Icon icon) (page 252)

JScrollPane Class

Class javax.swing.JScrollPane

JScrollBar getHorizontalScrollBar() (page 244)

void setColumnHeaderView(Component view) (page 245)

void setViewport(JViewport viewport) (page 245)

JTable Class

Class javax.swing.JTable

int getRowHeight() (page 293)

int getRowMargin() (page 293)

ListSelectionMode getSelectionModel() (page 243)

JTableHeader getTableHeader() (page 244)

boolean isCellEditable(int row, int column) (page 240)

void setAutoResizeMode(int mode) (page 239)

void setDefaultRenderer(Class columnClass, TableCellRenderer renderer) (page 253)

JTree Class

Class javax.swing.JTree

void makeVisible(TreePath path) (page 251)

void setSelectionRow(int row) (page 251)

JViewPort Class

Class javax.swing.JViewport

Point getViewPosition() (page 244)

void setView(Component view) (page 244)

Point getViewPosition() (page 244)

ListSelectionModel Interface

Interface javax.swing.ListSelectionModel

void addListSelectionListener(ListSelectionListener l) (page 273)

SwingUtilities Class

Class javax.swing.SwingUtilities

static void invokeLater(Runnable run) (page 255)

DefaultTableCellRenderer Class

Class javax.swing.table.DefaultTableCellRenderer

Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected, boolean hasFocus, int row, int column) (page 252), (page 253)

DefaultTableModel Class

Class javax.swing.table.DefaultTableModel

void fireTableStructureChanged() (page 242)

TableColumn Class

Class javax.swing.table.TableColumn

void setCellRenderer(TableCellRenderer renderer) (page 243), (page 253)

TableColumnModel Interface

Interface javax.swing.table.TableColumnModel

TableColumn getColumn(int index) (page 235)

int getColumnCount() (page 293)

int getColumnMargin() (page 293)

DefaultMutableTreeNode Class

Class javax.swing.tree.DefaultMutableTreeNode

DefaultMutableTreeNode(Object node) (page 250)

Enumeration depthFirstEnumeration() (page 251)

TreePath Class

Class javax.swing.tree.TreePath

TreePath(Object singlePath) (page 251)

Any Class

Class org.omg.CORBA.Any

void insert_double(double doublevalue) (page 126)

void insert_string(String stringvalue) (page 126)

IntHolder Class

Class org.omg.CORBA.IntHolder

IntHolder() (page 121)

ORB Class

Class org.omg.CORBA.ORB

status ORB init(String[] args, Properties props) (page 85), (page 117)

Any create_any() (page 125)

void connect(Object object) (page 85), (page 118)

void disconnect(Object object) (page 118)

String object_to_String(Object object) (page 87)

Object resolve_initial_references(String servicename) (page 86)

Object string_to_object(String stringvalue) (page 88)

NameComponent Class

Class org.omg.CosNaming.NameComponent

NameComponent(String nameid, String kind) (page 85), (page 119), (page 120), (page 122), (page 125)

NamingContext Interface

Interface org.omg.CosNaming.NamingContext

NamingContext bind_new_context(NameComponent[] nc) (page 85)

Object resolve_initial_references(String servicename) (page 86)

JNI C methods

CallVoidMethod(JNIEnv *env, jobject object, jmethodId methodid, object arg1) (page 219)

void DeleteLocalRef(JNIEnv *env, jobject localref) (page 228)

void ExceptionClear(JNIEnv *env) (page 219)

void ExceptionDescribe(JNIEnv *env) (page 220)

jthrowable ExceptionOccurred(JNIEnv *env) (page 220)

jclass FindClass(JNIenv *env, const char *name) (page 216)

jfieldID GetFieldID(JNIEnv *env, jclass class, const char *fieldname, const char *fieldsig) (page 220)

jint* GetIntArrayElements(JNIEnv *env, jintArray array, jboolean *iscopy) (page 219)

jobject GetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize index) (page 219)

jclass GetObjectClass(JNIEnv *env, jobject obj) (page 219)

jmethodID GetMethodId(JNIEnv *env, jclass class, const char *methodname, const char *methodsig) (page 219)

const char* GetStringUTFChars(JNIEnv *env, jstring string, jboolean *iscopy) (page 210)

const jchar *GetStringChars(JNIEnv *env, jstring string, jboolean *iscopy) (page 213)

jint JNI_CreateJavaVM(JavaVM **pvm, void **penv, void *args) (page 229)

jint MonitorEnter(JNIEnv *env, jobject object) (page 219)

jint MonitorExit(JNIEnv *env, jobject object) (page 219)

jbyteArray NewByteArray(JNIEnv *env, jsize length) (page 210), (page 215)

jobject NewGlobalRef(JNIEnv *env, jobject object) (page 228)

jintArray NewIntArray(JNIEnv *env, jsize length) (page 219)

jarray NewObjectArray(JNIEnv *env, jsize length, jclass elementClass, jobject initialElement) (page 224)

jstring NewStringUTF(JNIEnv *env, const char *bytes) (page 212), (page 213), (page 216)

void ReleaseByteArrayElements(JNIEnv *env, jbyteArray array, jbyte *elems, jint mode) (page 215)

void ReleaseStringChars(JNIEnv *env, jstring string, const jchar *chars) (page 213)

SetByteArrayRegion(JNIEnv *env, jbyteArray array, jsize startelement, jsize length, jbyte *buffer) (page 210), (page 215)

SetIntArrayRegion(JNIEnv *env, jintArray array, jsize startelement, jsize length, jint *buffer) (page 219)

void SetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize index, jobject value) (page 216)

Index

A
Abstract Window Toolkit
  see AWT
AbstractListModel class 246
AbstractListSelectionModel class 246
AbstractTableModel class 240
ACC_MACHINE_COMPILED value 353
ActionListener interface 251
actionPerformed method 251
activity diagram 6
actor (person or software) 2
add method 141
addActionListener method 250
addBatch method 136
addCount method 349
addWindowListener method 251
afterBegin method (sess synch) 61
afterCompletion method (sess synch) 62
agent password 311
Alarm monitor 322, 326
algebra library 217
AllPermission 433
analyzing performance 354
Any type (IDL) 114, 124
applet viewer 421
applets
  applet viewer 421
  deploy 405
  download speed 339
  image downloading 339
  signed 417
  signed in JDK 1.1 422
  SignedAppletDemo example code 426
application servers
  configuring 63
  described 15
ARCHIVE tag 341
ArrayHandler class 228
arrays
  IDL types 111
  JNI types 213
  multidimensional 217
  object 216
  pinning 215
AttachCurrentThread function 229
attaching threads 229
attributes (IDL) 112
auction house
  activity diagram 6
  APIs to use 7
  AuctionItem entity Bean 22
  BidderBean session Bean 25
  caching queries 363
  CORBA 115
  enterprise Beans 14, 17
  project described 3
  project modeled 2
  project requirements 2
  RegistrationBean entity Bean 24
  RMI 93
  searches 29
  SellerBean session Bean 25
  web pages 7
AuctionClient example code 270
AuctionItem class
  example code 33
  remote interface 22
AuctionItemBean class 17
  entity bean class 23
  example code 34
  interfaces and classes 22
  searches 70
AuctionItemHome class
  example code 34
  findAllMatchingItems 30, 32
  home interface 22

AuctionItemPK class
    example code 37
    primary key class 23
AUCTIONITEMS table 23
AuctionServlet class
    container-managed example 44
    customFind 124
    described 20, 145
    example code
        CORBA 191
        RMI 173
    listAllItems(out) 21
    methods 21
    searchItems method 29
        bean managed 68
        container managed 31
    service methods 147
auditAccounts method 105, 120
authentication, password 422
auto commit 60
AWT Toolkit debugging 318
AWTEventListener class 319
AWT-EventQueue-0 thread 329
AWT-Modal thread 330
AWTPermission 433
AWTTreeLock class 329
B
backing out transactions 60
batch database access 136
BatchUpdateException class 136
bean-managed
    persistence 55
    transactions 63
before method 255
beforeCompletion method (sess synch) 62
begin transaction 61
Bidder class
    example Code 39
    remote interface 25
BidderBean class 17
    example code 40
    getMatchingItemsList method 29, 31, 32
    interfaces and classes 25
BidderHome class
    example code 40
    home interface 25
Binary Large Objects (BLOBs) 140
BinClass lock monitor 325
bind method 83
boolean flag, debug 301
boot classpath option 303
BorderLayout layout manager 237
Box class 244, 274
branches of code, removing 302
breadthFirstEnumeration method 249
breakpoints
    clearing 309
    servlets 317
    setting 306
Bug Parade 323
byte compiler 351
ByteArrayInputStream class 137
C
C source file (JNI) 210
Cache-Control value 151
caching
    Cache-Control value 151
    database connections 344
    database results 133
    Expires header 152
    many objects 365
    Most Recently Used (MRU) 365
    one object 363
    pages 151
    performance tuning 363

Pragma value 151
queries 363
Secure Socket Layer (SSL) 152
Calendar class 141, 254, 255
CallableStatement class 131
callback from server 97, 105
cell editing 253
cell rendering 252
CellRenderer class 235
char types (IDL) 109
Character Large Objects (CLOBs) 140
check list, debugging 330
Child death monitor 326
CLAPACK/LAPACK algebra library 217
Class linking lock monitor 325
Class loading lock monitor 326
classes
CLASSPATH variable 299
Java virutal machine 299
loading 299, 300
system 299
classpath command 299
CLASSPATH variable 299
servlet debugging 315
version issues 334
clickCountToStart method 253
close method 137
Code rewrite lock monitor 325
CODE tag 341
codebase property 96
Collections API
caching 363
marshalling 98
COMM_FAILURE message 87
commands
classpath 299
cpu=sample option 360
cpu=times option 360
idltojava 116
jar command 400
jarsigner 420
java bootclasspath 303
java -debug 314
java -Djava.compiler=NONE 329, 354
java -Xdebug 302
java -Xrunhprof 360
cpu=times 349
help 355
java_g 302
javac -classpath classpath 299
javac -g 303
javac -sysclasspath classpath 299
javac -Xbootclasspath classpath 299
javah -jni 209
javap 223
javap -public 319
jdb 304
jdb -Xbootclasspath 304
jidl 116
keytool 420
kill -QUIT process_id 321
Mermory meter (Windows platform) 363
rmi 90
sar 361
SQL 55
sun.servlet.http.HttpServer 314
trace (Linux) 362
traceplus (Windows platform) 363
truss 361
commits
JDBC drivers 63
transaction commit mode 63
transactions 62
Common Object Request Broker
see CORBA

communication services
multitiered applications 19
RMI 97
Compiler class 352
compiler optimization 302
complete transaction 62
complications with data marshalling 98
ComponentPrinterFrame example code 283
components
CellRenderer object 235
container 235
heavyweight 234
layered 235
lightweight 234
peer 234
peerless 234
print AWT component 257
printing multiple components 264
Z-ordering 235
ComponentUI class 233
Connection class
callable Statements 131, 132
controlling transactions 142
RegistrationServer example 56, 57
ConnectionReaper class 346
connections
caching 344
database 70, 130
deadlocks 346
hangs 346
performance 344
pooling 27, 344
rollback method 140
wrapper classes 345
constant pool 354
constructors and JITs 353
container components 235
container-managed beans
defined 26
deployment descriptor 28
finder methods 29
persistence 26
session synchronization 60
containers
classes 25
container-managed beans 26
defined 18
deployment tools 19
context
EntityContext 27
transaction 64
Context class 19
CONTIGUOUS_TREE_SELECTION mode 247
Cookie class 149
cookies
restricting access 152
retrieving 150
servlets 149
setting 149
CORBA
defined 84
garbage collection 121
lookup services 83
name server, starting 86
object server, starting 86
ORBInitialPort class 118
ORBSingletonClass 119
RegistrationServer 85
type methods 114
using lookup code with 107
CosNaming IDL module 84
cpu=samples option 360
cpu=times option 360
create

bean-managed Bean 56
container-managed Bean 26
Interoperable Object Reference (IOR) 87
searches 71
createRigidArea method 244, 274
createStatement method 132
CustomButtonEditor class 239
CustomButtonRenderer class 239
CustomRenderer class 239
customSearch method 115
CW thread state 323
D
data
columns and rows 238
communication services 19
marshalling
Collections API 98
complications 98
described 20, 92, 97
IDL 116
ORBs 117
models 238
data types 109
Any type 114
arrays 111
char type 109
IDL
attribute 112
enum 113
exceptions 111
mappings 108
mapping in JDBC 143
methods and IDL operations 111
module 110
objects (IDL) 115
principal type 114
strings 109
structs 113
unions 113
DATA_CONVERSION exception 108
database
connection 56, 70
servers 15
SQL commands 55
Date class 141, 144
DATE type and JDBC mappings 144
dbmaplib.cc library 221
dead connection reaper 346
deadlocks, connections 346
debugging information 301
AWT Toolkit 318
AWTEventListener class 319
check list 330
debug boolean flag 301, 303
Java WebServer debug mode 316
jdb command 304
platform identification 322
release identification 321
remote debugging 310
remove branches of code 302
servlet debugging 314
session log 314
stack trace analysis 320
thread packages 322
version issues 334
DEFAULT_FRAME_LAYER band 236
DEFAULT_LAYER band 236
DefaultCellEditor class 253, 254, 255
DefaultListModel class 246
DefaultListSelectionModel class 246
DefaultTableModel class 239
DefaultTreeModel class 247
DefaultTreeMutableTreeNode class 247
DefaultXXXCellRenderer class 252
DELETE requests 147, 149

DeleteGlobalRef function 227
deleteRow method 136
DeleteWeakGlobalRef function 227
deployment
  applets 405
  JAR files 401
  JNDI 19
  Solaris platform 406
  tools and containers 19
  Windows platform 411
deployment descriptor
  AuctionItemBean 32
  beanhomename value 83
  finder descriptors 32
  finder methods 29, 32
  RegistrationBean 28
  transaction settings 63
  XML version 29, 63
depthFirstEnumeration method 249
destroy method 146
DestroyJavaVM function 230
diagnosing problems 302
Directory Name Service (DNS) 82
DISCONTIGUOUS_TREE_SELECTION mode 247
downloading applets 339
DRAG_LAYER band 236
drawRect method 258
Driver class 56, 70, 129
DriverManager class 56
drivers
  Driver class 129
  jdbc-odbc bridge 129
  JDCConnectionDriver class 129
  ODBC driver 129
  types 130
Duke's Auction web pages 8
dumpStack method 321
dynamic library 211
Dynamic loading lock monitor 325
E
editing cells 253
ejbCreate method
  Bean managed 56
  container managed 26
  JNDI lookup services 83
  searches 71
ejbFindByPrimaryKey method
  bean-managed search 59
  container-managed search 59
ejbLoad method
  bean managed 57
  container managed 27
ejbStore method 27
ENOENT error 362
ENOPERM error 362
enterprise Beans
  defined 14
  interfaces and classes 17
  JNDI 19
  RMI to marshal data 20
  specifications 17, 19
entity Beans
  defined 16
  persistent data storage 20
EntityContext instance 27
enum type (IDL) 113
environment problems 298
error codes (HTTP) 153
escape keyword 142
event handling (Project Swing) 255
example code
  AuctionItem class 33
  AuctionItemBean class 34
  AuctionItemHome class 34
  AuctionItemPK class 37

AuctionServlet class 44
CORBA 191
RMI 173
Bidder class 39
BidderBean class 40
BidderHome class 40
FileIO class 428
HttpServer class 384
HttpServerWorker class 383
InlineMe class 348
JDCConnectionDriver class 391
JDCConnectionPool class 391
JDCConnectoin class 393
JNI example 207
Login servlet 200
MyApplet class 369
PasswordSecurityManager class 427
Pool class 380
printing
AuctionClient class 270
ComponentPrinterFrame class 283
Print2Button class 287
PrintButton class 280
PrintButton class (Swing) 281
PrintPanel class 282
Report class 288
SalesReport class 291
ShapesPrint class 284
Project Swing
ResultsModel class 241
SimpleSearchTree class 249
SimpleTree class 248
ReadFile JNI example 207
Registration class
bean managed 37
RMI 169
Registration.idl file (CORBA) 187
RegistrationBean class
bean managed 72
connection pool 395
connection pooling 347
container managed 38
RegistrationHome class
bean managed 37
RMI 169
RegistrationPK class
bean managed 39
RMI 170
RegistrationServer class
CORBA 181
lookup 161
RMI 165
ReturnResults class (RMI) 172
SearchBean class 77
searchItems method 77
Seller class
bean managed 43
CORBA 191
SellerBean class
bean managed 75
container managed 43
CORBA 187
lookup 163
SellerHome class 43
Session class 204
SessionCache class 205
SignedAppletDemo class 426
TableExample3 class 355
Worker class 382
XPM definition 375
ExceptionClear function 225
ExceptionOccured function 225
exceptions
IDL exceptions 111
web servers 318
executeQuery method 132

executeUpdate method 132
Expires expiration header 152
exportObject method 90
F
FileIO example code 428
FilePermission class 434
Finalize me queue lock monitor 326
Finalizer Thread 322
findAllMatchingItems method 29
FindClass function 216
finder descriptors 32
finder methods 29
  Any IDL type 124
  bean managed 68
  container managed 29
  deployment descriptor 29
findLowCreditAccounts method 99, 115, 121
fireEditingStopped method 254
fireTableCellUpdated method 240
fireTableStructureChanged method 241
first method 135
forName method 129
FORTRAN language 217
functions
  CallVoidMethod 219
  DeleteLocalRef 228
  ExceptionClear 219
  ExceptionDescribe 220
  ExceptionOccurred 220
  FindClass 216
  GetFieldID 220
  GetIntArrayElements 219
  GetMethodID 219
  getMethodID 219
  GetObjectArrayElement 219
  GetObjectClass 219
  GetStringChars 213
  GetStringUTF 210
  JNI_CreateJavaVM 229
  MonitorEvent 219
  MonitorExit 219
  NewByteArray 210, 215
  NewGlobalRef 228
  NewIntArray 219
  NewObjectArray 224
  NewStringUTF 213, 216
  ReleaseStringChars 213
  SetByteArrayRegion 211, 215
  SetIntArrayRegion 219
  SetObjectArrayElements 216
G
-g none option 303
garbage collection
  CORBA 121
  Hotspot 351
  improved in Hotspot 351
  JNI 215
  leaseValue property 99
  memory issues (JNI) 227
  pinning array 216
  RMI 99
generational copying 351
GET
  reading value 153
  requests 147, 148
GetArrayLength function 217
GetByteArrayElements function 214
GetByteArrayRegion function 214
getBytes method 137
getCellEditorValue method 254
getColumnCount method 240
getConnection method 70, 345
getContentPane method 237
getContentType method 156
getDouble method 123, 135

GetField function 225
getFontMetrics method 293
getGlassPane method 238
getHorizontalScrollBar method 244, 274
getImageableHeight method 265
getImageableWidth method 265
getInstance method 141
getInt method 127
getMatchingItemsList method 70
getMetaData method 135
getMethod 156
GetObjectArrayElement function 217
getProtocol method 152, 154
getResultSet method 142
getRowCount method 240
getString method 58, 123, 134
GetStringChars function 213
GetStringUTFChar function 212
getTableCellEditorComponent method 254, 255
getTableCellRendererComponent method 253
getTime method 141, 144
getTreeLock monitor lock 329
getType method 134
getValueAt method 240
getXXXCellRenderer methods 252
GlassPane component 238
Graphics class 258, 264, 293
graphics context 256
Graphics2D class 265
green threads 228
H
hanging, connections 346
Has finalization queue lock monitor 326
hash table 133
HashMap class 363
header file (JNI) 209
Heap Analysis Tool (HAT)
  described 357
  profiling 355
Heap lock monitor 325
heavyweight components 234, 237
home interface defined 18
hostname property 96
HotSpot virtual machine 351
HTML Converter
  Solaris 408
  Windows 412
HTTP protocol
  HTTPS protocol 159
  HttpServletRequest object 147
  HttpServletResponse object 147
  requests 148
  sendRedirect method 152
  server 145
HTTPS protocol 159
HttpServer class 343
HttpServer example code 384
HttpServerWorker class 343
HttpServerWorker example code 383
I
I/O Monitor 326
IDL
  any type 114
  arrays 111
  attrbutes 112
  char types 109
  enum 113
  exceptions 111
  idltojava command 116
  interface types 110
  jidl command 116
  language mappings 84
  mapping scheme 108
  mapping setup 110

methods and IDL operations 111
module type 110
object type 115
ORB 117
principal type 114
string types 109
structs 113
unions 113
idltojava command 116
-iiop option 89
illegal page fault 321
images, downloading 339
InetAddress class 91
init method 146
initArgs parameter 146
initdb native method 221
InitialContext class 71
described 83
RMI-IIOP 89
inlined methods 348
InlineMe example code 348
insertRow method 136
installation problems 298
Interface Definition Language
see IDL
Internet Inter-ORB protocol (IIOP) 87
Interoperable Object Reference (IOR)
creating 87
described 87
invokeAndWait method 255
invokeLater method 255
invoker methods 352
isCellEditable method 238, 240
isolation levels
described 66
introduced 63
J
jar command 400
JAR files
command 400
deploying 401
file format 399
jarsigner command 420
performance 341
signed 417
jarsigner command 420
Java Archive (JAR) files
see JAR files
java -debug command 314
Java Debug Interface (JDI) 304
java Debug Wire Protocol (JDWP) 304
java -Djava.compiler=NONE command 329, 354
Java Naming and Directory Interface (JNDI)
described 82
InitialContext class 71
lookup services 19
Java Native Interface (JNI)
see JNI
Java Plug-in software
Solaris 406
Windows 411
Java Remote Method Protocol (JRMP) 88
Java Runtime Environment (JRE) 412
Java stack lock monitor 326
Java virtual machine
class loading 299
debug information 302
Debug Interface (JVMDI) 304
green threads 228
Hotspot 351
internal error 321
invocation 228
invoker methods 352
JBUG architecture 304

jdb debug command 304
JITs 352
method inlining 348
options 228
performance features 348
profile report 355
profiling 349
running threads 360
security manager 422
signals 320
thread and monitor snapshot 320
thread packages 322
threads 229
Java WebServer
debug mode 316
exceptions 318
java -Xdebug command 302
java -Xrunhprof command 360
cpu=times option 349
help option 355
java.awt.event.ActionListener interface 251
java.awt.event.WindowListener interface 251
java.awt.Graphics class 258, 264, 293
java.awt.Graphics2D class 265
java.sql.Connection class
controlling transactions 142
making connections 131
RegistrationPK example 57
statements 132
java.sql.PreparedStatement class
controlling transactions 141
Images 137
prepared statements 133
RegistrationBean example 59
RegistrationServer example 123, 127
java.sql.ResultSet class
caching results 134
images 137
RegistrationBean example 58
RegistrationServer example 123, 127
updating 135
java.util.Calendar class 141, 254, 255
java.util.Date class 141
java.utill.Calendar class 255
java_g command 302
Java_ReadFile_loadFile method 208
javac -g command 303
javah -jni command 209
javap command 223
javap -public command 319
javas.swing.Box class 244, 274
JavaServer Web Development Kit (JWSDK) 314
JavaVMInitArgs structure 228
JavaVMOption value 228
javax.servlet.ServletRequest interface 152, 154
javax.servlet.ServletResponse interface 154
javax.swing.DefaultCellEditor class 254, 255
javax.swing.JButton class 250
javax.swing.JFrame class 251
javax.swing.JLabel class 252
javax.swing.JScrollPane class 244, 245, 274, 275
jboolean type 214
JBUG architecture 304
JButton class 250
jbyte type 214
jchar type 214
JComponent class 330
JContentPane component 237
jdb debug tool

auto session 313
breakpoints 306, 309
clear command 309
cont command 309
host command 317
jdb.ini file (auto debug) 313
list command 306
locating source 307
next command 307
object details 310
print command 308
quit command 310
remote debugging 310
quit command 313
resume command 312
starting 311
thread command 311
servlet debugging 315
session log 314
starting 305
step up command 307
stop command 307, 308
where command 306
jdb.ini file 313
JDBC
batch jobs 136
binary data 136
BLOBs 139
caching 133
CLOBs 139
commit mode 63
connections 130
data type mapping 143
driver types 130
drivers 129
escaping characters 142
images 136
jdbc-odbc bridge 129
object storage and retrieval 137
ODBC driver 129
pool driver 102
query PreparedStatement 132
query Statement 132
ResultsSet interface 133
roll backs 140
setLogStream method 131
statement classes 131
transactions 60, 140
update PreparedStatement 133
update Statement 132
updating many tables 129
jdbc-odbc bridge 129
JDCConnection
class 346
example code 393
JDCConnectionDriver
class 129, 345
example code 391
JDCConnectionManager class 345
JDCConnectionPool
class 345
example code 391
jdouble type 214
jfloat type 214
JFrame class 251
jidl command 116
jint type 214
jintArray instance 217
JInternalFrame class 235
JIT_ARGS environment variable 353
JLabel class 252
JLayeredPane component 235
JList component 246
jlong type 214
JNI
arrays 213

C source file 210
call methods 223
Class linking lock 325
compile example 209
described 207
dynamic library 211
example code 207
fields 225
garbage collection 215, 216, 227
global reference lock monitor 325
green threads 228
header file 209
Java virtual machine invocation 228
javah -jni command 209
javap command 223
memory 227
method signature parameters 209
native code library 208
pinning lock 322
pinning lock monitor 325
retrieve class reference 222
retrieve method identifier 222
shared object library 211
string types 212
synchronization 226
threads 226
type to function mappings 214
Xbase database 220
JNI_ABORT value 215
JNI_COMMIT value 215
JNI_CreateJavaVM function 228
JNI_DestroyJavaVM function 228
JNI_FALSE value 212
JNI_TRUE value 212
JNIEnv parameter 210
jobject parameter 210
jobject type 214
jobjectarray type 216
JRootPane class 235
JScrollPane class 244, 245, 274, 275
jshort type 215
jstring parameter 210
jstring type 212
JTree class 247
Just-In-Time (JIT) compiler
boundary checking 354
constructor code 353
described 352
disabling 311, 354
performance 348
K
keystore database
described 420
import public key certificate 421
keytool command 420
kill -QUIT process_id command 321
L
layered components 235
leaseValue property 89, 99
length method 353
lightweight components 234
Lightweight Directory Access Protocol (LDAP) 82
linear algebra library 217
LinkedList class 363, 366
Linux platform 362
ListModel class 246
ListSelectionModel class 246
ListSelectionModel object 242
load
bean-managed data 57
container-managed data 27
refresh method 57
loadFile function 208
loading classes 300
locks

Collections API 366
getTreeLock 329
monitors 324
synchronization and performance 350
Login servlet example code 200
long raw 136
longvarbinary 136
lookup method 88
lookup services
change service 83
CORBA 84
CosNaming 84
defined 82
directory services 82
JNDI 82
multitiered applications 19
performance 91
M
MANDATORY attribute 65
mappings (IDL)
compiling 116
setup 110
MARSHAL exception 109
marshalling
Collections API 98
complications 98
data 20, 92
defined 97
IDL 116
ORBs 117
stub and skel classes 25
MDialogPeer class 330
memory
issues (JNI) 227
leaks 99, 121
Memory meter (Windows platform) 363
methods
actionPerformed 251
add 123, 141
addActionListener 250
addCookie 150
addWindowListener 251
append 263
before 255
begin 67
bind 85
bind_new_context 85
close 137
commit 68
connect 85, 118
Cookies 149
create_any 125
createRigidArea 244, 274
createStatement 132, 141
DefaultMutableTreeNode 250
defaultPage 262
depthFirstEnumeration 251
disconnect 118
drawRect 258
ejbActivate 39, 79
ejbPassivate 39, 79
ejbRemove 39, 79
executeBatch 136
executeQuery 132, 133
executeUpdate 59, 132, 142
fireEditingStopped 254
fireTableStructureChanged 242
first 135
getBytes 137
getCellEditorValue 254
getColumn 235
getConnection 56, 131
getCookie 150
getDate 131
getDouble 135
getHorizontalScrollBar 244, 274

getImageableX 258
getImageableY 258
getInitParameter 152
getInstance 141
getInt 127
getMetaData 135
getPrinterJob 262
getProtocol 152
getResultSet 58, 141, 142
getSecurityManager 103, 168
getSelectionModel 243, 273
getServletContext 153
getString 58, 123, 134
getTableCellEditorComponent 255
getTableCellRendererComponent 252, 253
getTableHeader 244, 274
getTime 141
getViewPosition 244, 275
getWriter 154
hasNext 121
IDL mappings 111
init 85, 117
inlining 348
insert_double 126
insert_string 126
IntHolder 121
invokeLater 256
isCellEditable 240
Iterator 121
LoadLibrary 209
loadLibrary 217, 218
lookup 88, 90
makeVisible 251
NameComponent
  Any type 125
  CORBA callbacks 122
  CORBA clients 120
  RegistrationServer example 85, 119
narrow 90
next 127, 134
object_to_string 88
oreoareStatenebt 123
pageDialog 262
prepareCall 131
prepareStatement
  controlling transactions 141
  prepared statements 133
  RegistrationBean example 57, 58
  RegistrationServer example 127
  searching 71
print 258, 262
printDialog 261
rebind 103, 168
registerOutParameter 131
resolve 86
resolve_initial_references 86
roll 255
rollback 67, 142
security permissions 447
sendRedirect 152
setAutoCommit 141
setAutoResizeMode 239
setBinaryStream 137
setCellRenderer 243, 253, 274
setColumnHeaderView 245, 275
setContentType 154
setDate 141
setDateHeader 152
setDefaultRenderer 253
setDomain 150
setDouble 59, 123, 127, 141
setFailureHandler 103, 168
setHeader 152
setIcon 252

setInt 137
setMaxAge 149
setOrientation 263
setPageable 263
setPath 149
setSecurityManager 103, 168
setSelectionRow 251
setString 59, 131, 133
setTime 254
setView 244, 245, 275
setViewport 245, 275
signature (JNI) 209
string_to_object 88
supportsResultSetConcurrency 135
toArray 123
TreePath 251
updateDouble 135
valueOf 126
windowClosing 251
modal Dialog 329
MODAL_LAYER band 236
ModalThread class 330
models
list 246
model-delegate architecture 233
Model-View-Controller (MVC) 233
table 239
tree 247
module type 110
Monitor cache lock monitor 325
Monitor IO lock monitor 325
Monitor registry 326
monitors
common registered 325
getTreeLock 329
monitor operations 324
registry 326
stack trace dump 324
sysMonitor 324
wait state 327
Most Recently Used (MRU) cache 365
Motif components 234
moveToFront method 237
moveToInsertRow method 136
MS thread state 323
multidimensional arrays 217
MULTIPLE_INTERVAL_SELECTION
mode 246
multitiered applications
behavior 19
disadvantages to 13
easy way to write 13
multitiered architecture
application servers 15
database servers 15
defined 15
MW thread state 323
MyApplet example code 369
N
Name and type hash table lock monitor 326
NameComponent helper class 84
naming services
described 82
lookup method 88
NamingContext interface 84
native code library 208
NetPermission 435
Netscape communicator installation 408
NEVER attribute 65
NewByteArray function 213
NewFloatArray function 213
NewGlobalReference function 227
NewObjectArray function 216
NewWeakGlobalRef function 227
next method 127, 134
NIS+ 82

NO_SUCH_PAGE value 265
O
object arrays 216
Object Request Broker (ORB)
    described 117
    plugging in 119
    see ORBs
object type (IDL) 115
object_to_string method 87
ObjectOutputStream class 137
OMG Object Transaction Service (OTS) 64
on-stack replacement, 351
Open DataBase Conectivity (ODBC) 129
OPTIONS requests 147, 149
orb.create_any 125
ORBClass 119
ORBInitialPort class 118
-ORBInitialPort portnumber 118
ORBs
    ORBClass 119
    ORBInitialPort class 118
    ORBSingletonClass 119
ORBSingletonClass 119
P
packages
    module IDL type 110
    threads 322
page caching 151
paint method 329
paintChildren method 330
paintComponent method 329
PALETTE_LAYER band 236
password identification 422
password, agent 311
PasswordSecurityManager class 423
PasswordSecurityManager example code 427
peer components 234
peerless components 234
performance
    analysis 354
    caching 363
    JITs 352, 354
    Linux platform commands 362
    lookup services 91
    profiling 349
    sar tool 361
    Solaris platform commands 361
    time used 360
    trace command (Linux) 362
    truss command 361
    Windows platform commands 363
performWork method 343
permissions
    see also security
    see also security manager
    AllPermission 433
    applets 417
    AWTPermission 433
    deployment 409
    FilePermission 434
    methods
        addAWTEventListener 449
        canRead 450
        canWrite 450
        ClassLoader 453
        createNewFile 450
        createTempFile 450
        delete 450
        deleteOnExit 450
        enableReplaceObject 451
        enableResolvedObject 451
        exists 450
        FileInputStream 450
        FileOutputStream 451

forName 452
getClasses 452
getClassLoader 452
getConstructors 453
getDeclaredClasses 452
getDeclaredConstructors 452
getDeclaredFields 452
getDeclaredMethods 452
getFields 453
getMethods 453
getPrintJob 449
getProtectionDomain 453
getSystemClibpoard 449
getSystemEventQueue 449
isDirectory 450
isFile 450
isHidden 450
lastModified 450
length 450
list 450
listFiles 450
mkdir 450
NetPermission 435
ObjectInputStream 451
ObjectOutputStream 451
PropertyPermission 436
RandomAccessFile 452
ReflectPermission 437
registerEditor 450
removeAWTEventListener 449
renameTo 450
setComposite 448
setDesignTime 449
setEditorSearchPath 450
setGuiAvailable 449
setLastModified 450
setReadOnly 450
synchroinized 449
Window 449
platform methods 447
RMI 94
SecurityPermission 440
SerializablePermission 443
SocketPermission 444
syntax 432
persistent data
enterprise Beans 20
transient keyword 26
pinning array 215
pluggable look and feel (PLAF) 233
policy files
install 409, 413
signatures 417
types 409, 412
policy property 96
Pool class 343
Pool example code 380
pooling
connection 344
constant, class file 354
thread 342
PortableRemoteObject 90
POST
reading the value 153
requests 147, 148
postorderEnumeration method 249
Pragma value 151
preorderEnumeration method 249
prepareCall method 131
PreparedStatement class
controlling transactions 141
CORBA callbacks 123
escaping characters 143
images 137
prepared statements 131, 133
RegistrationBean example 56, 59

RegistrationServer example 127
scrollable result sets 134
searches 70
prepareStatement method
controlling transactions 141
prepared statements 133
RegistrationBean example 57, 58
RegistrationServer example 127
searches 71
primary key class
bean managed 56
container managed 23
SearchBean 70
searches 68
principal type (IDL) 114
Print2Button example code 287
PrintButton example code 280, 281
printing
across pages 265
AWT component 257
collection of pages 262
dialog box 261
example code
AuctionClient class 270
ComponentPrinterFrame class 283
Print2Button class 287
PrintButton class (Project Swing) 281
PrintButton class(AWT) 280
PrintPanel class 282
Report class 288
SalesReport class 291
ShapesPrint class 284
footer 264
graphics 259
graphics context 256
multiple components 264
Project Swing Components 259
sales report 269
setup dialog 262
tables 268
PrintPanel example code 282
printStackTrace method 321
private key
keystore database 420
signing JAR files 419
profiling
inlined code 349
reports 355
time used 360
program policy file 409, 412
project requirements 2
Project Swing
data models 238
event handlling 255
print sales report 269
printing 259
across pages 265
collection of pages 262
footer 264
graphics 259
tables 268
ResultsModel example code 241
searches 249
SimpleSearchTree example code 249
SimpleTree example code 248
synchronization 330
properties
codebase 96
hostname 96
leaseValue 99
policy 96
Property object 82
PropertyPermission 436
public key
certificate 159

cryptography 159
export certificate 420
import certificate 421
keystore database 420
verify signature 419
PUT requests 147, 149
Q
query statement 132
R
R thread state 322
race conditions 301
ReadFile.h header file 209
rebind method 90, 103, 168
ReflectPermission 437
refresh method 57
Registered Monitor Dump, stack traces 322
Registration class
example code
container-managed 37
RMI 169
RMI 97, 103
REGISTRATION table 24
Registration.idl file
CORBA 115
example code 187
ReturnResults 122
RegistrationBean class 17
bean managed example code 72
deployment descriptor 28
ejbCreate method 26
ejbLoad method 27
ejbStore method 27
entity context methods 27
example code
bean managed 72
connection pooling 347, 395
container managed 38
getConnection method (database) 56
interfaces and classes 24
transient keyword 26
walk through 26
RegistrationHelper class 127
RegistrationHome
example code
RMI 169
RegistrationHome class
example code 37
container managed 37
findLowCreditAccounts 122
RMI 97, 104
RegistrationImpl class
remote reference to 99
serialization 137
RegistrationPK class
bean managed 56
example code 39
container managed 37
RMI 170
RegistrationServer class
CORBA 85, 115
example code
CORBA 181
lookup 161
RMI 165
Interoperable Object Reference 87
refresh method (retrieve object) 139
RMI 92, 98
searches 93
ReleaseStringUTFChars function 213
remote
access 98
communications 97
interfaces 18
Remote Method Invocation (RMI)
see RMI
remove code branches 302

rendering
across pages 265
AWT component 257
cells 252
collection of pages 262
graphics 259
graphics context 256
printing a footer 264
printing multiple components 264
printing sales report 269
Project Swing components 259
tables 268
Report example code 288
REQUIRED attribute 65
requirements (project) 2
REQUIRESNEW attribute 65
resolved
host name 91
IP address 91
ResultSet interface
caching 134
CORBA callbacks 123
described 133
images 137
navigating 135
query returns 133
RegistrationBean example 58
RegistrationServer example 127
scrolling 134
searches 70
updating 135
ResultsModel class
described 238
example code 241
ReturnResults interface 122
example code (RMI) 172
RMI 104
RMI
data marshalling 20
described 92
garbage collection 99
interfaces and classes 93
lookup services 88
RMIFailureHandler class 103
rmiregistry 89
security permissions 94
using lookup code with 91
rmic compiler
command 89
-vcompat option 98
RMIFailureHandler method 103
RMI-IIOP
client-side lookup 90
communicate with others 92
InitialContext 89
lookup services 89
URLs 88
roll method 255
rollback method 142
rolling back transactions 60
Running or runnable thread 322
S
S thread state 323
sales report printing 269
SalesReport example code 291
sar tool 361
scale method 265
scrolling results sets 134
SearchBean class
described 70
example code 77
searches
Any IDL type 124
complex 68
customSearch (CORBA) 115
finder methods

bean managed 68
container managed 29
Project Swing 249
RegistrationServer (RMI) 93
searchItems method
described 31
example code 77
Secure Socket Layer (SSL) 152, 159
security
see also permissions
see also security manager
authentication in servlets 159
cookies 152
custom access controls 422
custom verifications 422
deployment 409
encryption algorithm 159
FileIO example code 428
HTTPS 159
PasswordSecurityManager example code 427
policy files 409, 412
public key certificate 159
restrict access (JNI) 226
security manager
see also permissions
see also security
methods
checkAccept 465
checkAccess 465
checkAwtEventQueueAccess 465
checkConnect 465
checkCreateClassLoader 465
checkDelete 465
checkExec 466
checkExit 466
checkLink 466
checkListen 466
checkMemberAccess 466
checkMulticast 466
checkPackageAccess 466
checkPackageDefinition 466
checkPrintJobAccess 466
checkPropertiesAccess 466
checkPropertyAccess 466
checkRead 466
checkSecurityAccess 467
checkSetFactory 467
checkSystem 467
checktopLevelWindow 467
checkWrite 467
SecurityManager 467
platform methods and permissions 447
writing 422
SecurityPermission 440
segmentation violation 321
selective compilation 351
Seller class
bean managed example code 43
CORBA example code 191
SellerBean class 17
auditAccounts 124
auditAccounts method 120
bean-managed example code 75
classinsertItem method
sess synch 61
container-managed example code 43
customFind 124
example code
CORBA 187
lookup 163
insertItem method
CORBA 86
interfaces and classes 25
RMI 105
updateResults 122

SellerHome class
example code 43
reference _stub 88
sendRedirect method 152
sequence problems 301
SerializablePermission 443
serialization
defined 20
RMI 92
unserializable objects 98
servers
RMI callback 97
RMI callbacks 105
service method 147, 152
ServletConfig object 146
ServletRequest interface 152, 154
ServletResponse interface 154
servletrunner program 314
servlets
authentication 159
breakpoints 317
cookies 149
debugging 314
DELETE requests 147, 149
described 145
destroy method 146
get content type 156
GET requests 147, 148
get values 156
HTTP error codes 153
HTTP requests 148
init method 146
life cycle 145
multiple-part posts 156
OPTIONS requests 147, 149
POST requests 147, 148
properties file 146
PUT requests 147, 149
reading matching lines 156
restricting access with cookies 152
service method 147
session generation 151
thin clients 14
threading 158
TRACE requests 147, 149
session Beans defined 16
Session class
described 151
example code 204
session log 314
session synchronization 60
interface 61
methods 61
SessionCache example code 205
SetArrayElements function 214
SetArrayRegion function 214
setAutoCommit method 141
setBinaryStream method 137
setCellRenderer method 253
setClip method 264
setColumnHeaderView method 245, 275
setContentType method 154
setDate method 141
setDouble method 59, 123, 127, 141
setFailureHandler method 103, 168
setIcon method 252
setInt method 137
SetIntField function 225
setLayout method 237
setLogStream method 131
setNoCache method 151
SetObjectArrayElement function 216
setPageable method 263
setRowSelectionInterval method 242
setSecurityManager method 103, 168
setSelectionMode method 246

setString method 59, 133
setTime method 254
ShapesPrint example code 284
shared object library 211
show method 330
signature, policy file 417
signed applets 417
SignedAppletDemo example code 426
SimpleDateFormat class 144
SimpleJdbTest program 305
SimpleSearchTree example code 249
SimpleTree example code 248
SINGLE_INTERVAL_SELECTION mode 246
SINGLE_SELECTION mode 246
SINGLE_TREE_SELECTION mode 247
SingletonClass 119
skel class
    backwards compatible 98
    defined 26
    IDL 116
    RMI 97
sleep method 323
sleeping threads 342
SocketPermission 444
stack traces
    Alarm monitors 322
    analyzing 320
    AWT-Modal thread 330
    dumping 302
    example 1 327
    example 2 328
    generating 320
    permissions 432
    platforms 322
    reading 302
    releases 321
    thread packages 322
    thread states 322
Statement class 131
    escaping characters 143
    scrollable result sets 134
store
    bean-managed data 58
    container-managed data 27
String intern lock monitor 326
string types (IDL) 109
string_to_object method 88
struct type (IDL) 113
stub class
    backwards compatible 98
    defined 25
    IDL 116
    lookup services 83
    RMI 97
sun.servlet.http.HttpServer command 314
SUPPORTS attribute 65
supportsResultSetConcurrency method 135
suspended threads 323, 342
SwingUtilities class 255
synchronization
    Hotspot improvements 352
    JNI 226
    performance 350
    Project Swing 330
    synchronized wait 323
    wait state 327
-sysclasspath classpath option 299
sysMonitor operation 324
System Accounting Reports tool (sar) 361
System class loader lock monitor 325
system policy file 409, 412
T
TableColumnModel interface 239
TableExample3 program 355

TableModel interface 238
tables
  custom model 240
  layout 239
  model 239
  non-scrolling columns 241
  printing 268
  rows and columns 238
thin-client applications
  defined 14
  disadvantages to 13
  easy way to write 13
  servlets 14
Thread queue lock monitor 326
Thread suspended waiting monitor lock 323
Thread waiting on a condition variable 323
Thread waiting on a monitor lock 323
threads
  attaching 229
  AWT-Modal 330
  Finalizer Thread 322
  green 228
  Hotspot improvements 352
  JNI 226
  monitor registry 326
  platforms in stack traces 322
  pooling 342
  sample of running 360
  servlets 158
  sleeping or suspended 342
  states 322
  wait state 327
  Writer 1 322
tnameserv naming server 118
toolkit (AWT) debugging 318
Toolkit class 340
trace
  calls 302
  command (Linux) 362
  records, truss 361
TRACE requests 147, 149
traceMethodCalls line 302
Traceplus (Windows platform) 363
transaction context 64
TRANSACTION_READ_COMMITTED level 66
TRANSACTION_READ_UNCOMMITTED level 66
TRANSACTION_REPEATABLE_READ level 66
TRANSACTION_SERIALIZABLE level 66
transactions
  attributes 63, 64
  auto commit 60
  backing out 60
  bean-managed 63
  begin 61
  commit 62
  commit (sess synch) 61
  commit mode 63
  complete 62
  controlling 140
  isolation levels described 63, 66
  JDBC 60
  settings 63
  type 63
transient keyword 26
translate method 264, 265
truss command 361
TX_BEAN_MANAGED attribute 65
TX_MANDATORY attribute 65
TX_NOT_SUPPORTED attribute 65
TX_REQUIRED attribute 65
TX_REQUIRED_NEW attribute 65

TX_SUPPORTS attribute 65
type().kind().value() method 114
TypeArray function 213
TypeCode value 114
U
unexportObject method 90
UnicastRemoteObject 90
Unicode Transformation Format (UTF) 212
Unified Modeling Language (UML) 3
Uniform Resource Locator (URL)
    CORBA callbacks 121
    database connection 70
    jdbc odbc 129
    weblogic.jdbc.jts.Driver 56
union type 113
unsigned values 109
update statement 132, 133
updateAccounts method 100, 105
use case diagram 2
User Interface delegate (UI delegate) 233
user policy file 409, 412
User signal monitor 325
utf8 hash table lock 322
utf8 hash table monitor 325
V
validateSession method 152
validation test, connections 346
-vcompat option 98
verbose option 300
W
W32LockView method 360
web pages for Duke's Auction 7
weblogic.jdbc.jts.Driver 56
weblogic.jndi.TengahInitialContextFactory 82
windowClosing method 251
WindowListener interface 251
Worker example code 382
WorkerThread class 342
wrapper classes, connections 345
Writer 1 322
X
Xbase database 220
-Xbootclasspath classpath option 299
Xbootclasspath option 304
XImageSource class 340
XML deployment descriptor 29, 63
X-PixMap (XPM) encoding scheme 339
XPM definition 375
XpmParser class 340
Z
Z-ordering 235